The Art of Programming By July

July

The Art of Programming By July

程序员如何准备面试中的算法
第一章导读
字符的移动
字符串包含
带通配符的字符串匹配问题
字符串转换成整数
回文判断
字符串其它问题
第二章导读
寻找最小的k个数
求数组中给定下标区间内的第 K 小元素
求解500万以内的亲和数
寻找和为定值的两个数
第五章、寻找和为定值的多个数
求连续子数组的最大和
荷兰国旗问题
矩阵相乘之Strassen算法
K个最小和 (UVA 11997 K Smallest Sums)
数组其它问题
第二十五章：二分查找实现
第二十三章：杨氏矩阵查找
出现次数超过一半的数字
第三十三章：木块砌墙
完美洗牌算法
第三十九章、最近公共祖先LCA问题
第四十章、螺旋矩阵
第五章导读
第十一章：最长公共子序列（LCS）问题
最大连续乘积子串
第二十九章：字符串编辑距离
格子取数问题

本章习题
随机取出其中之一元素
其他问题
最短摘要的生成
第二十四章:倒排索引关键词不重复Hash编码
第二十六章:基于给定的文档生成倒排索引的编码与实践
第八章导读
Trie树（字典树）
教你透彻了解红黑树
关联式容器
hash表算法
Bitmap
外排序
教你一步一步用c语言实现sift算法、下
支持向量机通俗导论（理解SVM的三层境界）
K近邻算法与距离度量表示法

程序员如何准备面试中的算法

备战面试中算法的五个步骤

对于立志进一线互联网公司，同时不满足于一辈子干纯业务应用开发，希望在后端做点事情的同学来说，备战面试中的算法，分为哪几个步骤呢？如下：

1、掌握一门编程语言

首先你得确保你已掌握好一门编程语言：

- C的话，推荐Dennis M. Ritchie & Brian W. Kernighan合著的《C程序设计语言》，和《C和指针》；
- C++ 则推荐《C++ Primer》，《深度探索C++对象模型》，《Effective C++》；
- Java推荐《Thinking in Java》，《Core Java》，《Effictive Java》，《深入理解Java虚拟机》。

掌握一门语言并不容易，不是翻完一两本书即可了事，语言的细枝末节需要在平日不断的编程练习中加以熟练。

2、过一遍微软面试100题系列

我从2010年起开始整理微软面试100题系列，见过的题目不可谓不多，但不管题目怎般变化，依然是那些常见的题型和考察点，当然，不考察任何知识点，纯粹考察编程能力的题目也屡见不鲜。故不管千变万化，始终不离两点：①看你基本知识点的掌握情况；②编程基本功。

而当你看了一遍微软面试100题之后(不要求做完)，你自会意识到：数据结构和算法在笔试面试中的重要性。

3、苦补数据结构基础

如果学数据结构，可以看我们在大学里学的任一本数据结构教材都行，如果你觉得实在不够上档次，那么可以再看看《STL源码剖析》。

然后，你会发现：大部分的面试题都在围绕一个点：基于各种数据结构上的增删改查。如字符串的查找翻转，链表的查找遍历合并删除，树和图的查找遍历，后来为了更好的查找，我们想到了排序，排序仍然不够，我们有了贪心、动态规划，再后来东西多了，于是有了海量数据处理，资源有限导致人们彼此竞争，出现了博弈组合概率。

4、看算法导论

《算法导论》上的前大部分的章节都在阐述一些经典常用的数据结构和典型算法（如二分查找，快速排序、Hash表），以及一些高级数据结构（诸如红黑树、B树），如果你已经学完了一本数据结构教材，那么建议你着重看贪心、动态规划、图论等内容，这3个议题每一个议题都大有题目可出。同时，熟悉常用算法的时间复杂度。

如果算法导论看不懂，你可以参看本博客。

5、刷leetcode或cc150或编程艺术系列

- 如主要在国外找工作，推荐两个面试编程网站：一个是http://leetcode.com/，leetcode是国外一网站，它上面有不少编程题；另外一个是http://www.careercup.com/，而后这个网站的创始人写了本书，叫《careercup cracking coding interview》，最终这本英文书被图灵教育翻译出版为《程序员面试金典》。

- 若如果是国内找工作，则郑重推荐我编写的《程序员编程艺术》，有编程艺术博客版，以及在博客版本基础上精简优化的编程艺术github版。除此之外，还可看看《编程之美》，与《剑指offer》。

而不论是准备国内还是国外的海量数据处理面试题，此文必看：教你如何迅速秒杀掉：99%的海量数据处理面试题。

此外，多看看优秀的开源代码，如nginx或redis，多做几个项目加以实践之，尽早实习（在一线互联网公司实习3个月可能胜过你自个黑灯瞎火

摸爬滚打一年)。

当然,如果你是准备社招,且已经具备了上文所说的语言 & 数据结构 & 算法基础,可以直接跳到本第五步骤,开始刷leetcode或cc150或编程艺术系列。

后记

学习最忌心浮气躁,急功近利,即便练习了算法,也不一定代表能万无一失通过笔试面试关,因为总体说来,在一般的笔试面试中,70%**基础**+30%**coding能力**(含算法),故如果做到了上文中的5个步骤,还远远不够,最后,我推荐一份非算法的书单,以此为大家查漏补缺(不必全部看完,欢迎大家补充):

1. 《深入理解计算机系统》
2. W.Richard Stevens著的《TCP/IP详解三卷》,《UNIX网络编程二卷》,《UNIX环境高级编程:第2版》,详见此豆瓣页面;
3. 你如果要面机器学习一类的岗位,建议看看相关的算法(如支持向量机通俗导论(理解SVM的三层境界)),及老老实实补补数学基础,包括微积分、线性代数、概率论与数理统计*(除了教材,推荐一本《数理统计学简史》)*、矩阵论*(推荐《矩阵分析与应用》)*等..

最后望大家循序渐进,踏实前进,若实在觉得算法 & 编程太难,转产品、运营、测试、运维、前端、设计都是不错的选择,因为虽然编程有趣,但不一定人人适合编程。

第一章导读

一般来说，笔试面试中常考的知识点有：

- 数据结构：字符串、链表、数组、堆、哈希表、树（Trie树、 后缀树、 红黑树、B树、R树）、图（遍历：BFS、DFS、Dijkstra）；
- 算法：基于各个数据结构的查找（二分、二叉树）、排序、遍历，分治、递归、回溯，贪心算法、动态规划、海量数据，外加字符串匹配和资源调优；
- 数学：排列组合概率；
- 操作系统、网络协议、数据库等等。

本第一章，咱们便来看看基于字符串的面试题。

字符的移动

题目描述

给定一个字符串，要求把字符串前面的若干个字符移动到字符串的尾部，如把字符串“abcdef”前面的2个字符“ab” 移动到字符串的尾部，即变成“cdefab”。请写一个函数完成此功能，要求对长度为n的字符串操作的时间复杂度为 O(n)，空间复杂度为 O(1)。

分析与解法

解法一、暴力移位法

初看此题，可能最先想到的方法是按照题目所要求的，把需要移动的字符一个一个的移动到字符串的尾部，如此我们可以实现一个函数 `LeftShiftOne(char *s, int n)`，以完成移动一个字符到字符串尾部的功能，代码如下所示：

```
void LeftShiftOne(char *s, int n)
{
    assert(s != NULL);
    char t = s[0];  //保存第一个字符
    for (int i = 1; i < n; i++)
    {
        s[i - 1] = s[i];
    }
    s[n - 1] = t;
}
```

因此，若要把字符串开头的m个字符移动到字符串的尾部，则可以如下操作：

```
void LeftShiftString(char *s, int n, int m)
{
    while (m--)
    {
        LeftShiftOne(s, n);
```

```
    }
}
```

下面，我们来分析下这个方法的时间复杂度和空间复杂度。

针对长度为n的字符串来说，假设需要移动m个字符到字符串的尾部，那么总共需要 m*n 次操作，同时设立一个变量保存第一个字符，如此，时间复杂度为O(m n)，空间复杂度为O(1)，空间复杂度符合题目要求，但时间复杂度不符合，所以，我们得需要寻找其它更好的办法来降低时间复杂度。

解法二、三步反转法

对于这个问题，换一个角度：

将一个字符串分成X和Y两个部分，在每部分字符串上定义反转操作，如X^T，即把X的所有字符反转（如，X=“abc”，那么X^T=“cba”），那么就得到下面的结论：$(X^{TY}T)^T=YX$。显然这解决了字符串的反转问题。

例如字符串 abcdef，若要让def翻转到abc的前头，只要按下述3个步骤操作即可：

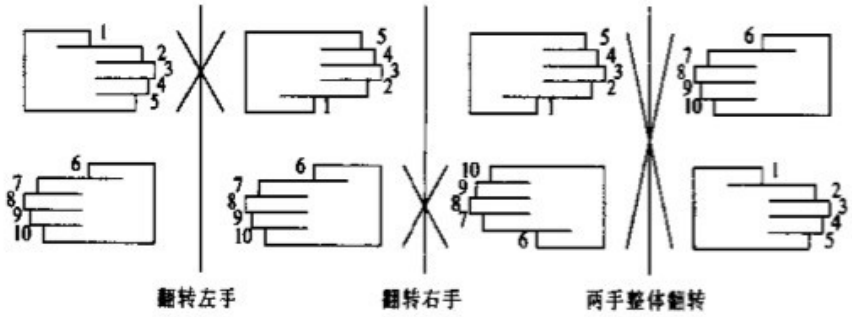
1. 首先分为俩部分，X:abc，Y:def；
2. 将X反转，X->X^T，即得：abc->cba；将Y反转，Y->Y^T，即得：def->fed。
3. 反转上述得到的结果字符串$X^{TY}T$，即字符串cbafed的两部分（cba和fed）给予反转，得到：cbafed->defabc，形式化表示为$(X^{TY}T)^T=YX$，这就实现了整个反转。

是不是一目了然了？

在《编程珠玑》上也有这样一个类似的问题，它的解法同本思路一致，如下图所示：

***. 可操作的证明方法-手摇法**

如果要将一个具有10个元素（我们只有10个手指啊）的数组向上旋转5个位置，先让两只手的掌心正对你自己，左右放在右手上面（其实两只手是同意平面上的），看下图：



代码则可以这么写：

```
//updated@caopengcs && July
//2014-1-6
void reverse(char *s,int from,int to)
{
    while (from < to)
    {
        char t = s[from];
        s[from++] = s[to];
        s[to--] = t;
    }
}

void LeftRotateString(char *s,int n,int m)
{
    m %= n;                 //若要左移动大于n位,那么和%n 是等价的
    reverse(s, 0, m - 1); //反转[0..m - 1],套用到上面举的例子中,就是X->X^T,即 abc->cba
    reverse(s, m, n - 1); //反转[m..n - 1],例如Y->Y^T,即 def->fed
    reverse(s, 0, n - 1); //反转[0..n - 1],即如整个反转,(X^TY^T)^T=YX,即 cbafed->defabc。
}
```

举一反三

1、链表翻转。给出一个链表和一个数k，比如链表1→2→3→4→5→6，k=2，则翻转后2→1→6→5→4→3，若k=3,翻转后3→2→1→6→5→4，若k=4，翻转后4→3→2→1→6→5，用程序实现。

2、编写程序，在原字符串中把尾部m个字符移动到字符串的头部，要求：长度为n字符串操作时间复杂度为O(n),空间复杂度为O(1)。如：原字符串为”Ilovebaofeng”，m=7，输出结果：”baofengIlove”。

3、单词翻转。输入一个英文句子，翻转句子中单词的顺序，但单词内字符的顺序不变，句子中单词以空格符隔开。为简单起见，标点符号和普通字母一样处理。例如输入“I am a student.”，则输出“student. a am I”。

字符串包含

题目描述

给定两个分别由字母组成的字符串A和字符串B，字符串B的长度较字符串A短。请问，如何最快地判断字符串B中所有字母是否都在字符串A里？也就是说判断字符串B是不是字符串A的真子集（为了简化，姑且认为两个集合都不是空集，即字符串都不为空）。且为了简单起见，我们规定输入的字符串只包含大写英文字母。

请实现函数bool StringContains(string &A,string &B)

比如，如果是下面两个字符串：

String 1: ABCDEFGHLMNOPQRS

String 2: DCGSRQPO

答案是true，即String2里的字母在String1里也都有，或者说String2是String1的真子集。

如果是下面两个字符串：

String 1: ABCDEFGHLMNOPQRS

String 2: DCGSRQPZ

答案是false，因为字符串String2里的Z字母不在字符串String1里。

分析与解法

题目描述虽长，但题意很明了，就是给定一长一短的两个字符串A，B，假设A长B短，要求判断B是否包含在字符串A中，即B?(-A。

初看似乎简单，但实现起来并不轻松，且如果面试官步步紧逼，一个一个否决你能想到的方法，要你给出更好、最好的方案时，恐怕就要伤不少脑筋了。

解法一、暴力轮询

判断string2中的字符是否在string1中?：

String 1: ABCDEFGHLMNOPQRS

String 2: DCGSRQPO

最直观也是最简单的思路是，针对string2中每一个字符，逐个与string1中每个字符比较，看它是否在String1中。

代码可如下编写：

```
//copyright@caopengcs 2013-01-14
bool StringContain(string &a,string &b)
{
    for (int i = 0; i < b.length(); ++i) {
        int j;
        for (j = 0; (j < a.length()) && (a[j] != b[i]); ++j)
            ;
        if (j >= a.length())
        {
            return false;
        }
    }
    return true;
}
```

假设n是字符串String1的长度，m是字符串String2的长度，那么此算法，需要O(n*m*)*次操作，以上面的例子来说，将会有*168 = 128次操作。显然，时间开销太大，应该找到一种更好的办法。

解法二、普通排序

如果允许排序的话，我们可以考虑下排序。比如可先对这两个字符串的字母进行排序，然后再同时对两个字串依次轮询。两个字串的排序需要

(常规情况)O(m log m) + O(n log n)次操作，之后的线性扫描需要O(m+n)次操作。

同样拿上面的字串做例子，将会需要16*4* + *8*3 = 88，再加上对两个字串线性扫描的16 + 8 = 24的操作(随着字串长度的增长，你会发现这个算法的效果会越来越好)。

关于排序方法，可采用最常用的快速排序，C有库函数qsort,C++有库函数sort，排序不是重点，因此使用库函数好一些。

```
//copyright@caopengcs 2014-01-14
//注意A B中可能包含重复字符,所以注意A下标不要轻易移动。这种方法改变了字符串。如不想改变请自己复制
bool StringContain(string &a,string &b)
{
    sort(a.begin(),a.end());
    sort(b.begin(),b.end());
    for (int pa = 0, pb = 0; pb < b.length();)
    {
        while ((pa < a.length()) && (a[pa] < b[pb]))
        {
            ++pa;
        }
        if ((pa >= a.length()) || (a[pa] > b[pb]))
        {
            return false;
        }
        //a[pa] == b[pb]
        ++pb;
    }
    return true;
}
```

解法三、计数比较

有没有比快速排序更好的方法呢？进一步，我们可以采用线性时间的计数方法，假设需要比较字符串A（n）中是否包含字符串B（m），统计A中出现的字符O(n)，比较B中是否有出现的字符O(m)，总计时间复杂度为：O(n+m)。

代码如下：

```
// copyright@caopengcs 2014-01-14
// modified by @古道西风 2014-01-14
bool StringContain(string &a,string &b)
{
    vector<int> have;
    have.resize(26,0);
    for (int i = 0; i < a.length(); ++i)
    {
        ++have[a[i] - 'A'];
    }
    for (int i = 0; i < b.length(); ++i)
    {
        //若A中只需要包含同一个相同的字符即可代表B中重复出现的字符
        //即A:CDEFAB B:AABBCC 合法
        if (have[b[i] - 'A'] == 0)
        {
            return false;
        }
        //若A中需要包含B中所有重复出现字符
        //即A:AAACCDBBCCE    B:AABBCC 合法
        //即A:BCDEFA B:AABBCC 非法
        //if (have[b[i] - 'A']-- == 0) {
        //  return false;
        //}
    }
    return true;
}
```

解法四、巧用hashtable

上述方案中，较好的方法是解法三中的计数方法，总的时间已经优化到了：O(m+n)，貌似到了极限，还有没有更好的办法？

更进一步，可以对短字符串进行轮询，把其中的每个字母都放入一个Hashtable里（始终设m为短字符串的长度，那么此项操作成本是O(m)或8次操作）。然后轮询长字符串，在Hashtable里查询短字符串的每个字符，看能否找到。如果找不到，说明没有匹配成功，轮询长字符串将消耗掉16次操作，这样两项操作加起来一共只有8+16=24次。

当然，理想情况是如果长字串的前缀就为短字串，只需消耗8次操作，这样总共只需8+8=16次。

具体算法流程如下：1. hash[26]，先全部清零，然后扫描短的字符串，若有相应的置1，2. 计算hash[26]中1的个数，记为m 3. 扫描长字符串的每个字符a；若原来hasha == 1，则修改hasha = 0，并将m减1；若hasha == 0，则不做处理 4. 若m == 0 or 扫描结束，退出循环。

代码实现，也不难，如下：

```
// copyright@caopengcs 2014-StringContains01-14
bool StringContain(string &a,string &b)
{
    vector<int> hash;
    hash.resize(26,0);
    int m = 0;
    for (int i = 0; i < b.length(); ++i)
    {
        int x = b[i] - 'A';
        if (hash[x] == 0)
        {
            hash[x] = 1;
            ++m;
        }
    }
    for (int i = 0; i < a.length() && m > 0; ++i)
    {
        int x = a[i] - 'A';
        if (hash[x] == 1)
        {
            --m;
            hash[x] = 0;
        }
    }
    return m == 0;
}
```

解法五、素数相乘

如果面试官继续追问，还有没有更好的办法呢?

我们换一种角度思考本问题：

假设有一个仅由字母组成字串，让每个字母与一个素数对应，从2开始，往后类推，A对应2，B对应3，C对应5，……。遍历第一个字串，把每个字母对应素数相乘。最终会得到一个整数。

利用上面字母和素数的对应关系，对应第二个字符串中的字母，然后轮询，用每个字母对应的素数除前面得到的整数。如果结果有余数，说明结果为false。如果整个过程中没有余数，则说明第二个字符串是第一个的子集了（判断是不是真子集，可以比较两个字符串对应的素数乘积，若相等则不是真子集）。

思路总结如下：- 1. 按照从小到大的顺序，用26个素数分别与字符'A'到'Z'一一对应。- 2. 遍历长字符串，求得每个字符对应素数的乘积。- 3. 遍历短字符串，判断乘积能否被短字符串中的字符对应的素数整除。- 4. 输出结果。

如前所述，算法的时间复杂度为O(m+n)的最好的情况为O(n)（遍历短的字符串的第一个数，与长字符串素数的乘积相除，即出现余数，便可退出程序，返回false），n为长字串的长度，空间复杂度为O(1)。

```
// copyright@caopengcs
//此方法只有理论意义，因为整数乘积很大，有溢出风险
bool StringContain(string &a,string &b)
{
    const int p[26] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53,
59,61, 67, 71, 73, 79, 83, 89, 97, 101};
    int f = 1;
    for (int i = 0; i < a.length(); ++i)
    {
        int x = p[a[i] - 'A'];
        if (f % x)
        {
            f *= x;
        }
    }
    for (int i = 0; i < b.length(); ++i)
    {
        int x = p[b[i] - 'A'];
        if (f % x)
        {
            return false;
        }
    }
    return true;
}
```

此种素数相乘的方法看似完美，但缺点是素数相乘的结果容易导致整数溢出。

解法六、位运算

最好的思路是对字符串A,用位运算（26bit整数表示)计算出一个“签名”，再用B中的字符到A里面进行查找。这个方法的实质是用一个整数代替了hashtable，空间复杂度可以降低为O(1)。时间复杂度还是O(n + m)。

```
// copyright@caopengcs
// “最好的方法”,时间复杂度O(n + m),空间复杂度O(1)
bool StringContain(string &a,string &b)
{
    int hash = 0;
    for (int i = 0; i < a.length(); ++i)
    {
        hash |= (1 << (a[i] - 'A'));
    }
    for (int i = 0; i < b.length(); ++i)
    {
        if ((hash & (1 << (b[i] - 'A'))) == 0)
        {
            return false;
        }
    }
    return true;
}
```

举一反三

1、本问题的几个变形 - 一个单词单词字母交换，可得另一个单词，如army->mary，成为兄弟单词。提供一个单词，在字典中找到它的兄弟。描述数据结构和查询过程。- 如果两个字符串的字符一样，但是顺序不一样，被认为是兄弟字符串，问如何在迅速匹配兄弟字符串（如，bad和adb就是兄弟字符串）。- 假设两个字符串中所含有的字符和个数都相同我们就叫这两个字符串匹配，比如：abcda和adabc,由于出现的字符个数都是相同，只是顺序不同，所以这两个字符串是匹配的，要求高效。

分析：上述3个问题的解决思路基本同本节的解决方法一致，只是题目做了个变形。

带通配符的字符串匹配问题

题目描述

字符串匹配问题：给定一个字符串，按照指定规则对其进行匹配，并将匹配的结果保存至output数组中。多个匹配项用空格间隔，最后一个不需要空格。

要求：

1. 匹配规则中包含通配符?和*，其中?表示匹配任意一个字符，*表示匹配任意多个（>=0）字符。
2. 匹配规则要求匹配最大的字符子串，例如a*d,匹配abbdd而非abbd,即最大匹配子串。
3. 匹配后的输入串不再进行匹配，从当前匹配后的字符串重新匹配其他字符串。

请实现函数：

```
char* my_find(char input[], char rule[])
```

举例说明：

input:abcadefg

rule:a?c

output:abc

input :newsadfanewfdadsf

rule: new

output: new new

input :breakfastfood

rule: f*d

output:fastfood

31.1.jpg

注意事项：

1. 自行实现函数my_find，勿在my_find函数里夹杂输出，且不准用C、C++库，和Java的String对象；
2. 请注意代码的时间，空间复杂度，及可读性，简洁性；
3. input=aaa，rule=aa时，返回一个结果aa，即可。

分析与解法

解法一、直接匹配

本题与“字符串转换成整数”的题目不同，那题偏向于考察思维的全面性和细节的处理。而本题则偏向于编程技巧。闲不多说，直接上代码：

```
//copyright@cao_peng 2013/4/23
int str_len(char *a)
{
    //字符串长度
    if (a == 0)
    {
        return 0;
    }
```

```
    char *t = a;
    for (; *t; ++t)
        ;
    return (int) (t - a);
}

void str_copy(char *a, const char *b, int len)
{
    //拷贝字符串 a = b
    for (; len > 0; --len, ++b, ++a)
    {
        *a = *b;
    }
    *a = 0;
}

char *str_join(char *a, const char *b, int lenb)
{
    //连接字符串 第一个字符串被回收
    char *t;
    if (a == 0)
    {
        t = (char *) malloc(sizeof(char) * (lenb + 1));
        str_copy(t, b, lenb);
        return t;
    }
    else
    {
        int lena = str_len(a);
        t = (char *) malloc(sizeof(char) * (lena + lenb + 2));
        str_copy(t, a, lena);
        *(t + lena) = ' ';
        str_copy(t + lena + 1, b, lenb);
        free(a);
        return t;
    }
}

int canMatch(char *input, char *rule)
{
    // 返回最长匹配长度 -1表示不匹配
    if (*rule == 0)
    {
        //已经到rule尾端
        return 0;
    }
    int r = -1 , may;
    if (*rule == '*')
```

```
    {
        r = canMatch(input, rule + 1);  // *匹配0个字符
        if (*input)
        {
            may = canMatch(input + 1, rule);  // *匹配非0个字符
            if ((may >= 0) && (++may > r))
            {
                r = may;
            }
        }
    }
    if (*input == 0)
    {
        //到尾端
        return r;
    }
    if ((*rule == '?') || (*rule == *input))
    {
        may = canMatch(input + 1, rule + 1);
        if ((may >= 0) && (++may > r))
        {
            r = may;
        }
    }
    return r;
}

char * my_find(char input[], char rule[])
{
    int len = str_len(input);
    int *match = (int *) malloc(sizeof(int) * len);  //input第i位最多能匹配多少位 匹配不
上是-1
    int i, max_pos = - 1;
    char *output = 0;

    for (i = 0; i < len; ++i)
    {
        match[i] = canMatch(input + i, rule);
        if ((max_pos < 0) || (match[i] > match[max_pos]))
        {
            max_pos = i;
        }
    }
    if ((max_pos < 0) || (match[max_pos] <= 0))
    {
        //不匹配
        output = (char *) malloc(sizeof(char));
        *output = 0;   // \0
```

```
            return output;
        }
        for (i = 0; i < len;)
        {
            if (match[i] == match[max_pos])
            {
                //找到匹配
                output = str_join(output, input + i, match[i]);
                i += match[i];
            }
            else
            {
                ++i;
            }
        }
        free(match);
        return output;
}
```

解法二、动态规划

本题也可以直接写出DP(Dynamic Programming, 动态规划)方程。如下所示：

```
//copyright@chpeih 2013/4/23
char* my_find(char input[], char rule[])
{
    //write your code here
    int len1, len2;
    for (len1 = 0; input[len1]; len1++);
    for (len2 = 0; rule[len2]; len2++);
    int MAXN = len1 > len2 ? (len1 + 1) : (len2 + 1);
    int  **dp;

    //dp[i][j]表示字符串1和字符串2分别以i j结尾匹配的最大长度
    //记录dp[i][j]是由之前那个节点推算过来  i*MAXN+j
    dp = new int *[len1 + 1];
    for (int i = 0; i <= len1; i++)
    {
        dp[i] = new int[len2 + 1];

    }

    dp[0][0] = 0;
    for (int i = 1; i <= len2; i++)
        dp[0][i] = -1;
```

```
for (int i = 1; i <= len1; i++)
    dp[i][0] = 0;

for (int i = 1; i <= len1; i++)
{
    for (int j = 1; j <= len2; j++)
    {
        if (rule[j - 1] == '*')
        {
            if (j >= 2 && rule[j - 2] == '*')
            {
                dp[i][j] = dp[i][j - 1];
                continue;
            }
            dp[i][j] = -1;
            if (dp[i - 1][j - 1] != -1)
            {
                dp[i][j] = dp[i - 1][j - 1] + 1;

            }
            if (dp[i - 1][j] != -1 && dp[i][j] < dp[i - 1][j] + 1)
            {
                dp[i][j] = dp[i - 1][j] + 1;
            }
        }
        else if (rule[j - 1] == '?')
        {
            if (dp[i - 1][j - 1] != -1)
            {
                dp[i][j] = dp[i - 1][j - 1] + 1;

            }
            else dp[i][j] = -1;
        }
        else
        {
            dp[i][j] = -1;
            if (input[i - 1] == rule[j - 1])
            {
                if (dp[i - 1][j - 1] != -1)
                {
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                }
                else if (j >= 2 && rule[j - 2] == '*')
                {
                    int tmp = j - 2;
                    while (tmp >= 0 && rule[tmp] == '*')
                    {
```

```
                        tmp--;
                    }
                    tmp++;
                    if (dp[i - 1][tmp] != -1)
                    {
                        dp[i][j] = dp[i - 1][tmp] + 1;
                    }
                }
            }
        }
    }

    int m = -1;//记录最大字符串长度
    int *ans = new int[len1];
    int count_ans = 0;//记录答案个数
    char *returnans = new char[len1 + 1];
    int count = 0;
    for (int i = 1; i <= len1; i++)
        if (dp[i][len2] > m)
        {
            m = dp[i][len2];
            count_ans = 0;
            ans[count_ans++] = i - m;
        }
        else if (dp[i][len2] != -1 && dp[i][len2] == m && ans[count_ans] + m  <= i -
m)
        {
            ans[count_ans++] = i - m;
        }

    if (count_ans != 0)
    {
        int len = ans[0];
        for (int i = 0; i < m; i++)
        {
            printf("%c", input[i + ans[0]]);
            returnans[count++] = input[i + ans[0]];
        }
        for (int j = 1; j < count_ans; j++)
        {
            printf(" ");
            returnans[count++] = ' ';
            len = ans[j];
            for (int i = 0; i < m; i++)
            {
                printf("%c", input[i + ans[j]]);
                returnans[count++] = input[i + ans[j]];
```

```
                }
            }
            printf("\n");
            returnans[count++] = '\0';
        }

        return returnans;
    }


    /**
     * 平均O(n+m)  最坏O(n*m)
     * 约定 * : ANY>=0    ,   ? : ANY=1
     *
     * @author Spance.Wong
     */
    static class WildCardMatcher {

        /**
         * 仅为了方便实验
         *
         * @param input
         * @param pattern
         * @return
         */
        static List<String> matches(String input, String pattern) {
            String[] pa = pattern.split("\\*+");    // 分割不是重点,故未做重点实现
            return matches(input, pa);
        }

        /**
         * 从input中查找通配符序列
         *
         * @param input
         * @param patterns
         * @return
         */
        static List<String> matches(CharSequence input, String[] patterns) {
            int n = input.length(), m = patterns.length;
            List<String> result = new ArrayList<String>();
            for (int i = 0; i < n; ) {
                int left = -1, right = -1;
                for (int j = 0; j < m; j++) {    // 以i为起点，执行m趟匹配，每趟i至少前进
p[j].length长度
                    long region = lookBehind(input, i, patterns[j]);
                    if (j != 0 && region >= 0) {    // 模式序列的第二个开始使用贪婪匹配
                        long greedyRegion;
                        for (int k = (int) region + 1; ; k = (int) greedyRegion + 1)
```

```
{
                            greedyRegion = lookBehind(input, k, patterns[j]);
                            if (greedyRegion > 0)     // 贪婪找到，继续贪婪尝试
                                region = greedyRegion;
                            else
                                break;
                        }
                    }
                    if (region < 0) {    // pattern[j]失败,则本趟失败
                        i = ((int) -region) + 1;
                        break;
                    } else {
                        i = (int) region + 1;
                        if (j == 0)      // 模式序列的第一个找到,记左边界,在高32位
                            left = (int) (region >> 32);
                        if (j == m - 1)      // 模式序列的最后一个找到,记右边界,在低32位
                            right = (int) region;
                    }
                }
                if (left >= 0 && right >= 0)
                    result.add(input.subSequence(left, right + 1).toString());
            }
            return result;
        }

        /**
         * 在input的i位置开始向后扫描非贪婪查找pattern,在pattern尾匹配时回溯确认
         *
         * @param in
         * @param i
         * @param pattern
         * @return
         */
        static long lookBehind(CharSequence in, int i, CharSequence pattern) {
            int len = in.length(), pLen = pattern.length(), _pMax = pLen - 1;
            char pEnd = pattern.charAt(_pMax);
            if (len - i >= pLen) {
                for (i = i + _pMax; i < len; i++) {  // 以 i + pLen - 1 起步
                    if (in.charAt(i) == pEnd || pEnd == '?') {      // 与pa末尾相同,i即
右边界
                        if (pLen == 1)
                            return ((long) i) << 32 | i;
                        for (int j = i - 1; j >= i - _pMax; j--) {    // 则至多回溯pLen
长找左边界
                            char p = pattern.charAt(_pMax - i + j);
                            if (in.charAt(j) == p || p == '?') {
                                if (j == i - _pMax)      // 找到左边界即j
                                    return ((long) j) << 32 | i;
```

```
                        } else
                            break;
                    }
                }
            }
        }
        return -i;
    }
}
```

举一反三

1、给定一个带通配符问号的数W，问号可以代表任意数字。 再给定一个整数X，和W具有同样的长度。问有多少个整数符合W的形式并且比X大？

字符串转换成整数

题目描述

输入一个由数字组成的字符串，把它转换成整数并输出。例如：输入字符串“345”，输出整数345。

给定函数原型`int StrToInt(const char *str)`，实现字符串转换成整数的功能。不能使用库函数atoi。

分析与解法

我们来一步一步分析，直至写出第一份准确的代码。本题考查的实际上就是字符串转换成整数的问题，或者说是要你自行实现atoi函数。那如何实现把表示整数的字符串正确地转换成整数呢？以“345”作为例子：- 当我们扫描到字符串的第一个字符’3’时，由于我们知道这是第一位，所以得到数字3。- 当扫描到第二个数字’4’时，而之前我们知道前面有一个3，所以便在后面加上一个数字4，那前面的3相当于30，因此得到数字：*3*10+4=34。*- 继续扫描到字符’5’，’5’的前面已经有了34，由于前面的34相当于340，加上后面扫描到的5，最终得到的数是：34*10+5=345。

因此，此题的基本思路：从左至右扫描字符串，把之前得到的数字乘以10，再加上当前字符表示的数字。

但有一些细节需要注意：
1. 正负符号：整数不仅包含数字，还有可能是以’+‘或’-‘开头表示正负整数，因此如果第一个字符是’-’号，则要把得到的整数转换成负整数。
2. 空指针输入：输入的是指针，在访问空指针时程序会崩溃，因此在使用指针之前需要先判断指针是否为空。
3. 非法字符：输入的字符串中可能含有不是数字的字符。因此，每当碰到这些非法的字符，程序应停止转换。
4. 整型溢出：输入的数字是以字符串的形式输入，因此输入一个很长的字符串将可能导致溢出。

如图，输入字符串用例和分析的结果（假定运行环境是32位系统，且编译环境是VS2008以上）：

“”	0
“1”	1
“+1”	1
“-1”	-1
“123”	123
“-123”	-123
“010”	10
“+00131204”	131204
“-01324000”	-1324000
“2147483647”	2147483647
“-2147483647”	-2147483647
“-2147483648”	-2147483648
“2147483648”	2147483647
“-2147483649”	-2147483648
“abc”	0
“-abc”	0
“1a”	1
“23a8f”	23
“-3924x8fc”	-3924
“ 321”	321
“ -321”	-321
“123 456”	123
“123 ”	123
“ - 321”	0
“ +4488”	4488
“ + 413”	0
“ ++c”	0
“ ++1”	0
“ --2”	0
“ -2”	-2

解法一

初步实现：

```
//copyright@zhedahht 2007
enum Status {kValid = 0, kInvalid};
int g_nStatus = kValid;

int StrToInt(const char* str)
{
    g_nStatus = kInvalid;
    long long num = 0;

    if (str != NULL)
```

```
    {
        const char* digit = str;

        // 判断字符串是否以'+'或'-'开头
        bool minus = false;
        if (*digit == '+')
            digit ++;
        else if (*digit == '-')
        {
            digit ++;
            minus = true;
        }

        // 从左至右扫描字符串
        while (*digit != '\0')
        {
            if (*digit >= '0' && *digit <= '9')
            {
                num = num * 10 + (*digit - '0');

                //处理型溢出问题
                if (num > std::numeric_limits<int>::max())
                {
                    num = 0;
                    break;
                }

                digit ++;
            }
            //处理字符问题。如“1a”
            else
            {
                num = 0;
                break;
            }
        }

        if (*digit == '\0')
        {
            g_nStatus = kValid;
            if (minus)
                num = 0 - num;
        }
    }
    return static_cast<int>(num);
}
```

这份实现有两个问题：

当输入的字符串不是数字，而是字符的时候，比如“1a”，上述程序直接返回了0（而正确的结果应该是得到1）：

```
//处理字符问题。如“1a”
else
{
    num = 0;
    break;
}
```

处理溢出时，有问题。因为它遇到溢出情况时，直接返回了0：

```
//处理型溢出问题
if (num > std::numeric_limits<int>::max())
{
    num = 0;
    break;
}
```

解法二

对代码做下微调。当发生溢出时，取最大或最小的int值。当输入字符串包含字母时，停止转换并返回结果。（注：库函数atoi规定：大于int值，取最大值maxint：2147483647；小于-int取最小值minint:-2147483648）

```
//copyright@SP_daiyq 2013/5/29
int StrToInt(const char* str)
{
    int res = 0; //存放结果
    int i = 0; // 字符串下标
    int signal = '+'; // 数字的符号,'+'或'-'
    int cur; // 当前数字
    //处理空指针问题
    if (!str)
        return 0;

    //跳过空格
    while (isspace(str[i]))
        i++;

    //跳过符号
    if (str[i] == '+' || str[i] == '-')
    {
```

```
        signal = str[i];
        i++;
    }

    //计算结果
    while (str[i] >= '0' && str[i] <= '9')
    {
        cur = str[i] - '0';

        //当发生正溢出时,返回INT_MAX;发生负溢出时，返回INT_MIN
        if ( (signal == '+') && (cur > INT_MAX - res * 10) )
        {
            res = INT_MAX;
            break;
        }
        else if ( (signal == '-') && (cur - 1 > INT_MAX - res * 10) )
        {
            res = INT_MIN;
            break;
        }

        res = res * 10 + cur;
        i++;
    }

    return (signal == '-') ? -res : res;
}
```

此时会发现,非法字符的问题解决了:

"-2147483648"	-2147483648	-2147483648	✓
"2147483648"	2147483647	2147483647	✓
"-2147483649"	-2147483648	-2147483648	✓
"abc"	0	0	✓
"-abc"	0	0	✓
"1a"	1	1	✓
"23a8f"	23	23	✓
"-3924x8fc"	-3924	-3924	✓
" 321"	321	321	✓
" -321"	-321	-321	✓
"123 456"	123	123	✓
"123 "	123	123	✓
" - 321"	0	0	✓
" +4488"	4488	4488	✓
" + 413"	0	0	✓
" ++c"	0	0	✓
" ++1"	0	0	✓
" --2"	0	0	✓
" -2"	-2	-2	✓

但整型溢出问题却没有解决：

当上述代码转换" 10522545459“时，它应得到2147483647，但程序运行结果却是：1932610867。所以程序没有解决好溢出问题。

分析下代码，看是如何具体处理溢出情况的：

```
// judge overlap or not
        if ( (signal == '+') && (cur > INT_MAX - res * 10) )
        {
            res = INT_MAX;
            break;
        }
        else if ( (signal == '-') && (cur - 1 > INT_MAX - res * 10) )
        {
            res = INT_MIN;
            break;
        }
```

给定字符串" 10522545459“，除去空格有11位。而MAX_INT是2147483647，有10位数。当扫描到最后一个字符‘9’的时候，程序会比较 9 和 2147483647 - 1052254545*10的大小。
问题立马就暴露出来了，因为此时让res*10，即让1052254545*10 > MAX_INT，溢出无疑。程序已经出错，再执行下面这行代码已无意义：

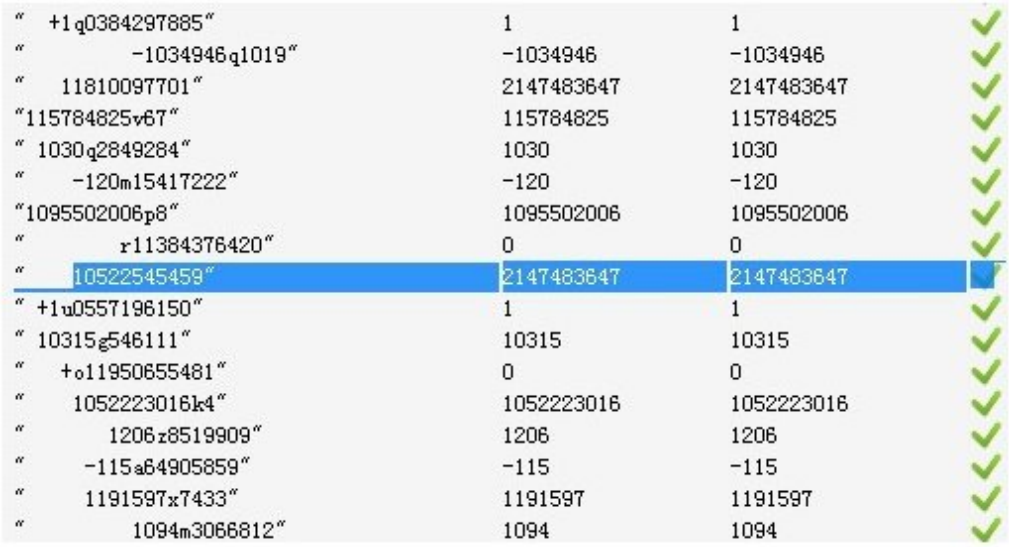
解法三

上面说给的程序没有“很好的解决溢出问题。由于输入的数字是以字符串的形式输入，因此有可能输入一个很大的数字转换之后会超过能够表示的最大的整数而溢出”。
看看下面的代码

```
//copyright@fuwutu 2013/5/29
int StrToInt(const char* str)
{
    bool negative = false;
    long long result = 0; // 返回值定义成了long long
    while (*str == ' ' || *str == '\t')
```

```
    {
        ++str;
    }
    if (*str == '-')
    {
        negative = true;
        ++str;
    }
    else if (*str == '+')
    {
        ++str;
    }

    while (*str != '\0')
    {
        int n = *str - '0';
        if (n < 0 || n > 9)
        {
            break;
        }

        if (negative)
        {
            result = result * 10 - n;
            if (result < -2147483648LL)
            {
                result = -2147483648LL;
            }
        }
        else
        {
            result = result * 10 + n;
            if (result > 2147483647LL)
            {
                result = 2147483647LL;
            }
        }
        ++str;
    }

    return result;
}
```

运行上述程序：



" +1q0384297885"	1	1	✓
" -1034946q1019"	-1034946	-1034946	✓
" 11810097701"	2147483647	2147483647	✓
"115784825v67"	115784825	115784825	✓
" 1030q2849284"	1030	1030	✓
" -120m15417222"	-120	-120	✓
"1095502006p8"	1095502006	1095502006	✓
" r11384376420"	0	0	✓
" 10522545459"	2147483647	2147483647	✓
" +1u0557196150"	1	1	✓
" 10315g546111"	10315	10315	✓
" +o11950655481"	0	0	✓
" 1052223016k4"	1052223016	1052223016	✓
" 1206z8519909"	1206	1206	✓
" -115a64905859"	-115	-115	✓
" 1191597x7433"	1191597	1191597	✓
" 1094m3066812"	1094	1094	✓

上图所示程序貌似通过了，然而实际上它还是未能处理数据溢出的问题。因为它只是做了个取巧：把返回的值result定义成了long long。严格说来，我们依然未写出准确的规范代码

解法四

根据我们最初的思路："从左至右扫描字符串，把之前得到的数字乘以10，再加上当前字符表示的数字"，相信读者已经觉察到，在扫描到最后一个字符的时候，如果之前得到的数比较大，此时若再让其扩大10倍，相对来说是比较容易溢出的。

但车到山前必有路，既然让一个比较大的int整型数括大10倍，比较容易溢出，那么在不好判断是否溢出的情况下，可以尝试使用除法：

1. 与其将n扩大10倍，，冒着溢出的风险，再与MAX_INT进行比较（如果已经溢出，则比较的结果没有意义），
2. 不如未雨绸缪先用n与MAX_INT/10进行比较： 若n>MAX_INT/10（当然同时还要考虑n=MAX_INT/10的情况），说明最终得到的整数一定会溢出，故此时可以当即进行溢出处理，直接返回最大值MAX_INT，从而也就免去了计算n*10这一步骤。

也就是说，计算n*10前，先比较n与MAX_INT/10大小，若n>MAX_INT/10，那么n*10肯定大于MAX_INT，即代表最后得到的整数n肯定溢出，既然溢出，不能再计算n*10，直接提前返回MAX_INT就行了。

一直以来，我们努力的目的归根结底是为了更好的处理溢出，但上述做法最重要的是巧妙的规避了计算n*10这一乘法步骤，转换成计算除法MAX_INT/10代替，不能不说此法颇妙。

对于正数来说，它溢出的可能性有两种： 1. 一种是诸如2147483650，即n > MAX/10 的；

2. 一种是诸如2147483649，即n == MAX/10 && c > MAX%10。

部分代码如下：

```
//copyright@njnu_mjn 2013
c = *str - '0';
if (sign > 0 && (n > MAX / 10 || (n == MAX / 10 && c > MAX % 10)))
{
    n = MAX;
    break;
}
else if (sign < 0 && (n > (unsigned)MIN / 10
          || (n == (unsigned)MIN / 10 && c > (unsigned)MIN % 10)))
{
    n = MIN;
    break;
}
```

测试结果如下，暂未发现什么问题

输入	输出
10522545459：	2147483647
-10522545459：	-2147483648

咱们再来总结下上述代码是如何处理溢出情况的。

仍有一些细节是值得改进的，如：

将MAX/10,MAX%10,(unsigned)MIN/10及(unsigned)MIN%10保存到变量中，防止重复计算

解法五

优化后，完整的代码为：

```
//copyright@njnu_mjn 2013
int StrToDecInt(const char* str)
{
    static const int MAX = (int)((unsigned)~0 >> 1);
    static const int MIN = -(int)((unsigned)~0 >> 1) - 1;
    static const int MAX_DIV = (int)((unsigned)~0 >> 1) / 10;
    static const int MIN_DIV = (int)((((unsigned)~0 >> 1) + 1) / 10);
    static const int MAX_R = (int)((unsigned)~0 >> 1) % 10;
    static const int MIN_R = (int)((((unsigned)~0 >> 1) + 1) % 10);
    int n = 0;
    int sign = 1;
    int c;

    while (isspace(*str))
        ++str;
    if (*str == '+' || *str == '-')
    {
        if (*str == '-')
            sign = -1;
        ++str;
    }
    while (isdigit(*str))
    {
        c = *str - '0';
        if (sign > 0 && (n > MAX_DIV || (n == MAX_DIV && c >= MAX_R)))
        {
            n = MAX;
            break;
        }
        else if (sign < 0 && (n > MIN_DIV
                  || (n == MIN_DIV && c >= MIN_R)))
        {
            n = MIN;
            break;
        }
        n = n * 10 + c;
        ++str;
    }
    return sign > 0 ? n : -n;
}
```

部分数据的测试结果如下图所示：

```
输入            输出
10522545459   : 2147483647
-10522545459  : -2147483648
2147483648    : 2147483647
-2147483648   : -2147483648
```

程序已实现题目的要求。但如程序作者MJN君所说“我的实现与linux内核的atoi函数的实现, 都有一个共同的问题: 即使出错, 函数也返回了一个值, 导致调用者误认为自己传入的参数是正确的, 但是可能会导致程序的其他部分产生莫名的错误且很难调试”。

类似问题

1. 实现string到double的转换

提醒：此题虽然类似于atoi函数，但毕竟double为64位，而且支持小数，因而边界条件更加严格，写代码时需要更加注意。

回文判断

题目描述

回文，英文palindrome，指一个顺着读和反过来读都一样的字符串，比如madam、我爱我，这样的短句在智力性、趣味性和艺术性上都颇有特色，中国历史上还有很多有趣的回文诗。

那么，我们的第一个问题就是：判断一个字串是否是回文？

分析与解法

回文判断是一类典型的问题，尤其是与字符串结合后呈现出多姿多彩，在实际中使用也比较广泛，而且也是面试题中的常客，所以本节就结合几个典型的例子来体味下回文之趣。

解法一

同时从字符串头尾开始向中间扫描字串，如果所有字符都一样，那么这个字串就是一个回文。采用这种方法的话，我们只需要维护头部和尾部两个扫描指针即可。

代码如下：：

```
/**
 *检查字符串s是不是回文， 字符串s的长度为n
 *Copyright(C) fairywell 2011
 */
bool IsPalindrome(const char *s, int n)
{
    if (s == NULL || n < 1) return false; // 非法输入
    char *front, *back;
    front = s; back = s + n - 1; // 初始化头指针和尾指针
    while (front < back) {
        if (*front != *back)
          return false; // 不是回文,立即返回
```

```
        ++front;
        --back;
    }
    return true; // 是回文
}
```

这是一个直白且效率不错的实现，时间复杂度：O(n)，空间复杂度：O(2)。

解法二

上述解法一从两头向中间扫描，那么是否还有其它办法呢？我们可以先从中间开始、然后向两边扩展查看字符是否相等。参考代码如下：

```
/**
 *检查字符串s是不是回文，字符串s的长度为n
 *Copyright(C) fairywell 2011
 */
bool IsPalindrome2(const char *s, int n)
{
    if (s == NULL || n < 1) return false; // 非法输入
    char *first, *second;
    int m = ((n >> 1) - 1) >= 0 ? (n >> 1) - 1 : 0; // m is themiddle point of s
    first = s + m; second = s + n - 1 - m;
    while (first >= s)
          if (s[first--] != s[second++]) return false; // not equal, so it's not
apalindrome
    return true; // check over, it's a palindrome
}
```

时间复杂度：O(n)，空间复杂度：O(1)。

虽然本解法二的时空复杂度和解法一是一样的，但很快我们会看到，在某些回文问题里面，这个方法有着自己的独到之处，可以方便的解决一类问题。

举一反三

1、判断一条单向链表是不是“回文”

分析:对于单链表结构,可以用两个指针从两端或者中间遍历并判断对应字符是否相等。但这里的关键就是如何朝两个方向遍历。由于单链表是单向的,所以要向两个方向遍历的话,可以采取经典的快慢指针的方法,即先位到链表的中间位置,再将链表的后半逆置,最后用两个指针同时从链表头部和中间开始同时遍历并比较即可。

2、判断一个栈是不是“回文”

分析:对于栈的话,只需要将字符串全部压入栈,然后依次将各字符出栈,这样得到的就是原字符串的逆置串,分别和原字符串各个字符比较,就可以判断了。

最长回文子串的长度

题目描述

给定一个字符串，求它的最长回文子串的长度。

分析与解法

最容易想到的办法是枚举所有的子串,分别判断其是否为回文。这个思路初看起来是正确的,但却做了很多无用功,如果一个长的子串包含另一个短一些的子串,那么对子串的回文判断其实是不需要的。

解法一

那么如何高效的进行判断呢?借鉴KMP算法的做法,既然对短的子串的判断和包含它的长的子串的判断重复了,我们何不复用下短的子串的判断呢,即让短的子串的判断成为长的子串的判断的一个部分。

没错,扩展法。从一个字符开始,向两边扩展,看看最多能到多长,使其保持为回文。这也就是为什么我们在上一节里面要提出解法二的原因。

具体而言,我们可以枚举中心位置,然后再在该位置上用扩展法,记录并更新得到的最长的回文长度,即为所求。代码如下:

```
/**
 *find the longest palindrome in a string, n is the length of string s
 *Copyright(C) fairywell 2011
 */
int LongestPalindrome(const char *s, int n)
{
    int i, j, max;
    if (s == 0 || n < 1) return 0;
    max = 0;
    for (i = 0; i < n; ++i) { // i is the middle point of the palindrome
         for (j = 0; (i-j >= 0) && (i+j < n); ++j) // if the lengthof the palindrome
is odd
            if (s[i-j] != s[i+j])
              break;
        if (j*2+1 > max)
          max = j * 2 + 1;
        for (j = 0; (i-j >= 0) && (i+j+1 < n); ++j) // for theeven case
            if (s[i-j] != s[i+j+1])
              break;
        if (j*2+2 > max)
          max = j * 2 + 2;
    }
    return max;
}
```

代码稍微难懂一点的地方就是内层的两个 for 循环，它们分别对于以 i 为中心的，长度为奇数和偶数的两种情况，整个代码遍历中心位置 i 并以之扩展，找出最长的回文。

当然，还有更先进但也更复杂的方法，比如用 s 和逆置 s’ 的组合 s$s’ 来建立后缀树的方法也能找到最长回文，但构建的过程比较复杂，所以在实践中用的比较少，感兴趣的朋友可以参考相应资料。–well。

解法二、O(N)解法

参考：http://bbs.dlut.edu.cn/bbstcon.php?board=Competition&gid=23474，或这篇文章：http://www.felix021.com/blog/read.php?2040，或：http://leetcode.com/2011/11/longest-palindromic-substring-part-ii.html，待续。

字符串其它问题

1、第一个只出现一次的字符

题目描述

在一个字符串中找到第一个只出现一次的字符。如输入abaccdeff，则输出b。

分析与解法

分析：这道题是2006年google 的一道笔试题。它在今年又出现了，不过换了一种形式。即最近的搜狐笔试大题：数组非常长，如何找到第一个只出现一次的数字，说明算法复杂度。此问题已经在程序员编程艺术系列第二章中有所阐述，在此不再作过多讲解。

代码，可编写如下：

```
//查找第一个只出现一次的字符，第2个程序
//copyright@ yansha
//July、updated,2011.04.24.
char FirstNotRepeatChar(char* pString)
{
    if (!pString)
        return '\0';

    const int tableSize = 256;
    int hashTable[tableSize] = {0}; //存入数组，并初始化为0

    char* pHashKey = pString;
    while (*(pHashKey) != '\0')
        hashTable[*(pHashKey++)]++;

    while (*pString != '\0')
    {
        if (hashTable[*pString] == 1)
            return *pString;
```

```
        pString++;
    }
    return '\0';   //没有找到满足条件的字符,退出
}
```

2、带通配符的字符串匹配问题

字符串匹配问题:给定一个字符串,按照指定规则对其进行匹配,并将匹配的结果保存至output数组中。多个匹配项用空格间隔,最后一个不需要空格。

要求:

1. 匹配规则中包含通配符?和*,其中?表示匹配任意一个字符,*表示匹配任意多个(>=0)字符。
2. 匹配规则要求匹配最大的字符子串,例如a*d,匹配abbdd而非abbd,即最大匹配子串。
3. 匹配后的输入串不再进行匹配,从当前匹配后的字符串重新匹配其他字符串。

请实现函数:

```
char* my_find(char input[], char rule[])
```

举例说明:

input:abcadefg

rule:a?c

output:abc

input :newsadfanewfdadsf

rule: new

output: new new

input :breakfastfood

rule: f*d

output:fastfood

31.1.jpg

注意事项：

1. 自行实现函数my_find，勿在my_find函数里夹杂输出，且不准用C、C++库，和Java的String对象；
2. 请注意代码的时间，空间复杂度，及可读性，简洁性；
3. input=aaa，rule=aa时，返回一个结果aa，即可。

第二章导读

慢慢的你你会发现解决面试编程题的几种常用思路，就是活用数据结构+算法，例如：

- 暴力穷举（递归回溯）——“万能的”，如求n个数的全排列 & 8皇后（N皇后问题）；
- 分治，分而治之，然后归并；
- 空间换时间，如活用hashtable；
- 巧用数据结构，如寻找最小的k个数中，用堆代替数组；
- 能排序，考虑排序，比如寻找和为定值的两个数中，用前后两个指针往中间扫；如果已经排好序了，如杨氏矩阵查找中，想想有无必要二分；
- 不能排序，考虑贪心，如最小生成树Prim, Krusal，以及最短路dijkstra，或动态规划，如01背包问题，每一步都在决策；
- 细节处理，注意边界条件。

寻找最小的k个数

题目描述

输入n个整数，输出其中最小的k个。

分析与解法

解法一

要求一个序列中最小的k个数，按照惯有的思维方式，则是先对这个序列从小到大排序，然后输出前面的最小的k个数。

至于选取什么的排序方法，我想你可能会第一时间想到快速排序（我们知道，快速排序平均所费时间为n*logn），然后再遍历序列中前k个元素输出即可。因此，总的时间复杂度：O(n * log n)+O(k)=O(n * log n)。

解法二

咱们再进一步想想，题目没有要求最小的k个数有序，也没要求最后n-k个数有序。既然如此，就没有必要对所有元素进行排序。这时，咱们想到了用选择或交换排序，即：

1、遍历n个数，把最先遍历到的k个数存入到大小为k的数组中，假设它们即是最小的k个数；
2、对这k个数，利用选择或交换排序找到这k个元素中的最大值kmax（找最大值需要遍历这k个数，时间复杂度为O(k)）；

3、继续遍历剩余n-k个数。假设每一次遍历到的新的元素的值为x，把x与kmax比较：如果x < kmax，用x替换kmax，并回到第二步重新找出k个元素的数组中最大元素kmax；如果x >= kmax，则继续遍历不更新数组。

每次遍历，更新或不更新数组的所用的时间为O(k)或O(0)。故整趟下来，时间复杂度为n*O(k)=O(n*k)。

解法三

更好的办法是维护容量为k的最大堆，原理跟解法二的方法相似：

- 1、用容量为k的最大堆存储最先遍历到的k个数，同样假设它们即是最小的k个数；
- 2、堆中元素是有序的，令k1<k2<…<kmax（kmax设为最大堆中的最大元素）
- 3、遍历剩余n-k个数。假设每一次遍历到的新的元素的值为x，把x与堆顶元素kmax比较：如果x < kmax，用x替换kmax，然后更新堆（用时logk）；否则不更新堆。

这样下来，总的时间复杂度:O(k+(n-k)*logk)=O(n*logk)。此方法得益于堆中进行查找和更新的时间复杂度均为：O(logk)（若使用解法二：在数组中找出最大元素，时间复杂度：O(k)）。

解法四

在《数据结构与算法分析–c语言描述》一书，第7章第7.7.6节中，阐述了一种在最坏情况下，时间复杂度为O(N)的快速选择算法。如下述文字：

> Since we can sort the file in O(nlog n) time, one might expect to obtain a better time bound for selection. The algorithm we present to find the kth smallest element in a set S is almost identical to quicksort. In fact, the first three steps are the same. We will call this algorithm quickselect（叫做快速选择）. Let |Si| denote the number of elements in Si（令|Si|为Si中元素的个数）. The steps of quickselect are（**步骤如下**）:
>
> 1. If |S| = 1, then k = 1 and return the elements in S as the answer. If a cutoff for small files is being used and |S| <=CUTOFF, then sort S and return the kth smallest element.
> 2. Pick a pivot element, v (- S.（选取S中一个元素作为枢纽元v）

3. Partition S - {v} into S1 and S2, as was done with quicksort.（将集合S-{v}分割成S1和S2，就像快速排序那样）
4. If k <= |S1|, then the kth smallest element must be in S1. In this case, return quickselect (S1, k). If k = 1 + |S1|, then the pivot is the kth smallest element and we can return it as the answer. Otherwise, the kth smallest element lies in S2, and it is the (k - |S1| - 1)st smallest element in S2. We make a recursive call and return quickselect (S2, k - |S1| - 1).（如果k<=|S1|，那么第k个最小元素必然在S1中。在这种情况下，返回quickselect（S1,k）。如果k=1+|S1|，那么枢纽元素就是第k个最小元素，即找到，直接返回它。否则，这第k个最小元素就在S2中，即S2中的第（k-|S1|-1）个最小元素，我们递归调用并返回quickselect（S2，k-|S1|-1））。

In contrast to quicksort, quickselect makes only one recursive call instead of two. The worst case of quickselect is identical to that of quicksort and is O(n2). Intuitively, this is because quicksort's worst case is when one of S1 and S2 is empty; thus, quickselect（快速选择）is not really saving a recursive call. The average running time, however, is O(n)（**不过，其平均运行时间为O（N）**）. The analysis is similar to quicksort's and is left as an exercise.

示例代码：

```
//copyright@ mark allen weiss
//July、updated,2011.05.05凌晨.

//q_select 将第k小的元素放在 a[k-1]
void q_select( int a[], int k, int left, int right )
{
    int i, j;
    int Pivot;

    if( left + Cutoff <= right )
    {
        Pivot = median3( a, left, right );
        //取三数中值作为枢纽元,可以消除最坏情况而保证此算法是O(N)
        i = left; j = right - 1;
        for( ; ; )
        {
            while( a[ ++i ] < Pivot ){ }
            while( a[ --j ] > Pivot ){ }
```

```
            if( i < j )
                swap( &a[ i ], &a[ j ] );
            else
                break;
        }
        swap( &a[ i ], &a[ right - 1 ] );  /* 重置枢纽元 */

        if( k <= i )
            q_select( a, k, left, i - 1 );
        else if( k > i + 1 )
            q_select( a, k, i + 1, right );
    }
    else
        insert_sort( a + left, right - left + 1 );
}
```

这个快速选择SELECT算法，类似快速排序的划分方法。N个数存储在数组S中，再从数组中选取“中位数的中位数”作为枢纽元X，把数组划分为Sa和Sb俩部分，Sa<=X<=Sb，如果要查找的k个元素小于Sa的元素个数，则返回Sa中较小的k个元素，否则返回Sa中所有元素+Sb中小的k-|Sa|个元素，这种解法在最坏情况下亦能做到O(N)的复杂度。

解法五

《算法导论》介绍了一个随机选取主元的选择算法RANDOMIZED-SELECT。它每次都是随机选取数列中的一个元素作为主元，在O(n)的时间内找到第k小的元素，然后遍历输出前面的k个小的元素。平均时间复杂度：O(n+k)=O(n)（当k比较小时）。

我们知道，快速排序是以固定的第一个或最后一个元素作为主元，每次递归划分都是不均等的，最后的平均时间复杂度为：O(n*logn)。而RANDOMIZED-SELECT与普通的快速排序不同，它每次递归都是随机选择序列，从第一个到最后一个元素中任一一个作为主元。

下面是RANDOMIZED-SELECT(A, p, r)完整伪码：

```
PARTITION(A, p, r)          //partition过程 p为第一个数，r为最后一个数
1  x ← A[r]                 //以最后一个元素作为主元
2  i ← p - 1
3  for j ← p to r - 1
4       do if A[j] ≤ x
```

```
5             then i ← i + 1
6                  exchange A[i] <-> A[j]
7  exchange A[i + 1] <-> A[r]
8  return i + 1

RANDOMIZED-PARTITION(A, p, r)       //随机快排的partition过程
1  i ← RANDOM(p, r)                                    //i  随机取p到r中个一个值
2  exchange A[r] <-> A[i]                             //以随机的 i作为主元
3  return PARTITION(A, p, r)              //调用上述原来的partition过程

RANDOMIZED-SELECT(A, p, r, i)        //以线性时间做选择,目的是返回数组A[p..r]中的第i 小的元素
1  if p = r           //p=r,序列中只有一个元素
2      then return A[p]
3  q ← RANDOMIZED-PARTITION(A, p, r)    //随机选取的元素q作为主元
4  k ← q - p + 1                         //k表示子数组 A[p…q]内的元素个数,处于划分低区的元素个数
加上一个主元元素
5  if i == k                            //检查要查找的i 等于子数组中A[p....q]中的元素个数k
6      then return A[q]          //则直接返回A[q]
7  else if i < k
8      then return RANDOMIZED-SELECT(A, p, q - 1, i)
         //得到的k 大于要查找的i 的大小,则递归到低区间A[p,q-1]中去查找
9  else return RANDOMIZED-SELECT(A, q + 1, r, i - k)
         //得到的k 小于要查找的i 的大小,则递归到高区间A[q+1,r]中去查找。
```

下面则是《算法导论》原版关于RANDOMIZED-SELECT(A, p, r)为$O(n)$的证明,阐述如下:

此RANDOMIZED-SELECT最坏情况下时间复杂度为Θ(n2),即使是要选择最小元素也是如此,因为在划分时可能极不走运,总是按余下元素中的最大元素进行划分,而划分操作需要O(n)的时间。

然而此算法的平均情况性能极好,因为它是随机化的,故没有哪一种特别的输入会导致其最坏情况的发生。

算法导论上,针对此*RANDOMIZED-SELECT算法平均时间复杂度为O(n)的证明*,引用如下,或许,能给你我多点的启示(本来想直接引用第二版中文版的翻译文字,但在中英文对照阅读的情况下,发现第二版中文版的翻译实在不怎么样,所以,得自己一个一个字的敲,最终敲完修正如下),分4步证明:

1. 当RANDOMIZED-SELECT作用于一个含有n个元素的输入数组A[p ..r]上时,所需时间是一个随机变量,记为T(n),我们可以

这样得到线性期望值E [T(n)]的下界：程序RANDOMIZED-PARTITION会以等同的可能性返回数组中任何一个元素为主元，因此，对于每一个k，（1 ≤k ≤n），子数组A[p ..q]有k个元素，它们全部小于或等于主元元素的概率为1/n.对k = 1, 2,…,n,我们定指示器X_k，为：

X_k = I{子数组A[p ..q]恰有k个元素} ,

我们假定元素的值不同，因此有

$E[X_k]=1/n$

当调用RANDOMIZED-SELECT并且选择A[q]作为主元元素的时候,我们事先不知道是否会立即找到我们所想要的第i小的元素，因为，我们很有可能需要在子数组A[p ..q - 1], 或A[q + 1 ..r]上递归继续进行寻找.具体在哪一个子数组上递归寻找，视第i小的元素与A[q]的相对位置而定.

2. 假设T(n)是单调递增的，我们可以将递归所需时间的界限限定在输入数组时可能输入的所需递归调用的最大时间（此句话，原中文版的翻译也是有问题的）.换言之,我们断定,为得到一个上界，我们假定第i小的元素总是在划分的较大的一边，对一个给定的RANDOMIZED-SELECT,指示器Xk刚好在一个k值上取1，在其它的k值时，都是取0.当Xk =1时，可能要递归处理的俩个子数组的大小分别为k-1，和n-k，因此可得到递归式为

$$\begin{aligned} T(n) &\le \sum_{k=1}^{n} X_k \cdot (T(\max(k-1, n-k)) + O(n)) \\ &= \sum_{k=1}^{n} (X_k \cdot T(\max(k-1, n-k)) + O(n)) . \end{aligned}$$

取期望值为：

$$
\begin{aligned}
&\mathrm{E}[T(n)] \\
&\le \mathrm{E}\left[\sum_{k=1}^{n} X_k \cdot T(\max(k-1, n-k)) + O(n)\right] \\
&= \sum_{k=1}^{n} \mathrm{E}[X_k \cdot T(\max(k-1, n-k))] + O(n) \qquad \text{(by linearity of expectation)} \\
&= \sum_{k=1}^{n} \mathrm{E}[X_k] \cdot \mathrm{E}[T(\max(k-1, n-k))] + O(n) \quad \text{(by equation (C.23))} \\
&= \sum_{k=1}^{n} \frac{1}{n} \cdot \mathrm{E}[T(\max(k-1, n-k))] + O(n) \qquad \text{(by equation (9.1))} \ .
\end{aligned}
$$

为了能应用等式*(C.23)*,我们依赖于X_k和T(max(k - 1,n - k))是独立的随机变量(这个可以证明,证明此处略)。

3. 下面,我们来考虑下表达式max(k - 1,n -k)的结果.我们有:

$$
\max(k-1, n-k) = \begin{cases} k-1 & \text{if } k > \lceil n/2 \rceil \ , \\ n-k & \text{if } k \le \lceil n/2 \rceil \ . \end{cases}
$$

如果n是偶数,从T(⌉)到T(n - 1)每个项在总和中刚好出现俩次,T(⌋)出现一次。因此,有

$$
\mathrm{E}[T(n)] \le \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} \mathrm{E}[T(k)] + O(n) \ .
$$

我们可以用替换法来解上面的递归式。假设对满足这个递归式初始条件的某个常数c,有T(n) ≤cn。我们假设对于小于某个常数c(稍后再来说明如何选取这个常数)的n,有T(n) =O(1)。同时,还要选择一个常数a,使得对于所有的n>0,由上式中O(n)项(用来描述这个算法的运行时间中非递归的部分)所描述的函数，可由an从上方限界得到(这里,原中文版的翻译的确是有点含糊)。利用这个归纳假设,可以得到:

$$
\begin{aligned}
\mathrm{E}[T(n)] &\le \frac{2}{n}\sum_{k=\lfloor n/2\rfloor}^{n-1} ck + an \\
&= \frac{2c}{n}\left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2\rfloor-1} k\right) + an \\
&= \frac{2c}{n}\left(\frac{(n-1)n}{2} - \frac{(\lfloor n/2\rfloor - 1)\,\lfloor n/2\rfloor}{2}\right) + an \\
&\le \frac{2c}{n}\left(\frac{(n-1)n}{2} - \frac{(n/2-2)(n/2-1)}{2}\right) + an \\
&= \frac{2c}{n}\left(\frac{n^2-n}{2} - \frac{n^2/4 - 3n/2 + 2}{2}\right) + an \\
&= \frac{c}{n}\left(\frac{3n^2}{4} + \frac{n}{2} - 2\right) + an \\
&= c\left(\frac{3n}{4} + \frac{1}{2} - \frac{2}{n}\right) + an \\
&\le \frac{3cn}{4} + \frac{c}{2} + an \\
&= cn - \left(\frac{cn}{4} - \frac{c}{2} - an\right) .
\end{aligned}
$$

> 4. 为了完成证明，还需要证明对足够大的n，上面最后一个表达式最大为cn，即要证明：cn/4 -c/2 -an ≥ 0.如果在俩边加上c/2，并且提取因子n，就可以得到n(c/4 -a) ≥c/2.只要我们选择的常数c能满足c/4 -a > 0, i.e.,即c > 4a,我们就可以将俩边同时除以c/4 -a, 最终得到：

$$
n \ge \frac{c/2}{c/4 - a} = \frac{2c}{c - 4a} .
$$

综上，如果假设对n < 2c/(c -4a),有T(n) =O(1)，我们就能得到E[T(n)] =O(n)。所以，最终我们可以得出这样的结论，并确认无疑：**在平均情况下，任何顺序统计量（特别是中位数）都可以在线性时间内得到**。

结论：RANDOMIZED-SELECT的时间复杂度为O(N)，但它在最坏情况下时间的复杂度为O(N^2)。

解法五

《算法导论》第九章第9.3节介绍了一个最坏情况线性时间的选择算法，如下：

9.3 Selection in worst-case linear time（*最坏情况下线性时间的选择算法*）

We now examine a selection algorithm whose running time isO(n) in the worst case（现在来看，一个最坏情况运行时间为O（N）的选择算法SELECT）. Like RANDOMIZED-SELECT, the algorithm SELECT finds the desired element by recursively partitioning the input array. The idea behind the algorithm, however, is toguarantee a good split when the array is partitioned. SELECT uses the deterministic partitioning algorithm PARTITION from quicksort (seeSection 7.1), modified to take the element to partition around as an input parameter（像RANDOMIZED-SELECT一样，SELECTT通过输入数组的递归划分来找出所求元素，但是，该算法的基本思想是要保证对数组的划分是个好的划分。SECLECT采用了取自快速排序的确定性划分算法partition，并做了修改，把划分主元元素作为其参数）.

The SELECT algorithm determines theith smallest of an input array ofn > 1 elements by executing the following steps. (Ifn = 1, then SELECT merely returns its only input value as theith smallest.)（算法SELECT通过执行下列步骤来确定一个有n>1个元素的输入数组中的第i小的元素。（如果n=1，则SELECT返回它的唯一输入数值作为第i个最小值。））

1. Divide then elements of the input array into⌊ groups of 5 elements each and at most one group made up of the remainingn mod 5 elements.
2. Find the median of each of the⌉ groups by first insertion sorting the elements of each group (of which there are at most 5) and then picking the median from the sorted list of group elements.
3. Use SELECT recursively to find the medianx of the⌉ medians found in step 2. (If there are an even number of medians, then by our convention,x is the lower median.)
4. Partition the input array around the median-of-mediansx using the modified version of PARTITION. Letk be one more than the number of elements on the low side of the partition, so thatx is

thekth smallest element and there aren-k elements on the high side of the partition.（利用修改过的partition过程，按中位数的中位数x对输入数组进行划分，让k比划低去的元素数目多1，所以，x是第k小的元素，并且有n-k个元素在划分的高区）

5. Ifi =k, then returnx. Otherwise, use SELECT recursively to find theith smallest element on the low side ifi k.（**如果要找的第i小的元素等于程序返回的k**，即i=k，则返回x。否则，如果ik，则在高区间找第（i-k）个最小元素）

1）将输入数组的 n 个元素划分为 $\lfloor n/5 \rfloor$ 组，每组 5 个元素，且至多只有一个组由剩下的 $n \bmod 5$ 个元素组成。

2）寻找 $\lceil n/5 \rceil$ 个组中每一组的中位数。首先对每组中的元素（至多为 5 个）进行插入排序，然后从排序过的序列中选出中位数。

3）对第 2 步中找出的 $\lceil n/5 \rceil$ 个中位数，递归调用 SELECT 以找出其中位数 x。（如果有偶数个中位数，根据约定，x 是下中位数。）

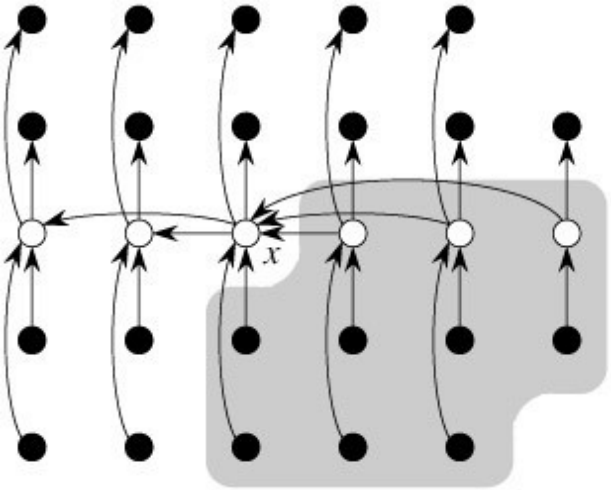
4）利用修改过的 PARTITION 过程，按中位数的中位数 x 对输入数组进行划分。让 k 比划分低区的元素数目多 1，所以 x 是第 k 小的元素，并且有 $n-k$ 个元素在划分的高区。

5）如果 $i=k$，则返回 x。否则，如果 $i<k$，则在低区递归调用 SELECT 以找出第 i 小的元素，如果 $i>k$，则在高区找第 $(i-k)$ 个最小元素。

（以上五个步骤，即本文上面的第四节末中所提到的所谓“五分化中项的中项”的方法。）

To analyze the running time of SELECT, we first determine a lower bound on the number of elements that are greater than the partitioning element x.（为了分析SELECT的运行时间，先来确定大于划分主元元素x的的元素数的一个下界）Figure 9.1 is helpful in visualizing this bookkeeping. At least half of the medians found in step 2 are greater than[1] the median-of-medians x. Thus, at least half of the ⌉ groupscontribute 3 elements that are greater than x, except for the one group that has fewer than 5 elements if 5 does not dividen exactly, and the one group containingx itself. Discounting these two groups, it follows that the number of elements greater thanx is at least：

$$3\left(\left\lceil \frac{1}{2}\left\lceil \frac{n}{5} \right\rceil \right\rceil - 2\right) \geq \frac{3n}{10} - 6 .$$



（Figure 9.1: 对上图的解释或称对SELECT算法的分析：n个元素由小圆圈来表示，并且每一个组占一纵列。组的中位数用白色表示，而各中位数的中位数x也被标出。（当寻找偶数数目元素的中位数时，使用下中位数）。箭头从比较大的元素指向较小的元素，从中可以看出，在x的右边，每一个包含5个元素的组中都有3个元素大于x，在x的左边，每一个包含5个元素的组中有3个元素小于x。大于x的元素以阴影背景表示。）

Similarly, the number of elements that are less thanx is at least 3n/10 - 6. Thus, in the worst case, SELECT is called recursively on at most 7n/10 + 6 elements in step 5.

We can now develop a recurrence for the worst-case running timeT(n) of the algorithm SELECT. Steps 1, 2, and 4 take O(n) time. (Step 2 consists ofO(n) calls of insertion sort on sets of sizeO(1).) Step 3 takes timeT(⌉), and step 5 takes time at mostT(7n/10+ 6), assuming thatT is monotonically increasing. We make the assumption, which seems unmotivated at first, that any input of 140 or fewer elements requiresO(1) time; the origin of the magic constant 140 will be clear shortly. We can therefore obtain the recurrence：

$$T(n) \le \begin{cases} \Theta(1) & \text{if } n \le 140, \\ T(\lceil n/5 \rceil) + T(7n/10 + 6) + O(n) & \text{if } n > 140. \end{cases}$$

We show that the running time is linear by substitution. More specifically, we will show thatT(n) ≤cn for some suitably large constant

c and alln > 0. We begin by assuming thatT(n) ≤cn for some suitably large constantc and alln ≤ 140; this assumption holds ifc is large enough. We also pick a constanta such that the function described by theO(n) term above (which describes the non-recursive component of the running time of the algorithm) is bounded above byan for alln > 0. Substituting this inductive hypothesis into the right-hand side of the recurrence yields

```
T(n) ≤ c⌉ +c(7n/10 + 6) +an
     ≤ cn/5 +c + 7cn/10 + 6c +an
     = 9cn/10 + 7c +an
     = cn + (-cn/10 + 7c +an) ,
```

which is at mostcn if
$-cn/10 + 7c + an \le 0$.

Inequality (9.2) is equivalent to the inequalityc ≥ 10a(n/(n - 70)) when n > 70. Because we assume thatn ≥ 140, we have n/(n - 70) ≤ 2, and so choosing c ≥ 20a will satisfyinequality (9.2). (Note that there is nothing special about the constant 140; we could replace it by any integer strictly greater than 70 and then choosec accordingly.) The worst-case running time of SELECT is therefore linear（**因此，此SELECT的最坏情况的运行时间是线性的**）.

As in a comparison sort (seeSection 8.1), SELECT and RANDOMIZED-SELECT determine information about the relative order of elements only by comparing elements. Recall fromChapter 8 that sorting requiresΩ(n lgn) time in the comparison model, even on average (see Problem 8-1). The linear-time sorting algorithms in Chapter 8 make assumptions about the input. In contrast, the linear-time selection algorithms in this chapter do not require any assumptions about the input. They are not subject to the Ω(n lgn) lower bound because they manage to solve the selection problem without sorting.

（与比较排序（算法导论8.1节）中的一样，SELECT和RANDOMIZED-SELECT仅通过元素间的比较来确定它们之间的相对次序。在算法导论第8章中，我们知道在比较模型中，即使在平均情况下，排序仍然要O(n*logn)的时间。第8章得线性时间排序算法

在输入上做了假设。**相反地，本节提到的此类似partition过程的SELECT算法不需要关于输入的任何假设，它们不受下界O(n*lgn)的约束，因为它们没有使用排序就解决了选择问题**（看到了没，道出了此算法的本质阿））

Thus, the running time is linear because these algorithms do not sort; the linear-time behavior is not a result of assumptions about the input, as was the case for the sorting algorithms inChapter 8. Sorting requiresΩ(n lgn) time in the comparison model, even on average (see Problem 8-1), and thus the method of sorting and indexing presented in the introduction to this chapter is asymptotically inefficient.（所以，本节中的选择算法之所以具有线性运行时间，是因为这些算法没有进行排序；线性时间的结论并不需要在输入上所任何假设，即可得到。）

举一反三

1、谷歌面试题：输入是两个整数数组，他们任意两个数的和又可以组成一个数组，求这个和中前k个数怎么做？

分析：

“假设两个整数数组为A和B，各有N个元素，任意两个数的和组成的数组C有N^2个元素。
那么可以把这些和看成N个有序数列：

```
A[1]+B[1] <= A[1]+B[2] <= A[1]+B[3] <=…
A[2]+B[1] <= A[2]+B[2] <= A[2]+B[3] <=…
…
A[N]+B[1] <= A[N]+B[2] <= A[N]+B[3] <=…
```

问题转变成，在这N^2个有序数列里，找到前k小的元素”

2、有两个序列A和B,A=(a1,a2,…,ak),B=(b1,b2,…,bk)，A和B都按升序排列。对于1<=i,j<=k，求k个最小的（ai+bj）。要求算法尽量高效。

3、给定一个数列a1,a2,a3,…,an和m个三元组表示的查询，对于每个查询(i，j，k)，输出ai，ai+1，…，aj的升序排列中第k个数。

求数组中给定下标区间内的第 K 小元素

题目描述

给定数组，给定区间，求第K小的数如何处理？求最小的k个元素用最大堆，求最大的k的元素用最小堆。

分析与解法

常规解法

1、排序，快速排序。我们知道，快速排序平均所费时间为$n*logn$，从小到大排序这n个数，然后再遍历序列中后k个元素输出，即可，总的时间复杂度为$O(n*logn+k)=O(n*logn)$。

2、排序，选择排序。用选择或交换排序，即遍历n个数，先把最先遍历到得k个数存入大小为k的数组之中，对这k个数，利用选择或交换排序，找到k个数中的最大数kmax（kmax设为k个元素的数组中最大元素），用时$O(k)$（你应该知道，插入或选择排序查找操作需要$O(k)$的时间），后再继续遍历后$n-k$个数，x与kmax比较：如果$x<kmax$，则x代替kmax，并再次重新找出k个元素的数组中最大元素kmax‘（多谢jiyeyuran 提醒修正）；如果$x>kmax$，则不更新数组。这样，每次更新或不更新数组的所用的时间为$O(k)$或$O(0)$，整趟下来，总的时间复杂度平均下来为：$n*O(k)=O(n*k)$。

3、维护k个元素的最大堆，原理与上述第2个方案一致，即用容量为k的最大堆存储最先遍历到的k个数，并假设它们即是最小的k个数，建堆费时$O(k)$，有$k1<k2<...kmax$（kmax设为最大堆中的最大元素）。继续遍历数列，每次遍历一个元素x，与堆顶元素比较，若$x<kmax$，则更新堆（用时$logk$），否则不更新堆。这样下来，总费时$O(k+(n-k)*logk)=O(N*logK)$。此方法得

益于在堆中，查找等各项操作时间复杂度均为logk（不然，就如上述思路2所述：直接用数组也可以找出最大的k个元素，用时O(n*k)）。

4、按编程之美上解法二的所述，类似快速排序的划分方法，N个数存储在数组S中，再从数组中随机选取一个数X，把数组划分为Sa和Sb俩部分，Sa<=X<=Sb，如果要查找的k个元素小于Sa的元素个数，则返回Sa中较小的k个元素，否则返回Sa中所有的元素+Sb中较小的k-|Sa|个元素。不断递归下去，把问题分解成更小的问题，平均时间复杂度为O(N)。

伴随数组解法

我们知道，通常来讲，寻找给定区间内的第k小的元素的问题是ACM中一类常用的数据结构的一个典型例题，即划分树/逆向归并树，通常用线段树的结构存储。

当然这里暂且不表，尚不说划分树思想的神奇，就是线段树的结构，一般没有ACM基础的人也都觉得难以理解。所以，这里提供一个时间效率尚可，空间代价还要略小的巧妙解法—伴随数组。

首先，定义一个结构体，一个是数组元素，另一个是数组原来的标号，记录每个数在数组的原顺序。

我们以下面的测试数据举例（红体部分表示下标为2~5之间的数5,2,6,3，浅色部分表示数组中的数各自对应的数组下标，淡蓝色部分为给定的下标区间，注，这里，我们让数组下标从1开始）：

a[i].data	1	5	2	6	3	7	4
a[i].num	1	2	3	4	5	6	7

现在，题目给定了下标区间，如在原序列中下标2~5（即下标为2,3,4,5）区间找到第3小的数。问题亦相当于要你找原序列里给定下标区间即第2个数到第5个数之中（5,2,6,3）第3小的数（当然，答案很明显，第3小的数就是5）。

那么对原数组进行排序，然后得到的序列应该是（**注：原下标始终保持不变**）：

a [i].data 1 2 3 4 5 6 7

a [i].num 1 3 5 7 2 4 6

如上，既然数据现在已经从小到大排好了，那么，我们只需要进行一次检索，从最小的数到最大的数，我们找第k(k=3)小的数，当我们发现下标a[i].num等于原给定下标区间在2~5中，即a[i].num==2 || 3 || 4 || 5的时候，k–，那么当k==0的时候，我们也就找到了第k(3)小的数了。如下（红色部分表示原给定下标区间中的数，浅色部分依然是原各数对应的下标，淡蓝色部分为原来给定的下标区间所对应的索引）：

a [i].data 1 2 3 4 **5** 6 7

a [i].num 1 3 5 7 2 4 6

k 3 2 1 1 0

故下标索引为2~5之间第k（3）小的数是5。

程序的构造与解释：由于排序后，我们能保证原序列已经从小到大的排好序了，所以，当遍历或扫描到原序列给定下标区间中的数时，则k–，最终能在k==0时， 找到第k小的数，且这个数是在原来给定下标区间中的某一个数。

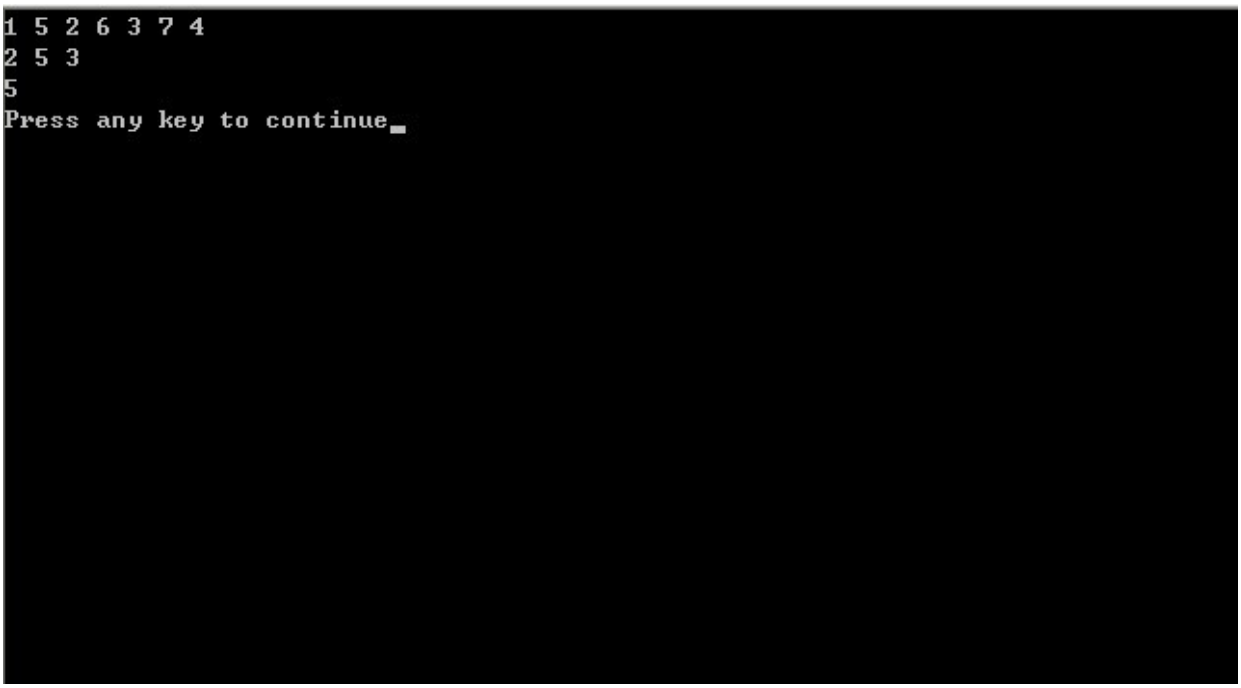
而这个伴随数组，或者说原序列各数的索引则帮我们或者说是帮电脑记下了原来的数，已让我们后来遍历时能识别排序后序列中的数是否是给定下标区间中的某一个数。如果是原给定下标区间中的数，则k–，否则k不变。

上述采用伴随数组的方法巧妙且简单，也很好理解和实现，关键 就是在于题目要求是在给定下标区间中找寻第k小（大）的元素，所以，基本上在排序n*logn完了之后，总能 在O(n)的时间内找到想找的数。源代码如下：

```
//copyright@ 水 && July
//总的时间复杂度为O(N*logN+N)= O(N*logN)。
//July、updated,2011.05.28.凌晨。
#include<iostream>
#include<algorithm>
using namespace std;

struct node{
```

```
    int num,data;
    bool operator < (const node &p) const
    {
        return data < p.data;
    }
};
node p[100001];

int main()
{
    int n=7;
    int i,j,a,b,c;//c:flag;

    for(i=1;i<=n;i++)
    {
        scanf("%d",&p[i].data);
        p[i].num = i;
    }
    sort(p+1,p+1+n);     //调用库函数sort完成排序,复杂度n*logn

    scanf("%d %d %d",&a,&b,&c);
    for(i=1;i<=n;i++)    //扫描一遍，复杂度n
    {
        if(p[i].num>=a && p[i].num<=b)
            c--;
        if(c == 0)
            break;
    }
    printf("%d/n",p[i].data);
    return 0;
}
```

程序测试：输入的第1行数字1 5 2 6 3 7 4代表给定的数组，第二行的数字中，2 5代表给定的下标区间2~5，3表示要在给定的下标区间2~5中寻找第3小的数，第三行的5表示找到的第3小的数。程序运行结果如下：



水原来写的代码(上面我的改造,是为了达到后来扫描时O(N)的视觉效果):

```
//copyright@ 水
#include<iostream>
#include<algorithm>
using namespace std;

struct node{
    int num,data;
    bool operator < (const node &p) const
    {
        return data < p.data;
    }
};
node p[100001];

int main()
{
    int n,m,i,j,a,b,c;//c:flag;
    while(scanf("%d %d",&n,&m)!=EOF)
    {
        for(i=1;i<=n;i++)
        {
            scanf("%d",&p[i].data);
            p[i].num = i;
        }
```

```
        sort(p+1,p+1+n);

        for(j=1;j<=m;j++)
        {
            scanf("%d %d %d",&a,&b,&c);
            for(i=1;i<=n;i++)
            {
                if(p[i].num>=a && p[i].num<=b)
                    c--;
                if(c == 0)
                    break;
            }
            printf("%d/n",p[i].data);
        }
    }
    return 0;
}
```

问题扩展

你可能会忽略一个重要的事实，不知读者是否意识到。题目是要求我们在数组中求给定下标区间内某一第k小的数，即我们只要找到这个第k小的数，就够了。但上述程序显示的一个弊端，就是它先对整个数组进行了排序，然后采用伴随数组的解法寻找到第k小的数。而事实是，我们不需要对整个数组进行排序，我们只需要对我们要寻找的那个数的数组中给定下标区间的数进行部分排序，即可。

对，事情就是这么简单。我们摒弃掉伴随数组的方法，只需要直接对数组中给定的那部分下标区间中的数进行排序，而不是对整个数组进行排序。如此的话，算法的时间复杂度降到了L*logK。其中，L=|b-a+1|，L为给定下标区间的长度，相对整个数组的程度n，L<=n。程序代码如下。

```
//copyright@ 苍狼
//直接对给定区间的数进行排序，没必要用伴随数组。
#include<iostream>
#include<algorithm>
using namespace std;

struct node{
    int data;
    bool operator < (const node &p) const
    {
        return data < p.data;
```

```
    }
};
node p[100001];

int main()
{
    int n=7;
    int i,a,b,c;//c:flag;

    for(i=1;i<=n;i++)
    {
        scanf("%d",&p[i].data);
    }

    scanf("%d%d%d", &a, &b, &c);    //b,a为原数组的下标索引
    sort(p+a, p+b+1);      //直接对给定区间进行排序,|b-a+1|*log(b-a+1)

    printf("The number is %d/n", p[a-1+c].data);
    return 0;
}
```

程序测试:我们同样采取第二节的测试用例。输入的第1行数字1 5 2 6 3 7 4代表给定的数组,第二行的数字中,2 5代表给定的下标区间2~5,3表示要在给定的下标区间2~5中的数,即从a[2]~a[5]中寻找第3小的数,第三行的5表示找到的第3小的数。程序运行结果如下。

貌似上述直接对给定区间内的数进行排序，效率上较第二节的伴随数组方案更甚一筹。既然如此，那么伴随数组是不是多此一举呢?其实不然，@水：假如，我对2-5之间进行了排序，那么数据就被摧毁了，怎么进行2次的操作？就是现在的2位置已经不是初始的2位置的数据了。也就是说，快排之后下标直接定位的方法明显只能用一次。

ok，更多请看下文第四节中的“百家争鸣”与“经典对白”。

解法总结

百家争鸣

- @雨翔：伴随数组这种方式确实比较新颖，伴随数组的前提是在排序后的，但总的复杂度**还是**`O(N*logN+N)=O(N*logN)`，找第K大的数的此类面试题都是有这几点限制：**1**、数很多，让你在内存中放不下，**2**、复杂度严格要求，即不能用排序。当然，即便第三节中，直接对给定下标区间进行排序，复杂度同样为`L*logL`，L为给定区间的长度。事实上，我们在解决“从给定下标区间中的数找寻第k小（大）的元素”这个问题，还是选择堆为好，在之前的基础上：入堆的时候 只需检测这个元素的下标是否是给定下标区间内的，不是则不入这样的复杂度会低，不需要排序。然后便是平均时间复杂度虽为O(N)，但并不常用的快速选择SELECT算法，参考：第三章再续：快速选择SELECT算法的深入分析与实现。

- @水：伴随数组的解法是为了达到预处理开销换查找开销目的。直接对给定不同的下标区间的数进行排序，在小数据量处理时复杂度还可以接受，但当面临大数据量，即海量数据处理时，比如10G的数据量，每次取1G的段的问题，则使用伴随数组的方法会凸显优势，只不过预处理的开销的确是大了点。伴随数组的精髓就是稳定的时间之内解决对相同数据的多次访问查找。说白了，就是同一个数组，要不断查找数组中给定的不同下标区间中的第k小的数时优势明显。具体，还可以看看这道题。

- @July：不用看我了，基本上同意上述水的观点。雨翔之所以认为伴随数组不可取，是因为没有考虑到水提出的问题，即如果要多次或不断的从数组中不同的下标区间中寻找第k小的数的情况。这时，伴

随数组的优势就体现出来了。ok，读者还可以继续看下面的经典对白。相信，你能找到你想要的答案。

经典对白

- 查找a[0]$_{a[n-1]内第K小，然后再找a[1]}$a[n]内第K小，依次往复，找个几次就优势明显了。其实是比较采取伴随数组解法n log n +m*n的代价（m为给定不同区间的个数）和直接排序m*(L*log L)（L为给定下标区间的长度）的代价，哪个更低。其中，采用伴随数组查找最差情况是`nlogn + m(n-1)`，而直接排序代价，最差情况为`m * ((n-1)*log(n-1))`。当m>>0且n>>0时，排序时间 - 伴随时间 = `m*n*logn - n*logn - mn` = `(m-1)n*logn -mn`恒正，结论：即在需要不断的从不同给定下标区间中寻找第k小数的情况下，当数据规模大的时候伴随数组效果恒优于每次都直接对给定的下标区间的部分数进行排序。

- 是的，好比我现在给定不同的另外一个下标区间，要你从中查找第k小的数，你总不能每次都排序吧。而采取伴随数组的方案的话，由于伴随数组记下了各自给定的下标区间对应的数。所以，第二次在不同的下标区间中查找第k小的数时，还是只要扫描一遍即可找到，复杂度还是 `O(N)`。从而，给定不同的下标区间查找第k小的数，复杂度为`m*N`加上之前排序预处理的复杂度，`N*logN`，总的时间复杂度为`O(N*logN + m*N)`（m为给定不同区间的个数）。而直接对给定下标区间中的数进行排序的代价则为`l1*logl1 + l2*logl2 + ... + li*logli`。当m>>0且n>>0时，哪个复杂度谁大谁小，一眼就看出来了伴随数组所体现的巨大优势。

- 恩，实际样例是这样的，我们有每天超过100万次点击的网页，我们常见的来源有n种，然后，我们要确定每天的每个时段和一周乃至整个月的点击来源地分析。数据库的库存数据量庞大，copy花销很大，内排序花销更大，如果要做出这样的统计图，我擦泪，如果每次都排序，玩死了。

原例重现

ok，说了这么多，你可能还根本就不明白到底是怎么一回事。让我们从第一节举的那个例子说起。我们要找给定下标区间2~5的数中第3小的

数。诚然，此时，我们有两种选择：**1**、如上第一节、第二节所述的伴随数组，**2**、直接对下标区间2-5的数进行排序。下面，只回顾下伴随数组的方案。

伴随数组

a[i].data 1 5 2 6 3 7 4

a[i].num 1 2 3 4 5 6 7

第一次排序后：

a [i].data 1 2 3 4 5 6 7

a [i].num 1 3 5 7 2 4 6

伴随数组方案查找：

a [i].data 1 2 3 4 **5** 6 7

a [i].num 1 3 5 7 2 4 6

k 3 2 1 1 0

好的，那么现在，如果题目要求你在之前数组的下标区间3~6的数中找第3小的数呢（答案很明显，为6）？

a[i].data 1 5 2 6 3 7 4

a[i].num 1 2 3 4 5 6 7

1. 直接排序么?ok，退万一步讲，假设有的读者可能还是会依然选择直接排序下标3~6之间的数。但你是否可曾想到，每次对不同的下标区间所对应的数进行排序，你不但破坏了原有的数据，而且如果**区间有覆盖**的话，那么将使得我们无法再能依靠原有的直接的下标定位找到原来的数据，且每进行一次排序，都要花费平均时间复杂度为`O(N*logN)`的时间开销。如上面的经典对白所述，这样下去的开销将非常大，将为`l1*logl1 + l2*logl2 + ... + li*logli`。

2. 那么，如果是采取伴随数组的方法，我们要怎么做呢?如下所示，我们在k=0的时候，同样找到了第3小的数6，如此是不是只要在之前的一次排序，以后不论是换各种不同的下标区间时都能扫描一遍$O(N)$搞定？复杂度为$O(N*logN + m*N)$（m为给定不同的下标区间的区间数）。

3. 由上面的经典对白里面的内容，我们已经知道，当m>>0且n>>0时（m为给定不同的下标区间的区间数，n为数组大小），排序时间 - 伴随时间 = $m*n*logn - n*logn - mn = (m-1)n*logn -mn$恒正。yeah，相信，你已经明白了。

伴随数组

第一次排序后：

```
a [i].data  1 2 3 4 5 6 7
a [i].num   1 3 5 7 2 4 6
```

再次扫描，直接$O(N)$搞定：

```
a [i].data   1 2 3 4 5 6 7
a [i].num    1 3 5 7 2 4 6
  k        3 2 1 1 1 0
```

（而之前有的读者意识不到伴随数组的意义，是因为一般的人只考虑找一次，不会想到第二次或多次查找）

编程独白

给你40分钟的时间，你可以思考十分钟，然后用三十分钟的时间来写代码，最后浪费在无谓的调试上；你也可以思考半个小时，彻底弄清问题的本质与程序的脉络，然后用十分钟的时间来编写代码，体会代码如行云流水而出的感觉。

本章完。

求解500万以内的亲和数

题目描述

求500万以内的所有亲和数

如果两个数a和b，a的所有真因数之和等于b,b的所有真因数之和等于a,则称a,b是一对亲和数。例如220和284，1184和1210，2620和2924。

思路分析

首先得明确到底是什么是亲和数?

亲和数问题最早是由毕达哥拉斯学派发现和研究的。他们在研究数字的规律的时候发现有以下性质特点的两个数:

220的真因子是：1、2、4、5、10、11、20、22、44、55、110；

284的真因子是：1、2、4、71、142。

而这两个数恰恰等于对方的真因子各自加起来的和（sum[i]表示数i 的各个真因子的和），即

220=1+2+4+71+142=sum[284],

284=1+2+4+5+10+11+20+22+44+55+110=sum[220]。

得284的真因子之和sum[284]=220，且220的真因子之和sum[220]=284，即有sum[220]=sum[sum[284]]=284。

如此，是否已看出丝毫端倪?

如上所示，考虑到1是每个整数的因子，把出去整数本身之外的所有因子叫做这个数的“真因子”。如果两个整数，其中每一个真因子的和都恰好

等于另一个数，那么这两个数，就构成一对“亲和数”（有关亲和数的更多讨论，可参考这里）。

求解：了解了什么是亲和数，接下来咱们一步一步来解决上面提出的问题：

1. 看到这个问题后，第一想法是什么？模拟搜索+剪枝？回溯？时间复杂度有多大？其中bn为an的伪亲和数，即bn是an的真因数之和大约是多少？至少是10^{10}的数量级。那么对于每秒千万次运算的计算机来说，大概在1000多天也就是3年内就可以搞定了（10^10 / 10^7 =1000000秒，大约 278 小时）。如果是基于这个基数在优化，你无法在一天内得到结果的。
2. 一个不错的算法应该在半小时之内搞定这个问题，当然这样的算法有很多。节约时间的做法是可以生成伴随数组，也就是空间换时间，但是那样，空间代价太大，因为数据规模庞大。
3. 在稍后的算法中，依然使用的伴随数组，只不过，因为题目的特殊性，只是它方便和巧妙地利用了下标作为伴随数组，来节约时间。同时，将回溯的思想换成递推的思想(预处理数组的时间复杂度为**O(N*logN)**(logN为调和级数)，扫描数组的时间复杂度为线性**O(N)**。所以，总的时间复杂度仍然为**O(N*logN)**(其中logN为调和级数))，扫描数组的时间与其相比是低阶的。

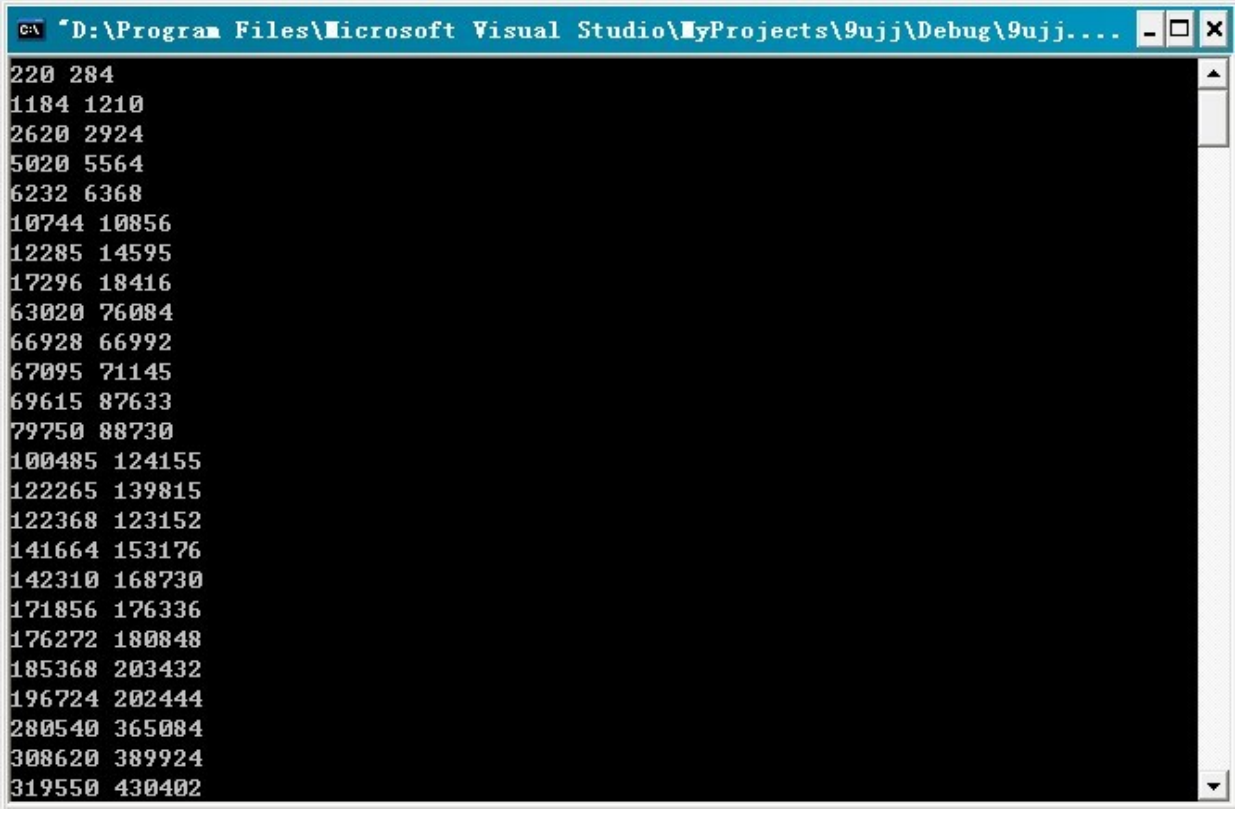
解法一、伴随数组线性遍历

依据上文的思路分析，可以编写如下代码：

```
//求解亲和数问题

//第一个for和第二个for循环是logN*N（调和级数）次遍历，第三个for循环扫描O(N)。
//所以总的时间复杂度为 O(N*logN) + O(n) =O(N*logN+N)（其中logN为调和级数）。

//关于第一个for和第二个for寻找中，调和级数的说明：
//比如给2的倍数加2，那么应该是  n/2次，3的倍数加3 应该是 n/3次，...
//那么其实就是n*(1+1/2+1/3+1/4+...1/(n/2))=n*（调和级数）=n*logn。

//copyright@ 上善若水
//July、updated，2011.05.24。
#include<stdio.h>

int sum[5000010];    //为防越界
```

```
int main()
{
    int i, j;
    for (i = 0; i <= 5000000; i++)
        sum[i] = 1;  //1是所有数的真因数所以全部置1

    for (i = 2; i + i <= 5000000; i++)  //预处理，预处理是logN（调和级数）*N。
        //@litaoye:调和级数1/2 + 1/3 + 1/4......的和近似为ln(n),
        //因此O(n *(1/2 + 1/3 + 1/4......)) = O(n * ln(n)) = O(N*log(N))。
    {
        //5000000以下最大的真因数是不超过它的一半的
        j = i + i;  //因为真因数,所以不能算本身,所以从它的2倍开始
        while (j <= 5000000)
        {
            //将所有i的倍数的位置上加i
            sum[j] += i;
            j += i;
        }
    }

    for (i = 220; i <= 5000000; i++)    //扫描，O(N)。
    {
        // 一次遍历,因为知道最小是220和284因此从220开始
        if (sum[i] > i && sum[i] <= 5000000 && sum[sum[i]] == i)
        {
            //去重,不越界,满足亲和
            printf("%d %d/n",i,sum[i]);
        }
    }
    return 0;
}
```

运行结果:



> @上善若水：> > 1、可能大家理解的还不是很清晰，我们建立一个5 000 000 的数组，从1到2 500 000开始，在每一个下标是i的倍数的位置上加上i，那么在循环结束之后，我们得到的是什么？是类似埃斯托拉晒求素数的数组（当然里面有真的亲和数），然后只需要一次遍历就可以轻松找到所有的亲和数了。时间复杂度，线性。

2、我们可以清晰的发现连续数据的映射可以通过数组结构本身的特点替代，用来节约空间，这是数据结构的艺术。在大规模连续数据的回溯处理上，可以通过转化为递推生成的方法，逆向思维操作，这是算法的艺术。

3、把最简单的东西运用的最巧妙的人，要比用复杂方法解决复杂问题的人要头脑清晰。

解法一的构造与解释

我再来具体解释下上述程序的原理，ok，举个例子，假设是求10以内的亲和数，求解步骤如下：

因为所有数的真因数都包含1，所以，先在各个数的下方全部置1

1. 然后取i=2,3,4,5（i<=10/2），j依次对应的位置为j=（4、6、8、10），（6、9），（8），（10）各数所对应的位置。依据j所找到的位置，在j所指的各个数的下面加上各个真因子i（i=2、3、4、5）。

2. 整个过程，即如下图所示（如sum[6]=1+2+3=6，sum[10]=1+2+5=8.）：

```
1  2  3  4  5  6  7  8  9  10
1  1  1  1  1  1  1  1  1  1
         2     2     2     2
               3           3
                     4
                           5
```

3. 然后一次遍历i从220开始到5000000，i每遍历一个数后，将i对应的数下面的各个真因子加起来得到一个和sum[i]，如果这个和sum[i]==某个i’，且sum[i‘]=i，那么这两个数i和i’，即为一对亲和数。

4. i=2；sum[4]+=2，sum[6]+=2，sum[8]+=2，sum[10]+=2，sum[12]+=2…i=3，sum[6]+=3，sum[9]+=3…

5. i=220 时，sum[220]=284，i=284 时，sum[284]=220；即sum[220]=sum[sum[284]]=284，得出220与284是一对亲和数。所以，最终输出220、284，…

解法二

> litaoye专门为本亲和数问题开帖子继续阐述，有兴趣的朋友可继续参见这里。同时，任何人对本亲和数问题有任何问题，也可以回复到上述帖子上。

深入理解真因子之和

假设一个整数分解因式形式为 img

那么它的所有**因子之和**（也就是真因子之和加上其本身）等于

img

举例说明：90分解质因数为

所以90的因子之和为

而这个函数f(n)有什么好性质呢？当n=p*q，并且p,q互质时，有：

$$f(n) = f(p) * f(q)$$

img

也就是说，我们只要知道n的任意一种pq分解，p、q互质，以及f(p)和f(q)的具体值，就可以快速知道f(n)的值了。

进一步，本例中我们只需要对真因子加和，设其为函数counter(n)。我们就会有counter(n)=f(n)-n的关系。如果我们能快速得到f(n)，counter(n)就不成问题，我们就能快速的验证counter(counter(n))=n来判断亲和数。

深入理解质数筛（Prime Sieves）

我们知道在解法一中，预处理时间为O(NlogN)，同时也能看到，为了给一个合数加够它的因子，计算次数相对于f(n)而言实在是太大了，而如果我们能够对每一个数最多操作一次就能知道它的f(n)值（进而知道counter(n)值），那这个算法就变成O(N)的了。

这种算法存在吗？答案是确定的。我们一边找素数一边去确定f(n)的值，那我们就不得不提到这个算法的基础算法：线性素数筛。

Eratosthenes近线性素数筛

首先介绍一种使用O(N)空间伴随数组的素数筛

```
//使用了一个布尔型伴随数组的素数筛
bool* prime(int n) {
  bool *notp = new bool[n];
```

```
  memset(notp, false, sizeof(bool)*n );
  for (int i = 2; i < n; i++)
    if (!notp[i])
      for (int j = 2*i; j < n; j+=i)
        notp[j] = true;
  return notp;
}
```

初始化伴随数组为全0后，我们对于每一个在notp中被标记0的下标i，对下标为i的2倍，3倍…的notp元素标记为“true”，即合数。这个算法的好处在于，因为我们从2开始顺序扫描到i的，如果发现i仍然没被标记过“true”，则它没有2到i-1的约束，肯定是素数。而如果一个合数被标记过“true”了，它的倍数肯定也被这个合数的第一个质因数都标记完“true”了，这个算法在标记素数上确实是不漏的，而在标记合数上，标记的次数是这个合数的不同质因数的个数。

整个算法的复杂度为**O(Nloglog(N))**，参考http://en.wikipedia.org/wiki/Prime_harmonic_series 的公式：

img

而且是渐进于这个**loglogN**的。因为loglogN工程上很小，可以当常量来估计。

Eratosthenes质数筛

进一步的我们对这个算法进行剪枝，是前述近线性素数筛的加强版，首先我们知道如果质数m小于质数p的平方，则在第p轮循环之前m就被比p还小的质数标记过了；同时对于另外的质数，如果发现它被标记过就跳过标记他。虽然加速了许多，但是考虑到比较大小和幅值的操作代价相差不多，复杂度上界并没有改变。每个合数m都被它的最小质因数标记了1次，其他质因数进行了notp[m]的寻址和比较。

```
//使用了一个布尔型伴随数组的素数筛
bool* prime_wiki(int n) {
  int i;
  int j;
  bool *notp = new bool[n];
```

```
  memset(notp, false, sizeof(bool)*n );
  double sqr = sqrt(double(n));
  for (i = 2; i < sqr; i++)
      if (!notp[i])
        for (j = i*i; j < n; j+=i)
          if(!notp[j]){notp[j] = true;}
  return notp;
}
```

线性时间筛

```
bool* prime_linear(int n) {
    bool *notp = new bool[n];
    int *plist = new int[2*n/log(double(n))+20];
    memset(notp, false, sizeof(bool)*n);
    memset(plist, 0, sizeof(int)*int(2*n/log(double(n))+20));
    int np = 0;
    int i,j;
    double half = double(n)/2;
    for (i = 2; i < n; i++) {
        if (!notp[i]) plist[np++] = i;
            if (i<half){
                  for (j = 0; (j < np) && (plist[j]*i < n); j++) {
                        notp[plist[j]*i] = 1;
                        if(i%plist[j] == 0) break;
                  }
            }
    }
    delete plist;
    return notp;
}
```

同时保存一个长度为N的伴随数组表示“p是否为合数”及长度为O(N/log(N))的数组来保存质数列表。这里，每一个质数p会保持notp[p]=0的标记并加入列表，每一个合数也只被它最大的真因数（也就是这个合数除以其最小质因数）标记一次。这样的好处我们可以看到，交换了标记次序并及时使用break，每个合数只需要1次标记和1次判断。

线性筛应用于亲和数问题

很多人都觉得近线性已经足够好了，根本不想花O(N/log(N))的空间存储素数。错！在本题中存储素数加上线性筛算法，恰恰保证了寻找合数最小质因数分解的快速性，以提高合数n对应的f(n)的计算，如下C#代码所示。

```
//求解亲和数问题
//copyright@ litaoye
//July、胡滨,updated,2011.05.26。
using System;
using System.Collections.Generic;

namespace CSharpTest
{
    class Program
    {
        public static void Main()
        {
            int max = 5000000;
            DateTime start = DateTime.Now;
            int[] counter = CreateCounter(max);

            for (int i = 0; i < counter.Length; i++)
            {
                int num = counter[i] - i;
                if (num < counter.Length && num > i && counter[num] == counter[i])
                    Console.WriteLine("{0} {1}", i, num);
            }
            Console.WriteLine((DateTime.Now - start).TotalSeconds);

            Console.ReadKey();
        }

        static int[] CreateCounter(int n)
        {
            List<int> primes = new List<int>();
            int[] counter = new int[n + 1];
            counter[1] = 1;

            for (int i = 2; i <= n; i++)
            {
                if (counter[i] == 0)
                {
                    counter[i] = i + 1;
                    primes.Add(i);
                }

                for (int j = 0; j < primes.Count; j++)
                {
                    if (primes[j] * i > n)
                        break;

                    if (i % primes[j] == 0)
                    {
```

```
                        int k = i;
                        int l = primes[j] * primes[j];

                        while (k % primes[j] == 0)
                        {
                            l *= primes[j];
                            k /= primes[j];
                        }

                         counter[primes[j] * i] = counter[k] * (l - 1) / (primes[j] -
1);
                        break;
                    }
                    else
                        counter[primes[j] * i] = counter[i] * (primes[j] + 1);
                }
            }

            return counter;
        }
    }
}

/*
测试结果：
0.484375
0.484375
0.46875
单位second。
*/
```

至此，这个算法融合了线性质数筛的最佳性质，它的步骤如下：

- 我们为1到N存储一个int型伴随数组，存储它们的因子和，而非伴随bool变量，同时也存储质数列表；
- 刷新伴随数组时，“因子和等于本身加1”等价于“质数”；
- 我们可以通过伴随数组精确的知道内层循环在哪一步break，进而知道：
  - break之前counter[j]和i本身就互质，直接得到了计算因子和的方案
  - break这一步通过获取得其最小质因数p，找到最大的k使得 img，从而实现 img，能够把m分解成两个互质整数的乘积。
- 知道了m的因子和，减去m就是m的真因子之和，进而考察counter[counter[m]] == m 析出亲和数。

寻找和为定值的两个数

题目描述

题目：输入一个数组和一个数字，在数组中查找两个数，使得它们的和正好是输入的那个数字。

要求时间复杂度是O(N)。如果有多对数字的和等于输入的数字，输出任意一对即可。

例如输入数组1、2、4、7、11、15和数字15。由于4+11=15，因此输出4和11。

解法一

咱们试着一步一步解决这个问题（注意阐述中数列有序无序的区别）：

直接穷举，从数组中任意选取两个数，判定它们的和是否为输入的那个数字。此举复杂度为O(N^2)。很显然，我们要寻找效率更高的解法。

解法二

题目相当于，对每个a[i]，查找sum-a[i]是否也在原始序列中，每一次要查找的时间都要花费为O(N)，这样下来，最终找到两个数还是需要O(N^2)的复杂度。那如何提高查找判断的速度呢？

答案是二分查找，可以将O(N)的查找时间提高到O(log N)，这样对于N个a[i]，都要花logN的时间去查找相对应的sum-a[i]是否在原始序列中，总的时间复杂度已降为O(N log N)，且空间复杂度为O(1)。（如果有序，直接二分O(N log N)，如果无序，先排序后二分，复杂度同样为O(N log N + N log N) = O(N log N)，空间复杂度总为O(1)）。

解法三

当然，你还可以构造hash表，正如编程之美上的所述，给定一个数字，根据hash映射查找另一个数字是否也在数组中，只需用O(1)的时间，这样的话，总体的算法同上述思路3 一样，也能降到O(N)，但有个缺陷，就是构造hash额外增加了O(N)的空间，此点同上述思路 3。不过，空间换时间，仍不失为在时间要求较严格的情况下的一种好办法。

解法四

有没有更好的办法呢？咱们可以依据上述思路2的思想，a[i]在序列中，如果a[i]+a[k]=sum的话，那么sum-a[i]（a[k])也必然在序列中。举个例子，如下：原始序列：1、2、4、7、11、15 用输入数字15减一下各个数，得到对应的序列为：14、13、11、8、4、0

第一个数组以一指针i 从数组最左端开始向右扫描，第二个数组以一指针j 从数组最右端开始向左扫描，如果第一个数组出现了和第二个数组一样的数，即a[*i]=a[*j]，就找出这俩个数来了。如上，i，j最终在第一个，和第二个序列中找到了相同的数4和11，所以符合条件的两个数，即为4+11=15。怎么样，两端同时查找，时间复杂度瞬间缩短到了O(N)，但却同时需要O(N)的空间存储第二个数组。

但事实上，也不一定非得开辟第二个数组，要达到O(N)的复杂度，第一个数组以一指针i 从数组最左端开始向右扫描，第二个数组以一指针j 从数组最右端开始向左扫描，首先初始i指向元素1，j指向元素0，谁指的元素小，谁先移动，由于1(i)> 0(j)，所以i不动，j向左移动。然后j移动到元素4发现大于元素1，故而停止移动j，开始移动i，直到i指向4，这时,i指向的元素与j指向的元素相同，故而判断4是满足条件的第一个数；然后同时移动i,j再进行判断，直到它们到达边界。

解法五

如果数组是无序的，先排序(N log N)，然后用两个指针i，j，各自指向数组的首尾两端，令i=0，j=n-1，然后i++，j–，逐次判断a[i]+a[j]?=sum，如果某一刻a[i]+a[j] > sum，则要想办法让sum的值减小，所以此刻i不动，j–，如

果某一刻a[i]+a[j] < sum，则要想办法让sum的值增大，所以此刻i++，j不动。所以，数组无序的时候，时间复杂度最终为O(N log N + N)=O(N log N)，若原数组是有序的，则不需要事先的排序，直接O(N)搞定，且空间复杂度还是O(1)，此思路是相对于上述所有思路的一种改进。(如果有序，直接两个指针两端扫描，时间O(N)，如果无序，先排序后两端扫描，时间O(N log N + N)=O(N log N)，空间始终都为O(1))。(与上述解法二相比，排序后的时间开销由之前的二分的n*logn降到了扫描的O(N))。

下面，咱们先来实现思路5(这里假定数组已经是有序的)，代码可以如下编写(两段代码实现)：

```
//代码一
//O(N)
Pair findSum(int *s, int n, int x)
{
    //sort(s, s+n);   如果数组非有序的,那就事先排好序O(N log N)

    int *begin = s;
    int *end = s+n-1;

    while(begin < end)     //俩头夹逼，或称两个指针两端扫描法，很经典的方法，O(N)
    {
        if(*begin + *end > x)
        {
            --end;
        }
        else if(*begin + *end < x)
        {
            ++begin;
        }
        else
        {
            return Pair(*begin, *end);
        }
    }

    return Pair(-1, -1);
}

//或者如下编写,
//代码二
//copyright@ zhedahht && yansha
//July、updated,2011.05.14。
bool find_num(int data[], unsigned int length, int sum, int& first_num, int&
```

```
second_num)
{
    if(length < 1)
        return false;

    int begin = 0;
    int end = length - 1;

    while(end > begin)
    {
        long current_sum = data[begin] + data[end];

        if(current_sum == sum)
        {
            first_num = data[begin];
            second_num = data[end];
            return true;
        }
        else if(current_sum > sum)
            end--;
        else
            begin++;
    }
    return false;
}
```

解法总结

不论原序列是有序还是无序，解决这类题有以下三种办法：1、二分（若无序，先排序后二分），时间复杂度总为O(N log N)，空间复杂度为O（1）；2、扫描一遍X-S[i] 映射到一个数组或构造hash表，时间复杂度为O(N)，空间复杂度为O(N)；3、两个指针两端扫描（若无序，先排序后扫描），时间复杂度最后为：有序O(N)，无序O(N log N + N)=O(N log N)，空间复杂度都为O(1)。

所以，要想达到时间O(N)，空间O(1)的目标，除非原数组是有序的（指针扫描法），不然，当数组无序的话，就只能先排序，后指针扫描法或二分（时间 O(N log N)，空间O(1)），或映射或hash（时间O(N)，空间O(N)）。时间或空间，必须牺牲一个，自己权衡吧。

综上，若是数组有序的情况下，优先考虑两个指针两端扫描法，以达到最佳的时（O(N)），空（O(1)）效应。否则，如果要排序的话，时间复杂度最快当然是只能达到O(N log N)，空间O(1)则是不在话下。

问题扩展

1. 如果在返回找到的两个数的同时，还要求你返回这两个数的位置列？
2. 如果把题目中的要你寻找的两个数改为“多个数”，或任意个数列？（请看下面第二节）
3. 二分查找时：`left <= right,right = middle - 1;left < right,right = middle;`

```
//算法所操作的区间,是左闭右开区间,还是左闭右闭区间,这个区间,需要在循环初始化,
//循环体是否终止的判断中,以及每次修改left,right区间值这三个地方保持一致,否则就可能出错.

//二分查找实现一
int search(int array[], int n, int v)
{
    int left, right, middle;

    left = 0, right = n - 1;

    while (left <= right)
    {
        middle = left + (right - left) / 2;
        if (array[middle] > v)
        {
            right = middle - 1;
        }
        else if (array[middle] < v)
        {
            left = middle + 1;
        }
        else
        {
            return middle;
        }
    }

    return -1;
}

//二分查找实现二
```

```
int search(int array[], int n, int v)
{
    int left, right, middle;

    left = 0, right = n;

    while (left < right)
    {
        middle = left + (right - left) / 2;

        if (array[middle] > v)
        {
            right = middle;
        }
        else if (array[middle] < v)
        {
            left = middle + 1;
        }
        else
        {
            return middle;
        }
    }

    return -1;
}
```

举一反三

1、在二元树中找出和为某一值的所有路径 输入一个整数和一棵二元树，从树的根结点开始往下访问一直到叶结点所经过的所有结点形成一条路径，然后打印出和与输入整数相等的所有路径。例如输入整数22和如下二元树

10

/
5 12
/
4 7

则打印出两条路径：10, 12和10, 5, 7。 其中，二元树节点的数据结构定义为：

```
struct BinaryTreeNode // a node in the binary tree
{
    int m_nValue; // value of node
    BinaryTreeNode *m_pLeft; // left child of node
    BinaryTreeNode *m_pRight; // right child of node
};
```

2、3-sum问题

3、4-sum问题

第五章、寻找和为定值的多个数

题目描述

输入两个整数 n 和 m, 从数列1, 2, 3.....n 中 随意取几个数,

使其和等于 m ,要求将其中所有的可能组合列出来。

解法一

我想, 稍后给出的程序已经足够清楚了, 就是要注意到放n, 和不放n个区别, 即可, 代码如下:

```
// 21题递归方法
//copyright@ July && yansha
//July、yansha,updated。
#include<list>
#include<iostream>
using namespace std;

list<int>list1;
void find_factor(int sum, int n)
{
    // 递归出口
    if(n <= 0 || sum <= 0)
        return;

    // 输出找到的结果
    if(sum == n)
    {
        // 反转list
        list1.reverse();
        for(list<int>::iterator iter = list1.begin(); iter != list1.end(); iter++)
            cout << *iter << " + ";
        cout << n << endl;
        list1.reverse();
    }

    list1.push_front(n);       //典型的01背包问题
```

```
    find_factor(sum-n, n-1);    //放n,n-1个数填满sum-n
    list1.pop_front();
    find_factor(sum, n-1);      //不放n,n-1个数填满sum
}

int main()
{
    int sum, n;
    cout << "请输入你要等于多少的数值sum:" << endl;
    cin >> sum;
    cout << "请输入你要从1.....n数列中取值的n:" << endl;
    cin >> n;
    cout << "所有可能的序列,如下:" << endl;
    find_factor(sum,n);
    return 0;
}
```

解法二

@zhouzhenren：

这个问题属于子集和问题（也是背包问题）。本程序采用 回溯法+剪枝

X数组是解向量，t=∑(1,..,k-1)Wi*Xi, r=∑(k,..,n)Wi

若t+Wk+W(k+1)<=M,则Xk=true，递归左儿子(X1,X2,..,X(k-1),1)；否则剪枝；

若t+r-Wk>=M && t+W(k+1)<=M,则置Xk=0，递归右儿子(X1,X2,..,X(k-1),0)；否则剪枝；

本题中W数组就是(1,2,..,n),所以直接用k代替WK值。

代码编写如下：

```
//copyright@ 2011 zhouzhenren

//输入两个整数 n 和 m,从数列1,2,3.......n 中 随意取几个数,
//使其和等于 m ,要求将其中所有的可能组合列出来。

#include <stdio.h>
```

```
#include <stdlib.h>
#include <memory.h>

/**
 * 输入t, r, 尝试Wk
 */
void sumofsub(int t, int k ,int r, int& M, bool& flag, bool* X)
{
    X[k] = true;    // 选第k个数
    if (t + k == M) // 若找到一个和为M,则设置解向量的标志位，输出解
    {
        flag = true;
        for (int i = 1; i <= k; ++i)
        {
            if (X[i] == 1)
            {
                printf("%d ", i);
            }
        }
        printf("/n");
    }
    else
    {   // 若第k+1个数满足条件,则递归左子树
        if (t + k + (k+1) <= M)
        {
            sumofsub(t + k, k + 1, r - k, M, flag, X);
        }
        // 若不选第k个数,选第k+1个数满足条件,则递归右子树
        if ((t + r - k >= M) && (t + (k+1) <= M))
        {
            X[k] = false;
            sumofsub(t, k + 1, r - k, M, flag, X);
        }
    }
}

void search(int& N, int& M)
{
    // 初始化解空间
    bool* X = (bool*) malloc(sizeof(bool) * (N+1));
    memset(X, false, sizeof(bool) * (N+1));
    int sum = (N + 1) * N * 0.5f;
    if (1 > M || sum < M) // 预先排除无解情况
    {
        printf("not found/n");
        return;
    }
    bool f = false;
```

```
    sumofsub(0, 1, sum, M, f, X);
    if (!f)
    {
        printf("not found/n");
    }
    free(X);
}

int main()
{
    int N, M;
    printf("请输入整数N和M/n");
    scanf("%d%d", &N, &M);
    search(N, M);
    return 0;
}
```

问题扩展

1、有N个数，组成的字符串，如012345，求出字串和取MOD3==0的字串，如012 12 123 45。

2、从一列数中筛除尽可能少的数使得从左往右看，这些数是从小到大再从大到小的（网易）。

3、有两个序列a,b，大小都为n,序列元素的值是任意整数，无序；

要求：通过交换a,b中的元素，使[序列a元素的和]与[序列b元素的和]之间的差最小。

例如:

var a=[100,99,98,1,2, 3];

var b=[1, 2, 3, 4,5,40];（微软100题第32题）。

@well：[fairywell]:

给出扩展问题 2 的一个解法：

1、从一列数中筛除尽可能少的数使得从左往右看，这些数是从小到大再从大到小的（网易）。

双端 LIS 问题，用 DP 的思想可解，目标规划函数 max{ b[i] + c[i] - 1 }，其中 b[i] 为从左到右， 0 ~ i 个数之间满足递增的数字个数；c[i] 为从右到左， n-1 ~ i 个数之间满足递增的数字个数。最后结果为 n - max + 1。其中 DP 的时候，可以维护一个 inc[] 数组表示递增数字序列，inc[i] 为从小到大第 i 大的数字，然后在计算 b[i] c[i] 的时候使用二分查找在 inc[] 中找出区间 inc[0] ~ inc[i-1] 中小于 a[i] 的元素个数（low）。源代码如下：

```
/**
* The problem:
* 从一列数中筛除尽可能少的数使得从左往右看，这些数是从小到大再从大到小的（网易）。
* use binary search, perhaps you should compile it with -std=c99
* fairywell 2011
*/
#include <stdio.h>

#define MAX_NUM     (1U<<31)

int
main()
{
    int i, n, low, high, mid, max;

    printf("Input how many numbers there are: ");
    scanf("%d/n", &n);
    /* a[] holds the numbers, b[i] holds the number of increasing numbers
    * from a[0] to a[i], c[i] holds the number of increasing numbers
    * from a[n-1] to a[i]
    * inc[] holds the increasing numbers
    * VLA needs c99 features, compile with -stc=c99
    */
    double a[n], b[n], c[n], inc[n];

    printf("Please input the numbers:/n");
    for (i = 0; i < n; ++i) scanf("%lf", &a[i]);

    // update array b from left to right
    for (i = 0; i < n; ++i) inc[i] = (unsigned) MAX_NUM;
    //b[0] = 0;
    for (i = 0; i < n; ++i) {
        low = 0; high = i;
```

```
        while (low < high) {
            mid = low + (high-low)*0.5;
            if (inc[mid] < a[i]) low = mid + 1;
            else high = mid;
        }
        b[i] = low + 1;
        inc[low] = a[i];
    }

    // update array c from right to left
    for (i = 0; i < n; ++i) inc[i] = (unsigned) MAX_NUM;
    //c[0] = 0;
    for (i = n-1; i >= 0; --i) {
        low = 0; high = i;
        while (low < high) {
            mid = low + (high-low)*0.5;
            if (inc[mid] < a[i]) low = mid + 1;
            else high = mid;
        }
        c[i] = low + 1;
        inc[low] = a[i];
    }

    max = 0;
    for (i = 0; i < n; ++i )
        if (b[i]+c[i] > max) max = b[i] + c[i];
        printf("%d number(s) should be erased at least./n", n+1-max);
        return 0;
}
```

@yansha：fairywell的程序很赞，时间复杂度O(N log N)，这也是我能想到的时间复杂度最优值了。不知能不能达到O(N)。

扩展题第3题

当前数组a和数组b的和之差为

```
A = sum(a) - sum(b)
```

a的第i个元素和b的第j个元素交换后，a和b的和之差为

```
A' = sum(a) - a[i] + b[j] - (sum(b) - b[j] + a[i])

       = sum(a) - sum(b) - 2 (a[i] - b[j])
```

```
= A - 2 (a[i] - b[j])
```

设x = a[i] - b[j]，得

```
|A| - |A'| = |A| - |A-2x|
```

假设A > 0,

当x 在 (0,A)之间时，做这样的交换才能使得交换后的a和b的和之差变小，x越接近A/2效果越好，

如果找不到在(0,A)之间的x，则当前的a和b就是答案。

所以算法大概如下：

在a和b中寻找使得x在(0,A)之间并且最接近A/2的i和j，交换相应的i和j元素，重新计算A后，重复前面的步骤直至找不到(0,A)之间的x为止。

接上，@yuan：

a[i]-b[j]要接近A/2，则可以这样想，

我们可以对于a数组的任意一个a[k],在数组b中找出与a[k]-C最接近的数（C就是常数，也就是0.5*A）

这个数要么就是a[k]-C，要么就是比他稍大，要么比他稍小，所以可以要二分查找。

查找最后一个小于等于a[k]-C的数和第一个大于等于a[k]-C的数，

然后看哪一个与a[k]-C更加接近，所以T(N) = N log N。

除此之外，受本文读者xiafei1987128启示，有朋友在stacoverflow上也问过一个类似的题，:-)，见此：http://stackoverflow.com/questions/9047908/swap-the-elements-of-two-sequences-such-that-the-difference-of-the-element-sums。感兴趣的可以看看。

举一反三

1、《挑战程序设计竞赛》的开票有个类似的抽签问题，挺有意思，题目如下：

将写有数字的n个纸片放入一个纸箱子中，然后你和你的朋友从纸箱子中抽取4张纸片，每次记下纸片上的数字后放回子箱子中，如果这4个数字的和是m，代表你赢，否则就是你的朋友赢。

请编写一个程序，当纸片上所写的数字是k1，k2，k3，k4，..，kn时，是否存在抽取4次和为m的方案，如果存在，输出YES；否则，输出NO。

限制条件：

- 1 <= n <= 50
- 1 <= m <= 10^8
- 1 <= ki <= 10^8

分析：最容易想到的方案是用4个for循环直接穷举所有方案，时间复杂度为O(N^4),主体代码如下：

```
//通过4重for循环枚举所有方案
for (int a = 0; a < n, a++)
{
    for (int b = 0; b < n; b++)
    {
        for (int c = 0; c < n; c++)
        {
            for (int d = 0; d < n; d++)
            {
                if (k[a] + k[b] + k[c] + k[d] == m)
                {
                    f = true;
                }
            }
        }
    }
}
```

但如果当n远大于50时，则程序会显得非常吃力，如此，我们需要找到更好的办法。

提供两个思路：

①最内侧关于d的循环所做的事情：检查是否有d满足ka+ kb +kc + kd = m，移动下式子，等价于：检查是否有d使得kd = m - ka - kb - kc，也就是说，只要检查k中所有元素，判断是否有等于m-ka-kb-ka 的元素即可。设m-ka-kb-ka = x，接下来，就是要看x是否存在于数组k中，此时，可以先把数组k排序，然后利用二分查找在数组k中找x；

②进一步，内侧的两个循环所做的事情：检查是否有c和d满足kc + kd = m - ka -kb。同样，可以预先枚举出kc+kd所得的n^2数字并排好序，便可以利用二分搜索继续求解。

2、给定整数a1，a2，a3，…，an，判断是否可以从中选出若干个数，使得它们的和等于k（k任意给定，且满足-10^8 <= k <= 10^8）。

分析：此题相对于本节“寻找满足条件的多个数”如出一辙，不同的是此题只要求判断，不要求把所有可能的组合给输出来。因为此题需要考虑到加上a[i]和不加上a[i]的情况，故可以采用深度优先搜索的办法，递归解决。

求连续子数组的最大和

题目描述

输入一个整形数组，数组里有正数也有负数。数组中连续的一个或多个整数组成一个子数组，每个子数组都有一个和。 求所有子数组的和的最大值，要求时间复杂度为O(n)。

例如输入的数组为`1, -2, 3, 10, -4, 7, 2, -5`，和最大的子数组为`3, 10, -4, 7, 2`，因此输出为该子数组的和18。

分析与解法

解法一：暴力循环

求一个数组的最大子数组和，如此序列`1, -2, 3, 10, -4, 7, 2, -5`，我想最最直观也是最野蛮的办法便是，三个for循环三层遍历，求出数组中每一个子数组的和，最终求出这些子数组的最大的一个值。currSum[i, …, j]为数组A中第i个元素到第j个元素的和（其中0 <= i <= j < n），currSum[i, …, j]，那么时间复杂度为O(N^3)：

```
int maxSubArray(int* A, int n)
{
    int maxSum = -INF;
    int currSum = 0;
    for (int i = 0; i < n; i++)
    {
        for (int j = i; j < n; j++)
        {
            for (int k = i; k <= j; k++)
            {
                currSum += A[k];
            }
            if (currSum > maxSum)
                maxSum = currSum;
```

```
            currSum = 0; //这里要记得清零，否则的话sum最终存放的是所有子数组的和。
        }
    }
    return maxSum;
}
```

解法二:O(N)遍历

当全是负数的情况时,我们可以让程序返回0,也可以让程序返回最大的那个负数,这里,我们让程序返回最大的那个负数,参考代码如下:

```
int maxSubArray(int* a, int n)
{
    int maxSum = a[0];        //全负情况,返回最大数
    int currSum = 0;
    for (int j = 0; j < n; j++)
    {
        if (currSum >= 0)     //如果加上某个元素,sum>=0的话,就加
            currSum += a[j];
        else
            currSum = a[j];   //如果加上某个元素,sum<0了,就不加
        if (currSum > maxSum)
            maxSum = currSum;
    }
    return maxSum;
}
```

解释下：例如输入的数组为1, -2, 3, 10, -4, 7, 2, -5，那么最大的子数组为3, 10, -4, 7, 2，因此输出为该子数组的和18。

所有的东西都在以下俩行，即：- currSum：0 1 - 1 3 13 9 16 18 13 - maxSum：0 1 1 3 13 13 16 18 18

其实算法很简单,当前面的几个数,加起来后,currSum<0后，把currSum重新赋值,置为下一个元素,currSum = a[i]。- 当currSum > maxSum,则更新maxSum = currSum; - 若currSum < maxSum,则maxSum保持原值,不更新。

解法三:动态规划

currSum[i] 为前i个元素中，包含第i个元素且和最大的连续子数组，result为已找到的子数组中和最大的。 对第i+1个元素有两种选择：- 做为新子数组的第一个元素；- 放入前面找到的子数组。

```
currSum[i+1] = max(a[i+1], currSum[i] + a[i+1])
maxSum = max(maxSum, currSum[i])
```

参考代码如下：

```
int maxSubArray(int* a, int n)
{
    if (a == nullptr || n == 0)
        return 0;

    int currSum = 0;
    int maxSum = a[0];        //全负情况，返回最大数

    for (int j = 0; j < n; j++)
    {
        currSum = (a[j] > currSum + a[j]) ? a[j] : currSum + a[j];
        maxSum = (maxSum > currSum) ? maxSum : currSum;

    }
    return maxSum;
}
```

问题扩展

1. 如果数组是二维数组，同样要你求最大子数组的和列?
2. 如果是要你求子数组的最大乘积列?
3. 如果同时要求输出子段的开始和结束列?

举一反三

1 给定整型数组，其中每个元素表示木板的高度，木板的宽度都相同，求这些木板拼出的最大矩形的面积。并分析时间复杂度。

此题类似leetcode里面关于连通器的题，需要明确的是高度可能为0，长度最长的矩形并不一定是最大矩形，还需要考虑高度很高，但长度较短

的矩形。如[5,4,3,2,4,5,0,7,8,4,6]中最大矩形的高度是[7,8,4,6]组成的矩形，面积为16。

2、环面上的最大子矩形

《算法竞赛入门经典》P89 页。

3、求一个矩阵中各个元素和加起来最的二维矩阵，例如:

1 2 0 3 4

2 3 4 5 1

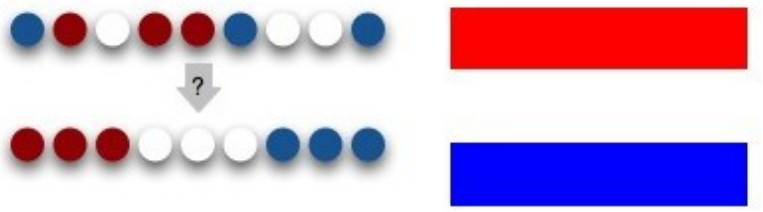
1 1 5 3 0

中最大的是: 4 5 5 3 要求:(1)写出算法;(2)分析时间复杂度;(3)用C写出关键代码。

荷兰国旗问题

题目描述

现有红白蓝三个不同颜色的小球，乱序排列在一起，请重新排列这些小球，使得红白蓝三色的同颜色的球在一起。这个问题之所以叫**荷兰国旗**，是因为我们可以将红白蓝三色小球想象成条状物，有序排列后正好组成荷兰国旗。如下图所示：



img

思路分析

初看此题，我们貌似除了暴力解决并无好的办法，但联想到我们所熟知的快速排序算法呢？我们知道，快速排序时基于分治模式处理的，对一个典型子数组A[p…r]排序的分治过程为三个步骤：

1. 分解：A[p..r]被划分为俩个（可能空）的子数组A[p ..q-1]和A[q+1…r]，使得A[p…q-1] <= A[q] <= A[q+1…r]
2. 解决：通过递归调用快速排序，对子数组A[p…q-1]和A[q+1…r]排序。
3. 合并。

也就是说，快速排序的主要思想便是依托于一个partition分治过程，每一趟排序的过程中，选取的主元都会把整个数组排列成一大一小的序列，继而递归排序完整个数组。

如下伪代码所示：

快速排序算法的关键是PARTITION过程，它对A[p…r]进行就地重排：

```
PARTITION(A, p, r)
1  x ← A[r]
2  i ← p - 1
3  for j ← p to r - 1
4       do if A[j] ≤ x
5             then i ← i + 1
6                  exchange A[i] <-> A[j]
7  exchange A[i + 1] <-> A[r]
8  return i + 1
```

继而递归完成整个排序过程：

```
QUICKSORT(A, p, r)
1 if p < r
2    then q ← PARTITION(A, p, r)    //关键
3         QUICKSORT(A, p, q - 1)
4         QUICKSORT(A, q + 1, r)
```

举个例子如下：$_i$指向数组头部前一个位置，$_j$指向数组头部元素，$_j$在前，$_i$在后，双双从左向右移动。

① j 指向元素2时，i 也指向元素2，2与2互换不变

```
i  p/j

    2   8   7   1   3   5   6   4(主元)
```

② 于是j 继续后移，直到指向了1，1 <= 4，于是i++，i 指向8，故j 所指元素1 与 i 所指元素8 位置互换：

```
    i       j

2   1   7   8   3   5   6   4
```

③ j 继续后移，指到了元素3，3 <= 4，于是同样i++，i 指向7，故j 所指元素3 与 i 所指元素7 位置互换：

```
        i       j

2   1   3   8   7   5   6   4
```

④ j 一路后移，没有再碰到比主元4小的元素：

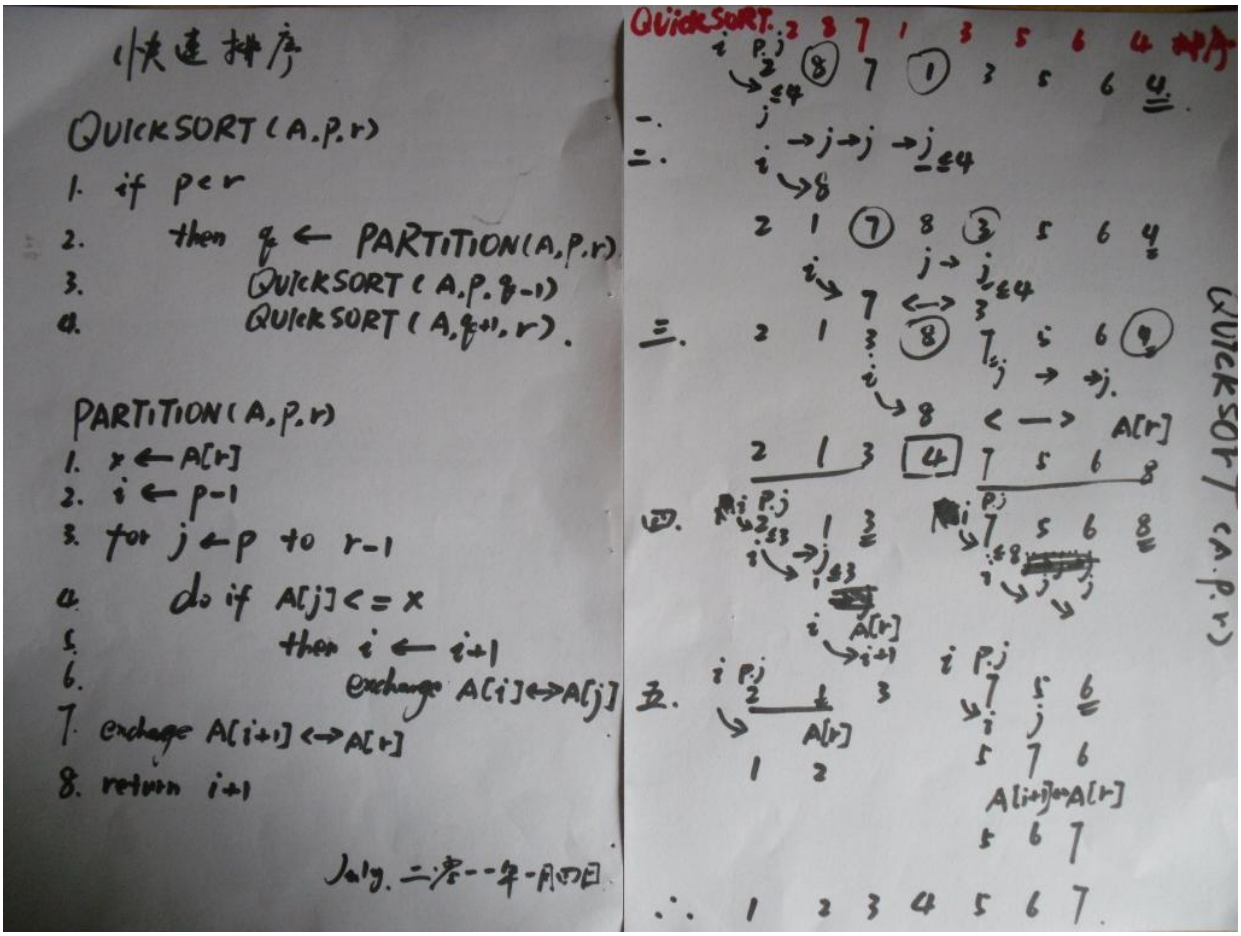
```
        i               j

2   1   3   8   7   5   6   4
```

⑤ 最后，A[i + 1] <-> A[r]，即8与4交换，所以，数组最终变成了如下形式：

```
    2   1   3   4   7   5   6   8
```

ok，至此快速排序第一趟完成。就这样，4把整个数组分成了俩部分，2 1 3,7 5 6 8，再递归对这俩部分分别进行排序。

全部过程可以参看此文：[快速排序算法](#)，或看下我以前在学校里画的图：



img

而我们面对的问题是，重新排列使得所有球排列成三个不同颜色的球，是否可以设定三个指针，借鉴partition过程呢？

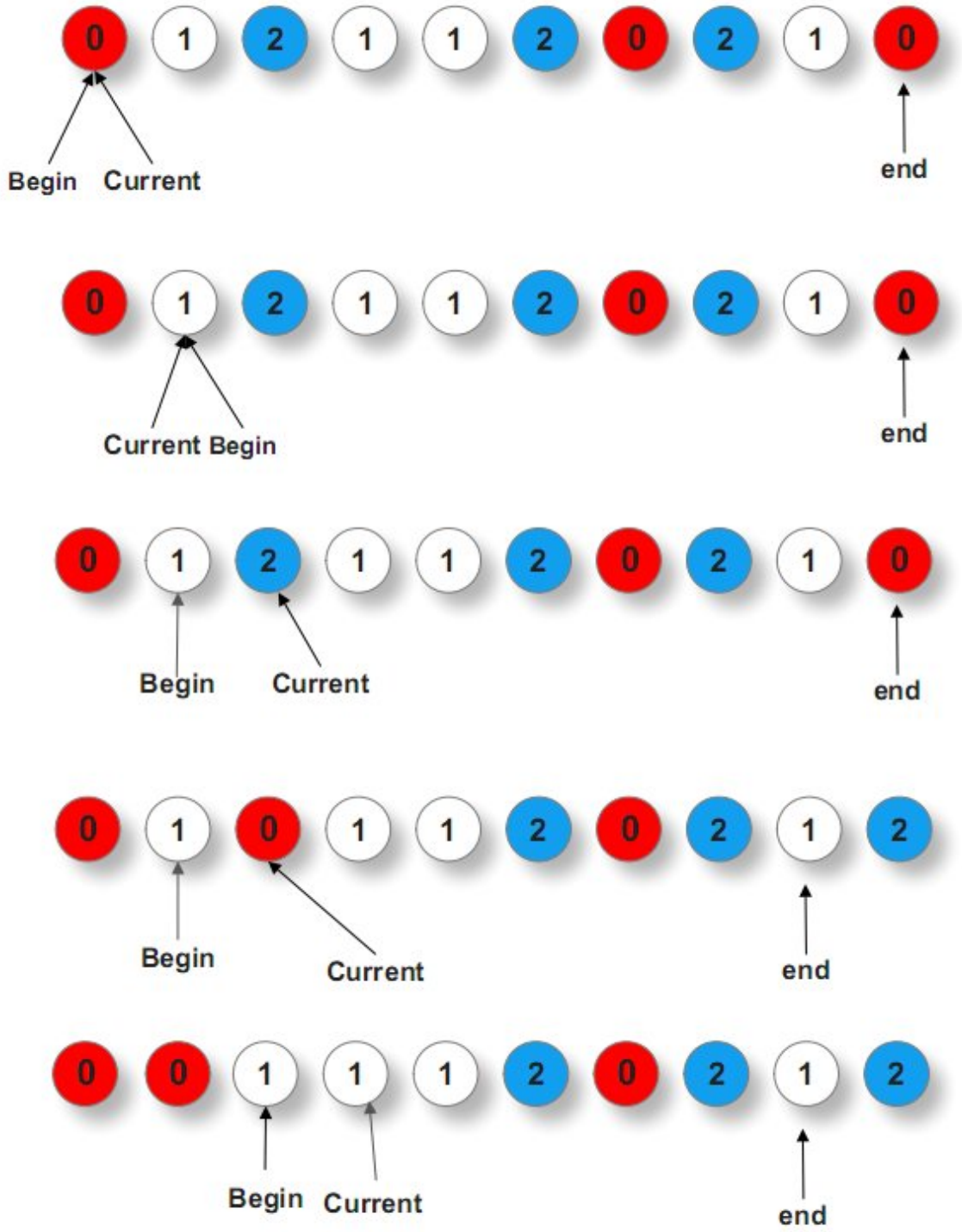
解法一、partition分治

通过前面的分析得知，这个问题类似快排中partition过程，只是需要用到三个指针：一个前指针begin，一个中指针current，一个后指针end，current指针遍历整个数组序列，当

1. current指针所指元素为0时，与begin指针所指的元素交换，而后current++，begin++；
2. current指针所指元素为1时，不做任何交换（即球不动），而后current++；
3. current指针所指元素为2时，与end指针所指的元素交换，而后，current指针不动，end–。

为什么上述第3点中，current指针所指元素为2时，与end指针所指元素交换之后，current指针不能动呢？因为第三步中current指针所指元素与end指针所指元素交换之前，如果end指针之前指的元素是0，那么与current指针所指元素交换之后，current指针此刻所指的元素是0，此时，current指针能动么？不能动，因为如上述第1点所述，如果current指针所指的元素是0，还得与begin指针所指的元素交换。

ok，说这么多，你可能不甚明了，直接引用下gnuhpc的图，就一目了然了：


0 1 2 1 1 2 0 2 1 0
Begin Current
end
0 1 2 1 1 2 0 2 1 0
Current Begin
end
0 1 2 1 1 2 0 2 1 0
Begin Current
end
0 1 0 1 1 2 0 2 1 2
Begin Current
end
0 0 1 1 1 2 0 2 1 2
Begin Current
end

img

参考代码如下：

```
//引用自gnuhpc
while( current<=end )
{
  if( array[current] ==0 )
   {
      swap(array[current],array[begin]);
      current++;
      begin++;
   }
   else if( array[current] == 1 )
   {
      current++;
   }

   else //When array[current] =2
   {
      swap(array[current],array[end]);
      end--;
   }
}
```

本章完。

矩阵相乘之Strassen算法

题目描述

请编程实现矩阵乘法，并考虑当矩阵规模较大时的优化方法。

分析与解法

根据wikipedia上的介绍：两个矩阵的乘法仅当第一个矩阵A的行数和另一个矩阵B的列数相等时才能定义。如A是m×n矩阵，B是n×p矩阵，它们的乘积AB是一个m×p矩阵，它的一个元素其中 $1 \le i \le m, 1 \le j \le p$。

值得一提的是，矩阵乘法满足结合律和分配率，但并不满足交换律，如下图所示的这个例子，两个矩阵交换相乘后，结果变了：

$$\begin{bmatrix}1 & 2\\3 & 4\end{bmatrix}\begin{bmatrix}0 & 1\\0 & 0\end{bmatrix}=\begin{bmatrix}0 & 1\\0 & 3\end{bmatrix},\qquad \begin{bmatrix}0 & 1\\0 & 0\end{bmatrix}\begin{bmatrix}1 & 2\\3 & 4\end{bmatrix}=\begin{bmatrix}3 & 4\\0 & 0\end{bmatrix}.$$

下面咱们来具体解决这个矩阵相乘的问题。

解法一、暴力解法

其实，通过前面的分析，我们已经很明显的看出，两个具有相同维数的矩阵相乘，其复杂度为O(n^3)，参考代码如下：

```
//矩阵乘法，3个for循环搞定
void Mul(int** matrixA, int** matrixB, int** matrixC)
{
    for(int i = 0; i < 2; ++i)
    {
        for(int j = 0; j < 2; ++j)
        {
            matrixC[i][j] = 0;
            for(int k = 0; k < 2; ++k)
            {
                matrixC[i][j] += matrixA[i][k] * matrixB[k][j];
            }
        }
    }
}
```

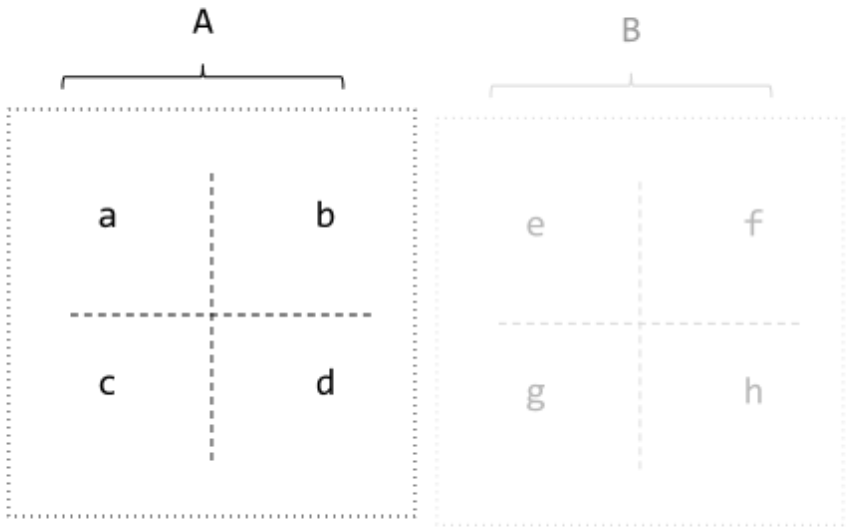
解法二、Strassen算法

在解法一中，我们用了3个for循环搞定矩阵乘法，但当两个矩阵的维度变得很大时，O（n^3）的时间复杂度将会变得很大，于是，我们需要找到一种更优的解法。

一般说来，当数据量一大时，我们往往会把大的数据分割成小的数据，各个分别处理。遵此思路，如果丢给我们一个很大的两个矩阵呢，是否可以考虑分治的方法循序渐进处理各个小矩阵的相乘，因为我们知道一个矩阵是可以分成更多小的矩阵的。

如下图，当给定一个两个二维矩阵A B时：

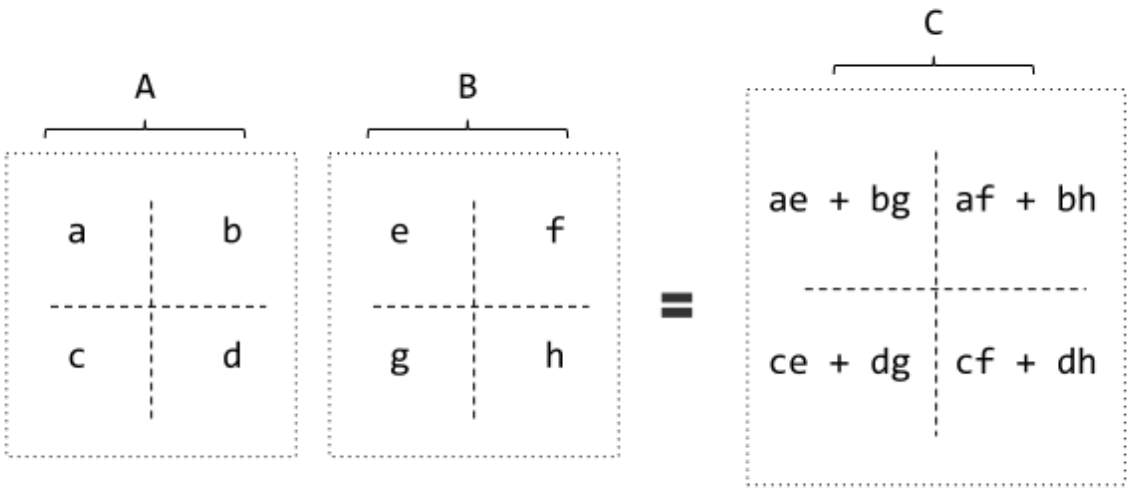
DIVIDE AND CONQUER



We can divide the matrix A in four square parts (in case n is a degree of 2).

这两个矩阵A B相乘时，我们发现在相乘的过程中，有8次乘法运算，4次加法运算：

DIVIDE AND CONQUER



矩阵乘法的复杂度主要就是体现在相乘上，而多一两次的加法并不会让复杂度上升太多。故此，我们思考，是否可以让矩阵乘法的运算过程中乘法的运算次数减少，从而达到降低矩阵乘法的复杂度呢？答案是肯定的。

1969年，德国的一位数学家Strassen证明$O(N^{3})$的解法并不是矩阵乘法的最优算法，他做了一系列工作使得最终的时间复杂度降低到了$O(n^{2.80})$。

他是怎么做到的呢？还是用上文A B两个矩阵相乘的例子，他定义了7个变量：

STRASSEN'S ALGORITHM

```
P1 = A(F - H)
P2 = (A + B)H
P3 = (C + D)E
P4 = D(G - E)
P5 = (A + D)(E + H)
P6 = (B - D)(G + H)
P7 = (A - C)(E + F)
```

$$AB = \begin{bmatrix} P5 + P4 - P2 + P6 & P1 + P2 \\ P3 + P4 & P1 + P5 - P3 - P7 \end{bmatrix}$$

After defining the sum P1 ... P7, the product AB can be done in O(n^lg(7))!

如此，Strassen算法的流程如下：

- 两个矩阵A B相乘时，将A, B, C分成相等大小的方块矩阵：

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_{1,1} & \mathbf{A}_{1,2} \\ \mathbf{A}_{2,1} & \mathbf{A}_{2,2} \end{bmatrix}, \mathbf{B} = \begin{bmatrix} \mathbf{B}_{1,1} & \mathbf{B}_{1,2} \\ \mathbf{B}_{2,1} & \mathbf{B}_{2,2} \end{bmatrix}, \mathbf{C} = \begin{bmatrix} \mathbf{C}_{1,1} & \mathbf{C}_{1,2} \\ \mathbf{C}_{2,1} & \mathbf{C}_{2,2} \end{bmatrix}$$

- 可以看出C是这么得来的：

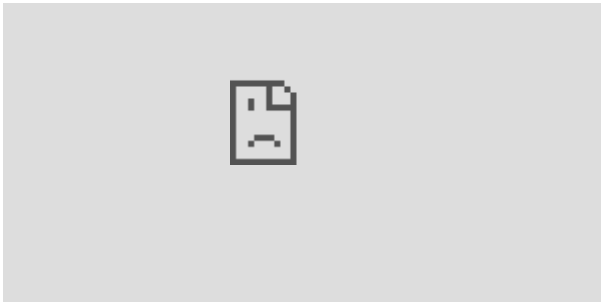
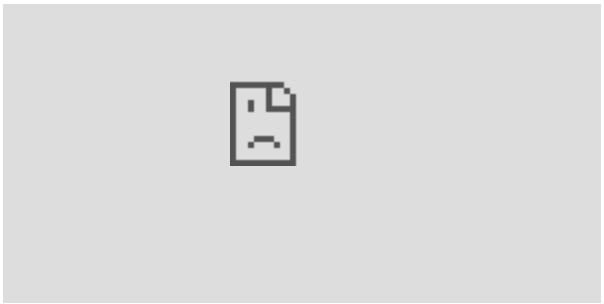
$$\begin{aligned} \mathbf{C}_{1,1} &= \mathbf{A}_{1,1}\mathbf{B}_{1,1} + \mathbf{A}_{1,2}\mathbf{B}_{2,1} \\ \mathbf{C}_{1,2} &= \mathbf{A}_{1,1}\mathbf{B}_{1,2} + \mathbf{A}_{1,2}\mathbf{B}_{2,2} \\ \mathbf{C}_{2,1} &= \mathbf{A}_{2,1}\mathbf{B}_{1,1} + \mathbf{A}_{2,2}\mathbf{B}_{2,1} \\ \mathbf{C}_{2,2} &= \mathbf{A}_{2,1}\mathbf{B}_{1,2} + \mathbf{A}_{2,2}\mathbf{B}_{2,2} \end{aligned}$$

- 现在定义7个新矩阵（*读者可以思考下，这7个新矩阵是如何想到的*）：

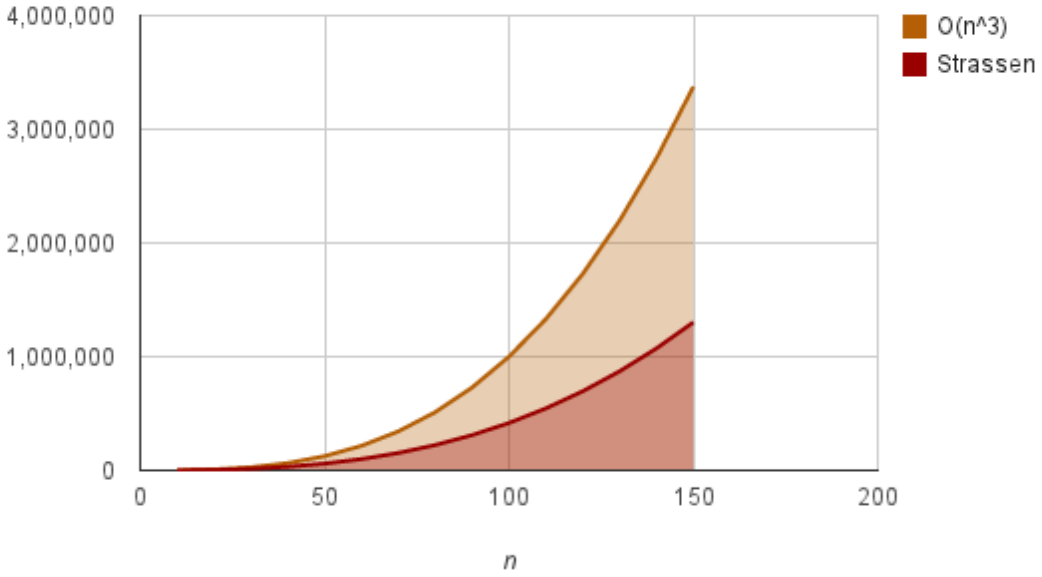
$$
\begin{aligned}
\mathbf{M}_1 &:= (\mathbf{A}_{1,1} + \mathbf{A}_{2,2})(\mathbf{B}_{1,1} + \mathbf{B}_{2,2}) \\
\mathbf{M}_2 &:= (\mathbf{A}_{2,1} + \mathbf{A}_{2,2})\mathbf{B}_{1,1} \\
\mathbf{M}_3 &:= \mathbf{A}_{1,1}(\mathbf{B}_{1,2} - \mathbf{B}_{2,2}) \\
\mathbf{M}_4 &:= \mathbf{A}_{2,2}(\mathbf{B}_{2,1} - \mathbf{B}_{1,1}) \\
\mathbf{M}_5 &:= (\mathbf{A}_{1,1} + \mathbf{A}_{1,2})\mathbf{B}_{2,2} \\
\mathbf{M}_6 &:= (\mathbf{A}_{2,1} - \mathbf{A}_{1,1})(\mathbf{B}_{1,1} + \mathbf{B}_{1,2}) \\
\mathbf{M}_7 &:= (\mathbf{A}_{1,2} - \mathbf{A}_{2,2})(\mathbf{B}_{2,1} + \mathbf{B}_{2,2})
\end{aligned}
$$

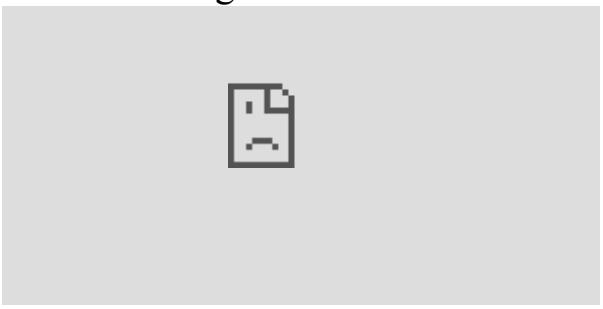
- 而最后的结果矩阵C 可以通过组合上述7个新矩阵得到：

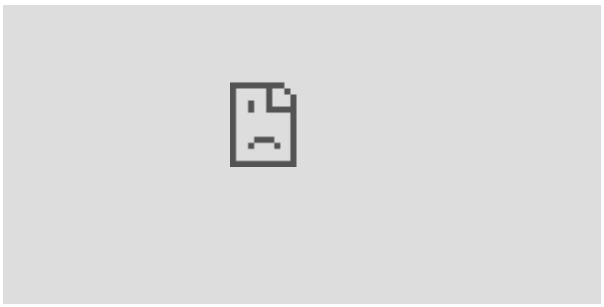
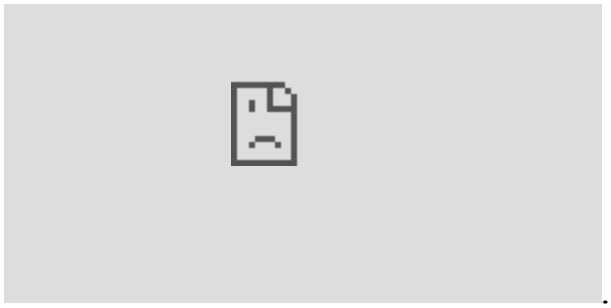
$$
\begin{aligned}
\mathbf{C}_{1,1} &= \mathbf{M}_1 + \mathbf{M}_4 - \mathbf{M}_5 + \mathbf{M}_7 \\
\mathbf{C}_{1,2} &= \mathbf{M}_3 + \mathbf{M}_5 \\
\mathbf{C}_{2,1} &= \mathbf{M}_2 + \mathbf{M}_4 \\
\mathbf{C}_{2,2} &= \mathbf{M}_1 - \mathbf{M}_2 + \mathbf{M}_3 + \mathbf{M}_6
\end{aligned}
$$

表面上看，Strassen算法仅仅比通用矩阵相乘算法好一点，因为通用矩阵相乘算法时间复杂度是  ，而Strassen算法复杂度只是  。但随着n的变大，比如当n >> 100时，Strassen算法是比通用矩阵相乘算法变得更有效率。

如下图所示：



根据wikipedia上的介绍，后来，Coppersmith–Winograd 算法把 N* N大小的矩阵乘法的时间复杂度降低到了：  ，而 2010 年，Andrew Stothers 再度把复杂度降低到了

 ，一年后的2011年，Virginia Williams

把复杂度最终定格为：.

K个最小和 (UVA 11997 K Smallest Sums)

题目大意：

You’re given k arrays, each array has k integers. There are kk ways to pick exactly one element in each array and calculate the sum of the integers. Your task is to find the k smallest sums among them.

Input

There will be several test cases. The first line of each case contains an integer k (2<=k<=750). Each of the following k lines contains k positive integers in each array. Each of these integers does not exceed 1,000,000. The input is terminated by end-of-file (EOF). The size of input file does not exceed 5MB.

Output

For each test case, print the k smallest sums, in ascending order.

Sample Input

3 1 8 5 9 2 5 10 7 6 2 1 1 1 2 #### Output for the Sample Input 9 10 12 2 2

给定一个k*k的一个矩阵，如果让你在每一行取出一个数，再将每一行取出的数相加，那么总共可以得到$k^{k种相加方法，现在让你求出这k}$k个结果中最小的k个结果。

分析与解法

仔细分析这个题目我们会发现其实这个问题是满足最优子结构的，比如：

如果我们已经计算出了前m行，每行取出一个数相加的最小的k个结果，分别是DP[1],DP[2]…DP[k] (注意这里的DP表示的是前m行每行一个相加的最小的前k个值)

假设第m+1行的值是A[1],A[2]…A[k] (注意这里的A[i]表示的是第m+1行的第i个数) 当我们推倒到第m+1行时，由于我们只计算了前m行的前k个最小值，那我们是不是有必要多计算一些来推导出第m+1行的前k个最小值呢？

答案是不必要的，我们可以通过以下数学公式严格证明：设DP[x]是前m行通过计算得出的第x(x>k)小的和，如果上述的假设成立，那么我们可以列出不等式：

```
DP[x] + A[y] < DP[m] + A[n] (1)
(DP[m]+A[n]表示只通过DP[1,2...k]计算出的前m+1行第k小的和)
```

上述不等式的含义是指在第m+1行存在一个数A[y],使得DP[x]+A[y]是前m+1行中前k小的结果。同时，我们注意到:

```
x>k ==> DP[x] > DP[k]  (2)
A[y] >= A[1]           (3)
```

由上面三个不等式(1),(2),(3)我们可以得到：

```
DP[k]+A[1] <= DP[x]+A[y] < DP[m]+A[n]
DP[k]+A[1] < DP[m]+A[n]
```

之前我们说过DP[m] + A[n] 是前m行第k大的和，然而：比DP[k]+A[1]小的数已经有

```
(DP[1]+A[1]),(DP[2]+A[1])...(DP[k-1]+A[1])共计k-1个,
```

所以DP[k]+A[1]是第k个最小的和，与假设的DP[m]+A[n]是第k个最小的和相矛盾，所以假设不成立。得证。

通过以上的证明我们可以得出结论要计算第m+1行的前k个最小和是只需要计算出前m行的前k个最小的和即可。这时，我们的目标就转化为了计算一个2*k的数组，在第一行取一个数，在第二行取一个数，得到k^2个和，求他们当中的最小的k个和。

为了计算它，我们把这n^2个数组织成如下n个有序表：

```
表1: A1+B1 <= A1+B2<=A1+B3<=......
表2: A2+B1 <= A2+B2<=A2+B3<=......
.
表n: An+B1 <= An+B2<=An+B3<=......
```

这时我们用一个二元组（sum，b）来保存以上的每一个元素，其中sum=Aa + Bb.

为什么不保存A的下标a呢？因为我们用不到a的值。如果我们需要在表(sum, b)中赵到下一个元素(sum', b+1)，只要计算sum' = s - Bb + B[b+1]，不需要知道a是多少。

实现代码

```
#include <cstdio>
#include <algorithm>
#include <queue>

using namespace std;

int  a[800][800],k;

struct node
{
    int s,b;
    bool operator<(const node &a) const
    {
        return s>a.s;
    }
};

void merge(int *A,int *B,int *C,int n)
{
    node tmp;
    priority_queue<node> q;
    for(int i=0;i<n;i++)
```

```
    {
        tmp.s=A[i]+B[0],tmp.b=0;
        q.push(tmp);
    }
    for(int i=0;i<n;i++)
    {
        tmp=q.top();
        q.pop();
        C[i]=tmp.s;
        if(tmp.b+1<n)
        {
            tmp.s=tmp.s-B[tmp.b]+B[tmp.b+1];
            tmp.b++;
            q.push(tmp);
        }
    }
}

int main()
{
    while(~scanf("%d",&k))
    {
        for(int i=0;i<k;i++)
        {
            for(int j=0;j<k;j++)
                scanf("%d",&a[i][j]);
            sort(a[i],a[i]+k);
        }
        for(int i=1;i<k;i++)
            merge(a[0],a[i],a[0],k);
        for(int i=0;i<k;i++)
            printf("%d%s",a[0][i],i!=k-1?" ":"\n");
    }
    return 0;
}
```

参考 http://www.cnblogs.com/gj-Acit/p/3583480.html

数组其它问题

1、不用除法运算。

a）一个长度为n的数组a[0],a[1],…,a[n-1]。现在更新数组的名个元素，即a[0]变为a[1]到a[n-1]的积，a[1]变为a[0]和a[2]到a[n-1]的积，…，a[n-1]为a[0]到a[n-2]的积（就是除掉当前元素，其他所有元素的积）。程序要求：具有线性复杂度，且不能使用除法运算符。

分析：left[i]标示着a[i]之前的乘积，right[i]标示着a[i]之后的乘积，但不申请空间，那么a[i]=left[i]*right[i]* 。*不过，left的计算从左往右扫的时候得出，right是从右往左扫得出。因为就是当中某个元素a[i]的左边所有元素与右边所有元素的乘积，就这么简单。所以计算a[i]=left[i]*right[i]时，直接出结果。

b）两个数组a[N]，b[N]，其中A[N]的各个元素值已知，现给b[i]赋值，b[i] = a[0]*a[1]*a[2]…*a[N-1]/a[i]；要求：- 1.不准用除法运算 - 2.除了循环计数值，a[N],b[N]外，不准再用其他任何变量（包括局部变量，全局变量等）- 3.满足时间复杂度O（n），空间复杂度O（1）。

分析：你要我求b=a[0]*a*…a[i-1]*a*a[i+1]..*a[N-1]/a ，就是求：a[0]*a[1]...*a[i-1]*a[i+1]..*a[N-1]。只是我把a[i]左边部分标示为left[i]，b[i]右边部分标示为right[i]，而实际上完全不申请left[i]，与right[i]变量，之所以那样标示，无非就是为了说明：除掉当前元素a[i]，其他所有元素(a[i]左边部分，和a[i]右边部分)的积。

第二十五章:二分查找实现

作者:July

出处:结构之法算法之道

引言

Jon Bentley:90%以上的程序员无法正确无误的写出二分查找代码。也许很多人都早已听说过这句话,但我还是想引用《编程珠玑》上的如下几段文字:

二分查找可以解决(预排序数组的查找)问题:只要数组中包含T(即要查找的值),那么通过不断缩小包含T的范围,最终就可以找到它。一开始,范围覆盖整个数组。将数组的中间项与T进行比较,可以排除一半元素,范围缩小一半。就这样反复比较,反复缩小范围,最终就会在数组中找到T,或者确定原以为T所在的范围实际为空。对于包含N个元素的表,整个查找过程大约要经过log(2)N次比较。

多数程序员都觉得只要理解了上面的描述,写出代码就不难了;但事实并非如此。如果你不认同这一点,最好的办法就是放下书本,自己动手写一写。试试吧。

我在贝尔实验室和IBM的时候都出过这道考题。那些专业的程序员有几个小时的时间,可以用他们选择的语言把上面的描述写出来;写出高级伪代码也可以。考试结束后,差不多所有程序员都认为自己写出了正确的程序。于是,我们花了半个钟头来看他们编写的代码经过测试用例验证的结果。几次课,一百多人的结果相差无几:90%的程序员写的程序中有bug(我并不认为没有bug的代码就正确)。

我很惊讶:在足够的时间内,只有大约10%的专业程序员可以把这个小程序写对。但写不对这个小程序的还不止这些人:高德纳在《计算机程序设计的艺术 第3卷 排序和查找》第6.2.1节的“历史与参考文献”部分指出,虽然早在1946年就有人将二分查找的方法公诸于世,但直到1962年才有

人写出没有bug的二分查找程序。”——乔恩·本特利，《编程珠玑（第1版）》第35-36页。

你能正确无误的写出二分查找代码么？不妨一试。

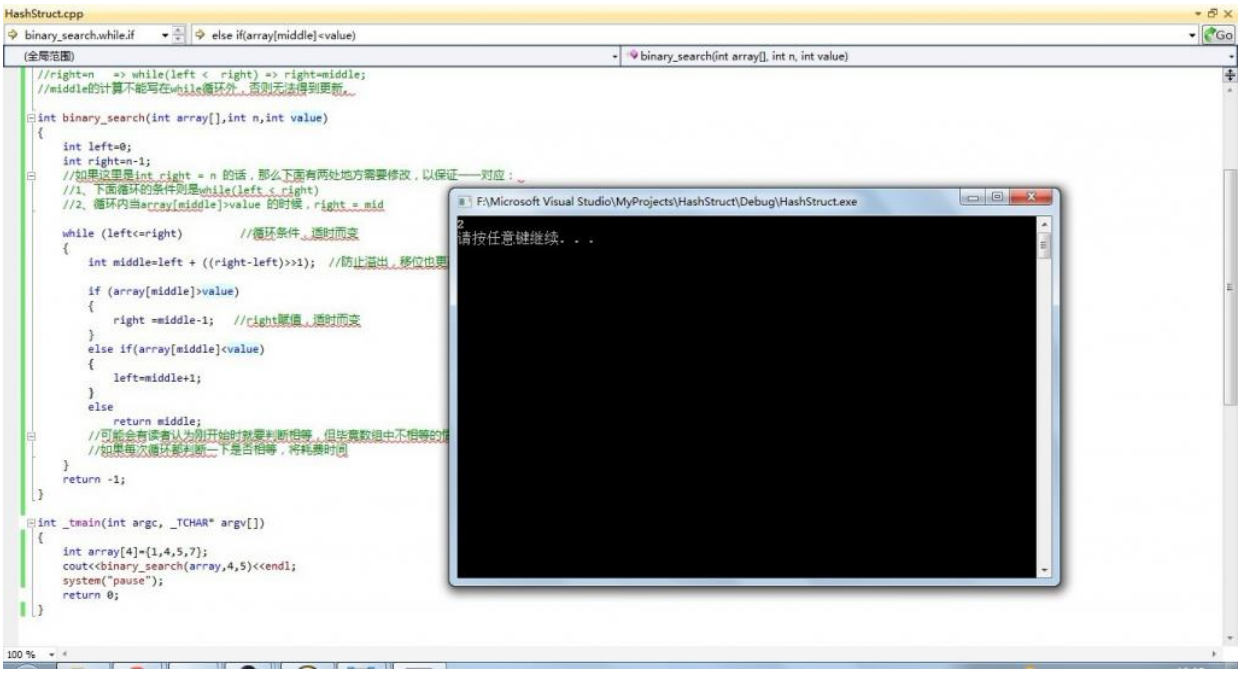
二分查找代码

二分查找的原理想必不用多解释了，不过有一点必须提醒读者的是，二分查找是针对的排好序的数组。OK，纸上谈来终觉浅，觉知此事要躬行。我先来写一份，下面是我写的一份二分查找的实现（之前去某一家公司面试也曾被叫当场实现二分查找，不过结果可能跟你一样，当时就未能完整无误写出），有任何问题或错误，恳请不吝指正：

```
//二分查找V0.1实现版
//copyright@2011 July

//首先要把握下面几个要点：
//right = n-1 => while(left <= right) => right = middle-1;
//right = n   => while(left <  right) => right = middle;
//middle的计算不能写在while循环外，否则无法得到更新。

int binary_search(int array[], int n, int value)
{
    int left = 0;
    int right = n - 1;
    //如果这里是int right = n 的话，那么下面有两处地方需要修改，以保证一一对应：
    //1、下面循环的条件则是while(left < right)
    //2、循环内当 array[middle] > value 的时候，right = mid

    while (left <= right)  //循环条件，适时而变
    {
        int middle = left + ((right - left) >> 1);  //防止溢出，移位也更高效。同时，每次循环都需要更新。

        if (array[middle] > value)
        {
            right = middle - 1;  //right赋值，适时而变
        }
        else if(array[middle] < value)
        {
            left = middle + 1;
        }
        else
            return middle;
```

```
            //可能会有读者认为刚开始时就要判断相等,但毕竟数组中不相等的情况更多
            //如果每次循环都判断一下是否相等,将耗费时间
        }
        return -1;
}
```

简单测试下，运行结果如下所示（当然，一次测试正确不代表程序便0 bug了，且测试深度远远不够）：



img

测试

也许你之前已经把二分查找实现过很多次了，但现在不妨再次测试一下。关闭所有网页，窗口，打开记事本，或者编辑器，或者直接在本文评论下，不参考上面我写的或其他任何人的程序，给自己十分钟到N个小时不等的时间，立即编写一个二分查找程序。

当然，能正确写出来不代表任何什么，不能正确写出来亦不代表什么，仅仅针对Jon Bentley的言论做一个简单的测试而已。

总结

本文发表后，马上就有很多朋友自己尝试了。根据从朋友们在本文评论下留下的代码，发现出错率最高的在以下这么几个地方：

注释里已经说得很明白了，可还是会有不少朋友犯此类的错误：

首先要把握下面几个要点：
`right = n-1 => while(left <= right) => right = middle-1;`
`middle`的计算不能写在`while`循环外，否则无法得到更新。

还有一个最最常犯的错误，具体可以参考Extra, Extra - Read All About It: Nearly All Binary Searches and Mergesorts are Broken，其中提到了JDK5中该BUG：

`middle = (left+right)>>1;` 这样的话`left`与`right`的值比较大的时候，其和可能溢出。

这个也是错误的:

```
right = n    => while(left <  right) => right = middle;
```

如果搜索数组中不存在的数字的时候,可能会导致死循环. 因为最后left = middle, right = middle + 1,这样每次更新left和middle总是不变的.

第二十三章:杨氏矩阵查找

题目描述

在一个m行n列二维数组中,每一行都按照从左到右递增的顺序排序,每一列都按照从上到下递增的顺序排序。请完成一个函数,输入这样的一个二维数组和一个整数,判断数组中是否含有该整数。

例如下面的二维数组就是每行、每列都递增排序。如果在这个数组中查找数字6,则返回true;如果查找数字5,由于数组不含有该数字,则返回false。

1	2	8	9
2	4	9	12
4	7	10	13
6	8	11	15

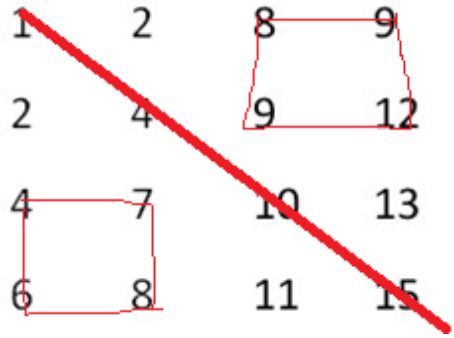
img

分析与解法

解法一、分治法

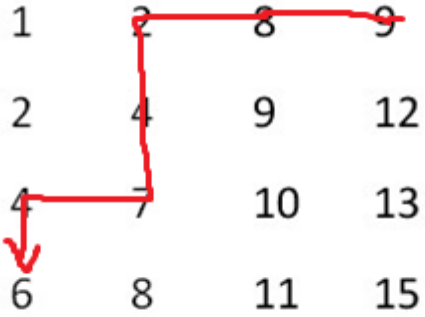
以查找数字6为例,把整个矩阵,分为四个小矩阵,配以二分查找,如果要找的数是6介于对角线上相邻的两个数4、10,可以排除掉左上和右下的

两个矩形，而递归在左下和右上的两个矩形继续找，如下图所示：

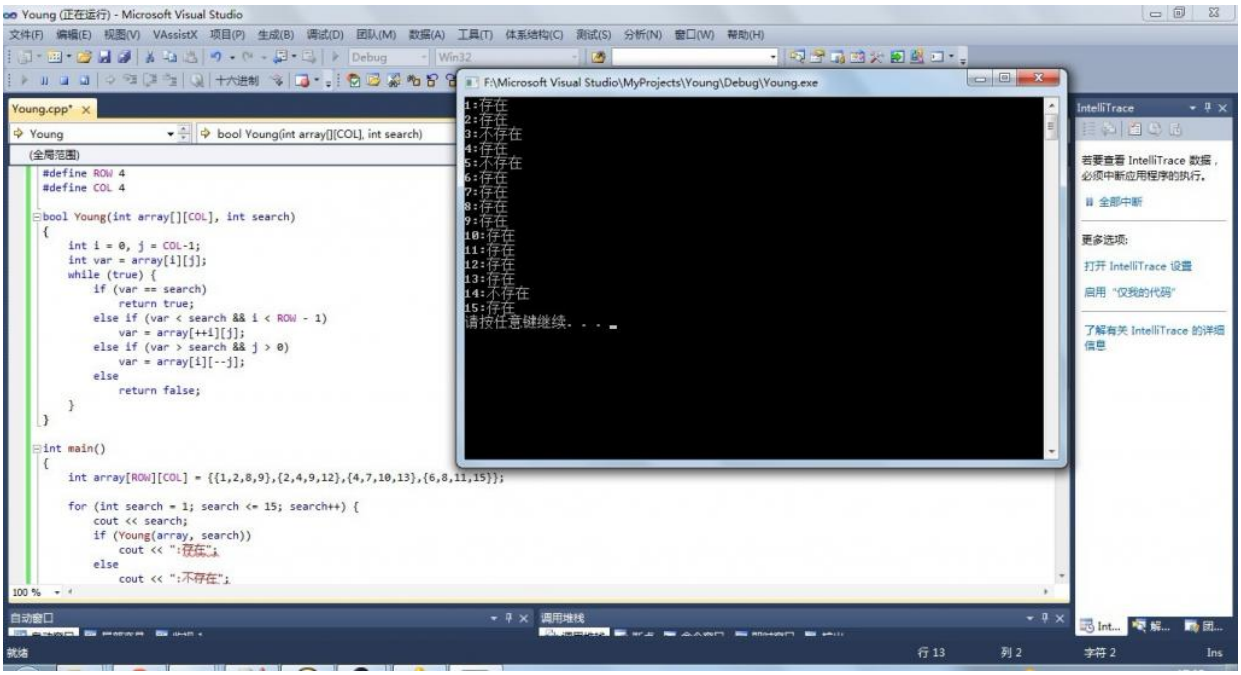


解法二、定位法

首先直接定位到最右上角的元素，再配以二分查找，比要找的数（6）大就往左走，比要找数（6）的小就往下走，直到找到要找的数字（6）为止，这个方法的时间复杂度O（m+n）。如下图所示：



上述方法二的关键代码+程序运行如下图所示：



试问，上述算法复杂么？不复杂，只要稍微动点脑筋便能想到，还可以参看友人老梦的文章，Young氏矩阵：http://blog.csdn.net/zhanglei8893/article/details/6234564，除此之外，何海涛先生一书剑指offer中也收集了此题，感兴趣的朋友也可以去看看。

类似问题

行列都递增的矩阵，求中位数。

出现次数超过一半的数字

题目描述

题目：数组中有一个数字出现的次数超过了数组长度的一半，找出这个数字。

分析:编程之美上也有这道题,不过它变换了题目的表述形式,如下:

Tango是微软亚洲研究院的一个试验项目,如图1所示。研究院的员工和实习生们都很喜欢在Tango上面交流灌水。传说,Tango有一大“水王”,他不但喜欢发帖,还会回复其他ID发的每个帖子。坊间风闻该“水王”发帖数目超过了帖子总数的一半。如果你有一个当前论坛上所有帖子（包括回帖）的列表，其中帖子作者的ID也在表中,你能快速找出这个传说中的Tango水王吗?

图1 Tango

分析与解法

一个数组中有很多数，现在我们要找出其中那个出现次数超过总数一半的数字，怎么找呢?大凡当我们碰到某一个杂乱无序的东西时，我们人的内心本质期望是希望把它梳理成有序的。所以，我们得分两种情况来讨论，无序和有序。

解法一

如果**无序**，那么我们是不是可以先把数组中所有这些数字**先进行排序**，至于选取什么排序方法则不在话下，最常用的快速排序O（N*logN*）即可。*排完序呢，直接遍历。在遍历整个数组的同时统计每个数字的出现次数，然后把那个出现次数超过一半的数字直接输出，题目便解答完成了。总的时间复杂度为O（N*logN+N）。

解法二

但各位再想想，**如果是有序的数组呢**或者经过上述由无序的数组变成有序后的数组呢？是否在排完序O（N*logN*）*后，真的还需要再遍历一次整个数组么?我们知道，既然是数组的话，那么我们可以根据数组索引支持直接定向到某一个数。我们发现，一个数字在数组中的出现次数超过了一半，那么在已排好序的数组索引的N/2处（从零开始编号），就一定是这个数字。自此，我们只需要对整个数组排完序之后，然后直接输出数组中的第N/2处的数字即可，这个数字即是整个数组中出现次数超过一半的数字，总的时间复杂度由于少了最后一次整个数组的遍历，缩小到O（N*logN）。

解法三

然不论是上述思路一的O（N*logN+N*），*还是思路二的O（N*logN），时间复杂度并无本质性的改变。我们需要找到一种更为有效的思路或方法。既要缩小总的时间复杂度，那么可以用查找时间复杂度为O（1）的**hash表**，即以空间换时间。哈希表的键值（Key）为数组中的数字，值（Value）为该数字对应的次数。然后直接遍历整个**hash表**，找出每一个数字在对应的位置处出现的次数，输出那个出现次数超过一半的数字即可。

解法四

Hash表需要O(N)的空间开销,且要设计hash函数,还有没有更好的办法呢?我们可以试着这么考虑,如果**每次删除两个不同的数**(不管是不是我们要查找的那个出现次数超过一半的数字),那么,在剩下的数中,我们要查找的数(出现次数超过一半)出现的次数仍然超过总数的一半。通过不断重复这个过程,不断排除掉其它的数,最终找到那个出现次数超过一半的数字。这个方法,免去了上述思路一、二的排序,也避免了思路三空间O(N)的开销,总得说来,时间复杂度只有O(N),空间复杂度为O(1),不失为最佳方法。

或许,你还没有明白上述思路4的意思,举个简单的例子吧,如数组a[5]={0,1,2,1,1};

很显然,若我们要找出数组a中出现次数超过一半的数字,这个数字便是1,若根据上述思路4所述的方法来查找,我们应该怎么做呢?通过一次性遍历整个数组,然后每次删除不相同的两个数字,过程如下简单表示:

```
0 1 2 1 1 =>2 1 1=>1,最终,1即为所找。
```

但是如果是5,5,5,5,1,还能运用上述思路么?额,别急,请看下文思路5。

解法五

咱们根据数组的特性进一步考虑@zhedahht:数组中有个数字出现的次数超过了数组长度的一半。也就是说,有个数字出现的次数比其他所有数字出现次数的和还要多。因此我们可以考虑在遍历数组的时候保存两个值:一个是数组中的一个数字,一个是次数。当我们遍历到下一个数字的时候,如果下一个数字和我们之前保存的数字相同,则次数加1。如果下一个数字和我们之前保存的数字不同,则次数减1。如果次数为零,我们需要保存下一个数字,并把次数重新设为1。

下面,举二个例子:

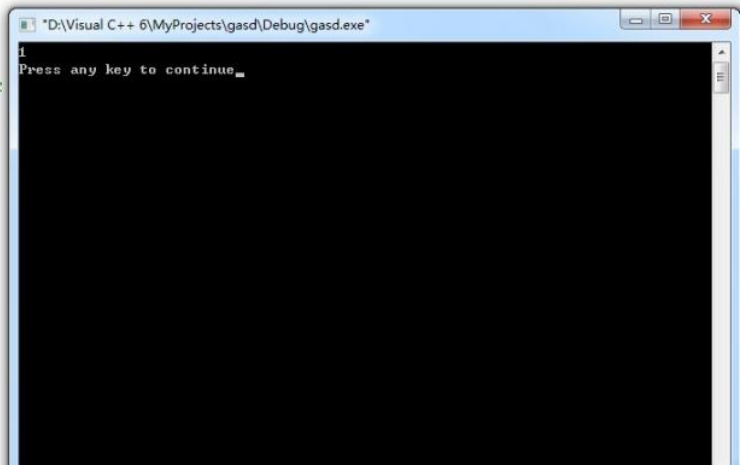
- 第一个例子,5,5,5,5,1:

  不同的相消,相同的累积。遍历到第四个数字时,candidate 是5, nTimes 是4; 遍历到第五个数字时,candidate 是5, nTimes 是3; nTimes不为0,那么candidate就是超过半数的。

- 第二个例子，0，1，2，1，1：

  开始时，保存candidate是数字0，nTimes为1；遍历到数字1后，与数字0不同，则nTimes减1变为零；接下来，遍历到数字2，2与1不同，candidate保存数字2，且nTimes重新设为1；继续遍历到第4个数字1时，与2不同，nTimes减1为零，同时candidate保存为1；最终遍历到最后一个数字还是1，与我们之前candidate保存的数字1相同，nTimes加1为1。最后返回的是之前保存的candidate为1。

```
//copyright@zhedahht
//July,updated,
//2011.04.16。
#include <iostream>
using namespace std;

//改自编程之美 2010
int Find(int* a, int N)  //a代表数组，N代表数组长度
{
    int candidate;
    int nTimes, i;
    for(i = nTimes = 0; i < N; i++)
    {
        if(nTimes == 0)
        {
            candidate = a[i], nTimes = 1;
        }
        else
        {
            if(candidate == a[i])
                nTimes++;
            else
                nTimes--;
        }
    }
    return candidate;  //返回出现次数超过一半的那个数字
}

int main()
{
    int a[5]={0,1,2,1,1};
    int* n=a;
    cout<<Find(a,5)<<endl;
    return 0;
}
```



针对上述程序，我再说详细点，0，1，2，1，1：

```
i=0,candidate=0,nTimes=1;
i=1,a[1] != candidate,nTimes--,=0;
i=2,candidate=2,nTimes=1;
i=3,a[3] != candidate,nTimes--,=0;
i=4,candidate=1,nTimes=1;
如果是0,1,2,1,1,1的话,那么i=5,a[5] == candidate,nTimes++,=2;.....
```

ok，思路清楚了，完整的代码如下：

```
//改自编程之美 2010
Type Find(Type* a, int N)  //a代表数组,N代表数组长度
{
    Type candidate;
    int nTimes, i;
    for(i = nTimes = 0; i < N; i++)
    {
        if(nTimes == 0)
        {
            candidate = a[i], nTimes = 1;
        }
        else
        {
            if(candidate == a[i])
                nTimes++;
            else
                nTimes--;
        }
    }
    return candidate;
}
```

或者：

```
//copyright@zhedahht
//updated@July
#include <iostream>
using namespace std;

bool g_Input = false;

int Num(int* numbers, unsigned int length)
{
    if(numbers == NULL && length == 0)
    {
        g_Input = true;
        return 0;
    }
    g_Input = false;

    int result = numbers[0];
    int times = 1;
    for(int i = 1; i < length; ++i)
    {
        if(numbers[i] == result)
            times++;
        else
            times--;
```

```
        if(times == 0)
        {
            result = numbers[i];
            times = 1;
        }
    }

    //检测输入是否有效。
    times = 0;
    for(int i = 0; i < length; ++i)
    {
        if(numbers[i] == result)
            times++;
    }
    if(times * 2 <= length)
        //检测的标准是：如果数组中并不包含这么一个数字，那么输入将是无效的。
    {
        g_Input = true;
        result = 0;
    }
    return result;
}

int main()
{
    int a[10]={1,2,3,4,6,6,6,6,6};
    int* n=a;
    cout<<Num(a,9)<<endl;
    return 0;
}
```

这段代码与上段代码本质上并无二致，不过有几个问题，还是需要我们注意：

1.当输入无效性时，要处理。比如数组长度为0。

2.最后，上述代码加了一个判断，如果数组中并不包含这么一个数字，那么输入也是无效的。因此在函数结束前还加了一段代码来验证输入是不是有效的。

举一反三

加强版水王：找出出现次数刚好是一半的数字

分析:我们知道,水王问题:有N个数,其中有一个数出现超过一半,要求在线性时间求出这个数。那么,我的问题是,加强版水王:有N个数,其中有一个数刚好出现一半次数,要求在线性时间内求出这个数。

因为,很明显,如果是刚好出现一半的话,如此例:0,1,2,1:

遍历到0时,candidate为0, times为1
遍历到1时,与candidate不同,times减为0
遍历到2时,times为0,则candidate更新为2,times加1
遍历到1时,与candidate不同,则times减为0;我们需要返回所保存candidate(数字2)的下一个数字,即数字1。

问题扩展

谁是选举中得票超半数的人?

美国选举一般要求找到票数超过一半的人。于是有人就开始研究如何用线性时间找到这个人或者证明这个人不存在。Robert S. Boyer 和 J Strother Moore 发表了一片关于这个算法的论文。内容和上文提到的算法是一致的,只是多了理论分析和FORTRAN实现部分。

MJRTY - A Fast Majority Vote Algorithm, with R.S. Boyer. In R.S. Boyer (ed.), Automated Reasoning: Essays in Honor of Woody Bledsoe, Automated Reasoning Series, Kluwer Academic Publishers, Dordrecht, The Netherlands, 1981, pp. 105-117.

论文链接: http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.56.3439&rep=rep1&type=pdf

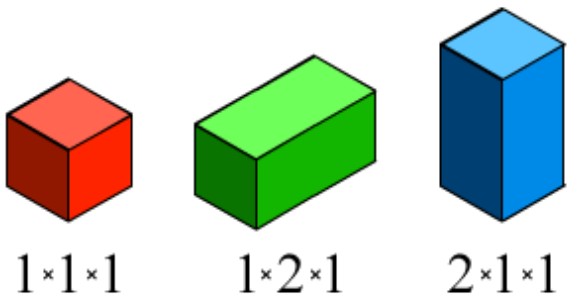
作者 J Strother Moore 关于这个算法的网址(英文): http://www.cs.utexas.edu/~moore/best-ideas/mjrty/index.html

第三十三章:木块砌墙

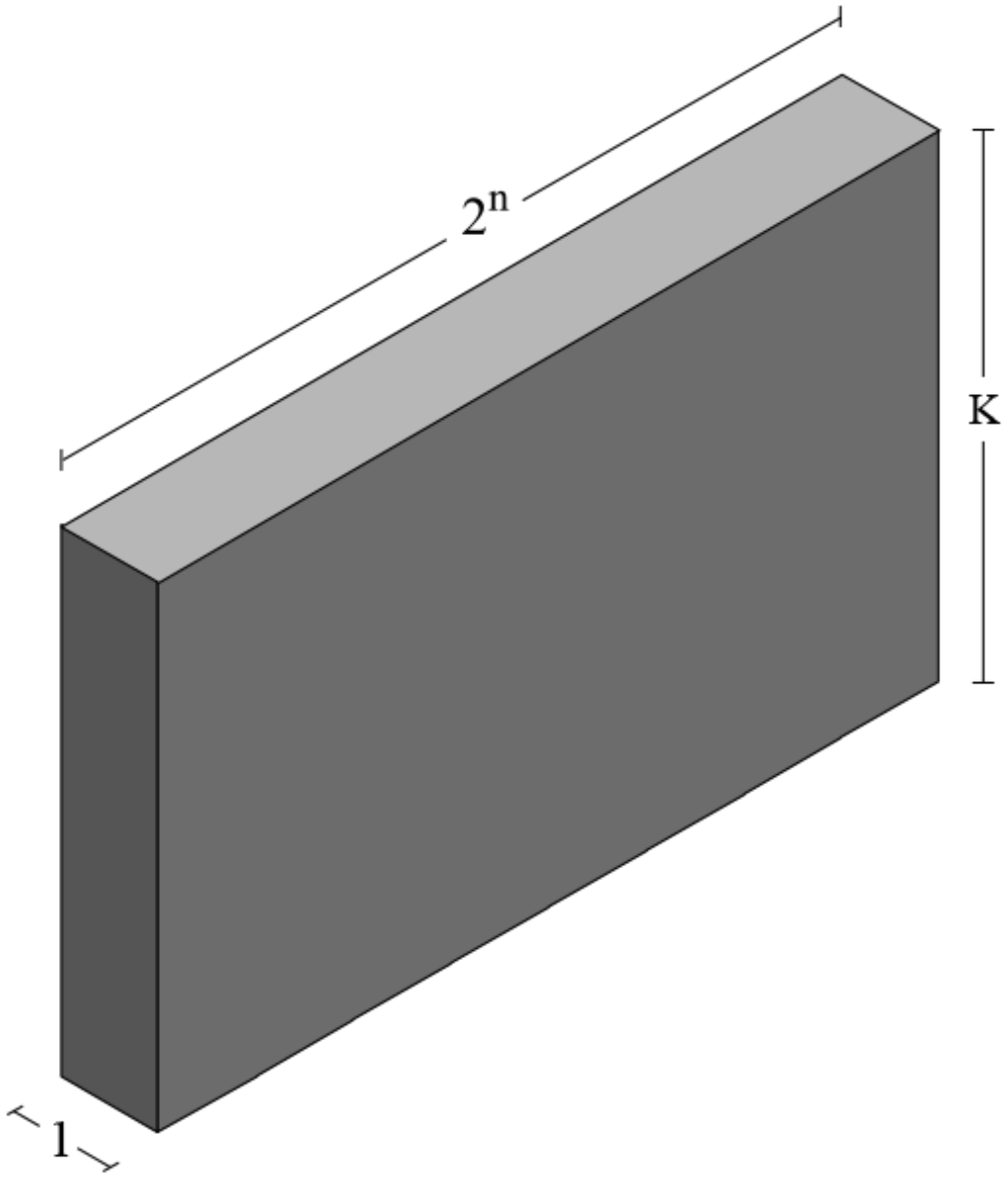
作者:July、caopengcs、红色标记。致谢:fuwutu、demo。

时间:二零一三年八月十二日

题目:用 1×1×1, 1×2×1以及2×1×1的三种木块(横绿竖蓝,且绿蓝长度均为2),



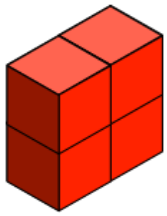
搭建高长宽分别为K × 2^N × 1的墙,不能翻转、旋转(其中,0<=N<=1024,1<=K<=4)

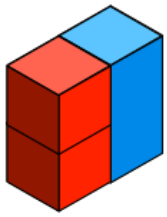


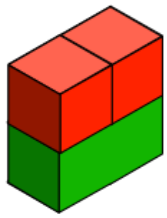
有多少种方案,输出结果

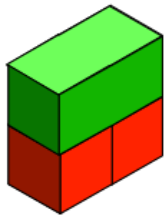
对1000000007取模。

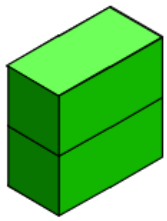
举个例子如给定高度和长度：N=1 K=2,则答案是7，即有7种搭法，如下图所示：

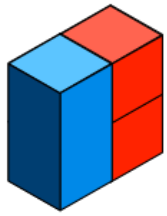


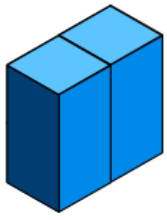




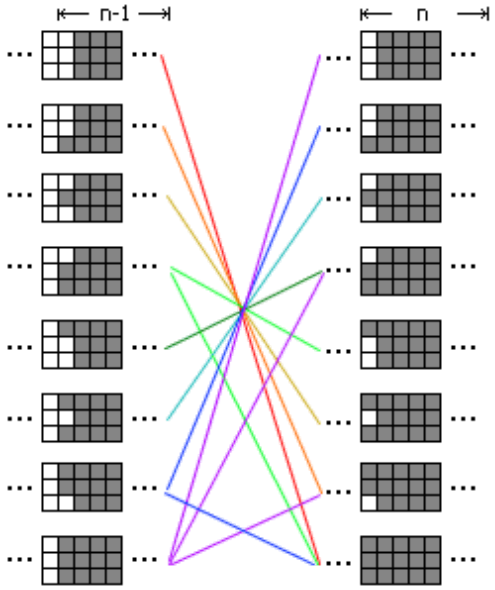


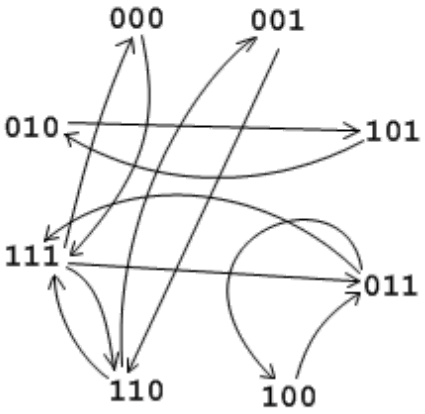






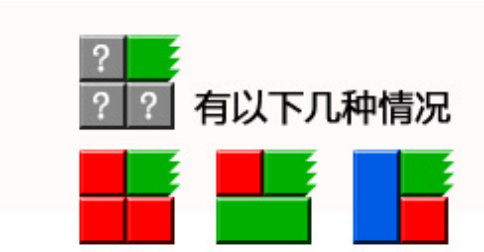
详解:此题很有意思,涉及的知识点也比较多,包括动态规划,快速矩阵幂,状态压缩,排列组合等等都一一考察了个遍。而且跟一个比较经典的矩阵乘法问题类似:即用1 x 2的多米诺骨牌填满M x N的矩形有多少种方案,M<=5,N<2^31,输出答案mod p的结果





OK,回到正题。下文使用的图示说明(所有看到的都是横切面):

首先说明“?方块”的作用



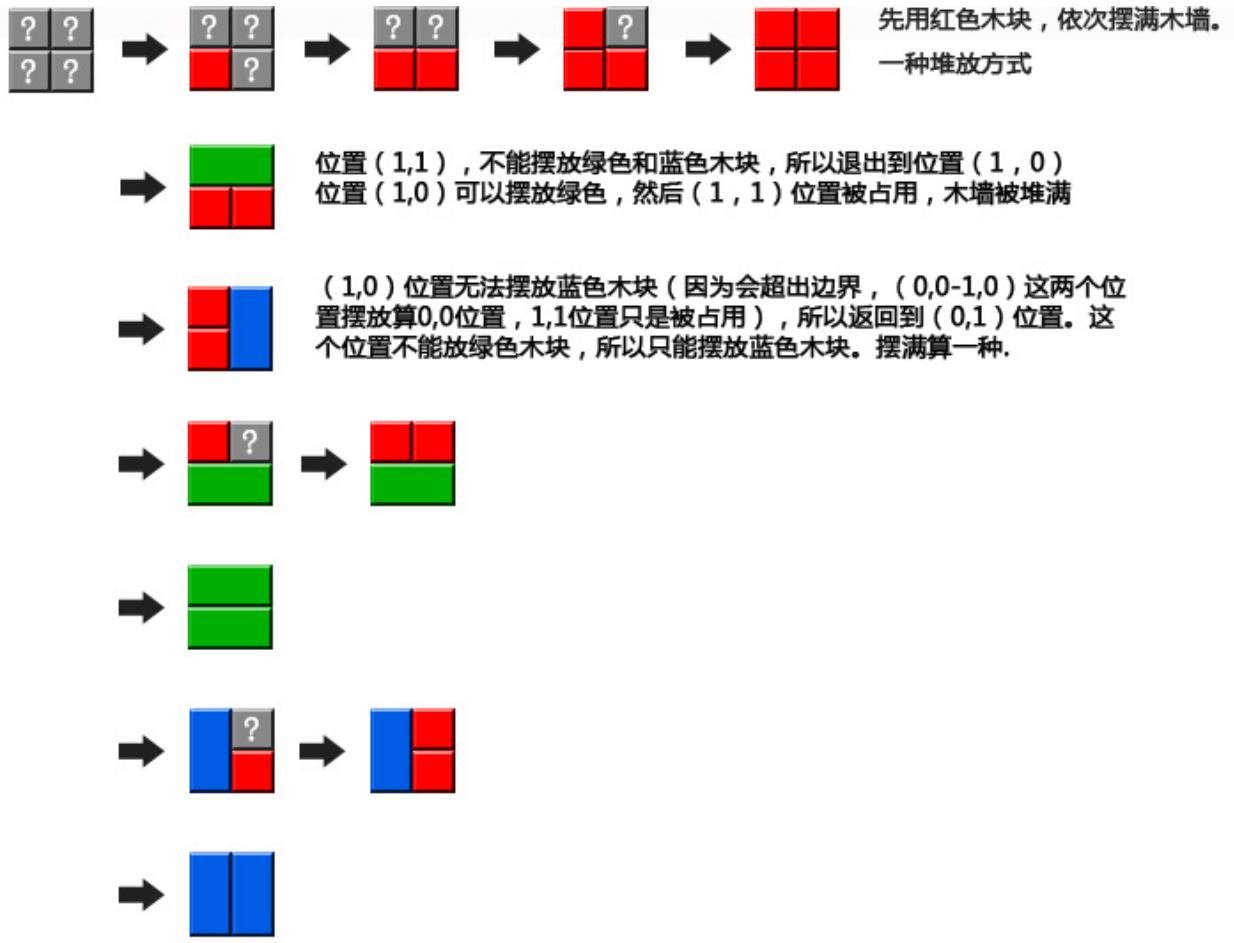
“?方块”，表示这个位置是空位置，可以任意摆放。

上图的意思就是，当右上角被绿色木块占用，此位置固定不变，其他位置任意摆放，在这种情况下的堆放方案数。

解法一、穷举遍历

初看此题，你可能最先想到的思路便是穷举：用二维数组模拟墙，从左下角开始摆放，从左往右，从下往上，最后一个格子是右上角那个位置；每个格子把每种可以摆放木块都摆放一次，每堆满一次算一种用摆放方法。为了便于描述，为木块的每个格子进行编号：

下面演示当n=1,k=2的算法过程（7种情况）：



穷举遍历在数据规模比较小的情况下还撑得住，但在0<=N<=1024这样的数据规模下，此方法则立刻变得有心无力，因此我们得寻找更优化的解法。

解法二、递归分解

递归求解就是把一个大问题，分解成小问题，逐个求解，然后再解决大问题。

2.1、算法演示

假如有墙规模为(n,k)，如果从中间切开，被分为规模问(n-1,k)的两堵墙，那么被分开的墙和原墙有什么关系呢？我们首先来看一下几组演示。

2.1.1 、n=1，k=2的情况

首先演示，**n=1,k=2**时的情况，如下图2-1：

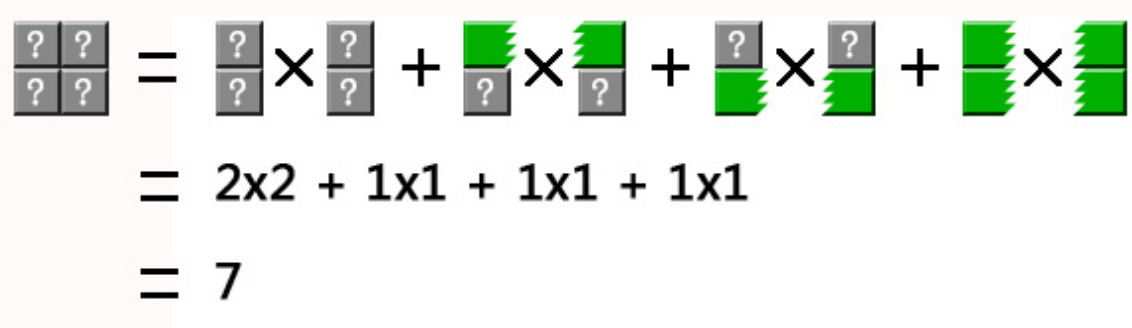
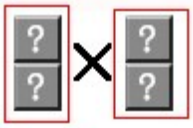


图 2-1

上图2-1中：


表示，左边墙的所有堆放方案数 * 右边墙所有堆放方案数 = 2 * 2 = 4

表示，当切开处有一个横条的时候，空位置存在的堆放方案数。左边*右边* = *1*1 = 2；剩余两组以此类推。

这个是排列组合的知识。

2.1.2 、n=2，k=3的情况

其次，我们再来演示下面更具一般性的计算分解，即当__n=2,k=3__的情况，如下图2-2：

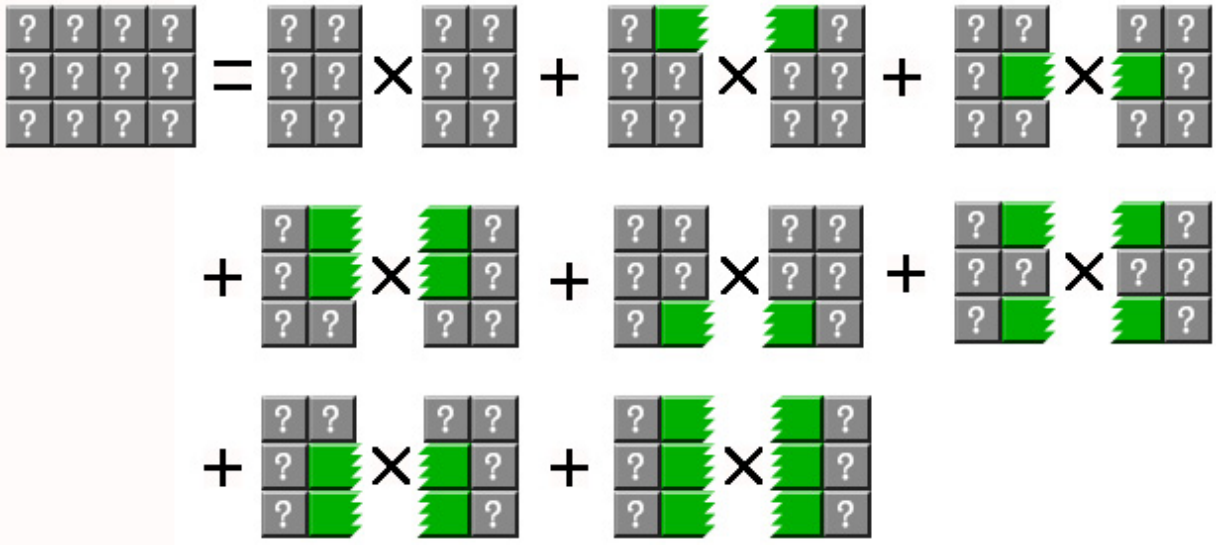


图 2-2

再从分解的结果中，挑选一组进行分解演示：

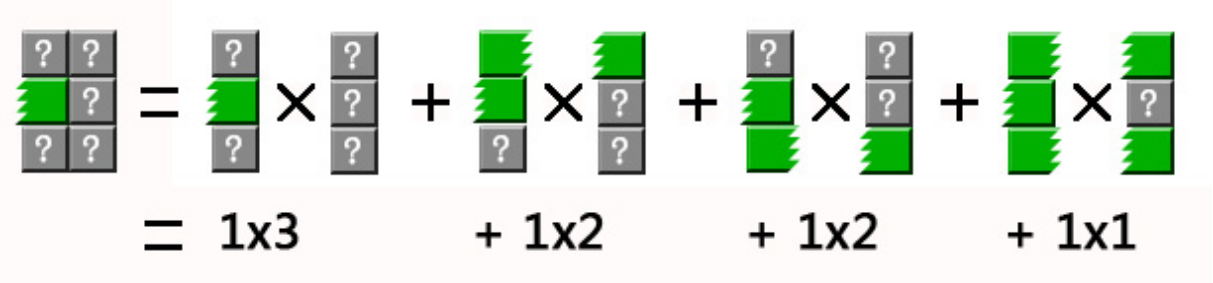


图 2-3

通过图2-2和图2-3的分解演示，可以说明，最终都是分解成一列求解。在逐级向上汇总。

2.1.3 、n=4，k=3的情况

我们再假设一堵墙n=4，k=3，也就是说，宽度是16，高度是3时，会有以下分解：

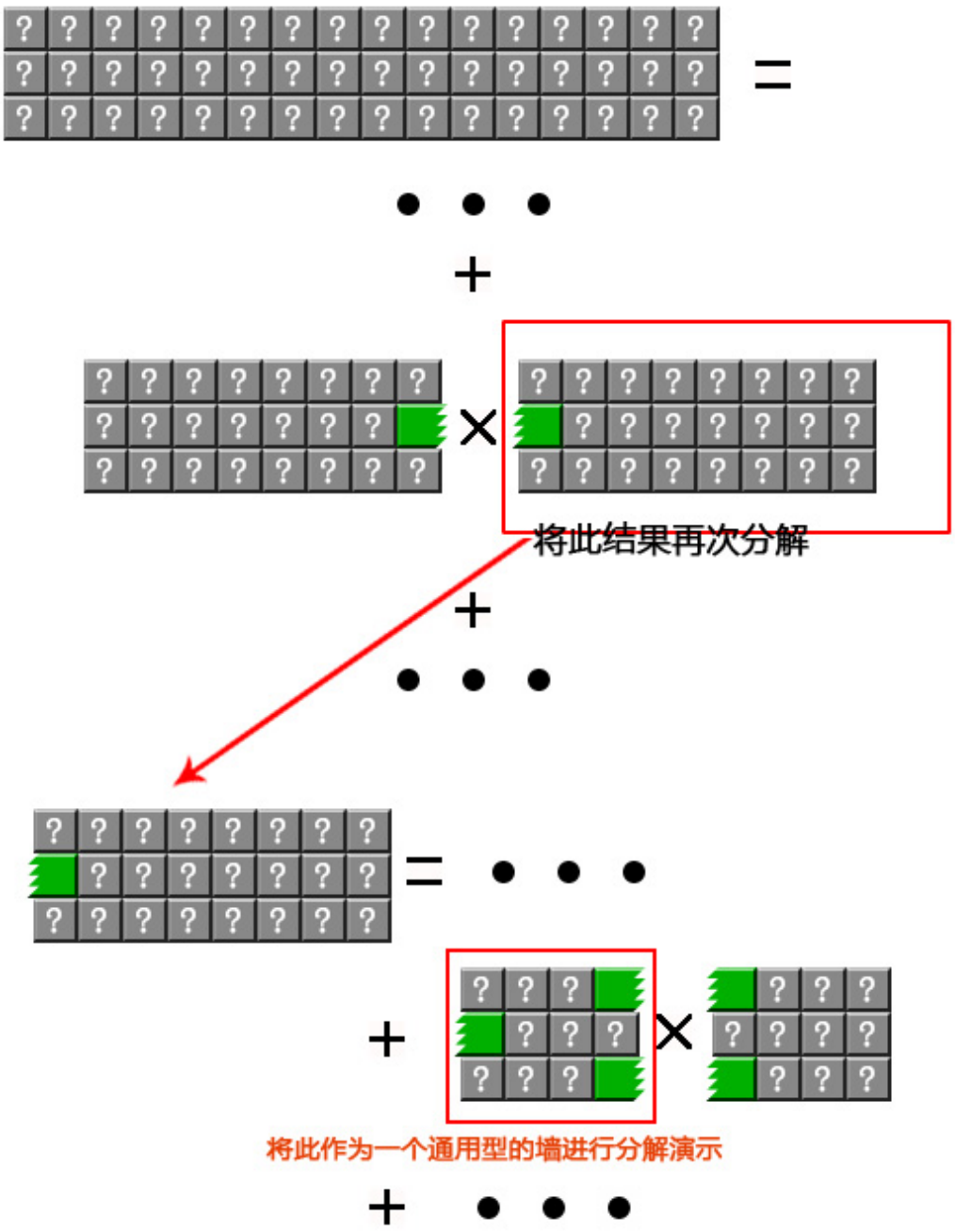


图2-4

根据上面的分解的一个中间结果，再进行分解，如下：

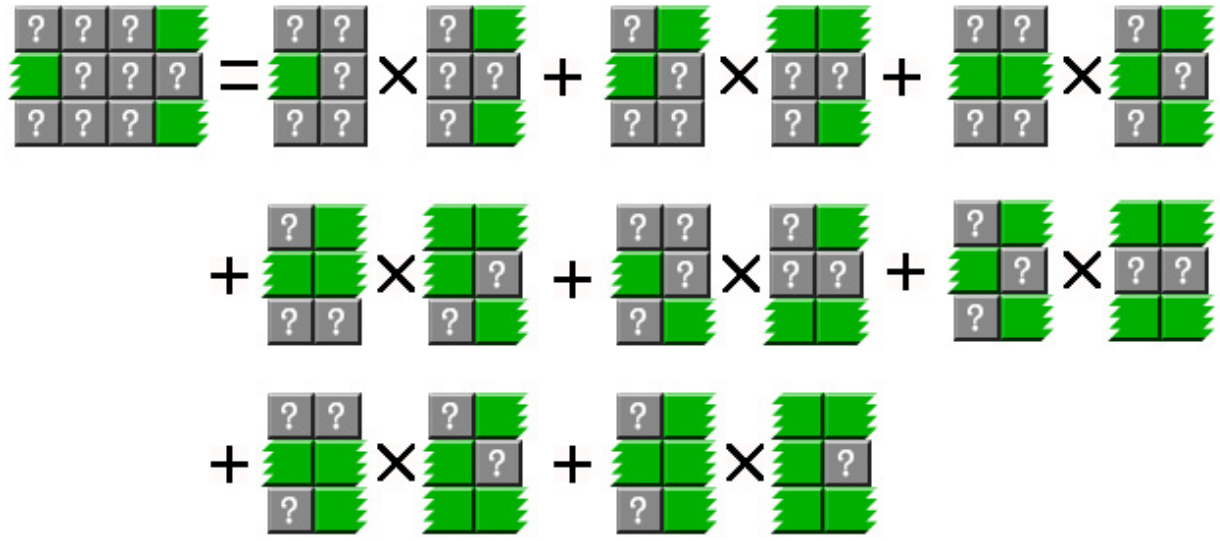


图2-5

通过上面图2-1~图2-5的演示可以明确如下几点：

1.假设f(n)用于计算问题，那么f(n)依赖于f(n-1)的多种情况。

2.切开处有什么特殊的地方呢？通过上面的演示，我们得知被切开的两堵墙从没有互相嵌入的木块（绿色木块）到全是互相连接的木块，相当于切口绿色木块的全排列（即有绿色或者没有绿色的所有排列），即有2^k种状态（比如k=2，且有绿色用1表示，没有绿色用0表示，那么就有00、01、10、11这4种状态）。根据排列组合的性质，把每一种状态下左右木墙堆放方案数相乘，再把所有乘积求和，就得到木墙的堆放结果数。以此类推，将问题逐步往下分解即可。

3.此外，从图2-5中可以看出，除了需要考虑切口绿色木块的状态，还需要考虑最左边一列和最右边一列的绿色木块状态。我们把这两种边界状态称为左边界状态和右边界状态，分别用leftState和rightState表示。

且在观察图2-5被切分后，所有左边的墙，他们的左边界ls状态始终保持不变，右边界rs状态从0~maxState, maxState = 2^k-1（有绿色方块表示1，没有表示0；ls表示左边界状态，rs表示右边界状态）：

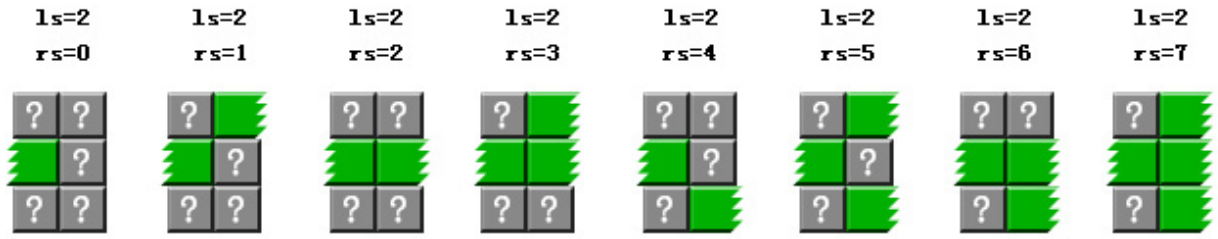


图2-6

同样可以看出右边的墙的右边界状态保持不变，而左边界状态从0~maxState。要堆砌的木墙可以看做是左边界状态=0，和右边界状态=0的一堵墙。

有一点可能要特别说明下，即上文中说，有绿色方块的状态表示标为1，无绿色方块的状态表示标为0，特意又拿上图2-6标记了一些数字，以让绝大部分读者能看得一目了然，如下所示：

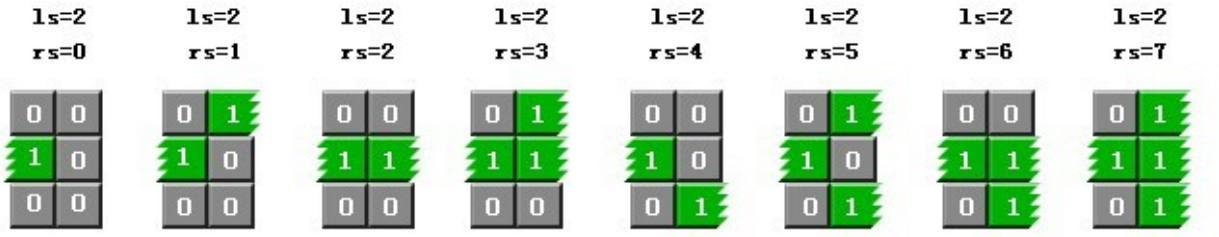


图2-7

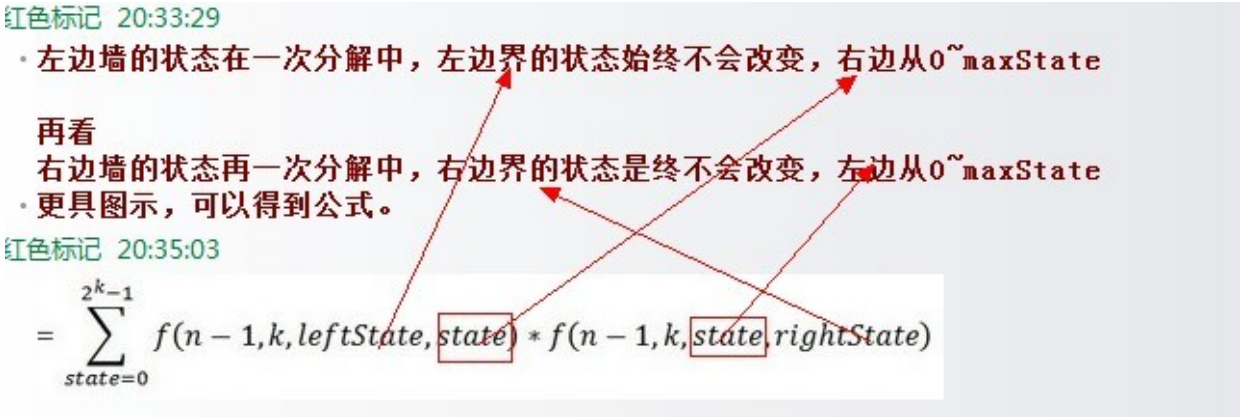
这下，你应该很清楚的看到，在上图中，左边木块的状态表示一律为010，右边木块的状态表示则是000~111（即从下至上开始计数，右边木块rs的状态用二进制表示为：000 001 010 011 100 101 110 111，它们各自分别对应整数则是：0 1 2 3 4 5 6 7）。

2.2、计算公式

通过图2-4、图2-5、图2-6的分解过程，我们可以总结出下面公式（leftState=最左边边界状态，rightState=最右边边界状态）：

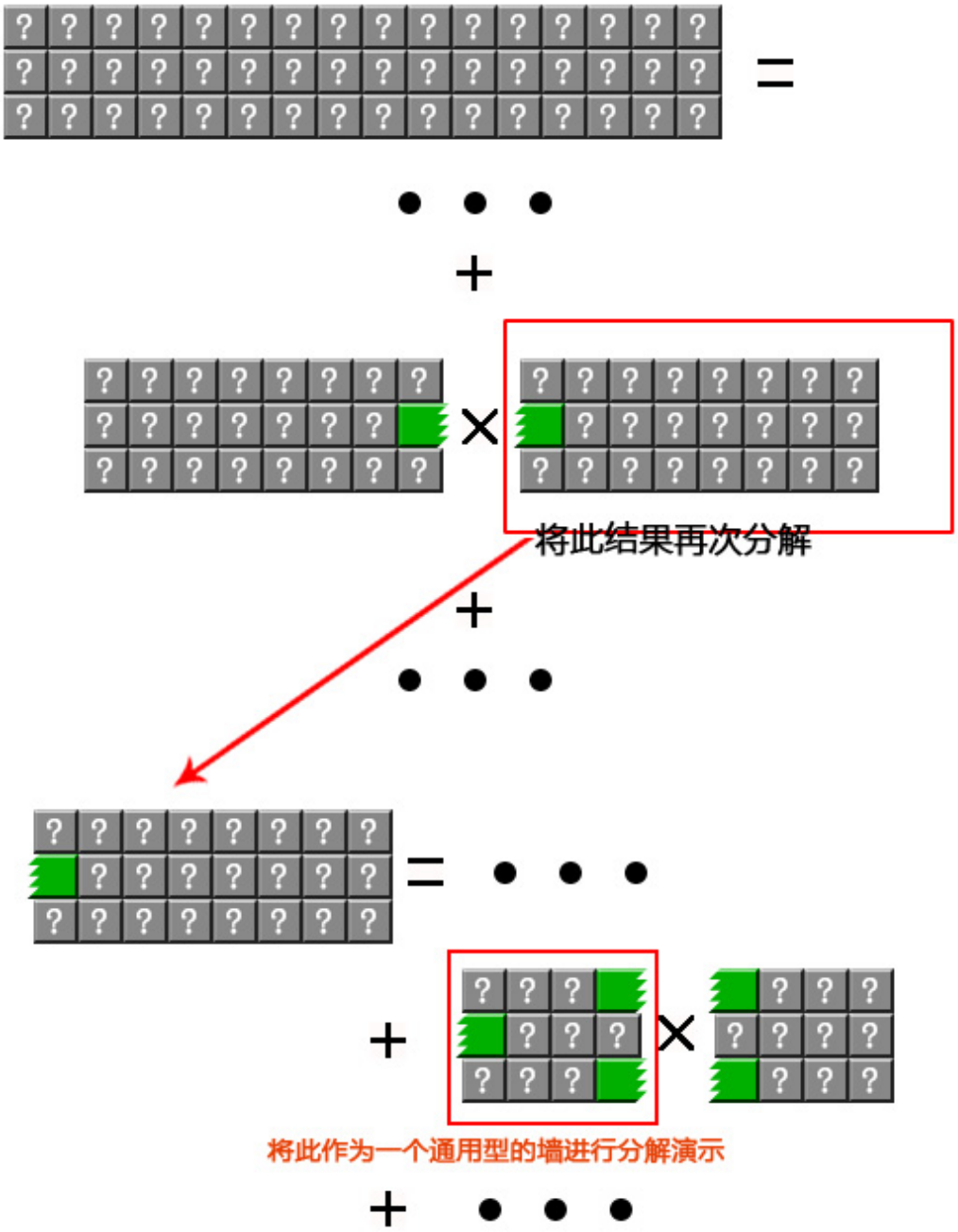
$$f(n,k,leftState,rightState)=\sum_{state=0}^{2^k-1} f(n-1,k,leftState,state)*f(n-1,k,state,rightState)$$

即：



接下来，分3点解释下上述公式：

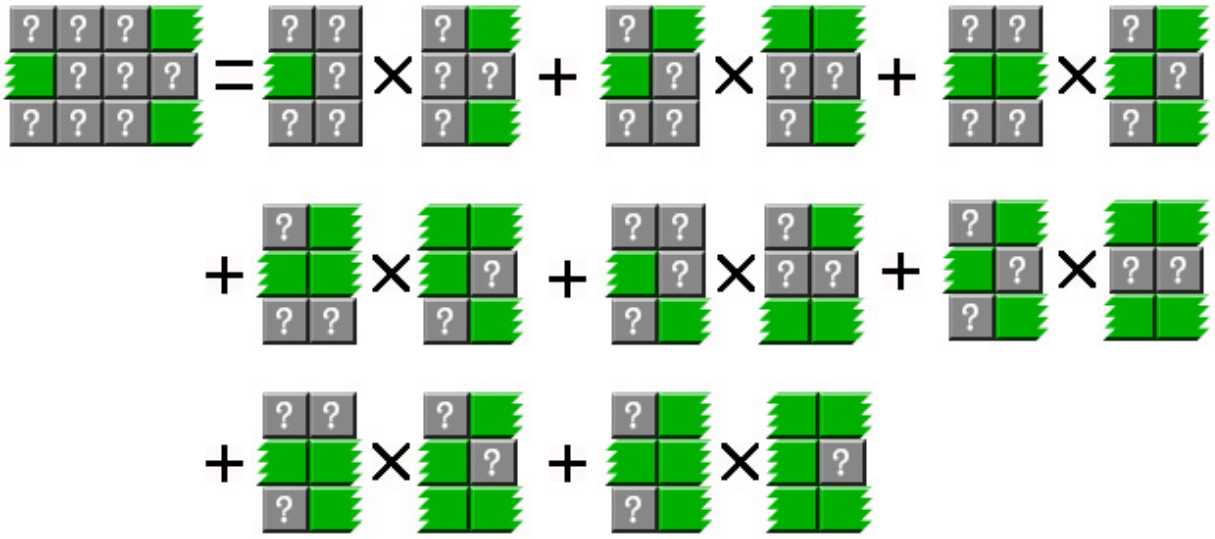
1、上述函数返回结果是当左边状态为=leftState，右边状态=rightState时木墙的堆砌方案数，相当于直接分解的左右状态都为0的情况，即直接分解f(n,k,0,0)即可。看到这，读者可能便有疑问了，既然直接分解f(n,k,0,0)即可，为何还要加leftstate和leftstate两个变量呢？回顾下2.1.3节中n=4，k=3的演示例子，即当n=4，k=3时，其分解过程即如下图（上文2.1.3节中的图2-4）

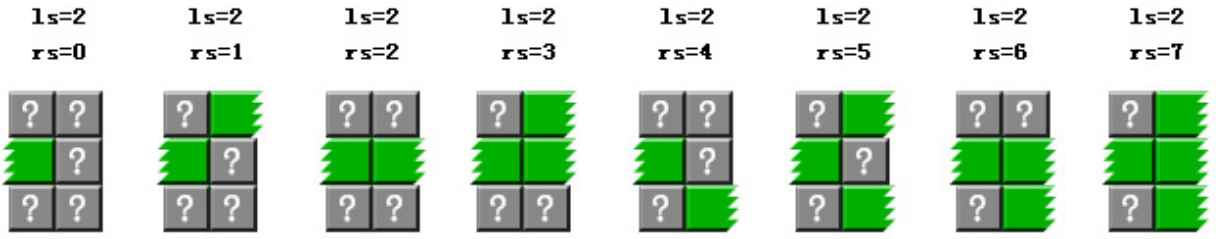


也就是说，刚开始直接分解f(4,3,0,0)，即n=4，k=3，leftstate=0，rightstate=0，但分解过程中leftstate和rightstate皆从0变化到了maxstate，故才让函数的第3和第4个参数采用leftstate和rightstate这两个变量的形式，公式也就理所当然的写成了f(n,k,leftstate,rightstate)。

2、然后我们再看下当n=4，k=3分解的一个中间结果，即给定如上图最下面部分中红色框框所框住的木块时：

它用方程表示即为 f(2,3,2,5)，怎么得来的呢？其实还是又回到了上文2.1.3节中，当n=2，k=3 时（下图即为上文2.1.3节中的图2-5和图2-6）





左边界ls状态始终保持不变时，右边界rs状态从$0_{maxState}$；右边界状态保持不变时，而左边界状态从$0_{maxState}$。

故上述分解过程用方程式可表示为：

f(2,3,2,5) = f(1,3,2,0) * f(1,3,0,5)

+ f(1,3,2,1) * f(1,3,1,5)

+ f(1,3,2,2) * f(1,3,2,5)

+ f(1,3,2,3) * f(1,3,3,5)

+ f(1,3,2,4) * f(1,3,4,5)

+ f(1,3,2,5) * f(1,3,5,5)

+ f(1,3,2,6) * f(1,3,6,5)

+ f(1,3,2,7) * f(1,3,7,5)

说白了，我们曾在2.1节中从图2-2到图2-6正推推导出了公式，然上述过程中，则又再倒推推了一遍公式进行了说明。

3、最后，作者是怎么想到引入 leftstate 和rightstate 这两个变量的呢？如红色标记所说：“因为切开后，发现绿色条，在分开处不断的变化，当时也进入了死胡同，我就在想，蓝色的怎么办。后来才想明白，与蓝色无关。每一种变化就是一种状态，所以就想到了引入leftstate 和rightstate这两个变量。”

2.3、参考代码

下面代码就是根据上面函数原理编写的。最终执行效率，n=1024,k=4时，用时0.2800160秒（之前代码用的是字典作为缓存，用时在1.3秒左右，后来改为数组结果，性能大增）。“”

```
//copyright@红色标记 12/8/2013
//updated@July 13/8/2013
using System;
using System.Collections.Generic;
using System.Text;
using System.Collections;

namespace HeapBlock
{
    public class WoolWall
    {
```

```
        private int n;
        private int height;
        private int maxState;
        private int[, ,] resultCache;    //结果缓存数组

        public WoolWall(int n, int height)
        {
            this.n = n;
            this.height = height;
            maxState = (1 << height) - 1;
            resultCache = new int[n + 1, maxState + 1, maxState + 1];    //构建缓存数组,
每个值默认为0；
        }

        /// <summary>
        /// 静态入口。计算堆放方案数。
        /// </summary>
        /// <param name="n"></param>
        /// <param name="k"></param>
        /// <returns></returns>
        public static int Heap(int n, int k)
        {
            return new WoolWall(n, k).Heap();
        }

        /// <summary>
        /// 计算堆放方案数。
        /// </summary>
        /// <returns></returns>
        public int Heap()
        {
            return (int)Heap(n, 0, 0);
        }

        private long Heap(int n, int lState, int rState)
        {
            //如果缓存数组中的值不为0,则表示该结果已经存在缓存中。
            //直接返回缓存结果。
            if (resultCache[n, lState, rState] != 0)
            {
                return resultCache[n, lState, rState];
            }

            //在只有一列的情况,无法再进行切分
            //根据列状态计算一列的堆放方案
            if (n == 0)
            {
                return CalcOneColumnHeapCount(lState);
```

```
            }

            long result = 0;
            for (int state = 0; state <= maxState; state++)
            {
                if (n == 1)
                {
                    //在只有两列的情况,判断当前状态在切分之后是否有效
                    if (!StateIsAvailable(n, lState, rState, state))
                    {
                        continue;
                    }
                     result += Heap(n - 1, state | lState, state | lState)  //合并状态。因为只有一列,所以lState和rState相同。
                        * Heap(n - 1, state | rState, state | rState);
                }
                else
                {
                          result += Heap(n - 1, lState, state) * Heap(n - 1, state, rState);
                }
                result %= 1000000007;//为了防止结果溢出，根据题目要求求模。
            }

            resultCache[n, lState, rState] = (int)result;    //将结果写入缓存数组中
            resultCache[n, rState, lState] = (int)result;    //对称的墙结果相同，所以直接写入缓存。
            return result;
        }

        /// <summary>
        /// 根据一列的状态,计算列的堆放方案数。
        /// </summary>
        /// <param name="state">状态</param>
        /// <returns></returns>
        private int CalcOneColumnHeapCount(int state)
        {
            int sn = 0; //连续计数
            int result = 1;
            for (int i = 0; i < height; i++)
            {
                if ((state & 1) == 0)
                {
                    sn++;
                }
                else
                {
                    if (sn > 0)
```

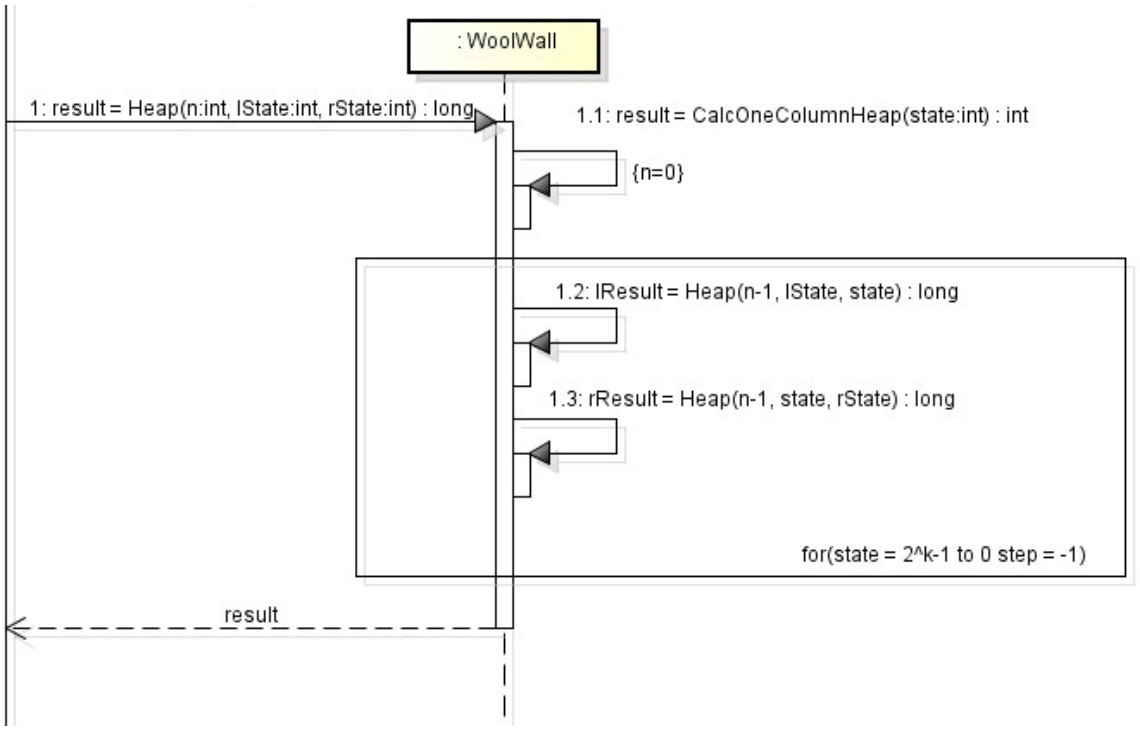
```
                    {
                        result *= CalcAllState(sn);
                    }
                    sn = 0;
                }
                state >>= 1;
            }
            if (sn > 0)
            {
                result *= CalcAllState(sn);
            }

            return result;
        }

        /// <summary>
        /// 类似于斐波那契序列。
        /// f(1)=1
        /// f(2)=2
        /// f(n) = f(n-1)*f(n-2);
        /// 只是初始值不同。
        /// </summary>
        /// <param name="k"></param>
        /// <returns></returns>
        private static int CalcAllState(int k)
        {
            return k <= 2 ? k : CalcAllState(k - 1) + CalcAllState(k - 2);
        }

        /// <summary>
        /// 判断状态是否可用。
        /// 当n=1时，分割之后，左墙和右边墙只有一列。
        /// 所以state的状态码可能会覆盖原来的边缘状态。
        /// 如果有覆盖,则该状态不可用;没有覆盖则可用。
        /// 当n>1时，不存在这种情况，都返回状态可用。
        /// </summary>
        /// <param name="n"></param>
        /// <param name="lState">左边界状态</param>
        /// <param name="rState">右边界状态</param>
        /// <param name="state">切开位置的当前状态</param>
        /// <returns>状态有效返回 true,状态不可用返回 false</returns>
        private bool StateIsAvailable(int n, int lState, int rState, int state)
        {
            return (n > 1) || ((lState | state) == lState + state && (rState | state)
== rState + state);
        }
    }
}
```

上述程序中，

- WoolWall.Heap(1024,4); //直接通过静态方法获得结果
- new WoolWall(n, k).Heap();//通过构造对象获得结果

2.3.1、核心算法讲解

因为它最终都是分解成一列的情况进行处理，这就会导致很慢。为了提高速度，本文使用了缓存机制来提高性能。缓存原理就是，n,k,leftState,rightState相同的墙，返回的结果肯定相同。利用这个特性，每计算一种结果就放入到缓存中，如果下次计算直接从缓存取出。刚开始缓存用字典类实现，有网友给出了更好的缓存方法——数组。这样性能好了很多，也更加简单。程序结构如下图所示：



上图反应了Heap调用的主要方法调用，在循环中，result 累加 lResult 和 rResult。

①在实际代码中，首先是从缓存中读取结果，如果没有从缓存中读取结果再进行计算。

分解到一列时，不再分解，直接计算结果

```
if (n == 0)
{
     return CalcOneColumnHeap(lState);
}
```

②下面是整个程序的核心代码，通过for循环，求和state=0到state=2^k-1的两边木墙乘积：

```
for (int state = 0; state <= maxState; state++)
{
    if (n == 1)
    {
        if (!StateIsAvailable(n, lState, rState, state))
        {
            continue;
        }
        result += Heap(n - 1, state | lState, state | lState) *
            Heap(n - 1, state | rState, state | rState);
    }
    else
    {
        result += Heap(n - 1, lState, state)
            * Heap(n - 1, state, rState);
    }
    result %= 1000000007;
}
```

当n=1切分时，需要特殊考虑。如下图：

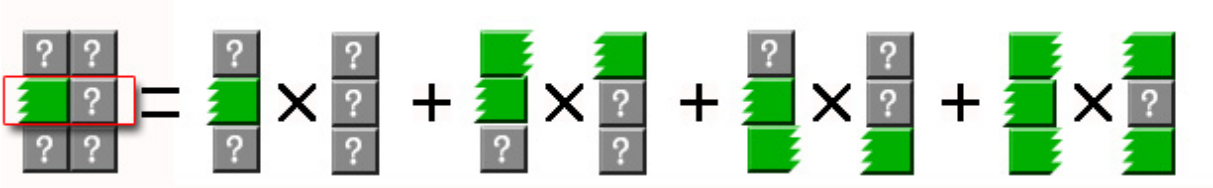
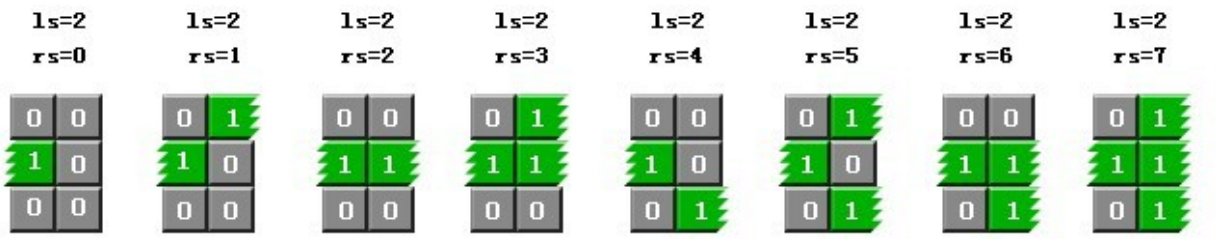


图2-8

看上图中，因为左边墙中间被绿色方块占用，所以在(1,0)-(1,1)这个位置（位置的标记方法同解法一）不能再放绿色方块。所以一些状态需要排除，如state=2需要排除。同时在还需要合并状态，如state=1时，左边墙的状态=3。

特别说明下:依据我们上文2.2节中的公式,如果第i行有这种木块,state对应$2^{(i-1)}$,加上所有行的贡献就得到state(0就是没有这种横跨木块,2^k-1就是所有行都是横跨木块),然后遍历state,还记得上文中的图2-7么?



当第i行被这样的木块 或这样的木块 占据时,其各自对应的state值分别为:

1.当第1行被占据,state=1;

2.当第2行被占据,state=2;

3.当第1和第2行都被占据,state=3;

4.当第3行被占据,state=4;

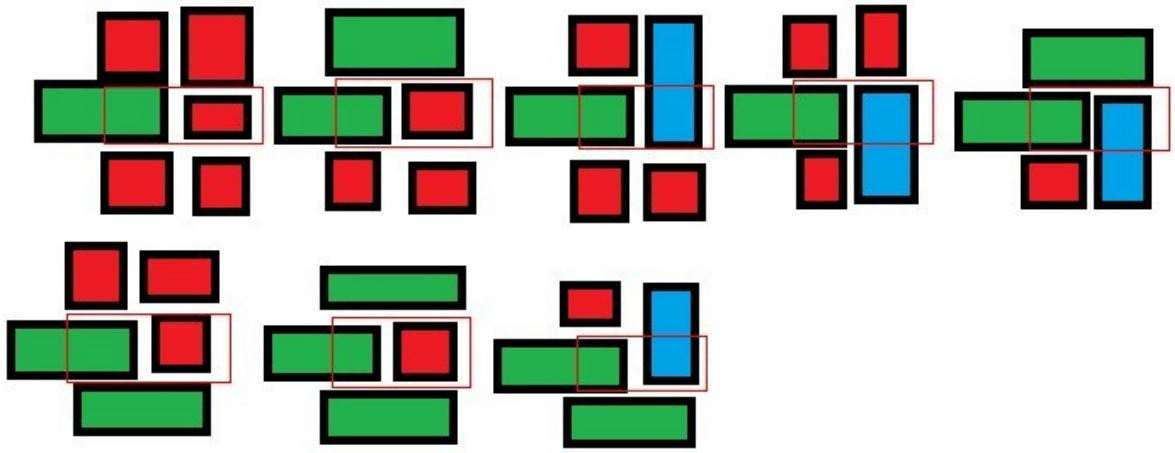
5.当第1和第3行被占据,state=5;

6.当第2和第3行被占据,state=6;

7.当第1、2、3行全部都被占据,state=7。

至于原因,即如2.1.3节节末所说:二进制表示为:000 001 010 011 100 101 110 111,它们各自分别对应整数则是:0 1 2 3 4 5 6 7。

具体来说,下面图中所有框出来的位置,不能有绿色的:



③CalcOneColumnHeap(int state)函数用于计算一列时摆放方案数。

计算方法是， 求和被绿色木块分割开的每一段连续方格的摆放方案数。每一段连续的方格的摆放方案通过CalcAllState方法求得。经过分析，可以得知CalcAllState是类似斐波那契序列的函数。

举个例子如下（分步骤讲述）：

1.令state = 4546（state=2^k-1，k最大为4，故本题中state最大在15，而这里取state=4546只是为了演示如何计算），二进制是：1000111000010。位置上为1，表示被绿色木块占用，0表示空着，可以自由摆放。

2.1000111000010 被分割后 1 000 111 0000 1 0, 那么就有 000=3个连续位置，0000=4个连续位置，0=1个连续位置。

3.堆放结果=CalcAllState(3) + CalcAllState(4) + CalcAllState(1) = 3 + 5 + 1 = 9。

2.4、再次优化

上面程序因为调用性能的树形结构，形成了大量的函数调用和缓存查找，所以其性能不是很高。为了得到更高的性能，可以让所有的运算直接依赖于上一次运算的结果，以防止更多的调用。即如果每次运算都算出

所有边界状态的结果，那么就能为下一次运算提供足够的信息。后续优化请查阅此文第3节。

解法三、动态规划

相信读到上文，不少读者都已经意识到这个问题其实就是一个动态规划问题，接下来咱们换一个角度来分析此问题。

3.1、暴力搜索不可行

首先，因为木块的宽度都是1，我们可以想成2维的问题。也就是说三种木板的规格分别为1* 1, 1 * 2, 2 * 1。

通过上文的解法一，我们已经知道这个问题最直接的想法就是暴力搜索，即对每个空格尝试放置哪种木板。但是看看数据规模就知道，这种思路是不可行的。因为有一条边范围长度高达2^{1024}，普通的电脑，2^{30}左右就到极限了。于是我们得想想别的方法。

3.2、另辟蹊径

为了方便，我们把墙看做有2^n行，k列的矩形。这是因为虽然矩形木块不能翻转，但是我们同时拥有1*2*和*2*1的两种木块。

假设我们从上到下，从左到右考虑每个1*1的格子是如何被覆盖的。显然，我们每个格子都要被覆盖住。木块的特点决定了我们覆盖一个格子最多只会影响到下一行的格子。这就可以让我们暂时只考虑两行。

假设现我们已经完全覆盖了前(i–1)行。那么由于覆盖前(i-1)行导致第i行也不“完整”了。如下图：

xxxxxxxxx

ooxooxoxo

我们用x表示已经覆盖的格子，o表示没覆盖的格子。为了方便，我们使用9列。

我们考虑第i行的状态，上图中，第1列我们可以用1*1的覆盖掉，也可以用1*2*的覆盖前两列。第4、5列的覆盖方式和第1、2列是同样的情况。第7列需要覆盖也有两种方式，即用1*1的覆盖或者用2*1的覆盖，但是这样会导致第（i+1)行第7列也被覆盖。第9列和第7列的情况是一样的。这样把第i行覆盖满了之后，我们再根据第(i+1)行被影响的状态对下一行进行覆盖。

那么每行有多少种状态呢？显然有2^k，由于k很小，我们只有大约16种状态。如果我们对于这些状态之间的转换制作一个矩阵，矩阵的第i行第j列的数表示的是我们第m行是状态i，我们把它完整覆盖掉，并且使得第(m + 1)行变成状态j的可能的方法数，这个矩阵我们可以暴力搜索出来，搜索的方式就是枚举第m行的状态，然后尝试放木板，用所有的方法把第m行覆盖掉之后，下一行的状态。当然，我们也可以认为只有两行，并且第一行是2k种状态的一种，第二行起初是空白的，求使得第一行完全覆盖掉，第二行的状态有多少种类型以及每种出现多少次。

3.3、动态规划

这个矩阵作用很大，其实我们覆盖的过程可以认为是这样：第一行是空的，我们看看把它覆盖了，第2行是什么样子的。根据第二行的状态，我们把它覆盖掉，看看第3行是什么样子的。

如果我们知道第i行的状态为s,怎么考虑第i行完全覆盖后，第(i+1)行的状态？那只要看那个矩阵的状态s对应的行就可以了。我们可以考虑一下，把两个这样的方阵相乘得到得结果是什么。这个方阵的第i行第j个元素是这样得到的，是第i行第k个元素与第k行第j个元素的对k的叠加。它的意义是上一行是第m行是状态i，把第m行和第(m+1)行同时覆盖住，第(m+2)行的状态是j的方法数。这是因为中间第(m+1)行的所有状态k，我们已经完全遍历了。

于是我们发现，每做一次方阵的乘法，我们相当于把状态推动了一行。那么我们要坐多少次方阵乘法呢？就是题目中墙的长度2^n,这个数太大了。但是事实上，我们可以不断地平方n次。也就是说我们可以算出A^2,A^4, A^8, A^{16}……方法就是不断用结果和自己相乘，这样乘n次就可以了。

因此，我们最关键的问题就是建立矩阵A。我们可以这样表示一行的状态，从左到右分别叫做第0列，第1列……覆盖了我们认为是1，没覆盖我

们认为是0，这样一行的状态可以表示为一个整数。某一列的状态我们可以用位运算来表示。例如，状态x第i列是否被覆盖，我们只需要判断x & (1 << i) 是否非0即可，或者判断(x >> i) & 1，用右移位的目的是防止溢出，但是本题不需要考虑溢出，因为k很小。接下来的任务就是递归尝试放置方案了

3.4、参考代码

最终结果，我们最初的行是空得，要求最后一行之后也不能被覆盖，所以最终结果是矩阵的第[0][0]位置的元素。另外，本题在乘法过程中会超出32位int的表示范围，需要临时用C/C++的long long，或者java的long。

参考代码如下：

```
//copyright@caopengcs 12/08/2013
#ifdef WIN32
#define ll __int64
#else
#define ll long long
#endif

// 1 covered 0 uncovered

void cal(int a[6][32][32],int n,int col,int laststate,int nowstate)
{
    if (col >= n)
    {
        ++a[n][laststate][nowstate];
        return;
    }
    //不填 或者用1*1的填
    cal(a,n, col + 1, laststate, nowstate);
    if (((laststate >> col) & 1) == 0)
    {
        cal(a,n, col + 1, laststate, nowstate | (1 << col));
        if ((col + 1 < n) && (((laststate >> (col + 1)) & 1) == 0))
        {
            cal(a,n, col + 2, laststate, nowstate);
        }
    }
}

inline int mul(ll x, ll y)
```

```
{
    return x * y % 1000000007;
}

void multiply(int n,int a[][32],int b[][32])
{ // b = a * a
    int i,j, k;
    for (i = 0; i < n; ++i)
    {
        for (j = 0; j < n; ++j)
        {
            for (k = b[i][j] = 0; k < n; ++k)
            {
                if ((b[i][j] += mul(a[i][k],a[k][j])) >= 1000000007)
                {
                    b[i][j] -= 1000000007;
                }
            }
        }
    }
}

int calculate(int n,int k)
{
    int i, j;
    int a[6][32][32],mat[2][32][32];
    memset(a,0,sizeof(a));
    for (i = 1; i <= 5; ++i)
    {
        for (j = (1 << i) - 1; j >= 0; --j)
        {
            cal(a,i, 0, j, 0);
        }
    }
    memcpy(mat[0], a[k],sizeof(mat[0]));
    k = (1 << k);
    for (i = 0; n; --n)
    {
        multiply(k, mat[i], mat[i ^ 1]);
        i ^= 1;
    }
    return mat[i][0][0];
}
```

参考链接及推荐阅读

1. caopengcs，木块砌墙
2. 红色标记，木块砌墙
3. LoveHarvy，木块砌墙
4. 在线编译测试木块砌墙问题
5. hero上木块砌墙一题

完美洗牌算法

题目详情

有个长度为2n的数组{a1,a2,a3,…,an,b1,b2,b3,…,bn}，希望排序后{a1,b1,a2,b2,….,an,bn}，请考虑有无时间复杂度o(n)，空间复杂度0(1)的解法。

题目来源：此题是去年2013年UC的校招笔试题，看似简单，按照题目所要排序后的字符串蛮力变化即可，但若要完美的达到题目所要求的时空复杂度，则需要我们花费不小的精力。OK，请看下文详解，一步步优化。

分析与解法

解法一、蛮力变换

题目要我们怎么变换，咱们就怎么变换。此题@陈利人也分析过，在此，引用他的思路进行说明。为了便于分析，我们取n=4，那么题目要求我们把

a1，a2，a3，a4，**b1，b2，b3，b4**

变成

a1，b1，a2，b2，a3，b3，a4，b4

1.1、步步前移

仔细观察变换前后两个序列的特点，我们可做如下一系列操作：

第①步、确定b1的位置，即让b1跟它前面的a2，a3，a4交换：

a1，b1，a2，a3，a4，**b2，b3，b4**

第②步、接着确定b2的位置，即让b2跟它前面的a3，a4交换：

a1，b1，a2，b2，a3，a4，**b3，b4**

第③步、b3跟它前面的a4交换位置：

a1，b1，a2，b2，a3，b3，a4，b4

b4已在最后的位置，不需要再交换。如此，经过上述3个步骤后，得到我们最后想要的序列。但此方法的时间复杂度为O（N^2），我们得继续寻找其它方法，看看有无办法能达到题目所预期的O（N）的时间复杂度。

1.2、中间交换

当然，除了如上面所述的让b1，b2，b3，b4步步前移跟它们各自前面的元素进行交换外，我们还可以每次让序列中最中间的元素进行交换达到目的。还是用上面的例子，针对a1，a2，a3，a4，b1，b2，b3，b4

第①步：交换最中间的两个元素a4，b1，序列变成（待交换的元素用粗体表示）：

a1，a2，a3，b1，a4，**b2，b3，b4**

第②步，让最中间的两对元素各自交换：

a1，a2，b1，a3，b2，a4，**b3，b4**

第③步，交换最中间的三对元素，序列变成：

a1，b1，a2，b2，a3，b3，a4，b4

同样，此法同解法1.1、步步前移一样，时间复杂度依然为O（N^2），我们得下点力气了。

解法二、完美洗牌算法

玩过扑克牌的朋友都知道，在一局完了之后洗牌，洗牌人会习惯性的把整副牌大致分为两半，两手各拿一半对着对着交叉洗牌，如下图所示：

如果这副牌用a1 a2 a3 a4 b1 b2 b3 b4表示（为简化问题，假设这副牌只有8张牌），然后一分为二之后，左手上的牌可能是a1 a2 a3 a4，右手上的牌是b1 b2 b3 b4，那么在如上图那样的洗牌之后，得到的牌就可能是b1 a1 b2 a2 b3 a3 b4 a4。

技术来源于生活，2004年，microsoft的Peiyush Jain在他发表一篇名为：“A Simple In-Place Algorithm for In-Shuffle”的论文中提出了完美洗牌算法。

这个算法解决一个什么问题呢？跟本题有什么联系呢？

Yeah，顾名思义，完美洗牌算法解决的就是一个完美洗牌问题。什么是完美洗牌问题呢？即给定一个数组a1,a2,a3,…an,b1,b2,b3..bn,最终把它置换成b1,a1,b2,a2,…bn,an。读者可以看到，这个完美洗牌问题本质上与本题完全一致，只要在完美洗牌问题的基础上对它最后的序列swap两两相邻元素即可。

即：

```
a1,a2,a3,...an,b1,b2,b3..bn
```

通过完美洗牌问题，得到：

```
b1,a1,b2,a2,b3,a3...  bn,an
```

再让上面相邻的元素两两swap，即可达到本题的要求：

```
a1,b1,a2,b2,a3,b3....,an,bn
```

也就是说，如果我们能通过完美洗牌算法（时间复杂度O(N)，空间复杂度O(1)）解决了完美洗牌问题，也就间接解决了本题。

虽然网上已有不少文章对上篇论文或翻译或做解释说明，但对于初学者来说，理解难度实在太大，再者，若直接翻译原文，根本无法看出这个算法怎么一步步得来的，故下文将从完美洗牌算法的最基本的原型开始说起，以让读者能对此算法一目了然。

2.1、位置置换pefect_shuffle1算法

为方便讨论，我们设定数组的下标从1开始，下标范围是[1..2n]。 还是通过之前n=4的例子，来看下每个元素最终去了什么地方。

起始序列：a1 a2 a3 a4 b1 b2 b3 b4 数组下标：1 2 3 4 5 6 7 8 最终序列：b1 a1 b2 a2 b3 a3 b4 a4

从上面的例子我们能看到，前n个元素中，> 第1个元素a1到了原第2个元素a2的位置，即1->2；> 第2个元素a2到了原第4个元素a4的位置，即2->4；> 第3个元素a3到了原第6个元素b2的位置，即3->6；> 第4个元素a4到了原第8个元素b4的位置，即4->8；>

那么推广到一般情况即是：前n个元素中，第i个元素去了 第（2 * i）的位置。

上面是针对前n个元素，那么针对后n个元素，可以看出：> 第5个元素b1到了原第1个元素a1的位置，即5->1；> 第6个元素b2到了原第3个元素a3的位置，即6->3；> 第7个元素b3到了原第5个元素b1的位置，即7->5；> 第8个元素b4到了原第7个元素b3的位置，即8->7；

推广到一般情况是，后n个元素，第i个元素去了第 (2 * (i - n)) - 1 = 2 * i - (2 * n + 1) = (2 * i) % (2 * n + 1) 个位置。

再综合到任意情况，任意的第i个元素，我们最终换到了 (2 * i) % (2 * n + 1)的位置。为何呢？因为：> 当0 < i < n时， 原式= (2i) % (2 * n + 1) = 2i；> 当i > n时，原式(2 * i) % (2 * n + 1)保持不变。

因此，如果题目允许我们再用一个数组的话，我们直接把每个元素放到该放得位置就好了。也就产生了最简单的方法pefect_shuffle1，参考代码如下：

```
// 时间O(n),空间O(n) 数组下标从1开始
void pefect_shuffle1(int *a, int n)
{
    int n2 = n * 2, i, b[N];
    for (i = 1; i <= n2; ++i)
    {
        b[(i * 2) % (n2 + 1)] = a[i];
    }
    for (i = 1; i <= n2; ++i)
    {
        a[i] = b[i];
    }
}
```

但很明显，它的时间复杂度虽然是O(n)，但其空间复杂度却是O(n)，仍不符合本题所期待的时间O(n)，空间O(1)。我们继续寻找更优的解法。

与此同时，我也提醒下读者，根据上面变换的节奏，我们可以看出有两个圈，> 一个是1 -> 2 -> 4 -> 8 -> 7 -> 5 -> 1；> 一个是3 -> 6 -> 3。

下文2.2.1、走圈算法cycle_leader将再次提到这两个圈。

2.2、完美洗牌算法perfect_shuffle2

2.2.1、走圈算法cycle_leader

因为之前perfect_shuffle1算法未达到时间复杂度O（N）并且空间复杂度O（1）的要求，所以我们必须得再找一种新的方法，以期能完美的解决本节开头提出的完美洗牌问题。

让我们先来回顾一下2.1节位置置换perfect_shuffle1算法，还记得我之前提醒读者的关于当n=4时，通过位置置换让每一个元素到了最后的位

置时，所形成的两个圈么？我引用下2.1节的相关内容：

当n=4的情况：

起始序列：a1 a2 a3 a4 b1 b2 b3 b4 数组下标：1 2 3 4 5 6 7 8 最终序列：b1 a1 b2 a2 b3 a3 b4 a4

即通过置换，我们得到如下结论：

“于此同时，我也提醒下读者，根据上面变换的节奏，我们可以看出有两个圈，> 一个是1 -> 2 -> 4 -> 8 -> 7 -> 5 -> 1；> 一个是3 -> 6 -> 3。”

这两个圈可以表示为（1,2,4,8,7,5）和（3,6），且perfect_shuffle1算法也已经告诉了我们，不管你n是奇数还是偶数，每个位置的元素都将变为第（2*i）%（2n+1）个元素：

因此我们只要知道圈里最小位置编号的元素即圈的头部，顺着圈走一遍就可以达到目的，且因为圈与圈是不想交的，所以这样下来，我们刚好走了O(N)步。

还是举n=4的例子，且假定我们已经知道第一个圈和第二个圈的前提下，要让1 2 3 4 5 6 7 8变换成5 1 2 7 3 8 4：

第一个圈：1 -> 2 -> 4 -> 8 -> 7 -> 5 -> 1 第二个圈：3 -> 6 -> 3：

原始数组：1 2 3 4 5 6 7 8 数组小标：1 2 3 4 5 6 7 8

走第一圈：5 1 3 2 7 6 8 4 走第二圈：5 1 6 2 7 3 8 4

上面沿着圈走的算法我们给它取名为cycle_leader，这部分代码如下：

```
//数组下标从1开始，from是圈的头部，mod是要取模的数 mod 应该为 2 * n + 1，时间复杂度O(圈长)
void cycle_leader(int *a, int from, int mod)
{
    int t,i;

    for (i = from * 2 % mod; i != from; i = i * 2 % mod)
    {
        t = a[i];
        a[i] = a[from];
```

```
            a[from] = t;
        }
    }
```

2.2.2、神级结论：若2*n=(3^k - 1)，则可确定圈的个数及各自头部的起始位置

下面我要引用此论文“A Simple In-Place Algorithm for In-Shuffle”的一个结论了，即 对于2*n = (3^{k-1})这种长度的数组，恰好只有k个圈，且每个圈头部的起始位置分别是$1,3,9,\ldots 3^{(k-1)}$。

论文原文部分为：

Thus we have the following ***In-shuffle Algorithm:***

Input: An array $A[1,\ldots,2n]$

Step 1. Find a $2m = 3^k - 1$ such that $3^k \leq 2n < 3^{k+1}$

Step 2. Do a right cyclic shift of $A[m+1,\ldots,n+m]$ by a distance m

Step 3. For each $i \in \{0,1,\ldots,k-1\}$, starting at 3^i, do the cycle leader algorithm for the in-shuffle permutation of order $2m$

Step 4. Recursively do the in-shuffle algorithm on $A[2m+1,\ldots,2n]$.

也就是说，利用上述这个结论，我们可以解决这种特殊长度2*n = (3^k-1)的数组问题，那么若给定的长度n是任意的咋办呢？此时，我们可以采取分而治之算法的思想，把整个数组一分为二，即拆分成两个部分：

让一部分的长度满足神级结论：若2*m = (3^{k-1})，则恰好k个圈，且每个圈头部的起始位置分别是$1,3,9,\ldots 3^{(k-1)}$。其中m < n，m往神级结论所需的值上套；

剩下的n-m部分单独计算；

当把n分解成m和n-m两部分后，原始数组对应的下标如下（为了方便描述，我们依然只需要看数组下标就够了）：

原始数组下标：1..m m+1.. n，n+1 .. n+m, n+m+1,..2*n

且为了能让前部分的序列满足神级结论2*m = (3^{k-1})，我们可以把中间那两段长度为n-m和m的段交换位置，即相当于把m+1..n，n+1..n+m的段循环右移m次（为什么要这么做？因为如此操作后，数组的前部分的长度为2m，而根据神级结论：当2m=3^{k-1}时，可知这长度2m的部分恰好有k个圈）。

而如果读者看过本系列第一章、左旋转字符串的话，就应该意识到循环位移是有O（N）的算法的，其思想即是把前n-m个元素（m+1.. n）和后m个元素（n+1 .. n+m）先各自翻转一下，再将整个段（m+1.. n，n+1 .. n+m）翻转下。

这个翻转的代码如下：

```
//翻转字符串时间复杂度O(to - from)
void reverse(int *a, int from, int to)
{
    int t;
    for (; from < to; ++from, --to)
    {
        t = a[from];
        a[from] = a[to];
        a[to] = t;
    }
}

//循环右移num位 时间复杂度O(n)
void right_rotate(int *a, int num, int n)
{
    reverse(a, 1, n - num);
    reverse(a, n - num + 1, n);
    reverse(a, 1, n);
}
```

翻转后，得到的目标数组的下标为：

```
目标数组下标:1..m n+1..n+m    m+1 .. n       n+m+1,..2*n
```

OK，理论讲清楚了，再举个例子便会更加一目了然。当给定n=7时，若要满足神级结论2*n=3^k-1，k只能取2，继而推得n‘=m=4。

```
原始数组:a1 a2 a3 a4       a5 a6 a7     b1 b2 b3 b4   b5 b6 b7
```

既然m=4,即让上述数组中有下划线的两个部分交换,得到:

目标数组:a1 a2 a3 a4 b1 b2 b3 b4 a5 a6 a7 b5 b6 b7

继而目标数组中的前半部分a1 a2 a3 a4 b1 b2 b3 b4部分可以用2.2.1、走圈算法cycle_leader搞定,于此我们最终求解的n长度变成了n'=3,即n的长度减小了4,单独再解决后半部分a5 a6 a7 b5 b6 b7即可。

2.2.3、完美洗牌算法perfect_shuffle3

从上文的分析过程中也就得出了我们的完美洗牌算法,其算法流程为:> 输入数组 A[1..2 * n] > step 1 找到 2 * m = 3^k - 1 使得 3^k <= 2 * n < 3^(k +1) > step 2 把a[m + 1..n + m]那部分循环移m位 > step 3 对每个i = 0,1,2..k - 1,3^i是个圈的头部,做cycle_leader算法,数组长度为m,所以对2 * m + 1取模。> step 4 对数组的后面部分A[2 * m + 1.. 2 * n]继续使用本算法,这相当于n减小了m。

上述算法流程对应的论文原文为:

以上各个步骤对应的时间复杂度分析如下:> 因为循环不断乘3的,所以时间复杂度O(logn) > 循环移位O(n) > 每个圈,每个元素只走了一次,一共2*m个元素,所以复杂度omega(m), 而m < n,所以 也在O(n)内。T(n - m) > 因此总的时间复杂度为 T(n) = T(n - m) + O(n) ,m = omega(n) ,解得:T(n) = O(n)。

此完美洗牌算法实现的参考代码如下:

```
//copyright@caopengcs 8/24/2013
//时间O(n),空间O(1)
void perfect_shuffle2(int *a, int n)
{
    int n2, m, i, k, t;
    for (; n > 1;)
    {
        // step 1
        n2 = n * 2;
        for (k = 0, m = 1; n2 / m >= 3; ++k, m *= 3)
          ;
        m /= 2;
```

```
        // 2m = 3^k - 1 , 3^k <= 2n < 3^(k + 1)

        // step 2
        right_rotate(a + m, m, n);

        // step 3
        for (i = 0, t = 1; i < k; ++i, t *= 3)
        {
          cycle_leader(a , t, m * 2 + 1);
        }

        //step 4
        a += m * 2;
        n -= m;

    }
    // n = 1
    t = a[1];
    a[1] = a[2];
    a[2] = t;
}
```

2.2.4 perfect_shuffle2算法解决其变形问题

啊哈!以上代码即解决了完美洗牌问题,那么针对本章要解决的其变形问题呢?是的,如本章开头所说,在完美洗牌问题的基础上对它最后的序列swap两两相邻元素即可，代码如下：

```
//copyright@caopengcs 8/24/2013
//时间复杂度O(n),空间复杂度O(1),数组下标从1开始,调用perfect_shuffle3
void shuffle(int *a, int n)
{
    int i, t, n2 = n * 2;
    perfect_shuffle2(a, n);
    for (i = 2; i <= n2; i += 2)
    {
        t = a[i - 1];
        a[i - 1] = a[i];
        a[i] = t;
   }
}
```

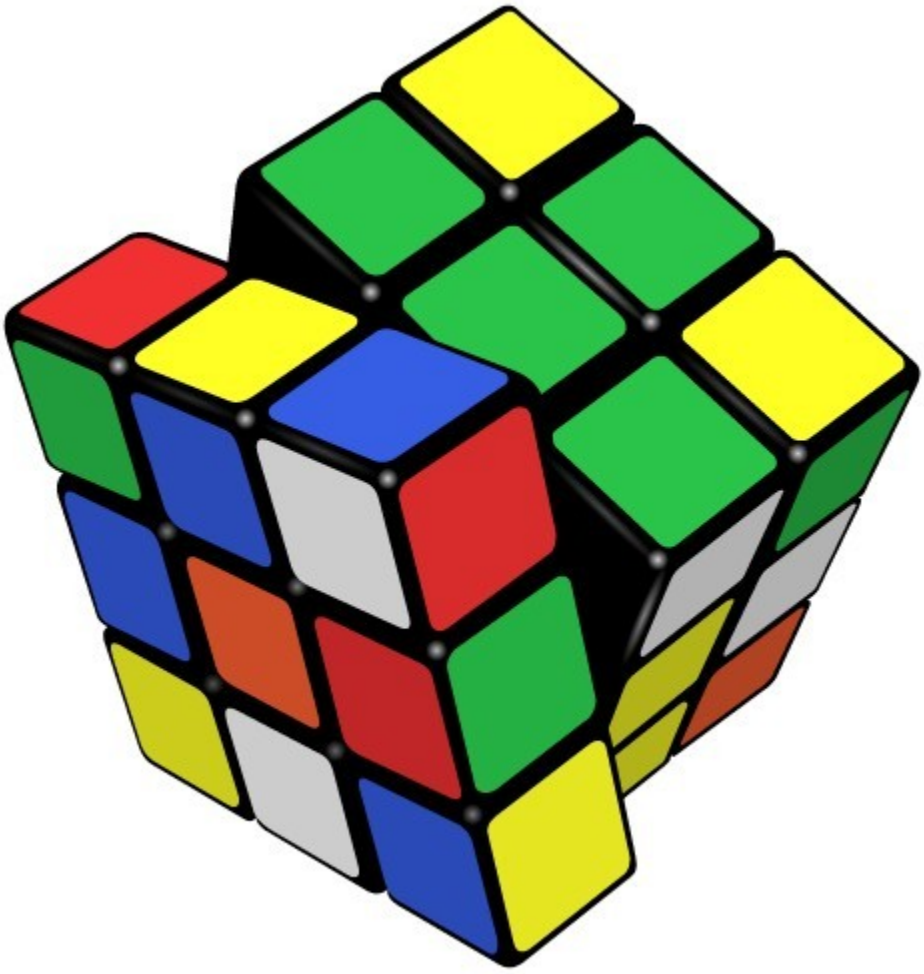
上述的这个“在完美洗牌问题的基础上对它最后的序列swap两两相邻元素”的操作（当然，你也可以让原数组第一个和最后一个不变，中间的

2 * (n - 1)项用原始的标准完美洗牌算法做），只是在完美洗牌问题时间复杂度O(N)空间复杂度O(1)的基础上再增加O(N)的时间复杂度，故总的时间复杂度O(N)不变，且理所当然的保持了空间复杂度O(1)。至此，咱们的问题得到了圆满解决！

问题扩展

神级结论是如何来的？

我们的问题得到了解决，但本章尚未完，即决定完美洗牌算法的神级结论：若2*n=（3^k - 1），则恰好只有k个圈，且每个圈头部的起始位置分别是1,3,9，…3^(k-1)，是如何来的呢？



要证明这个结论的关键就是:这所有的圈合并起来必须包含从1到M之间的所有正数,一个都不能少。这个证明有点麻烦,因为证明过程中会涉及到群论等数论知识,但再远的路一步步走也能到达。

首先,让咱们明确以下相关的概念,定理,及定义(搞清楚了这些东西,咱们便证明了一大半):> 概念1 mod表示对一个数取余数,比如3 mod 5 =3,5 mod 3 =2;> 定义1 欧拉函数ϕ(m) 表示为不超过m(即小于等于m)的数中,与m互素的正整数个数 > 定义2 若ϕ(m)=Ordm(a) 则称a为m的原根,其中Ordm(a)定义为:a ^d (mod m),其中d=0,1,2,3…,但取让等式成立的最小的那个d。

结合上述定义1、定义2可知, 2是3的原根, 因为2^0 mod 3 = 1, 2^1 mod 3 = 2, 2^2 mod 3 = 1, 2^3 mod 3 = 2,{a^0 mod m,a^1 mod m,a^2}得到集合S={1,2},包含了所有和3互质的数,也即d=ϕ(2)=2,满足原根定义。

而2不是7的原根,这是因为2^0 mod 7 = 1, 2^1 mod 7 = 2, 2^2 mod 7 = 4, 2^3 mod 7 = 1,2^4 mod 7 = 2,2^5 mod 7 = 4,2^6 mod 7 = 1,从而集合S={1,2,4}中始终只有1、2、4三种结果,而没包含全部与7互质的数(3、6、5便不包括),,即d=3,但ϕ(7)=6,从而d != ϕ(7),不满足原根定义。

再者,如果说一个数a,是另外一个数m的原根,代表集合S = {a^0 mod m, a^1 mod m, a^2 mod m…… },得到的集合包含了所有小于m并且与m互质的数,否则a便不是m的原根。而且集合S = {a^0 mod m, a^1 mod m, a^2 mod m…… }中可能会存在重复的余数,但当a与m互质的时候,得到的{a^0 mod m, a^1 mod m, a^2 mod m}集合中,保证了第一个数是a^0 mod m,故第一次发现重复的数时,这个重复的数一定是1,也就是说,出现余数循环一定是从开头开始循环的。> 定义3 对模指数,a对模m的原根定义为 $Ord_m(a) = d$,st: $a^d \equiv 1(mod m)$ 中最小的正整数d

再比如,2是9的原根,因为$2^d \equiv 1(mod 9)$, 为了让2^d除以9的余数恒等于1,可知最小的正整数d=6,而ϕ(m)=6,满足原根的定义。 > 定理1 同余定理: 两个整数a,b,若它们除以正整数m所得的余数相等,则称a,b对于模m同余,记作$a \equiv b \pmod{m}$, 读做a与b关于模m同余。> 定理2 当p为奇素数且a是p^2的原根时⇒ a也是p^k的原根 > 定理3 费马小定理:如果

a和m互质，那么a^ϕ(m) mod m = 1 > 定理4 若(a,m)=1 且a为m的原根，那么a是(Z/mZ)*的生成元。

取a = 2, m = 3。

我们知道2是3的原根，2是9的原根，我们定义S(k)表示上述的集合S，并且取x = 3^k（x表示为集合S中的数）。

所以：

```
S(1) = {1, 2}
S(2) = {1, 2, 4, 8, 7, 5}
```

我们没改变圈元素的顺序，由前面的结论S(k)恰好是一个圈里的元素，且认为从1开始循环的，也就是说从1开始的圈包含了所有与3^k互质的数。

那与3^k不互质的数怎么办？如果0 < i < 3^k与 3^k不互质，那么i 与3^k的最大公约数一定是3^t的形式（只包含约数3），并且 t < k。即gcd(i , 3^k) = 3^t，等式两边除以个3 ^ t，即得gcd(i/(3^t), $3^{(k - t)}$) = 1， i/(3^t) 都与3^(k - t) 互质了，并且i / (3^t) < 3^(k - t), 根据S(k)的定义，可见i/(3^t) 在集合S(k - t)中。

同理，任意S(k - t)中的数x，都满足gcd(x , 3^k) = 1,于是gcd(3^k , x* 3^t) = 3 ^ t, 并且x*3^t < 3^k。可见S(k - t)中的数x*3^t 与 i形成了一一对应的关系。

也就是说S(k - t)里每个数x* 3^t形成的新集合包含了所有与3^k的最大公约数为3^t的数，它也是一个圈,原先圈的头部是1，这个圈的头部是3^t。

于是，对所有的小于 3^k的数，根据它和3^k的最大公约数，我们都把它分配到了一个圈里去了，且k个圈包含了所有的小于3^k的数。

下面，举个例子，如caopengcs所说，当我们取“a = 2, m = 3时，

我们知道2是3的原根，2是9的原根，我们定义S(k)表示上述的集合S，并且x= 3^k。

所以S(1) = {1, 2}

S(2) = {1, 2, 4, 8, 7, 5}

比如k = 3。我们有：

S(3) = {1, 2 ,4 , 8, 16, 5, 10, 20, 13, 26, 25, 23, 19, 11, 22, 17, 7, 14} 包含了小于27且与27互质的所有数，圈的首部为1，这是原根定义决定的。

那么与27最大公约数为3的数，我们用S(2)中的数乘以3得到。S(2) * 3 = {3, 6, 12, 24, 21, 15}, 圈中元素的顺序没变化，圈的首部是3。

与27最大公约数为9的数，我们用S(1)中的数乘以9得到。S(1) * 9 = {9, 18}, 圈中得元素的顺序没变化，圈的首部是9。

因为每个小于27的数和27的最大公约数只有1, 3, 9这3种情况，又由于前面所证的一一对应的关系，所以S(2) * 3包含了所有小于27且与27的最大公约数为3的数，S(1) * 9 包含了所有小于27且和27的最大公约数为9的数。”

换言之，若定义为整数，假设/N定义为整数Z除以N后全部余数的集合，包括{0…N-1}等N个数，而(/N)*则定义为这Z/N中{0…N-1}这N个余数内与N互质的数集合。

则当n=13时，2n+1=27，即得/N ={0,1,2,3,…..,26}，(/N)*相当于就是{0,1,2,3,…..,26}中全部与27互素的数的集合；

而2^k(mod 27)可以把(/27)*取遍，故可得这些数分别在以下3个圈内：

取头为1，(/27)*＝{1,2,4,8,16,5,10,20,13,26,25,23,19,11,22,17,7,14}，也就是说，与27互素且小于27的正整数集合为{1,2,4,8,16,5,10,20,13,26,25,23,19,11,22,17,7,14}，因此ϕ(m) = ϕ(27)=18，从而满足$a^d \equiv 1(mod m)$的最小d = 18，故得出2为27的原根；

取头为3，就可以得到{3,6,12,24,21,15}，这就是以3为头的环，这个圈的特点是所有的数都是3的倍数，且都不是9的倍数。为什么呢？因为2^k和27互素。

具体点则是：如果3×2^k除27的余数能够被9整除，则有一个n使得$3*2^k=9n(mod\ 27)$，即$3*2^k-9n$能够被27整除，从而$3*2^{k-9n}=27m$，其中n，m为整数，这样一来，式子约掉一个3，我们便能得到$2^k=9m+3n$，也就是说，2^k是3的倍数，这与2^k与27互素是矛盾的，所以，3×2^k除27的余数不可能被9整除。

此外，2^k除以27的余数可以是3的倍数以外的所有数，所以，2^k除以27的余数可以为1,2,4,5,7,8，当余数为1时，即存在一个k使得2^k-1=27m，m为整数。

式子两边同时乘以3得到：$3*2^k-3=81m$是27的倍数，从而$3*2^k$除以27的余数为3；

同理，当余数为2时，2^k - 2 = 27m，=> $3*2^k- 6 =81m$，从而$3*2^k$除以27的余数为6；

当余数为4时，2^k - 4 = 37m，=> $3*2^k - 12 =81m$，从而$3*2^k$除以27的余数为12；

同理，可以取到15，21，24。从而也就印证了上面的结论：取头为3，就可以得到{3,6,12,24,21,15}。取9为头，这就很简单了，这个圈就是{9,18}

你会发现，小于27的所有自然数，要么在第一个圈里面，也就是那些和27互素的数；要么在第二个圈里面，也就是那些是3的倍数，但不是9的倍数的数；要么在第三个圈里面，也就是是9倍数的数，而之所以能够这么做，就是因为2是27的本原根。证明完毕。

最后，咱们也再验证下上述过程：

因为$i \in \{1,2,\ldots,2n\}$，故：

i = 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27

由于n=13，2n+1 = 27，据此$i \to 2i(mod(2n+1))$公式可知，上面第 i 位置的数将分别变成下述位置的：

i = 2 4 6 8 10 12 14 16 18 20 22 24 26 1 3 5 7 9 11 13 15 17 19 21 23 25 0

根据i 和 i‘ 前后位置的变动，我们将得到3个圈：

```
1->2->4->8->16->5->10->20->13->26->25->23->19->11->22->17->7->14->1;
3->6->12->24->21->15->3
9->18->9
```

没错，这3个圈的数字与咱们之前得到的3个圈一致吻合，验证完毕。

举一反三

至此，本章开头提出的问题解决了，完美洗牌算法的证明也证完了，是否可以止步了呢？OH，NO！读者有无思考过下述问题：

1、既然完美洗牌问题是给定输入：a1,a2,a3,……aN,b1,b2,b3,……bN，要求输出：b1,a1,b2,a2,……bN,aN；那么有无考虑过它的逆问题：即给定b1,a1,b2,a2,……bN,aN,，要求输出a1,a2,a3,……aN,b1,b2,b3,……bN ？

2、完美洗牌问题是两手洗牌，假设有三只手同时洗牌呢？那么问题将变成：输入是a1,a2,……aN, b1,b2,……bN, c1,c2,……cN，要求输出是c1,b1,a1,c2,b2,a2,……cN,bN,aN，这个时候，怎么处理？

第三十九章、最近公共祖先LCA问题

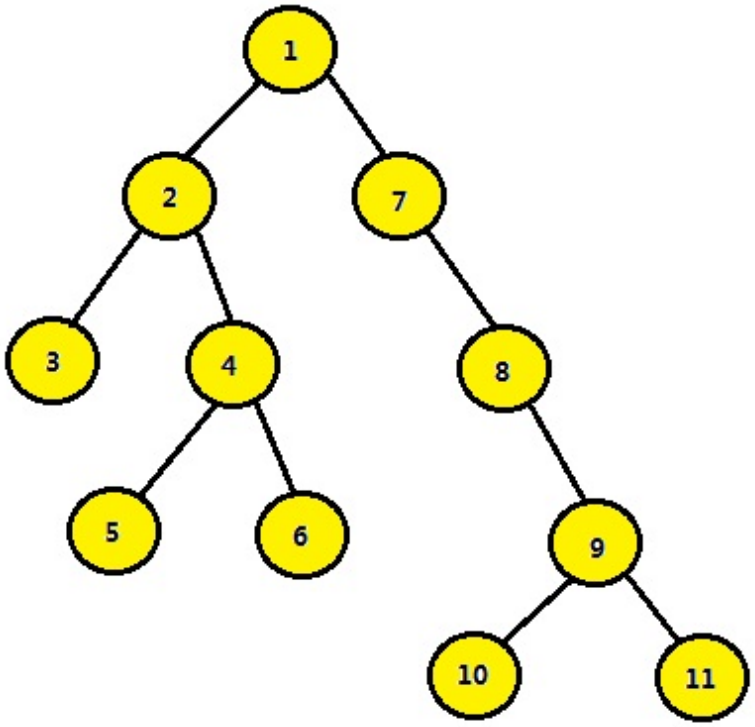
问题描述：求二叉树的任意两个节点的最近公共祖先。

题意分析

这个问题来自去年10月整理的腾讯笔试题，即此文第31题，网上也有很多文章阐述了这个问题，然要么是阐述不够细致规范，要么千篇一律的晦涩难懂，希望本文能把这个问题阐述的明明白白。

解答这个问题之前，咱们得先搞清楚到底什么是最近公共祖先。最近公共祖先简称LCA，所谓LCA，是当给定一个有根树T时，对于任意两个结点u、v，找到一个离根最远的结点x，使得x同时是u和v的祖先，x便是u、v的最近公共祖先。

举个例子，如针对下图所示的一棵普通的二叉树来讲：



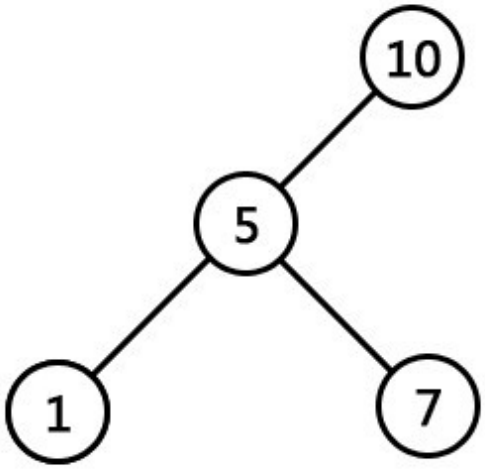
结点3和结点4的最近公共祖先是结点2，即LCA（3 4）=2 。在此，需要注意到当两个结点在同一棵子树上的情况，如结点3和结点2的最近公共祖先为2，即 LCA（3 2）=2。同理：LCA（5 6）=4，LCA（6 10）=1。

明确了题意，咱们便来试着解决这个问题。一般文章的做法，可能是针对是否为二叉查找树分情况讨论，想必这也是一般人最先想到的思路。除此之外，还有所谓的Tarjan算法、倍增算法、以及转换为RMQ问题（求某段区间的极值）。

解法一、暴力对待

1.1、是二叉查找树

在当这棵树是二叉查找树的情况下，如下图：



那么从树根开始：

- 如果当前结点t 大于结点u 、v，说明u 、v都在t 的左侧，所以它们的共同祖先必定在t 的左子树中，故从t 的左子树中继续查找；
- 如果当前结点t 小于结点u 、v，说明u 、v都在t 的右侧，所以它们的共同祖先必定在t 的右子树中，故从t 的右子树中继续查找；
- 如果当前结点t 满足 u <t < v，说明u和v分居在t 的两侧，故当前结点t 即为最近公共祖先；
- 而如果u是v的祖先，那么返回u的父结点，同理，如果v是u的祖先，那么返回v的父结点。

代码如下所示：

```
//copyright@eriol 2011
//modified by July 2014
public int query(Node t, Node u, Node v) {
    int left = u.value;
    int right = v.value;
    Node parent = null;

    //二叉查找树内，如果左结点大于右结点，不对，交换
    if (left > right) {
        int temp = left;
        left = right;
        right = temp;
    }
```

```
    while (true) {
        //如果t小于u ~v,往t的右子树中查找
        if (t.value < left) {
            parent = t;
            t = t.right;

        //如果t大于u ~v,往t的左子树中查找
        } else if (t.value > right) {
            parent = t;
            t = t.left;
        } else if (t.value == left || t.value == right) {
            return parent.value;
        } else {
            return t.value;
        }
    }
}
```

1.2、不是二叉查找树

但如果这棵树不是二叉查找树，只是一棵普通的二叉树呢？一网友何海涛在他博客中用了一种蛮力方法。由于每个结点都有一个指针指向它的父结点，于是我们可以从任何一个结点出发，得到一个到达树根结点的单向链表。因此这个问题转换为两个单向链表的第一个公共结点。

我不想再在这里重复赘述，有兴趣的可以看原文。

事实上，正如本文读者allantop所说，如果给出根节点，LCA问题可以用递归很快解决。而关于树的问题一般都可以转换为递归（因为树本来就是递归描述），故可写下如下代码：

```
//copyright@allantop 2014-1-22-20:01
node* getLCA(node* root, node* node1, node* node2)
{
    if(root == null)
        return null;
    if(root== node1 || root==node2)
        return root;

    node* left = getLCA(root->left, node1, node2);
    node* right = getLCA(root->right, node1, node2);

    if(left != null && right != null)
```

```
        return root;
    else if(left != null)
        return left;
    else if (right != null)
        return right;
    else
        return null;
}
```

然不论是针对普通的二叉树，还是针对二叉查找树，上面的解法有一个很大的弊端就是：如需N 次查询，则总体复杂度会扩大N 倍，故这种暴力解法仅适合一次查询，不适合多次查询。

接下来的解法，将不再区别对待是否为二叉查找树，而是一致当做是一棵普通的二叉树。总体来说，由于可以把LCA问题看成是询问式的，即给出一系列询问，程序对每一个询问尽快做出反应。故处理这类问题一般有两种解决方法：

- 一种是在线算法，相当于循序渐进处理；
- 另外一种则是离线算法，如Tarjan算法，相当于一次性批量处理，一开始就知道了全部查询，只待询问。

解法二、Tarjan算法

如上文末节所述，不论咱们所面对的二叉树是二叉查找树，或不是二叉查找树，都可以把求任意两个结点的最近公共祖先，当做是查询的问题，如果是只求一次，则是单次查询；如果要求多个任意两个结点的最近公共祖先，则相当于是批量查询。

涉及到批量查询的时候，咱们可以借鉴离线处理的方式，这就引出了解决此LCA问题的Tarjan离线算法。

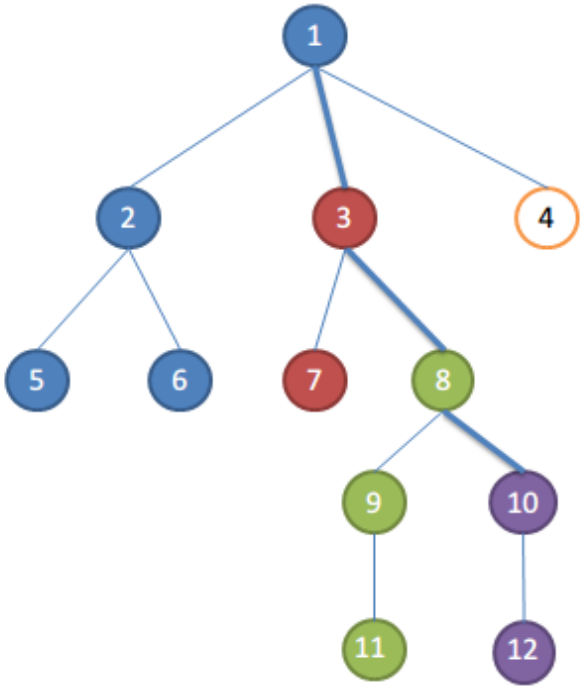
2.1、什么是Tarjan算法

Tarjan算法（以发现者Robert Tarjan命名）是一个在图中寻找强连通分量的算法。算法的基本思想为：任选一结点开始进行深度优先搜索dfs（若深度优先搜索结束后仍有未访问的结点，则再从中任选一点再次进行）。搜

索过程中已访问的结点不再访问。搜索树的若干子树构成了图的强连通分量。

应用到咱们要解决的LCA问题上，则是：对于新搜索到的一个结点u，先创建由u构成的集合，再对u的每颗子树进行搜索，每搜索完一棵子树，这时候子树中所有的结点的最近公共祖先就是u了。

引用此文的一个例子，如下图（不同颜色的结点相当于不同的集合）：



假设遍历完10的孩子,要处理关于10的请求了，取根节点到当前正在遍历的节点的路径为关键路径,即1-3-8-10，集合的祖先便是关键路径上距离集合最近的点。

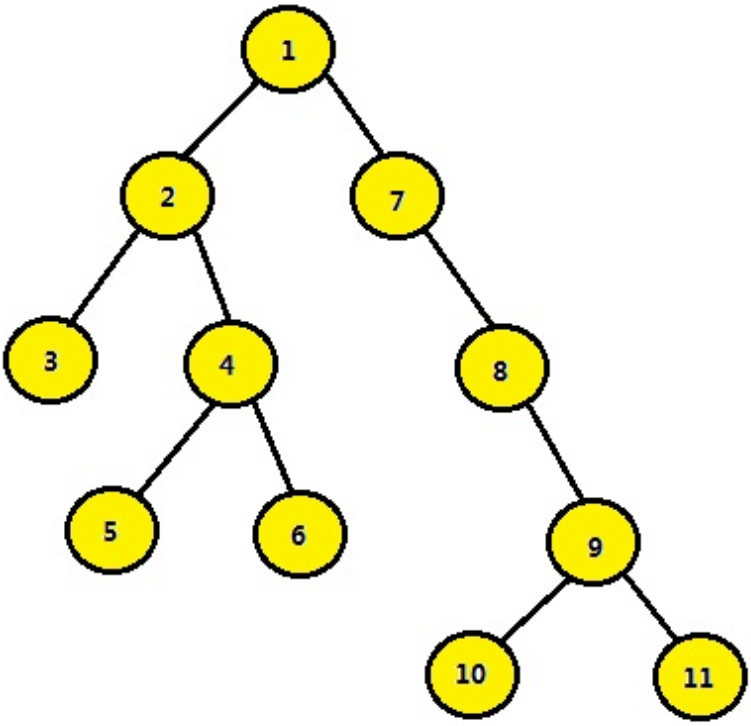
比如：

- 1，2，5，6为一个集合,祖先为1，集合中点和10的LCA为1
- 3，7为一个集合，祖先为3，集合中点和10的LCA为3
- 8，9，11为一个集合，祖先为8，集合中点和10的LCA为8
- 10，12为一个集合，祖先为10，集合中点和10的LCA为10

得出的结论便是:LCA(u,v)便是根至u的路径上到节点v最近的点。

2.2 、Tarjan算法如何而来

但关键是 Tarjan算法是怎么想出来的呢？再给定下图，你是否能看出来：分别从结点1的左右子树当中，任取一个结点，设为u 、v，这两个任意结点u 、v的最近公共祖先都为1。



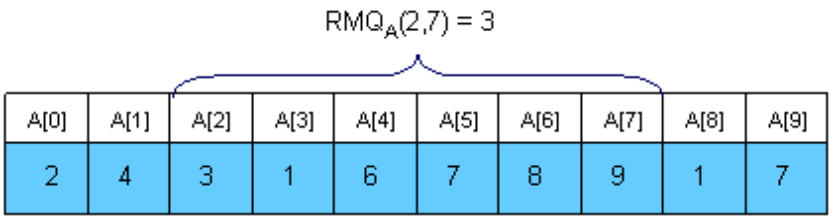
于此，我们可以得知：若两个结点u 、v分别分布于某节点t 的左右子树，那么此节点 t即为u和v的最近公共祖先。更进一步，考虑到一个节点自己就是LCA的情况，得知：

- 若某结点t 是两结点u 、v的祖先之一，且这两结点并不分布于该结点t 的一棵子树中，而是分别在结点t 的左子树、右子树中，那么该结点t 即为两结点u 、v的最近公共祖先。

这个定理就是Tarjan算法的基础。

一如上文1.1节我们得到的结论：“如果当前结点t 满足 u 。我们将用RMQA(i, j) 来表示数组A 中索引i 和 j 之间最小值的位置。u和v的离树T根结点最远的公共祖先用LCA T(u, v)表示。

如下图所示，RMQA(2,7)则表示求数组A中从A[2]~A[7]这段区间中的最小值：



很显然，从上图中，我们可以看出最小值是A[3] = 1，所以也就不难得出最小值的索引值RMQA(2,7) = 3。

3.2、如何解决RMQ问题

3.2.1 Trivial algorithms for RMQ

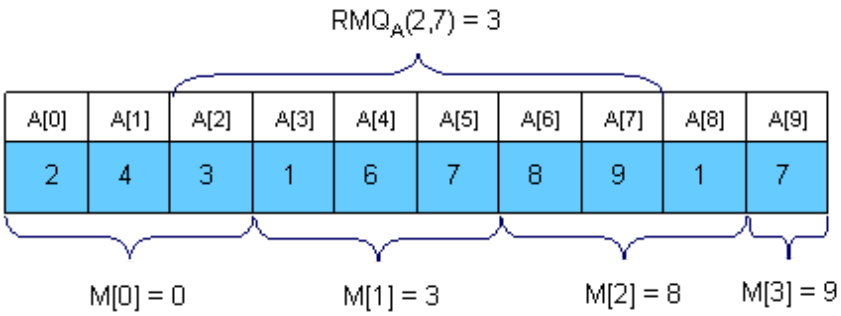
下面，我们对对每一对索引(i, j)，将数组中索引i 和 j 之间最小值的位置RMQA(i, j) 存储在M[0, N-1][0, N-1]表中。RMQA(i, j) 有不同种计算方法，你会看到，随着计算方法的不同，它的时空复杂度也不同：

- 普通的计算将得到一个 复杂度的算法。尽管如此，通过使用一个简单的动态规划方法，我们可以将复杂度降低到。如何做到的呢？方法如下代码所示：

```
//copyright@
//modified by July 2014
void process1(int M[MAXN][MAXN], int A[MAXN], int N)
{
    int i, j;
    for (i =0; i < N; i++)
        M[i][i] = i;
```

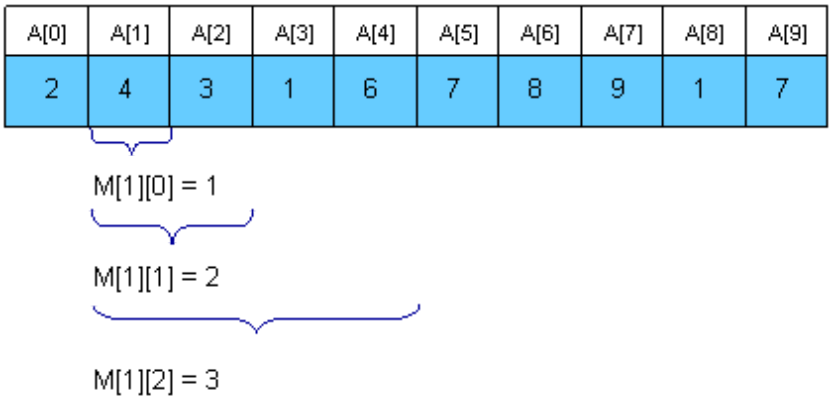
```
    for (i = 0; i < N; i++)
        for (j = i + 1; j < N; j++)
            //若前者小于后者,则把后者的索引值付给M[i][j]
            if (A[M[i][j - 1]] < A[j])
                M[i][j] = M[i][j - 1];
            //否则前者的索引值付给M[i][j]
            else
                M[i][j] = j;
}
```

- 一个比较有趣的点子是把向量分割成sqrt(N)大小的段。我们将在M[0,sqrt(N)-1]为每一个段保存最小值的位置。如此，M可以很容易的在O(N)时间内预处理。



- 一个更好的方法预处理RMQ 是对2^k 的长度的子数组进行动态规划。我们将使用数组M[0, N-1][0, logN]进行保存，其中M[i][j] 是以i 开始，长度为 2^j 的子数组的最小值的索引。这就引出了咱们接下来要介绍的Sparse Table (ST) algorithm。

3.2.2 Sparse Table (ST) algorithm



在上图中，我们可以看出：* 在A[1]这个长度为2^0的区间内，最小值即为A[1] = 4，故最小值的索引M[1][0]为1； * 在A[1]、A[2]这个长度为2^1的区间内，最小值为A[2] = 3，故最小值的索引为M[1][1] = 2；* 在A[1]、A[2]、A[3]、A[4]这个长度为2^2的区间内，最小值为A[3] = 1，故最小值的索引M[1][2] = 3。

为了计算M[i][j]我们必须找到前半段区间和后半段区间的最小值。很明显小的片段有着2^(j-1)长度，因此递归如下

$$M[i][j] = \begin{cases} M[i][j-1], & A[M[i][j-1]] <= A[M[i+2^{j-1}-1][j-1]] \\ M[i+2^{j-1}-1][j-1], & otherwise \end{cases}$$

根据上述公式，可以写出这个预处理的递归代码，如下：

```
void process2(int M[MAXN][LOGMAXN], int A[MAXN], int N)
{
    int i, j;
    //initialize M for the intervals with length 1

    for (i = 0; i < N; i++)
        M[i][0] = i;

    //compute values from smaller to bigger intervals
    for (j = 1; 1 << j <= N; j++)
        for (i = 0; i + (1 << j) - 1 < N; i++)
            if (A[M[i][j - 1]] < A[M[i + (1 << (j - 1))][j - 1]])
                M[i][j] = M[i][j - 1];
            else
```

```
                M[i][j] = M[i + (1 << (j - 1))][j - 1];
}
```

经过这个O(N logN)时间复杂度的预处理之后，让我们看看怎样使用它们去计算 RMQA(i, j)。思路是选择两个能够完全覆盖区间[i..j]的块并且找到它们之间的最小值。设k = [log(j - i + 1)]。

为了计算 RMQA(i, j)，我们可以使用下面的公式：

$$RMQ_A(i,j) = \begin{cases} M[i][k], & A[M[i][k]] <= A[M[j-2^k+1][k]] \\ M[j-2^k+1][k], & otherwise \end{cases}$$

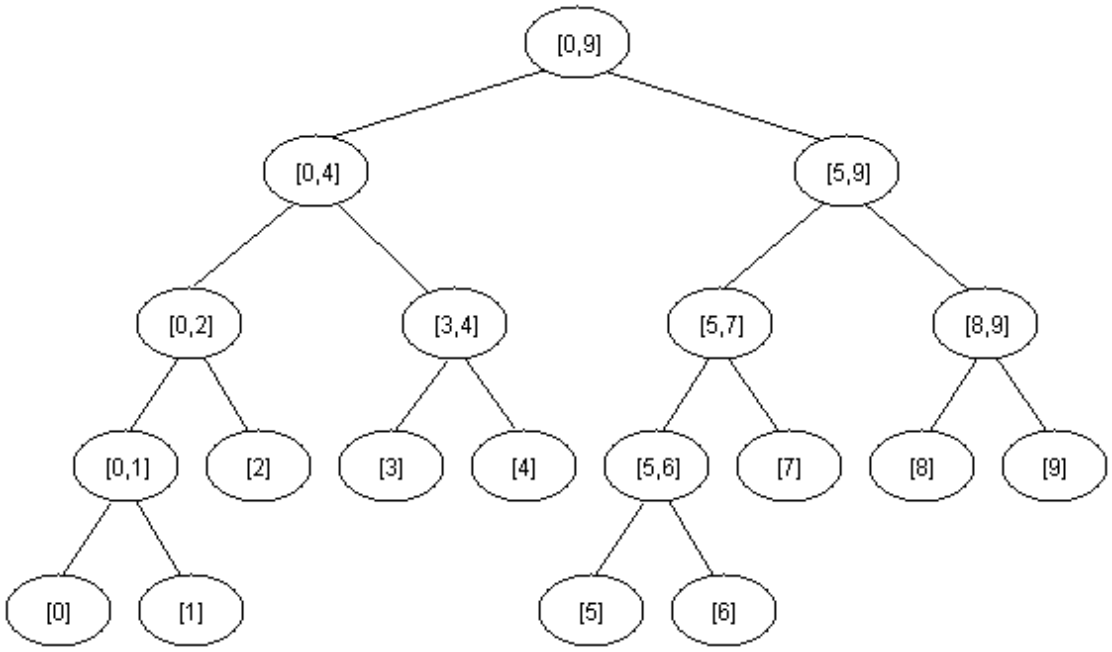
故，综合来看，咱们预处理的时间复杂度从O(N3)降低到了O(N logN)，查询的时间复杂度为O(1)，所以最终的整体复杂度为：。

3.2.3、线段树Segment trees

解决RMQ问题也可以用所谓的线段树。线段树是一个类似堆的数据结构，可以在基于区间数组上用对数时间进行更新和查询操作。我们用下面递归方式来定义线段树的[i, j]区间：

- 第一个结点将保存区间[i, j]区间的信息
- 如果i<j 左右的孩子结点将保存区间[i, (i+j)/2]和[(i+j)/2+1, j] 的信息

注意具有N个区间元素的线段树的高度为[logN] + 1。下面是区间[0,9]的线段树：



线段树和堆具有相同的结构，因此我们定义x是一个非叶结点，那么左孩子结点为2*x*,*而右孩子结点为*2x+1。想要使用线段树解决RMQ问题，我们则要要使用数组 M[1, 2 * 2[logN] + 1]，这里M[i]保存结点i区间最小值的位置。初始时M的所有元素为-1。树应当用下面的函数进行初始化(b和e是当前区间的范围)：

```
void initialize(int node, int b, int e, int M[MAXIND], int A[MAXN], int N)
{
    if (b == e)
        M[node] = b;
    else
    {
        //compute the values in the left and right subtrees
        initialize(2 * node, b, (b + e) / 2, M, A, N);
        initialize(2 * node + 1, (b + e) / 2 + 1, e, M, A, N);

        //search for the minimum value in the first and
        //second half of the interval
        if (A[M[2 * node]] <= A[M[2 * node + 1]])
            M[node] = M[2 * node];
        else
            M[node] = M[2 * node + 1];
    }
}
```

上面的函数映射出了这棵树建造的方式。当计算一些区间的最小值位置时，我们应当首先查看子结点的值。调用函数的时候使用 node = 1, b = 0 和e = N-1。

现在我们可以开始进行查询了。如果我们想要查找区间[i, j]中的最小值的位置时，我们可以使用下一个简单的函数：

```
int query(int node, int b, int e, int M[MAXIND], int A[MAXN], int i, int j)
{
    int p1, p2;
    //if the current interval doesn't intersect
    //the query interval return -1
    if (i > e || j < b)
        return -1;

    //if the current interval is included in
    //the query interval return M[node]
    if (b >= i && e <= j)
        return M[node];

    //compute the minimum position in the
    //left and right part of the interval
    p1 = query(2 * node, b, (b + e) / 2, M, A, i, j);
    p2 = query(2 * node + 1, (b + e) / 2 + 1, e, M, A, i, j);

    //return the position where the overall
    //minimum is
    if (p1 == -1)
        return M[node] = p2;
    if (p2 == -1)
        return M[node] = p1;
    if (A[p1] <= A[p2])
        return M[node] = p1;
    return M[node] = p2;
}
```

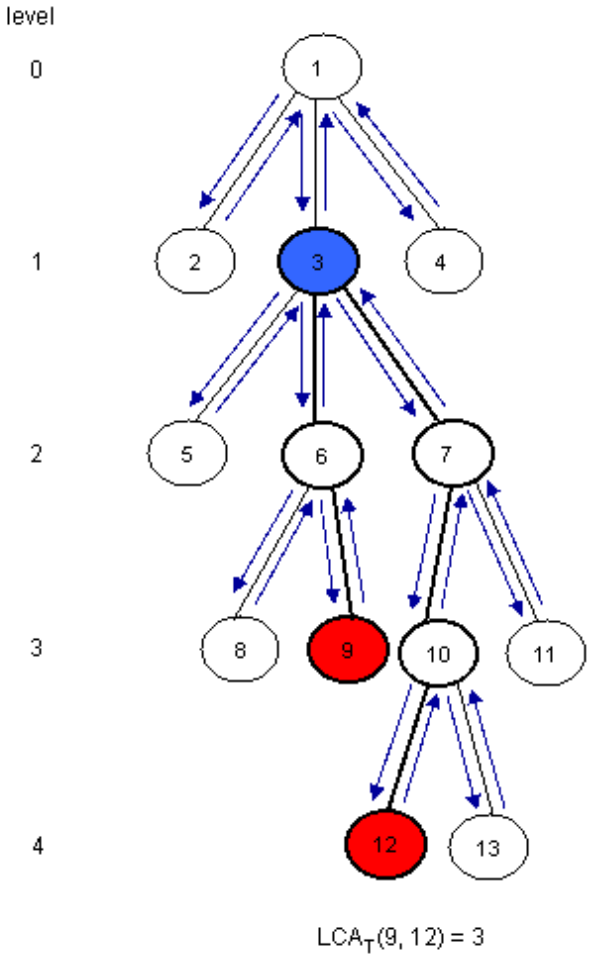
你应该使用node = 1, b = 0和e = N - 1来调用这个函数，因为分配给第一个结点的区间是[0, N-1]。

可以很容易的看出任何查询都可以在O(log N)内完成。注意当我们碰到完整的in/out区间时我们停止了，因此数中的路径最多分裂一次。用线段树我们获得了的算法。

线段树非常强大，不仅仅是因为它能够用在RMQ上，还因为它是一个非常灵活的数据结构，它能够解决动态版本的RMQ问题和大量的区间搜索问题。

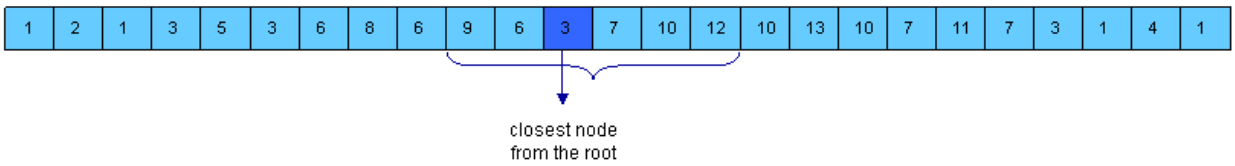
3.3 LCA与RMQ的关联性

现在，让我们看看怎样用RMQ来计算LCA查询。事实上，我们可以在线性时间里将LCA问题规约到RMQ问题，因此每一个解决RMQ的问题都可以解决LCA问题。让我们通过例子来说明怎么规约的：



$LCA_T(9, 12) = 3$

Euler Tour:

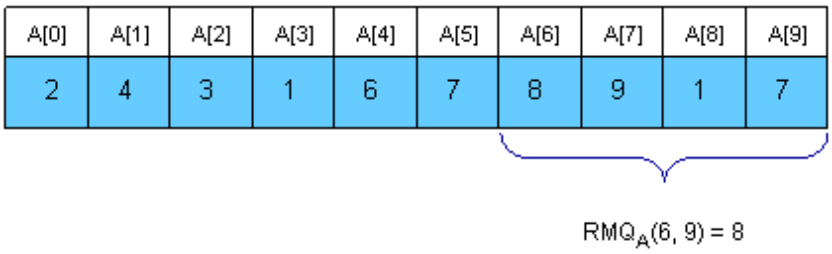


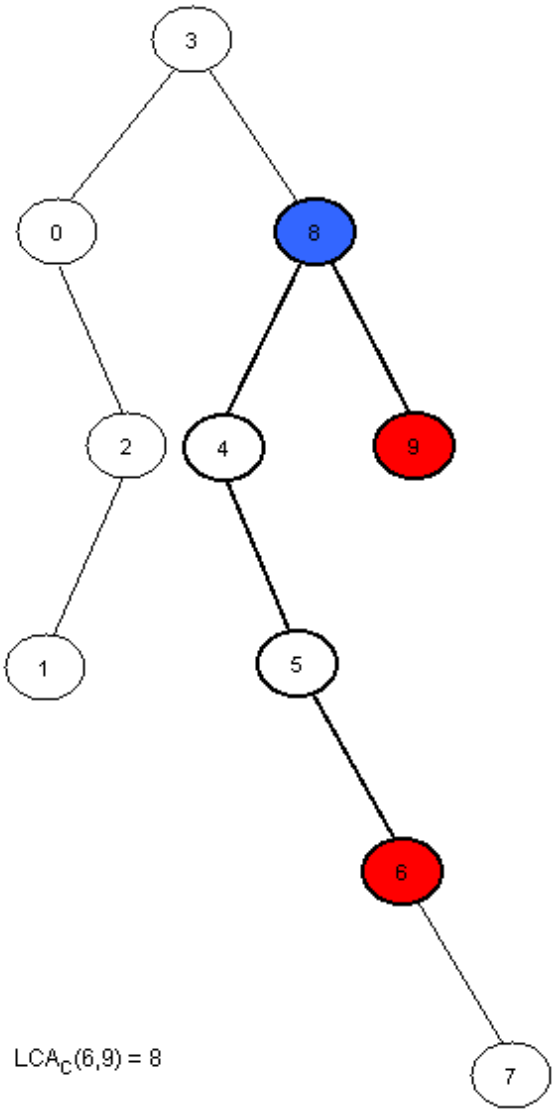
注意LCAT(u, v)是在对T进行dfs过程当中在访问u和v之间离根结点最近的点。因此我们可以考虑树的欧拉环游过程u和v之间所有的结点，并找到它们之间处于最低层的结点。为了达到这个目的，我们可以建立三个数组：

- E[1, 2*N-1] - 对T进行欧拉环游过程中所有访问到的结点;E[i]是在环游过程中第i个访问的结点
- L[1,2*N-1] - 欧拉环游中访问到的结点所处的层数;L[i]是E[i]所在的层数
- H[1, N] - H[i] 是E中结点i第一次出现的下标(任何出现i的地方都行，当然选第一个不会错)

假定H[u]0时，这个笛卡尔树的左子结点是A[0,i-1]构成的笛卡尔树，其他情况没有左子结点。右结点类似的用A[i+1,N-1]定义。注意对于具有相同元素的数组A，笛卡尔树并不唯一。在本文中，将会使用第一次出现的最小值，因此笛卡尔树看作唯一。可以很容易的看到RMQA(i, j) = LCAC(i, j)。

下面是一个例子：





现在我们需要做的仅仅是用线性时间计算C(A)。这个可以使用栈来实现。

- 初始栈为空。
- 然后我们在栈中插入A的元素。
- 在第i步,A[i]将会紧挨着栈中比A[i]小或者相等的元素插入,并且所有较大的元素将会被移除。
- 在插入结束之前栈中A[i]位置前的元素将成为i的左儿子,A[i]将会成为它之后一个较小元素的右儿子。

在每一步中,栈中的第一个元素总是笛卡尔树的根。

如果使用栈来保存元素的索引而不是值，我们可以很轻松的建立树。由于A中的每个元素最多被增加一次和最多被移除一次，所以建树的时间复杂度为O(N)。最终查询的时间复杂度为O(1)，故综上可得，咱们整个问题的最终时间复杂度为：。

现在，对于询问 RMQA(i, j) 我们有两种情况:

- i和j在同一个块中,因此我们使用在P和T中计算的值
- i和j在不同的块中，因此我们计算三个值:从i到i所在块的末尾的P和T中的最小值，所有i和j中块中的通过与处理得到的最小值以及从j所在块i和j在同一个块中,因此我们使用在P和T中计算的值j的P和T的最小值；最后我们我们只要计算三个值中最小值的位置即可。

RMQ和LCA是密切相关的问题，因为它们之间可以相互规约。有许多算法可以用来解决它们，并且他们适应于一类问题。

更多细节请看原文。

其余解法

除此之外，还有倍增法、重链剖分算法和后序遍历也可以解决该问题。其中，倍增思路相当于层序遍历，逐层或几层跳跃查，查询时间复杂度为O(log n)，空间复杂度为nlogn，对于每个节点先存储向上1层2层4层的节点，每个点有depth信息。

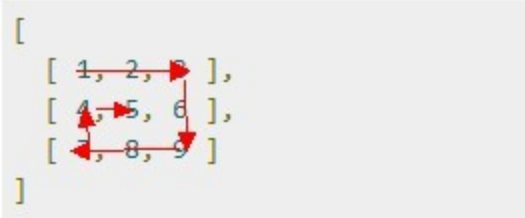
第四十章、螺旋矩阵

Given a matrix of m x n elements (m rows, n columns), return all elements of the matrix in spiral order。一句话，即为螺旋矩阵问题。

举个例子，给定如下的一个矩阵:

```
[
     [1,2,3],
     [4,5,6],
     [7,8,9]
]
```

你应该返回:[1,2,3,6,9,8,7,4,5]。如下图所示，遍历顺序为螺旋状:



以下是一份参考代码:

```
//代码来源:http://discuss.leetcode.com/questions/29/spiral-matrix
class Solution {
public:
    vector<int> spiralOrder(vector<vector<int> >& matrix) {
        vector<int> result;
        if (matrix.empty()) return result;
        ssize_t beginX = 0, endX = matrix[0].size() - 1;
        ssize_t beginY = 0, endY = matrix.size() - 1;

        while (true) {
            // From left to right
            for (ssize_t i = beginX; i <= endX; ++i)
                result.push_back(matrix[beginY][i]);
            if (++beginY > endY) break;
```

```
            // From top down
            for (ssize_t i = beginY; i <= endY; ++i)
                result.push_back(matrix[i][endX]);
            if (beginX > --endX) break;

            // From right to left
            for (ssize_t i = endX; i >= beginX; --i)
                result.push_back(matrix[endY][i]);
            if (beginY > --endY) break;

            // From bottom up
            for (ssize_t i = endY; i >= beginY; --i)
                result.push_back(matrix[i][beginX]);
            if (++beginX > endX) break;
        }
        return result;
    }
};
```

第五章导读

学习一个算法，先知其解决什么，后知其解决策略，最后了解彼此之间的联系。如图算法中，- ①广搜：一层一层往外遍历，寻找最短路径，策略：队列，- ②最小生成树：最小代价连接所有点，策略：贪心（Prim：贪心+权重队列），- ③Dijkstra：寻找单源最短路径，策略：贪心+非负权重队列，- ④Floyd：多结点对的最短路径，策略：动态规划。

而贪心和动态规划之间的联系是，贪心：最优子结构+局部最优，动态规划：最优独立重叠子结构+全局最优。此外，你还可以放空自己，不知道任何算法，去尝试解决一个个问题，后发现，原来解决某某问题的方法就是某某算法。

第十一章:最长公共子序列(LCS)问题

问题描述

什么是最长公共子序列呢?好比一个数列 S,如果分别是两个或多个已知数列的子序列,且是所有符合此条件序列中最长的,则S 称为已知序列的最长公共子序列。

举个例子,如:有两条随机序列,如 1 3 4 5 5 ,and 2 4 5 5 7 6,则它们的最长公共子序列便是:4 5 5。

分析与解法

注意最长公共子串(Longest CommonSubstring)和最长公共子序列(LongestCommon Subsequence, LCS)的区别:子串(Substring)是串的一个连续的部分,子序列(Subsequence)则是从不改变序列的顺序,而从序列中去掉任意的元素而获得的新序列;更简略地说,前者(子串)的字符的位置必须连续,后者(子序列LCS)则不必。比如字符串acdfg同akdfc的最长公共子串为df,而他们的最长公共子序列是adf。LCS可以使用动态规划法解决。下文具体描述。

解法一

- 穷举法
  解最长公共子序列问题时最容易想到的算法是穷举搜索法,即对X的每一个子序列,检查它是否也是Y的子序列,从而确定它是否为X和Y的公共子序列,并且在检查过程中选出最长的公共子序列。X和Y的所有子序列都检查过后即可求出X和Y的最长公共子序列。X的一个子序列相应于下标序列{1, 2, …, m}的一个子序列,因此,X共有2m个不同子序列(Y亦如此,如为2^n),从而穷举搜索法需要指数时间($2^m * 2^n$)。

解法二

- 动态规划算法

事实上，最长公共子序列问题也有最优子结构性质。

记：
Xi=<x1，…，xi>即X序列的前i个字符 (1≤i≤m)（前缀）

Yj=<y1，…，yj>即Y序列的前j个字符 (1≤j≤n)（前缀）

假定Z=<z1，…，zk>∈LCS(X , Y)。

- 若**xm=yn**（最后一个字符相同），则不难用反证法证明：该字符必是X与Y的任一最长公共子序列Z（设长度为k）的最后一个字符，即有zk = xm = yn 且显然有Zk-1∈LCS(Xm-1 , Yn-1)即Z的前缀**Zk-1是Xm-1与Yn-1的最长公共子序列。**此时，问题化归成求Xm-1与Yn-1的LCS（LCS(X , Y)的长度等于LCS(Xm-1 , Yn-1)的长度加1）。

- 若**xm≠yn**，则亦不难用反证法证明：要么Z∈LCS(Xm-1, Y)，要么Z∈LCS(X , Yn-1)。由于zk≠xm与zk≠yn其中至少有一个必成立，若zk≠xm则有Z∈LCS(Xm-1 , Y)，类似的，若zk≠yn 则有Z∈LCS(X , Yn-1)。此时，问题化归成求Xm-1与Y的LCS及X与Yn-1的LCS。LCS(X , Y)的长度为：max{LCS(Xm-1 , Y)的长度, LCS(X , Yn-1)的长度}。

由于上述当**xm≠yn**的情况中，求LCS(Xm-1 , Y)的长度与LCS(X , Yn-1)的长度，这两个问题不是相互独立的：两者都需要求LCS(Xm-1，Yn-1)的长度。另外两个序列的LCS中包含了两个序列的前缀的LCS，故问题具有最优子结构性质考虑用动态规划法。

也就是说，解决这个LCS问题，你要求三个方面的东西：1、LCS（Xm-1，Yn-1）+1；2、LCS（Xm-1，Y），LCS（X，Yn-1）；3、max{LCS（Xm-1，Y），LCS（X，Yn-1）}。

最长公共子序列的结构

最长公共子序列的结构有如下表示：

设序列X=< x1, x2, …, xm >和Y=< y1, y2, …, yn >的一个最长公共子序列Z=< z1, z2, …, zk >，则：

1. 若xm=yn，则zk=xm=yn且Zk-1是Xm-1和Yn-1的最长公共子序列；
2. 若xm≠yn且zk≠xm，则Z是Xm-1和Y的最长公共子序列；
3. 若xm≠yn且zk≠yn，则Z是X和Yn-1的最长公共子序列。
   其中Xm-1 = < x1, x2, …, xm-1 >，Yn-1 = < y1, y2, …, yn-1 >，Zk-1 = < z1, z2, …, zk-1 >。

子问题的递归结构

由最长公共子序列问题的最优子结构性质可知，要找出X=< x1, x2, …, xm >和Y=< y1, y2, …, yn >的最长公共子序列，可按以下方式递归地进行：当xm=yn时，找出Xm-1和Yn-1的最长公共子序列，然后在其尾部加上xm(=yn)即可得X和Y的一个最长公共子序列。当xm≠yn时，必须解两个子问题，即找出Xm-1和Y的一个最长公共子序列及X和Yn-1的一个最长公共子序列。这两个公共子序列中较长者即为X和Y的一个最长公共子序列。

由此递归结构容易看到最长公共子序列问题具有子问题重叠性质。例如，在计算X和Y的最长公共子序列时，可能要计算出X和Yn-1及Xm-1和Y的最长公共子序列。而这两个子问题都包含一个公共子问题，即计算Xm-1和Yn-1的最长公共子序列。

与矩阵连乘积最优计算次序问题类似，我们来建立子问题的最优值的递归关系。用c[i,j]记录序列Xi和Yj的最长公共子序列的长度。其中Xi=< x1, x2, …, xi >，Yj=< y1, y2, …, yj >。当i=0或j=0时，空序列是Xi和Yj的最长公共子序列，故c[i,j]=0。其他情况下，由定理可建立递归关系如下：

$$c[i,j]=\begin{cases}0 & \text{if } i=0 \text{ or } j=0,\\ c[i-1,j-1]+1 & \text{if } i,j>0 \text{ and } x_i=y_j,\\ \max(c[i,j-1],c[i-1,j]) & \text{if } i,j>0 \text{ and } x_i\neq y_j.\end{cases}$$

计算最优值

直接利用上节节末的递归式，我们将很容易就能写出一个计算c[i,j]的递归算法，但其计算时间是随输入长度指数增长的。由于在所考虑的子问题空间中，总共只有θ(m*n)个不同的子问题，因此，用动态规划算法自底向上地计算最优值能提高算法的效率。

计算最长公共子序列长度的动态规划算法LCS_LENGTH(X,Y)以序列X=< x1, x2, …, xm >和Y=< y1, y2, …, yn >作为输入。输出两个数组c[0..m ,0..n]和b[1..m ,1..n]。其中c[i,j]存储Xi与Yj的最长公共子序列的长度，b[i,j]记录指示c[i,j]的值是由哪一个子问题的解达到的，这在构造最长公共子序列时要用到。最后，X和Y的最长公共子序列的长度记录于c[m,n]中。

```
Procedure LCS_LENGTH(X,Y);
begin
  m:=length[X];
  n:=length[Y];
  for i:=1 to m do c[i,0]:=0;
  for j:=1 to n do c[0,j]:=0;
  for i:=1 to m do
    for j:=1 to n do
      if x[i]=y[j] then
        begin
          c[i,j]:=c[i-1,j-1]+1;
          b[i,j]:="↖";
        end
      else if c[i-1,j]≥c[i,j-1] then
        begin
          c[i,j]:=c[i-1,j];
          b[i,j]:="↑";
        end
      else
        begin
          c[i,j]:=c[i,j-1];
          b[i,j]:="←"
        end;
  return(c,b);
end;
```

由算法LCS_LENGTH计算得到的数组b可用于快速构造序列X=< x1, x2, …, xm >和Y=< y1, y2, …, yn >的最长公共子序列。首先从b[m,n]开始，沿

着其中的箭头所指的方向在数组b中搜索。

- 当b[i,j]中遇到“↖”时（*意味着xi=yi是LCS的一个元素*），表示Xi与Yj的最长公共子序列是由Xi-1与Yj-1的最长公共子序列在尾部加上xi得到的子序列；
- 当b[i,j]中遇到“↑”时，表示Xi与Yj的最长公共子序列和Xi-1与Yj的最长公共子序列相同；
- 当b[i,j]中遇到“←”时，表示Xi与Yj的最长公共子序列和Xi与Yj-1的最长公共子序列相同。

这种方法是按照反序来找LCS的每一个元素的。由于每个数组单元的计算耗费O(1)时间，算法LCS_LENGTH耗时O(mn)。

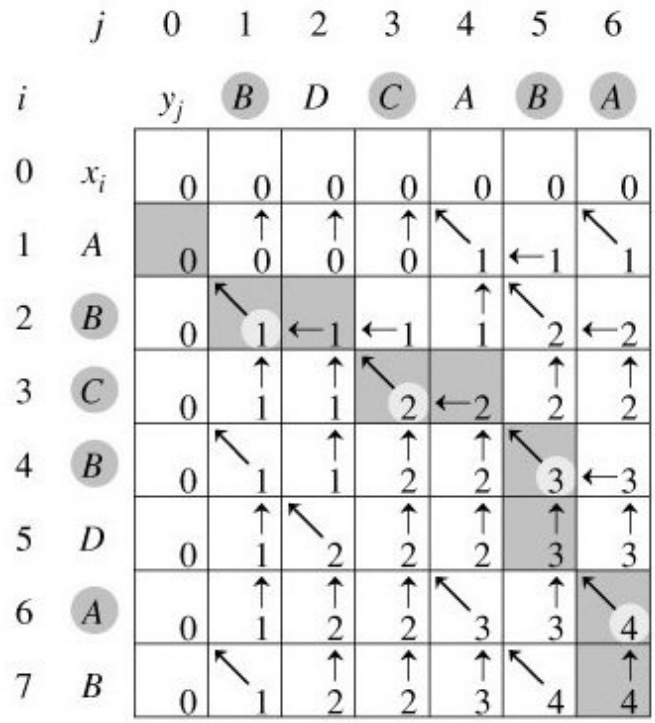
构造最长公共子序列

下面的算法LCS(b,X,i,j)实现根据b的内容打印出Xi与Yj的最长公共子序列。通过算法的调用LCS(b,X,length[X],length[Y])，便可打印出序列X和Y的最长公共子序列。

```
Procedure LCS(b,X,i,j);
begin
  if i=0 or j=0 then return;
  if b[i,j]="↖" then
    begin
      LCS(b,X,i-1,j-1);
      print(x[i]); {打印x[i]}
    end
  else if b[i,j]="↑" then LCS(b,X,i-1,j)
                     else LCS(b,X,i,j-1);
end;
```

在算法LCS中，每一次的递归调用使i或j减1，因此算法的计算时间为O(m+n)。

例如，设所给的两个序列为X=< A，B，C，B，D，A，B >和Y=< B，D，C，A，B，A >。由算法LCS_LENGTH和LCS计算出的结果如下图所示：



i	j	0	1	2	3	4	5	6
		y_j	B	D	C	A	B	A
0	x_i	0	0	0	0	0	0	0
1	A	0	↑ 0	↑ 0	↑ 0	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	↑ 1	↖ 2	← 2
3	C	0	↑ 1	↑ 1	↖ 2	← 2	↑ 2	↑ 2
4	B	0	↖ 1	↑ 1	↑ 2	↑ 2	↖ 3	← 3
5	D	0	↑ 1	↖ 2	↑ 2	↑ 2	↑ 3	↑ 3
6	A	0	↑ 1	↑ 2	↑ 2	↖ 3	↑ 3	↖ 4
7	B	0	↖ 1	↑ 2	↑ 2	↑ 3	↖ 4	↑ 4

- 我来说明下此图(参考算法导论)*。在序列X={A, B, C, B, D, A, B}和 Y={B, D, C, A, B, A}上, 由LCS_LENGTH计算出的表c和b。第i行和第j列中的方块包含了c[i, j]的值以及指向b[i, j]的箭头。在c[7,6]的项4, 表的右下角为X和Y的一个LCS < B, C, B, A >的长度。对于i, j>0, 项c[i, j]仅依赖于是否有xi=yi, 及项c[i-1, j]和c[i, j-1]的值, 这几个项都在c[i, j]之前计算。为了重构一个LCS的元素, 从右下角开始跟踪b[i, j]的箭头即可, 这条路径标示为阴影, 这条路径上的每一个“↖”对应于一个使xi=yi为一个LCS的成员的项(高亮标示)。

  所以根据上述图所示的结果, 程序将最终输出:“B C B A”。

算法的改进

对于一个具体问题，按照一般的算法设计策略设计出的算法，往往在算法的时间和空间需求上还可以改进。这种改进，通常是利用具体问题的一些特殊性。

例如, 在算法LCS_LENGTH和LCS中, 可进一步将数组b省去。事实上, 数组元素c[i,j]的值仅由c[i-1,j-1], c[i-1,j]和c[i,j-1]三个值之一确定, 而数组

元素b[i,j]也只是用来指示c[i,j]究竟由哪个值确定。因此，在算法LCS中，我们可以不借助于数组b而借助于数组c本身临时判断c[i,j]的值是由c[i-1,j-1], c[i-1,j]和c[i,j-1]中哪一个数值元素所确定，代价是O(1)时间。既然b对于算法LCS不是必要的，那么算法LCS_LENGTH便不必保存它。这一来，可节省θ(mn)的空间，而LCS_LENGTH和LCS所需要的时间分别仍然是O(mn)和O(m+n)。不过，由于数组c仍需要O(mn)的空间，因此这里所作的改进，只是在空间复杂性的常数因子上的改进。

另外，如果只需要计算最长公共子序列的长度，则算法的空间需求还可大大减少。事实上，在计算c[i,j]时，只用到数组c的第i行和第i-1行。因此，只要用2行的数组空间就可以计算出最长公共子序列的长度。更进一步的分析还可将空间需求减至min(m, n)。

编码实现LCS问题

动态规划的一个计算最长公共子序列的方法如下，以两个序列 X、Y 为例子：

设有二维数组 f[i][j] 表示 X 的 i 位和 Y 的 j 位之前的最长公共子序列的长度，则有：

f[1][1] = same(1,1)
f[i][j] = max{f[i − 1][j − 1] +same(i,j), f[i − 1][j] ,f[i][j − 1]}

其中，same(a,b)当 X 的第 a 位与 Y 的第 b 位完全相同时为“1”，否则为“0”。

此时，f[i][j]中最大的数便是 X 和 Y 的最长公共子序列的长度，依据该数组回溯，便可找出最长公共子序列。

该算法的空间、时间复杂度均为O(n2)，经过优化后，空间复杂度可为O(n)，时间复杂度为O(nlogn)。

以下是此算法的java代码：

```
import java.util.Random;

public class LCS{
    public static void main(String[] args){

        //设置字符串长度
        int substringLength1 = 20;
        int substringLength2 = 20;  //具体大小可自行设置

        // 随机生成字符串
        String x = GetRandomStrings(substringLength1);
        String y = GetRandomStrings(substringLength2);

        Long startTime = System.nanoTime();
        // 构造二维数组记录子问题x[i]和y[i]的LCS的长度
        int[][] opt = new int[substringLength1 + 1][substringLength2 + 1];

        // 动态规划计算所有子问题
        for (int i = substringLength1 - 1; i >= 0; i--){
            for (int j = substringLength2 - 1; j >= 0; j--){
                if (x.charAt(i) == y.charAt(j))
                                        opt[i][j]  =  opt[i  +  1][j  +  1]  +  1;
//参考上文我给的公式。
                else
                    opt[i][j] = Math.max(opt[i + 1][j], opt[i][j + 1]);         //参考
上文我给的公式。
            }
        }
```

理解上段,参考上文我给的公式:

根据上述结论,可得到以下公式,

如果我们记字符串Xi和Yj的LCS的长度为c[i,j],我们可以递归地求c[i,j]:

```
          /      0                              if i=0 or j=0
c[i,j]=           c[i-1,j-1]+1                   if i,j>0 and xi=xj
         /        max(c[i,j-1],c[i-1,j])          if i,j>0 and xi≠xj
```

```
        System.out.println("substring1:"+x);
        System.out.println("substring2:"+y);
        System.out.print("LCS:");
```

```
        int i = 0, j = 0;
        while (i < substringLength1 && j < substringLength2){
            if (x.charAt(i) == y.charAt(j)){
                System.out.print(x.charAt(i));
                i++;
                j++;
            } else if (opt[i + 1][j] >= opt[i][j + 1])
                i++;
            else
                j++;
        }
        Long endTime = System.nanoTime();
        System.out.println(" Totle time is " + (endTime - startTime) + " ns");
    }

    //取得定长随机字符串
    public static String GetRandomStrings(int length){
        StringBuffer buffer = new StringBuffer("abcdefghijklmnopqrstuvwxyz");
        StringBuffer sb = new StringBuffer();
        Random r = new Random();
        int range = buffer.length();
        for (int i = 0; i < length; i++){
            sb.append(buffer.charAt(r.nextInt(range)));
        }
        return sb.toString();
    }
}
```

改进的算法

下面咱们来了解一种不同于动态规划法的一种新的求解最长公共子序列问题的方法,该算法主要是把求解公共字符串问题转化为求解矩阵L(p,m)的问题,在利用定理求解矩阵的元素过程中

(1)while(i < k),L(k,i)=null,
(2)while(L(k,i)=k),L(k,i+1)=L(k,i+2)=…L(k,m)=k;

求出每列元素,一直到发现第p+1 行都为null 时退出循环，得出矩阵L(k,m)后,B[L(1,m-p+1)]B[L(2,m-p+2)]…B[L(p,m)]即为A 和B 的LCS,其中p 为LCS 的长度。

主要定义及定理

- 定义 1 子序列(Subsequence)：给定字符串A=A[1]A[2]…A[m]，(A[i]是A 的第i 个字母，A[i]∈字符集Σ，l<= i<m = A ，A 表示字符串A 的长度)，字符串B 是A 的子序列是指B=A[1 i]A[2 i]…A[k i],其中1 i < 2 i <…< k i 且k<=m.

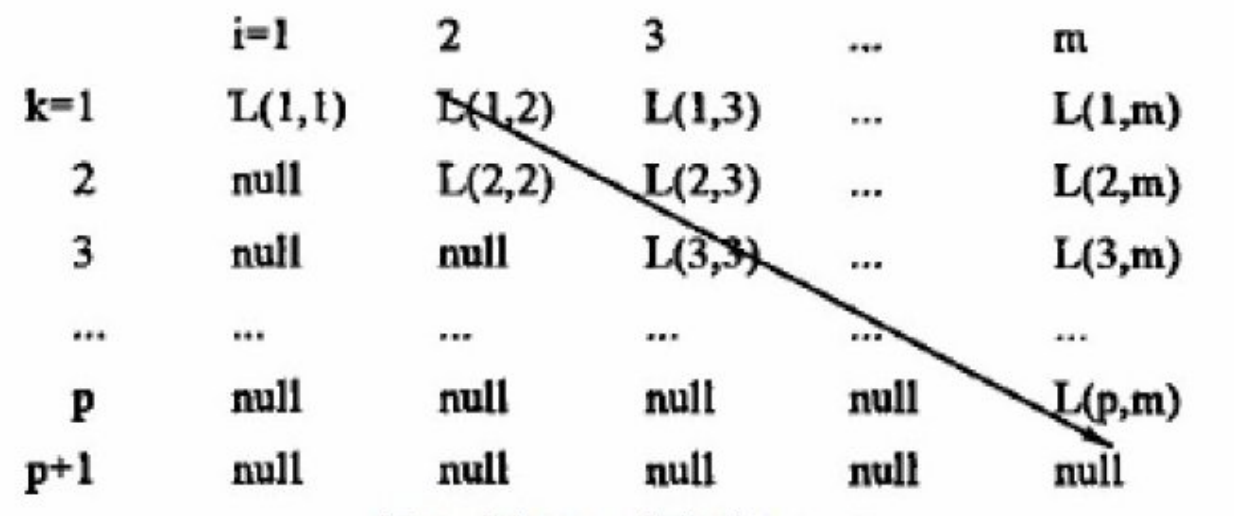
- 定义2 公共子序列(Common Subsequence)：给定字符串A 、B 、C，C 称为A 和B 的公共子序列是指C 既是A 的子序列，又是B 的子序列。

- 定义3 最长公共子序列(Longest Common Subsequence 简称LCS)：

  给定字符串A 、B 、C，C 称为A 和B 的最长公共子序列是指C 是A 和B 的公共子序列，且对于A 和B 的任意公共子序列D，都有D <= C 。给定字符串A 和B，A =m，B =n，不妨设m<=n，LCS 问题就是要求出A 和B 的LCS。

- 定义4 给定字符串A=A[1]A[2]…A[m]和字符串B=B[1]B[2]…[n]，A(1:i)表示A 的连续子序列A[1]A[2]…A[i]，同样B(1:j)表示B 的连续子序列B[1]B[2]…[j] 。Li(k)表示所有与字符串A(1:i) 有长度为k 的LCS 的字符串B(l:j) 中j 的最小值。用公式表示就是Li(k)=Minj(LCS(A(1:i)，B(l:j))=k) [3]。

定理1 ∀ i∈[1，m]，有Li(l) < Li(2) < Li(3) < … < Li(m) .
定理2 ∀i∈[l，m-1]，∀k∈[l，m]，有i 1 L + (k)<= i L (k).
定理3 ∀ i∈[l，m-1]，∀ k∈[l，m-l]，有i L (k)< i 1 L + (k+l).
以上三个定理都不考虑Li(k)无定义的情况。
定理4[3] i 1 L + (k)如果存在，那么它的取值必为: i 1 L + (k)=Min(j, i L (k))。这里j 是满足以下条件的最小整数:A[i+l]=B[j]且j> i L (k-1)。



	i=1	2	3	...	m
k=1	L(1,1)	L(1,2)	L(1,3)	...	L(1,m)
2	null	L(2,2)	L(2,3)	...	L(2,m)
3	null	null	L(3,3)	...	L(3,m)
...	...	...	...	...	...
p	null	null	null	null	L(p,m)
p+1	null	null	null	null	null

图 1 求解 LCS 的矩阵 L(p,m)

Fig. 1 L(p,m)of solving matrix LCS

矩阵中元素L(k，i)=Li(k)，这里(1 < i <= m，1 < k <= m)，null 表示L(k,i)不存在。当i < k 时，显然L(k，i)不存在。

设p=Maxk(L(k ， m) ≠ null) ， 可以证明L 矩阵中L(p,m) 所在的对角线,L(1,m-p+1),L(2,m-p+2)…L(p-1,m-1),L(p,m) 所对应的子序列B[L(1,m-p+1)]B[L(2,m-p+2)]…B[L(p,m)]即为A 和B 的LCS，p 为该LCS 的长度。这样，LCS 问题的求解就转化为对m m L × 矩阵的求解。

算法思想

根据定理,第一步求出第一行元素,L(1,1),L(1,2),…L(1,m),第二步求第二行,一直到发现第p+1 行都为null 为止。在计算过程中遇到i < k 时,L(k,i)=null, 及L(k,i)=k时,L(k,i+1)=L(k,i+2)=…L(k,m)=k。这样,计算每行的时间复杂度为O(n),则整个时间复杂度为O(pn)。在求L 矩阵的过程中不用存储整个矩阵,只需存储当前行和上一行即可。空间复杂度为O(m+n)。

下面给出一个例子来说明:给定字符串A 和B，A=acdabbc，B=cddbacaba，(m= A =7，n= B =9)。按照定理给出的递推公式，求出A 和B 的L 矩阵如图

2, 其中的$表示NULL。

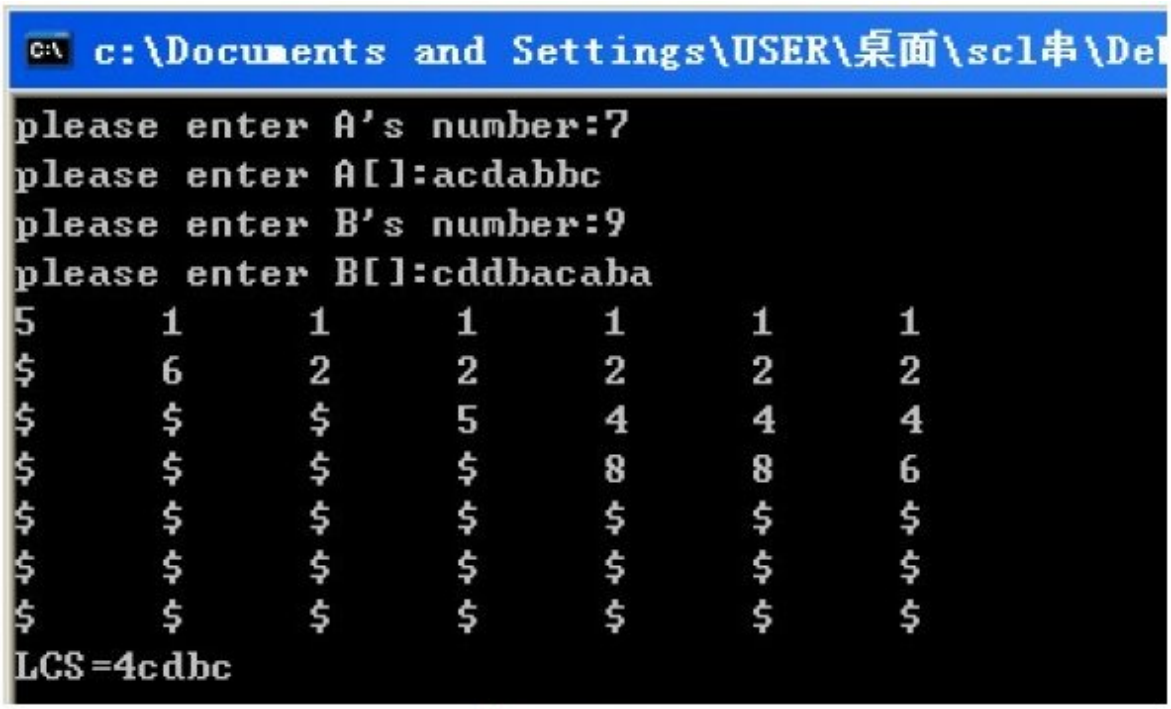


图 2　L(p.m)
Fig. 2　L(p.m)

则A 和B 的LCS 为B[1]B[2]B[4]B[6]=cdbc, LCS 的长度为4。

算法伪代码

算法 L(A,B,L)
输入 长度分别为m,n 的字符串A,B
输出 A,B 的最长公共子序列LCS

```
L(A,B,L){//字符串A,B,所求矩阵L
  for(k=1;k<=m;k++){ //m 为A 的长度
    for(i=1;i<=m;i++){
      if(i<k) L[k][i]=N;//i<k 时,L(k,i)=null,N 代表无穷大
      if(L[k][i]==k)//L(k,i)=k 时,L(k,i+1)=L(k,i+2)=…L(k,m)=k
      for(l=i+1;l<=m;l++)
       { L[k][l]=k;
         Break;}
      for(j=1;j<=n;j++){//定理4 的实现
       if(A[i+1]==B[j]&&j>L[k-1][i]){
```

```
        L[k][i+1]=(j<L[k][i]?j:L[k][i]);
        break;
       }
       if(L[k][i+1]==0)
        L[k][i]=N;
      }
      if(L[k][m]==N)
       {p=k-1;break;}
  }
  p=k-1;
}
```

结语

本节主要描述区别于动态规划法的一种新的求解最长公共子序列问题的方法，在不影响精确度的前提下，提高序列匹配的速度，根据定理i 1 L + (k)=Min(j, i L (k))得出矩阵，在求解矩阵的过程中对最耗时的L(p,m)进行条件约束优化。我们在Intel(R) Core(TM)2 Quad 双核处理器、1G 内存，软件环境：windows xp 下试验结果证明，本文算法与其他经典的比对算法相比,不但能够取得准确的结果,而且速度有了较大的提高（本节参考了刘佳梅女士的论文）。

举一反三

1、最长递增子序列LIS（Longest Increasing Subsequence）

给定一个长度为N的数组，找出一个最长的单调自增子序列（不一定连续，但是顺序不能乱）。例如：给定一个长度为6的数组A{5，6，7，1，2，8}，则其最长的单调递增子序列为{5，6，7，8}，长度为4。

分析：其实此LIS问题可以转换成最长公子序列问题，为什么呢？

- 原数组为A {5，6，7，1，2，8}
- 排序后：A‘{1，2，5，6，7，8}

因为，原数组A的子序列顺序保持不变，而且排序后A‘本身就是递增的，这样，就保证了两序列的最长公共子序列的递增特性。如此，若想求数组A的最长递增子序列，其实就是求数组A与它的排序数组A‘的最长公共子序列。

此外，本题也可以使用动态规划来求解，读者可以继续思考。

最大连续乘积子串

题目描述：

给一个浮点数序列，取最大乘积连续子串的值，例如 -2.5，4，0，3，0.5，8，-1，则取出的最大乘积连续子串为3，0.5，8。也就是说，上述数组中，3 0.5 8这3个数的乘积3*0.5*8=12是最大的，而且是连续的。

提醒：此最大乘积连续子串与最大乘积子序列不同，请勿混淆，前者子串要求连续，后者子序列不要求连续。也就是说：最长公共子串（Longest CommonSubstring）和最长公共子序列（LongestCommon Subsequence，LCS）的区别：

- 子串（Substring）是串的一个连续的部分，
- 子序列（Subsequence ）则是从不改变序列的顺序，而从序列中去掉任意的元素而获得的新序列；

更简略地说，前者（子串）的字符的位置必须连续，后者（子序列LCS ）则不必。比如字符串“ acdfg ”同“ akdfc ”的最长公共子串为“ df ”，而它们的最长公共子序列LCS是“ adf ”，LCS可以使用动态规划法解决。

分析与解法

解法一

或许，读者初看此题，自然会想到最大乘积子序列问题类似于[最大子数组和问题]，可能立马会想到用最简单粗暴的方式：两个for循环直接轮询。

```
double max = 0;
double start = 0;
double end = 0;
for (int i = 0; i < num; i++)
{
```

```
    double x = arrs[i];
    for (int j = i + 1; j < num; j++)
    {
        x *= arrs[j];
        if (x > max)
        {
            max = x;
            start = arrs[i];
            end = arrs[j];
        }
    }
}
```

解法二

虽说类似于最大子数组和问题，但实际上具体处理起来诸多不同。为什么呢，因为乘积子序列中有正有负也还可能有0。我们可以把问题简化成这样：数组中找一个子序列，使得它的乘积最大；同时找一个子序列，使得它的乘积最小（负数的情况）。因为虽然我们只要一个最大积，但由于负数的存在，我们同时找这两个乘积做起来反而方便。也就是说，不但记录最大乘积，也要记录最小乘积。So, 我们让

- maxCurrent表示当前最大乘积的candidate,
- minCurrent反之,表示当前最小乘积的candidate,
- 而maxProduct则记录到目前为止所有最大乘积candidates的最大值。(以上用candidate这个词是因为只是可能成为新一轮的最大/最小乘积)

由于空集的乘积定义为1,在搜索数组前,maxCurrent,minCurrent,maxProduct都赋为1。假设在任何时刻你已经有了maxCurrent和minCurrent这两个最大/最小乘积的candidates,新读入数组的元素x(i)后,新的最大乘积candidate只可能是maxCurrent或者minCurrent与x(i)的乘积中的较大者,如果x(i) < 0导致maxCurrent < minCurrent,需要交换这两个candidates的值。

当任何时候maxCurrent<1,由于1(空集)是比maxCurrent更好的candidate,所以更新maxCurrent为1,类似的可以更新minCurrent。任何时候maxCurrent如果比最好的maxProduct大,更新maxProduct。

参考代码如下：

```
template <typename Comparable>
Comparable maxprod( const vector<Comparable>&v)
{
    int i;
    Comparable maxProduct = 1;
    Comparable minProduct = 1;
    Comparable maxCurrent = 1;
    Comparable minCurrent = 1;
    //Comparable t;

    for ( i = 0; i < v.size() ; i++)
    {
        maxCurrent *= v[i];
        minCurrent *= v[i];
        if (maxCurrent > maxProduct)
            maxProduct = maxCurrent;
        if (minCurrent > maxProduct)
            maxProduct = minCurrent;
        if (maxCurrent < minProduct)
            minProduct = maxCurrent;
        if (minCurrent < minProduct)
            minProduct = minCurrent;
        if (minCurrent > maxCurrent)
            swap(maxCurrent, minCurrent);
        if (maxCurrent < 1)
            maxCurrent = 1;
        //if(minCurrent>1)
        //    minCurrent =1;
    }
    return maxProduct;
}
```

解法三

本题除了上述类似最大子数组和的解法，也可以直接用动态规划求解（其实，上述的解法一本质上也是动态规划，只是解题所表现出来的具体形式与接下来的解法二不同罢了。这个不同就在于下面的解法二会写出动态规划问题中经典常见的DP方程，而解法一是直接求解）。具体解法如下：

假设数组为a[]，直接利用动归来求解，考虑到可能存在负数的情况，我们用Max来表示以a结尾的最大连续子串的乘积值，用Min表示以a结尾

的最小的子串的乘积值，那么状态转移方程为：

```
  Max=max{a, Max[i-1]*a, Min[i-1]*a};
  Min=min{a, Max[i-1]*a, Min[i-1]*a};
```

初始状态为Max[1]=Min[1]=a[1]。

C/C++代码一：

```
double func(double *a, const int n)
{
    double *maxA = new double[n];
    double *minA = new double[n];
    maxA[0] = minA[0] = a[0];
    double value = maxA[0];
    for (int i = 1 ; i < n ; ++i)
    {
        maxA[i] = max(max(a[i], maxA[i - 1] * a[i]), minA[i - 1] * a[i]);
        minA[i] = min(min(a[i], maxA[i - 1] * a[i]), minA[i - 1] * a[i]);
        value = max(value, maxA[i]);
    }
    return value;
}
```

C/C++代码二：

```
/*
 给定一个浮点数数组，有正有负数，0，正数组成,数组下标从1算起
 求最大连续子序列乘积,并输出这个序列,如果最大子序列乘积为负数,那么就输出-1
 用Max[i]表示以a[i]结尾乘积最大的连续子序列
 用Min[i]表示以a[i]结尾乘积最小的连续子序列   因为有复数，所以保存这个是必须的
*/
void longest_multiple(double *a, int n)
{
    double *Min = new double[n + 1]();
    double *Max = new double[n + 1]();
    double *p = new double[n + 1]();
    //初始化
    for (int i = 0; i <= n; i++)
    {
        p[i] = -1;
    }
    Min[1] = a[1];
    Max[1] = a[1];
    double max_val = Max[1];
```

```
    for (int i = 2; i <= n; i++)
    {
        Max[i] = max(Max[i - 1] * a[i], Min[i - 1] * a[i], a[i]);
        Min[i] = min(Max[i - 1] * a[i], Min[i - 1] * a[i], a[i]);
        if (max_val < Max[i])
            max_val = Max[i];
    }
    if (max_val < 0)
        printf("%d", -1);
    else
        printf("%d", max_val);
    //内存释放
    delete [] Max;
    delete [] Min;
}
```

举一反三

1、给定一个长度为N的整数数组，只允许用乘法，不能用除法，计算任意（N-1）个数的组合中乘积最大的一组，并写出算法的时间复杂度。

分析：我们可以把所有可能的（N-1）个数的组合找出来，分别计算它们的乘积，并比较大小。由于总共有N个（N-1）个数的组合，总的时间复杂度为O（N2），显然这不是最好的解法。

OK，以下解答来自编程之美

解法1

在计算机科学中，时间和空间往往是一对矛盾体，不过，这里有一个折中方法。可以通过“空间换时间”或“时间换空间”的策略来达到优化某一方面的目的。在这里，是否可以通过“空间换时间”来降低时间复杂度呢？

计算（N-1）个数的组合乘积，假设第 i 个（$0 \leqslant i \leqslant N-1$）元素被排除在乘积之外（如图 2-16 所示）。

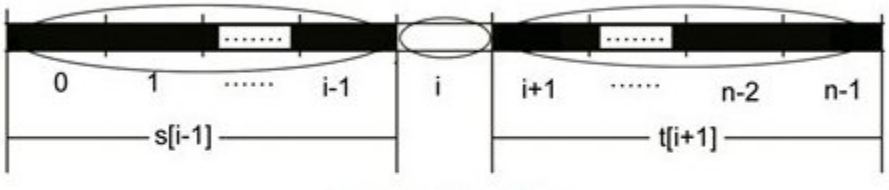


图 2-16　组合示意图

设 $array[]$为初始数组，$s[i]$表示数组前 i 个元素的乘积 $s[i]=\prod_{1}^{i} array[i-1]$，其中 $1 \leqslant i \leqslant N$，$s[0]=1$（边界条件），那么 $s[i]=s[i-1] \times array[i-1]$，其中 $i=1,2,\cdots,N-1,N$；

设 $t[i]$表示数组后（N-i）个元素的乘积 $t[i]=\prod_{i}^{n} array[i]$，其中 $1 \leqslant i \leqslant N$，$t[N+1]=1$（边界条件），那么 $t[i]=t[i+1] \times array[i]$，其中 $i=1,2,\cdots,N-1,N$；

那么设 $p[i]$为数组除第 i 个元素外，其他 N-1 个元素的乘积，即有：

$p[i]=s[i-1] \times t[i+1]$。

由于只需要从头至尾，和从尾至头扫描数组两次即可得到数组 $s[]$和 $t[]$，进而线性时间可以得到 $p[]$。所以，很容易就可以得到 $p[]$的最大值（只需遍历 $p[]$一次）。总的时间复杂度等于计算数组 $s[]$、$t[]$、$p[]$的时间复杂度加上查找 $p[]$最大值的时间复杂度等于 $O(N)$。

解法2、

此外，还可以通过分析，进一步减少解答问题的计算量。假设N个整数的乘积为P，针对P的正负性进行如下分析（其中，AN-1表示N-1个数的组合，PN-1表示N-1个数的组合的乘积）。

1. P为0

那么，数组中至少包含有一个0。假设除去一个0之外，其他N-1个数的乘积为Q，根据Q的正负性进行讨论：

Q为0
说明数组中至少有两个0，那么N-1个数的乘积只能为0，返回0；
Q为正数
返回Q，因为如果以0替换此时AN-1中的任一个数，所得到的PN-1为0，

必然小于Q；
Q为负数
如果以0替换此时AN-1中的任一个数，所得到的PN-1为0，大于Q，乘积最大值为0。

2. P为负数

根据“负负得正”的乘法性质，自然想到从N个整数中去掉一个负数，使得PN-1为一个正数。而要使这个正数最大，这个被去掉的负数的绝对值必须是数组中最小的。我们只需要扫描一遍数组，把绝对值最小的负数给去掉就可以了。

3. P为正数

类似地，如果数组中存在正数值，那么应该去掉最小的正数值，否则去掉绝对值最大的负数值。
上面的解法采用了直接求N个整数的乘积P，进而判断P的正负性的办法，但是直接求乘积在编译环境下往往会有溢出的危险（这也就是本题要求不使用除法的潜在用意），事实上可做一个小的转变，不需要直接求乘积，而是求出数组中正数（+）、负数（-）和0的个数，从而判断P的正负性，其余部分与以上面的解法相同。

在时间复杂度方面，由于只需要遍历数组一次，在遍历数组的同时就可得到数组中正数（+）、负数（-）和0的个数，以及数组中绝对值最小的正数和负数，时间复杂度为O（N）。

第二十九章:字符串编辑距离

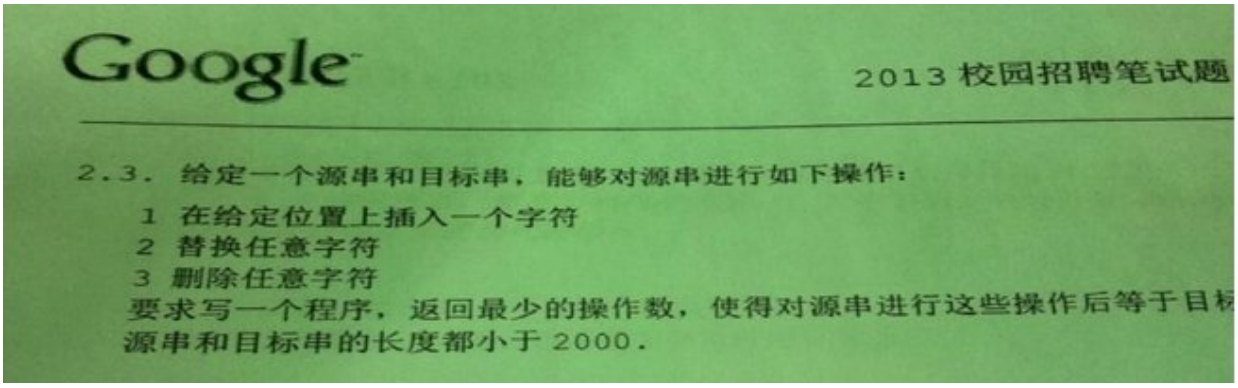
前言

上一章和这一章的问题皆是各大IT公司最喜欢出的笔试面试题之一,比如说前者是小米2013年校招笔试原题,而后者则更是反复出现,如去年9月26日百度一二面试题,10月9日腾讯面试题第1小题,10月13日百度2013校招北京站笔试题第二 大道题第3小题,及去年10月15日2013年Google校招笔试最后一道大题皆是考察的这个字符串编辑距离问题。

题目描述: 给定一个源串和目标串，能够对源串进行如下操作:
1. 在给定位置上插入一个字符
2. 替换任意字符
3. 删除任意字符

写一个程序,返回最小操作数,使得对源串进行这些操作后等于目标串,源串和目标串的长度都小于2000。

提醒:上文前言中已经说过了,此题反复出现,最近考的最多的是百度和Google的笔试面试经常考察。下图则是2013年Google的校招试题原景重现:



解法一、此题跟上面的最大连续乘积子串类似,常见的思路是动态规划,下面是简单的DP状态方程:

```
// 动态规划:

// f[i,j]表示s[0...i]与t[0...j]的最小编辑距离。
f[i, j] = min { f[i - 1, j] + 1,  f[i, j - 1] + 1,  f[i - 1, j - 1] + (s[i] == t[j] ? 0 : 1) }

// 分别表示:添加1个,删除1个,替换1个(相同就不用替换)。
```

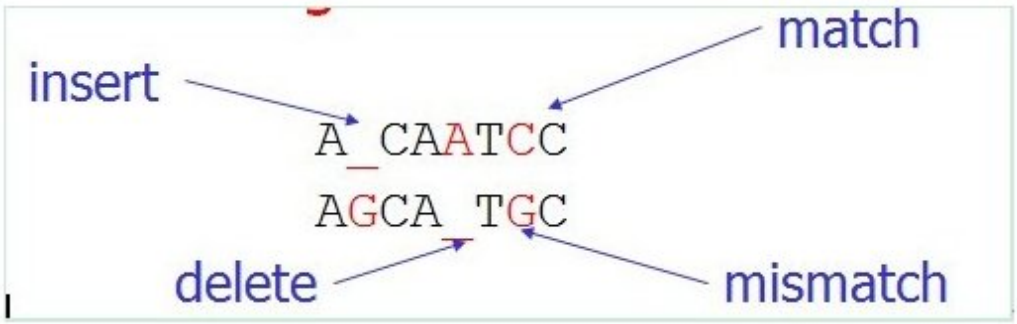
解法二、本解法来自[为学论坛](#)。

编辑距离的定义和计算方法如下：

Given two strings A and B, edit A to B with the minimum number of edit operations:

a. .Replace a letter with another letter
b. .Insert a letter
c. .Delete a letter

e.g.
A = interestingly _i**nterestingly**
B = bioinformatics bioinformatics
1011011011001111
Edit distance = 11
Instead of minimizing the number of edge operations, we can associate a cost function to the operations and minimize the total cost. Such cost is called edit distance. Instead of using string edit, in computational biology, people like to use string alignment.We use similarity function, instead of cost function, to evaluate the goodness of the alignment.
e.g. of similarity function: match – 2, mismatch, insert, delete – -1.
Consider two strings ACAATCC and AGCATGC.
One of their alignment is



In the above alignment, space (‘_’) is introduced to both strings. There are 5 matches, 1 mismatch, 1 insert, and 1 delete.The alignment has similarity score 7.

A_CAATCC
AGCA_TGC

Note that the above alignment has the maximum score.Such alignment is called optimal alignment.String alignment problem tries to find the alignment with the maximum similarity score!String alignment problem is also called global alignment problem.

Needleman-Wunsch algorithm Consider two strings S[1..n] and T[1..m].Define V(i, j) be the score of the optimal alignment between S[1..i] and T[1..j].

Basis:

V(0, 0) = 0

V(0, j) = V(0, j-1) + d(_, T[j]) : Insert j times

V(i, 0) = V(i-1, 0) + d(S,_) : Delete i times

that is:

$$V(i,j)=\max\begin{cases} V(i-1,j-1)+\delta(S[i],T[j]) & \text{Match/mis} \\ V(i-1,j)+\delta(S[i],_) & \text{Dele} \\ V(i,j-1)+\delta(_,T[j]) & \text{Ins} \end{cases}$$

Example :

	_	A	G	C	A	T	G	C
_	0	-1	-2	-3	-4	-5	-6	-7
A	-1	2	1	0	-1	-2	-3	-4
C	-2	1	1	3	2	1	0	-1
A	-3	0	0	2	5	4	3	2
A	-4	-1	-1	1	4	4	3	2
T	-5	-2	-2	0	3	6	5	4
C	-6	-3	-3	0	2	5	5	7
C	-7	-4	-4	-1	1	4	4	7

下面是代码,测试数据比较少,若有问题请指正:

```
//copyright@ peng_weida
//实现代码如下:
//头文件StrEditDistance.h
#pragma once
#include <string>
class CStrEditDistance
{
public:
    CStrEditDistance(std::string& vStrRow, std::string& vStrColumn);
    ~CStrEditDistance(void);
    int  getScore()
    {
        return m_Score;
    }
    int  getEditDis()
    {
        return m_EditDis;
    }
```

```
    void setEditDis(int vDis)
    {
        m_EditDis = vDis;
    }
    void setScore(int vScore)
    {
        m_Score = vScore;
    }
private:
    void process(const std::string& vStrRow, const std::string& vStrColumn);
    int getMaxValue(int a, int b, int c)
    {
        if (a < b)
        {
            if (b < c) return c;
            return b;
        }
        else
        {
            if (b > c) return a;
            return a < c ? c : a;
        }
    }
private:
    int   m_EditDis;
    int   m_Score;
};
//源文件StrEditDistance.cpp
#include "StrEditDistance.h"
#include <iostream>
#include <iomanip>
#define MATCH        2
#define MISS_MATCH   -1
#define INSERT       -1
#define DELETE       -1
CStrEditDistance::CStrEditDistance(std::string& vStrRow, std::string& vStrColumn)
{
    process(vStrRow, vStrColumn);
}
CStrEditDistance::~CStrEditDistance(void)
{
}
//FUNCTION:
void   CStrEditDistance::process(const   std::string&   vStrRow,   const   std::string&
vStrColumn)
{
    int editDis = 0;      //编辑距离
    int row = vStrColumn.length();
```

```
int column = vStrRow.length();
const int sizeR = row + 1;
const int sizeC = column + 1;

int **pScore = new int*[sizeR];  //二维指针
for (int i = 0; i <= row; i++)
    pScore[i] = new int[sizeC];

//初始化第一行和第一列
for (int c = 0; c <= column; c++)
    pScore[0][c] = 0 - c;
for (int r = 0; r <= row; r++)
    pScore[r][0] = 0 - r;

//从v(1,1)开始每列计算
for (int c = 1; c <= column; c++)
{
    for (int r = 1; r <= row; r++)
    {
        //计算v(i,j)
        int valueMatch;
        if (vStrColumn[r - 1] == vStrRow[c - 1])
            valueMatch = MATCH;
        else
            valueMatch = MISS_MATCH;
        int A = pScore[r - 1][c] + INSERT;
        int B = pScore[r][c - 1] + DELETE;
        int C = pScore[r - 1][c - 1] + valueMatch;
        pScore[r][c] = getMaxValue(A, B, C);
    }
}

//计算编辑距离
int r = row, c = column;
while (r > 0 && c > 0)
{
    if (pScore[r][c] + 1 == pScore[r - 1][c])
    {
        editDis++;
        r--;
        continue;
    }
    else if (pScore[r][c] + 1 == pScore[r][c - 1])
    {
        editDis++;
        c--;
        continue;
    }
```

```
        else if (pScore[r][c] + 1 == pScore[r - 1][c - 1])
        {
            editDis++;
            r--;
            c--;
            continue;
        }
        else
        {
            r--;
            c--;
        }
    }
    if (r > 0 && c == 0)  editDis += r;
    else if (c > 0 && r == 0) editDis += c;

    std::cout << std::endl;
    //----------------DEBUG-------------------//
    //打印数据
    for (int i = 0; i <= row; i++)
    {
        for (int j = 0; j <= column; j++)
            std::cout << std::setw(2) << pScore[j] << "  ";
        std::cout << std::endl;
    }
    std::cout << std::endl;

    //设置编辑距离和得分
    setEditDis(editDis);
    setScore(pScore[row][column]);

    for (int i = 0; i <= row; i++)    //释放内存
    {
        delete pScore;
        pScore = NULL;
    }
    delete[] pScore;
}
```

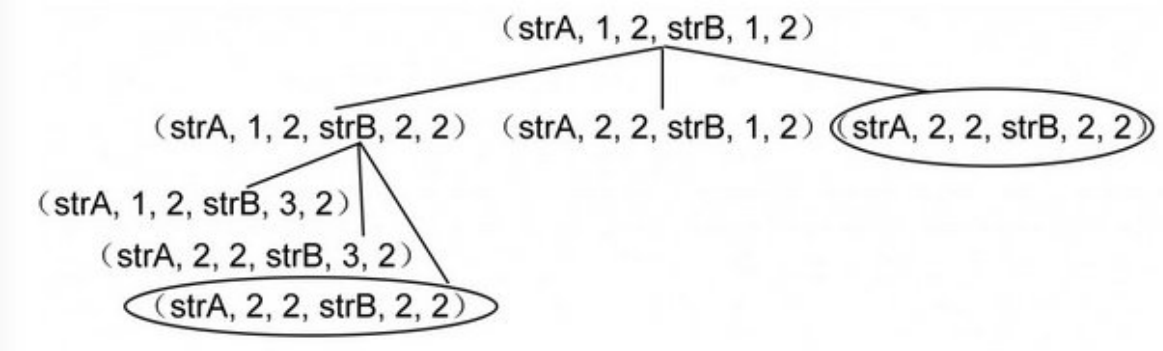
类似

上述问题类似于编程之美上的下述一题「以下内容摘自编程之美第3.3节」：

许多程序会大量使用字符串。对于不同的字符串，我们希望能够有办法判断其相似程度。我们定义了一套操作方法来把两个不相同的字符串变得相同，具体的操作方法为：

1. 修改一个字符（如把“a”替换为“b”）；
2. 增加一个字符（如把“abdd”变为“aebdd”）；
3. 删除一个字符（如把“travelling”变为“traveling”）。

比如，对于“abcdefg”和“abcdef”两个字符串来说，我们认为可以通过增加/减少一个“g”的方式来达到目的。上面的两种方案，都仅需要一次操作。把这个操作所需要的次数定义为两个字符串的距离，而相似度等于“距离+1”的倒数。也就是说，“abcdefg”和“abcdef”的距离为1，相似度为1 / 2 = 0.5。
给定任意两个字符串，你是否能写出一个算法来计算出它们的相似度呢？



这样，很快就可以完成一个递归程序，如下所示：

```
Int CalculateStringDistance(string strA, int pABegin, int pAEnd,
                            string strB, int pBBegin, int pBEnd)
{
    if (pABegin > pAEnd)
    {
        if (pBBegin > pBEnd)
            return 0;
        else

            return pBEnd - pBBegin + 1;
    }

    if (pBBegin > pBEnd)
    {
        if (pABegin > pAEnd)
            return 0;
        else
            return pAEnd - pABegin + 1;
    }
```

```
    if (strA[pABegin] == strB[pBBegin])
    {
        return CalculateStringDistance(strA, pABegin + 1, pAEnd,
                                       strB, pBBegin + 1, pBEnd);
    }
    else
    {
        int t1 = CalculateStringDistance(strA, pABegin, pAEnd, strB,
                                         pBBegin + 1, pBEnd);
        int t2 = CalculateStringDistance(strA, pABegin + 1, pAEnd,
                                         strB, pBBegin, pBEnd);
        int t3 = CalculateStringDistance(strA, pABegin + 1, pAEnd,
                                         strB, pBBegin + 1, pBEnd);
        return minValue(t1, t2, t3) + 1;
    }
}
```

上面的递归程序，有什么地方需要改进呢？在递归的过程中，有些数据被重复计算了。比如，如果开始我们调用CalculateStringDistance(strA,1, 2, strB, 1, 2)，下图是部分展开的递归调用。

不难看出，两个字符串的距离肯定不超过它们的长度之和（我们可以通过删除操作把两个串都转化为空串）。虽然这个结论对结果没有帮助，但至少可以知道，任意两个字符串的距离都是有限的。

考虑如何才能把这个问题转化成规模较小的同样的问题。如果有两个串 $A = xabcdae$ 和 $B = xfdfa$，它们的第一个字符是相同的，只要计算 $A[2,\cdots,7] = abcdae$ 和 $B[2,\cdots,5] = fdfa$ 的距离就可以了。但是如果两个串的第一个字符不相同，那么可以进行如下的操作（len$A$ 和 len$B$ 分别是 A 串和 B 串的长度）。

1. 删除 A 串的第一个字符，然后计算 $A[2,\cdots,lenA]$和 $B[1,\cdots,lenB]$的距离。
2. 删除 B 串的第一个字符，然后计算 $A[1,\cdots,lenA]$和 $B[2,\cdots,lenB]$的距离。
3. 修改 A 串的第一个字符为 B 串的第一个字符，然后计算 $A[2,\cdots,lenA]$和 $B[2,\cdots,lenB]$的距离。
4. 修改 B 串的第一个字符为 A 串的第一个字符，然后计算 $A[2,\cdots,lenA]$和 $B[2,\cdots,lenB]$的距离。
5. 增加 B 串的第一个字符到 A 串的第一个字符之前，然后计算 $A[1,\cdots,lenA]$和 $B[2,\cdots,lenB]$的距离。
6. 增加 A 串的第一个字符到 B 串的第一个字符之前，然后计算 $A[2,\cdots,lenA]$和 $B[1,\cdots,lenB]$的距离。

在这个题目中，我们并不在乎两个字符串变得相等之后的字符串是怎样的。所以，可以将上面 6 个操作合并为：

1. 一步操作之后，再将 $A[2,\cdots,lenA]$和 $B[1,\cdots,lenB]$变成相同字符串；
2. 一步操作之后，再将 $A[1,\cdots,lenA]$和 $B[2,\cdots,lenB]$变成相同字符串；
3. 一步操作之后，再将 $A[2,\cdots,lenA]$和 $B[2,\cdots,lenB]$变成相同字符串。

可以看到，圈中的两个子问题被重复计算了。为了避免这种不必要的重复计算，可以把子问题计算后的解存储起来。如何修改递归程序呢？还是DP！请看此链接。

深入

1.详细读者朋友们也已经看到了，百度/Google经常喜欢出这个字符串编辑距离，实际上，关于这个“编辑距离”问题在搜索引擎中有着重要的作用，如搜索引擎关键字查询中拼写错误的提示，如下图所示，当你输入“Jult”后，因为没有这个单词“Jult”，所以搜索引擎猜测你可能是输入错误，进而会提示你是不是找“July”：

但这个拼写错误检查的原理是什么呢？Google是基于贝叶斯统计推断的方法，相关原理详情可以看下Google的研发总监Peter Norvig写的这篇文章，以及fuanyif写的这篇。

2.关于什么是“编辑距离”：一个快速、高效的Levenshtein算法实现，这个是计算两个字符串的算法，Levenshtein距离又称为“编辑距离”，是指两个字符串之间，由一个转换成另一个所需的最少编辑操作次数。当然，次数越小越相似。这里有一个BT树的数据结构，挺有意思的。

3.最后，Lucene中也有这个算法的实现(我想，一般的搜索引擎一般都应该会有此项拼写错误检查功能的实现)，下面是lucene的源码(并没有太多优化，与实际工程中java注重实用性的原则并不背离)：

```
public final class LevensteinDistance
{

    public LevensteinDistance () { }

    // Compute Levenshtein distance:
    // see org.apache.commons.lang.StringUtils#getLevenshteinDistance(String, String)

    public float getDistance (String target, String other)
    {
        char[] sa;
        int n;
```

```
int p[];
//'previous' cost array, horizontally
int d[];
// cost array, horizontally
int _d[];
//placeholder to assist in swapping p and d

sa = target.toCharArray();
n = sa.length;
p = new int[n + 1];
d = new int[n + 1];

final int m = other.length();
if (n == 0 || m == 0)
{
    if (n == m)
    {
        return 1;
    }
    else
    {
        return 0;
    }
}

// indexes into strings s and t
int i;
// iterates through s
int j;
// iterates through t
char t_j;
// jth character of t
int cost;

for (i = 0; i <= n; i++)
{
    p[i] = i;
}

for (j = 1; j <= m; j++)
{
    t_j = other.charAt(j - 1);
    d[0] = j;

    for (i = 1; i <= n; i++)
    {
        cost = sa[i - 1] == t_j ? 0 : 1;
```

```
                // minimum of cell to the left+1, to the top+1, diagonally left and
up +cost
                d[i] = Math.min(Math.min(d[i - 1] + 1, p[i] + 1),  p[i - 1] + cost);
            }


            // copy current distance counts to 'previous row' distance counts
            _d = p;
            p = d;
            d = _d;
        }


        // our last action in the above loop was to switch d and p, so p now

        // actually has the most recent cost counts
        return 1.0f - ((float) p[n] / Math.max(other.length(), sa.length));
    }

}
```

扩展

当然，面试官还可以继续问下去，如请问，如何设计一个比较这篇文章和上一篇文章相似性的算法？这个问题的讨论可以看看这里。OK，字符串编辑距离这个问题实用性很强，限于篇幅，详情读者自己深入吧。

格子取数问题

题目描述

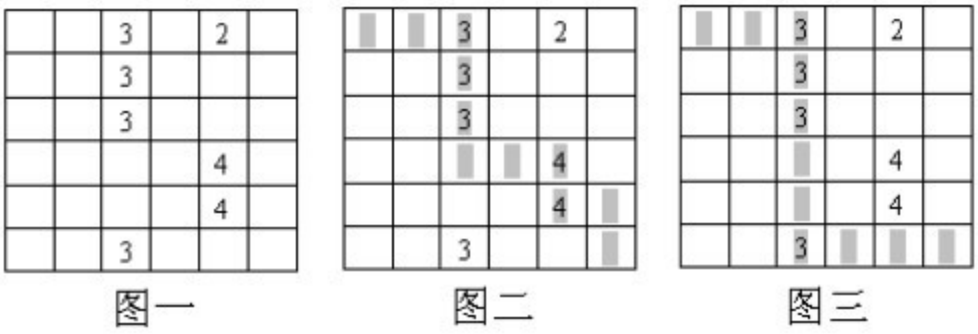
有n*n个格子，每个格子里有正数或者0，从最左上角往最右下角走，只能向下和向右，一共走两次（即从左上角走到右下角走两趟），把所有经过的格子的数加起来，求最大值SUM，且两次如果经过同一个格子，则最后总和SUM中该格子的计数只加一次。

	1	2	3	4	5	6	7	8
1	0	0	0	0	0	0	0	0
2	0	0	13	0	0	6	0	0
3	0	0	0	0	7	0	0	0
4	0	0	0	14	0	0	0	0
5	0	21	0	0	0	4	0	0
6	0	0	15	0	0	0	0	0
7	0	14	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0

分析与解法

初看到此题，因为要让两次走下来的路径总和最大，读者可能最初想到的思路可能是让每一次的路径都是最优的，即不顾全局，只看局部，让第一次和第二次的路径都是最优。

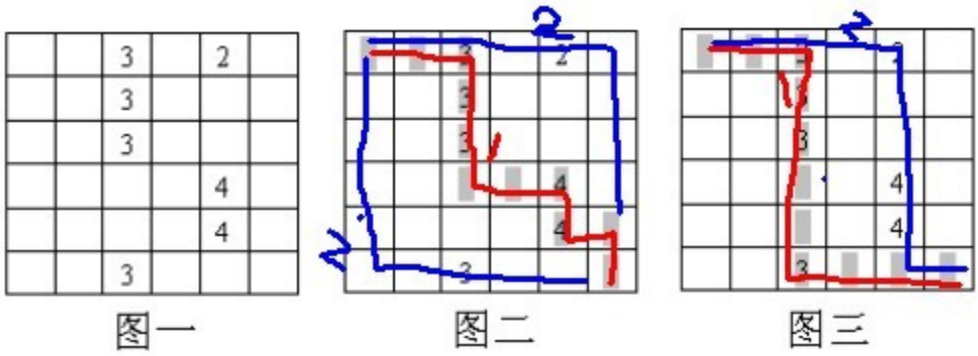
但问题马上就来了，虽然这一算法保证了连续的两次走法都是最优的，但却不能保证总体最优，相应的反例也不难给出，请看下图：



上图中，图一是原始图，那么我们有以下两种走法可供我们选择：

- 如果按照上面的局部贪优走法，那么第一次势必会如图二那样走，导致的结果是第二次要么取到2，要么取到3，
- 但若不按照上面的局部贪优走法，那么第一次可以如图三那样走，从而第二次走的时候能取到2 4 4，很显然，这种走法求得的最终SUM值更大；

为了便于读者理解，我把上面的走法在图二中标记出来，而把应该正确的走法在上图三中标示出来，如下图所示：



也就是说，上面图二中的走法太追求每一次最优，所以第一次最优，导致第二次将是很差；而图三第一次虽然不是最优，但保证了第二次不差，所以图三的结果优于图二。由此可知不要只顾局部而贪图一时最优，而丧失了全局最优。

解法一、直接搜索

局部贪优不行，我们可以考虑穷举，但最终将导致复杂度过高，所以咱们得另寻良策。@西芹_new，针对此题，可以使用直接搜索法，一共搜（2n-2）步，每一步有四种走法，考虑不相交等条件可以剪去很多枝，代码如下：

```
//copyright@西芹_new 2013
#include "stdafx.h"
#include <iostream>
using namespace std;

#define N 5
int map[5][5] =
{
    {2, 0, 8, 0, 2},
    {0, 0, 0, 0, 0},
    {0, 3, 2, 0, 0},
    {0, 0, 0, 0, 0},
    {2, 0, 8, 0, 2}
};
int sumMax = 0;
int p1x = 0;
int p1y = 0;
int p2x = 0;
int p2y = 0;
int curMax = 0;

void dfs( int index)
{
    if ( index == 2 * N - 2)
    {
        if ( curMax > sumMax)
            sumMax = curMax;
        return;
    }

    if ( !(p1x == 0 && p1y == 0) && !(p2x == N - 1 && p2y == N - 1))
    {
        if ( p1x >= p2x && p1y >= p2y )
            return;
    }

    //right right
    if ( p1x + 1 < N && p2x + 1 < N )
    {
        p1x++;
        p2x++;
```

```
            int sum = map[p1x][p1y] + map[p2x][p2y];
            curMax += sum;
            dfs(index + 1);
            curMax -= sum;
            p1x--;
            p2x--;
        }

        //down down
        if ( p1y + 1 < N && p2y + 1 < N )
        {
            p1y++;
            p2y++;
            int sum = map[p1x][p1y] + map[p2x][p2y];
            curMax += sum;
            dfs(index + 1);
            curMax -= sum;
            p1y--;
            p2y--;
        }

        //rd
        if ( p1x + 1 < N && p2y + 1 < N )
        {
            p1x++;
            p2y++;
            int sum = map[p1x][p1y] + map[p2x][p2y];
            curMax += sum;
            dfs(index + 1);
            curMax -= sum;
            p1x--;
            p2y--;
        }

        //dr
        if ( p1y + 1 < N && p2x + 1 < N )
        {
            p1y++;
            p2x++;
            int sum = map[p1x][p1y] + map[p2x][p2y];
            curMax += sum;
            dfs(index + 1);
            curMax -= sum;
            p1y--;
            p2x--;
        }
    }
```

```
int _tmain(int argc, _TCHAR* argv[])
{
    curMax = map[0][0];
    dfs(0);
    cout << sumMax - map[N - 1][N - 1] << endl;
    return 0;
}
```

解法二、动态规划

上述解法一的搜索解法是的时间复杂度是指数型的，如果是只走一次的话，是经典的dp。

2.1 、DP思路详解

故正如@绿色夹克衫所说：此题也可以用动态规划求解，主要思路就是同时DP 2次所走的状态。

1、先来分析一下这个问题，为了方便讨论，先对矩阵做一个编号，且以5*5的矩阵为例（给这个矩阵起个名字叫M1）：

M1 0 1 2 3 4 1 2 3 4 5 2 3 4 5 6 3 4 5 6 7 4 5 6 7 8

从左上(0)走到右下(8)共需要走8步（2*5-2）。我们设所走的步数为s。因为限定了只能向右和向下走，因此无论如何走，经过8步后（s = 8)都将走到右下。而DP的状态也是依据所走的步数来记录的。

再来分析一下经过其他s步后所处的位置，根据上面的讨论，可以知道：

- 经过8步后，一定处于右下角(8)；
- 那么经过5步后(s = 5)，肯定会处于编号为5的位置；
- 3步后肯定处于编号为3的位置；
- s = 4的时候，处于编号为4的位置，此时对于方格中，共有5（相当于n）个不同的位置，也是所有编号中最多的。

故推广来说，对于n*n的方格，总共需要走2n - 2步，且当s = n - 1时，编号为n个，也是编号数最多的。

如果用DP[s,i,j]来记录2次所走的状态获得的最大值，其中s表示走s步，i和j分别表示在s步后第1趟走的位置和第2趟走的位置。

2、为了方便描述，再对矩阵做一个编号（给这个矩阵起个名字叫M2）：

M2 0 0 0 0 0 1 1 1 1 1 2 2 2 2 2 3 3 3 3 3 4 4 4 4 4

把之前定的M1矩阵也再贴下：

M1 0 1 2 3 4 1 2 3 4 5 2 3 4 5 6 3 4 5 6 7 4 5 6 7 8 我们先看M1，在经过6步后，肯定处于M1中编号为6的位置。而M1中共有3个编号为6的，它们分别对应M2中的2 3 4。故对于M2来说，假设第1次经过6步走到了M2中的2，第2次经过6步走到了M2中的4，DP[s,i,j] 则对应 DP[6,2,4]。由于s = 2n - 2,0 <= i <= j <= n，所以这个DP共有O(n^3)个状态。

M1 0 1 2 3 4 1 2 3 4 5 2 3 4 5 6 3 4 5 6 7 4 5 6 7 8 再来分析一下状态转移，以DP[6,2,3]为例(就是上面M1中加粗的部分)，可以到达DP[6,2,3]的状态包括DP[5,1,2]，DP[5,1,3]，DP[5,2,2]，DP[5,2,3]。

3、下面，我们就来看看这几个状态：DP[5,1,2]，DP[5,1,3]，DP[5,2,2]，DP[5,2,3]，用加粗表示位置DP[5,1,2] DP[5,1,3] DP[5,2,2] DP[5,2,3]（加红表示要达到的状态DP[6,2,3]）

0 1 2 3 4 0 1 2 3 4 0 1 2 3 4 0 1 2 3 4 1 2 3 4 5 1 2 3 4 5 1 2 3 4 5 1 2 3 4 5 2 3 4 5 6 2 3 4 5 6 2 3 4 5 6 2 3 4 5 6 3 4 5 6 7 3 4 5 6 7 3 4 5 6 7 4 5 6 7 8 4 5 6 7 8 4 5 6 7 8 4 5 6 7 8 因此：

DP[6,2,3] = Max(DP[5,1,2] ,DP[5,1,3],DP[5,2,2],DP[5,2,3]) + 6,2和6,3格子中对应的数值 (式一)

上面（式一）所示的这个递推看起来没有涉及："如果两次经过同一个格子，那么该数只加一次的这个条件"，讨论这个条件需要换一个例子，以DP[6,2,2]为例： DP[6,2,2]可以由DP[5,1,1]，DP[5,1,2]，DP[5,2,2]到达，但由于i = j，也就是2次走到同一个格子，那么数值只能加1次。所以当i = j时，

DP[6,2,2] = Max(DP[5,1,1] , DP[5,1,2] , DP[5,2,2]) + 6,2 格子中对应的数值（式二）

4、故，综合上述的（式一），（式二）最后的递推式就是

if(i != j) DP[s, i ,j] = Max(DP[s - 1, i - 1, j - 1], DP[s - 1, i - 1, j], DP[s - 1, i, j - 1], DP[s - 1, i, j]) + W[s,i] + W[s,j] else DP[s, i ,j] = Max(DP[s - 1, i - 1, j - 1], DP[s - 1, i - 1, j], DP[s - 1, i, j]) + W[s,i] 其中W[s,i]表示经过s步后，处于i位置，位置i对应的方格中的数字。下一节我们将根据上述DP方程编码实现。

2.2 、DP方法实现

为了便于实现，我们认为所有不能达到的状态的得分都是负无穷，参考代码如下：

```
//copyright@caopengcs 2013
const int N = 202;
const int inf = 1000000000;  //无穷大
int dp[N * 2][N][N];
bool isValid(int step, int x1, int x2, int n) //判断状态是否合法
{
    int y1 = step - x1, y2 = step - x2;
    return ((x1 >= 0) && (x1 < n) && (x2 >= 0) && (x2 < n) && (y1 >= 0) && (y1 < n) && (y2 >= 0) && (y2 < n));
}

int getValue(int step, int x1, int x2, int n)  //处理越界 不存在的位置 给负无穷的值
{
    return isValid(step, x1, x2, n) ? dp[step][x1][x2] : (-inf);
}

//状态表示dp[step][i][j] 并且i <= j, 第step步  两个人分别在第i行和第j行的最大得分 时间复杂度O(n^3) 空间复杂度O(n^3)
int getAnswer(int a[N][N], int n)
{
    int P = n * 2 - 2; //最终的步数
    int i, j, step;

    //不能到达的位置 设置为负无穷大
    for (i = 0; i < n; ++i)
    {
        for (j = i; j < n; ++j)
```

```
        {
            dp[0][i][j] = -inf;
        }
    }
    dp[0][0][0] = a[0][0];

    for (step = 1; step <= P; ++step)
    {
        for (i = 0; i < n; ++i)
        {
            for (j = i; j < n; ++j)
            {
                dp[step][i][j] = -inf;
                if (!isValid(step, i, j, n))    //非法位置
                {
                    continue;
                }
                //对于合法的位置进行dp
                if (i != j)
                {
                     dp[step][i][j] = max(dp[step][i][j], getValue(step - 1, i - 1, j
- 1, n));
                      dp[step][i][j] = max(dp[step][i][j], getValue(step - 1, i - 1,
j, n));
                      dp[step][i][j] = max(dp[step][i][j], getValue(step - 1, i, j -
1, n));
                       dp[step][i][j] = max(dp[step][i][j], getValue(step - 1, i, j,
n));
                     dp[step][i][j] += a[i][step - i] + a[j][step - j];  //不在同一个格
子,加两个数
                }
                else
                {
                     dp[step][i][j] = max(dp[step][i][j], getValue(step - 1, i - 1, j
- 1, n));
                      dp[step][i][j] = max(dp[step][i][j], getValue(step - 1, i - 1,
j,  n));
                       dp[step][i][j] = max(dp[step][i][j], getValue(step - 1, i, j,
n));
                    dp[step][i][j] += a[i][step - i]; // 在同一个格子里,只能加一次
                }
            }
        }
    }
    return dp[P][n - 1][n - 1];
}
```

复杂度分析：状态转移最多需要统计4个变量的情况，看做是O(1)的，共有$O(n^3)$个状态，所以总的时间复杂度是$O(n^3)$的，且dp数组开了N^3大小，故其空间复杂度亦为$O(n^3)$。

2.3 、DP实现优化版

如上节末所说，2.2节实现的代码的复杂度空间复杂度是$O(n^3)$，事实上，空间上可以利用滚动数组优化，由于每一步的递推只跟上1步的情况有关，因此可以循环利用数组，将空间复杂度降为$O(n^2)$。

即我们在推算dp[step]的时候，只依靠它上一次的状态dp[step - 1]，所以dp数组的第一维，我们只开到2就可以了。即step为奇数时，我们用dp[1][i][j]表示状态，step为偶数我们用dp[0][i][j]表示状态，这样我们只需要O(n^2)的空间，这就是滚动数组的方法。滚动数组写起来并不复杂，只需要对上面的代码稍作修改即可，优化后的代码如下：

```
//copyright@caopengcs 8/24/2013
int dp[2][N][N];

bool isValid(int step, int x1, int x2, int n) //判断状态是否合法
{
    int y1 = step - x1, y2 = step - x2;
    return ((x1 >= 0) && (x1 < n) && (x2 >= 0) && (x2 < n) && (y1 >= 0) && (y1 < n)
&& (y2 >= 0) && (y2 < n));
}

int getValue(int step, int x1, int x2, int n)  //处理越界 不存在的位置 给负无穷的值
{
    return isValid(step, x1, x2, n) ? dp[step % 2][x1][x2] : (-inf);
}

//状态表示dp[step][i][j] 并且i <= j, 第step步  两个人分别在第i行和第j行的最大得分 时间复杂度
O(n^3) 使用滚动数组 空间复杂度O(n^2)
int getAnswer(int a[N][N], int n)
{
    int P = n * 2 - 2; //最终的步数
    int i, j, step, s;

    //不能到达的位置 设置为负无穷大
    for (i = 0; i < n; ++i)
```

```
    {
        for (j = i; j < n; ++j)
        {
          dp[0][i][j] = -inf;
        }
    }
    dp[0][0][0] = a[0][0];

    for (step = 1; step <= P; ++step)
    {
        for (i = 0; i < n; ++i)
        {
            for (j = i; j < n; ++j)
            {
                dp[step][i][j] = -inf;
                if (!isValid(step, i, j, n))    //非法位置
                {
                    continue;
                }
                s = step % 2;   //状态下表标
                //对于合法的位置进行dp
                if (i != j)
                {
                     dp[s][i][j] = max(dp[s][i][j], getValue(step - 1, i - 1, j - 1,
n));
                    dp[s][i][j] = max(dp[s][i][j], getValue(step - 1, i - 1, j, n));
                    dp[s][i][j] = max(dp[s][i][j], getValue(step - 1, i, j - 1, n));
                    dp[s][i][j] = max(dp[s][i][j], getValue(step - 1, i, j, n));
                     dp[s][i][j] += a[i][step - i] + a[j][step - j];  //不在同一个格子,
加两个数
                }
                else
                {
                     dp[s][i][j] = max(dp[s][i][j], getValue(step - 1, i - 1, j - 1,
n));
                        dp[s][i][j] = max(dp[s][i][j], getValue(step - 1, i - 1, j,
n));
                    dp[s][i][j] = max(dp[s][i][j], getValue(step - 1, i, j,  n));
                    dp[s][i][j] += a[i][step - i]; // 在同一个格子里,只能加一次
                }
            }
        }
    }
    return dp[P % 2][n - 1][n - 1];
}
```

本章习题

本书中我们详细讲了0-1背包问题(第二章的2.5节:“寻找和为定值的多个数),最大连续子数组和(2.6节:求连续子数组的最大和),最长公共子序列LCS(5.1节),最大连续乘积子串(5.2节),字符串编辑距离(5.3节),下面我们看看其它的问题。同时,这些问题中大部分只会做个简单的思路分析,留给大家课后深入思考。

1、交替字符串

输入三个字符串s1、s2和s3,判断第三个字符串s3是否由前两个字符串s1和s2交错而成,即不改变s1和s2中各个字符原有的相对顺序,例如当s1 = “aabcc”,s2 = “dbbca”,s3 = “aadbbcbcac”时,则输出true,但如果s3=“accabdbbca”,则输出false。

分析:此题不能简单的排序,因为一旦排序,便改变了s1或s2中各个字符原始的相对顺序,既然不能排序,咱们可以考虑下用动态规划的方法,令dp[i][j]代表s3[0…i+j-1]是否由s1[0…i-1]和s2[0…j-1]的字符组成 - 如果s1当前字符(即s1[i-1])等于s3当前字符(即s3[i+j-1]),而且dp[i-1][j]为真,那么可以取s1当前字符而忽略s2的情况,dp[i][j]返回真; - 如果s2当前字符等于s3当前字符,并且dp[i][j-1]为真,那么可以取s2而忽略s1的情况,dp[i][j]返回真,其它情况,dp[i][j]放回假

参考代码如下:

```
public boolean isInterleave(String s1, String 2, String 3){
    int n = s1.length(), m = s2.length(), s = s3.length();

    //如果长度不一致,则s3不可能由s1和s2交错组成
    if (n + m != s)
        return false;

    boolean[][]dp = new boolean[n + 1][m + 1];

    //在初始化边界时,我们认为空串可以由空串组成,因此dp[0][0]赋值为true。
    dp[0][0] = true;
```

```
    for (int i = 0; i < n + 1; i++){
        for (int j = 0; j < m + 1; j++){
            if ( dp[i][j] || (i - 1 >= 0 && dp[i - 1][j] == true &&
                //取s1字符
                s1.charAT(i - 1) == s3.charAT(i + j - 1)) ||

                (j - 1 >= 0 && dp[i][j - 1] == true &&
                //取s2字符
                s2.charAT(j - 1) == s3.charAT(i + j - 1)) )

                dp[i][j] = true;
            else
                dp[i][j] = false;
        }
    }
    return dp[n][m]
}
```

理解本题及上段代码，对真正理解动态规划有一定帮助。

2、最长递增子序列

给定一个长度为N的数组，找出一个最长的单调递增子序列（注：子序列不要求连续，顺序不乱即可）。例如：给定一个长度为6的数组A{5，6，7，1，2，8}，则其最长的单调递增子序列为{5，6，7，8}，长度为4。

分析：

解法一：转换为最长公共子序列问题。比如原数组为A{5，6，7，1，2，8}，当我们对这个数组进行排序后，排序后的数组为A‘{1，2，5，6，7，8}。然后想求数组A的最长递增子序列，其实就是求数组A与它的排序数组A‘的最长公共子序列，原因是原数组A的子序列顺序保持不变，而且排序后A‘本身就是递增的，这样，就保证了两序列的最长公共子序列的递增特性。如此，若想求数组A的最长递增子序列，其实就是求数组A与它的排序数组A‘的最长公共子序列。

解法二：动态规划。

子序列个数，

TSP（旅行商），

爬楼梯，

格子取数，

矩阵链乘法，

流水装配线，

最优二叉搜索树，

数塔，

导弹拦截，

铺砖问题，

木块砌墙。

40亿个数中快速查找

题目描述

给40亿个不重复的unsigned int的整数，没排过序的，然后再给一个数，如何快速判断这个数是否在那40亿个数当中？

分析与解法

海量数据处理往往会很有趣，有趣在什么地方呢？

- 空间，available的内存不够，需要反复交换内存
- 时间，速度太慢不行，毕竟那是海量数据
- 处理，数据是一次调用还是反复调用，因为针对时间和空间，通常来说，多次调用的话，势必会增加预处理以减少每次调用的时候的时间代价。

解法一

咱们回到眼前要解决的这个问题，1个unsigned int占用4字节，40亿大约是4G个数，那么一共大约要用16G的内存空间，如果内存不够大，反复和硬盘交换数据的话，后果不堪设想。

那么怎么储存这么多的数据呢？还记得伴随数组么？还是那种思想，利用内存地址代替下标。

先举例，在内存中应该是1个byte=8bit，那么明显有

0 = 0000 0000

255 = 1111 1111

69 = 0100 0101

那么69可以表示0.2.6三个数存在，其余的7以下的数不存在，0表示0-7都不存在，255表示0-7都存在，这就是位图算法：通过全部置0，存在置1，这样一种模式来通过连续的地址存贮数据，和检验数据的方法。

那么1个 unsigned int代表多少个数呢？1个unsigned int 是一个2^32以内的数，那么也就是这样的1个数，可以表示32个数是否存在。同理申请一个unsigned int的数组a[n]则可以表示连续的 n*32的数。也就是a[0]表示0-31的数是否存在，a[1]表示32-63的数是否存在，依次类推。

这时候需要用多大的内存呢？

16G/32=512M

512M和16G之间的区别，却是是否一个32位寻址的CPU能否办得到的事儿了，众所周知，32位CPU最大寻址不超过4G，固然，你会说，现在都是64位的CPU之类的云云，但是，对于底层的设计者来说，寻址范围越小越好操控的事实是不争的。

问题到这里，其实基本上已经完事了，判断本身，在位图算法这里就是找到对应的内存位置是否为1就可以了。

解法二

当然，下面就要开始说一说，当数据超出了可以接受的范围之后的事情了。比如，2^66范围的数据检索，也会是一个问题

4倍于64位CPU寻址范围，如果加上CPU本身的偏移寄存器占用的资源，可能应该是6-8个64位CPU的寻址范围，如果反复从内存到硬盘的读写，过程本身就是可怕的。

算法，更多的是用来解决瓶颈的，就想现在，根本不用考虑内存超出8M的问题，但是20年前，8086的年代，内存4M，或者内存8M，你怎么处理？固然做软件的不需要完全考虑摩尔定律，但是摩尔定律绝对是影响软件和算法编写者得想法的。

再比如，乌克兰俄罗斯的一批压缩高手，比如国内有名的R大，为什么压缩会出现？就是因为，要么存不下，要么传输时间过长。网络再好，64G的高清怎么的也得下遍历n个元素取出等概率载个一段时间吧。海量数据处理，永远是考虑超过了当前硬件条件的时候，该怎么办？！

那么我们可以发现一个更加有趣的问题，如果存不下，但是还要存，怎么办！

压缩！这里简单的说一嘴，无损压缩常见的为Huffman算法和LZW(Lenpel-Ziv &Welch)压缩算法，前者研究不多，后者却经常使用。

因为上面提到了位图算法，我就用常见的位图类的数据举例：

以下引自我的摘抄出处忘记了，请作者见谅：

对原始数据ABCCAABCDDAACCDB进行LZW压缩

原始数据中，只包括4个字符(Character),A,B,C,D,四个字符可以用一个2bit的数表示，0-A,1-B,2-C,3-D,从最直观的角度看，原始字符串存在重复字符：ABCCAABCDDAACCDB，用4代表AB,5代表CC，上面的字符串可以替代表示为:45A4CDDAA5DB,这样是不是就比原数据短了一些呢！

问题扩展

为了区别代表串的值(Code)和原来的单个的数据值(String)，需要使它们的数值域不重合，上面用0-3来代表A-D,那么AB就必须用大于3的数值来代替，再举另外一个例子，原来的数值范围可以用8bit来表示，那么就认为原始的数的范围是0～255，压缩程序生成的标号的范围就不能为0～255（如果是0-255，就重复了）。只能从256开始，但是这样一来就超过了8位的表示范围了，所以必须要扩展数据的位数，至少扩展一位，但是这样不是增加了1个字符占用的空间了么？但是却可以用一个字符代表几个字符，比如原来255是8bit,但是现在用256来表示254，255两个数，还是划得来的。从这个原理可以看出LZW算法的适用范围是原始数据串最好是有大量的子串多次重复出现，重复的越多，压缩效果越好。反之则越差，可能真的不减反增了。

伪代码如下

```
STRING = get input character
WHILE there are still input characters DO
  CHARACTER = get input character
  IF STRING+CHARACTER is in the string table then
    STRING = STRING+character
  ELSE
    output the code for STRING
    add STRING+CHARACTER to the string table
    STRING = CHARACTER
  END of IF
END of WHILE
output the code for STRING
```

看过上面的适用范围再联想本题，数据有多少种，根据同余模的原理，可以惊人的发现，其实真的非常适合压缩，但是压缩之后，尽管存下了，在查找的时候，势必又需要解码，那么又回到了我们当初学习算法时的那句经典话，算法本身，就是为了解决时间和空间的均衡问题，要么时间换空间，要么空间换时间。

更多的，请读者自行思考，因为，压缩本身只是想引起读者思考，已经是题外话了~本部分完–**上善若水.qinyu**。

随机取出其中之一元素

题目描述

一个文件中含有n个元素(n未知),要求在只能遍历一遍这n个元素的情况下,等概率随机的取出其中之一个元素。

分析与解法

假设5个人轮流抽签,只有其中某一个人能中签,那么,这5个人每个人中签的概率是相等的。不信的话,咱们可以具体计算下。

首先,第一个人中签的概率是1/5,第二个人中签的情况只能在第一个人未中时才有可能,所以第二个人中签的概率是4/5 X 1/4 = 1/5(4/5表示第一个人未中,1/4表示第二个人在剩下的4个签里中签的概率),所以,第二个人最终的中签概率也是1/5,

同理,第三个人中签的概率为:第一个人未中的概率 * 第二个人未中的概率 * 第三个人中的概率,即为:4/5 * 3/4 * 1/3 = 1/5,

一样的可以求出第四和第五个人的概率都为1/5,也就是说先后抽签顺序不影响每个人中签概率的大小。

回到咱们的问题,在明确了先后抽签顺序不影响不公平的原则之后,下面,给出选取策略:

顺序遍历, 当前遍历的元素为第L个元素,变量e表示之前选取了的某一个元素,此时生成一个随机数r,如果r%L == 0(当然0也可以是0~L-1中的任何一个,概率都是一样的), 我们将e的值替换为当前值,否则扫描下一个元素直到文件结束。

你要是给面试官说明了这样一个策略后,面试官可能会问你这样做是等概率吗?那我们来证明一下。

在遍历到第1个元素的时候，即L为1，那么r%L必然为0，所以e为第一个元素，p=100%。遍历到第2个元素时，L为2，r%L==0的概率为1/2，这个时候，第1个元素不被替换的概率为1*(1-1/2)=1/2,第1个元素被替换，也就是第2个元素被选中的概率为1/2,你可以看到，只有2时，这两个元素是等概率的机会被选中的。

同理，当遍历到第3个元素的时候，r%L==0的概率为1/3，前面被选中的元素不被替换的概率为1/2*(1-1/3)=1/3,前面被选中的元素被替换的概率，即第3个元素被选中的概率为1/3。

归纳法证明，这样走到第L个元素时，这L个元素中任一被选中的概率都是1/L，那么走到L+1时，第L+1个元素选中的概率为1/(L+1)，之前选中的元素不被替换，即继续被选中的概率为1/L*(1-1/(L+1)) = 1/(L+1)。证毕。

也就是说，走到文件最后，每一个元素最终被选出的概率为1/n，n为文件中元素的总数。

下面给出一个此选取策略的伪代码：

```
Element RandomPick(file):
Int length = 1;
While (length <= file.size)
  If (rand() % length == 0)
    Picked = File[length];
  Length++;
Return picked
```

举一反三

一个文件含有n个元素，n未知的情况下，顺序遍历一遍，要求等概率随机取r个，其中r < n。

第十六章、全排列问题

题目描述

输入一个字符串，打印出该字符串中字符的所有排列。

例如输入字符串abc，则输出由字符a、b、c 所能排列出来的所有字符串

abc、acb、bac、bca、cab 和 cba。

分析与解法

解法一、递归实现

从集合中依次选出每一个元素，作为排列的第一个元素，然后对剩余的元素进行全排列，如此递归处理，从而得到所有元素的全排列。以对字符串abc进行全排列为例，我们可以这么做：以abc为例

固定a，求后面bc的排列：abc，acb，求好后，a和b交换，得到bac

固定b，求后面ac的排列：bac，bca，求好后，c放到第一位置，得到cba

固定c，求后面ba的排列：cba，cab。代码可如下编写所示：

```
template <typename T>
void CalcAllPermutation_R(T perm[], int first, int num)
{
    if (num <= 1)
    {
        return;
    }

    for (int i = first; i < first + num; ++i)
    {
        swap(perm[i], perm[first]);
        CalcAllPermutation_R(perm, first + 1, num - 1);
        swap(perm[i], perm[first]);
    }
}
```

或者如此编写，亦可：

```
void Permutation(char* pStr, char* pBegin);
```

```
void Permutation(char* pStr)
{
    Permutation(pStr, pStr);
}

void Permutation(char* pStr, char* pBegin)
{
    if (!pStr || !pBegin)
        return;

    if (*pBegin == '\0')
    {
        printf("%s\n", pStr);
    }
    else
    {
        for (char* pCh = pBegin; *pCh != '\0'; ++ pCh)
        {
            // swap pCh and pBegin
            char temp = *pCh;
            *pCh = *pBegin;
            *pBegin = temp;

            Permutation(pStr, pBegin + 1);
            // restore pCh and pBegin
            temp = *pCh;
            *pCh = *pBegin;
            *pBegin = temp;
        }
    }
}
```

解法二、字典序排列

把升序的排列（当然，也可以实现为降序）作为当前排列开始，然后依次计算当前排列的下一个字典序排列。

对当前排列从后向前扫描，找到一对为升序的相邻元素，记为i和j（i < j）。如果不存在这样一对为升序的相邻元素，则所有排列均已找到，算法结束；否则，重新对当前排列从后向前扫描，找到第一个大于i的元素k，交换i和k，然后对从j开始到结束的子序列反转，则此时得到的新排列就为下一个字典序排列。这种方式实现得到的所有排列是按字典序有序的，这也是C++ STL算法next_permutation的思想。算法实现如下：

```
template <typename T>
void CalcAllPermutation(T perm[], int num)
{
    if (num < 1)
        return;

    while (true)
    {
        int i;
        for (i = num - 2; i >= 0; --i)
        {
            if (perm[i] < perm[i + 1])
                break;
        }

        if (i < 0)
            break;  // 已经找到所有排列

        int k;
        for (k = num - 1; k > i; --k)
        {
            if (perm[k] > perm[i])
                break;
        }

        swap(perm[i], perm[k]);
        reverse(perm + i + 1, perm + num - 1);
    }
}
```

扩展：如果不是求字符的所有排列，而是求字符的所有组合，应该怎么办呢？当输入的字符串中含有相同的字符串时，相同的字符交换位置是不同的排列，但是同一个组合。举个例子，如果输入abc，它的组合有a、b、c、ab、ac、bc、abc。

类似问题

已知字符串里的字符是互不相同的，现在任意组合，比如ab，则输出aa，ab，ba，bb，编程按照字典序输出所有的组合。

点评：非简单的全排列问题（跟全排列的形式不同，abc全排列的话，只有6个不同的输出）。本题可用递归的思想，设置一个变量表示已输出的个数，然后当个数达到字符串长度时，就输出。

```
//copyright@ 一直很安静 && World Gao
//假设str已经有序
void perm(char* result, char *str, int size, int resPos)
{
  if(resPos == size)
    printf("%s\n", result);
  else
    {
      for(int i = 0; i < size; ++i)
        {
          result[resPos] = str[i];
          perm(result, str, size, resPos + 1);
        }
    }
}
```

跳台阶问题

题目描述

一个台阶总共有n 级，如果一次可以跳1 级，也可以跳2 级。

求总共有多少总跳法，并分析算法的时间复杂度。

分析与解法

解法一

首先考虑最简单的情况。如果只有1级台阶，那显然只有一种跳法。如果有2级台阶，那就有两种跳的方法了：一种是分两次跳，每次跳1级；另外一种就是一次跳2级。

现在我们再来讨论一般情况。我们把n级台阶时的跳法看成是n的函数，记为f(n)。当n>2时，第一次跳的时候就有两种不同的选择：一是第一次只跳1级，此时跳法数目等于后面剩下的n-1级台阶的跳法数目，即为f(n-1)；另外一种选择是第一次跳2级，此时跳法数目等于后面剩下的n-2级台阶的跳法数目，即为f(n-2)。因此n级台阶时的不同跳法的总数f(n)=f(n-1)+f(n-2)。

我们把上面的分析用一个公式总结如下：

```
        /  1                              n = 1
f(n)=      2                              n = 2
        \  f(n-1) + f(n-2)                n > 2
```

原来上述问题就是我们平常所熟知的Fibonacci数列问题。可编写代码，如下：

```
long long Fibonacci_Solution1(unsigned int n)
{
    int result[3] = {0, 1, 2};
    if (n <= 2)
        return result[n];

    return Fibonacci_Solution1(n - 1) + Fibonacci_Solution1(n - 2);
}
```

那么，如果是人人笔试那道题呢?一个人上台阶可以一次上1个，2个，或者3个，岂不是可以轻而易举的写下如下公式：

```
        / 1                                   n = 1
f(n)=     2                                   n = 2
          4                                   n = 3       //111, 12, 21, 3
        \ f(n-1)+f(n-2)+f(n-3)                n > 3
```

解法二

行文至此，你可能会认为问题已经解决了，但事实上没有。

用递归方法计算的时间复杂度是以n的指数的方式递增的，我们可以尝试用递推方法解决。具体如何操作，读者自行思考。

有一种方法，能在O(logn)的时间复杂度内求解Fibonacci数列问题，你能想到么?

同时，有朋友指出对于这个台阶问题只需求幂就可以了（求复数幂C++库里有），不用任何循环且复杂度为O(1)，如下图所示，是否真如此?:

$$f(n)=f(n-1)+f(n-2)+f(n-3)$$
$$f(1)=1, f(2)=2, f(3)=4;$$
$$x_1=1.839286755214161$$
$$x_2=-0.419643377607081+0.606290729207199i$$
$$x_3=-0.419643377607081-0.606290729207199i$$
$$A=0.618419922319393$$
$$B=0.190790038840305-0.018700583111741i$$
$$C=0.190790038840304+0.018700583111740i$$
$$f(n)=Ax_1^n+Bx_2^n+Cx_3^n, n=1,2,3\cdots$$

解法三

可以使用动态规划求解，开辟一个足够长度的数组Arr[n]，递推计算即可。

```
Arr[n] = Arr[n-1] + Arr[n-2]; /* 从其他地方可达路径*/
```

举一反三

1、兔子繁殖问题

13世纪意大利数学家斐波那契在他的《算盘书》中提出这样一个问题：有人想知道一年内一对兔子可繁殖成多少对，便筑了一道围墙把一对兔子关在里面。已知一对兔子每一个月可以生一对小兔子，而一对兔子出生后.第三个月开始生小兔子假如一年内没有发生死亡，则一对兔子一年内能繁殖成多少对？

分析:这就是斐波那契数列的由来,本节的跳台阶问题便是此问题的变形,只是换了种表述形式。

2、换硬币问题。

想兑换100元钱,有1,2,5,10四种钱,问总共有多少兑换方法。

```
const int N = 100;
int dimes[] = {1,2,5,10};
int arr[N+1]  = {1};
for (int i = 0; i < 101; ++i)
{
    for(int j = 0; j < sizeof(dimes)/sizeof(int); ++j)
    {
        if (i + dimes[j] < 101)
            arr[i + dimes[j]] += arr[i];
    }
}
/* arr[100]  即为可以兑换的种类。*/
```

此问题还有一个变形,就是打印出路径目前只想到要使用递归来解决这个问题。对此,利用一个vector来保存路径,每进入一层,push_back一个路径,每退出一层,pop_back一个路径。

奇偶调序

题目描述

输入一个整数数组，调整数组中数字的顺序，使得所有奇数位于数组的前半部分，所有偶数位于数组的后半部分。要求时间复杂度为O(n)。

分析与解法

解法一

最容易想到的办法是从头扫描这个数组,每碰到一个偶数时,拿出这个数字,并把位于这个数字后面的所有数字往前挪动一位。挪完之后在数组的末尾有一个空位,这时把该偶数放入这个空位。由于碰到一个偶数,

需要移动O(n)个数字，只是这种方法总的时间复杂度是O(n^2)，不符合要求。

解法二

再细想下，我们发觉可以考虑维护两个指针，一个指针指向数组的第一个数字，向后移动；一个指针指向最后一个数字，向前移动。如果第一个指针指向的数字是偶数而第二个指针指向的数字是奇数，我们就交换这两个数字。

思路有了，接下来，写代码实现：

```
//思路，很简答，俩指针，一首一尾
//如果第一个指针指向的数字是偶数而第二个指针指向的数字是奇数，
//我们就交换这两个数字

// 2 1 3 4 6 5 7
// 7 1 3 4 6 5 2
// 7 1 3 5 6 4 2

//如果限制空间复杂度为O(1)，时间为O(N)，且奇偶数之间相对顺序不变，就相当于正负数间顺序调整的那道题了。

//copyright@2010 zhedahht
void Reorder(int *pData, unsigned int length, bool (*func)(int));
bool isEven(int n);
void ReorderOddEven(int *pData, unsigned int length)
{
    if (pData == NULL || length == 0)
        return;

    Reorder(pData, length, isEven);
}
void Reorder(int *pData, unsigned int length, bool (*func)(int))
{
    if (pData == NULL || length == 0)
        return;
    int *pBegin = pData;
    int *pEnd = pData + length - 1;
    while (pBegin < pEnd)
    {
        // if *pBegin does not satisfy func, move forward
        if (!func(*pBegin))   //如果是奇数，则向后移一个位置
        {
```

```
            pBegin ++;
            continue;
        }

        // if *pEnd does not satisfy func, move backward
        if (func(*pEnd))      //如是是偶数，则向前移一个位置
        {
            pEnd --;
            continue;
        }
        // if *pBegin satisfy func while *pEnd does not,
        // swap these integers
        int temp = *pBegin;
        *pBegin = *pEnd;
        *pEnd = temp;
    }
}
bool isEven(int n)
{
    return (n & 1) == 0;
}
```

问题扩展

细心的同学一看到上述解法二，可能会恍然大悟道，一前一后两个指针往中间扫描，这不就是快速排序中的其中一种实现么？的确如此。

然而，普林斯顿大学教授Robert在Coursera上讲快速排序时，他提到过Unix下的qsort有一个隐藏很久的bug，那就是当快速排序遇到大量的重复元素时，其效率会非常低下。

试想一下：假设每次将第一个元素设为pivot（即比较的向标），在一次partition结束后，所有小于该元素在pivot左边，所有大于该元素位于其右边。维护两个指针，beg_ptr和end_ptr，beg_ptr从左至右移动，end_ptr从右至左移动。当beg_ptr位置的元素比pivot小时，++beg_ptr；当beg_ptr位置的元素比pivot大时，比较pivot和end_ptr位置的元素，若end_ptr位置的元素比它小，则交换beg_ptr和end_ptr位置的元素，然后++beg_ptr和–end_ptr。

在上述描述中当出现beg_ptr或end_ptr位置的元素和pivot相等时，如何操作？上述算法能保证：当pivot为5时，所有等于5的值在一次partition

后都位于正确的位置上吗？

4 3 1 3 3 5 5 5 5 5 7 8 6 9

显然利用上述简单的快速排序是无法实现这个目的的。提到这个问题，其实是想说明我们早就接触过奇偶排序，那就是快速排序中的partition。将上述partition中的交换规则换成奇数和偶数即是奇偶排序。

现在回到刚才那个问题上，即快速排序中出现很多重复的元素怎么办？Dijkstra提出了一种三向快速排序算法。三向快速排序相对于传统的快速排序其需要维护3个指针：beg_ptr, cur_ptr, end_ptr。其思想如下：

初始时：beg_ptr = cur_ptr指向首位，设首位是pivot, end_ptr指向末位

更新规则：

- while cur_ptr <= end_ptr
- 若cur_ptr位置元素和beg_ptr位置元素相等，++cur_ptr, continue
- 若cur_ptr位置的元素比beg_ptr位置元素大，swap(beg_ptr, end_ptr), –end_ptr, continue
- 若cur_ptr位置元素比beg_ptr位置元素小，则swap(beg_ptr, cur_ptr), ++beg_ptr, ++cur_ptr, continue

这样做以后就可以实现partition后，所有重复的pivot都在中间。这种三向快速排序的partition还可以实现一个比奇偶排序更多的功能：

将一个整数数组，其元素分为正数，负数和0，将正数在左边，负数在右边，0在中间（可能聪明的读者会再次恍然大悟道，咦，这不就是第二章所述的荷兰国旗问题么？是的，有兴趣进一步追究的读者可以参阅2.7节）。

在C++的STL中已经实现了非常通用的partition算法，在头文件中，

```
template <class ForwardIterator, class Predicate>
ForwardIterator partition(ForwardIterator first,
                          ForwardIterator last, Predicate pred)
template <class ForwardIterator, class Predicate>
ForwardIterator stable_partition(ForwardIterator first, ForwardIterator last,
                                 Predicate pred);
```

pred是一个仿函数或者是函数指针，其返回一个bool型变量。

下面用STL的partition实现一个奇偶排序：

```
#include<iostream>
#include<algorithm>
#include<iterator>
using namespace std;

struct ODD_EVEN
{
    bool operator()(const int op)
    {
        return op&0x1;
    }
};
int main()
{
    int arr[] = {1,3,2,4,5,4,5};
    partition(arr, arr + sizeof(arr)/sizeof(int), ODD_EVEN());
    copy(arr, arr + sizeof(arr)/sizeof(int),ostream_iterator<int>(cout, " "));
    return EXIT_SUCCESS;
}
```

举一反三

一个未排序整数数组，有正负数，重新排列使负数排在正数前面，并且要求不改变原来的正负数之间相对顺序，比如：input: 1,7,-5,9,-12,15 ans: -5,-12,1,7,9,15 要求时间复杂度O(N),空间O(1)。

分析：如果本题没有这个要求“并且要求不改变原来的正负数之间相对顺序”，那么同奇偶数排序是一道题，但加上这个不能改变正负数之间的相对顺序后，便使得问题变得比较艰难了，若有兴趣，读者可以参考这篇论文《STABLE MINIMUM SPACE PARTITIONING IN LINEAR TIME》。

第三十二章、最小操作数

题目描述

给定一个单词集合Dict，其中每个单词的长度都相同。现从此单词集合Dict中抽取两个单词A、B，我们希望通过若干次操作把单词A变成单词B，每次操作可以改变单词的一个字母，同时，新产生的单词必须是在给定的单词集合Dict中。求所有行得通步数最少的修改方法。

举个例子如下：

Given: A = “hit” B = “cog” Dict = [“hot”,“dot”,“dog”,“lot”,“log”] Return [[“hit”,“hot”,“dot”,“dog”,“cog”], [“hit”,“hot”,“lot”,“log”,“cog”]]

即把字符串A = “hit”转变成字符串B = “cog”，有以下两种可能：

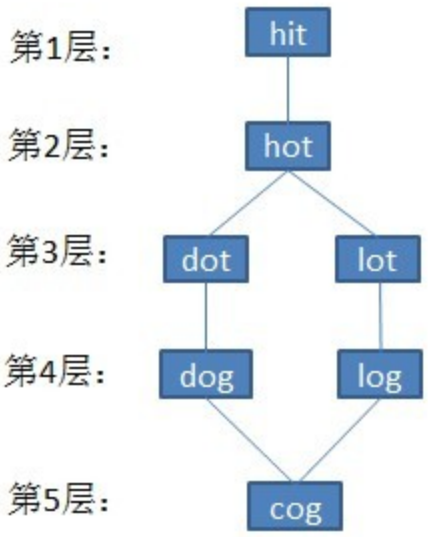
“hit” -> “hot” -> “dot” -> “dog” -> “cog”；

“hit” -> “hot” -> “lot” -> “log” ->“cog”。

分析与解法

本题是一个典型的图搜索算法问题。此题看似跟本系列的第29章的字符串编辑距离相似，但其实区别特别大，原因是最短编辑距离是让某个单词增加一个字符或减少一个字符或修改一个字符达到目标单词，来求变换的最少次数，但此最小操作数问题就只是改变一个字符。

通过此文，我们知道，在图搜索算法中，有深度优先遍历DFS和广度优先遍历BFS，而题目中并没有给定图，所以需要我们自己建立图。



涉及到图就有这么几个问题要思考，节点是什么？边如何建立？图是有方向的还是无方向的？包括建好图之后，如何记录单词序列等等都是我们要考虑的问题。

解法一、单向BFS法

1、建图

对于本题，我们的图的节点就是字典里的单词，两个节点有连边，对应着我们可以把一个单词按照规则变为另外一个单词。比如我们有单词hat，它应该与单词cat有一条连边，因为我们可以把h变为c，反过来我们也可以把c变为h，所以我们建立的连边应该是无向的。

如何建图？有两种办法，

- 第一种方法是：我们可以把字典里的任意两个单词，通过循环判断一下这两个单词是否只有一个位置上的字母不同。即假设字典里有n个单词，我们遍历任意两个单词的复杂度是O(n2)，如果每个单词长度为length，我们判断两个单词是否连边的复杂度是O(length)，所以这个建图的总复杂度是O(n2*length)。但当n比较大时，这个复杂度非常高，有没有更好的方法呢？

- 第二种方法是:我们把字典里地每个单词的每个位置的字母修改一下,从字典里查找一下(若用基于red-black tree的map查找，其查找复杂度为O(logn),若用基于hashmap的unordered_map,则查找复杂度为O(1)),修改后的单词是否在字典里出现过。即我们需要遍历字典里地每一个单词O(n),尝试修改每个位置的每个字母，对每个位置我们需要尝试26个字母（其实是25个，因为要改得和原来不同），因此这部分复杂度是O(26*length),总复杂度是O(26* n * length)（第二种方法优化版:这第二种方法能否更优?在第二种方法中,我们对每个单词每个位置尝试了26次修改,事实上我们可以利用图是无向的这一特点,我们对每个位置试图把该位置的字母变到字典序更大的字母。例如,我们只考虑cat变成hat,而不考虑hat变成cat,因为再之前已经把无向边建立了。这样,只进行一半的修改次数,从而减少程序的运行时间。当然这个优化从复杂度上来讲是常数的,因此称为常数优化,此虽算是一种改进,但不足以成为第三种方法,原因是我们经常忽略O背后隐藏的常数)。

OK,上面两种方法孰优孰劣呢?直接比较n2*length* 与 *26* n * length的大小。很明显,通常情况下,字典里的单词个数非常多,也就是n比较大,因此第二种方法效果会好一些,稍后的参考代码也会选择上述第二种方法的优化。

2、记录单词序列

对于最简单的bfs,我们是如何记录路径的?如果只需要记录一条最短路径的话,我们可以对每个走到的位置,记录走到它的前一个位置。这样到终点后,我们可以不断找到它的前一个位置。我们利用了最短路径的一个特点:即第二次经过一个节点的时候,路径长度不比第一次经过它时短。因此这样的路径是没有圈的。

但是本题需要记录全部的路径,我们第二次经过一个节点时,路径长度可能会和第一次经过一个节点时路径长度一样。这是因为,我们可能在第i层中有多个节点可以到达第(i + 1)层的同一个位置，这样那个位置有多条路径都是最短路径。

如何解决呢?——我们记录经过这个位置的前面所有位置的集合。这样一个节点的前驱不是一个节点,而是一个节点的集合。如此,当我们第二

次经过一个第(i+ 1)层的位置时，我们便保留前面那第i层位置的集合作为前驱。

3、遍历

解决了以上两个问题，我们最终得到的是什么？如果有解的话，我们最终得到的是从终点开始的前一个可能单词的集合，对每个单词，我们都有能得到它的上一个单词的集合，直到起点。这就是bfs分层之后的图，我们从终点开始遍历这个图的到起点的所有路径，就得到了所有的解，这个遍历我们可以采用之前介绍的dfs方法（路径的数目可能非常多）。

其实，为了简单起见，我们可以从终点开始bfs，因为记录路径记录的是之前的节点，也就是反向的。这样最终可以按顺序从起点遍历到终点的所有路径。

参考代码如下：

```
//copyright@caopengcs
//updated@July 08/12/2013
class Solution
{
public:
    // help 函数负责找到所有的路径
        void  help(intx,vector<int>  &d,  vector<string>  &word,vector<vector<int>  >
&next,vector<string> &path,vector<vector<string> > &answer)
    {
        path.push_back(word[x]);
        if (d[x] == 0)
        {   //已经达到终点了
            answer.push_back(path);
        }
        else
        {
            int i;
            for (i = 0; i <next[x].size(); ++i)
            {
                help(next[x][i],d, word, next,path,answer);
            }
        }
        path.pop_back();    //回溯
    }

    vector<vector<string>> findLadders(string start, string end, set<string>& dict)
```

```
{
    vector<vector<string> > answer;
    if (start == end)
    {   //起点终点恰好相等
        return answer;
    }
    //把起点终点加入字典的map
    dict.insert(start);
    dict.insert(end);
    set<string>::iterator dt;
    vector<string> word;
    map<string,int>allword;
    //把set转换为map,这样每个单词都有编号了。
    for (dt = dict.begin(); dt!= dict.end(); ++dt)
    {
        word.push_back(*dt);
        allword.insert(make_pair(*dt, allword.size()));
    }

    //建立连边 邻接表
    vector<vector<int> > con;
    int i,j,n =word.size(),temp,len = word[0].length();
    con.resize(n);
    for (i = 0; i < n; ++i)
    {
        for (j = 0; j <len; ++j)
        {
            char c;
            for (c =word[i][j] + 1; c <= 'z'; ++c)
            {  //根据上面第二种方法的优化版的思路,让每个单词每个位置变更大
                char last =word[i][j];
                word[i][j] =c;
                map<string,int>::iterator t = allword.find(word[i]);
                if (t !=allword.end())
                {
                    con[i].push_back(t->second);
                    con[t->second].push_back(i);
                }
                word[i][j] =last;
            }
        }
    }

    //以下是标准bfs过程
    queue<int> q;
    vector<int> d;
    d.resize(n, -1);
    int from = allword[start],to = allword[end];
```

```
        d[to] = 0;  //d记录的是路径长度，-1表示没经过
        q.push(to);
        vector<vector<int> > next;
        next.resize(n);
        while (!q.empty())
        {
            int x = q.front(), now= d[x] + 1;
            //now相当于路径长度
            //当now > d[from]时，则表示所有解都找到了
            if ((d[from] >= 0)&& (now > d[from]))
            {
                break;
            }
            q.pop();
            for (i = 0; i <con[x].size(); ++i)
            {
                int y = con[x][i];
                //第一次经过y
                if (d[y] < 0)
                {
                    d[y] = now;
                    q.push(y);
                    next[y].push_back(x);
                }
                //非第一次经过y
                else if (d[y] ==now)
                {  //是从上一层经过的,所以要保存
                    next[y].push_back(x);
                }
            }
        }
        if (d[from] >= 0)
        {  //有解
            vector<string>path;
            help(from, d,word,next, path,answer);
        }
        return answer;
    }
};
```

解法二、双向BFS法

BFS需要把每一步搜到的节点都存下来，很有可能每一步的搜到的节点个数越来越多，但最后的目的节点却只有一个。后半段的很多搜索都是白耗时间了。

上面给出了单向BFS的解法，但看过此前blog中的这篇文章“A*、Dijkstra、BFS算法性能比较演示”可知：双向BFS性能优于单向BFS。

举个例子如下，第1步，是起点，1个节点，第2步，搜到2个节点，第3步，搜到4个节点，第4步搜到8个节点，第5步搜到16个节点，并且有一个是终点。那这里共出现了31个节点。从起点开始广搜的同时也从终点开始广搜，就有可能在两头各第3步，就相遇了，出现的节点数不超过(1+2+4)*2=14个，如此就节省了一半以上的搜索时间。

下面给出双向BFS的解法，参考代码如下：

```
//copyright@fuwutu 6/26/2013
class Solution
{
public:
    vector<vector<string>> findLadders(string start, string end, set<string>& dict)
    {
        vector<vector<string>> result;
        if (dict.erase(start) == 1 && dict.erase(end) == 1)
        {
            map<string, vector<string>> kids_from_start;
            map<string, vector<string>> kids_from_end;

            set<string> reach_start;
            reach_start.insert(start);
            set<string> reach_end;
            reach_end.insert(end);

            set<string> meet;
            while (meet.empty() && !reach_start.empty() && !reach_end.empty())
            {
                if (reach_start.size() < reach_end.size())
                {
                     search_next_reach(reach_start, reach_end, meet, kids_from_start,
dict);
                }
                else
                {
                       search_next_reach(reach_end, reach_start, meet, kids_from_end,
dict);
                }
            }
```

```
            if (!meet.empty())
            {
                    for (set<string>::iterator it = meet.begin(); it != meet.end();
++it)
                {
                    vector<string> words(1, *it);
                    result.push_back(words);
                }

                walk(result, kids_from_start);
                for (size_t i = 0; i < result.size(); ++i)
                {
                    reverse(result[i].begin(), result[i].end());
                }
                walk(result, kids_from_end);
            }
        }

        return result;
    }

private:
       void search_next_reach(set<string>& reach, const set<string>& other_reach,
set<string>& meet, map<string, vector<string>>& path, set<string>& dict)
    {
        set<string> temp;
        reach.swap(temp);

        for (set<string>::iterator it = temp.begin(); it != temp.end(); ++it)
        {
            string s = *it;
            for (size_t i = 0; i < s.length(); ++i)
            {
                char back = s[i];
                for (s[i] = 'a'; s[i] <= 'z'; ++s[i])
                {
                    if (s[i] != back)
                    {
                        if (reach.count(s) == 1)
                        {
                            path[s].push_back(*it);
                        }
                        else if (dict.erase(s) == 1)
                        {
                            path[s].push_back(*it);
                            reach.insert(s);
                        }
                        else if (other_reach.count(s) == 1)
```

```
                        {
                            path[s].push_back(*it);
                            reach.insert(s);
                            meet.insert(s);
                        }
                    }
                }
                s[i] = back;
            }
        }
    }

    void walk(vector<vector<string>>& all_path, map<string, vector<string>> kids)
    {
        vector<vector<string>> temp;
        while (!kids[all_path.back().back()].empty())
        {
            all_path.swap(temp);
            all_path.clear();
                  for (vector<vector<string>>::iterator it = temp.begin(); it != 
temp.end(); ++it)
            {
                vector<string>& one_path = *it;
                vector<string>& p = kids[one_path.back()];
                for (size_t i = 0; i < p.size(); ++i)
                {
                    all_path.push_back(one_path);
                    all_path.back().push_back(p[i]);
                }
            }
        }
    }
};
```

其他问题

第二十章、一致性哈希算法

tencent2012笔试题附加题

问题描述： 例如手机朋友网有n个服务器，为了方便用户的访问会在服务器上缓存数据，因此用户每次访问的时候最好能保持同一台服务器。

已有的做法是根据ServerIPIndex[QQNUM%n]得到请求的服务器，这种方法很方便将用户分到不同的服务器上去。但是如果一台服务器死掉了，那么n就变为了n-1，那么ServerIPIndex[QQNUM%n]与ServerIPIndex[QQNUM%(n-1)]基本上都不一样了，所以大多数用户的请求都会转到其他服务器，这样会发生大量访问错误。

问： 如何改进或者换一种方法，使得：

1. 一台服务器死掉后，不会造成大面积的访问错误，
2. 原有的访问基本还是停留在同一台服务器上；
3. 尽量考虑负载均衡。(思路:往分布式一致哈希算法方面考虑。)

1. 最土的办法还是用模余方法:做法很简单，假设有N台服务器，现在完好的是M(M<=N),先用N求模，如果不落在完好的机器上，然后再用N-1求模，直到M.这种方式对于坏的机器不多的情况下，具有更好的稳定性。
2. 一致性哈希算法。

下面，本文剩下部分重点来讲讲这个一致性哈希算法。

应用场景

在做服务器负载均衡时候可供选择的负载均衡的算法有很多，包括：轮循算法（Round Robin）、哈希算法（HASH）、最少连接算法（Least Connection）、响应速度算法（Response Time）、加权法（Weighted）等。其中哈希算法是最为常用的算法.

典型的应用场景是：有N台服务器提供缓存服务，需要对服务器进行负载均衡，将请求平均分发到每台服务器上，每台机器负责1/N的服务。

常用的算法是对hash结果取余数 (hash() mod N)：对机器编号从0到N-1，按照自定义的hash()算法，对每个请求的hash()值按N取模，得到余数i，然后将请求分发到编号为i的机器。但这样的算法方法存在致命问题，如果某一台机器宕机，那么应该落在该机器的请求就无法得到正确的处理，这时需要将当掉的服务器从算法从去除，此时候会有(N-1)/N的服务器的缓存数据需要重新进行计算；如果新增一台机器，会有N /(N+1)的服务器的缓存数据需要进行重新计算。对于系统而言，这通常是不可接受的颠簸（因为这意味着大量缓存的失效或者数据需要转移）。那么，如何设计一个负载均衡策略，使得受到影响的请求尽可能的少呢？

在 Memcached 、Key-Value Store 、Bittorrent DHT 、LVS 中都采用了 Consistent Hashing算法，可以说Consistent Hashing 是分布式系统负载均衡的首选算法。### Consistent Hashing算法描述

下面以Memcached中的Consisten Hashing算法为例说明。

consistent hashing 算法早在 1997 年就在论文 Consistent hashing and random trees 中被提出，目前在 cache 系统中应用越来越广泛；

基本场景

比如你有 N 个 cache 服务器（后面简称 cache ），那么如何将一个对象 object 映射到 N 个 cache 上呢，你很可能会采用类似下面的通用方法计算 object 的 hash 值，然后均匀的映射到到 N 个 cache ；

hash(object)%N

一切都运行正常，再考虑如下的两种情况；

1. 一个 cache 服务器 m down 掉了（在实际应用中必须要考虑这种情况），这样所有映射到 cache m 的对象都会失效，怎么办，需要把 cache m 从 cache 中移除，这时候 cache 是 N-1 台，映射公式变成了 hash(object)%(N-1)；
2. 由于访问加重，需要添加 cache，这时候 cache 是 N+1 台，映射公式变成了 hash(object)%(N+1)；

1 和 2 意味着什么？这意味着突然之间几乎所有的 cache 都失效了。对于服务器而言，这是一场灾难，洪水般的访问都会直接冲向后台服务器；再来考虑第三个问题，由于硬件能力越来越强，你可能想让后面添加的节点多做点活，显然上面的 hash 算法也做不到。

有什么方法可以改变这个状况呢，这就是consistent hashing。

hash 算法和单调性

Hash 算法的一个衡量指标是单调性（ Monotonicity ），定义如下：

单调性是指如果已经有一些内容通过哈希分派到了相应的缓冲中，又有新的缓冲加入到系统中。哈希的结果应能够保证原有已分配的内容可以被映射到新的缓冲中去，而不会被映射到旧的缓冲集合中的其他缓冲区。

容易看到，上面的简单 hash 算法 hash(object)%N 难以满足单调性要求。

consistent hashing 算法的原理

consistent hashing 是一种 hash 算法，简单的说，在移除 / 添加一个 cache 时，它能够尽可能小的改变已存在 key 映射关系，尽可能的满足单调性的要求。

下面就来按照 5 个步骤简单讲讲 consistent hashing 算法的基本原理。

环形hash 空间

考虑通常的 hash 算法都是将 value 映射到一个 32 为的 key 值，也即是 0~2^32-1 次方的数值空间；我们可以将这个空间想象成一个首（ 0 ）尾（ 2^32-1 ）相接的圆环，如下面图 1 所示的那样。

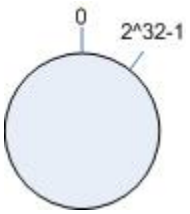


图 1 环形 hash 空间

把对象映射到hash 空间

接下来考虑 4 个对象 object1~object4，通过 hash 函数计算出的 hash 值 key 在环上的分布如图 2 所示。

hash(object1) = key1;

… …

hash(object4) = key4;

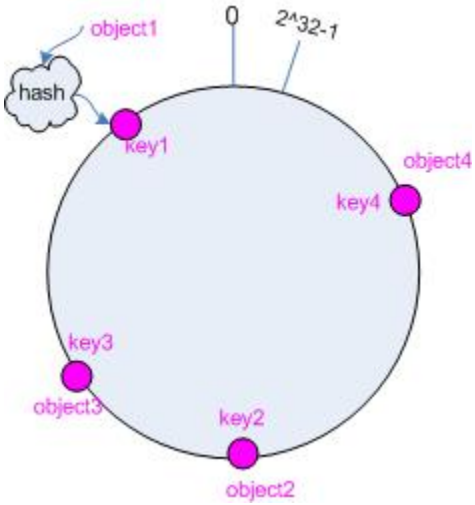


图 2 4 个对象的 key 值分布

把cache 映射到hash 空间

Consistent hashing 的基本思想就是将对象和 cache 都映射到同一个 hash 数值空间中，并且使用相同的hash 算法。

假设当前有 A,B 和 C 共 3 台 cache，那么其映射结果将如图 3 所示，他们在 hash 空间中，以对应的 hash值排列。

hash(cache A) = key A;

… …

hash(cache C) = key C;

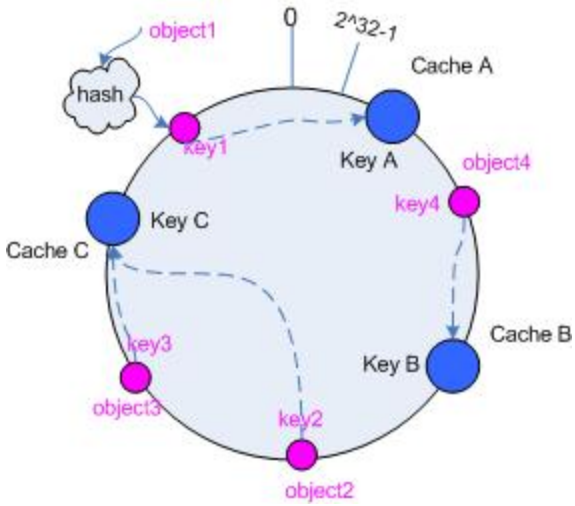


图 3 cache 和对象的 key 值分布

说到这里，顺便提一下 cache 的 hash 计算，一般的方法可以使用 cache 机器的 IP 地址或者机器名作为hash 输入。

把对象映射到cache

现在 cache 和对象都已经通过同一个 hash 算法映射到 hash 数值空间中了，接下来要考虑的就是如何将对象映射到 cache 上面了。

在这个环形空间中，如果沿着顺时针方向从对象的 key 值出发，直到遇见一个 cache，那么就将该对象存储在这个 cache 上，因为对象和 cache

的 hash 值是固定的，因此这个 cache 必然是唯一和确定的。这样不就找到了对象和 cache 的映射方法了吗?!

依然继续上面的例子（参见图 3），那么根据上面的方法，对象 object1 将被存储到 cache A 上；object2和 object3 对应到 cache C；object4 对应到 cache B；

考察cache 的变动

前面讲过，通过 hash 然后求余的方法带来的最大问题就在于不能满足单调性，当 cache 有所变动时，cache 会失效，进而对后台服务器造成巨大的冲击，现在就来分析分析 consistent hashing 算法。

移除 cache

考虑假设 cache B 挂掉了，根据上面讲到的映射方法，这时受影响的将仅是那些沿 cache B 逆时针遍历直到下一个 cache（ cache C ）之间的对象，也即是本来映射到 cache B 上的那些对象。

因此这里仅需要变动对象 object4，将其重新映射到 cache C 上即可；参见图 4 。

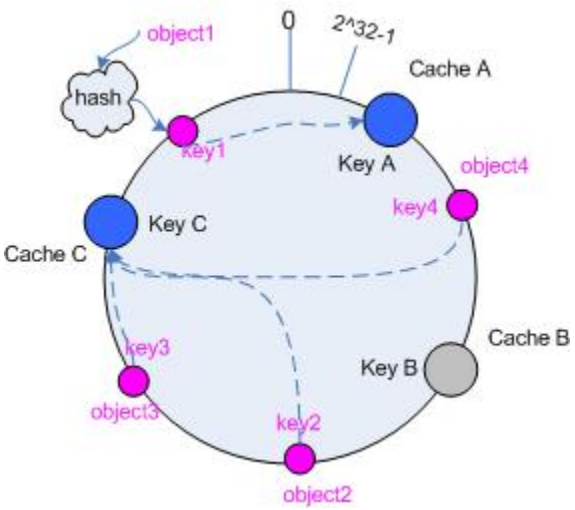


图 4 Cache B 被移除后的 cache 映射

添加 cache

再考虑添加一台新的 cache D 的情况，假设在这个环形 hash 空间中，cache D 被映射在对象 object2 和object3 之间。这时受影响的将仅是那些沿 cache D 逆时针遍历直到下一个 cache（ cache B ）之间的对象（它们是也本来映射到 cache C 上对象的一部分），将这些对象重新映射到 cache D 上即可。

因此这里仅需要变动对象 object2，将其重新映射到 cache D 上；参见图 5 。

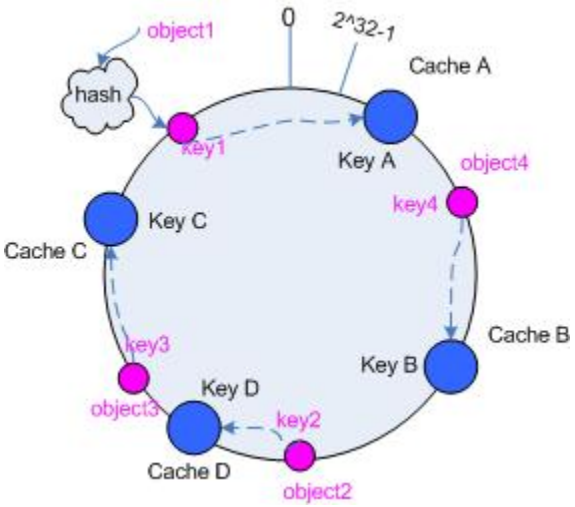


图 5 添加 cache D 后的映射关系

虚拟节点

考量 Hash 算法的另一个指标是平衡性 (Balance)，定义如下：

平衡性

平衡性是指哈希的结果能够尽可能分布到所有的缓冲中去，这样可以使得所有的缓冲空间都得到利用。

hash 算法并不是保证绝对的平衡，如果 cache 较少的话，对象并不能被均匀的映射到 cache 上，比如在上面的例子中，仅部署 cache A 和 cache C 的情况下，在 4 个对象中，cache A 仅存储了 object1，而 cache C 则存储了 object2 、object3 和 object4 ；分布是很不均衡的。

为了解决这种情况， consistent hashing 引入了“虚拟节点”的概念，它可以如下定义：

“虚拟节点”（ virtual node ）是实际节点在 hash 空间的复制品（ replica ），一实际个节点对应了若干个“虚拟节点”，这个对应个数也成为“复制个数”，“虚拟节点”在 hash 空间中以 hash 值排列。

仍以仅部署 cache A 和 cache C 的情况为例，在图 4 中我们已经看到，cache 分布并不均匀。现在我们引入虚拟节点，并设置“复制个数”为 2，这就意味着一共会存在 4 个“虚拟节点”， cache A1, cache A2 代表了 cache A ； cache C1, cache C2 代表了 cache C ；假设一种比较理想的情况，参见图 6 。

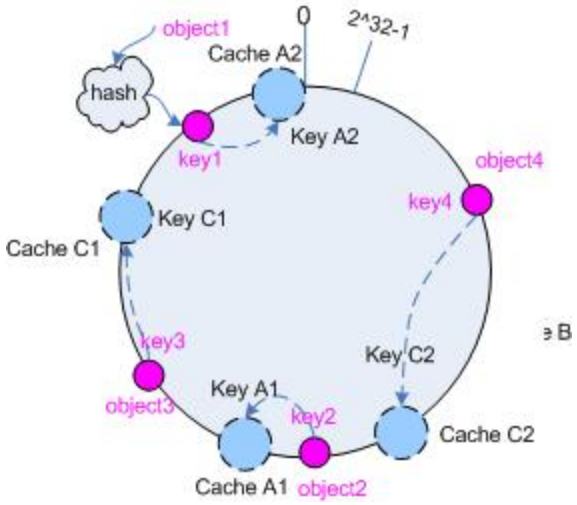


图 6 引入“虚拟节点”后的映射关系

此时，对象到“虚拟节点”的映射关系为：

objec1->cache A2 ； objec2->cache A1 ； objec3->cache C1 ； objec4->cache C2 ；

因此对象 object1 和 object2 都被映射到了 cache A 上，而 object3 和 object4 映射到了 cache C 上；平衡性有了很大提高。

引入“虚拟节点”后，映射关系就从 { 对象 -> 节点 } 转换到了 { 对象 -> 虚拟节点 } 。查询物体所在 cache时的映射关系如图 7 所示。

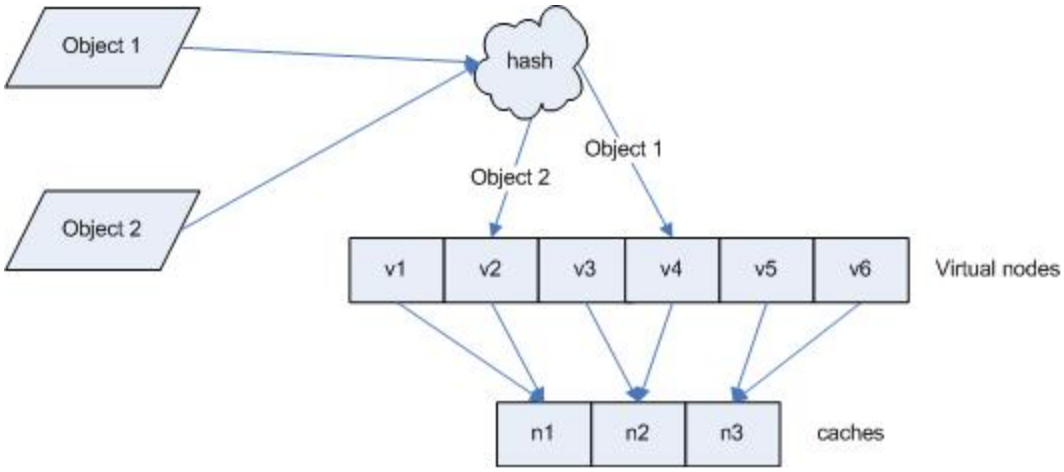


图 7 查询对象所在 cache

“虚拟节点”的 hash 计算可以采用对应节点的 IP 地址加数字后缀的方式。例如假设 cache A 的 IP 地址为202.168.14.241 。

引入“虚拟节点”前，计算 cache A 的 hash 值：

Hash(“202.168.14.241”);

引入“虚拟节点”后，计算“虚拟节”点 cache A1 和 cache A2 的 hash 值：

Hash(“202.168.14.241#1”); // cache A1

Hash(“202.168.14.241#2”); // cache A2

最短摘要的生成

题目描述

你我在百度或谷歌搜索框中敲入本博客名称的前4个字“结构之法”，便能在第一个选项看到本博客的链接，如下图2所示：

图2 谷歌中搜索关键字“结构之法”

在上面所示的图2中，搜索结果“结构之法算法之道-博客频道-CSDN.NET”下有一段说明性的文字：“程序员面试、算法研究、编程艺术、红黑树4大经典原创系列集锦与总结 作者：July–结构之法算法…”，我们把这段文字称为那个搜索结果的摘要，亦即最短摘要。我们的问题是，请问，这个最短摘要是怎么生成的呢？

分析与解法

这个问题比较完整正规的说明是：

给定一段产品的英文描述，包含M个英文字母，每个英文单词以空格分隔，无其他标点符号；再给定N个英文单词关键字，请说明思路并编程实现方法

```
String extractSummary(String description,String[] key words)
```

目标是找出此产品描述中包含N个关键字(每个关键词至少出现一次)的长度最短的子串,作为产品简介输出(不限编程语言。

简单分析如下:

@owen:扫描过程始终保持一个[left,right]的range,初始化确保[left,right]的range里包含所有关键字则停止。然后每次迭代:

1.试图右移动left,停止条件为再移动将导致无法包含所有关键字。

2.比较当前range's length和best length,更新最优值。

3.右移right,停止条件为使任意一个关键字的计数+1。

4.重复迭代。

更进一步,我们可以对问题进行如下的简化:

1.假设给定的已经是经过网页分词之后的结果,词语序列数组为W。其中W[0], W[1],…, W[N]为一些已经分好的词语。

2.假设用户输入的搜索关键词为数组Q。其中Q[0], Q[1],…, Q[m]为所有输入的搜索关键词。

这样,生成的最短摘要实际上就是一串相互联系的分词序列。比如从W[i]到W[j],其中,0 < i < j<= N。例如上图所示的摘要"程序员面试、算法研究、编程艺术、红黑树4大经典原创吸了集锦与总结 作者:July–结构之法算法之道blog之博主….."中包含了关键字——"结构之法"。

那么,我们该怎么做呢?

解法一

在分析问题之前,先通过一个实际的例子来探讨。比如在本博客第一篇置顶文章的开头,有这么一段话:

"程序员面试、算法研究、编程艺术、红黑树4大经典原创系列集锦与总结

作者:July--结构之法算法之道blog之博主。

时间:2010年10月-2011年6月。

出处:http://blog.csdn.net/v_JULY_v。

声明:版权所有,侵犯必究。"

那么，我们可以猜想一下可能的分词结果：

"程序员/面试/、/算法/研究/、/编程/艺术/、/红黑树/4/大/经典/原创/系列/集锦/与/总结/ /作者/:/July/--/结构/之/法/算法/之/道/blog/之/博主/....";（网页的分词效果W数组）

这也就是我们期望的W数组序列。

之前的Q数组序列为：

"结构之法"（用户输入的关键字Q数组）

再看下下面这个W-Q序列：

w0,w1,w2,w3,q0,w4,w5,q1,w6,w7,w8,q0,w9,q1

上述序列上面的是W数组（经过网页分词之后的结果），W[0], W[1],…, W[N]为一些已经分好的词语，上述序列下面的是Q数组（用户输入的搜索关键词）。其中Q[0], Q[1],…, Q[m]为所有输入的搜索关键词。

ok，如果你不甚明白，我说的通俗点：如上W-Q序列中，我们可以把,q0,w4,w5,q1作为摘要，q0,w9,q1的也可以作为摘要，同样都包括了所有的关键词q0，q1，那么选取哪个是最短摘要呢？答案很明显，后一个更短，选取q0,w9,q1的作为最短摘要，这便是最短摘要的生成。

我们可以进一步可以想象，如下：

从用户的角度看：当我们在百度的搜索框中输入"结构之法"4个字时，搜索引擎将在索引数据库中（关于搜索引擎原理的大致介绍，可参考本博客中这篇文章：搜索引擎技术之概要预览）查找和匹配这4个字的网页，最终第一个找到了本博客的置顶的第一篇文章：[置顶]程序员面试、算法研究、编程艺术、红黑树4大系列集锦与总结；

从搜索引擎的角度看：搜索引擎经过把上述网页分词后，便得到了上述的分词效果，然后在这些分词中查找"结构之法"4个关键字，但这4个关键字不一定只会出现一遍，它可能会在这篇文章中出现多次，就如上面的W-Q序列一般。咱们可以假想出下面的结果（结构之法便出现了两次）：

“程序员/面试/、/算法/研究/、/编程/艺术/、/红黑树/4/大/经典/原创/系列/集锦/与/总结/ /作者/:/July/--/结构/之/法/算法/之/道/blog/之/博主/././././转载/请/注明/出处/:/结构/之/法/算法/之/道/CSDN/博客/./././.”

由此,我们可以得出解决此问题的思路,如下:

1.从W数组的第一个位置开始查找出一段包含所有关键词数组Q的序列(第一个位置”程“开始:程序员/面试/、/算法/研究/、/编程/艺术/、/红黑树/4/大/经典/原创/系列/集锦/与/总结/ /作者/:/July/–/结构/之/法/查找包含关键字“结构之法”所有关键词的序列)。计算当前的最短长度,并更新Seq数组。

2.对目标数组W进行遍历,从第二个位置开始,重新查找包含所有关键词数组Q的序列(第二个位置”序“处开始:程序员/面试/、/算法/研究/、/编程/艺术/、/红黑树/4/大/经典/原创/系列/集锦/与/总结/ /作者/:/July/–/结构/之/法/查找包含关键字”结构之法“所有关键词的序列),同样计算出其最短长度,以及更新包含所有关键词的序列Seq,然后求出最短距离。

3.依次操作下去,一直到遍历至目标数组W的最后一个位置为止。

最终,通过比较,咱们确定如下分词序列作为最短摘要,即搜索引擎给出的分词效果:

“程序员面试、算法研究、编程艺术、红黑树4大经典原创系列集锦与总结 作者:July--结构之法算法之道blog之博主。时间:2010年10月-2011年6月。出处:http://...”

那么,这个算法的时间复杂度如何呢?

要遍历所有其他的关键词(M),对于每个关键词,要遍历整个网页的词(N),而每个关键词在整个网页中的每一次出现,要遍历所有的Seq,以更新这个关键词与所有其他关键词的最小距离。所以算法复杂度为:O(N^2 * M)。

解法二

我们试着降低此问题的复杂度。因为上述思路一再进行查找的时候,总是重复地循环,效率不高。那么怎么简化呢?先来看看这些序列:

w0,w1,w2,w3,q0,w4,w5,q1,w6,w7,w8,q0,w9,q1

问题在于，如何一次把所有的关键词都扫描到，并且不遗漏。扫描肯定是无法避免的，但是如何把两次扫描的结果联系起来呢？这是一个值得考虑的问题。

沿用前面的扫描方法，再来看看。第一次扫描的时候，假设需要包含所有的关键词，从第一个位置w0处将扫描到w6处：

```
w0,w1,w2,w3,q0,w4,w5,q1,w6,w7,w8,q0,w9,q1
```

那么，下次扫描应该怎么办呢？先把第一个被扫描的位置挪到q0处。

```
w0,w1,w2,w3,q0,w4,w5,q1,w6,w7,w8,q0,w9,q1
```

然后把第一个被扫描的位置继续往后面移动一格，这样包含的序列中将减少了关键词q0。那么，我们便可以把第二个扫描位置往后移，这样就可以找到下一个包含所有关键词的序列。即从w4扫描到w9处，便包含了q1，q0：

```
w0,w1,w2,w3,q0,w4,w5,q1,w6,w7,w8,q0,w9,q1
```

这样，问题就和第一次扫描时碰到的情况一样了。依次扫描下去，在w中找出所有包含q的序列，并且找出其中的最小值，就可得到最终的结果。编程之美上给出了如下参考代码：

```
//July、updated,2011.10.21
int nTargetLen = N + 1;            // 设置目标长度为总长度+1
int pBegin = 0;                      // 初始指针
int pEnd = 0;                        // 结束指针
int nLen = N;                        // 目标数组的长度为N
int nAbstractBegin = 0;            // 目标摘要的起始地址
int nAbstractEnd = 0;            // 目标摘要的结束地址

while(true)
{
    // 假设未包含所有的关键词，并且后面的指针没有越界，往后移动指针
    while(!isAllExisted() && pEnd < nLen)
    {
        pEnd++;
    }

    // 假设找到一段包含所有关键词信息的字符串
    while(isAllExisted())
    {
```

```
        if(pEnd - pBegin < nTargetLen)
        {
            nTargetLen = pEnd - pBegin;
            nAbstractBegin = pBegin;
            nAbstractEnd = pEnd - 1;
        }
        pBegin++;
    }
    if(pEnd >= N)
        Break;
}
```

小结：上述思路二相比于思路一，很明显提高了不小效率。我们在匹配的过程中利用了可以省去其中某些死板的步骤，这让我想到了KMP算法的匹配过程。同样是经过观察，比较，最后总结归纳出的高效算法。我想，一定还有更好的办法，只是我们目前还没有看到，想到，待我们去发现，创造。

解法三

以下是读者jiaotao1983回复于本文评论下的反馈，非常感谢。

关于最短摘要的生成，我觉得July的处理有些简单，我以July的想法为基础，提出了自己的一些想法，这个问题分以下几步解决：

1.将传入的key words[]生成哈希表，便于以后的字符串比较。结构为KeyHash，如下：

```
struct KeyHash
{
  int cnt;
  char key[];
  int hash;
}
```

结构体中的hash代表了关键字的哈希值，key代表了关键字，cnt代表了在当前的扫描过程中，扫描到的该关键字的个数。

当然，作为哈希表结构，该结构体中还会有其它值，这里不赘述。

初始状态下，所有哈希结构的cnt字段为0。

2.建立一个KeyWord结构，结构体如下：

```
struct KeyWord
{
  int start;
  KeyHash* key;
  KeyWord* next;
  KeyWord* prev;
}
```

key字段指向了建立的一个KeyWord代表了当前扫描到的一个关键字，扫描到的多个关键字组成一个双向链表。

start字段指向了关键字在文章中的起始位置。

3.建立几个全局变量:

KeyWord* head，指向了双向链表的头，初始为NULL。

KeyWord* tail，指向了双向链表的尾，初始为NULL。

int minLen，当前扫描到的最短的摘要的长度，初始为0。

int minStartPos，当前扫描到的最短摘要的起始位置。

int needKeyCnt，还需要几个关键字才能够包括全部的关键字，初始为关键字的个数。

4.开始对文章进行扫描。每扫描到一个关键字时，就建立一个KeyWord的结构并且将其连入到扫描到的双向链表中，更新head和tail结构，同时将对应的KeyHash结构中的cnt加1，表示扫描到了关键字。如果cnt由0变成了1，表示扫描到一个新的关键字，因此needKeyCnt减1。

5.当needKeyCnt变成0时，表示扫描到了全部的关键字了。此时要进行一个操作：链表头优化。链表头指向的word是摘要的起始点，可是如果对应的KeyHash结构中的cnt大于1，表示扫描到的摘要中还有该关键字，因此可以跳过该关键字。因此，此时将链表头更新为下一个关键字，同时，将对应的KeyHash中的结构中的cnt减1，重复这样的检查，直至某个链表头对应的KeyHash结构中的cnt为1，此时该结构不能够少了。

6.如果找到更短的minLength，则更新minLength和minStartPos。

7.开始新一轮的搜索。此时摘除链表的第一个节点，将needKeyCnt加1，将下一个节点作为链表头，同样的开始链表头优化措施。搜索从上一次的搜索结束处开始，不用回溯。就是所，搜索在整个算法的过程中是一直沿着文章向下的，不会回溯，直至文章搜索完毕。

这样的算法的复杂度初步估计是O(M+N)。

8.另外，我觉得该问题不具备实际意义，要具备实际意义，摘要应该包含完整的句子，所以摘要的起始和结束点应该以句号作为分隔。

这里，新建立一个结构：Sentence，结构体如下：

```
struct Sentence
{
  int start; //句子的起始位置
  int end; //句子的结束位置
  KeyWord* startKey; //句子包含的起始关键字
  KeyWord* endKey; //句子包含的结束关键字
  Sentence* prev; //下一个句子结构
  Sentence* next; //前一个句子结构
}
```

扫描到的多个句子结构组成一个链表。增加两个全局变量，分别指向了Sentence链表的头和尾。

扫描时，建立关键字链表时，也要建立Sentence链表。当扫描到包含了所有的关键字时，必须要扫描到一个完整句子的结束。开始做Sentence头节点优化。做法是：查看Sentence结构中的全部key结构，如果全部的key对应的KeyHash结构的cnt属性全部大于1，表明该句子是多余的，去掉它，去掉它的时候更新对应的HashKey结构的关键字，因为减去了很多的关键字。然后对下一个Sentence结构做同样的操作，直至某个Sentence结构是必不可少的，就是说它包含了当前的摘要中只出现过一次的关键字！

扫描到了一个摘要后，在开始新的扫描。更新Sentence链表的头结点为下一个节点，同时更新对应的KeyHash结构中的cnt关键字，当某个cnt变

成0时，就递增needKeycnt变量。再次扫描时仍然是从当前的结束位置开始扫描。

初步估计时间也是O(M+N)。

ok，留下一个编程之美一书上的扩展问题：当搜索一个词语后，有许多的相似页面出现，如何判断两个页面相似，从而在搜索结果中隐去这类结果？

第二十四章:倒排索引关键词不重复Hash编码

作者:July、yansha。编程艺术室出品。

出处:结构之法算法之道

本章要介绍这样一个问题,对倒排索引中的关键词进行编码。那么,这个问题将分为两个个步骤:

1. 首先,要提取倒排索引内词典文件中的关键词;
2. 对提取出来的关键词进行编码。本章采取hash编码的方式。既然要用hash编码,那么最重要的就是要解决hash冲突的问题,下文会详细介绍。

有一点必须提醒读者的是,倒排索引包含词典和倒排记录表两个部分,词典一般有词项(或称为关键词)和词项频率(即这个词项或关键词出现的次数),倒排记录表则记录着上述词项(或关键词)所出现的位置,或出现的文档及网页ID等相关信息。

24.1、正排索引与倒排索引

咱们先来看什么是倒排索引,以及倒排索引与正排索引之间的区别:

我们知道,搜索引擎的关键步骤就是建立倒排索引,所谓倒排索引一般表示为一个关键词,然后是它的频度(出现的次数),位置(出现在哪一篇文章或网页中,及有关的日期,作者等信息),它相当于为互联网上几千亿页网页做了一个索引,好比一本书的目录、标签一般。读者想看哪一个主题相关的章节,直接根据目录即可找到相关的页面。不必再从书的第一页到最后一页,一页一页的查找。

接下来,阐述下正排索引与倒排索引的区别:

一般索引(正排索引)

正排表是以文档的ID为关键字，表中记录文档中每个字的位置信息，查找时扫描表中每个文档中字的信息直到找出所有包含查询关键字的文档。正排表结构如图1所示，这种组织方法在建立索引的时候结构比较简单，建立比较方便且易于维护;因为索引是基于文档建立的，若是有新的文档假如，直接为该文档建立一个新的索引块，挂接在原来索引文件的后面。若是有文档删除，则直接找到该文档号文档对因的索引信息，将其直接删除。但是在查询的时候需对所有的文档进行扫描以确保没有遗漏，这样就使得检索时间大大延长，检索效率低下。

尽管正排表的工作原理非常的简单,但是由于其检索效率太低,除非在特定情况下,否则实用性价值不大。

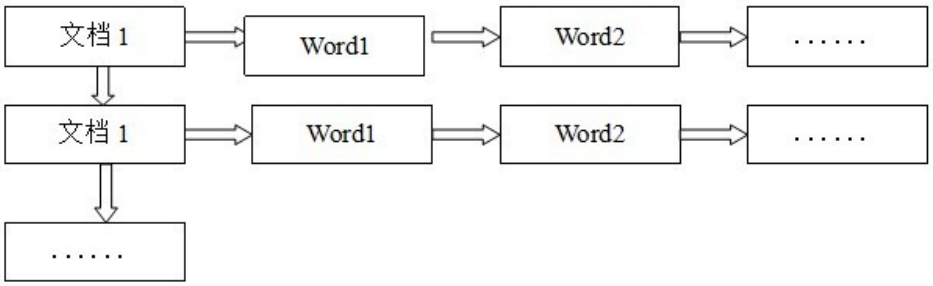


图 1 正排表结构图

倒排索引

倒排表以字或词为关键字进行索引,表中关键字所对应的记录表项记录了出现这个字或词的所有文档,一个表项就是一个字表段,它记录该文档的ID和字符在该文档中出现的位置情况。由于每个字或词对应的文档数量在动态变化,所以倒排表的建立和维护都较为复杂,但是在查询的时候由于可以一次得到查询关键字所对应的所有文档,所以效率高于正排表。在全文检索中,检索的快速响应是一个最为关键的性能,而索引建立由于在后台进行,尽管效率相对低一些,但不会影响整个搜索引擎的效率。

倒排表的结构图如图2：

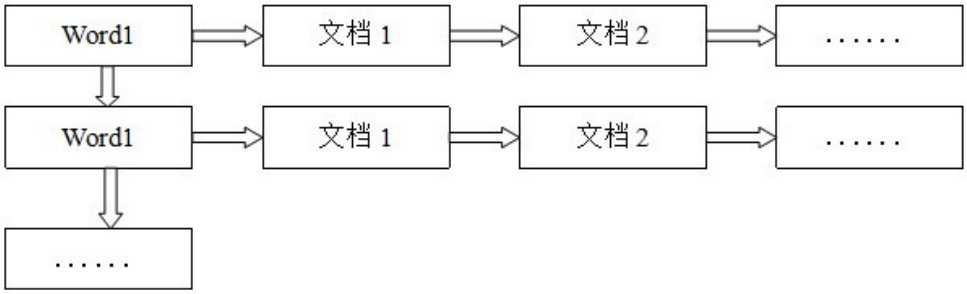


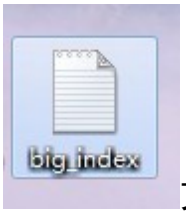
图 2 一倒排表结构图

倒排表的索引信息保存的是字或词后继数组模型、互关联后继数组模型条在文档内的位置，在同一篇文档内相邻的字或词条的前后关系没有被保存到索引文件内。

24.2、倒排索引中提取关键词

倒排索引是搜索引擎之基石。建成了倒排索引后，用户要查找某个query，如在搜索框输入某个关键词：“结构之法”后，搜索引擎不会再次使用爬虫又一个一个去抓取每一个网页，从上到下扫描网页，看这个网页有没有出现这个关键词，而是会在它预先生成的倒排索引文件中查找和匹配包含这个关键词“结构之法”的所有网页。找到了之后，再按相关性度排序，最终把排序后的结果显示给用户。

如下，即是一个倒排索引文件（不全），我们把它取名为big_index，

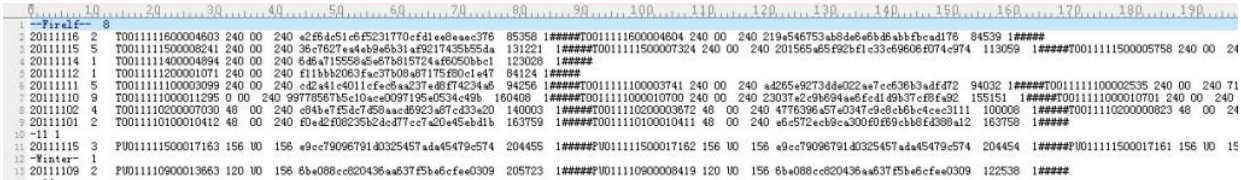


文件中每一较短的，不包含有“#####”符号的便是某个关键词，及这个关键词的出现次数。现在要从这个大索引文件中提取出这些关键词，–Firelf–，-11，-Winter-，.，007，007：天降杀机，02Chan..如何做到呢？一行一行的扫描整个索引文件么？

何意？之前已经说过：倒排索引包含词典和倒排记录表两个部分，词典一般有词项（或称为关键词）和词项频率（即这个词项或关键词出现的次数），倒排记录表则记录着上述词项（或关键词）所出现的位置，或出现的文档及网页ID等相关信息。

最简单的讲，就是要提取词典中的词项（关键词）：–Firelf–，-11，-Winter-，.，007，007：天降杀机，02Chan…。

–Firelf–（关键词）8（出现次数）



我们可以试着这么解决：通过查找#####便可判断某一行出现的词是不是关键词，但如果这样做的话，便要扫描整个索引文件的每一行，代价实在巨大。如何提高速度呢？对了，关键词后面的那个出现次数为我们问题的解决起到了很好的作用，如下注释所示：

// 本身没有##### 的行判定为关键词行，后跟这个关键词的行数N（即词项频率）// 接下来，截取关键词–Firelf–，然后读取后面关键词的行数N // 再跳过N行（滤过和避免扫描中间的倒排记录表信息）// 读取下一个关键词..

有朋友指出，上述方法虽然减少了扫描的行数，但并没有减少I0开销。读者是否有更好地办法？欢迎随时交流。

24.3、为提取出来的关键词编码

爱思考的朋友可能会问，上述从倒排索引文件中提取出那些关键词（词项）的操作是为了什么呢？其实如我个人微博上12月12日所述的Hash词典编码：

词典文件的编码：1、词典怎么生成（存储和构造词典）；2、如何运用hash对输入的汉字进行编码；3、如何更好的解决冲突，即不重复以及追加功能。具体例子为：事先构造好词典文件后，输入一个词，要求找到这个词的编码，然后将其编码输出。且要有不断能添加词的功能，不得重复。

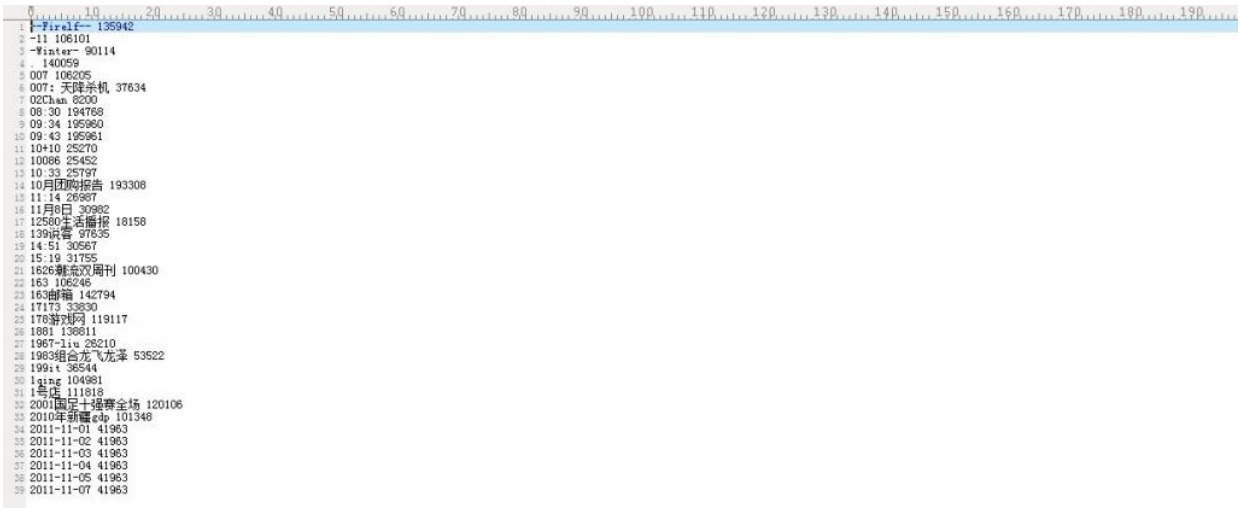
步骤应该是如下：1、读索引文件；2、提取索引中的词出来；3、词典怎么生成，存储和构造词典；4、词典文件的编码：不重复与追加功能。编码比如，输入中国，他的编码可以为10001，然后输入银行，他的编码可以为10002。只要实现不断添加词功能，以及不重复即可，词典类的大文件，hash最重要的是怎样避免冲突。

也就是说，现在我要对上述提取出来后的关键词进行编码，采取何种方式编码呢？暂时用hash函数编码。编码之后的效果将是每一个关键词都有一个特定的编码，如下图所示（与上文big_index文件比较一下便知）：

–Firelf– 对应编码为：135942

-11 对应编码为：106101

....



但细心的朋友一看上图便知，其中第34~39行显示，有重复的编码，那么如何解决这个不重复编码的问题呢？用hash表编码？但其极易产生冲突碰撞，为什么？请看：

哈希表是一种查找效率极高的数据结构，很多语言都在内部实现了哈希表。PHP中的哈希表是一种极为重要的数据结构，不但用于表示Array数据类型，还在Zend虚拟机内部用于存储上下文环境信息（执行上下文的变量及函数均使用哈希表结构存储）。

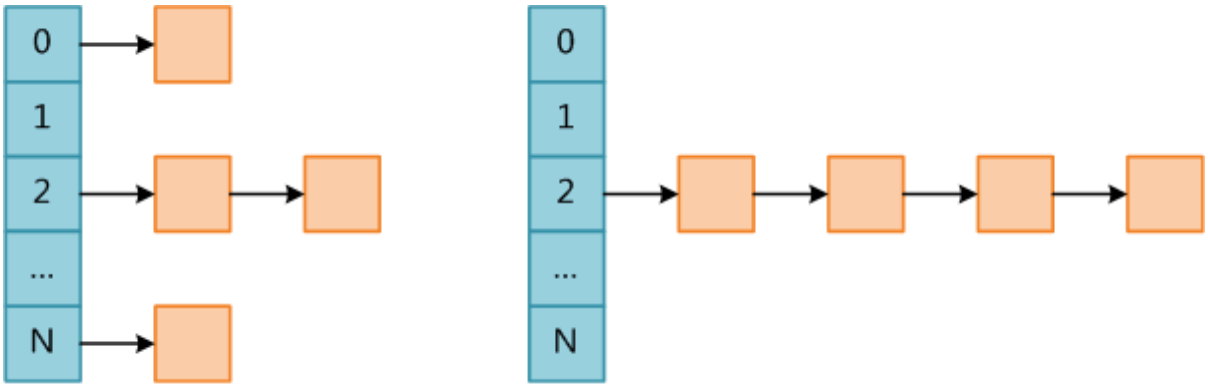
理想情况下哈希表插入和查找操作的时间复杂度均为O(1)，任何一个数据项可以在一个与哈希表长度无关的时间内计算出一个哈希值（key），然后在常量时间内定位到一个桶（术语bucket，表示哈希表中的一个位置）。当然这是理想情况下，因为任何哈希表的长度都是有限的，所以一定存在不同的数据项具有相同哈希值的情况，此时不同数据项被定为到同一个桶，称为碰撞（collision）。

哈希表的实现需要解决碰撞问题，碰撞解决大体有两种思路，

1. 第一种是根据某种原则将被碰撞数据定为到其它桶，例如线性探测——如果数据在插入时发生了碰撞，则顺序查找这个桶后面的桶，将其放入第一个没有被使用的桶；
2. 第二种策略是每个桶不是一个只能容纳单个数据项的位置，而是一个可容纳多个数据的数据结构（例如链表或红黑树），所有碰撞的数据以某种数据结构的形式组织起来。

不论使用了哪种碰撞解决策略，都导致插入和查找操作的时间复杂度不再是O(1)。以查找为例，不能通过key定位到桶就结束，必须还要比较原始key（即未做哈希之前的key）是否相等，如果不相等，则要使用与插入相同的算法继续查找，直到找到匹配的值或确认数据不在哈希表中。

PHP是使用单链表存储碰撞的数据，因此实际上PHP哈希表的平均查找复杂度为O(L)，其中L为桶链表的平均长度；而最坏复杂度为O(N)，此时所有数据全部碰撞，哈希表退化成单链表。下图PHP中正常哈希表和退化哈希表的示意图。



正常哈希表　　退化哈希表

哈希表碰撞攻击就是通过精心构造数据，使得所有数据全部碰撞，人为将哈希表变成一个退化的单链表，此时哈希表各种操作的时间均提升了一个数量级，因此会消耗大量CPU资源，导致系统无法快速响应请求，从而达到拒绝服务攻击（DoS）的目的。

可以看到，进行哈希碰撞攻击的前提是哈希算法特别容易找出碰撞，如果是MD5或者SHA1那基本就没戏了，幸运的是（也可以说不幸的是）大多数编程语言使用的哈希算法都十分简单（这是为了效率考虑），因此可以不费吹灰之力之力构造出攻击数据.（上述五段文字引自：http://www.codinglabs.org/html/hash-collisions-attack-on-php.html）。

24.4、暴雪的Hash算法

值得一提的是，在解决Hash冲突的时候，搞的焦头烂额，结果今天上午在自己的博客内的一篇文章（十一、从头到尾彻底解析Hash表算法）内找

到了解决办法:网上流传甚广的暴雪的Hash算法。OK,接下来,咱们回顾下暴雪的hash表算法:

接下来,咱们来具体分析一下一个最快的Hash表算法。

我们由一个简单的问题逐步入手:有一个庞大的字符串数组,然后给你一个单独的字符串,让你从这个数组中查找是否有这个字符串并找到它,你会怎么做?

有一个方法最简单,老老实实从头查到尾,一个一个比较,直到找到为止,我想只要学过程序设计的人都能把这样一个程序作出来,但要是有程序员把这样的程序交给用户,我只能用无语来评价,或许它真的能工作,但…也只能如此了。

最合适的算法自然是使用HashTable(哈希表),先介绍介绍其中的基本知识,所谓Hash,一般是一个整数,通过某种算法,可以把一个字符串"压缩" 成一个整数。当然,无论如何,一个32位整数是无法对应回一个字符串的,但在程序中,两个字符串计算出的Hash值相等的可能非常小,下面看看在MPQ中的Hash算法:

函数prepareCryptTable以下的函数生成一个长度为0x500(合10进制数:1280)的cryptTable[0x500]

```
//函数prepareCryptTable以下的函数生成一个长度为0x500(合10进制数:1280)的cryptTable[0x500]
void prepareCryptTable()
{
    unsigned long seed = 0x00100001, index1 = 0, index2 = 0, i;

    for( index1 = 0; index1 < 0x100; index1++ )
    {
        for( index2 = index1, i = 0; i < 5; i++, index2 += 0x100 )
        {
            unsigned long temp1, temp2;

            seed = (seed * 125 + 3) % 0x2AAAAB;
            temp1 = (seed & 0xFFFF) << 0x10;

            seed = (seed * 125 + 3) % 0x2AAAAB;
            temp2 = (seed & 0xFFFF);

            cryptTable[index2] = ( temp1 | temp2 );
        }
```

```
    }
}
```

函数HashString以下函数计算lpszFileName 字符串的hash值，其中dwHashType 为hash的类型，

```
//函数HashString以下函数计算lpszFileName 字符串的hash值，其中dwHashType 为hash的类型，
unsigned long HashString(const char *lpszkeyName, unsigned long dwHashType )
{
    unsigned char *key  = (unsigned char *)lpszkeyName;
    unsigned long seed1 = 0x7FED7FED;
    unsigned long seed2 = 0xEEEEEEEE;
    int ch;

    while( *key != 0 )
    {
        ch = *key++;
        seed1 = cryptTable[(dwHashType<<8) + ch] ^ (seed1 + seed2);
        seed2 = ch + seed1 + seed2 + (seed2<<5) + 3;
    }
    return seed1;
}
```

Blizzard的这个算法是非常高效的，被称为“One-Way Hash”(A one-way hash is a an algorithm that is constructed in such a way that deriving the original string (set of strings, actually) is virtually impossible)。举个例子，字符串“unitneutralacritter.grp”通过这个算法得到的结果是0xA26067F3。

是不是把第一个算法改进一下，改成逐个比较字符串的Hash值就可以了呢，答案是，远远不够，要想得到最快的算法，就不能进行逐个的比较，通常是构造一个哈希表(Hash Table)来解决问题，哈希表是一个大数组，这个数组的容量根据程序的要求来定义，

例如1024，每一个Hash值通过取模运算 (mod) 对应到数组中的一个位置，这样，只要比较这个字符串的哈希值对应的位置有没有被占用，就可以得到最后的结果了，想想这是什么速度？是的，是最快的O(1)，现在仔细看看这个算法吧：

```
typedef struct
{
    int nHashA;
```

```
    int nHashB;
    char bExists;
   ......
} SOMESTRUCTRUE;
//一种可能的结构体定义？
```

函数GetHashTablePos下述函数为在Hash表中查找是否存在目标字符串，有则返回要查找字符串的Hash值，无则，return -1.

```
//函数GetHashTablePos下述函数为在Hash表中查找是否存在目标字符串，有则返回要查找字符串的Hash值，无则，return -1.
int GetHashTablePos( har *lpszString, SOMESTRUCTURE *lpTable )
//lpszString要在Hash表中查找的字符串，lpTable为存储字符串Hash值的Hash表。
{
     int nHash = HashString(lpszString);   //调用上述函数HashString，返回要查找字符串lpszString的Hash值。
    int nHashPos = nHash % nTableSize;

        if ( lpTable[nHashPos].bExists    &&    !strcmp( lpTable[nHashPos].pString, lpszString ) )
    {  //如果找到的Hash值在表中存在，且要查找的字符串与表中对应位置的字符串相同，
        return nHashPos;     //返回找到的Hash值
    }
    else
    {
        return -1;
    }
}
```

看到此，我想大家都在想一个很严重的问题:“如果两个字符串在哈希表中对应的位置相同怎么办?”,毕竟一个数组容量是有限的，这种可能性很大。解决该问题的方法很多，我首先想到的就是用“链表”,感谢大学里学的数据结构教会了这个百试百灵的法宝，我遇到的很多算法都可以转化成链表来解决，只要在哈希表的每个入口挂一个链表，保存所有对应的字符串就OK了。事情到此似乎有了完美的结局，如果是把问题独自交给我解决，此时我可能就要开始定义数据结构然后写代码了。然而Blizzard的程序员使用的方法则是更精妙的方法。基本原理就是:他们在哈希表中不是用一个哈希值而是用三个哈希值来校验字符串。

MPQ使用文件名哈希表来跟踪内部的所有文件。但是这个表的格式与正常的哈希表有一些不同。首先，它没有使用哈希作为下标，把实际的文件名存储在表中用于验证，实际上它根本就没有存储文件

名。而是使用了3种不同的哈希：一个用于哈希表的下标，两个用于验证。这两个验证哈希替代了实际文件名。

当然了，这样仍然会出现2个不同的文件名哈希到3个同样的哈希。但是这种情况发生的概率平均是：1:188894659314785808547 84，这个概率对于任何人来说应该都是足够小的。现在再回到数据结构上，Blizzard使用的哈希表没有使用链表，而采用“顺延”的方式来解决问题。”下面，咱们来看看这个网上流传甚广的暴雪hash算法：

函数GetHashTablePos中，lpszString 为要在hash表中查找的字符串；lpTable 为存储字符串hash值的hash表；nTableSize 为hash表的长度：

```
//函数GetHashTablePos中，lpszString 为要在hash表中查找的字符串；lpTable 为存储字符串hash值的hash表；nTableSize 为hash表的长度：
int GetHashTablePos( char *lpszString, MPQHASHTABLE *lpTable, int nTableSize )
{
    const int  HASH_OFFSET = 0, HASH_A = 1, HASH_B = 2;

    int  nHash = HashString( lpszString, HASH_OFFSET );
    int  nHashA = HashString( lpszString, HASH_A );
    int  nHashB = HashString( lpszString, HASH_B );
    int  nHashStart = nHash % nTableSize;
    int  nHashPos = nHashStart;

    while ( lpTable[nHashPos].bExists )
   {
//      如果仅仅是判断在该表中时候存在这个字符串，就比较这两个hash值就可以了，不用对结构体中的字符串进行比较。
//          这样会加快运行的速度？减少hash表占用的空间？这种方法一般应用在什么场合？
        if (    lpTable[nHashPos].nHashA == nHashA
        &&  lpTable[nHashPos].nHashB == nHashB )
        {
             return nHashPos;
        }
        else
        {
             nHashPos = (nHashPos + 1) % nTableSize;
        }

        if (nHashPos == nHashStart)
              break;
    }
     return -1;
}
```

上述程序解释：

1. 计算出字符串的三个哈希值（一个用来确定位置，另外两个用来校验)
2. 察看哈希表中的这个位置
3. 哈希表中这个位置为空吗？如果为空，则肯定该字符串不存在，返回-1。
4. 如果存在，则检查其他两个哈希值是否也匹配，如果匹配，则表示找到了该字符串，返回其Hash值。
5. 移到下一个位置，如果已经移到了表的末尾，则反绕到表的开始位置起继续查询
6. 看看是不是又回到了原来的位置，如果是，则返回没找到
7. 回到3。

24.5、不重复Hash编码

有了上面的暴雪Hash算法。咱们的问题便可解决了。不过，有两点必须先提醒读者：
1. Hash表起初要初始化；
2. 暴雪的Hash算法对于查询那样处理可以，但对插入就不能那么解决。

关键主体代码如下：

```
//函数prepareCryptTable以下的函数生成一个长度为0x500(合10进制数:1280)的cryptTable[0x500]
void prepareCryptTable()
{
    unsigned long seed = 0x00100001, index1 = 0, index2 = 0, i;

    for( index1 = 0; index1 <0x100; index1++ )
    {
        for( index2 = index1, i = 0; i < 5; i++, index2 += 0x100)
        {
            unsigned long temp1, temp2;
            seed = (seed * 125 + 3) % 0x2AAAAB;
            temp1 = (seed & 0xFFFF)<<0x10;
            seed = (seed * 125 + 3) % 0x2AAAAB;
            temp2 = (seed & 0xFFFF);
            cryptTable[index2] = ( temp1 | temp2 );
        }
    }
}
```

```
//函数HashString以下函数计算lpszFileName 字符串的hash值，其中dwHashType 为hash的类型，
unsigned long HashString(const char *lpszkeyName, unsigned long dwHashType )
{
    unsigned char *key  = (unsigned char *)lpszkeyName;
    unsigned long seed1 = 0x7FED7FED;
    unsigned long seed2 = 0xEEEEEEEE;
    int ch;

    while( *key != 0 )
    {
        ch = *key++;
        seed1 = cryptTable[(dwHashType<<8) + ch] ^ (seed1 + seed2);
        seed2 = ch + seed1 + seed2 + (seed2<<5) + 3;
    }
    return seed1;
}

//////////////////////////////////////////////////////////////////////
//function: 哈希词典 编码
//parameter:
//author: lei.zhou
//time: 2011-12-14
//////////////////////////////////////////////////////////////////////
MPQHASHTABLE TestHashTable[nTableSize];
int TestHashCTable[nTableSize];
int TestHashDTable[nTableSize];
key_list test_data[nTableSize];

//直接调用上面的hashstring,nHashPos就是对应的HASH值。
int insert_string(const char *string_in)
{
    const int HASH_OFFSET = 0, HASH_C = 1, HASH_D = 2;
    unsigned int nHash = HashString(string_in, HASH_OFFSET);
    unsigned int nHashC = HashString(string_in, HASH_C);
    unsigned int nHashD = HashString(string_in, HASH_D);
    unsigned int nHashStart = nHash % nTableSize;
    unsigned int nHashPos = nHashStart;
    int ln, ires = 0;

    while (TestHashTable[nHashPos].bExists)
    {
//       if (TestHashCTable[nHashPos]  == (int) nHashC && TestHashDTable[nHashPos] ==
(int) nHashD)
//          break;
//      //...
//      else
        //如之前所提示读者的那般,暴雪的Hash算法对于查询那样处理可以,但对插入就不能那么解决
```

```
            nHashPos = (nHashPos + 1) % nTableSize;

        if (nHashPos == nHashStart)
            break;
    }

    ln = strlen(string_in);
    if (!TestHashTable[nHashPos].bExists && (ln < nMaxStrLen))
    {
        TestHashCTable[nHashPos] = nHashC;
        TestHashDTable[nHashPos] = nHashD;

        test_data[nHashPos] = (KEYNODE *) malloc (sizeof(KEYNODE) * 1);
        if(test_data[nHashPos] == NULL)
        {
            printf("10000 EMS ERROR !!!!\n");
            return 0;
        }

        test_data[nHashPos]->pkey = (char *)malloc(ln+1);
        if(test_data[nHashPos]->pkey == NULL)
        {
            printf("10000 EMS ERROR !!!!\n");
            return 0;
        }

        memset(test_data[nHashPos]->pkey, 0, ln+1);
        strncpy(test_data[nHashPos]->pkey, string_in, ln);
        *((test_data[nHashPos]->pkey)+ln) = 0;
        test_data[nHashPos]->weight = nHashPos;

        TestHashTable[nHashPos].bExists = 1;
    }
    else
    {
        if(TestHashTable[nHashPos].bExists)
            printf("30000 in the hash table %s !!!\n", string_in);
        else
            printf("90000 strkey error !!!\n");
    }
    return nHashPos;
}
```

接下来要读取索引文件big_index对其中的关键词进行编码（为了简单起见，直接一行一行扫描读写，没有跳过行数了）：

```
void bigIndex_hash(const char *docpath, const char *hashpath)
{
    FILE *fr, *fw;
    int len;
    char *pbuf, *p;
    char dockey[TERM_MAX_LENG];

    if(docpath == NULL || *docpath == '\0')
        return;

    if(hashpath == NULL || *hashpath == '\0')
        return;

    fr = fopen(docpath, "rb");  //读取文件docpath
    fw = fopen(hashpath, "wb");
    if(fr == NULL || fw == NULL)
    {
        printf("open read or write file error!\n");
        return;
    }

    pbuf = (char*)malloc(BUFF_MAX_LENG);
    if(pbuf == NULL)
    {
        fclose(fr);
        return ;
    }

    memset(pbuf, 0, BUFF_MAX_LENG);

    while(fgets(pbuf, BUFF_MAX_LENG, fr))
    {
        len = GetRealString(pbuf);
        if(len <= 1)
            continue;
        p = strstr(pbuf, "#####");
        if(p != NULL)
            continue;

        p = strstr(pbuf, "  ");
        if (p == NULL)
        {
            printf("file contents error!");
        }

        len = p - pbuf;
        dockey[0] = 0;
        strncpy(dockey, pbuf, len);
```

```
            dockey[len] = 0;

            int num = insert_string(dockey);

            dockey[len] = ' ';
            dockey[len+1] = '\0';
            char str[20];
            itoa(num, str, 10);

            strcat(dockey, str);
            dockey[len+strlen(str)+1] = '\0';
            fprintf (fw, "%s\n", dockey);

        }
        free(pbuf);
        fclose(fr);
        fclose(fw);
}
```

主函数已经很简单了,如下:

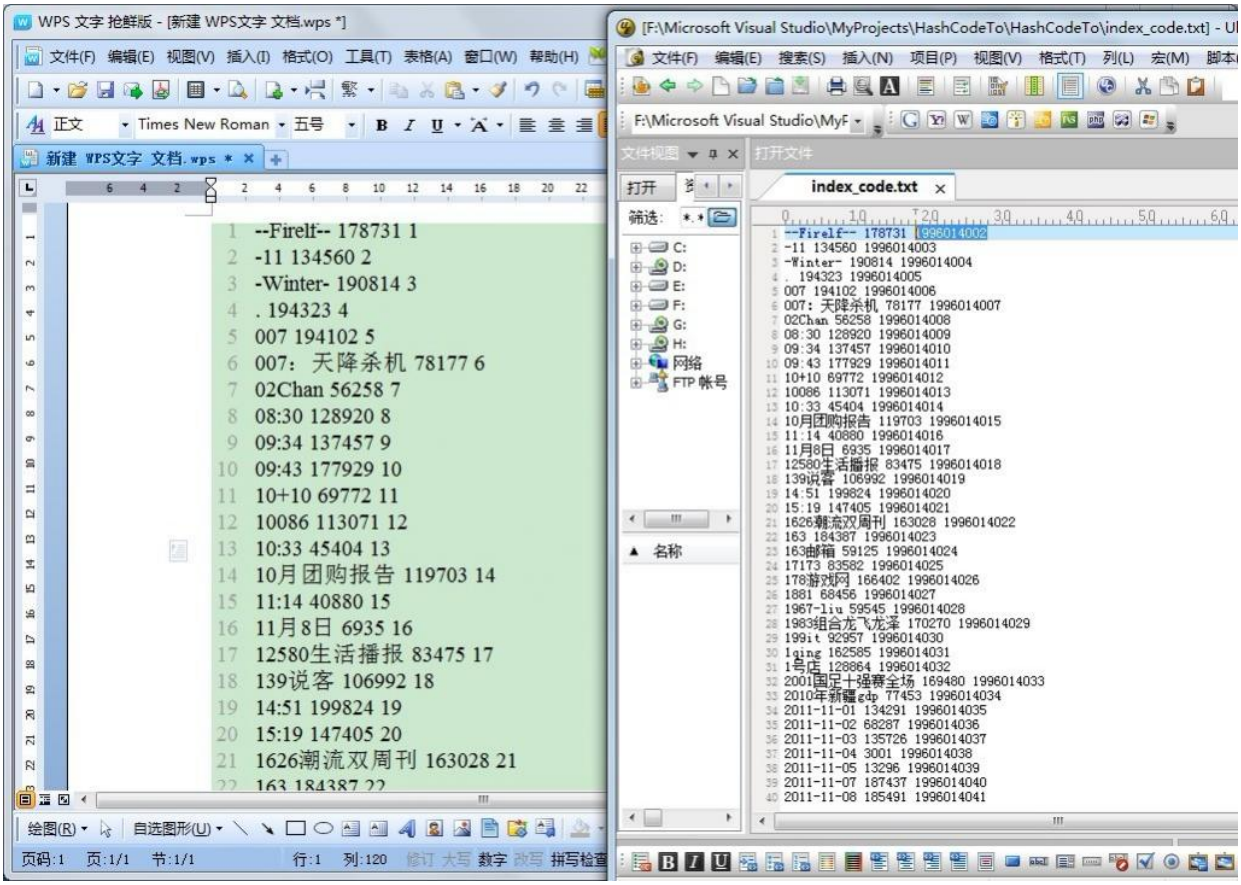
```
int main()
{
    prepareCryptTable();  //Hash表起初要初始化

    //现在要把整个big_index文件插入hash表,以取得编码结果
    bigIndex_hash("big_index.txt", "hashpath.txt");
    system("pause");

    return 0;
}
```

程序运行后生成的hashpath.txt文件如下：

如上所示，采取暴雪的Hash算法并在插入的时候做适当处理，当再次对上文中的索引文件big_index进行Hash编码后，冲突问题已经得到初步解决。当然，还有待更进一步更深入的测试。

后续添上数目索引1~10000…

后来又为上述文件中的关键词编了码一个计数的内码，不过，奇怪的是，同样的代码，在Dev C++ 与VS2010上运行结果却不同（左边dev上计数从“1”开始，VS上计数从“1994014002”开始），如下图所示：



在上面的bigIndex_hashcode函数的基础上，修改如下，即可得到上面的效果：

```
void bigIndex_hashcode(const char *in_file_path, const char *out_file_path)
{
    FILE *fr, *fw;
    int len, value;
    char *pbuf, *pleft, *p;
    char keyvalue[TERM_MAX_LENG], str[WORD_MAX_LENG];

    if(in_file_path == NULL || *in_file_path == '\0') {
        printf("input file path error!\n");
        return;
    }

    if(out_file_path == NULL || *out_file_path == '\0') {
        printf("output file path error!\n");
```

```
    return;
}

fr = fopen(in_file_path, "r");  //读取in_file_path路径文件
fw = fopen(out_file_path, "w");

if(fr == NULL || fw == NULL)
{
    printf("open read or write file error!\n");
    return;
}

pbuf = (char*)malloc(BUFF_MAX_LENG);
pleft = (char*)malloc(BUFF_MAX_LENG);
if(pbuf == NULL || pleft == NULL)
{
    printf("allocate memory error!");
    fclose(fr);
    return ;
}

memset(pbuf, 0, BUFF_MAX_LENG);

int offset = 1;
while(fgets(pbuf, BUFF_MAX_LENG, fr))
{
    if (--offset > 0)
        continue;

    if(GetRealString(pbuf) <= 1)
        continue;

    p = strstr(pbuf, "#####");
    if(p != NULL)
        continue;

    p = strstr(pbuf, "  ");
    if (p == NULL)
    {
        printf("file contents error!");
    }

    len = p - pbuf;

    // 确定跳过行数
    strcpy(pleft, p+1);
    offset = atoi(pleft) + 1;
```

```
            strncpy(keyvalue, pbuf, len);
            keyvalue[len] = '\0';
            value = insert_string(keyvalue);

            if (value != -1) {

                // key value中插入空格
                keyvalue[len] = ' ';
                keyvalue[len+1] = '\0';

                itoa(value, str, 10);
                strcat(keyvalue, str);

                keyvalue[len+strlen(str)+1] = ' ';
                keyvalue[len+strlen(str)+2] = '\0';

                keysize++;
                itoa(keysize, str, 10);
                strcat(keyvalue, str);

                // 将key value写入文件
                fprintf (fw, "%s\n", keyvalue);

            }
        }
        free(pbuf);
        fclose(fr);
        fclose(fw);
}
```

小结

本文有一点值得一提的是，在此前的这篇文章（十一、从头到尾彻底解析Hash表算法）之中，只是对Hash表及暴雪的Hash算法有过学习和了解，但尚未真正运用过它，而今在本章中体现，证明还是之前写的文章，及之前对Hash表等算法的学习还是有一定作用的。同时，也顺便对暴雪的Hash函数算是做了个测试，其的确能解决一般的冲突性问题，创造这个算法的人不简单呐。

第二十六章:基于给定的文档生成倒排索引的编码与实践

作者:July、yansha。

出处:结构之法算法之道

引言

本周实现倒排索引。实现过程中,寻找资料,结果发现找份资料诸多不易:1、网上搜倒排索引实现,结果千篇一律,例子都是那几个同样的单词;2、到谷歌学术上想找点稍微有价值水平的资料,结果下篇论文还收费或者要求注册之类;3、大部分技术书籍只有理论,没有实践。于是,朋友戏言:网上一般有价值的东西不多。希望,本blog的出现能改变此现状。

在第二十四章、倒排索引关键词不重复Hash编码中,我们针对一个给定的倒排索引文件,提取出其中的关键词,然后针对这些关键词进行Hash不重复编码。本章,咱们再倒退一步,即给定一个正排文档(暂略过文本解析,分词等步骤,日后会慢慢考虑这些且一并予以实现),要求生成对应的倒排索引文件。同时,本章还是基于Hash索引之上(运用暴雪的Hash函数可以比较完美的解决大数据量下的冲突问题),日后自会实现B+树索引。

与此同时,本编程艺术系列逐步从为面试服务而转到实战性的编程当中了,教初学者如何编程,如何运用高效的算法解决实际应用中的编程问题,将逐步成为本编程艺术系列的主旨之一。

OK,接下来,咱们针对给定的正排文档一步一步来生成倒排索引文件,有任何问题,欢迎随时不吝赐教或批评指正。谢谢。

第一节、索引的构建方法

- 根据信息检索导论（Christtopher D.Manning等著，王斌译）一书给的提示，我们可以选择两种构建索引的算法：BSBI算法，与SPIMI算法。

BSBI算法，基于磁盘的外部排序算法，此算法首先将词项映射成其ID的数据结构，如Hash映射。而后将文档解析成词项ID-文档ID对，并在内存中一直处理，直到累积至放满一个固定大小的块空间为止，我们选择合适的块大小，使之能方便加载到内存中并允许在内存中快速排序，快速排序后的块转换成倒排索引格式后写入磁盘。

建立倒排索引的步骤如下：

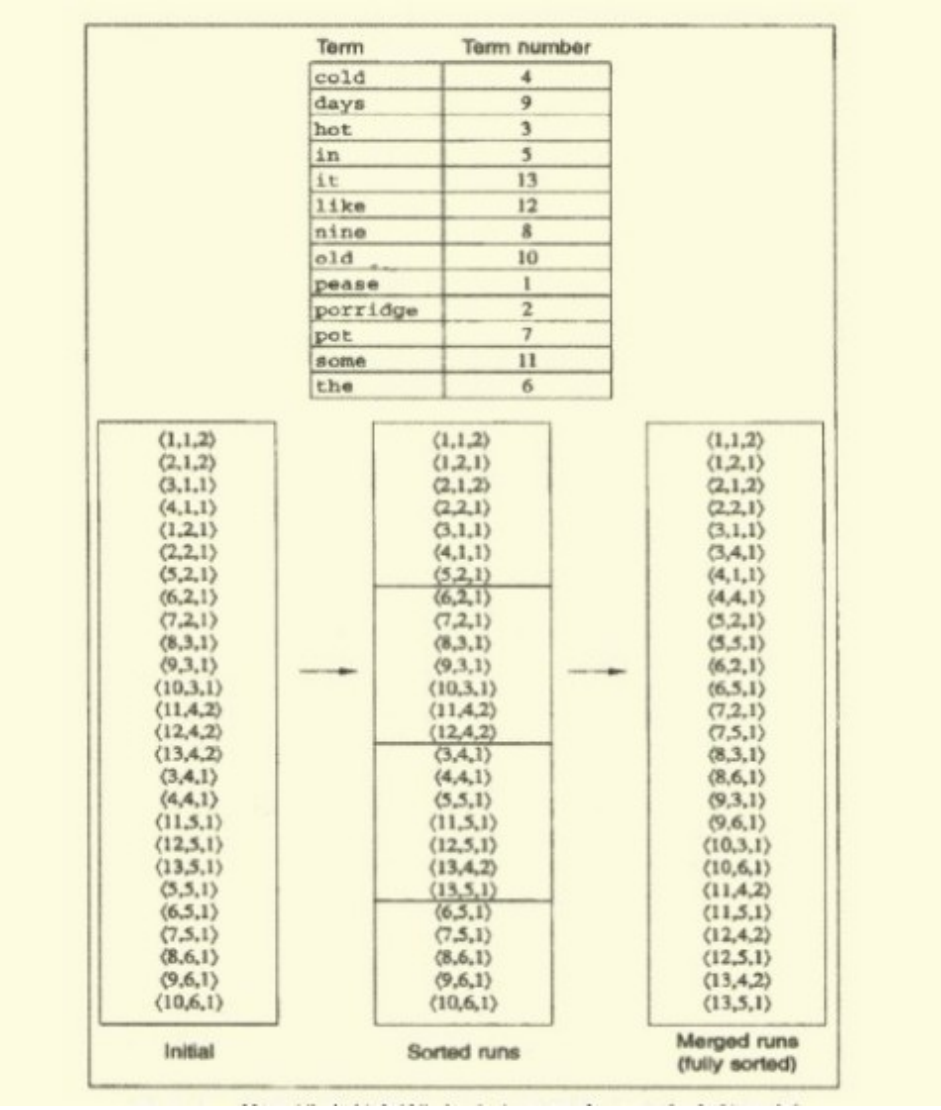
- 将文档分割成几个大小相等的部分；
- 对词项ID-文档ID进行排序；
- 将具有同一词项ID的所有文档ID放到倒排记录表中，其中每条倒排记录仅仅是一个文档ID；
- 将基于块的倒排索引写到磁盘上。

此算法假如说最后可能会产生10个块。其伪码如下：

```
BSBI NDEXConSTRUCTION()
n <- 0
while(all documents have not been processed)
    do n<-n+1
        block <- PARSENEXTBLOCK()     //文档分析
        BSBI-INVERT(block)
        WRITEBLOCKTODISK(block,fn)
        MERGEBLOCKS(f1,...,fn;fmerged)
```

（基于块的排序索引算法，该算法将每个块的倒排索引文件存入文件f1,…,fn中，最后合并成fmerged 如果该算法应用最后一步产生了10个块，那么接下来便会将10个块索引同时合并成一个索引文件。）

合并时，同时打开所有块对应的文件，内存中维护了为10个块准备的读缓冲区和一个为最终合并索引准备的写缓冲区。每次迭代中，利用优先级队列（如堆结构或类似的数据结构）选择最小的未处理的词项ID进行处理。如下图所示（图片引自深入搜索引擎–海里信息的压缩、索引和查询，梁斌译），分块索引，分块排序，最终全部合并（说实话，跟MapReduce还是有些类似的）：



读入该词项的倒排记录表并合并，合并结果写回磁盘中。需要时，再次从文件中读入数据到每个读缓冲区。

BSBI算法主要的时间消耗在排序上，选择什么排序方法呢，简单的快速排序足矣，其时间复杂度为O（N*logN），其中N是所需要排序的项（词项ID-文档ID对）的数目的上界。

SPIMI算法，内存式单遍扫描索引算法

与上述BSBI算法不同的是：SPIMI使用词项而不是其ID，它将每个块的词典写入磁盘，对于写一块则重新采用新的词典，只要硬盘空间足够大，它能索引任何大小的文档集。

倒排索引 = 词典(关键词或词项+词项频率)+倒排记录表。建倒排索引的步骤如下:

- 从头开始扫描每一个词项-文档ID(信息)对,遇一词,构建索引;
- 继续扫描,若遇一新词,则再建一新索引块(加入词典,通过Hash表实现,同时,建一新的倒排记录表);若遇一旧词,则找到其倒排记录表的位置,添加其后
- 在内存内基于分块完成排序,后合并分块;
- 写入磁盘。

其伪码如下:

```
SPIMI-Invert(Token_stream)
output.file=NEWFILE()
dictionary = NEWHASH()
while (free memory available)
    do token <-next(token_stream) //逐一处理每个词项-文档ID对
        if term(token) !(- dictionary
           /*如果词项是第一次出现,那么加入hash词典,同时,建立一个新的倒排索引表*/
           then postings_list = AddToDictionary(dictionary,term(token))
        /*如果不是第一次出现,那么直接返回其倒排记录表,在下面添加其后*/
        else postings_list = GetPostingList(dictionary,term(token))
    if full(postings_list)
        then postings_list =DoublePostingList(dictionary,term(token))
    /*SPIMI与BSBI的区别就在于此,前者直接在倒排记录表中增加此项新纪录*/
    AddToPosTingsList (postings_list,docID(token))
sorted_terms <- SortTerms(dictionary)
WriteBlockToDisk(sorted_terms,dictionary,output_file)
return output_file
```

SPIMI与BSBI的主要区别:

SPIMI当发现关键词是第一次出现时,会直接在倒排记录表中增加一项(与BSBI算法不同)。同时,与BSBI算法一开始就整理出所有的词项ID-文档ID,并对它们进行排序的做法不同(而这恰恰是BSBI的做法),这里的每个倒排记录表都是动态增长的(也就是说,倒排记录表的大小会不断调整),同时,扫描一遍就可以实现全体倒排记录表的收集。

SPIMI这样做有两点好处:

由于不需要排序操作，因此处理的速度更快，由于保留了倒排记录表对词项的归属关系，因此能节省内存，词项的ID也不需要保存。这样，每次单独的SPIMI-Invert调用能够处理的块大小可以非常大，整个倒排索引的构建过程也可以非常高效。

但不得不提的是，由于事先并不知道每个词项的倒排记录表大小，算法一开始只能分配一个较小的倒排记录表空间，每次当该空间放满的时候，就会申请加倍的空间，

与此同时，自然而然便会浪费一部分空间（当然，此前因为不保存词项ID，倒也省下一点空间，总体而言，算作是抵销了）。

不过，至少SPIMI所用的空间会比BSBI所用空间少。当内存耗尽后，包括词典和倒排记录表的块索引将被写到磁盘上，但在此之前，为使倒排记录表按照词典顺序来加快最后的合并操作，所以要对词项进行排序操作。

小数据量与大数据量的区别

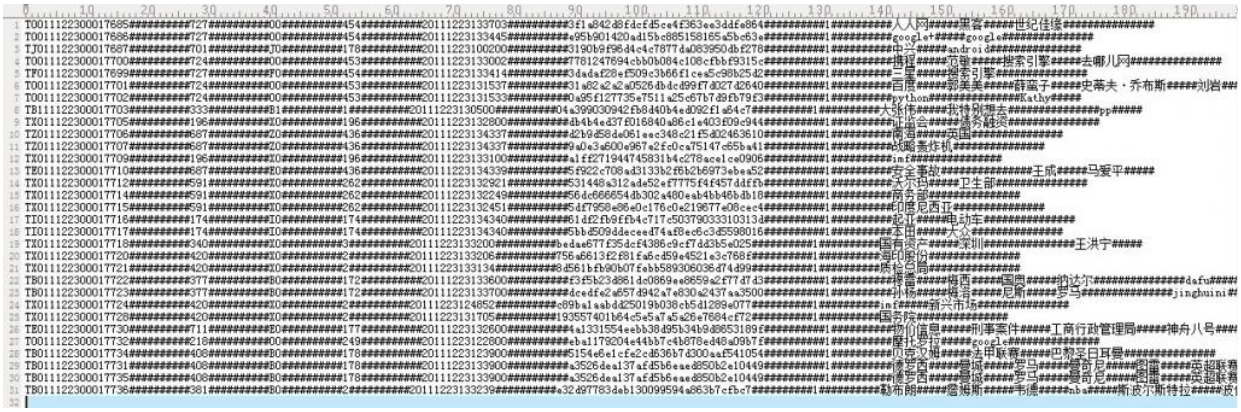
- 在小数据量时，有足够的内存保证该创建过程可以一次完成；
- 数据规模增大后，可以采用分组索引，然后再归并索 引的策略。该策略是，

建立索引的模块根据当时运行系统所在的计算机的内存大小，将索引分为 k 组，使得每组运算所需内存都小于系统能够提供的最大使用内存的大小。 按照倒排索引的生成算法，生成 k 组倒排索引。 然后将这 k 组索引归并，即将相同索引词对应的数据合并到一起，就得到了以索引词为主键的最终的倒排文件索引，即反向索引。

为了测试的方便，本文针对小数据量进行从正排文档到倒排索引文件的实现。而且针对大数量的K路归并算法或基于磁盘的外部排序算法本编程艺术系列第十章中已有详细阐述。

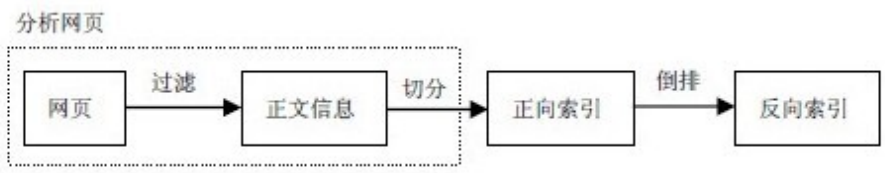
第二节、Hash表的构建与实现

如下，给定如下图所示的正排文档，每一行的信息分别为（中间用##########隔开）：文档ID、订阅源（子频道）、 频道分类、 网站类ID（大频道）、时间、 md5、文档权值、关键词、作者等等。



要求基于给定的上述正排文档。生成如第二十四章所示的倒排索引文件（注，关键词所在的文章如果是同一个日期的话，是挨在同一行的，用“#”符号隔开）：

我们知道：为网页建立全文索引是网页预处理的核心部分，包括分析网页和建立倒排文件。二者是顺序进行，先分析网页，后建立倒排文件（也称为反向索引），如图所示：



正如上图粗略所示，我们知道倒排索引创建的过程如下：

- 写爬虫抓取相关的网页，而后提取相关网页或文章中所有的关键词；
- 分词，找出所有单词；
- 过滤不相干的信息（如广告等信息）；
- 构建倒排索引，关键词=>（文章ID 出现次数 出现的位置）生成词典文件 频率文件 位置文件；
- 压缩。

因为已经给定了正排文档，接下来，咱们跳过一系列文本解析，分词等中间步骤，直接根据正排文档生成倒排索引文档（幸亏有yansha相助，不然，寸步难行，其微博地址为：http://weibo.com/yanshazi，欢迎关注他）。

OK，闲不多说，咱们来一步一步实现吧。

建相关的数据结构

根据给定的正排文档，我们可以建立如下的两个结构体表示这些信息：文档ID、订阅源（子频道）、 频道分类、 网站类ID（大频道）、时间、md5、文档权值、关键词、作者等等。如下所示：

```
typedef struct key_node
{
    char *pkey;      // 关键词实体
    int count;       // 关键词出现次数
    int pos;         // 关键词在hash表中位置
    struct doc_node *next;   // 指向文档结点
}KEYNODE, *key_list;

key_list key_array[TABLE_SIZE];

typedef struct doc_node
{
    char id[WORD_MAX_LEN];   //文档ID
    int classOne;            //订阅源（子频道）
    char classTwo[WORD_MAX_LEN];     //频道分类
    int classThree;                  //网站类ID(大频道)
    char time[WORD_MAX_LEN];         //时间
    char md5[WORD_MAX_LEN];          //md5
    int weight;                      //文档权值
```

```
    struct doc_node *next;
}DOCNODE, *doc_list;
```

我们知道，通过第二十四章的暴雪的Hash表算法，可以比较好的避免相关冲突的问题。下面，我们再次引用其代码：基于暴雪的Hash之上的改造算法

```
//函数prepareCryptTable以下的函数生成一个长度为0x100的cryptTable[0x100]
void PrepareCryptTable()
{
    unsigned long seed = 0x00100001, index1 = 0, index2 = 0, i;

    for( index1 = 0; index1 <0x100; index1++ )
    {
        for( index2 = index1, i = 0; i < 5; i++, index2 += 0x100)
        {
            unsigned long temp1, temp2;
            seed = (seed * 125 + 3) % 0x2AAAAB;
            temp1 = (seed & 0xFFFF)<<0x10;
            seed = (seed * 125 + 3) % 0x2AAAAB;
            temp2 = (seed & 0xFFFF);
            cryptTable[index2] = ( temp1 | temp2 );
        }
    }
}

//函数HashString以下函数计算lpszFileName 字符串的hash值，其中dwHashType 为hash的类型，
unsigned long HashString(const char *lpszkeyName, unsigned long dwHashType )
{
    unsigned char *key  = (unsigned char *)lpszkeyName;
    unsigned long seed1 = 0x7FED7FED;
    unsigned long seed2 = 0xEEEEEEEE;
    int ch;

    while( *key != 0 )
    {
        ch = *key++;
        seed1 = cryptTable[(dwHashType<<8) + ch] ^ (seed1 + seed2);
        seed2 = ch + seed1 + seed2 + (seed2<<5) + 3;
    }
    return seed1;
}

//按关键字查询，如果成功返回hash表中索引位置
key_list SearchByString(const char *string_in)
{
    const int HASH_OFFSET = 0, HASH_C = 1, HASH_D = 2;
```

```
    unsigned int nHash = HashString(string_in, HASH_OFFSET);
    unsigned int nHashC = HashString(string_in, HASH_C);
    unsigned int nHashD = HashString(string_in, HASH_D);
    unsigned int nHashStart = nHash % TABLE_SIZE;
    unsigned int nHashPos = nHashStart;

    while (HashTable[nHashPos].bExists)
    {
          if (HashATable[nHashPos] == (int) nHashC && HashBTable[nHashPos] == (int)
nHashD)
        {
            break;
            //查询与插入不同，此处不需修改
        }
        else
        {
            nHashPos = (nHashPos + 1) % TABLE_SIZE;
        }

        if (nHashPos == nHashStart)
        {
            break;
        }
    }

    if( key_array[nHashPos] && strlen(key_array[nHashPos]->pkey))
    {
        return key_array[nHashPos];
    }

    return NULL;
}

//按索引查询,如果成功返回关键字(此函数在本章中没有被用到,可以忽略)
key_list SearchByIndex(unsigned int nIndex)
{
    unsigned int nHashPos = nIndex;
    if (nIndex < TABLE_SIZE)
    {
        if(key_array[nHashPos] && strlen(key_array[nHashPos]->pkey))
        {
            return key_array[nHashPos];
        }
    }

    return NULL;
}
```

```
//插入关键字,如果成功返回hash值
int InsertString(const char *str)
{
    const int HASH_OFFSET = 0, HASH_A = 1, HASH_B = 2;
    unsigned int nHash = HashString(str, HASH_OFFSET);
    unsigned int nHashA = HashString(str, HASH_A);
    unsigned int nHashB = HashString(str, HASH_B);
    unsigned int nHashStart = nHash % TABLE_SIZE;
    unsigned int nHashPos = nHashStart;
    int len;

    while (HashTable[nHashPos].bExists)
    {
        nHashPos = (nHashPos + 1) % TABLE_SIZE;

        if (nHashPos == nHashStart)
            break;
    }

    len = strlen(str);
    if (!HashTable[nHashPos].bExists && (len < WORD_MAX_LEN))
    {
        HashATable[nHashPos] = nHashA;
        HashBTable[nHashPos] = nHashB;

        key_array[nHashPos] = (KEYNODE *) malloc (sizeof(KEYNODE) * 1);
        if(key_array[nHashPos] == NULL)
        {
            printf("10000 EMS ERROR !!!!\n");
            return 0;
        }

        key_array[nHashPos]->pkey = (char *)malloc(len+1);
        if(key_array[nHashPos]->pkey == NULL)
        {
            printf("10000 EMS ERROR !!!!\n");
            return 0;
        }

        memset(key_array[nHashPos]->pkey, 0, len+1);
        strncpy(key_array[nHashPos]->pkey, str, len);
        *((key_array[nHashPos]->pkey)+len) = 0;
        key_array[nHashPos]->pos = nHashPos;
        key_array[nHashPos]->count = 1;
        key_array[nHashPos]->next = NULL;
        HashTable[nHashPos].bExists = 1;
        return nHashPos;
    }
```

```
    if(HashTable[nHashPos].bExists)
        printf("30000 in the hash table %s !!!\n", str);
    else
        printf("90000 strkey error !!!\n");
    return -1;
}
```

有了这个Hash表，接下来，我们就可以把词插入Hash表进行存储了。

第三节、倒排索引文件的生成与实现

Hash表实现了（存于HashSearch.h中），还得编写一系列的函数，如下所示（所有代码还只是初步实现了功能，稍后在第四部分中将予以改进与优化）：

```
//处理空白字符和空白行
int GetRealString(char *pbuf)
{
    int len = strlen(pbuf) - 1;
      while (len > 0 && (pbuf[len] == (char)0x0d || pbuf[len] == (char)0x0a ||
pbuf[len] == ' ' || pbuf[len] == '\t'))
    {
        len--;
    }

    if (len < 0)
    {
        *pbuf = '\0';
        return len;
    }
    pbuf[len+1] = '\0';
    return len + 1;
}

//重新strcoll字符串比较函数
int strcoll(const void *s1, const void *s2)
{
    char *c_s1 = (char *)s1;
    char *c_s2 = (char *)s2;
    while (*c_s1 == *c_s2++)
    {
        if (*c_s1++ == '\0')
        {
            return 0;
```

```
        }
    }

    return *c_s1 - *--c_s2;
}

//从行缓冲中得到各项信息,将其写入items数组
void GetItems(char *&move, int &count, int &wordnum)
{
    char *front = move;
    bool flag = false;
    int len;
    move = strstr(move, "#####");
    if (*(move + 5) == '#')
    {
        flag = true;
    }

    if (move)
    {
        len = move - front;
        strncpy(items[count], front, len);
    }
    items[count][len] = '\0';
    count++;

    if (flag)
    {
        move = move + 10;
    } else
    {
        move = move + 5;
    }
}

//保存关键字相应的文档内容
doc_list SaveItems()
{
    doc_list infolist = (doc_list) malloc(sizeof(DOCNODE));
    strcpy_s(infolist->id, items[0]);
    infolist->classOne = atoi(items[1]);
    strcpy_s(infolist->classTwo, items[2]);
    infolist->classThree = atoi(items[3]);
    strcpy_s(infolist->time, items[4]);
    strcpy_s(infolist->md5, items[5]);
    infolist->weight = atoi(items[6]);
    return infolist;
}
```

```
//得到目录下所有文件名
int GetFileName(char filename[][FILENAME_MAX_LEN])
{
    _finddata_t file;
    long handle;
    int filenum = 0;
    //C:\Users\zhangxu\Desktop\CreateInvertedIndex\data
                                                    if              ((handle              =
_findfirst("C:\\Users\\zhangxu\\Desktop\\CreateInvertedIndex\\data\\*.txt",     &file))
== -1)
    {
        printf("Not Found\n");
    }
    else
    {
        do
        {
            strcpy_s(filename[filenum++], file.name);
        } while (!_findnext(handle, &file));
    }
    _findclose(handle);
    return filenum;
}

//以读方式打开文件,如果成功返回文件指针
FILE* OpenReadFile(int index, char filename[][FILENAME_MAX_LEN])
{
    char *abspath;
    char dirpath[] = {"data\\"};
    abspath = (char *)malloc(ABSPATH_MAX_LEN);
    strcpy_s(abspath, ABSPATH_MAX_LEN, dirpath);
    strcat_s(abspath, FILENAME_MAX_LEN, filename[index]);

    FILE *fp = fopen (abspath, "r");
    if (fp == NULL)
    {
        printf("open read file error!\n");
        return NULL;
    }
    else
    {
        return fp;
    }
}

//以写方式打开文件,如果成功返回文件指针
FILE* OpenWriteFile(const char *in_file_path)
```

```
{
    if (in_file_path == NULL)
    {
        printf("output file path error!\n");
        return NULL;
    }

    FILE *fp = fopen(in_file_path, "w+");
    if (fp == NULL)
    {
        printf("open write file error!\n");
    }
    return fp;
}
```

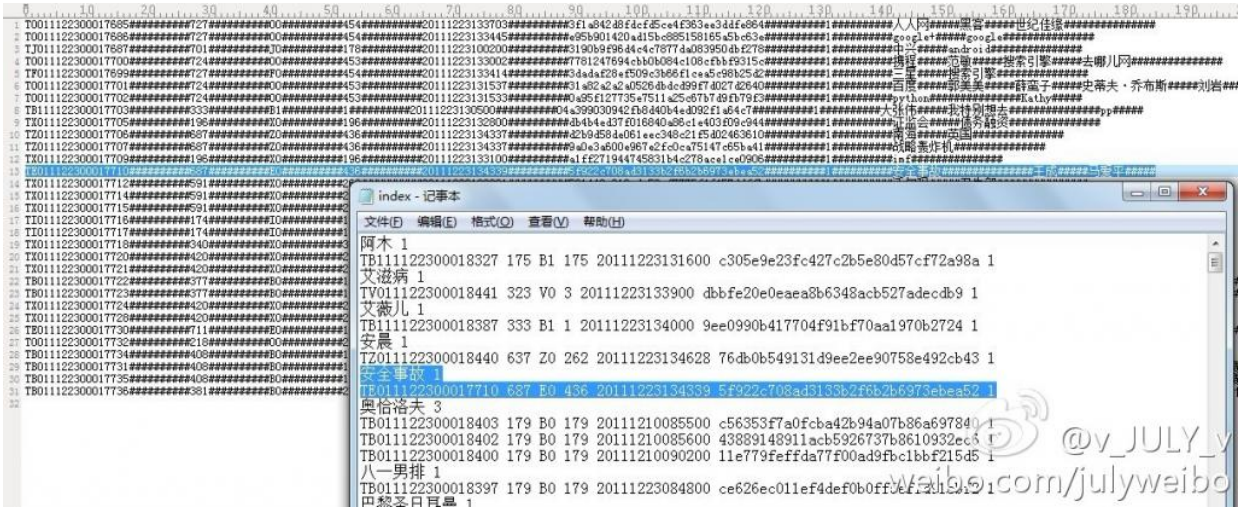
最后,主函数编写如下:

```
int main()
{
    key_list keylist;
    char *pbuf, *move;
    int filenum = GetFileName(filename);
    FILE *fr;
    pbuf = (char *)malloc(BUF_MAX_LEN);
    memset(pbuf, 0, BUF_MAX_LEN);

    FILE *fw = OpenWriteFile("index.txt");
    if (fw == NULL)
    {
        return 0;
    }

    PrepareCryptTable();    //初始化Hash表

    int wordnum = 0;
    for (int i = 0; i < filenum; i++)
    {
        fr = OpenReadFile(i, filename);
        if (fr == NULL)
        {
            break;
        }

        // 每次读取一行处理
        while (fgets(pbuf, BUF_MAX_LEN, fr))
        {
            int count = 0;
```

```
        move = pbuf;
        if (GetRealString(pbuf) <= 1)
            continue;

        while (move != NULL)
        {
            // 找到第一个非'#'的字符
            while (*move == '#')
                move++;

            if (!strcmp(move, ""))
                break;

            GetItems(move, count, wordnum);
        }

        for (int i = 7; i < count; i++)
        {
            // 将关键字对应的文档内容加入文档结点链表中
            if (keylist = SearchByString(items[i]))      //到hash表内查询
            {
                doc_list infolist = SaveItems();
                infolist->next = keylist->next;
                keylist->count++;
                keylist->next = infolist;
            }
            else
            {
                // 如果关键字第一次出现,则将其加入hash表
                int pos = InsertString(items[i]);        //插入hash表
                keylist = key_array[pos];
                doc_list infolist = SaveItems();
                infolist->next = NULL;
                keylist->next = infolist;
                if (pos != -1)
                {
                    strcpy_s(words[wordnum++], items[i]);
                }
            }
        }
    }
}

// 通过快排对关键字进行排序
qsort(words, WORD_MAX_NUM, WORD_MAX_LEN, strcoll);

// 遍历关键字数组,将关键字及其对应的文档内容写入文件中
for (int i = 0; i < WORD_MAX_NUM; i++)
```

```
    {
        keylist = SearchByString(words[i]);
        if (keylist != NULL)
        {
            fprintf(fw, "%s %d\n", words[i], keylist->count);
            doc_list infolist = keylist->next;
            for (int j = 0; j < keylist->count; j++)
            {
                //文档ID,订阅源（子频道） 频道分类 网站类ID(大频道）时间  md5,文档权值
                    fprintf(fw, "%s %d %s %d %s %s %d\n", infolist->id, infolist->classOne,
                            infolist->classTwo, infolist->classThree, infolist->time, infolist->md5, infolist->weight);
                infolist = infolist->next;
            }
        }
    }

    free(pbuf);
    fclose(fr);
    fclose(fw);
    system("pause");
    return 0;
}
```

程序编译运行后，生成的倒排索引文件为index.txt，其与原来给定的正排文档对照如下：



有没有发现关键词奥恰洛夫出现在的三篇文章是同一个日期1210的，貌似与本文开头指定的倒排索引格式要求不符？因为第二部分开头中，已明确说明：“注，关键词所在的文章如果是同一个日期的话，是挨在同一

行的，用“#”符号隔开”。OK，有疑问是好事，代表你思考了，请直接转至下文第4部分。

第四节、程序需求功能的改进

对相同日期与不同日期的处理

细心的读者可能还是会注意到：在第二部分开头中，要求基于给定的上述正排文档。生成如第二十四章所示的倒排索引文件是下面这样子的，即是：

也就是说，上面建索引的过程本该是如下的：

与第一部分所述的SMIPI算法有什么区别？对的，就在于对在同一个日期的出现的关键词的处理。如果是遇一旧词，则找到其倒排记录表的位置：相同日期，添加到之前同一日期的记录之后（第一个记录的后面记下同一日期的记录数目）；不同日期，另起一行新增记录。

- 相同（单个）日期，根据文档权值排序
- 不同日期，根据时间排序

代码主要修改如下：

```
//function: 对链表进行冒泡排序
void ListSort(key_list keylist)
{
    doc_list p = keylist->next;
    doc_list final = NULL;
    while (true)
    {
        bool isfinish = true;
        while (p->next != final) {
            if (strcmp(p->time, p->next->time) < 0)
            {
                SwapDocNode(p);
                isfinish = false;
            }
            p = p->next;
        }
        final = p;
        p = keylist->next;
        if (isfinish || p->next == final) {
            break;
        }
    }
}

int main()
{
    key_list keylist;
    char *pbuf, *move;
    int filenum = GetFileName(filename);
    FILE *frp;
    pbuf = (char *)malloc(BUF_MAX_LEN);
    memset(pbuf, 0, BUF_MAX_LEN);

    FILE *fwp = OpenWriteFile("index.txt");
    if (fwp == NULL) {
        return 0;
    }

    PrepareCryptTable();

    int wordnum = 0;
    for (int i = 0; i < filenum; i++)
    {
        frp = OpenReadFile(i, filename);
        if (frp == NULL) {
            break;
```

```
    }

    // 每次读取一行处理
    while (fgets(pbuf, BUF_MAX_LEN, frp))
    {
        int count = 0;
        move = pbuf;
        if (GetRealString(pbuf) <= 1)
            continue;

        while (move != NULL)
        {
            // 找到第一个非'#'的字符
            while (*move == '#')
                move++;

            if (!strcmp(move, ""))
                break;

            GetItems(move, count, wordnum);
        }

        for (int i = 7; i < count; i++) {
            // 将关键字对应的文档内容加入文档结点链表中
            // 如果关键字第一次出现,则将其加入hash表
            if (keylist = SearchByString(items[i])) {
                doc_list infolist = SaveItems();
                infolist->next = keylist->next;
                keylist->count++;
                keylist->next = infolist;
            } else {
                int pos = InsertString(items[i]);
                keylist = key_array[pos];
                doc_list infolist = SaveItems();
                infolist->next = NULL;
                keylist->next = infolist;
                if (pos != -1) {
                    strcpy_s(words[wordnum++], items[i]);
                }
            }
        }
    }
}

// 通过快排对关键字进行排序
qsort(words, WORD_MAX_NUM, WORD_MAX_LEN, strcoll);

// 遍历关键字数组,将关键字及其对应的文档内容写入文件中
```

```
    int rownum = 1;
    for (int i = 0; i < WORD_MAX_NUM; i++) {
        keylist = SearchByString(words[i]);
        if (keylist != NULL) {
            doc_list infolist = keylist->next;

            char date[9];

            // 截取年月日
            for (int j = 0; j < keylist->count; j++)
            {
                strncpy_s(date, infolist->time, 8);
                date[8] = '\0';
                strncpy_s(infolist->time, date, 9);
                infolist = infolist->next;
            }

            // 对链表根据时间进行排序
            ListSort(keylist);

            infolist = keylist->next;
            int *count = new int[WORD_MAX_NUM];
            memset(count, 0, WORD_MAX_NUM);
            strcpy_s(date, infolist->time);
            int num = 0;
            // 得到单个日期的文档数目
            for (int j = 0; j < keylist->count; j++)
            {
                if (strcmp(date, infolist->time) == 0) {
                    count[num]++;
                } else {
                    count[++num]++;
                }
                strcpy_s(date, infolist->time);
                infolist = infolist->next;
            }
            fprintf(fwp, "%s %d %d\n", words[i], num + 1, rownum);
            WriteFile(keylist, num, fwp, count);
            rownum++;
        }
    }

    free(pbuf);
//  fclose(frp);
    fclose(fwp);
    system("pause");
    return 0;
}
```

修改后编译运行，生成的index.txt文件如下：

为关键词添上编码

如上图所示，已经满足需求了。但可以再在每个关键词的背后添加一个计数表示索引到了第多少个关键词：

第五节、算法的二次改进

省去二次Hash

针对本文评论下读者的留言，做了下思考，自觉可以省去二次hash：

```
        for (int i = 7; i < count; i++)
        {
            // 将关键字对应的文档内容加入文档结点链表中
            //也就是说当查询到hash表中没有某个关键词之,后便会插入
            //而查询的时候,search会调用hashstring,得到了nHashC ,nHashD
            //插入的时候又调用了一次hashstring,得到了nHashA,nHashB
                //而如果查询的时候,是针对同一个关键词查询的,所以也就是说nHashC&nHashD,与nHashA&nHashB是相同的,无需二次hash
            //所以,若要改进,改的也就是下面这个if~else语句里头。July,2011.12.30。
            if (keylist = SearchByString(items[i]))      //到hash表内查询
            {
                doc_list infolist = SaveItems();
                infolist->next = keylist->next;
                keylist->count++;
                keylist->next = infolist;
            }
            else
            {
                // 如果关键字第一次出现,则将其加入hash表
                int pos = InsertString(items[i]);       //插入hash表
                keylist = key_array[pos];
                doc_list infolist = SaveItems();
                infolist->next = NULL;
                keylist->next = infolist;
                if (pos != -1)
                {
                    strcpy_s(words[wordnum++], items[i]);
                }
            }
        }
    }
}
// 通过快排对关键字进行排序
qsort(words, WORD_MAX_NUM, WORD_MAX_LEN, strcoll);
```

除去排序,针对不同日期的记录直接插入

```
//对链表进行冒泡排序。这里可以改成快速排序：等到统计完所有有关这个关键词的文章之后，才能对他集体快排。
//但其实完全可以用插入排序,不同日期的,根据时间的先后找到插入位置进行插入：
//假如说已有三条不同日期的记录 A B C
//来了D后,发现D在C之前,B之后,那么就必须为它找到B C之间的插入位置,
```

```
//A B D C°July´2011.12.31°
void ListSort(key_list keylist)
{
    doc_list p = keylist->next;
    doc_list final = NULL;
    while (true)
    {
        bool isfinish = true;
        while (p->next != final) {
            if (strcmp(p->time, p->next->time) < 0) //不同日期的按最早到最晚排序
            {
                SwapDocNode(p);
                isfinish = false;
            }
            p = p->next;
        }
        final = p;
        p = keylist->next;
        if (isfinish || p->next == final) {
            break;
        }
    }
}
```

综上两节免去冒泡排序和，省去二次hash和免去冒泡排序,修改后如下:

```
        for (int i = 7; i < count; i++) {
            // 将关键字对应的文档内容加入文档结点链表中
            // 如果关键字第一次出现,则将其加入hash表
            InitHashValue(items[i], hashvalue);
            if (keynode = SearchByString(items[i], hashvalue)) {
                doc_list infonode = SaveItems();
                doc_list p = keynode->next;
                // 根据时间由早到晚排序
                if (strcmp(infonode->time, p->time) < 0) {
                    //考虑infonode插入keynode后的情况
                    infonode->next = p;
                    keynode->next = infonode;
                } else {
                    //考虑其他情况
                    doc_list pre = p;
                    p = p->next;
                    while (p)
                    {
                        if (strcmp(infonode->time, p->time) > 0) {
                            p = p->next;
                            pre = pre->next;
```

```
                        } else {
                            break;
                        }
                    }
                    infonode->next = p;
                    pre->next = infonode;
                }
                keynode->count++;
            } else {
                int pos = InsertString(items[i], hashvalue);
                keynode = key_array[pos];
                doc_list infolist = SaveItems();
                infolist->next = NULL;
                keynode->next = infolist;
                if (pos != -1) {
                    strcpy_s(words[wordnum++], items[i]);
                }
            }
        }
    }
}

// 通过快排对关键字进行排序
qsort(words, WORD_MAX_NUM, WORD_MAX_LEN, strcoll);
```

修改后编译运行的效果图如下(用了另外一份更大的数据文件进行测试)：

第三十六章、搜索关键词智能提示suggestion

题目详情：百度搜索框中，输入“北京”，搜索框下面会以北京为前缀，展示“北京爱情故事”、“北京公交”、“北京医院”等等搜索词，输入“结构之”，会提示“结构之法”，“结构之法 算法之道”等搜索词。请问，如何设计此系统，使得空间和时间复杂度尽量低。

题目分析：本题来源于去年2012年百度的一套实习生笔试题中的系统设计题（*为尊重原题，本章主要使用百度搜索引擎展开论述，而不是google等其它搜索引擎，但原理不会差太多。然脱离本题，平时搜的时候，鼓励用...*），题目比较开放，考察的目的在于看应聘者解决问题的思路是否清晰明确，其次便是看能考虑到多少细节。

我去年整理此题的时候，曾简单解析过，提出的方法是：

- 直接上**Trie树**「Trie树的介绍见：从Trie树（字典树）谈到后缀树」+ **TOP K**「hashmap+堆，hashmap+堆 统计出如10个近似的热词，也就是说，只存与关键词近似的比如10个热词」

方法就是这样子的：Trie树+TOP K算法，但在实际中，真的只要Trie树 + TOP K算法就够了么，有什么需要考虑的细节？OK，请看下文娓娓道来。

解法一、Trie树 + TOP K

步骤一、trie树存储前缀后缀

若看过博客内这篇介绍Trie树和后缀树的文章的话，应该就能对trie树有个大致的了解，为示本文完整性，引用下原文内容，如下：

1.1、什么是Trie树

Trie树，即字典树，又称单词查找树或键树，是一种树形结构，是一种哈希树的变种。典型应用是用于统计和排序大量的字符串（但不仅限于字符串），所以经常被搜索引擎系统用于文本词频统计。它的优点是：最大限度地减少无谓的字符串比较，查询效率往往比哈希表高。

Trie的核心思想是空间换时间。利用字符串的公共前缀来降低查询时间的开销以达到提高效率的目的。

它有3个基本性质：

1. 根节点不包含字符，除根节点外每一个节点都只包含一个字符。
2. 从根节点到某一节点，路径上经过的字符连接起来，为该节点对应的字符串。
3. 每个节点的所有子节点包含的字符都不相同。

1.2、 树的构建

举个在网上流传颇广的例子，如下：

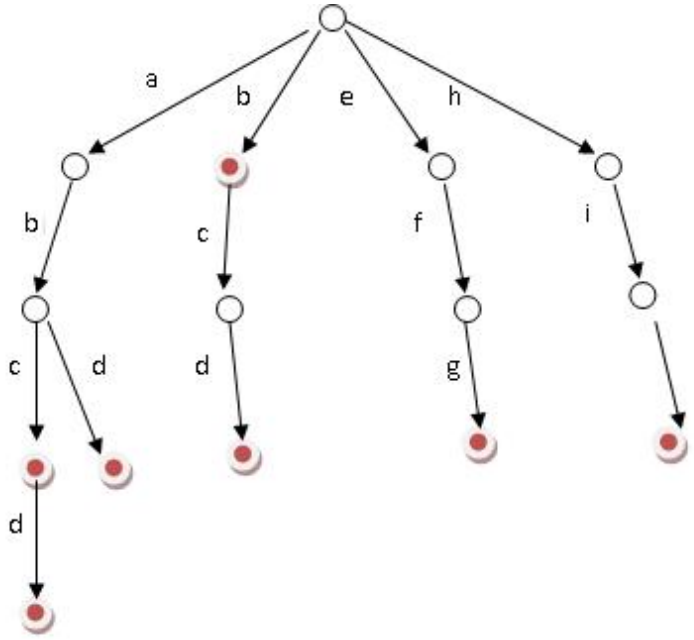
题目：给你100000个长度不超过10的单词。对于每一个单词，我们要判断他出没出现过，如果出现了，求第一次出现在第几个位置。

分析：这题当然可以用hash来解决，但是本文重点介绍的是trie树，因为在某些方面它的用途更大。比如说对于某一个单词，我们要询问它的前缀是否出现过。这样hash就不好搞了，而用trie还是很简单。

现在回到例子中，如果我们用最傻的方法，对于每一个单词，我们都要去查找它前面的单词中是否有它。那么这个算法的复杂度就是

O(n^2)。(字符串比较的复杂度是否以strcmp次数计算更妥当，当此处为了讨论简化了细节)。显然对于100000的范围难以接受。现在我们换个思路想。假设我要查询的单词是abcd，那么在他前面的单词中，以b，c，d，f之类开头的我显然不必考虑。而只要找以a开头的中是否存在abcd就可以了。同样的，在以a开头中的单词中，我们只要考虑以b作为第二个字母的，一次次缩小范围和提高针对性，这样一个树的模型就渐渐清晰了。

好比假设有b，abc，abd，bcd，abcd，efg，hii 这6个单词，我们构建的树就是如下图这样的：



当时第一次看到这幅图的时候，便立马感到此树之不凡构造了。单单从上幅图便可窥知一二，好比大海搜人，立马就能确定东南西北中的到底哪个方位，如此迅速缩小查找的范围和提高查找的针对性，不失为一创举。

ok，如上图所示，对于每一个节点，从根遍历到他的过程就是一个单词，如果这个节点被标记为红色，就表示这个单词存在，否则不存在。

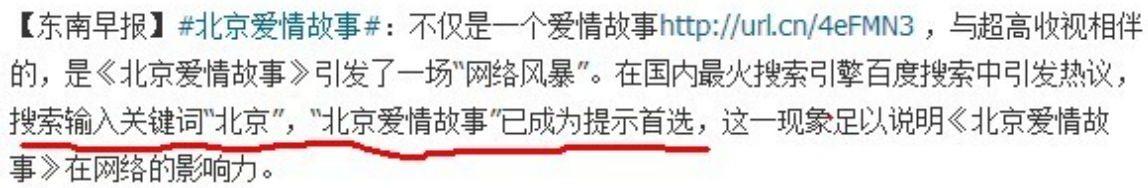
那么，对于一个单词，我只要顺着他从根走到对应的节点，再看这个节点是否被标记为红色就可以知道它是否出现过了。把这个节点标记为红色，就相当于插入了这个单词。

借用上面的图，当用户输入前缀a的时候，搜索框可能会展示以a为前缀的“abcd”，“abd”等关键词，再当用户输入前缀b的时候，搜索框下面可能会提示以b为前缀的“bcd”等关键词，如此，实现搜索引擎智能提示suggestion的第一个步骤便清晰了，即用trie树存储大量字符串，当前缀固定时，存储相对来说比较热的后缀。那又如何统计热词呢？请看下文步骤二、TOP K算法统计热词。

步骤二、TOP K算法统计热词

当每个搜索引擎输入一个前缀时，下面它只会展示0~10个候选词，但若是碰到那种候选词很多的时候，如何取舍，哪些展示在前面，哪些展示在后面？这就是一个搜索热度的问题。

如本题描述所说，在去年的这个时候，当我在搜索框内搜索“北京”时，它下面会提示以“北京”为前缀的诸如“北京爱情故事”，“北京公交”，“北京医院”，且“ 北京爱情故事”展示在第一个：



为何输入“北京”，会首先提示“北京爱情故事”呢？因为去年的这个时候，正是《北京爱情故事》这部电视剧上映正火的时候（其上映日期为2012年1月8日，火了至少一年），那个时候大家都一个劲的搜索这部电视剧的相关信息，当10个人中输入“北京”后，其中有8个人会继续敲入“爱情故事”（连起来就是“北京爱情故事”）的时候，搜索引擎对此当然不会无动于衷。

也就是说，搜索引擎知道了这个时间段，大家都在疯狂查找北京爱情故事，故当用户输入以“北京”为前缀的时候，搜索引擎猜测用户有80%的机率是要查找“北京爱情故事”，故把“北京爱情故事”在下面提示出来，并放在第一个位置上。

但为何今年这个时候再次搜索“北京”的时候，它展示出来的词不同了呢？

原因在于随着时间变化，人们对《北京爱情故事》这部电视剧的关注度逐渐下降，与此同时，又出现了新的热词，或新的电影，故现在虽然同样是输入“北京”，后面提示的词也相应跟着起了变化。那解决这个问题的办法是什么呢？如开头所说：定期分析某段时间内的人们搜索的关键词，统计出搜索次数比较多的热词，继而当用户输入某个前缀时，优先展示热词。

故说白了，这个问题的第二个步骤便是统计热词，我们把统计热词的方法称为TOP K算法，此算法的应用场景便是此文中的第2个问题，再次原文引用：

寻找热门查询，300万个查询字符串中统计最热门的10个查询

原题：搜索引擎会通过日志文件把用户每次检索使用的所有检索串都记录下来，每个查询串的长度为1-255字节。假设目前有一千万个记录（这些查询串的重复度比较高，虽然总数是1千万，但如果除去重复后，不超过3百万个。一个查询串的重复度越高，说明查询它的用户越多，也就是越热门），请你统计最热门的10个查询串，要求使用的内存不能超过1G。

解答：由上面第1题，我们知道，数据大则划为小的，如一亿个Ip求Top 10，可先将Ip地址使用Ip2long函数（各种语言或库皆提供本函数）转为长整型Ipld后，再按Ipld%1000将ip分到1000个小文件中去，并保证一种ip只出现在一个文件中，再对每个小文件中的ip进行hashmap计数统计并按数量排序，最后归并或者最小堆依次处理每个小文件的top10以得到最后的结果。（小提示：常用的Ip hash还有一种基于bit的方法，聪明的你一定想到了吧^_^）

但如果数据规模本身就比较小，能一次性装入内存呢？比如这第2题，虽然有一千万个Query，但是由于重复度比较高，因此事实上只有300万的Query，每个Query255Byte，因此我们可以考虑把他们都放进内存中去（300万个字符串假设没有重复，都是最大长度，那么最多占用内存3M*1K/4=0.75G（实际占用会略大一点，因具体实现而异）。所以可以将所有字符串都存放在内存中进行处理），而现在只是需要一个合适的数据结构，在这里，HashTable绝对是我们优先的选择。

所以对数据量较少（可以全部放入内存的时候）我们放弃分而治之+hash映射的步骤，直接上hash统计，然后排序。针对此类典型的TOP K问

题，采取的对策往往是：hashmap + 堆。如下所示：

1. **hashmap统计：** 先对这批海量数据预处理。具体方法是：维护一个Key为Query字串，Value为该Query出现次数的HashTable，即hash_map(Query, Value)，每次读取一个Query，如果该字串不在Table中，那么加入该字串，并且将Value值设为1；如果该字串在Table中，那么将该字串的计数加一即可。最终我们在O(N)的时间复杂度内用Hash表完成了统计；
2. **堆排序：**第二步、借助堆这个数据结构，找出Top K，时间复杂度为N‘logK。即借助堆结构，我们可以在对数量级的时间内查找和调整。因此，维护一个K(该题目中是10)大小的小根堆，然后遍历300万的Query对应的计数，分别和根元素进行对比。所以，我们最终的时间复杂度是：O（N）+ N’ * O（logK），（N为1000万，N’为300万）。

别忘了这篇文章中所述的堆排序思路：‘维护k个元素的最小堆，即用容量为k的最小堆存储最先遍历到的k个数，并假设它们即是最大的k个数，建堆费时O（k），并调整堆(费时O（logk）)后，有k1>k2>…kmin（kmin设为小顶堆中最小元素）。继续遍历数列，每次遍历一个元素x，与堆顶元素比较，若x>kmin，则更新堆（x入堆，用时logk），否则不更新堆。这样下来，总费时O（k*logk+（n-k）*logk）=O（n*logk）。此方法得益于在堆中，查找等各项操作时间复杂度均为logk。’–第三章续、Top K算法问题的实现。

当然，你也可以采用trie树，关键字域存该查询串出现的次数，没有出现为0。最后用10个元素的最小推来对出现频率进行排序。

相信，如此，也就不难理解开头所提出的方法了：Trie树+ TOP K「hashmap+堆，hashmap+堆 统计出如10个近似的热词，也就是说，只存与关键词近似的比如10个热词」。

而且你以后就可以告诉你身边的伙伴们，为何输入“结构之”，会提示出来一堆以“结构之”为前缀的词了：

方法貌似成型了，但有哪些需要注意的细节呢？如@江申_Johnson所说：“实际工作里，比如当前缀很短的时候，候选词很多的时候，查询和排序性能可能有问题，也许可以加一层索引trie（这层索引可以只索引频率高于某一个阈值的词，很短的时候查这个就可以了。数量不够的话再去查索引了全部词的trie树）；而且有时候不能根据query频率来排，而要引导用户输入信息量更全面的query，或者或不仅仅是前缀匹配这么简单。”

当然，在实际的工程中，排序的依据还有很多，例如对于突发的热点问题，虽然之前用户没有输入过，但是却能排在最前面，是因为在系统内部有一些相应的模块进行了所谓的“调权”。此外，上面我们谈到trie的时候，都采取了简化的模型，即trie通常只是针对英语单词而言的，如果是中文，会有一个严重的问题：），你想到了吗？（小提示：英语和中文的基本字符，差别是不是很大？）当然，如果考虑到我们喜欢用拼音来表述中文，这个问题不是没有解决方法的^_^。

此外，在工程实现的时候，前端往往采用了ajax技术来保障速度上的优势，不然等用户输入完毕了咱的推荐词还在慢悠悠的传输中，那也就没

有多少意义了。由于前端讨论和咱们这里的讨论关系不大，感兴趣的朋友可以参考相关资料。

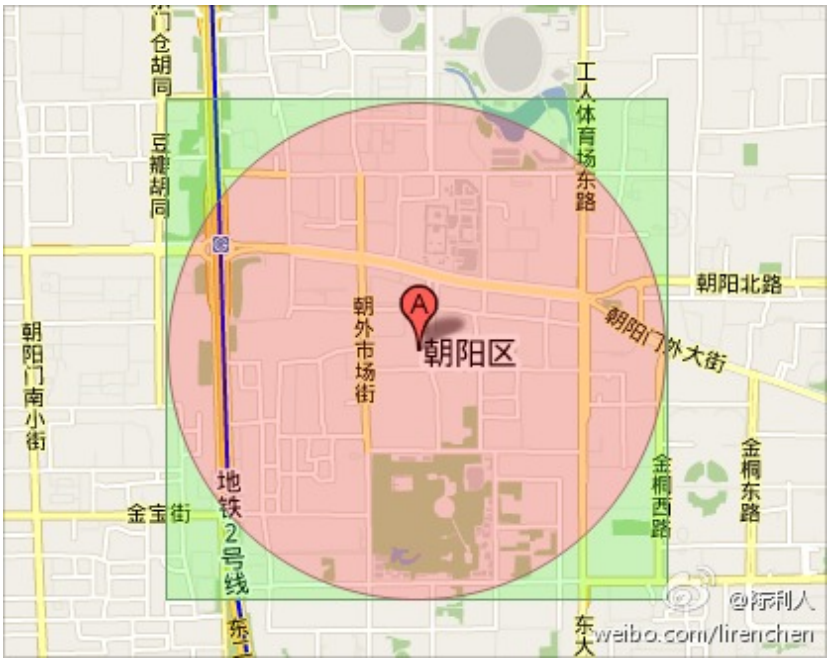
扩展阅读

除了上文提到的trie树，三叉树或许也是一个不错的解决方案：点击此处。此外，StackOverflow上也有两个讨论帖子，大家可以看看：帖子1，帖子2。

第三十七章、附近地点搜索

题目详情：找一个点集中与给定点距离最近的点，同时，给定的二维点集都是固定的，查询可能有很多次，时间复杂度O(n)无法接受，请设计数据结构和相应的算法。

题目分析：此题是去年微软的三面题，类似于一朋友@陈利人 出的这题：附近地点搜索，就是搜索用户附近有哪些地点。随着GPS和带有GPS功能的移动设备的普及，附近地点搜索也变得炙手可热。在庞大的地理数据库中搜索地点，索引是很重要的。但是，我们的需求是搜索附近地点，例如，坐标(39.91, 116.37)附近500米内有什么餐馆，那么让你来设计，该怎么做？



解法一、R树二维搜索

假定只允许你初中数学知识，那么你可能建一个X-Y坐标系，即以坐标(39.91, 116.37)为圆心，以500的长度为半径，画一个园，然后一个一个坐标点的去查找。此法看似可行，但复杂度可想而知，即便你自以为聪明的说把整个平面划分为四个象限，一个一个象限的查找，此举虽然优化程度不够，但也说明你一步步想到点子上去了。

即不一个一个坐标点的查找，而是一个一个区域的查找，相对来说，其平均查找速度和效率会显著提升。如此，便自然而然的想到了有没有一种一次查找定位于一个区域的数据结构呢？

若看过博客内之前介绍R树的这篇文章的读者立马便能意识到，R树就是解决这个区域查找继而不断缩小规模的问题。特直接引用原文：

> **R树的数据结构**
>
> R树是B树在高维空间的扩展，是一棵平衡树。每个R树的叶子结点包含了多个指向不同数据的指针，这些数据可以是存放在硬盘中的，也可以是存在内存中。根据R树的这种数据结构，当我们需要进行一个高维空间查询时，我们只需要遍历少数几个叶子结点所包含的指针，查看这些指针指向的数据是否满足要求即可。这种方式使我们不必遍历所有数据即可获得答案，效率显著提高。下图1是R树的一个简单实例：

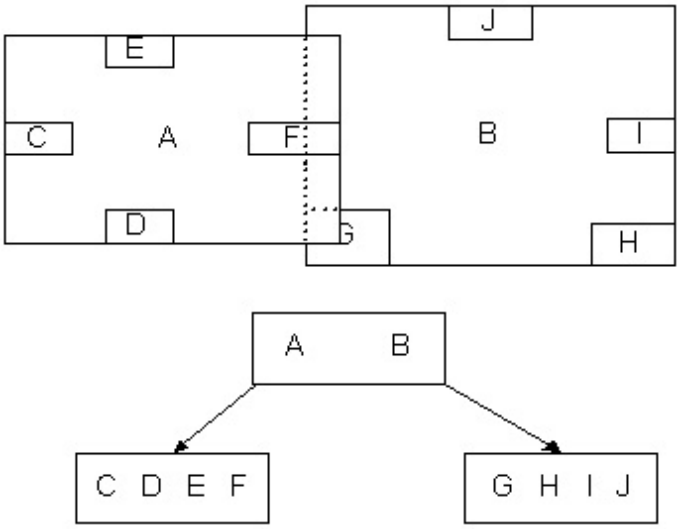
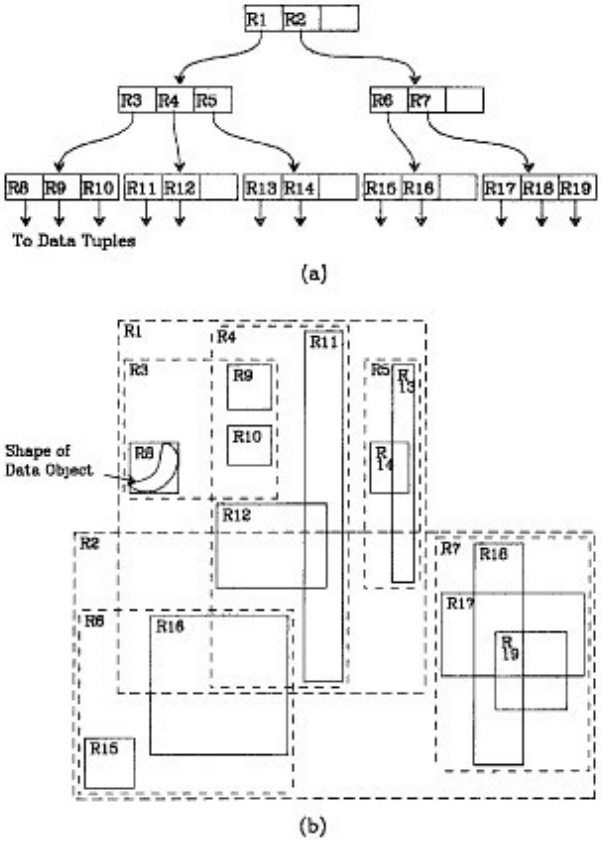


图1：一个R树实例

我们在上面说过，R树运用了空间分割的理念，这种理念是如何实现的呢？R树采用了一种称为MBR(Minimal Bounding Rectangle)的方法，在此我把它译作“最小边界矩形”。从叶子结点开始用矩形（rectangle）将空间框起来，结点越往上，框住的空间就越大，以此对空间进行分割。有点不懂？没关系，继续往下看。在这里我还想提一下，R树中的R应该代表的是Rectangle（此处参考wikipedia上关于R树的介绍），而不是大多数国内教材中所说的Region（很多书把R树称为区域树，这是有误的）。我们就拿二维空间来举例。下图是Guttman论文中的一幅图：



我来详细解释一下这张图。

1. 先来看图(b)，首先我们假设所有数据都是二维空间下的点，图中仅仅标志了R8区域中的数据，也就是那个shape of data object。别把那一块不规则图形看成一个数据，我们把它看作是多个数据围成的一个区域。为了实现R树结构，我们用一个最小边界矩形恰好框住这个不规则区域，这样，我们就构造出了一个区域：R8。R8的特点很明显，就是正正好好框住所有在此区域中的数据。
2. 其他实线包围住的区域，如R9，R10，R12等都是同样的道理。这样一来，我们一共得到了12个最最基本的最小矩形。这些矩形都将被存储在子结点中。
3. 下一步操作就是进行高一层次的处理。我们发现R8，R9，R10三个矩形距离最为靠近，因此就可以用一个更大的矩形R3恰好框住这3个矩形。
4. 同样道理，R15，R16被R6恰好框住，R11，R12被R4恰好框住，等等。所有最基本的最小边界矩形被框入更大的矩形中之后，再次迭代，

用更大的框去框住这些矩形。

我想大家都应该理解这个数据结构的特征了。用地图的例子来解释，就是所有的数据都是餐厅所对应的地点，先把相邻的餐厅划分到同一块区域，划分好所有餐厅之后，再把邻近的区域划分到更大的区域，划分完毕后再次进行更高层次的划分，直到划分到只剩下两个最大的区域为止。要查找的时候就方便了。

下面就可以把这些大大小小的矩形存入我们的R树中去了。根结点存放的是两个最大的矩形，这两个最大的矩形框住了所有的剩余的矩形，当然也就框住了所有的数据。下一层的结点存放了次大的矩形，这些矩形缩小了范围。每个叶子结点都是存放的最小的矩形，这些矩形中可能包含有n个数据。

地图查找的实例

讲完了基本的数据结构，我们来讲个实例，如何查询特定的数据。又以餐厅为例，假设我要查询广州市天河区天河城附近一公里的所有餐厅地址怎么办？

1. 打开地图（也就是整个R树），先选择国内还是国外（也就是根结点）；
2. 然后选择华南地区（对应第一层结点），选择广州市（对应第二层结点），
3. 再选择天河区（对应第三层结点）；
4. 最后选择天河城所在的那个区域（对应叶子结点，存放有最小矩形）；

遍历所有在此区域内的结点，看是否满足我们的要求即可。怎么样，其实R树的查找规则跟查地图很像吧？对应下图：

一棵R树满足如下的性质：

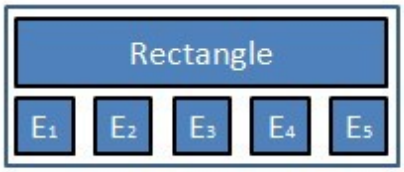
1. 除非它是根结点之外，所有叶子结点包含有m至M个记录索引（条目）。作为根结点的叶子结点所具有的记录个数可以少于m。通常，m=M/2。
2. 对于所有在叶子中存储的记录（条目），I是最小的可以在空间中完全覆盖这些记录所代表的点的矩形（注意：此处所说的“矩形”是可以扩展到高维空间的）。
3. 每一个非叶子结点拥有m至M个孩子结点，除非它是根结点。
4. 对于在非叶子结点上的每一个条目，i是最小的可以在空间上完全覆盖这些条目所代表的店的矩形（同性质2）。
5. 所有叶子结点都位于同一层，因此R树为平衡树。

叶子结点的结构

先来探究一下叶子结点的结构。叶子结点所保存的数据形式为：(I, tuple-identifier)。

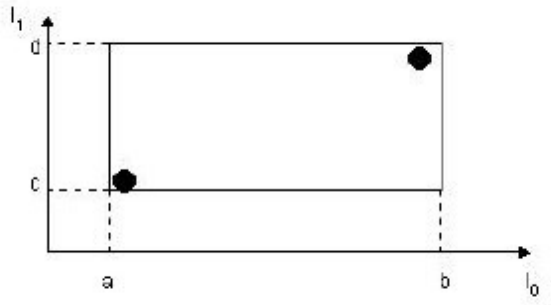
其中，tuple-identifier表示的是一个存放于数据库中的tuple，也就是一条记录，它是n维的。I是一个n维空间的矩形，并可以恰好框住这个叶子结

点中所有记录代表的n维空间中的点。I=(I0,I1,…,In-1)。其结构如下图所示：



R树中结点的存储结构。E代表Entry，即指向孩子结点的条目

下图描述的就是在二维空间中的叶子结点所要存储的信息。



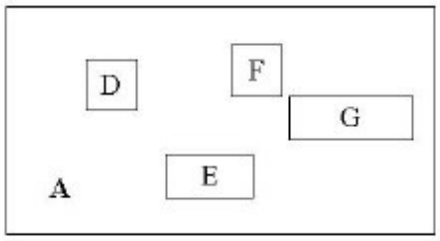
在这张图中，I所代表的就是图中的矩形，其范围是a<=I0<=b，c<=I1<=d。有两个tuple-identifier，在图中即表示为那两个点。这种形式完全可以推广到高维空间。大家简单想想三维空间中的样子就可以了。这样，叶子结点的结构就介绍完了。

非叶子结点

非叶子结点的结构其实与叶子结点非常类似。想象一下B树就知道了，B树的叶子结点存放的是真实存在的数据，而非叶子结点存放的是这些数据的“边界”，或者说也算是一种索引（有疑问的读者可以回顾一下上述第一节中讲解B树的部分）。

同样道理，R树的非叶子结点存放的数据结构为：(I, child-pointer)。

其中，child-pointer是指向孩子结点的指针，I是覆盖所有孩子结点对应矩形的矩形。这边有点拗口，但我想不是很难懂？给张图：



D,E,F,G为孩子结点所对应的矩形。A为能够覆盖这些矩形的更大的矩形。这个A就是这个非叶子结点所对应的矩形。这时候你应该悟到了吧？无论是叶子结点还是非叶子结点，它们都对应着一个矩形。树形结构上层的结点所对应的矩形能够完全覆盖它的孩子结点所对应的矩形。根结点也唯一对应一个矩形，而这个矩形是可以覆盖所有我们拥有的数据信息在空间中代表的点的。

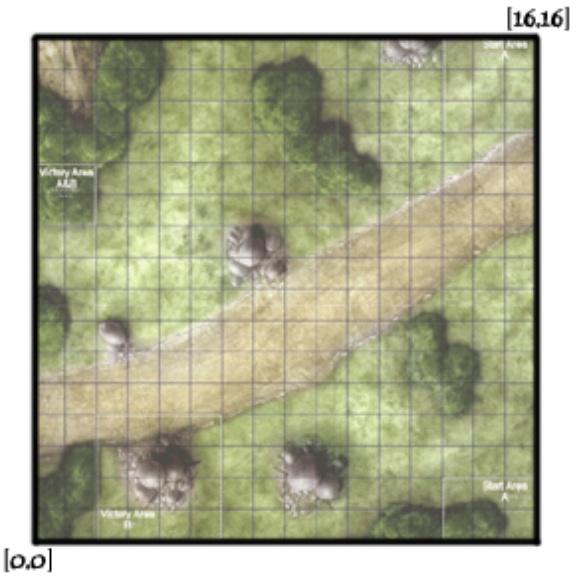
我个人感觉这张图画的不那么精确，应该是矩形A要恰好覆盖D,E,F,G，而不应该再留出这么多没用的空间了。但为尊重原图的绘制者，特不作修改。

但R树有些什么问题呢？如@宋枭_CD所说："单纯用R树来作索引，搜索附近的地点，可能会遍历树的很多个分支。而且当全国的地图或者全省的地图时候，树的叶节点数目很多，树的深度也会是一个问题。一般会把地理位置上附近的节点（二维地图中点线面）预处理成page(大小为4K的倍数)，在这些page上建立R树的索引。"

解法二、GeoHash算法索引地理位置信息

我在微博上跟一些朋友讨论这个附近点搜索的问题时，除了谈到R树，有几个朋友都指出GeoHash算法可以解决，故才了解了下GeoHash算法，此文 清晰阐述了MongoDB借助GeoHash算法实现地理位置索引的原理，特引用其内容加以说明，如下：

支持地理位置索引是MongoDB的一大亮点，这也是全球最流行的LBS服务foursquare 选择MongoDB的原因之一。我们知道，通常的数据库索引结构是B+ Tree，如何将地理位置转化为可建立B+Tree的形式。首先假设我们将需要索引的整个地图分成16×16的方格，如下图（左下角为坐标0,0 右上角为坐标16,16）：



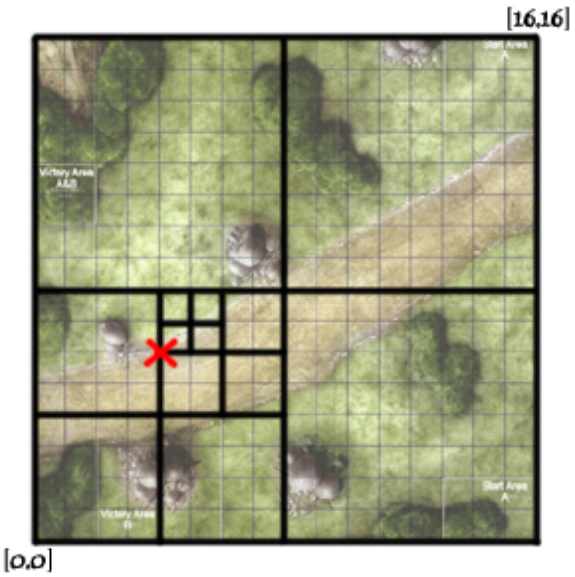
单纯的 [x, y] 的数据是无法建立索引的，所以MongoDB在建立索引的时候，会根据相应字段的坐标计算一个可以用来做索引的hash值，这个值叫做geohash，下面我们以地图上坐标为[4, 6]的点（图中红叉位置）为例。我们第一步将整个地图分成等大小的四块，如下图：

划分成四块后我们可以定义这四块的值，如下（左下为00，左上为01，右下为10，右上为11）：

01	11
00	10

这样 [4，6] 点的geohash值目前为 00然后再将四个小块每一块进行切割，如下：

这时 [4，6] 点位于右上区域，右上的值为11，这样 [4，6] 点的geohash值变为：0011继续往下做两次切分：



最终得到[4, 6]点的geohash值为：00110100

这样我们用这个值来做索引，则地图上同一个分块内相近的点就可以转化成有相同前缀的geohash值了。

我们可以看到，这个geohash值的精确度是与划分地图的次数成正比的，上例对地图划分了四次。而MongoDB默认是进行26次划分，这个值在建立索引时是可控的。具体建立二维地理位置索引的命令如下：

db.map.ensureIndex({point : “2d”}, {min : 0, max : 16, bits : 4})

其中的bits参数就是划分几次，默认为26次。

读者点评@yuotulck：首先多谢博主的文章，不过如果是新手（例如我）看到geohash那里可能会有误解：是否相邻可以靠前缀来比较？其实这是错的，例如边界那一块的相邻区域编码的前缀从第一个就不一样了，也就是说在geohash里相近的点hash值不一定相近。

上面的知识点了解自这篇文章，而geohash的进一步用法在这里可以了解到。

本章完。

第八章导读

在前面的第二章，我们介绍了数组，数组因为是下标定位，所以查找极快，而hashtable(key,value)借鉴数组O(1)定位的特点，key映射value，所以查找也相当快，只是需要耗费一定的空间存储key和value，所以是典型的空间换时间的思想。

其中，理解红黑树，先理解二叉查找树和2-3树，为何？首先，二叉查找树中的结点是2-结点（一个键两条链），引入3-结点（两个键三条链），即成2-3树；再者，将2-3树中3-结点分解，即成红黑树，故结合二叉查找树易查找和2-3树易插入的特点，便成了红黑二叉查找树，简称红黑树。

进一步，理解了2-3树，也就理解了B树、B+树、B*树，因为2-3树就是一棵3阶的B树，而一颗3阶的B树各个结点关键字数满足1-2，故结点关键字数多于2达到饱和时分裂，结点关键字数少于1时则从兄弟结点“借”关键字补充。

但为何有了红黑树，还要出现B树呢？原因是当计算机要处理的数据量一大，便无法一次性装入内存处理，于是，计算机会把大部分备用的数据存在磁盘中，有需要的时候，就从磁盘调取到在内存中处理，如果处理时修改了数据，则再次写会磁盘，如此导致了不断的磁盘IO读写，而树的高度越到，查找文件所需要的磁盘IO读写次数越多。

因此，为了进一步降低树的高度，具有多个孩子的B树的应运而生，正因为B树每一个结点可以有几个到几千个孩子，在结点数目一定的情况下，树的高度会大大降低，此外，B树的根结点和部分顶层数据在内存中，大部分下层数据在磁盘上。

详情请阅读第8章全部内容。

Trie树（字典树）

引言

常关注本blog的读者朋友想必看过此篇文章：从B树、B+树、B*树谈到R 树，这次，咱们来讲另外两种树：Tire树与后缀树。不过，在此之前，先来看两个问题。

第一个问题：一个文本文件，大约有一万行，每行一个词，要求统计出其中最频繁出现的前10个词，请给出思想，给出时间复杂度分析。

之前在此文：海量数据处理面试题集锦与Bit-map详解中给出的参考答案：用trie树统计每个词出现的次数，时间复杂度是O(n*le)（le表示单词的平均长度），然后是找出出现最频繁的前10个词。也可以用堆来实现（具体的操作可参考第三章、寻找最小的k个数），时间复杂度是O(n*lg10)。所以总的时间复杂度，是O(n*le)与O(n*lg10)中较大的哪一个。

第二个问题：找出给定字符串里的最长回文。例子：输入XMADAMYX。则输出MADAM。这道题的流行解法是用后缀树（Suffix Tree），但其用途远不止如此，它能高效解决一大票复杂的字符串编程问题（当然，它有它的弱点，如算法实现复杂以及空间开销大），概括如下：

- 查询字符串S是否包含子串S1。主要思想是：如果S包含S1，那么S1必定是S的某个后缀的前缀；又因为S的后缀树包含了所有的后缀，所以只需对S的后缀树使用和Trie相同的查找方法查找S1即可（使用后缀树实现的复杂度同流行的KMP算法的复杂度相当）。
- 找出字符串S的最长重复子串S1。比如abcdabcefda里abc同da都重复出现，而最长重复子串是abc。
- 找出字符串S1同S2的最长公共子串。注意最长公共子串（Longest CommonSubstring）和最长公共子序列（LongestCommon Subsequence, LCS）的区别：子串（Substring）是串的一个连续的部分，子序列（Subsequence）则是从不改变序列的顺序，而从序列中

去掉任意的元素而获得的新序列；更简略地说，前者（子串）的字符的位置必须连续，后者（子序列LCS）则不必。比如字符串acdfg同akdfc的最长公共子串为df，而他们的最长公共子序列是adf。LCS可以使用动态规划法解决。

- Ziv-Lampel无损压缩算法。LZW算法的基本原理是利用编码数据本身存在字符串重复特性来实现数据压缩，所以一个很好的选择是使用后缀树的形式来组织存储字符串及其对应压缩码值的字典。
- 找出字符串S的最长回文子串S1。例如：XMADAMYX的最长回文子串是MADAM（此即为上面所说的第二个问题：最长回文问题，本文第二部分将详细阐述此问题）。
- 多模式串的模式匹配问题（suffix_array + 二分）。

本文**第一部分**，咱们就来了解这个**Trie树**，然后自然而然过渡到**第二部分、后缀树**权作此番阐述，以备不时之需，在需要的时候便可手到擒来。ok，有任何问题，欢迎不吝指正或赐教。谢谢。

第一部分、Trie树

1.1、什么是Trie树

Trie树，即字典树，又称单词查找树或键树，是一种树形结构，是一种哈希树的变种。典型应用是用于统计和排序大量的字符串（但不仅限于字符串），所以经常被搜索引擎系统用于文本词频统计。它的优点是：最大限度地减少无谓的字符串比较，查询效率比哈希表高。

Trie的核心思想是空间换时间。利用字符串的公共前缀来降低查询时间的开销以达到提高效率的目的。

它有3个基本性质：

1. 根节点不包含字符，除根节点外每一个节点都只包含一个字符。
2. 从根节点到某一节点，路径上经过的字符连接起来，为该节点对应的字符串。
3. 每个节点的所有子节点包含的字符都不相同。

1.2、 树的构建

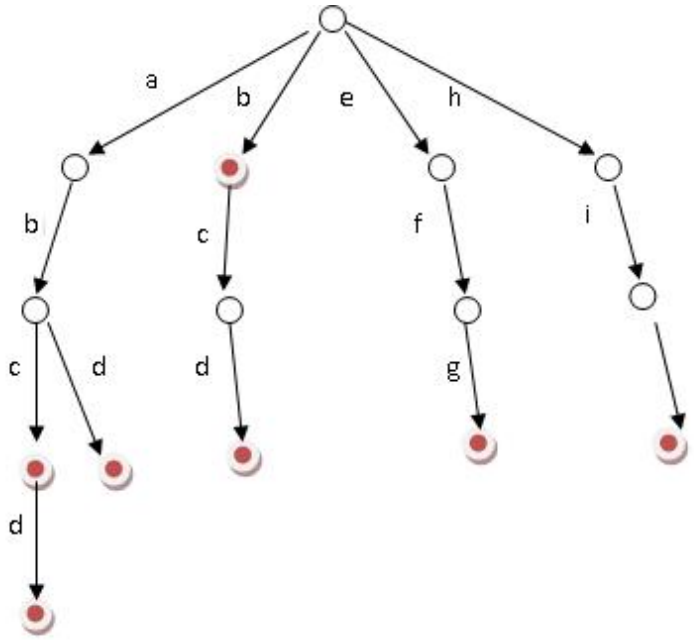
举个在网上流传颇广的例子，如下：

题目：给你100000个长度不超过10的单词。对于每一个单词，我们要判断他出没出现过，如果出现了，求第一次出现在第几个位置。

分析:这题当然可以用hash来解决,但是本文重点介绍的是trie树，因为在某些方面它的用途更大。比如说对于某一个单词，我们要询问它的前缀是否出现过。这样hash就不好搞了,而用trie还是很简单。

现在回到例子中，如果我们用最傻的方法，对于每一个单词，我们都要去查找它前面的单词中是否有它。那么这个算法的复杂度就是O(n^2)。显然对于100000的范围难以接受。现在我们换个思路想。假设我要查询的单词是abcd,那么在他前面的单词中,以b,c,d,f之类开头的我显然不必考虑。而只要找以a开头的中是否存在abcd就可以了。同样的,在以a开头中的单词中,我们只要考虑以b作为第二个字母的,一次次缩小范围和提高针对性,这样一个树的模型就渐渐清晰了。

好比假设有b,abc,abd,bcd,abcd,efg,hii 这6个单词，我们构建的树就是如下图这样的：



当时第一次看到这幅图的时候,便立马感到此树之不凡构造了。单单从上幅图便可窥知一二,好比大海搜人,立马就能确定东南西北中的到底

哪个方位，如此迅速缩小查找的范围和提高查找的针对性，不失为一创举。

ok，如上图所示，对于每一个节点，从根遍历到他的过程就是一个单词，如果这个节点被标记为红色，就表示这个单词存在，否则不存在。

那么，对于一个单词，我只要顺着他从根走到对应的节点，再看这个节点是否被标记为红色就可以知道它是否出现过了。把这个节点标记为红色，就相当于插入了这个单词。

这样一来我们查询和插入可以一起完成（重点体会这个查询和插入是如何一起完成的，稍后，下文具体解释），所用时间仅仅为单词长度，在这一个样例，便是10。

我们可以看到，trie树每一层的节点数是26^i级别的。所以为了节省空间。我们用动态链表，或者用数组来模拟动态。空间的花费，不会超过单词数×单词长度。

1.3、前缀查询

上文中提到”比如说对于某一个单词，我们要询问它的前缀是否出现过。这样hash就不好搞了，而用trie还是很简单“。下面，咱们来看看这个前缀查询问题：

已知n个由小写字母构成的平均长度为10的单词，判断其中**是否存在某个串为另一个串的前缀子串**。下面对比3种方法：

1. 最容易想到的：即从字符串集中从头往后搜，看每个字符串是否为字符串集中某个字符串的前缀，复杂度为O(n^2)。
2. 使用hash：我们用hash存下所有字符串的所有的前缀子串，建立存有子串hash的复杂度为O(n*len)，而查询的复杂度为O(n)* O(1)= O(n)。
3. 使用trie：因为当查询如字符串abc是否为某个字符串的前缀时，显然以b,c,d….等不是以a开头的字符串就不用查找了。所以建立trie的复杂度为O(n*len)，而**建立+查询在trie中是可以同时执行的**，建立的过程也就可以成为查询的过程，hash就不能实现这个功能。所以总的复杂度为O(n*len)，实际查询的复杂度也只是O(len)。（说白了，就是

Trie树的平均高度h为len，所以Trie树的查询复杂度为O（h）=O（len）。好比一棵二叉平衡树的高度为logN，则其查询，插入的平均时间复杂度亦为O（logN））。

下面解释下上述方法3中所说的为什么hash不能将建立与查询同时执行，而Trie树却可以：

- 在hash中，例如现在要输入两个串911，911456，如果要同时查询这两个串，且查询串的同时若hash中没有则存入。那么，这个查询与建立的过程就是先查询其中一个串911，没有，然后存入9、91、911；而后查询第二个串911456，没有然后存入9、91、911、9114、91145、911456。因为程序没有记忆功能，所以并不知道911在输入数据中出现过，只是照常以例行事，存入9、91、911、9114、911…。也就是说用hash必须先存入所有子串，然后for循环查询。
- 而trie树中，存入911后，已经记录911为出现的字符串，在存入911456的过程中就能发现而输出答案；倒过来亦可以，先存入911456，在存入911时，当指针指向最后一个1时，程序会发现这个1已经存在，说明911必定是某个字符串的前缀。

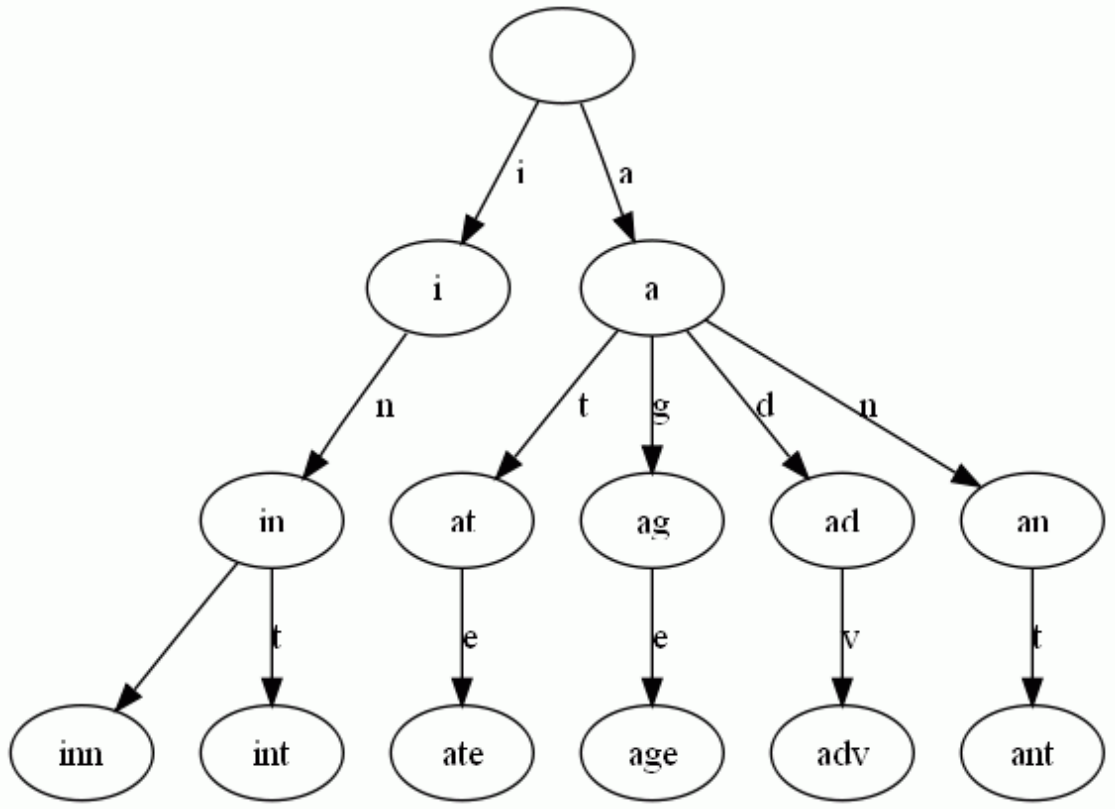
读者反馈@悠悠长风：关于这点，我有不同的看法。hash也是可以实现边建立边查询的啊。当插入911时，需要一个额外的标志位，表示它是一个完整的单词。在处理911456时，也是按照前面的查询9,91,911，当查询911时，是可以找到前面插入的911，且通过标志位知道911为一个完整单词。那么就可以判断出911为911456的前缀啊。虽然trie树更适合这个问题，但是我认为hash也是可以实现边建立，边查找。

至于，有关Trie树的查找，插入等操作的实现代码，网上遍地开花且千篇一律，诸君尽可参考，想必不用我再做多余费神。

1.4、查询

Trie树是简单但实用的数据结构，通常用于实现字典查询。我们做即时响应用户输入的AJAX搜索框时，就是Trie开始。本质上，Trie是一颗存储多个字符串的树。相邻节点间的边代表一个字符，这样树的每条分支代表一则子串，而树的叶节点则代表完整的字符串。和普通树不同的地方

是，相同的字符串前缀共享同一条分支。下面，再举一个例子。给出一组单词，inn, int, at, age, adv, ant, 我们可以得到下面的Trie：



可以看出：

- 每条边对应一个字母。
- 每个节点对应一项前缀。叶节点对应最长前缀，即单词本身。
- 单词inn与单词int有共同的前缀“in”, 因此他们共享左边的一条分支，root->i->in。同理，ate, age, adv, 和ant共享前缀“a”，所以他们共享从根节点到节点“a”的边。

查询操纵非常简单。比如要查找int，顺着路径i -> in -> int就找到了。

搭建Trie的基本算法也很简单，无非是逐一把每则单词的每个字母插入Trie。插入前先看前缀是否存在。如果存在，就共享，否则创建对应的节点和边。比如要插入单词add，就有下面几步：

1. 考察前缀“a”，发现边a已经存在。于是顺着边a走到节点a。

2. 考察剩下的字符串“dd”的前缀“d”，发现从节点a出发，已经有边d存在。于是顺着边d走到节点ad
3. 考察最后一个字符“d”，这下从节点ad出发没有边d了，于是创建节点ad的子节点add，并把边ad->add标记为d。

1.5、Trie树的应用

除了本文引言处所述的问题能应用Trie树解决之外，Trie树还能解决下述问题（节选自此文：海量数据处理面试题集锦与Bit-map）：

- 3. 有一个1G大小的一个文件，里面每一行是一个词，词的大小不超过16字节，内存限制大小是1M。返回频数最高的100个词。
- 9. 1000万字符串，其中有些是重复的，需要把重复的全部去掉，保留没有重复的字符串。请怎么设计和实现？
- 10. 一个文本文件，大约有一万行，每行一个词，要求统计出其中最频繁出现的前10个词，请给出思想，给出时间复杂度分析。
- 13.寻找热门查询：
  搜索引擎会通过日志文件把用户每次检索使用的所有检索串都记录下来，每个查询串的长度为1-255字节。假设目前有一千万个记录，这些查询串的重复读比较高，虽然总数是1千万，但是如果去除重复和，不超过3百万个。一个查询串的重复度越高，说明查询它的用户越多，也就越热门。请你统计最热门的10个查询串，要求使用的内存不能超过1G。

1. 请描述你解决这个问题的思路；
2. 请给出主要的处理流程，算法，以及算法的复杂度。

有了Trie，后缀树就容易理解了。本文接下来的第二部分，介绍后缀树。

后缀树

2.1、后缀树的定义

后缀树（Suffix tree）是一种数据结构，能快速解决很多关于字符串的问题。后缀树的概念最早由Weiner 于1973年提出，既而由McCreight 在1976

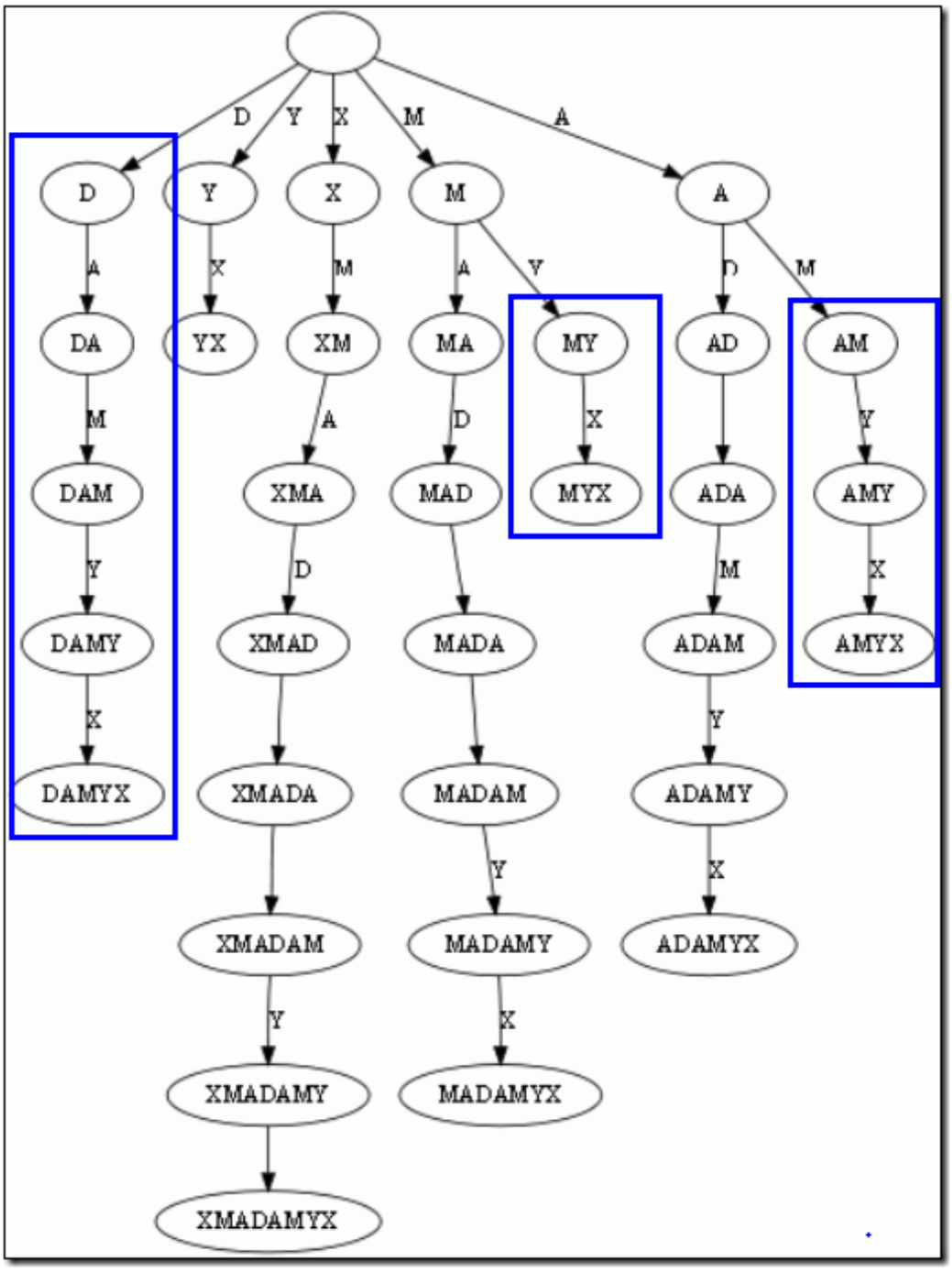
年和Ukkonen在1992年和1995年加以改进完善。

后缀，顾名思义，甚至通俗点来说，就是所谓后缀就是后面尾巴的意思。比如说给定一长度为n的字符串S=S1S2..Si..Sn，和整数i，1 <= i <= n，子串SiSi+1…Sn便都是字符串S的后缀。

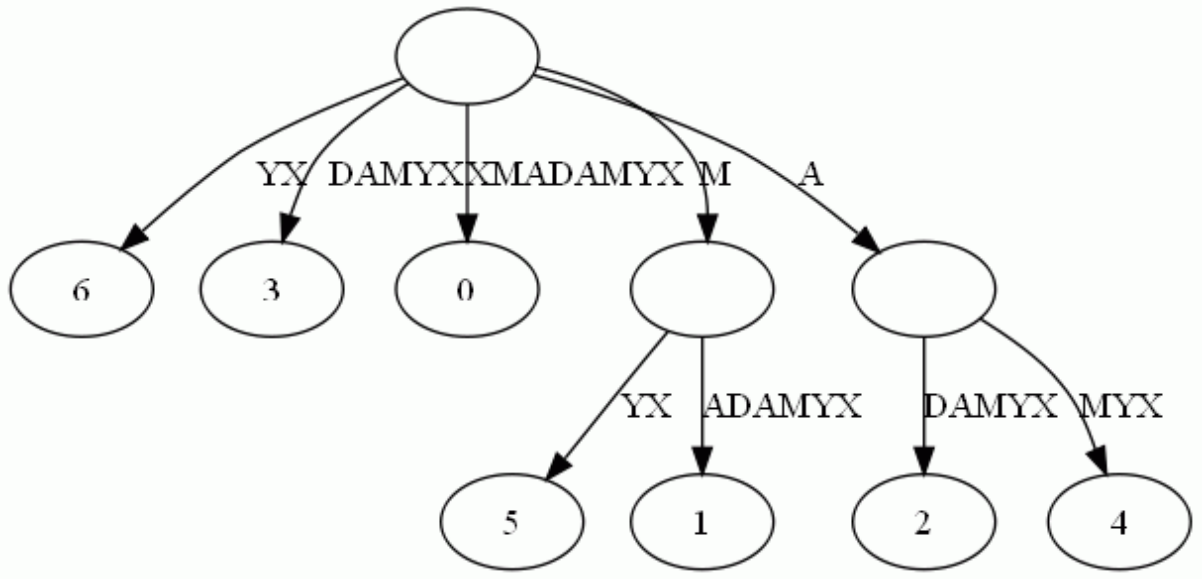
以字符串S=XMADAMYX为例，它的长度为8，所以S[1..8], S[2..8], … , S[8..8]都算S的后缀，我们一般还把空字串也算成后缀。这样，我们一共有如下后缀。对于后缀S[i..n]，我们说这项后缀起始于i。

S[1..8], XMADAMYX，也就是字符串本身，起始位置为1
S[2..8], MADAMYX，起始位置为2
S[3..8], ADAMYX，起始位置为3
S[4..8], DAMYX，起始位置为4
S[5..8], AMYX，起始位置为5
S[6..8], MYX，起始位置为6
S[7..8], YX，起始位置为7
S[8..8], X，起始位置为8
空字串，记为$。

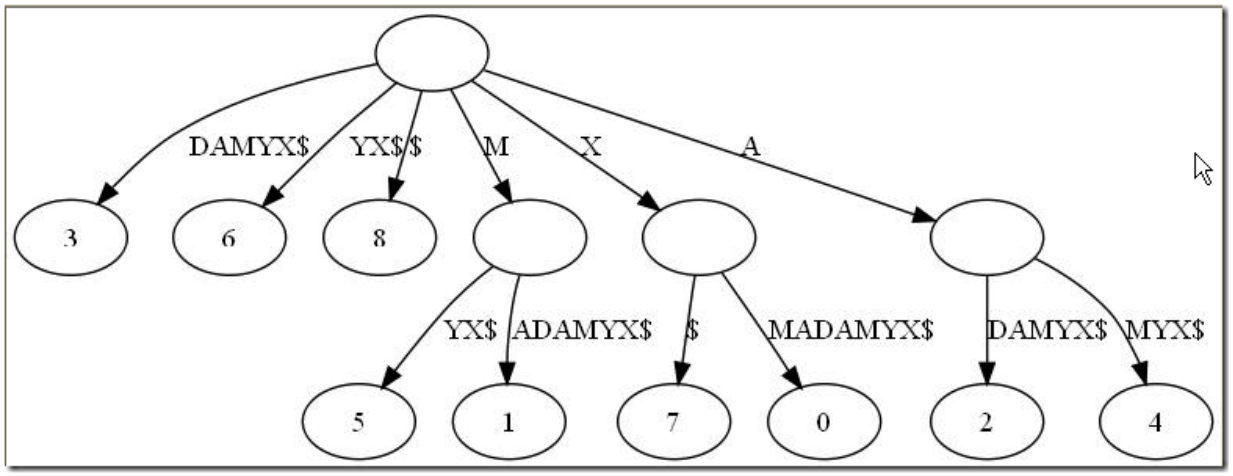
而后缀树，就是包含一则字符串所有后缀的压缩Trie。把上面的后缀加入Trie后，我们得到下面的结构：



仔细观察上图，我们可以看到不少值得压缩的地方。比如蓝框标注的分支都是独苗，没有必要用单独的节点同边表示。如果我们允许任意一条边里包含多个字 母，就可以把这种没有分叉的路径压缩到一条边。另外每条边已经包含了足够的后缀信息，我们就不用再给节点标注字符串信息了。我们只需要**在叶节点上标注上每项后缀的起始位置**。于是我们得到下图：



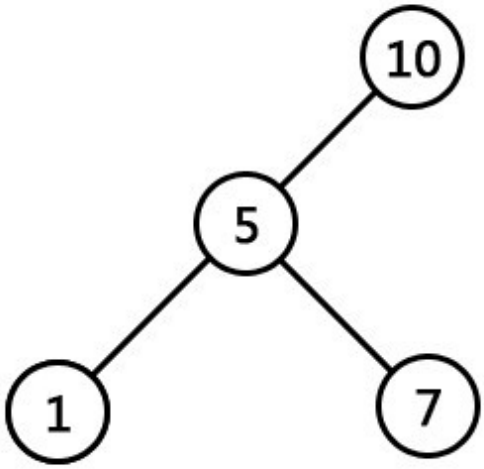
这样的结构丢失了某些后缀。比如后**缀X在上图中消失了**，因为它正好是字符串XMADAMYX的前缀。为了避免这种情况，我们也规定**每项后缀不能是其它后缀的前缀**。要解决这个问题其实挺简单，在**待处理的子串后加一个空字串**就行了。例如我们处理XMADAMYX前，先把XMADAMYX变为 XMADAMYX$，于是就得到suffix tree–后缀树了，如下图所示：



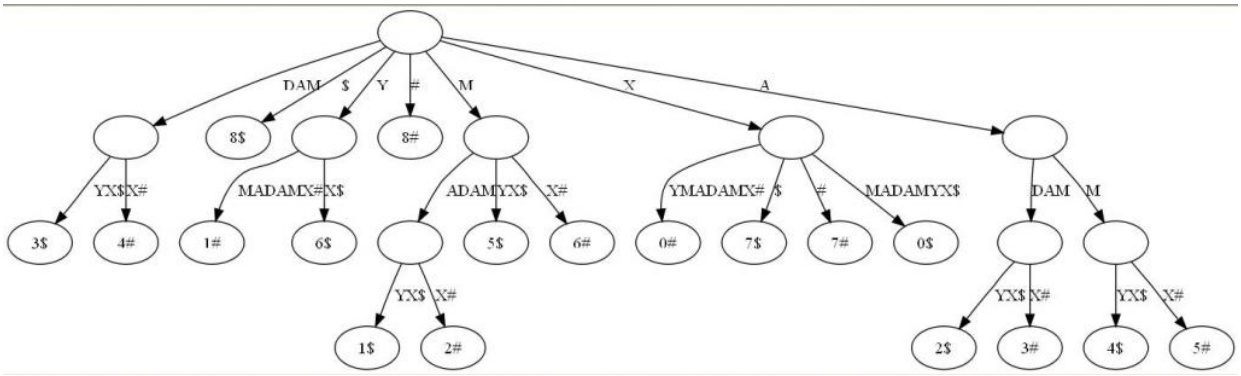
2.2、后缀树与回文问题的关联

那后缀树同最长回文有什么关系呢？我们得先知道两个简单概念：

- 最低共有祖先，**LCA**（Lowest Common Ancestor)，也就是任意两节点（多个也行）最长的共有前缀。比如下图中，节点7同节点1的共同祖先是节点5与节点10，但最低共同祖先是5。查找LCA的算法是O(1)的复杂度，当然，代价是需要对后缀树做复杂度为O(n)的预处理。



- 广义后缀树(Generalized Suffix Tree)。传统的后缀树处理一坨单词的所有后缀。广义后缀树存储任意多个单词的所有后缀。例如下图是单词**XMADAMYX与XYMADAMX的广义后缀 树**。注意我们需要区分不同单词的后缀，所以叶节点用不同的特殊符号与后缀位置配对。



2.3、最长回文问题的解决

有了上面的概念，本文引言中提出的查找最长回文问题就相对简单了。咱们来回顾下引言中提出的回文问题的具体描述：找出给定字符串里的最长回文。例如输入XMADAMYX，则输出MADAM。

思维的突破点在于考察回文的半径，而不是回文本身。所谓半径，就是回文对折后的字串。比如回文MADAM 的半径为MAD，半径长度为3，半径的中心是字母D。显然，最长回文必有最长半径，且两条半径相等。还是以MADAM为例，以D为中心往左，我们得到半径 DAM；以D为中心向右，我们得到半径DAM。二者肯定相等。因为MADAM已经是单词XMADAMYX里的最长回文，我们可以肯定从**D往左数的字串 DAMX与从D往右数的子串DAMYX共享最长前缀DAM**。而这，正是解决回文问题的关键。现在我们有后缀树，怎么把从D向左数的字串DAMX变成后缀呢？

到这个地步，答案应该明显：**把单词 XMADAMYX 翻转（XMADAMYX=>XYMADAMX，DAMX**就变成后缀了）就行了。于是我们把寻找回文的问题转换成了寻找两坨后缀的**LCA**的问题。当然，我们还需要知道 到底查询那些后缀间的LCA。很简单，给定字符串S，如果最长回文的中心在i，那从位置i向右数的后缀刚好是S(i)，而向左数的字符串刚好是翻转S后得到的字符串S‘的后缀S’(n-i+1)。这里的n是字符串S的长度。

可能上面的阐述还不够直观，我再细细说明下：

1. 首先，还记得本第二部分开头关于后缀树的定义么：“先说说后缀的定义，顾名思义，甚至通俗点来说，就是所谓后缀就是后面尾巴的意思。比如说给定一长度为n的字符串S=S1S2..Si..Sn，和整数i，**1 <= i <= n，子串SiSi+1…Sn便都是字符串S的后缀**。”

以字符串S=XMADAMYX为例，它的长度为8，所以S[1..8], S[2..8], … , S[8..8]都算S的后缀，我们一般还把空字串也算成后缀。这样，我们一共有如下后缀。对于后缀S[i..n]，我们说这项后缀起始于i。

S[1..8], XMADAMYX，也就是字符串本身，起始位置为1
S[2..8], MADAMYX，起始位置为2
S[3..8], ADAMYX，起始位置为3

S[4..8], DAMYX，起始位置为4
S[5..8], AMYX，起始位置为5
S[6..8], MYX，起始位置为6
S[7..8], YX，起始位置为7
S[8..8], X，起始位置为8
空字串，记为$。

2. 对单词XMADAMYX而言，回文中心为D，那么D向右的后缀**DAMYX**假设是S(i)（当N=8，i从1开始计数，i=4时，便是S(4..8)）;而对于翻转后的单词XYMADAMX而言，回文中心D向右对应的后缀为**DAMX**，也就是S'(N-i+1)(（N=8，i=4，便是S‘（5..8））。此刻已经可以得出，它们共享最长前缀，即**LCA（DAMYX，DAMX）=DAM**。有了这套直观解释，算法自然呼之欲出:

1. 预处理后缀树，使得查询LCA的复杂度为O(1)。这步的开销是O(N)，N是单词S的长度；
2. 对单词的每一位置i(也就是从0到N-1)，获取LCA(S(i), S‘(N-i+1)) 以及LCA(S(i+1), S'(n-i+1))。查找两次的原因是我们需要考虑奇数回文和偶数回文的情况。这步要考察每坨i，所以复杂度是O(N)；
3. 找到最大的LCA，我们也就得到了回文的中心i以及回文的半径长度，自然也就得到了最长回文。总的复杂度O(n)。

用上图做例子，i为4时，LCA(4$, 5#)为DAM，正好是最长半径。当然，这只是直观的叙述。

上面大致描述了后缀树的基本思路。要想写出实用代码，至少还得知道下面的知识：

- 创建后缀树的O(n)算法。此算法有很多种，无论Peter Weiner的73年年度最佳算法，还是Edward McCreight1976的改进算法，还是1995年E. Ukkonen大幅简化的算法（本文第4部分将重点阐述这种方法），还是Juha Kärkkäinen 和 Peter Sanders2003年进一步简化的线性算法，都是O（n）的时间复杂度。至于实际中具体选择哪一种算法，可依实际情况而定。

- 实现后缀树用的数据结构。比如常用的子结点加兄弟节点列表，Directed 优化后缀树空间的办法。比如不存储子串，而存储读取子串必需的位置。以及Directed Acyclic Word Graph，常缩写为黑哥哥们挂在嘴边的DAWG。

2.4、后缀树的应用

后缀树的用途，总结起来大概有如下几种:

1. 查找字符串o是否在字符串S中。
   方案：用S构造后缀树，按在trie中搜索字串的方法搜索o即可。
   原理：若o在S中，则o必然是S的某个后缀的前缀。
   例如S: leconte，查找o: con是否在S中,则o(con)必然是S(leconte)的后缀之一conte的前缀.有了这个前提，采用trie搜索的方法就不难理解了。
2. 指定字符串T在字符串S中的重复次数。
   方案：用S+'$'构造后缀树，搜索T节点下的叶节点数目即为重复次数
   原理：如果T在S中重复了两次，则S应有两个后缀以T为前缀，重复次数就自然统计出来了。
3. 字符串S中的最长重复子串
   方案：原理同2，具体做法就是找到最深的非叶节点。这个深是指从root所经历过的字符个数，最深非叶节点所经历的字符串起来就是最长重复子串。
   为什么要非叶节点呢?因为既然是要重复，当然叶节点个数要>=2。
4. 两个字符串S1，S2的最长公共部分
   方案：将S1#S2$作为字符串压入后缀树，找到最深的非叶节点，且该节点的叶节点既有#也有$(无#)。

教你透彻了解红黑树

作者：July 、saturnman 2010年12月29日

本文参考：Google、算法导论、STL源码剖析、计算机程序设计艺术

推荐阅读：Left-Leaning Red-Black Trees, Dagstuhl Workshop on Data Structures, Wadern, Germany, February, 2008.
直接下载：。

二叉查找树

由于红黑树本质上就是一棵二叉查找树，所以在了解红黑树之前，咱们先来看下二叉查找树。

二叉查找树（Binary Search Tree），也称有序二叉树（ordered binary tree），排序二叉树（sorted binary tree），是指一棵空树或者具有下列性质的二叉树：

- 若任意节点的左子树不空，则左子树上所有结点的值均小于它的根结点的值；
- 任意节点的右子树不空，则右子树上所有结点的值均大于它的根结点的值；
- 任意节点的左、右子树也分别为二叉查找树。
- 没有键值相等的节点（no duplicate nodes）。

因为，一棵由n个结点，随机构造的二叉查找树的高度为lgn，所以顺理成章，一般操作的执行时间为O（lgn）.（至于n个结点的二叉树高度为lgn的证明，可参考算法导论 第12章 二叉查找树 第12.4节）。

但二叉树若退化成了一棵具有n个结点的线性链后，则此些操作最坏情况运行时间为O（n）。后面我们会看到一种基于二叉查找树-红黑树，它通过一些性质使得树相对平衡，使得最终查找、插入、删除的时间复杂度最坏情况下依然为O（lgn）。

红黑树

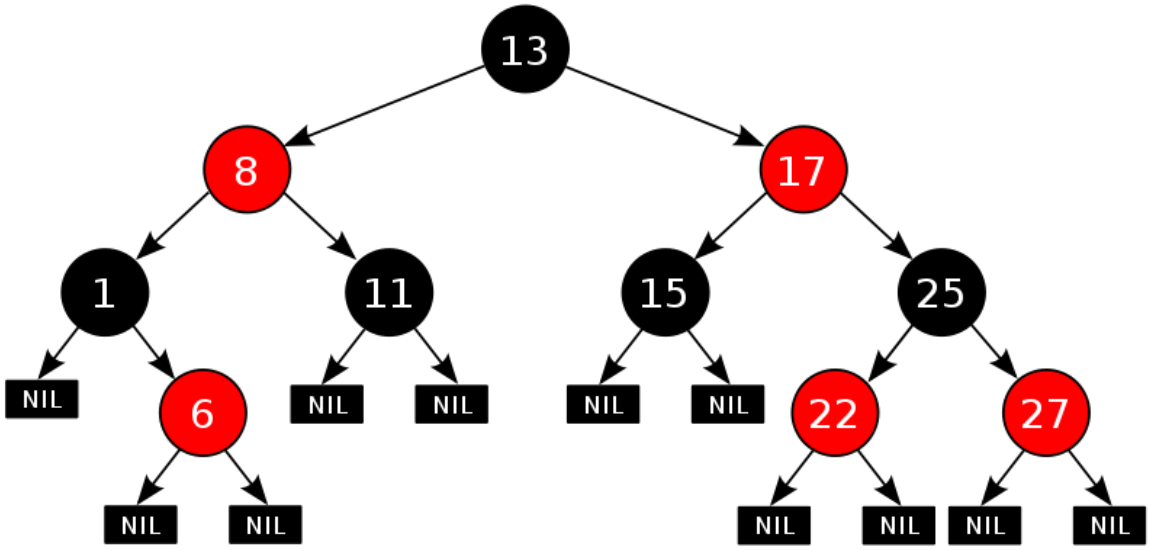
前面我们已经说过，红黑树，本质上来说就是一棵二叉查找树，但它在二叉查找树的基础上增加了着色和相关的性质使得红黑树相对平衡，从而保证了红黑树的查找、插入、删除的时间复杂度最坏为O(log n)。

下面，咱们便来具体看看红黑树的5条性质：

1）每个结点要么是红的，要么是黑的。
2）根结点是黑的。
3）每个叶结点（叶结点即指树尾端NIL指针或NULL结点）是黑的。
4）如果一个结点是红的，那么它的俩个儿子都是黑的。
5）对于任一结点而言，其到叶结点树尾端NIL指针的每一条路径都包含相同数目的黑结点。

注：上述第3、5点性质中所说的NULL结点（包括wikipedia.算法导论上所认为的叶子结点），即为树尾端的NIL指针，或者说NULL结点。然百度百科以及网上一些其它博文直接说的叶结点，则易引起误会，因，此叶结点非子结点。

如下图所示，即是一颗红黑树(下图引自wikipedia：http://t.cn/hgvH1l)：



上文中我们所说的“叶结点”或“NULL结点”，它不包含数据而只充当树在此结束的指示，这些节点以及它们的父结点，在绘图中都会经常被省略。

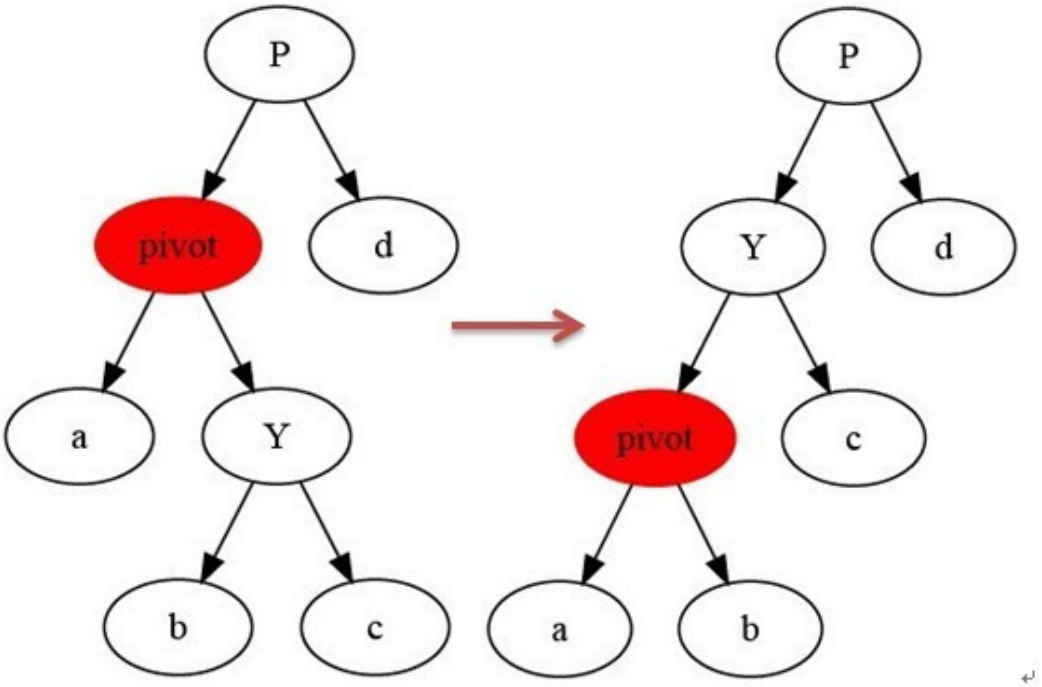
树的旋转知识

当我们在对红黑树进行插入和删除等操作时，对树做了修改，那么可能会违背红黑树的性质。

为了继续保持红黑树的性质，我们可以通过对结点进行重新着色，以及对树进行相关的旋转操作，即修改树中某些结点的颜色及指针结构，来达到对红黑树进行插入或删除结点等操作后，继续保持它的性质或平衡。

树的旋转，分为左旋和右旋，以下借助图来做形象的解释和介绍：

1.左旋



如上图所示：

当在某个结点pivot上，做左旋操作时，我们假设它的右孩子y不是NIL[T]，pivot可以为树内任意左孩子而不是NIL[T]的结点。

左旋以pivot到y之间的链为“支轴”进行，它使y成为该孩子树新的根，而y的左孩子b则成为pivot的右孩子。

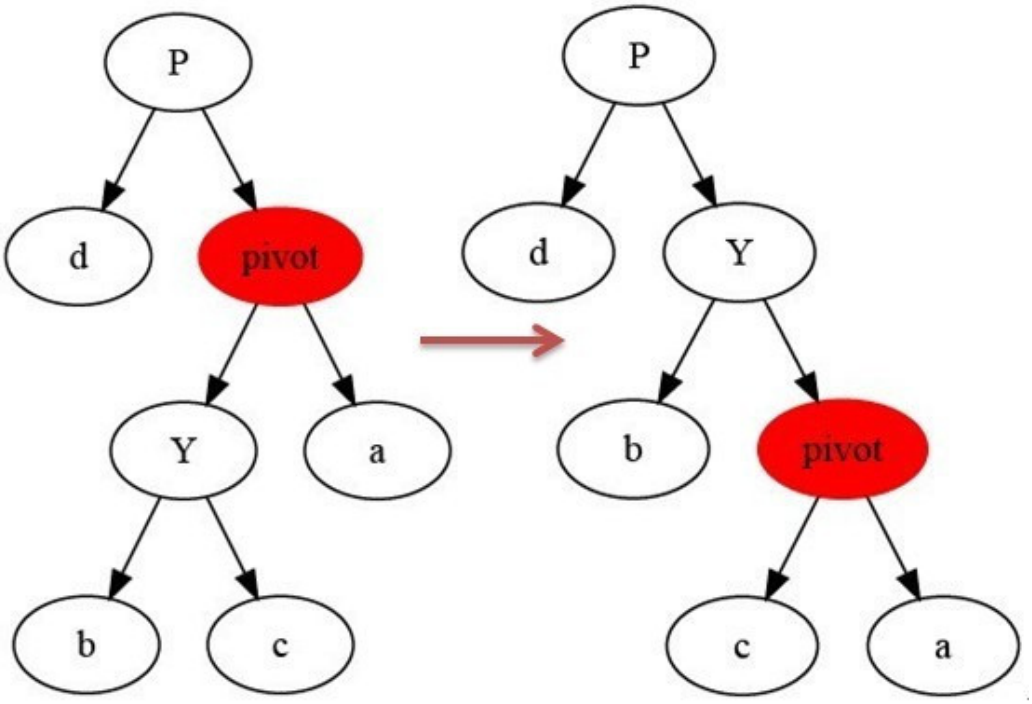
左旋操作的参考代码如下所示（以x代替上述的pivot）：

```
 LEFT-ROTATE(T, x)
1  y ← right[x] ▷ Set y.
2  right[x] ← left[y]       ▷ Turn y's left subtree into x's right subtree.
3  p[left[y]] ← x
4  p[y] ← p[x]              ▷ Link x's parent to y.
5  if p[x] = nil[T]
```

```
6     then root[T] ← y
7     else if x = left[p[x]]
8             then left[p[x]] ← y
9             else right[p[x]] ← y
10  left[y] ← x             ▷ Put x on y's left.
11  p[x] ← y
```

2.右旋

右旋与左旋差不多,再此不做详细介绍。



对于树的旋转，能保持不变的只有原树的搜索性质，而原树的红黑性质则不能保持，在红黑树的数据插入和删除后可利用旋转和颜色重涂来恢复树的红黑性质。

红黑树的插入

要真正理解红黑树的插入和删除,还得先理解二叉查找树的插入和删除。磨刀不误砍柴工,咱们再来分别了解下二叉查找树的插入和删除。

二叉查找树的插入

如果要在二叉查找树中插入一个结点，首先要查找到结点插入的位置，然后进行插入，假设插入的结点为z的话，插入的伪代码如下：

```
TREE-INSERT(T, z)
 1  y ← NIL
 2  x ← root[T]
 3  while x ≠ NIL
 4       do y ←  x
 5          if key[z] < key[x]
 6             then x ← left[x]
 7             else x ← right[x]
 8  p[z] ← y
 9  if y = NIL
10     then root[T] ← z              ✣ Tree T was empty
11     else if key[z] < key[y]
12              then left[y] ← z
13              else right[y] ← z
```

可以看到，上述第3-7行代码即是在二叉查找树中查找z待插入的位置，如果插入结点z小于当前遍历到的结点，则到当前结点的左子树中继续查找，如果z大于当前结点，则到当前结点的右子树中继续查找，第9-13行代码找到待插入的位置，如果z依然比此刻遍历到的新的当前结点小，则z作为当前结点的左孩子，否则作为当前结点的右孩子。

红黑树的插入和插入修复

现在我们了解了二叉查找树的插入，接下来，咱们便来具体了解红黑树的插入操作。红黑树的插入相当于在二叉查找树插入的基础上，为了重新恢复平衡，继续做了插入修复操作。

假设插入的结点为z，红黑树的插入伪代码具体如下所示：

```
RB-INSERT(T, z)
 1  y ← nil[T]
 2  x ← root[T]
 3  while x ≠ nil[T]
 4       do y ← x
 5          if key[z] < key[x]
 6             then x ← left[x]
 7             else x ← right[x]
 8  p[z] ← y
 9  if y = nil[T]
10     then root[T] ← z
11     else if key[z] < key[y]
12              then left[y] ← z
13              else right[y] ← z
```

```
14  left[z] ← nil[T]
15  right[z] ← nil[T]
16  color[z] ← RED
17  RB-INSERT-FIXUP(T, z)
```

我们把上面这段红黑树的插入代码，跟我们之前看到的二叉查找树的插入代码，可以看出，RB-INSERT(T, z)前面的第1-13行代码基本就是二叉查找树的插入代码，然后第14-16行代码把z的左孩子、右孩子都赋为叶结点nil，再把z结点着为红色，最后为保证红黑性质在插入操作后依然保持，调用一个辅助程序RB-INSERT-FIXUP来对结点进行重新着色，并旋转。

换言之 - 如果插入的是根结点，因为原树是空树，此情况只会违反性质2，所以直接把此结点涂为黑色。
- 如果插入的结点的父结点是黑色，由于此不会违反性质2和性质4，红黑树没有被破坏，所以此时也是什么也不做。

但当遇到下述3种情况时：- 插入修复情况1：如果当前结点的父结点是红色且祖父结点的另一个子结点（叔叔结点）是红色 - 插入修复情况2：当前节点的父节点是红色,叔叔节点是黑色，当前节点是其父节点的右子 - 插入修复情况3：当前节点的父节点是红色,叔叔节点是黑色，当前节点是其父节点的左子

又该如何调整呢？答案就是根据红黑树插入代码RB-INSERT(T, z)最后一行调用的RB-INSERT-FIXUP（T,z）所示操作进行，具体如下所示：

```
RB-INSERT-FIXUP(T,z)
 1 while color[p[z]] = RED
 2     do if p[z] = left[p[p[z]]]
 3           then y ← right[p[p[z]]]
 4                if color[y] = RED
 5                   then color[p[z]] ← BLACK                    ▷ Case 1
 6                        color[y] ← BLACK                       ▷ Case 1
 7                        color[p[p[z]]] ← RED                   ▷ Case 1
 8                        z ← p[p[z]]                            ▷ Case 1
 9                   else if z = right[p[z]]
10                           then z ← p[z]                       ▷ Case 2
11                                LEFT-ROTATE(T, z)              ▷ Case 2
12                           color[p[z]] ← BLACK                 ▷ Case 3
13                           color[p[p[z]]] ← RED                ▷ Case 3
14                           RIGHT-ROTATE(T, p[p[z]])            ▷ Case 3
15           else (same as then clause
                        with "right" and "left" exchanged)
16 color[root[T]] ← BLACK
```

下面，咱们来分别处理上述3种插入修复情况。

插入修复情况1：当前结点的父结点是红色且祖父结点的另一个子结点（叔叔结点）是红色。

即如下代码所示：

```
1 while color[p[z]] = RED
2     do if p[z] = left[p[p[z]]]
3           then y ← right[p[p[z]]]
4                if color[y] = RED
```

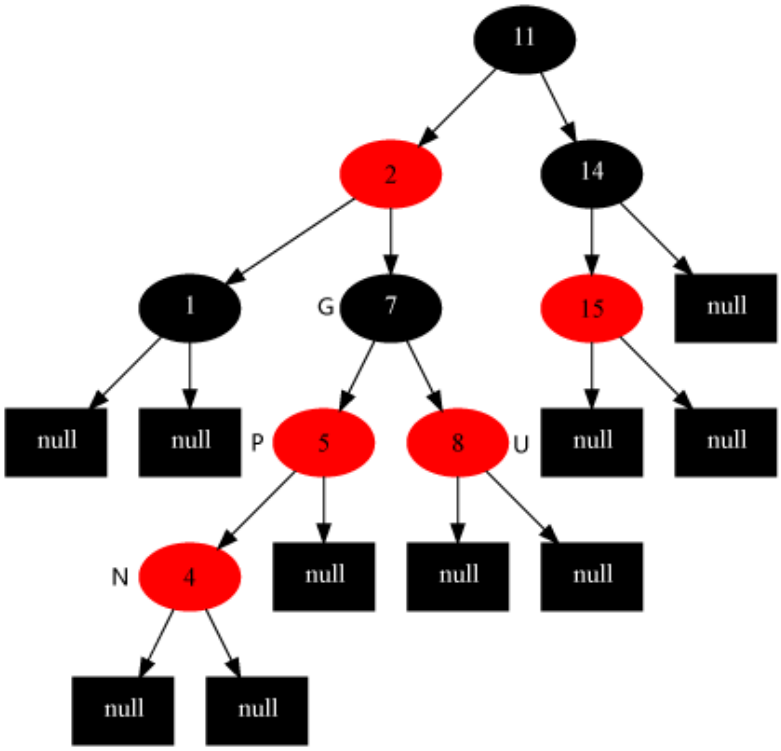
此时父结点的父结点一定存在，否则插入前就已不是红黑树。
与此同时，又分为父结点是祖父结点的左子还是右子，对于对称性，我们只要解开一个方向就可以了。

在此，我们只考虑父结点为祖父左子的情况。
同时，还可以分为当前结点是其父结点的左子还是右子，但是处理方式是一样的。我们将此归为同一类。

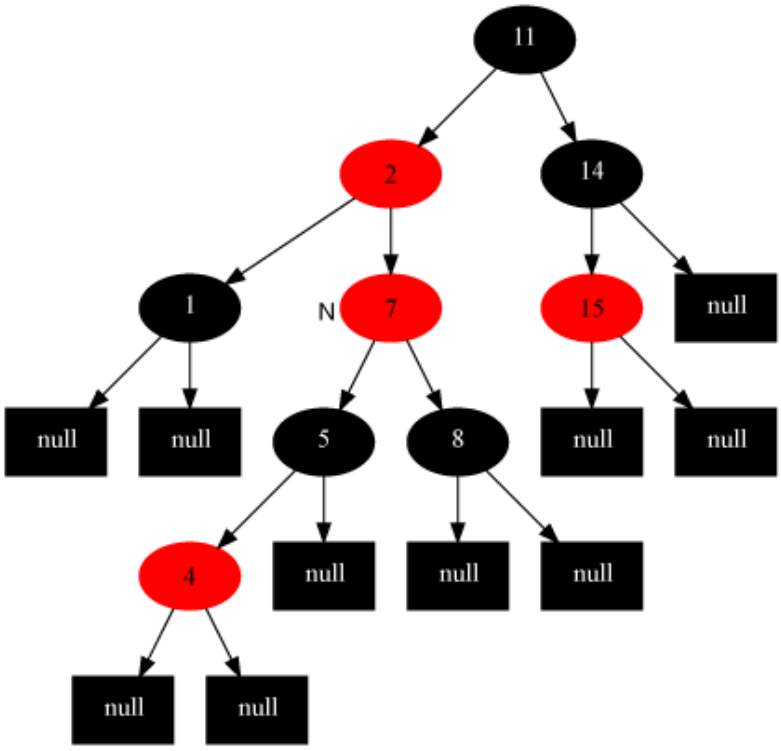
对策：将当前节点的父节点和叔叔节点涂黑，祖父结点涂红，把当前结点指向祖父节点，从新的当前节点重新开始算法。即如下代码所示：

```
5                    then color[p[z]] ← BLACK                    ▷ Case 1
6                         color[y] ← BLACK                       ▷ Case 1
7                         color[p[p[z]]] ← RED                   ▷ Case 1
8                         z ← p[p[z]]                            ▷ Case 1
```

针对情况1，变化前（图片来源：saturnman）[当前节点为4节点]：


11
2
14
1
G
7
15
null
null
null
P
5
8
U
null
null
N
4
null
null
null
null
null

变化后：


11
2
14
1
N
7
15
null
null
null
5
8
null
null
4
null
null
null
null
null

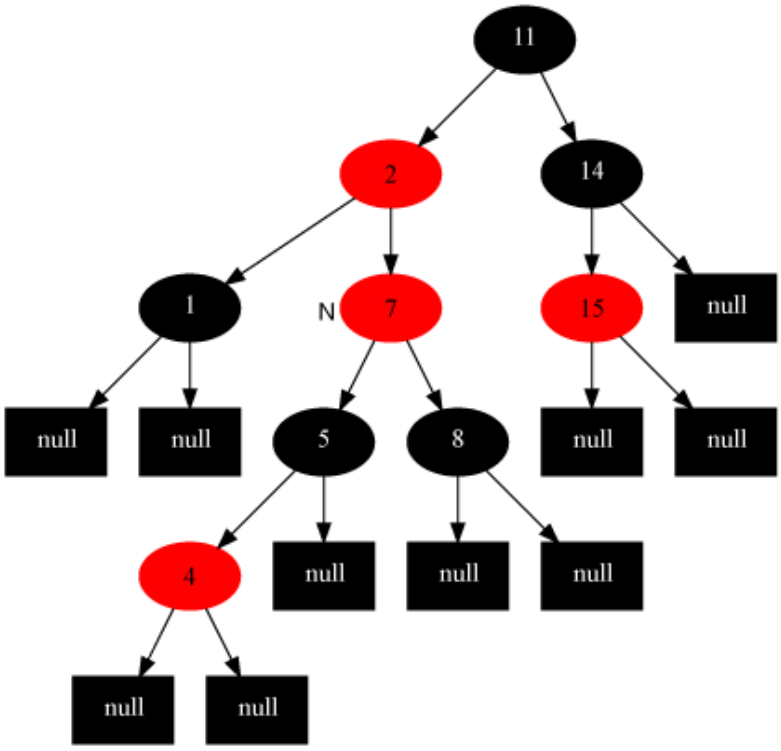
插入修复情况2:当前节点的父节点是红色,叔叔节点是黑色,当前节点是其父节点的右子

对策：当前节点的父节点做为新的当前节点，以新当前节点为支点左旋。

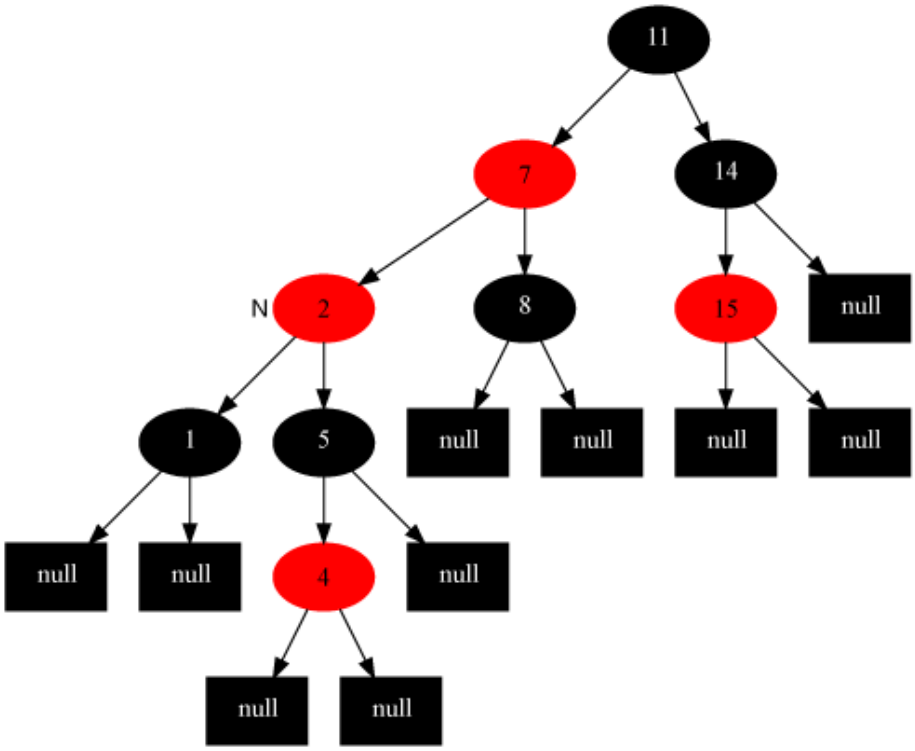
即如下代码所示：

```
 9                    else if z = right[p[z]]
10                            then z ← p[z]                          ▷ Case 2
11                                 LEFT-ROTATE(T, z)                 ▷ Case 2
```

如下图所示,变化前[当前节点为7节点]:



变化后：



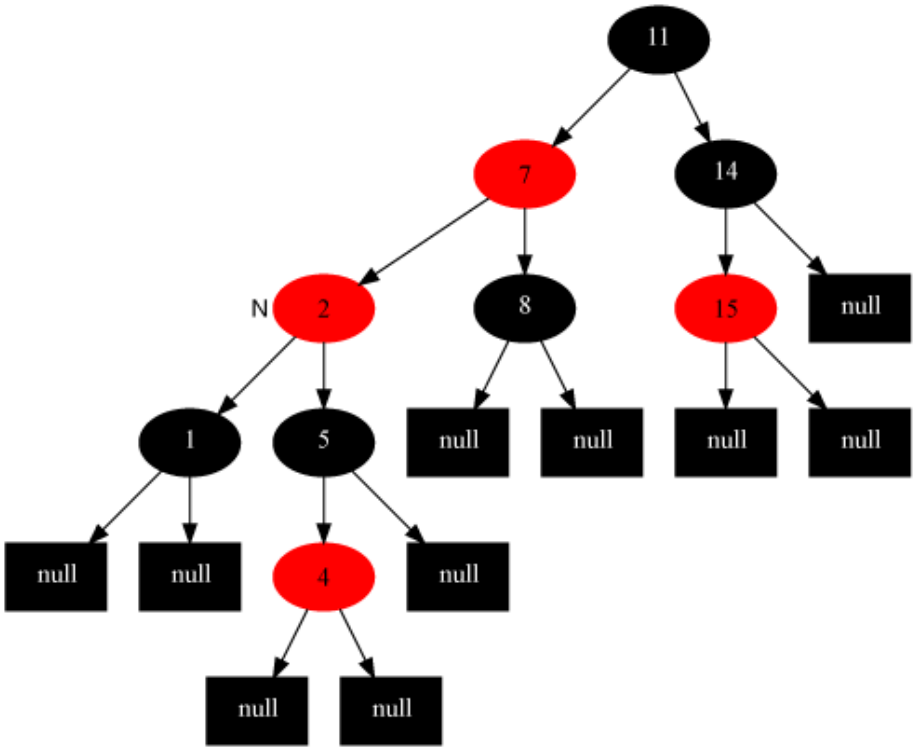
插入修复情况3:当前节点的父节点是红色,叔叔节点是黑色,当前节点是其父节点的左子

解法:父节点变为黑色,祖父节点变为红色,在祖父节点为支点右旋,操作代码为:

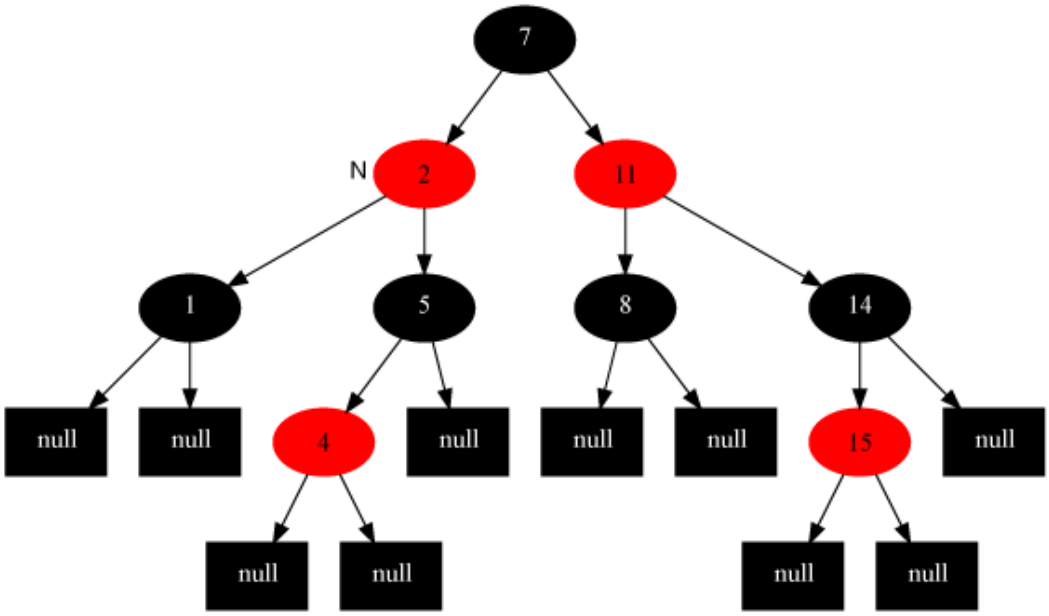
```
12                      color[p[z]] ← BLACK                 ▷ Case 3
13                      color[p[p[z]]] ← RED                ▷ Case 3
14                      RIGHT-ROTATE(T, p[p[z]])            ▷ Case 3
```

最后,把根结点涂为红色,整棵红黑树便重新恢复了平衡。

如下图所示[当前节点为2节点]



变化后：



红黑树的删除

ok，接下来，咱们最后来了解，红黑树的删除操作。

“我们删除的节点的方法与常规二叉搜索树中删除节点的方法是一样的，如果被删除的节点不是有双非空子女，则直接删除这个节点，用它的唯一子节点顶替它的位置，如果它的子节点分是空节点，那就用空节点顶替它的位置，如果它的双子全为非空，我们就把它的直接后继节点内容复制到它的位置，之后以同样的方式删除它的后继节点，它的后继节点不可能是双子非空，因此此传递过程最多只进行一次。”

二叉查找树的删除

继续讲解之前，补充说明下二叉树结点删除的几种情况，待删除的节点按照儿子的个数可以分为三种：

1. 没有儿子，即为叶结点。直接把父结点的对应儿子指针设为NULL，删除儿子结点就OK了。
2. 只有一个儿子。那么把父结点的相应儿子指针指向儿子的独生子，删除儿子结点也OK了。
3. 有两个儿子。这是最麻烦的情况，因为你删除节点之后，还要保证满足搜索二叉树的结构。其实也比较容易，我们可以选择左儿子中的最大元素或者右儿子中的最小元素放到待删除节点的位置，就可以保证结构的不变。当然，你要记得调整子树，毕竟又出现了节点删除。习惯上大家选择左儿子中的最大元素，其实选择右儿子的最小元素也一样，没有任何差别，只是人们习惯从左向右。这里咱们也选择左儿子的最大元素，将它放到待删结点的位置。左儿子的最大元素其实很好找，只要顺着左儿子不断的去搜索右子树就可以了，直到找到一个没有右子树的结点。那就是最大的了。

二叉查找树的删除代码如下所示：

```
TREE-DELETE(T, z)
 1  if left[z] = NIL or right[z] = NIL
 2      then y ← z
 3      else y ← TREE-SUCCESSOR(z)
 4  if left[y] ≠ NIL
 5      then x ← left[y]
 6      else x ← right[y]
 7  if x ≠ NIL
 8      then p[x] ← p[y]
 9  if p[y] = NIL
10      then root[T] ← x
11      else if y = left[p[y]]
12              then left[p[y]] ← x
13              else right[p[y]] ← x
```

```
14  if y ≠ z
15       then key[z] ← key[y]
16            copy y's satellite data into z
17  return y
```

红黑树的删除和删除修复

OK, 回到红黑树上来, 红黑树结点删除的算法实现是:

RB-DELETE(T, z) 单纯删除结点的总操作

```
 1 if left[z] = nil[T] or right[z] = nil[T]
 2    then y ← z
 3    else y ← TREE-SUCCESSOR(z)
 4 if left[y] ≠ nil[T]
 5    then x ← left[y]
 6    else x ← right[y]
 7 p[x] ← p[y]
 8 if p[y] = nil[T]
 9    then root[T] ← x
10    else if y = left[p[y]]
11            then left[p[y]] ← x
12            else right[p[y]] ← x
13 if y ≠ z
14    then key[z] ← key[y]
15         copy y's satellite data into z
16 if color[y] = BLACK
17    then RB-DELETE-FIXUP(T, x)
18 return y
```

“在删除节点后, 原红黑树的性质可能被改变, 如果删除的是红色节点, 那么原红黑树的性质依旧保持, 此时不用做修正操作, 如果删除的节点是黑色节点, 原红黑树的性质可能会被改变, 我们要对其做修正操作。那么哪些树的性质会发生变化呢, 如果删除节点不是树唯一节点, 那么删除节点的那一个支的到各叶节点的黑色节点数会发生变化, 此时性质5被破坏。如果被删节点的唯一非空子节点是红色, 而被删节点的父节点也是红色, 那么性质4被破坏。如果被删节点是根节点, 而它的唯一非空子节点是红色, 则删除后新根节点将变成红色, 违背性质2。”

RB-DELETE-FIXUP(T, x) 恢复与保持红黑性质的工作

```
 1 while x ≠ root[T] and color[x] = BLACK
 2     do if x = left[p[x]]
 3           then w ← right[p[x]]
 4                if color[w] = RED
 5                   then color[w] ← BLACK                        ▷ Case 1
```

```
 6                       color[p[x]] ← RED                          ▷ Case 1
 7                       LEFT-ROTATE(T, p[x])                       ▷ Case 1
 8                       w ← right[p[x]]                            ▷ Case 1
 9               if color[left[w]] = BLACK and color[right[w]] = BLACK
10                  then color[w] ← RED                             ▷ Case 2
11                       x ← p[x]                                   ▷ Case 2
12                  else if color[right[w]] = BLACK
13                          then color[left[w]] ← BLACK             ▷ Case 3
14                               color[w] ← RED                     ▷ Case 3
15                               RIGHT-ROTATE(T, w)                 ▷ Case 3
16                               w ← right[p[x]]                    ▷ Case 3
17                       color[w] ← color[p[x]]                     ▷ Case 4
18                       color[p[x]] ← BLACK                        ▷ Case 4
19                       color[right[w]] ← BLACK                    ▷ Case 4
20                       LEFT-ROTATE(T, p[x])                       ▷ Case 4
21                       x ← root[T]                                ▷ Case 4
22       else (same as then clause with "right" and "left" exchanged)
23 color[x] ← BLACK
```

“上面的修复情况看起来有些复杂，下面我们用一个分析技巧：我们从被删节点后来顶替它的那个节点开始调整，并认为它有额外的一重黑色。这里额外一重黑色是什么意思呢，我们不是把红黑树的节点加上除红与黑的另一种颜色，这里只是一种假设，我们认为我们当前指向它，因此空有额外一种黑色，可以认为它的黑色是从它的父节点被删除后继承给它的，它现在可以容纳两种颜色，如果它原来是红色，那么现在是红+黑，如果原来是黑色，那么它现在的颜色是黑+黑。有了这重额外的黑色，原红黑树性质5就能保持不变。现在只要恢复其它性质就可以了，做法还是尽量向根移动和穷举所有可能性。”– saturnman。

如果是以下情况，恢复比较简单：- a)当前节点是红+黑色
解法，直接把当前节点染成黑色，结束此时红黑树性质全部恢复。
- b)当前节点是黑+黑且是根节点，解法：什么都不做，结束。

但如果是以下情况呢？：- 删除修复情况1：当前节点是黑+黑且兄弟节点为红色(此时父节点和兄弟节点的子节点分为黑) - 删除修复情况2：当前节点是黑加黑且兄弟是黑色且兄弟节点的两个子节点全为黑色 - 删除修复情况3：当前节点颜色是黑+黑，兄弟节点是黑色，兄弟的左子是红色，右子是黑色 - 删除修复情况4：当前节点颜色是黑-黑色，它的兄弟节点是黑色，但是兄弟节点的右子是红色，兄弟节点左子的颜色任意

此时，我们需要调用RB-DELETE-FIXUP(T, x)，来恢复与保持红黑性质的工作。

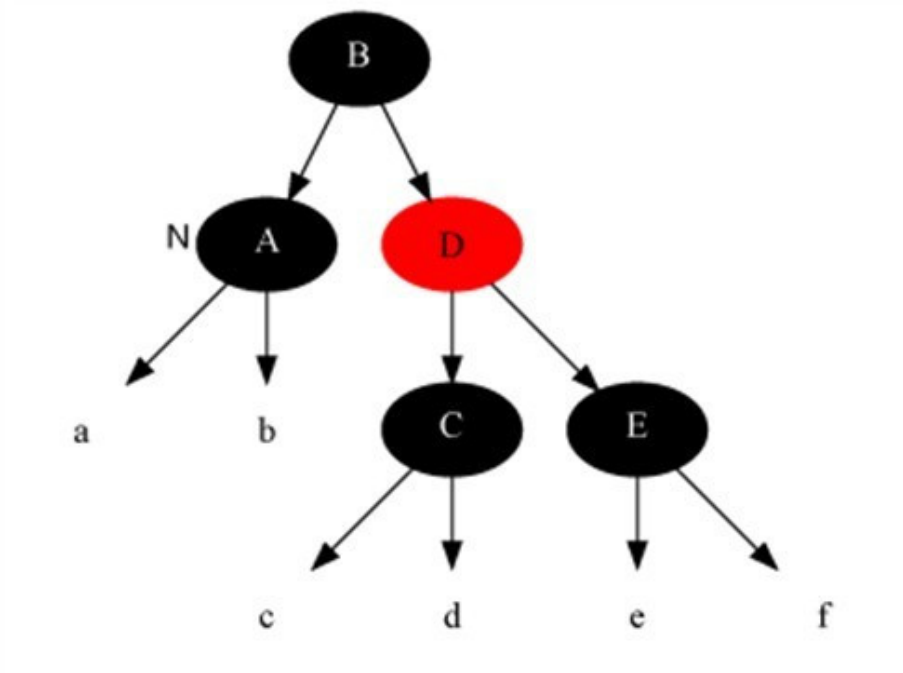
下面，咱们便来分别处理这4种删除修复情况。

删除修复情况1：当前节点是黑+黑且兄弟节点为红色(此时父节点和兄弟节点的子节点分为黑)。

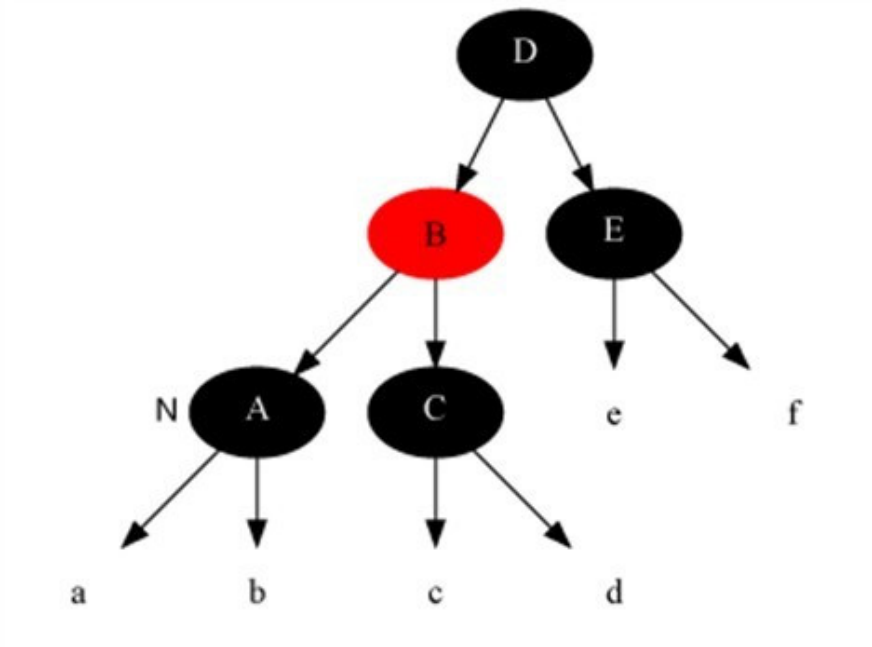
解法：把父节点染成红色，把兄弟结点染成黑色，之后重新进入算法（我们只讨论当前节点是其父节点左孩子时的情况）。此变换后原红黑树性质5不变，而把问题转化为兄弟节点为黑色的情况(注：变化前，原本就未违反性质5，只是为了**把问题转化为兄弟节点为黑色的情况**)。即如下代码操作：

```
//调用RB-DELETE-FIXUP(T, x) 的1-8行代码
 1 while x ≠ root[T] and color[x] = BLACK
 2     do if x = left[p[x]]
 3           then w ← right[p[x]]
 4                if color[w] = RED
 5                   then color[w] ← BLACK                          ▹ Case 1
 6                        color[p[x]] ← RED                         ▹ Case 1
 7                        LEFT-ROTATE(T, p[x])                      ▹ Case 1
 8                        w ← right[p[x]]                           ▹ Case 1
```

变化前：



变化后：

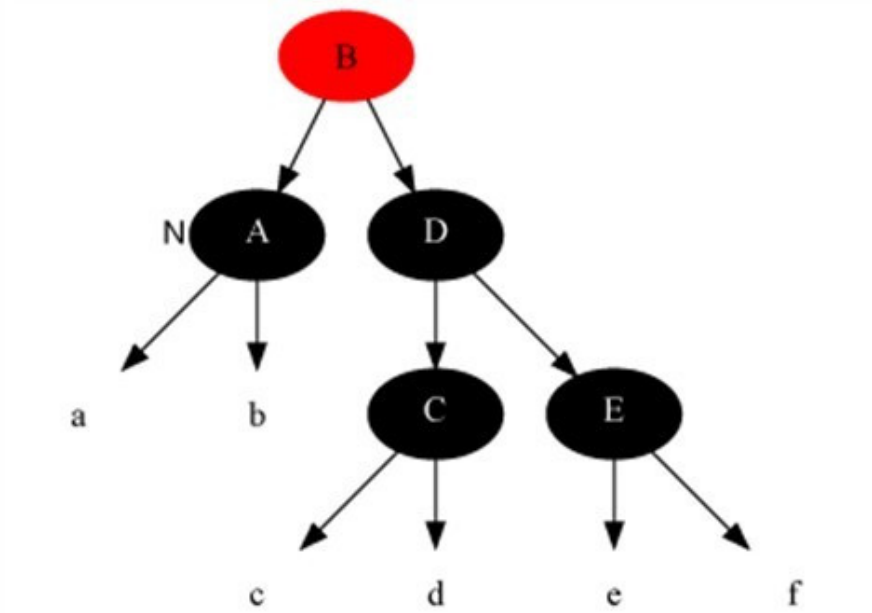


删除修复情况2：当前节点是黑加黑且兄弟是黑色且兄弟节点的两个子节点全为黑色。

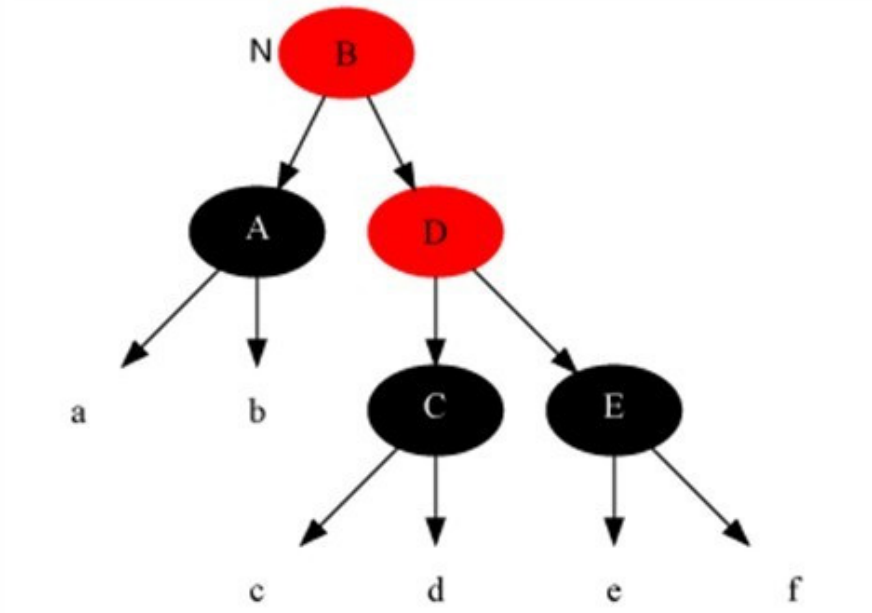
解法：把当前节点和兄弟节点中抽取一重黑色追加到父节点上，把父节点当成新的当前节点，重新进入算法。（此变换后性质5不变），即调用RB-INSERT-FIXUP(T, z) 的第9-10行代码操作，如下：

```
//调用RB-DELETE-FIXUP(T, x) 的9-11行代码
9                if color[left[w]] = BLACK and color[right[w]] = BLACK
10                   then color[w] ← RED                          ▷  Case 2
11                        x p[x]                                  ▷  Case 2
```

变化前：



变化后：



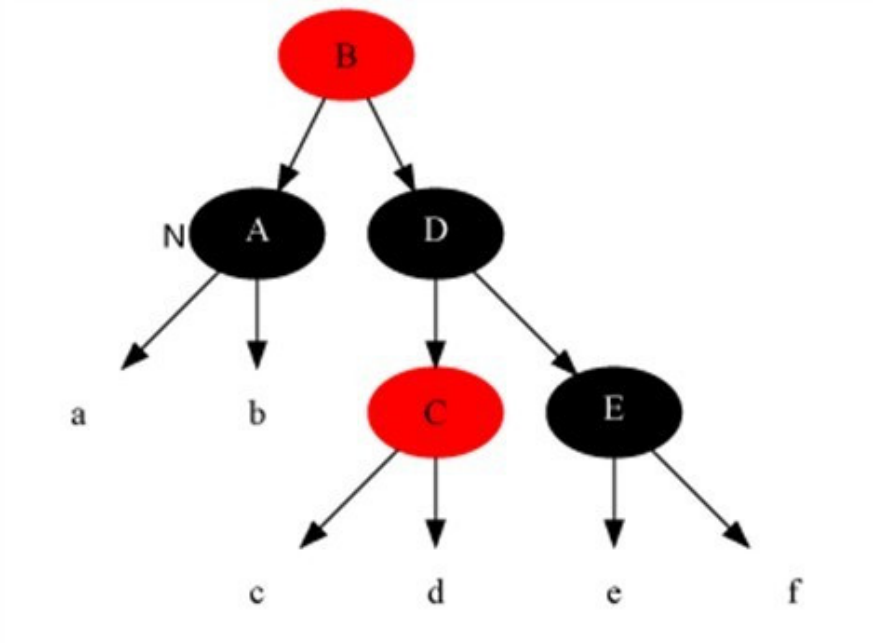
删除修复情况3：当前节点颜色是黑+黑，兄弟节点是黑色，兄弟的左子是红色，右子是黑色。

解法：把兄弟结点染红，兄弟左子节点染黑，之后再在兄弟节点为支点解右旋，之后重新进入算法。此是把当前的情况转化为情况4，而性质5得以保持，

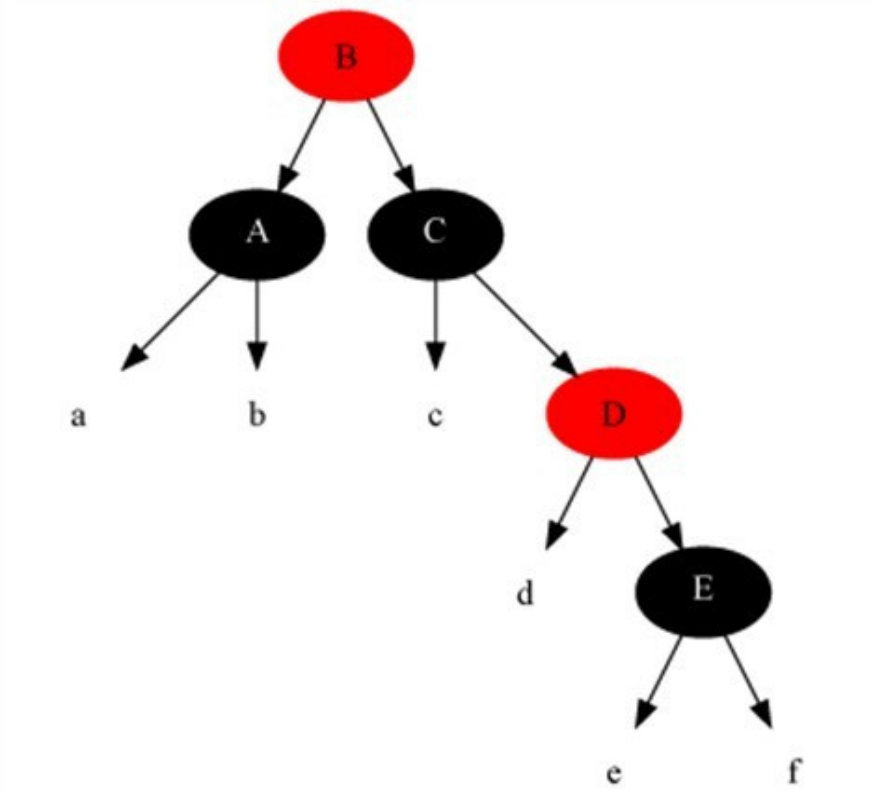
即调用RB-INSERT-FIXUP(T, z) 的第12-16行代码，如下所示：

```
//调用RB-DELETE-FIXUP(T, x) 的第12-16行代码
12                    else if color[right[w]] = BLACK
13                            then color[left[w]] ← BLACK          ▷ Case 3
14                                 color[w] ← RED                  ▷ Case 3
15                                 RIGHT-ROTATE(T, w)              ▷ Case 3
16                                 w ← right[p[x]]                 ▷ Case 3
```

变化前：



变化后：

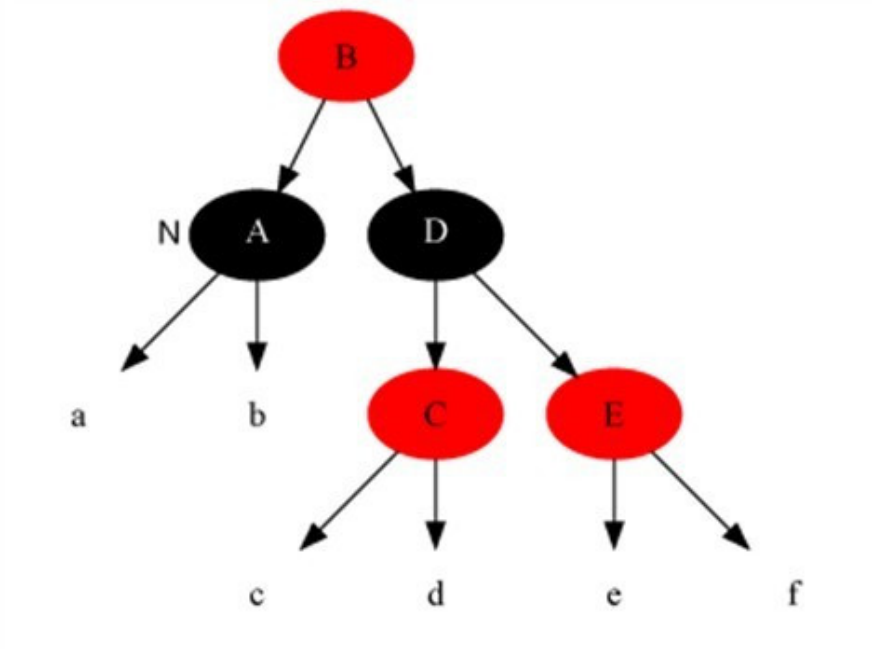


删除修复情况4：当前节点颜色是黑-黑色，它的兄弟节点是黑色，但是兄弟节点的右子是红色，兄弟节点左子的颜色任意。

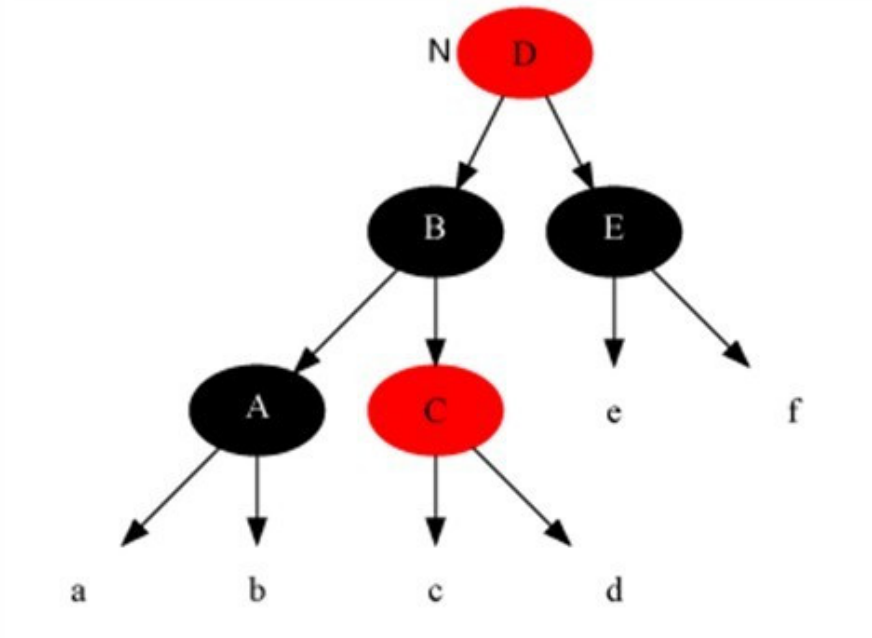
解法：把兄弟节点染成当前节点父节点的颜色，把当前节点父节点染成黑色，兄弟节点右子染成黑色，之后以当前节点的父节点为支点进行左旋，此时算法结束，红黑树所有性质调整正确，即调用RB-INSERT-FIXUP(T, z)的第17-21行代码，如下所示：

```
//调用RB-DELETE-FIXUP(T, x) 的第17-21行代码
17                          color[w] ← color[p[x]]              ▷  Case 4
18                          color[p[x]] ← BLACK                 ▷  Case 4
19                          color[right[w]] ← BLACK             ▷  Case 4
20                          LEFT-ROTATE(T, p[x])                ▷  Case 4
21                          x ← root[T]                         ▷  Case 4
```

变化前：



变化后：



July、二零一四年五月六日、再次更新。

B树

1.前言：

动态查找树主要有：二叉查找树（Binary Search Tree），平衡二叉查找树（Balanced Binary Search Tree），红黑树(Red-Black Tree)，B-tree/B+-tree/ B*-tree (B~Tree)。前三者是典型的二叉查找树结构，其查找的时间复杂度O(log2N)与树的深度相关，那么降低树的深度自然会提高查找效率。

但是咱们有面对这样一个实际问题：就是大规模数据存储中，实现索引查询这样一个实际背景下，树节点存储的元素数量是有限的（如果元素数量非常多的话，查找就退化成节点内部的线性查找了），这样导致二叉查找树结构由于**树的深度过大而造成磁盘I/O读写过于频繁，进而导致查询效率低下**（为什么会出现这种情况，待会在外部存储器-磁盘中有所解释），那么如何减少树的深度（当然是不能减少查询的数据量），一个基本的想法就是：采用**多叉树**结构（由于树节点元素数量是有限的，自然该节点的子树数量也就是有限的）。

也就是说，因为磁盘的操作费时费资源，如果过于频繁的多次查找势必效率低下。那么如何提高效率，即如何避免磁盘过于频繁的多次查找呢？根据磁盘查找存取的次数往往由树的高度所决定，所以，只要我们通过某种较好的树结构减少树的结构尽量减少树的高度，那么是不是便能有效减少磁盘查找存取的次数呢？那这种有效的树结构是一种怎样的树呢？

这样我们就提出了一个新的查找树结构——多路查找树。根据平衡二叉树的启发，自然就想到平衡多路查找树结构，也就是这篇文章所要阐述的第一个主题B~tree，即B树结构(后面，我们将看到，B树的各种操作能使B树保持较低的高度，从而达到有效避免磁盘过于频繁的查找存取操作，从而有效提高查找效率)。

B-tree（B-tree树即B树*，B即Balanced，平衡的意思）这棵神奇的树是在Rudolf Bayer, Edward M. McCreight(1970) 写的一篇论文《Organization and Maintenance of Large Ordered Indices》中首次提出的（wikipedia中：http://en.wikipedia.org/wiki/B-tree，阐述了B-tree名字来源以及相关的开源地址）。

在开始介绍B_{tree}之前，先了解下相关的硬件知识，才能很好的了解为什么需要Btree这种外存数据结构。

2.外存储器—磁盘

计算机存储设备一般分为两种：内存储器(main memory)和外存储器(external memory)。内存存取速度快，但容量小，价格昂贵，而且不能长期保存数据(在

不通电情况下数据会消失)。

外存储器—磁盘是一种直接存取的存储设备(DASD)。它是以存取时间变化不大为特征的。可以直接存取任何字符组，且容量大、速度较其它外存设备更快。

2.1 磁盘的构造

磁盘是一个扁平的圆盘(与电唱机的唱片类似)。盘面上有许多称为磁道的圆圈，数据就记录在这些磁道上。磁盘可以是单片的，也可以是由若干盘片组成的盘组，每一盘片上有两个面。如下图11.3中所示的6片盘组为例，除去最顶端和最底端的外侧面不存储数据之外，一共有10个面可以用来保存信息。

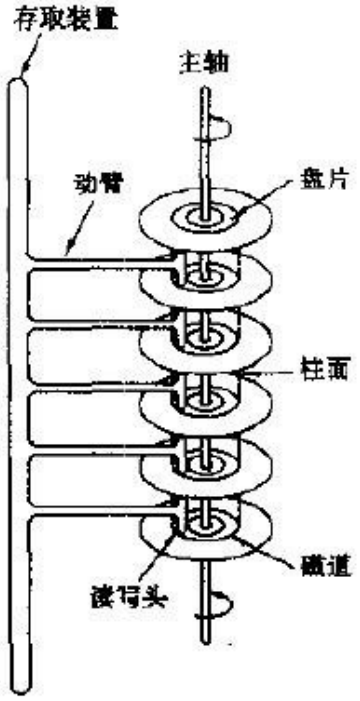


图 11.3　活动头盘示意图

当磁盘驱动器执行读/写功能时。盘片装在一个主轴上，并绕主轴高速旋转，当磁道在读/写头(又叫磁头) 下通过时，就可以进行数据的读 / 写了。

一般磁盘分为固定头盘(磁头固定)和活动头盘。固定头盘的每一个磁道上都有独立的磁头，它是固定不动的，专门负责这一磁道上数据的读/写。

活动头盘 (如上图)的磁头是可移动的。每一个盘面上只有一个磁头(磁头是双向的，因此正反盘面都能读写)。它可以从该面的一个磁道移动到另一个磁道。所有磁头都装在同一个动臂上，因此不同盘面上的所有磁头都是同时移动的(行动整齐划一)。当盘片绕主轴旋转的时候，磁头与旋转的盘片形成一个圆柱体。各个盘面上半径相同的磁道组成了一个圆柱面，我们称为柱面 。因此，柱面的个数也就是盘面上的磁道数。

2.2 磁盘的读/写原理和效率

磁盘上数据必须用一个三维地址唯一标示：柱面号、盘面号、块号(磁道上的盘块)。

读/写磁盘上某一指定数据需要下面3个步骤：

1. 首先移动臂根据柱面号使磁头移动到所需要的柱面上，这一过程被称为定位或查找 。
2. 如上图11.3中所示的6盘组示意图中，所有磁头都定位到了10个盘面的10条磁道上(磁头都是双向的)。这时根据盘面号来确定指定盘面上的磁道。
3. 盘面确定以后，盘片开始旋转，将指定块号的磁道段移动至磁头下。

经过上面三个步骤，指定数据的存储位置就被找到。这时就可以开始读/写操作了。

访问某一具体信息，由3部分时间组成：

- 查找时间(seek time) Ts: 完成上述步骤(1)所需要的时间。这部分时间代价最高，最大可达到0.1s左右。
- 等待时间(latency time) Tl: 完成上述步骤(3)所需要的时间。由于盘片绕主轴旋转速度很快，一般为7200转/分(电脑硬盘的性能指标之一, 家用的普通硬盘的转速一般有5400rpm(笔记本) ，7200rpm几种)。因此一般旋转一圈大约0.0083s。
- 传输时间(transmission time) Tt: 数据通过系统总线传送到内存的时间，一般传输一个字节(byte)大概0.02us=2*10^(-8)s

磁盘读取数据是以盘块(block)**为基本单位的。**位于同一盘块中的所有数据都能被一次性全部读取出来。而磁盘IO代价主要花费在查找时间Ts上。因此我们应该尽量将相关信息存放在同一盘块，同一磁道中。或者至少放在同一

柱面或相邻柱面上，以求在**读/写信息时尽量减少磁头来回移动的次数，避免过多的查找时间Ts。**

所以，在大规模数据存储方面，大量数据存储在外存磁盘中，而在外存磁盘中读取/写入块(block)中某数据时，首先需要定位到磁盘中的某块，如何有效地查找磁盘中的数据，需要一种合理高效的外存数据结构，就是下面所要重点阐述的B-tree结构，以及相关的变种结构：B+-tree结构和B*-tree结构。

3.B- 树

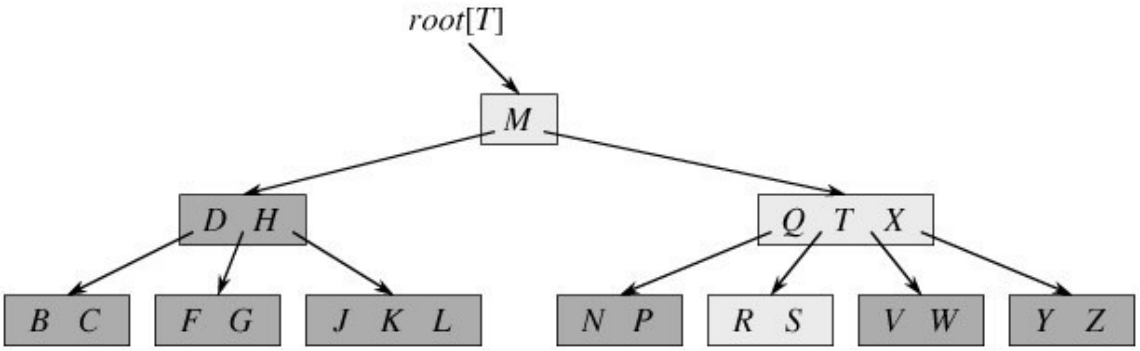
3.1 什么是B-树

具体讲解之前，有一点，**再次强调下：B-树，即为B树**。因为B树的原英文名称为B-tree，而国内很多人喜欢把B-tree译作B-树，其实，这是个非常不好的直译，很容易让人产生误解。如人们可能会以为B-树是一种树，而B树又是一种一种树。而事实上是，**B-tree就是指的B树**。特此说明。

我们知道，B 树是为了磁盘或其它存储设备而设计的一种多叉（下面你会看到，相对于二叉，B树每个内结点有多个分支，即多叉）平衡查找树。与本blog之前介绍的红黑树很相似，但在降低磁盘I/0操作方面要更好一些。许多数据库系统都一般使用B树或者B树的各种变形结构，如下文即将要介绍的B+树，B*树来存储信息。

B树与红黑树最大的不同在于，B树的结点可以有许多子女，从几个到几千个。那为什么又说B树与红黑树很相似呢?因为与红黑树一样，一棵含n个结点的B树的高度也为$O(\lg n)$，但可能比一棵红黑树的高度小许多，因为它的分支因子比较大。所以，B树可以在$O(\log n)$时间内，实现各种如插入（insert），删除（delete）等动态集合操作。

如下图所示，即是一棵B树，一棵关键字为英语中辅音字母的B树，现在要从树中查找字母R（包含n[x]个关键字的内结点x，x有n[x]+1个子女（也就是说，一个内结点x若含有n[x]个关键字，那么x将含有n[x]+1个子女）。所有的叶结点都处于相同的深度，带阴影的结点为查找字母R时要检查的结点）：



相信，从上图你能轻易的看到，一个内结点x若含有n[x]个关键字，那么x将含有n[x]+1个子女。如含有2个关键字D H的内结点有3个子女，而含有3个关键字Q T X的内结点有4个子女。

B树的定义，从下文中，你将看到，或者是用阶，或者是用度，如下段文字所述：

> Unfortunately, the literature on B-trees is not uniform in its use of terms relating to B-Trees. (Folk & Zoellick 1992, p. 362) Bayer & McCreight (1972), Comer (1979), and others define the order of B-tree as the minimum number of keys in a non-root node. Folk & Zoellick (1992) points out that terminology is ambiguous because the maximum number of keys is not clear. An order 3 B-tree might hold a maximum of 6 keys or a maximum of 7 keys. (Knuth 1998,TAOCP p. 483) avoids the problem by defining the order to be maximum number of children (which is one more than the maximum number of keys).

Definition

According to Knuth's definition, a B-tree of order *m* (the maximum number of children for each node) is a tree which satisfies the following properties:

1. Every node has at most *m* children.
2. Every node (except root) has at least $\lceil m/2 \rceil$ children.
3. The root has at least two children if it is not a leaf node.
4. A non-leaf node with *k* children contains *k*−1 keys.
5. All leaves appear in the same level, and carry information.

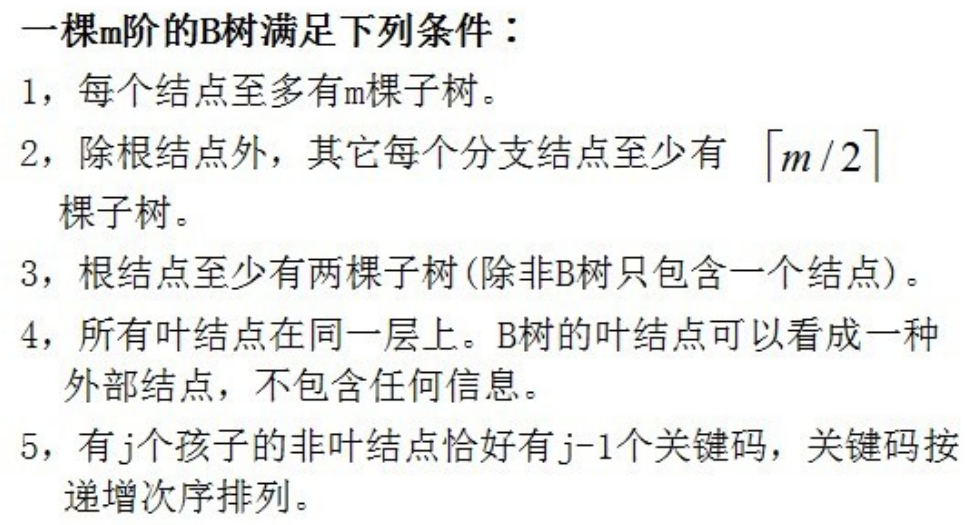
from http://en.wikipedia.org/wiki/Btree#Technical_description.

用阶定义的**B树**

B 树又叫平衡多路查找树。**一棵m阶的B 树** (注:切勿简单的认为一棵m阶的B树是m叉树,虽然存在四叉树,八叉树,KD树，及vp/R树/R*树/R+树/X树/M树/线段树/希尔伯特R树/优先R树等空间划分树，但与B树完全不等同)**的特性如下:**

1. 树中每个结点最多含有m个孩子(m>=2);
2. 除根结点和叶子结点外,其它每个结点至少有[ceil(m / 2)]个孩子(其中ceil(x)是一个取上限的函数);
3. 若根结点不是叶子结点,则至少有2个孩子(特殊情况:没有孩子的根结点,即根结点为叶子结点,整棵树只有一个根节点);
4. 所有叶子结点都出现在同一层,叶子结点不包含任何关键字信息(可以看做是外部结点或查询失败的结点,实际上这些结点不存在,指向这些结点的指针都为null);(读者反馈@冷岳:这里有错，叶子节点只是没有孩子和指向孩子的指针，这些节点也存在，也有元素。@研究者July:其实,关键是把什么当做叶子结点,因为如红黑树中,每一个NULL指针即当做叶子结点,只是没画出来而已)。
5. 每个非终端结点中包含有n个关键字信息:(n,P0,K1,P1,K2,P2,……,Kn,Pn)。其中:

a. Ki (i=1…n)为关键字，且关键字按顺序升序排序K(i-1)< Ki。
b. Pi为指向子树根的结点，且指针P(i-1)指向子树种所有结点的关键字均小于Ki,但都大于K(i-1)。
c. 关键字的个数n必须满足:[ceil(m / 2)-1]<= n <= m-1。如下图所示:



用度定义的**B**树

针对上面的5点，再阐述下：B树中每一个结点能包含的关键字（如之前上面的*D H*和*Q T X*）数有一个上界和下界。这个下界可以用一个称作B树的最小度数（算法导论中文版上译作度数，最小度数即内节点中节点最小孩子数目）m（m>=2）表示。

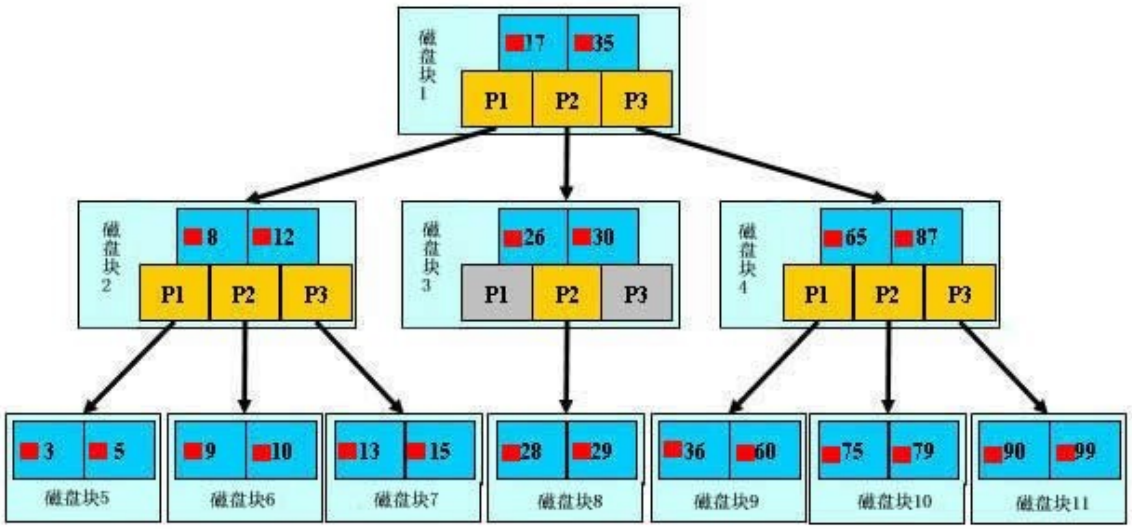
- 每个非根的内结点至多有m个子女，每个非根的结点必须至少含有m-1个关键字，如果树是非空的，则根结点至少包含一个关键字；
- 每个结点可包含至多2m-1个关键字。所以一个内结点至多可有2m个子女。如果一个结点恰好有2m-1个关键字，我们就说这个结点是满的（而稍后介绍的B*树作为B树的一种常用变形，B*树中要求每个内结点至少为2/3满，而不是像这里的B树所要求的至少半满）；
- **当关键字数m=2（t=2的意思是，mmin=2，m可以>=2）时的B树是最简单的**（有很多人会因此误认为B树就是二叉查找树，但二叉查找树就是二叉查找树，B树就是B树，B树是一棵含有m（m>=2）个关键字的平衡多路查找树），此时，每个内结点可能因此而含有2个、3个或4个子女，亦即一棵2-3-4树，然而在实际中，通常采用大得多的t值。

B树中的每个结点根据实际情况可以包含大量的关键字信息和分支(当然是不能超过磁盘块的大小，根据磁盘驱动(disk drives)的不同，一般块的大小在1k~4k左右)；这样树的深度降低了，这就意味着查找一个元素只要很少结点从外存磁盘中读入内存，很快访问到要查找的数据。如果你看完上面关于B树定义的介绍，思维感觉不够清晰，请继续参阅下文第6小节、B树的插入、删除操作 部分。

3.2 B树的类型和节点定义

B树的类型和节点定义如下图所示：

```
#define m 1024
struct BTNode;
typedef struct BTNode * PBTNode;
struct BTNode  {
    int  keyNum; /* 实际关键字个数，keyNum<m */
    PBTNode  parent;                /* 指向父结点 */
    PBTNode  *ptr;    /* 子树指针向量: ptr[0]…ptr[keyNum] */
    KeyType  *key;    /* 关键字向量: key [0]…key [keyNum－1] */
  }
typedef struct BTNode *BTree;
typedef BTree * PBTree;
```



3.3 文件查找的具体过程(涉及磁盘IO操作)

为了简单，这里用少量数据构造一棵3叉树的形式，实际应用中的B树结点中关键字很多的。上面的图中比如根结点，其中17表示一个磁盘文件的文件名；小红方块表示这个17文件内容在硬盘中的存储位置；p1表示指向17左子树的指针。

其结构可以简单定义为：

```
typedef struct {
    /*文件数*/
    int  file_num;
    /*文件名(key)*/
    char * file_name[max_file_num];
    /*指向子节点的指针*/
     BTNode * BTptr[max_file_num+1];
     /*文件在硬盘中的存储位置*/
     FILE_HARD_ADDR offset[max_file_num];
}BTNode;
```

假如每个盘块可以正好存放一个B树的结点（正好存放2个文件名）。那么一个BTNODE结点就代表一个盘块，而子树指针就是存放另外一个盘块的地址。

下面，咱们来模拟下查找文件29的过程：

1. 根据根结点指针找到文件目录的根磁盘块1，将其中的信息导入内存。【磁盘IO操作 1次】
2. 此时内存中有两个文件名17、35和三个存储其他磁盘页面地址的数据。

   根据算法我们发现：17<29<35，因此我们找到指针p2。
3. 根据p2指针，我们定位到磁盘块3，并将其中的信息导入内存。【磁盘IO操作 2次】
4. 此时内存中有两个文件名26，30和三个存储其他磁盘页面地址的数据。

   根据算法我们发现：26<29<30，因此我们找到指针p2。
5. 根据p2指针，我们定位到磁盘块8，并将其中的信息导入内存。【磁盘IO操作 3次】
6. 此时内存中有两个文件名28，29。根据算法我们查找到文件名29，并定位了该文件内存的磁盘地址。

分析上面的过程，发现需要**3次磁盘IO操作和3次内存查找**操作。关于内存中的文件名查找，由于是一个有序表结构，可以利用折半查找提高效率。至于IO操作是影响整个B树查找效率的决定因素。

当然，如果我们使用平衡二叉树的磁盘存储结构来进行查找，磁盘4次，最多5次，而且文件越多，B树比平衡二叉树所用的磁盘IO操作次数将越少，效率也越高。

3.4 B树的高度

根据上面的例子我们可以看出，对于辅存做IO读的次数取决于B树的高度。而B树的高度由什么决定的呢？

若B树某一非叶子节点包含N个关键字，则此非叶子节点含有N+1个孩子结点，而所有的叶子结点都在第I层，我们可以得出：

1. 因为根至少有两个孩子，因此第2层至少有两个结点。
2. 除根和叶子外，其它结点至少有 ┌m/2┐ 个孩子，
3. 因此在第3层至少有2* ┌m/2┐ 个结点，
4. 在第4层至少有2*(┌m/2┐ ^2)个结点，
5. 在第 I 层至少有2*(┌m/2┐ ^(l-2))个结点，于是有：N+1 ≥ 2 ┌m/2┐ I-2；
6. 考虑第L层的结点个数为N+1，那么2*(┌m/2┐ ^(l-2))≤N+1，也就是L层的最少结点数刚好达到N+1个，即：I≤ log ┌m/2┐ ((N+1)/2)+2；

所以

- 当B树包含N个关键字时，B树的最大高度为l-1（因为计算B树高度时，叶结点所在层不计算在内），即：l - 1 = **log ┌m/2┐ ((N+1)/2)+1**。

这个B树的高度公式从侧面显示了B树的查找效率是相当高的。

曾在一次面试中被问到，**一棵含有N个总关键字数的m阶的B树的最大高度是多少**?答曰：

log_ceil（m/2）(N+1)/2 + 1（上面中关于m阶B树的第1点特性已经提到：树中每个结点含有最多含有m个孩子，即m满足：ceil(m/2)<=**m**<=m。而树中每个结点含孩子数越少，树的高度则越大，故如此）。在2012微软4月份的笔试中也问到了此问题。

此外，还有读者反馈，说上面的B树的高度计算公式与算法导论一书上的不同，而后我特意翻看了算法导论第18章关于B树的高度一节的内容，如下图所示：

B 树的高度

B树上大部分操作所需的磁盘存取次数与B树的高度成正比。下面来分析B树的最坏情况高度。

定理 18.1 如果 $n \geqslant 1$，则对任意一棵包含 n 个关键字、高度为 h、最小度数 $t \geqslant 2$ 的B树 T，有：

$$h \leqslant \log_t \frac{n+1}{2}$$

证明： 如果一棵B树的高度为 h，其根结点包含至少一个关键字而其他结点包含至少 $t-1$ 个关键字。这样，在深度1至少有两个结点，在深度2至少有 $2t$ 个结点，在深度3至少有 $2t^2$ 个结点，等等，直到深度 h 至少有 $2t^{h-1}$ 个结点。图 18-4 给出了 $h=3$ 时的一棵树。因此，关键字的
439 个数 n 满足不等式

$$n \geqslant 1 + (t-1)\sum_{i=1}^{h} 2t^{i-1} = 1 + 2(t-1)\left(\frac{t^h - 1}{t-1}\right) = 2t^h - 1$$

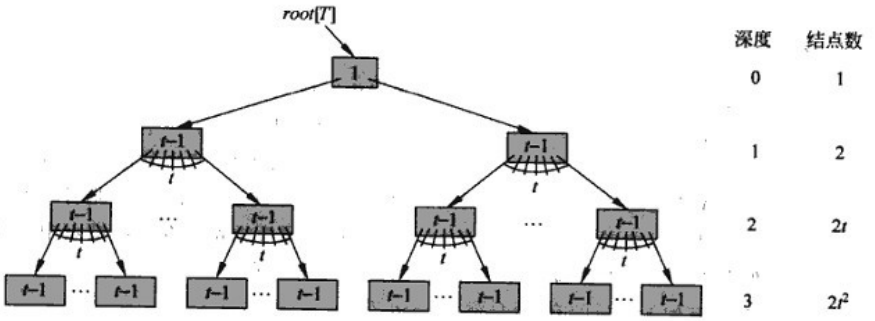


图 18-4 一棵高度为3的B树，它包含最小可能的关键字数。在每个结点 x 内显示的是 $n[x]$

⊖ B* 树是B树的另一种常用变形，在这种树中，要求每个内结点至少为2/3满，而不是像B树要求的至少半满。

在上图中书上所举的例子中，也许，根据我们大多数人的理解，它的高度应该是4，而书上却说的是“一棵高度为3的B树”。我想，此时，你也就明白了，算法导论一书上的高度的定义是从“0”开始计数的，而我们中国人的习惯是树的高度是从“1”开始计数的。特此说明。July、二零一二年九月二十七日。

6、B树的插入、删除操作

上面第3小节简单介绍了利用B树这种结构如何访问外存磁盘中的数据的情况，下面咱们通过另外一个实例来对这棵B树的插入（insert），删除（delete）基本操作进行详细的介绍。

但在此之前，咱们还得简单回顾下一棵m阶的B 树的特性，如下：

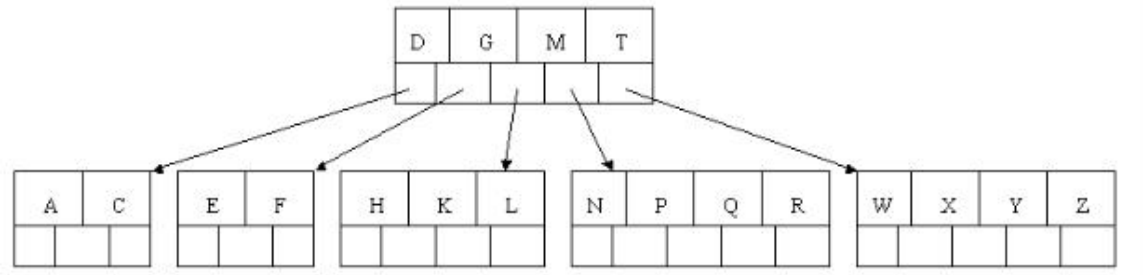
1. 树中每个结点含有最多含有m个孩子，即m满足：ceil(m/2)<=**m**<=m；
2. 除根结点和叶子结点外，其它每个结点至少有[ceil(m / 2)]个孩子（其中ceil(x)是一个取上限的函数）；

3. 若根结点不是叶子结点，则至少有2个孩子（特殊情况：没有孩子的根结点，即根结点为叶子结点，整棵树只有一个根节点）；
4. 所有叶子结点都出现在同一层，叶子结点不包含任何关键字信息(可以看做是外部结点或查询失败的结点，实际上这些结点不存在，指向这些结点的指针都为null)；
5. 每个非终端结点中包含有n个关键字信息：(n，P0，K1，P1，K2，P2，……，Kn，Pn)。其中：

a. Ki (i=1…n)为关键字，且关键字按顺序升序排序K(i-1)< Ki。
b. Pi为指向子树根的结点，且指针P(i-1)指向子树种所有结点的关键字均小于Ki，但都大于K(i-1)。
c. 除根结点之外的结点的关键字的个数n必须满足：[ceil(m / 2)-1]<= n <= m-1（叶子结点也必须满足此条关于关键字数的性质，根结点除外）。

ok，下面咱们以一棵**5阶**（即树中任一结点至多含有4个关键字，5棵子树）B树实例进行讲解(如下图所示)：

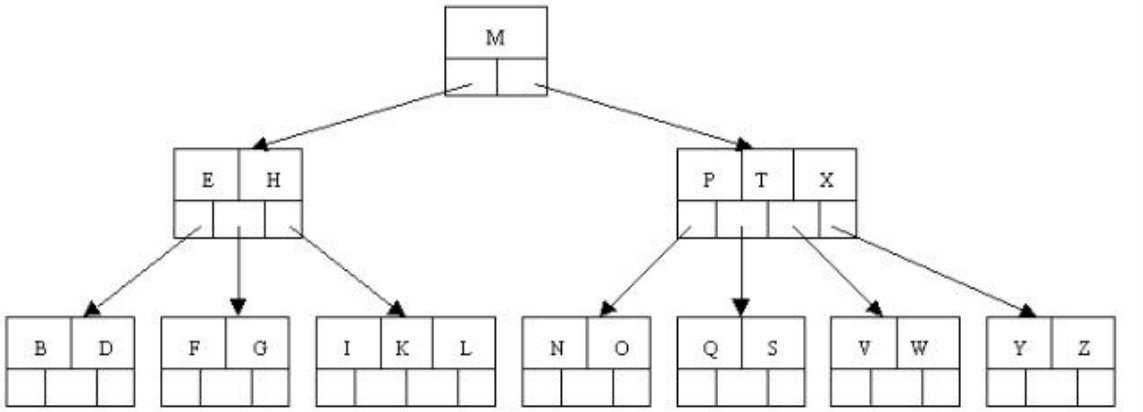
备注：**关键字数（2-4个）针对–非根结点（包括叶子结点在内），孩子数（3-5个）–针对根结点和叶子结点之外的内结点。当然，根结点是必须至少有2个孩子的，不然就成直线型搜索树了。**下图中，读者可以看到关键字数2-4个，内结点孩子数3-5个：



关键字为大写字母，顺序为字母升序。

结点定义如下：

```
typedef struct{
   int Count;          // 当前节点中关键元素数目
   ItemType Key[4];    // 存储关键字元素的数组
   long Branch[5];     // 伪指针数组，（记录数目）方便判断合并和分裂的情况
} NodeType;
```



6.1 插入(insert)操作

插入一个元素时,首先在B树中是否存在，如果不存在，即在叶子结点处结束，然后在叶子结点中插入该新的元素，注意：如果叶子结点空间足够，这里需要向右移动该叶子结点中大于新插入关键字的元素，如果空间满了以致没有足够的空间去添加新的元素，则将该**结点**进行“分裂”，将一半数量的关键字元素分裂到新的其相邻右结点中，中间关键字元素上移到父结点中（当然，如果父结点空间满了，也同样需要“分裂”操作），而且当结点中关键元素向右移动了，相关的指针也需要向右移。如果在根结点插入新元素，空间满了，则进行分裂操作，这样原来的根结点中的中间关键字元素向上移动到新的根结点中，因此导致树的高度增加一层。如下图所示：

在B树中插入关键码key的思路

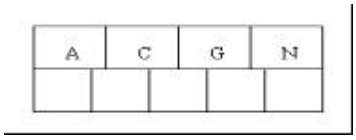
对高度为h的m阶B树，新结点一般是插在第h层。通过检索可以确定关键码应插入的结点位置。然后分两种情况讨论：

1，若该结点中关键码个数小于m-1，则直接插入即可。

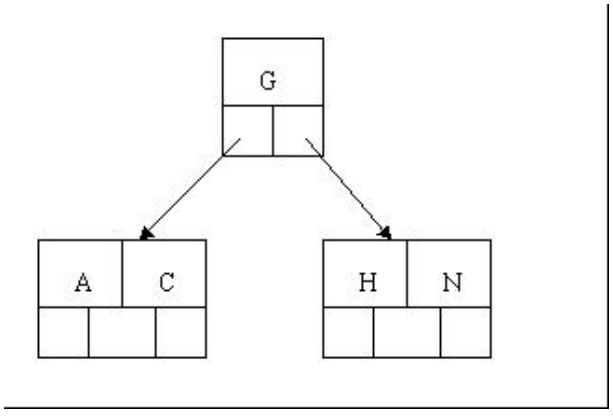
2，若该结点中关键码个数等于m-1，则将引起结点的分裂。以中间关键码为界将结点一分为二，产生一个新结点，并把中间关键码插入到父结点（h-1层）中；

重复上述工作，最坏情况一直分裂到根结点，建立一个新的根结点，整个B树增加一层。

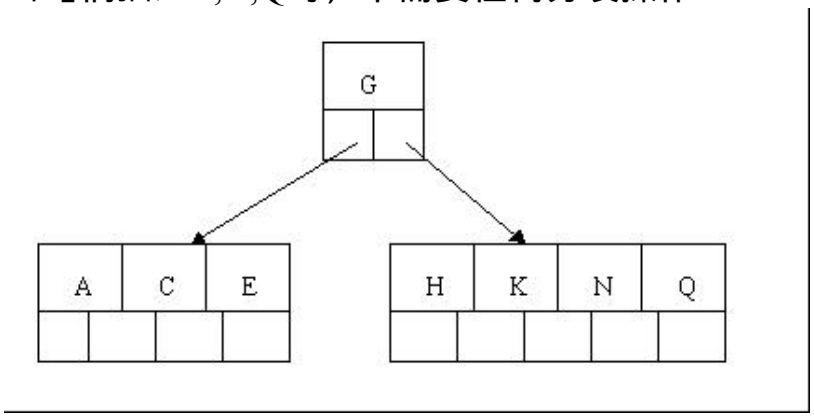
1. OK,下面咱们通过一个实例来逐步讲解下。插入以下字符字母到一棵空的B 树中（非根结点**关键字数**小了(小于2个)就合并,大了(超过4个)就分裂):C N G A H E K Q M F W L T Z D P R X Y S,首先,结点空间足够,4个字母插入相同的结点中,如下图:



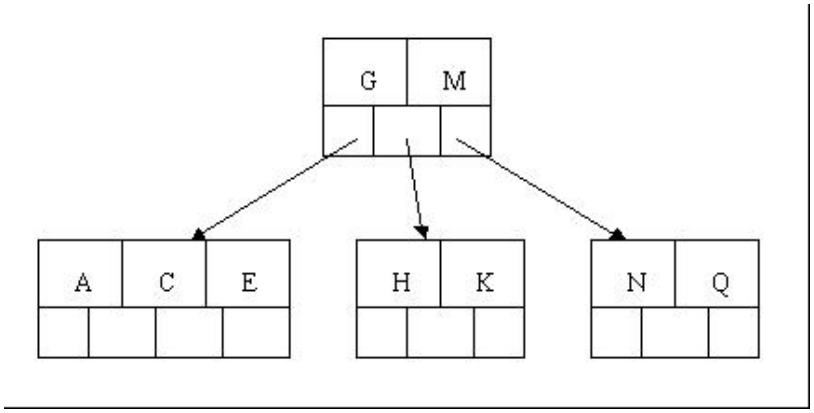
2. 当咱们试着插入H结点时，发现空间不够，以致将其分裂成2个结点，移动中间元素G上移到新的根结点中,在实现过程中,咱们把A和C留在当前结点中,而H和N放置新的其右邻居结点中。如下图:



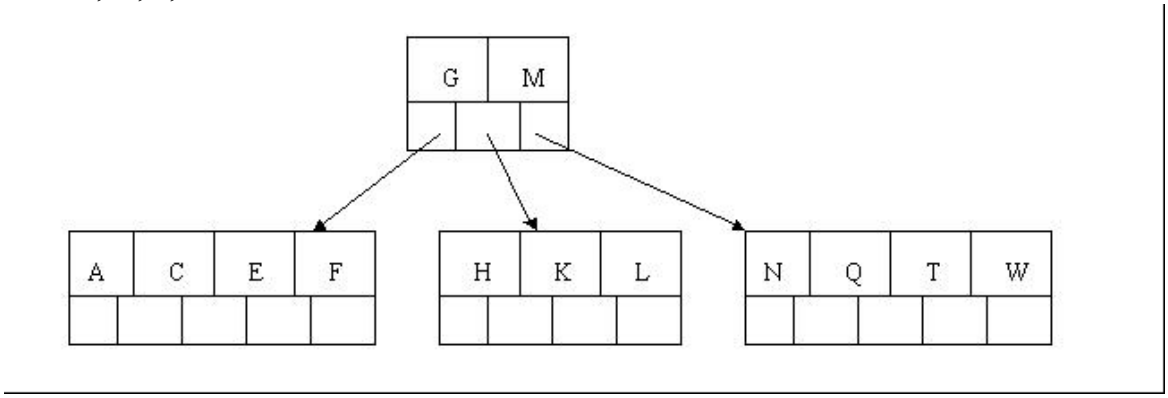
3. 当咱们插入E,K,Q时，不需要任何分裂操作



4. 插入M需要一次分裂，注意M恰好是中间关键字元素，以致向上移到父节点中

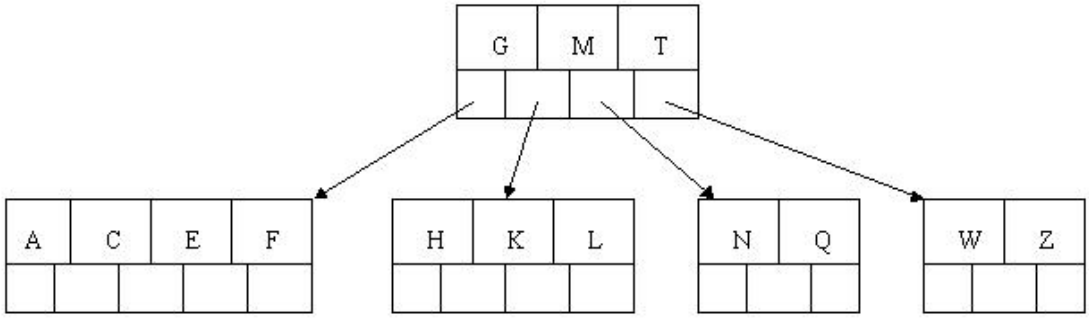


5. 插入F,W,L,T不需要任何分裂操作

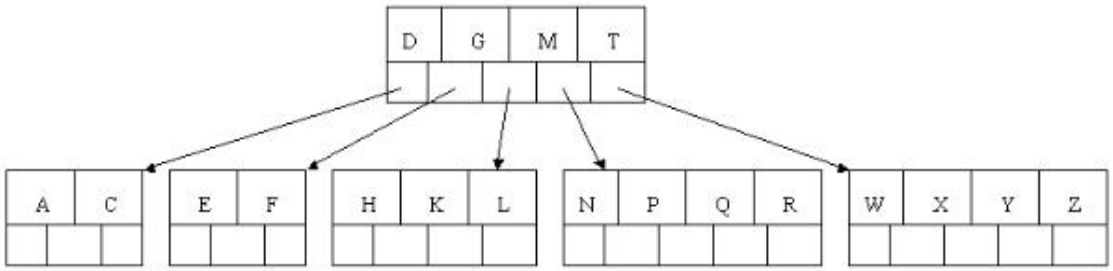


6. 插入Z时，最右的叶子结点空间满了，需要进行分裂操作，中间元素T上移到父节点中，注意通过上移中间元素，树最终还是保持平衡，分裂结

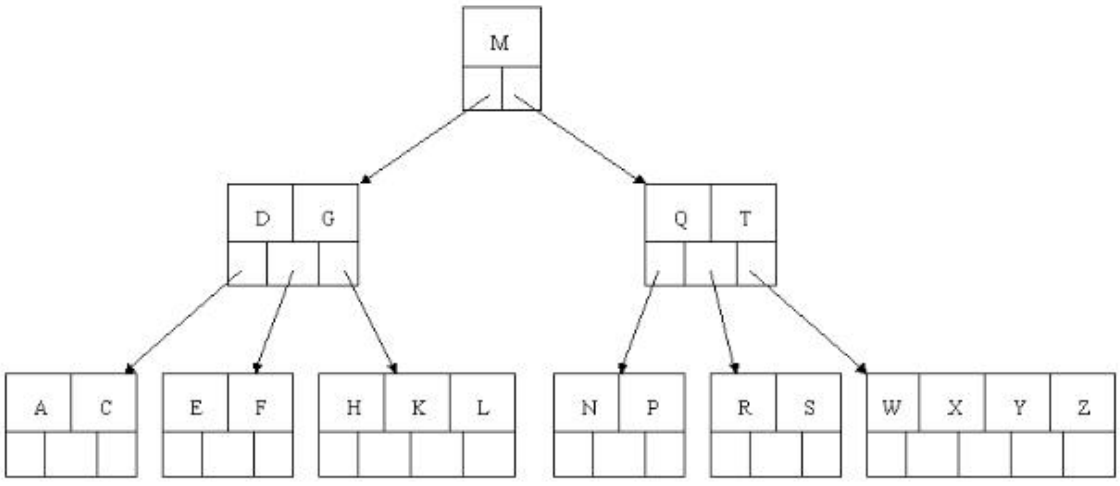
果的结点存在2个关键字元素。



7. 插入D时，导致最左边的叶子结点被分裂，D恰好也是中间元素，上移到父节点中，然后字母P,R,X,Y陆续插入不需要任何分裂操作（别忘了，任一结点课允许最多有4个关键字）。



8. 最后，当插入S时，含有N,P,Q,R的结点需要分裂，把中间元素Q上移到父节点中，但是问题来了，因为Q上移导致父结点 “D G M T” 也满了，所以也要进行分裂，将父节点中的中间元素M上移到新形成的根结点中，从而致使树的高度增加一层。



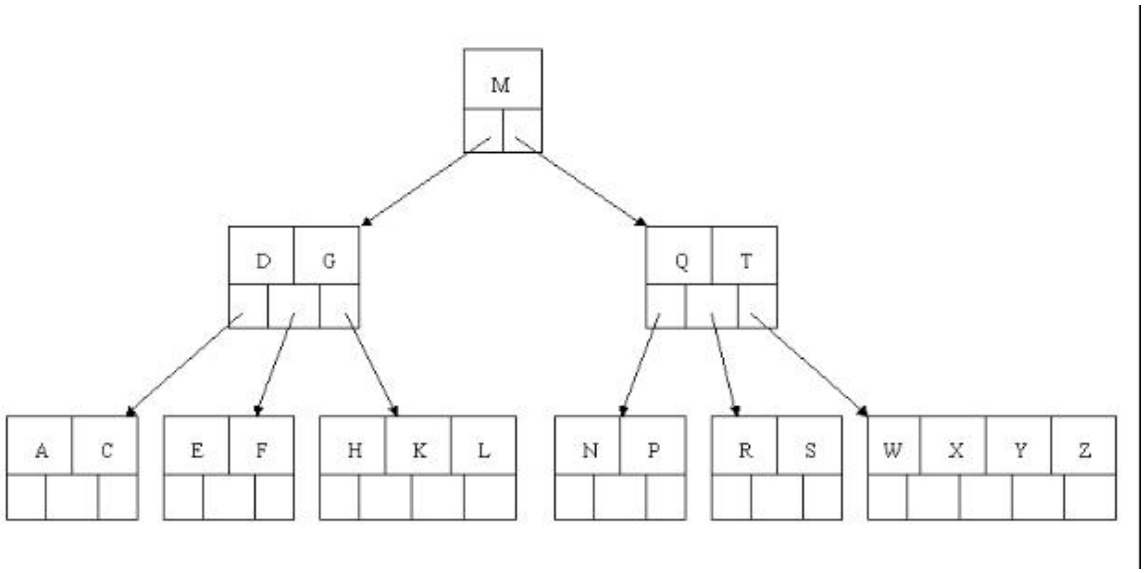
6.2、删除(delete)操作

下面介绍删除操作，删除操作相对于插入操作要考虑的情况多点。

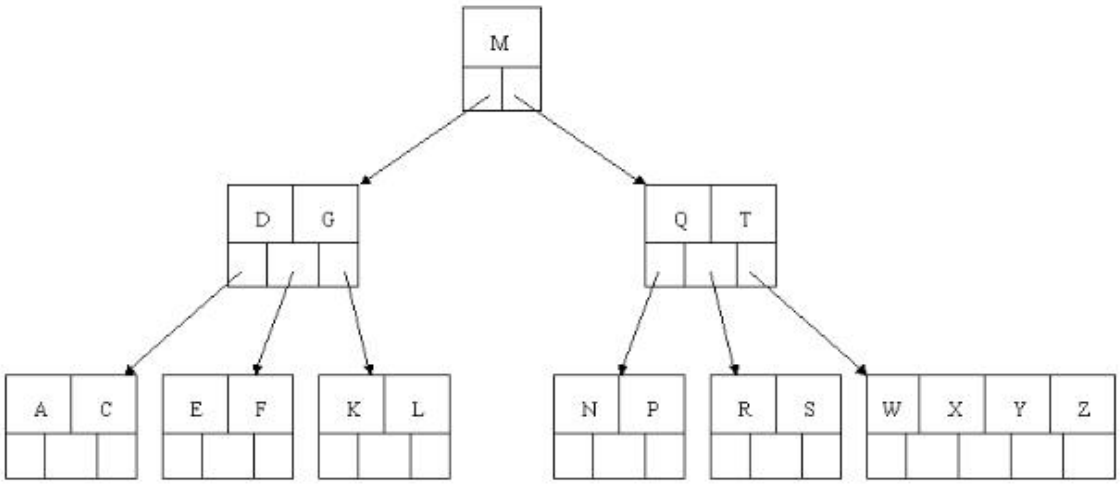
首先查找B树中需删除的元素,如果该元素在B树中存在，则将该元素在其结点中进行删除，如果删除该元素后，首先判断该元素是否有左右孩子结点，如果有，则上移孩子结点中的某相近元素(“左孩子最右边的节点”或“右孩子最左边的节点”)到父节点中，然后是移动之后的情况；如果没有，直接删除后，移动之后的情况。

删除元素，移动相应元素之后，如果某结点中元素数目（即关键字数）小于**ceil(m/2)-1**，则需要看其某相邻兄弟结点是否丰满（结点中元素个数大于ceil(m/2)-1）（**还记得第一节中关于B树的第5个特性中的c点么?：c)除根结点之外的结点（包括叶子结点）的关键字的个数n必须满足：（ceil(m / 2)-1）<= n <= m-1 。m表示最多含有m个孩子，n表示关键字数。在本小节中举的一颗B树的示例中，关键字数n满足：2<=n<=4**），如果丰满，则向父节点借一个元素来满足条件；如果其相邻兄弟都刚脱贫，即借了之后其结点数目小于ceil(m/2)-1，则该结点与其相邻的某一兄弟结点进行“合并”成一个结点，以此来满足条件。那咱们通过下面实例来详细了解吧。

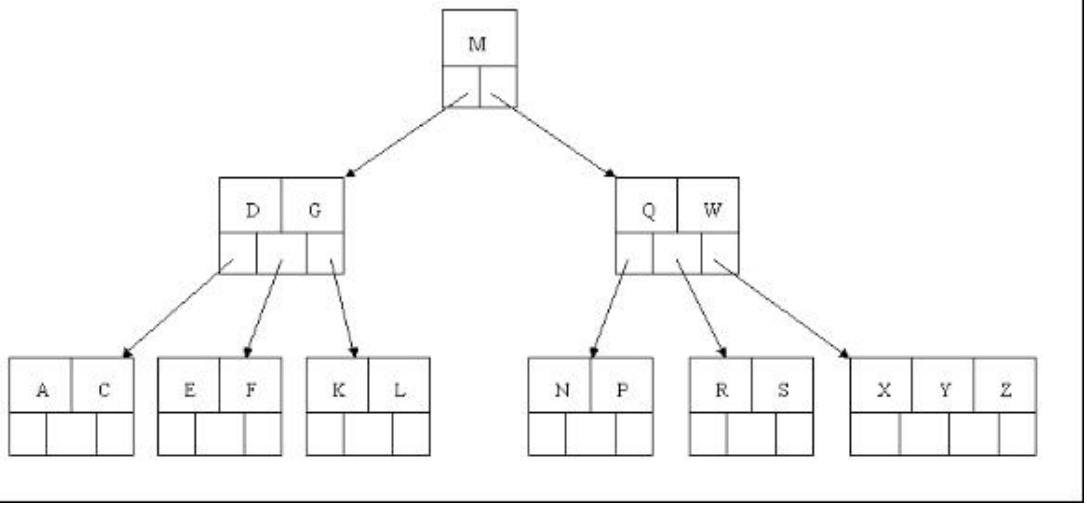
以上述插入操作构造的一棵5阶B树（树中最多含有m（m=5）个孩子，因此关键字数最小为ceil(m / 2)-1=2。还是这句话，**关键字数小了（小于2个）就合并，大了（超过4个）就分裂）**为例，依次删除H,T,R,E。



1. 首先删除元素H，当然首先查找H，H在一个叶子结点中，且该叶子结点元素数目3大于最小元素数目ceil(m/2)-1=2，则操作很简单，咱们只需要移动K至原来H的位置，移动L至K的位置（也就是结点中删除元素后面的元素向前移动）

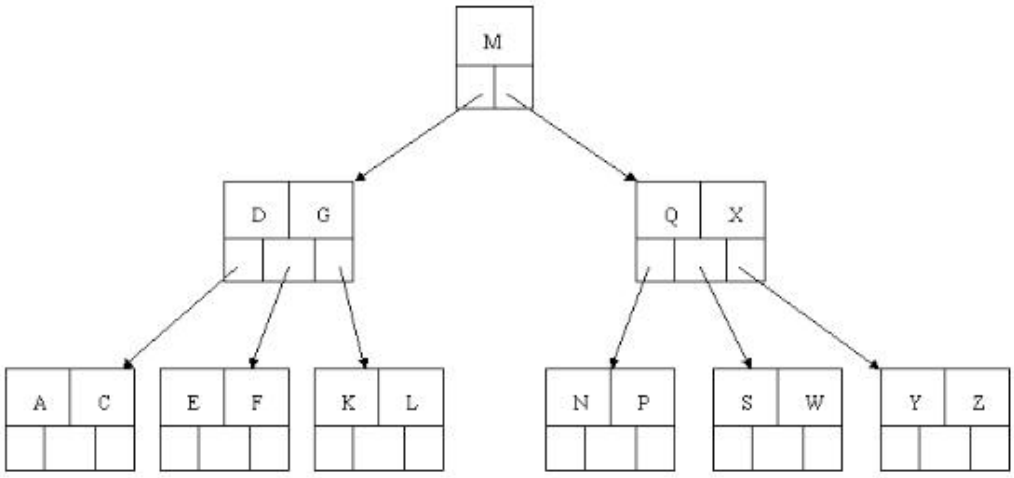


2. 下一步，删除T,因为T没有在叶子结点中，而是在中间结点中找到，咱们发现他的继承者W(字母升序的下个元素)，将W上移到T的位置，然后将原包含W的孩子结点中的W进行删除，这里恰好删除W后，该孩子结点中元素个数大于2，无需进行合并操作。

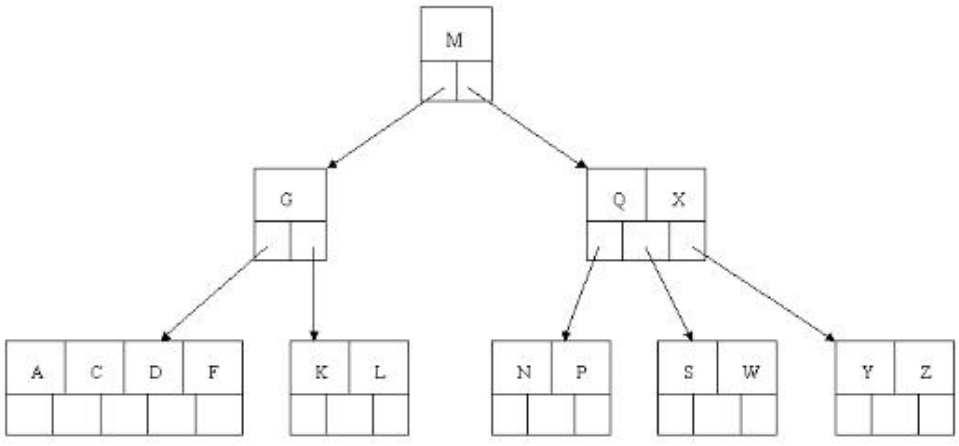


3. 下一步删除R，R在叶子结点中,但是该结点中元素数目为2，删除导致只有1个元素，已经小于最小元素数目ceil(5/2)-1=2,而由前面我们已经知道：**如果其某个相邻兄弟结点中比较丰满（元素个数大于ceil(5/2)-1=2），则可以向父结点借一个元素，然后将最丰满的相邻兄弟结点中上移最后或最前一个元素到父节点中**（有没有看到红黑树中左

旋操作的影子?)，在这个实例中，右相邻兄弟结点中比较丰满(3个元素大于2)，所以先向父节点借一个元素W下移到该叶子结点中，代替原来S的位置，S前移；然后X在相邻右兄弟结点中上移到父结点中，最后在相邻右兄弟结点中删除X，后面元素前移。

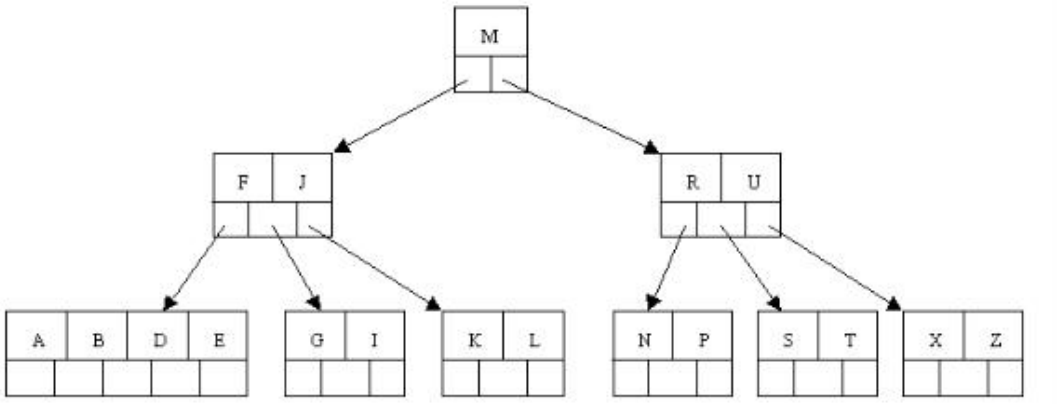


4. 最后一步删除E，删除后会导致很多问题，因为E所在的结点数目刚好达标，刚好满足最小元素个数(ceil(5/2)-1=2),而相邻的兄弟结点也是同样的情况，删除一个元素都不能满足条件，所以需要**该节点与某相邻兄弟结点进行合并操作**；首先移动父结点中的元素(该元素在两个需要合并的两个结点元素之间)下移到其子结点中，然后将这两个结点进行合并成一个结点。所以在该实例中，咱们首先将父节点中的元素D下移到已经删除E而只有F的结点中，然后将含有D和F的结点和含有A,C的相邻兄弟结点进行合并成一个结点。



5. 也许你认为这样删除操作已经结束了，其实不然，在看看上图，对于这种特殊情况，你立即会发现父节点只包含一个元素G，没达标(因为非根节

点包括叶子结点的关键字数n必须满足于2=最小元素个数2)，这样就可以想父结点借元素了，把父结点中的J下移到该结点中，相应的如果结点中J后有元素则前移，然后相邻兄弟结点中的第一个元素（或者最后一个元素）上移到父节点中，后面的元素（或者前面的元素）前移（或者后移）；注意含有K，L的结点以前依附在M的左边，现在变为依附在J的右边。这样每个结点都满足B树结构性质。



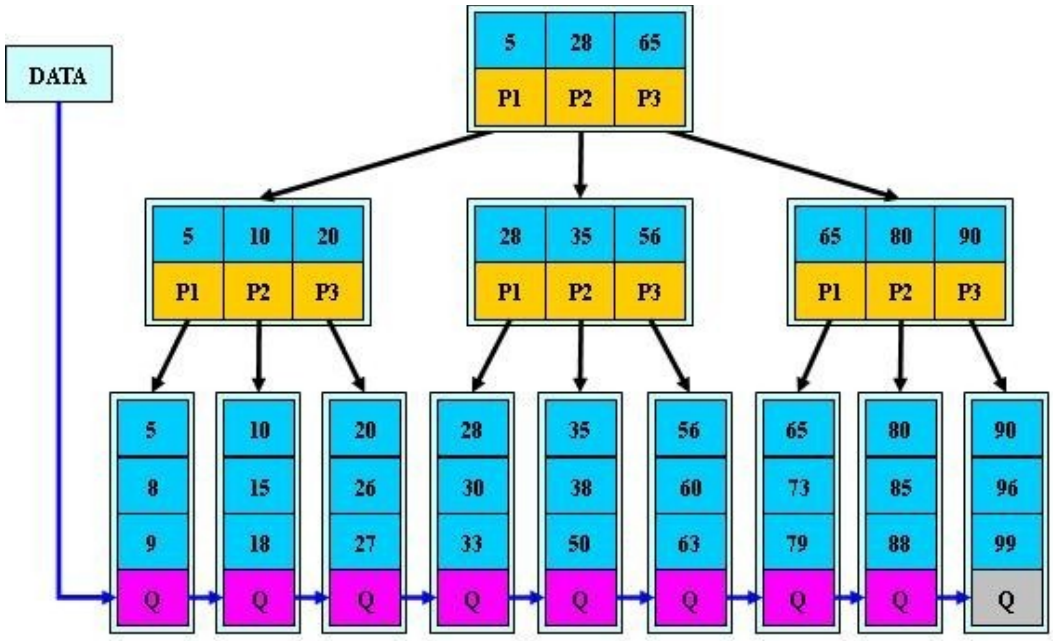
从以上操作可看出：除根结点之外的结点（包括叶子结点）的关键字的个数n满足：（ceil(m / 2)-1）<= n <= m-1，即2<=n<=4。这也佐证了咱们之前的观点。删除操作完。

4.B+-tree

B+-tree：是应文件系统所需而产生的一种B-tree的变形树。

一棵m阶的B+树和m阶的B树的异同点在于：

1. 有n棵子树的结点中含有n-1 个关键字；(与B 树n棵子树有n-1个关键字保持一致，参照：http://en.wikipedia.org/wiki/B%2B_tree#Overview，而下面**B+树的图可能有问题**，请读者注意)
2. 所有的叶子结点中包含了全部关键字的信息，及指向含有这些关键字记录的指针，且叶子结点本身依关键字的大小自小而大的顺序链接。(而B树的叶子节点并没有包括全部需要查找的信息)
3. **所有的非终端结点可以看成是索引部分**，结点中仅含有其子树根结点中最大（或最小）关键字。(而B 树的非终节点也包含需要查找的有效信息)



a. 为什么说B+-tree比B 树更适合实际应用中操作系统的文件索引和数据库索引？

1. B+-tree的磁盘读写代价更低 B+-tree的内部结点并没有指向关键字具体信息的指针。因此其内部结点相对B 树更小。如果把所有同一内部结点的关键字存放在同一盘块中，那么盘块所能容纳的关键字数量也越多。一次性读入内存中的需要查找的关键字也就越多。相对来说IO读写次数也就降低了。

   举个例子，假设磁盘中的一个盘块容纳16bytes，而一个关键字2bytes，一个关键字具体信息指针2bytes。一棵9阶B-tree(一个结点最多8个关键字)的内部结点需要2个盘快。而B+ 树内部结点只需要1个盘快。当需要把内部结点读入内存中的时候，B 树就比B+ 树多一次盘块查找时间(在磁盘中就是盘片旋转的时间)。

2. B+-tree的查询效率更加稳定
   由于非终结点并不是最终指向文件内容的结点，而只是叶子结点中关键字的索引。所以任何关键字的查找必须走一条从根结点到叶子结点的路。所有关键字查询的路径长度相同，导致每一个数据的查询效率相当。

   读者点评
   本文评论下第149楼，fanyy1991针对上文所说的两点，道：个人觉得这两个原因都不是主要原因。数据库索引采用B+树的主要原因是 B树在

提高了磁盘IO性能的同时并没有解决元素遍历的效率低下的问题。正是为了解决这个问题，B+树应运而生。B+树只要遍历叶子节点就可以实现整棵树的遍历。而且在数据库中基于范围的查询是非常频繁的，而B树不支持这样的操作（或者说效率太低）。

b. B+-tree的应用：VSAM(虚拟存储存取法)文件(来源论文 *the ubiquitous Btree* 作者：D COMER - 1979)

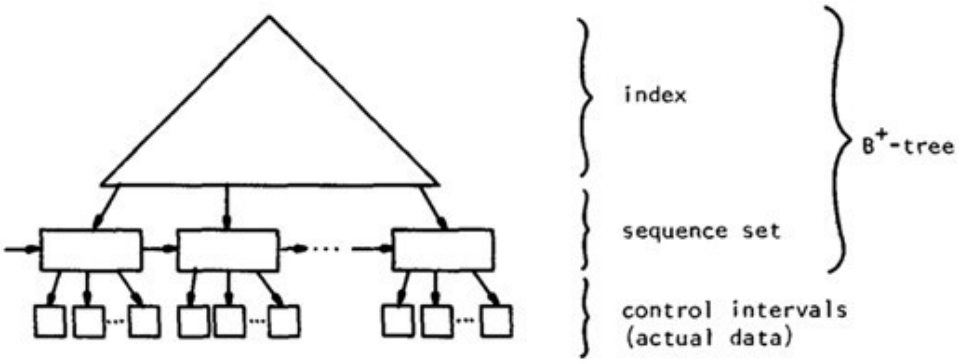
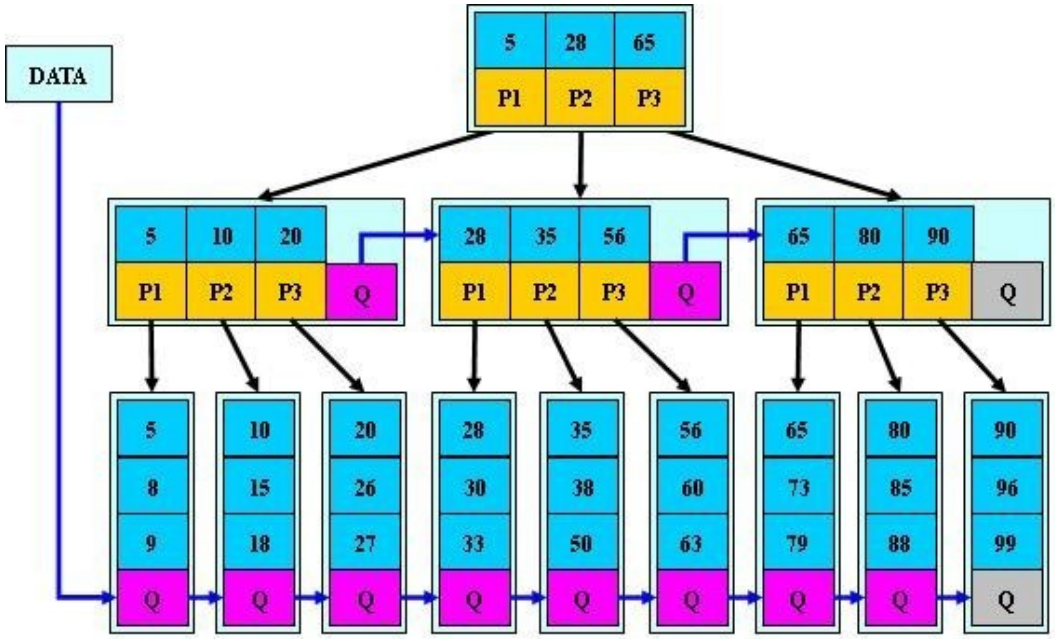


FIGURE 18. A VSAM file with actual data (associated information) stored in the leaves.

5.B*-tree

B*-tree是B+-tree的变体，在B+树的基础上(所有的叶子结点中包含了全部关键字的信息，及指向含有这些关键字记录的指针)，B*树中非根和非叶子结点再增加指向兄弟的指针；B*树定义了非叶子结点关键字个数至少为(2/3)*M，即块的最低使用率为2/3（代替B+树的1/2）。给出了一个简单实例，如下图所示：



B+树的分裂：当一个结点满时，分配一个新的结点，并将原结点中1/2的数据复制到新结点，最后在父结点中增加新结点的指针；B+树的分裂只影响原结点和父结点，而不会影响兄弟结点，所以它不需要指向兄弟的指针。

B*树的分裂：当一个结点满时，如果它的下一个兄弟结点未满，那么将一部分数据移到兄弟结点中，再在原结点插入关键字，最后修改父结点中兄弟结点的关键字（因为兄弟结点的关键字范围改变了）；如果兄弟也满了，则在原结点与兄弟结点之间增加新结点，并各复制1/3的数据到新结点，最后在父结点增加新结点的指针。

所以，B*树分配新结点的概率比B+树要低，空间使用率更高；

7.总结

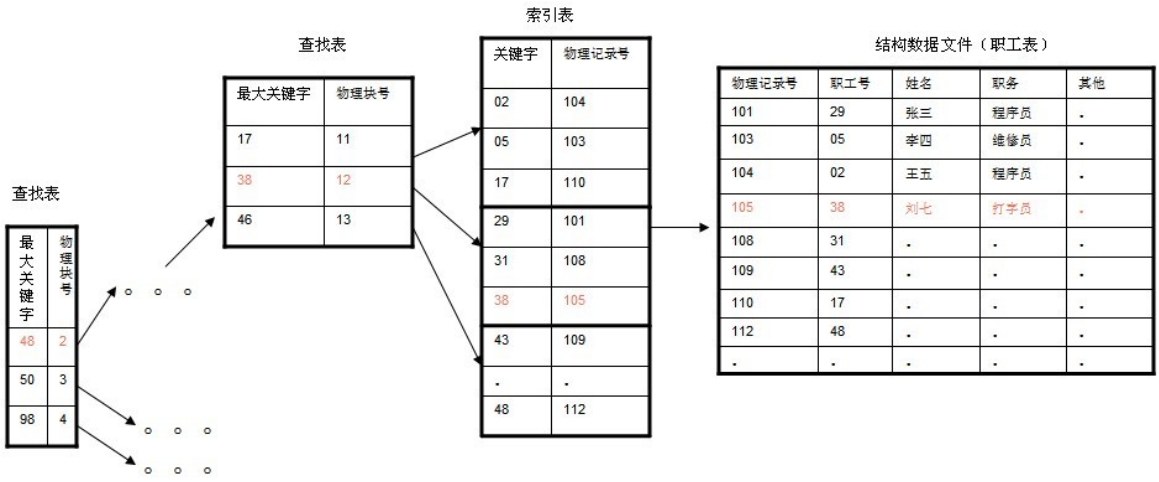
通过以上介绍，大致将B树，B+树，B*树总结如下：

- B树：有序数组+平衡多叉树；
- B+树：有序数组链表+平衡多叉树；
- B*树：一棵丰满的B+树。

在大规模数据存储的文件系统中，B~tree系列数据结构，起着很重要的作用，对于存储不同的数据，节点相关的信息也是有所不同，这里根据自己的理解，画的一个查找以职工号为关键字，职工号为38的记录的简单示意图。(这里假

设每个物理块容纳3个索引，磁盘的I/O操作的基本单位是块（block),磁盘访问很费时，采用B+树有效的减少了访问磁盘的次数。）

对于像MySQL，DB2，Oracle等数据库中的索引结构得有较深入的了解才行，建议去找一些B 树相关的开源代码研究。



走进搜索引擎的作者梁斌老师针对B树、B+树给出了他的意见（为了真实性，特引用其原话，未作任何改动）：“B+树还有一个最大的好处，方便扫库，B树必须用中序遍历的方法按序扫库，而B+树直接从叶子结点挨个扫一遍就完了，B+树支持range-query非常方便，而B树不支持。这是数据库选用B+树的最主要原因。

比如要查 5-10之间的，B+树一把到5这个标记，再一把到10，然后串起来就行了，B树就非常麻烦。B树的好处，就是成功查询特别有利，因为树的高度总体要比B+树矮。不成功的情况下，B树也比B+树稍稍占一点点便宜。

B树比如你的例子中查，17的话，一把就得到结果了，有很多基于频率的搜索是选用B树，越频繁query的结点越往根上走，前提是需要对query做统计，而且要对key做一些变化。

另外B树也好B+树也好，根或者上面几层因为被反复query，所以这几块基本都在内存中，不会出现读磁盘IO，一般已启动的时候，就会主动换入内存。”非常感谢。

> “mysql 底层存储是用B+树实现的，知道为什么么。内存中B+树是没有优势的，但是一到磁盘，B+树的威力就出来了”。——Bucket Li

第二节、R树：处理空间存储问题

相信经过上面第一节的介绍，你已经对B树或者B+树有所了解。这种树可以非常好的处理一维空间存储的问题。B树是一棵平衡树，它是把一维直线分为若干段线段，当我们查找满足某个要求的点的时候，只要去查找它所属的线段即可。依我看来，这种思想其实就是先找一个大的空间，再逐步缩小所要查找的空间，最终在一个自己设定的最小不可分空间内找出满足要求的解。一个典型的B树查找如下：

要查找某一满足条件的点，先去找到满足条件的线段，然后遍历所在线段上的点，即可找到答案。

B树是一种相对来说比较复杂的数据结构，尤其是在它的删除与插入操作过程中，因为它涉及到了叶子结点的分解与合并。由于本文第一节已经详细介绍了B树和B+树，下面直接开始介绍我们的第二个主角：R树。

简介

984年，加州大学伯克利分校的Guttman发表了一篇题为“R-trees: a dynamic index structure for spatial searching”的论文，向世人介绍了R树这种处理高维空间存储问题的数据结构。本文便是基于这篇论文写作完成的，因此如果大家对R树非常有兴趣，我想最好还是参考一下原著：）。为表示对这位牛人的尊重，给个引用先：Guttman, A.; “R-trees: a dynamic index structure for spatial searching,” ACM, 1984, 14

R树在数据库等领域做出的功绩是非常显著的。它很好的解决了在高维空间搜索等问题。举个R树在现实领域中能够解决的例子：查找20英里以内所有的餐厅。如果没有R树你会怎么解决？一般情况下我们会把餐厅的坐标(x,y)分为两个字段存放在数据库中，一个字段记录经度，另一个字段记录纬度。这样的话我们就需要遍历所有的餐厅获取其位置信息，然后计算是否满足要求。如果一个地区有100家餐厅的话，我们就要进行100次位置计算操作了，如果应用到谷歌地图这种超大数据库中，这种方法便必定不可行了。

R树就很好的解决了这种高维空间搜索问题。它把B树的思想很好的扩展到了多维空间，采用了B树分割空间的思想，并在添加、删除操作时采用合并、分解结点的方法，保证树的平衡性。因此，R树就是一棵用来存储高维数据的平衡树。

OK，接下来，本文将详细介绍R树的数据结构以及R树的操作。至于R树的扩展与R树的性能问题，可以查阅相关论文。

R树的数据结构

如上所述，R树是B树在高维空间的扩展，是一棵平衡树。每个R树的叶子结点包含了多个指向不同数据的指针，这些数据可以是存放在硬盘中的，也可以是存在内存中。根据R树的这种数据结构，当我们需要进行一个高维空间查询时，我们只需要遍历少数几个叶子结点所包含的指针，查看这些指针指向的数据是否满足要求即可。这种方式使我们不必遍历所有数据即可获得答案，效率显著提高。下图1是R树的一个简单实例：

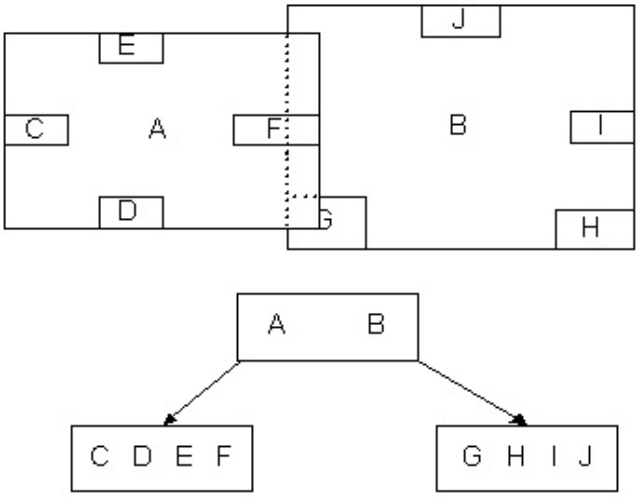
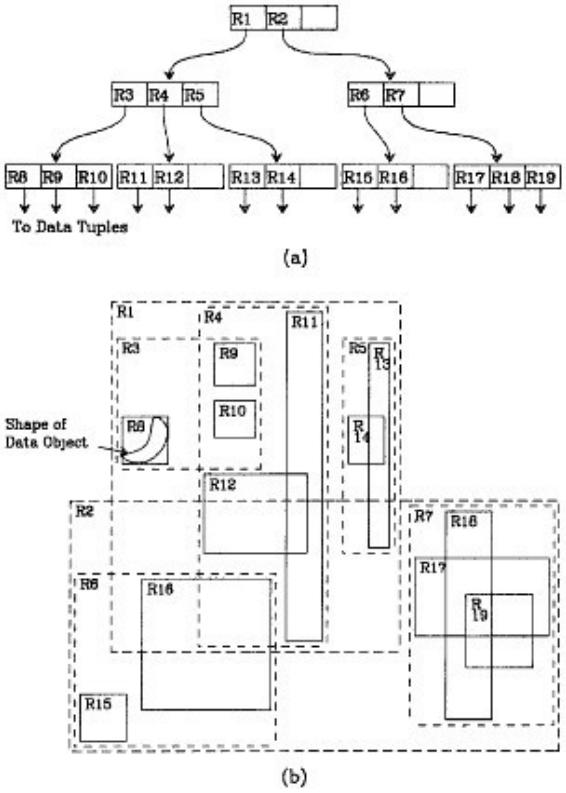


图1：一个R树实例

我们在上面说过，R树运用了空间分割的理念，这种理念是如何实现的呢？R树采用了一种称为MBR(Minimal Bounding Rectangle)的方法，在此我把它译作“最小边界矩形”。从叶子结点开始用矩形（rectangle）将空间框起来，结点越往上，框住的空间就越大，以此对空间进行分割。有点不懂？没关系，继续往下看。在这里我还想提一下，R树中的R应该代表的是Rectangle（此处参考wikipedia上关于R树的介绍），而不是大多数国内教材中所说的Region（很

多书把R树称为区域树，这是有误的）。我们就拿二维空间来举例。下图是Guttman论文中的一幅图：



我来详细解释一下这张图。先来看图(b)

1. 首先我们假设所有数据都是二维空间下的点，图中仅仅标志了R8区域中的数据，也就是那个shape of data object。别把那一块不规则图形看成一个数据，我们把它看作是多个数据围成的一个区域。为了实现R树结构，我们用一个最小边界矩形恰好框住这个不规则区域，这样，我们就构造出了一个区域：R8。R8的特点很明显，就是正正好好框住所有在此区域中的数据。其他实线包围住的区域，如R9，R10，R12等都是同样的道理。这样一来，我们一共得到了12个最最基本的最小矩形。这些矩形都将被存储在子结点中。
2. 下一步操作就是进行高一层次的处理。我们发现R8，R9，R10三个矩形距离最为靠近，因此就可以用一个更大的矩形R3恰好框住这3个矩形。
3. 同样道理，R15，R16被R6恰好框住，R11，R12被R4恰好框住，等等。所有最基本的最小边界矩形被框入更大的矩形中之后，再次迭代，用更大的框去框住这些矩形。

我想大家都应该理解这个数据结构的特征了。**用地图的例子来解释，就是所有的数据都是餐厅所对应的地点，先把相邻的餐厅划分到同一块区域，划分好所有餐厅之后，再把邻近的区域划分到更大的区域，划分完毕后再次进行更高层次的划分，直到划分到只剩下两个最大的区域为止。要查找的时候就方便了。**

下面就可以把这些大大小小的矩形存入我们的R树中去了。根结点存放的是两个最大的矩形，这两个最大的矩形框住了所有的剩余的矩形，当然也就框住了所有的数据。下一层的结点存放了次大的矩形，这些矩形缩小了范围。每个叶子结点都是存放的最小的矩形，这些矩形中可能包含有n个数据。

在这里，读者先不要去纠结于如何划分数据到最小区域矩形，也不要纠结怎样用更大的矩形框住小矩形，这些都是下一节我们要讨论的。

讲完了基本的数据结构，我们来讲个实例，如何查询特定的数据。又以餐厅为例，假设我要查询广州市天河区天河城附近一公里的所有餐厅地址怎么办？

1. 打开地图（也就是整个R树），先选择国内还是国外（也就是根结点）。
2. 然后选择华南地区（对应第一层结点），选择广州市（对应第二层结点），
3. 再选择天河区（对应第三层结点），
4. 最后选择天河城所在的那个区域（对应叶子结点，存放有最小矩形），遍历所有在此区域内的结点，看是否满足我们的要求即可。

怎么样，其实R树的查找规则跟查地图很像吧？对应下图：

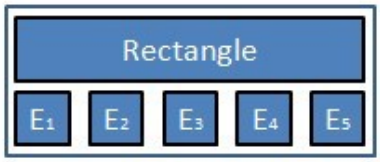
一棵R树满足如下的性质：

1. 除非它是根结点之外，所有叶子结点包含有m至M个记录索引（条目）。作为根结点的叶子结点所具有的记录个数可以少于m。通常，m=M/2。
2. 对于所有在叶子中存储的记录（条目），I是最小的可以在空间中完全覆盖这些记录所代表的点的矩形（注意：此处所说的“矩形”是可以扩展到高维空间的）。
3. 每一个非叶子结点拥有m至M个孩子结点，除非它是根结点。
4. 对于在非叶子结点上的每一个条目，i是最小的可以在空间上完全覆盖这些条目所代表的店的矩形（同性质2）。
5. 所有叶子结点都位于同一层，因此R树为平衡树。

叶子结点的结构

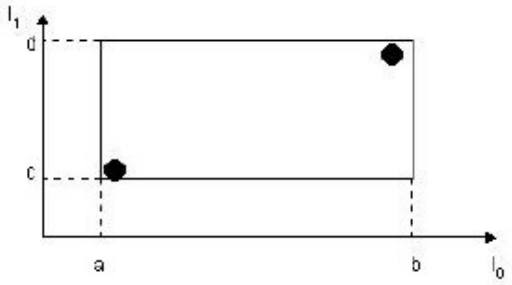
先来探究一下叶子结点的结构。叶子结点所保存的数据形式为：(I, tuple-identifier)。

其中，tuple-identifier表示的是一个存放于数据库中的tuple，也就是一条记录，它是n维的。I是一个n维空间的矩形，并可以恰好框住这个叶子结点中所有记录代表的n维空间中的点。I=(I0,I1,…,In-1)。其结构如下图所示：



R树中结点的存储结构。E代表Entry，即指向孩子结点的条目

下图描述的就是在二维空间中的叶子结点所要存储的信息。



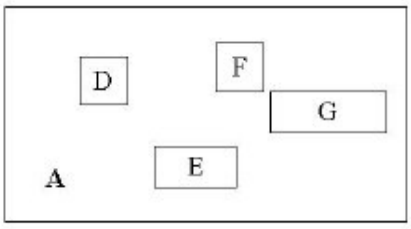
在这张图中，I所代表的就是图中的矩形，其范围是a<=I0<=b，c<=I1<=d。有两个tuple-identifier，在图中即表示为那两个点。这种形式完全可以推广到高维空间。大家简单想想三维空间中的样子就可以了。这样，叶子结点的结构就介绍完了。

非叶子结点

非叶子结点的结构其实与叶子结点非常类似。想象一下B树就知道了，B树的叶子结点存放的是真实存在的数据，而非叶子结点存放的是这些数据的“边界”，或者说也算是一种索引（有疑问的读者可以回顾一下上述第一节中讲解B树的部分）。

同样道理，R树的非叶子结点存放的数据结构为：(I, child-pointer)。

其中，child-pointer是指向孩子结点的指针，I是覆盖所有孩子结点对应矩形的矩形。这边有点拗口，但我想不是很难懂？给张图：



D,E,F,G为孩子结点所对应的矩形。A为能够覆盖这些矩形的更大的矩形。这个A就是这个非叶子结点所对应的矩形。这时候你应该悟到了吧？无论是叶子结点还是非叶子结点，它们都对应着一个矩形。树形结构上层的结点所对应的矩形能够完全覆盖它的孩子结点所对应的矩形。根结点也唯一对应一个矩形，而这个矩形是可以覆盖所有我们拥有的数据信息在空间中代表的点的。

我个人感觉这张图画的不那么精确，应该是矩形A要恰好覆盖D,E,F,G，而不应该再留出这么多没用的空间了。但为尊重原图的绘制者，特不作修改。

R树的操作

这一部分也许是编程者最关注的问题了。这么高效的数据结构该如何去实现呢？这便是这一节需要阐述的问题。

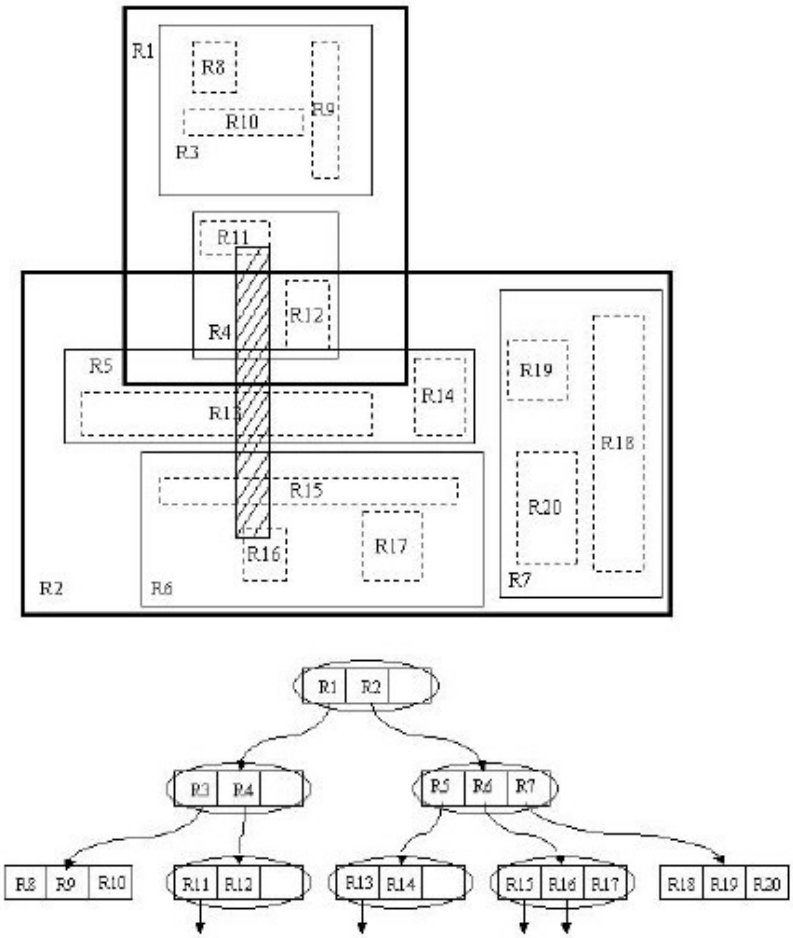
搜索

R树的搜索操作很简单，跟B树上的搜索十分相似。它返回的结果是所有符合查找信息的记录条目。而输入是什么？就我个人的理解，输入不仅仅是一个范围了，它更可以看成是一个空间中的矩形。也就是说，我们输入的是一个搜索矩形。

先给出伪代码：

Function：Search
描述：假设T为一棵R树的根结点，查找所有搜索矩形S覆盖的记录条目。
S1:[查找子树] 如果T是非叶子结点，如果T所对应的矩形与S有重合，那么检查所有T中存储的条目，对于所有这些条目，使用Search操作作用在每一个条目所指向的子树的根结点上（即T结点的孩子结点）。
S2:[查找叶子结点] 如果T是叶子结点，如果T所对应的矩形与S有重合，那么直接检查S所指向的所有记录条目。返回符合条件的记录。

我们通过下图来理解这个Search操作。



阴影部分所对应的矩形为搜索矩形。它与根结点对应的最大的矩形（未画出）有重叠。这样将Search操作作用在其两个子树上。两个子树对应的矩形分别为R1与R2。搜索R1，发现与R1中的R4矩形有重叠，继续搜索R4。最终在R4所包含的R11与R12两个矩形中查找是否有符合条件的记录。搜索R2的过程同样如此。很显然，该算法进行的是一个迭代操作。

插入

R树的插入操作也同B树的插入操作类似。当新的数据记录需要被添加入叶子结点时，若叶子结点溢出，那么我们需要对叶子结点进行分裂操作。显然，叶子结点的插入操作会比搜索操作要复杂。插入操作需要一些辅助方法才能够完成。

来看一下伪代码：

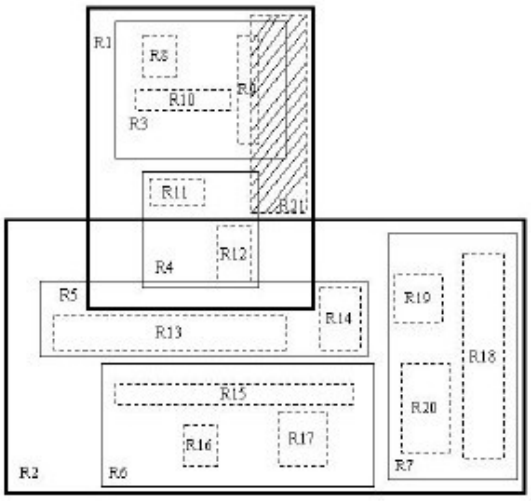
Function：Insert
描述：将新的记录条目E插入给定的R树中。
I1：[为新记录找到合适插入的叶子结点] 开始ChooseLeaf方法选择叶子结点L以放置记录E。
I2：[添加新记录至叶子结点] 如果L有足够的空间来放置新的记录条目，则向L中添加E。如果没有足够的空间，则进行SplitNode方法以获得两个结点L与LL，这两个结点包含了所有原来叶子结点L中的条目与新条目E。
I3：[将变换向上传递] 开始对结点L进行AdjustTree操作，如果进行了分裂操作，那么同时需要对LL进行AdjustTree操作。
I4：[对树进行增高操作] 如果结点分裂，且该分裂向上传播导致了根结点的分裂，那么需要创建一个新的根结点，并且让它的两个孩子结点分别为原来那个根结点分裂后的两个结点。

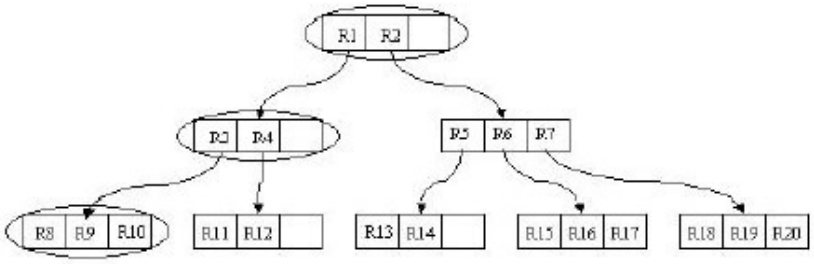
Function：ChooseLeaf

描述：选择叶子结点以放置新条目E。
CL1：[Initialize] 设置N为根结点。
CL2：[叶子结点的检查] 如果N为叶子结点，则直接返回N。
CL3：[选择子树] 如果N不是叶子结点，则遍历N中的结点，找出添加E.I时扩张最小的结点，并把该结点定义为F。如果有多个这样的结点，那么选择面积最小的结点。
CL4：[下降至叶子结点] 将N设为F，从CL2开始重复操作。

Function：AdjustTree
描述：叶子结点的改变向上传递至根结点以改变各个矩阵。在传递变换的过程中可能会产生结点的分裂。AT1：[初始化] 将N设为L。
AT2：[检验是否完成] 如果N为根结点，则停止操作。
AT3：[调整父结点条目的最小边界矩形] 设P为N的父节点，EN为指向在父节点P中指向N的条目。调整EN.I以保证所有在N中的矩形都被恰好包围。
AT4：[向上传递结点分裂] 如果N有一个刚刚被分裂产生的结点NN，则创建一个指向NN的条目ENN。如果P有空间来存放ENN，则将ENN添加到P中。如果没有，则对P进行SplitNode操作以得到P和PP。
AT5：[升高至下一级] 如果N等于L且发生了分裂，则把NN置为PP。从AT2开始重复操作。

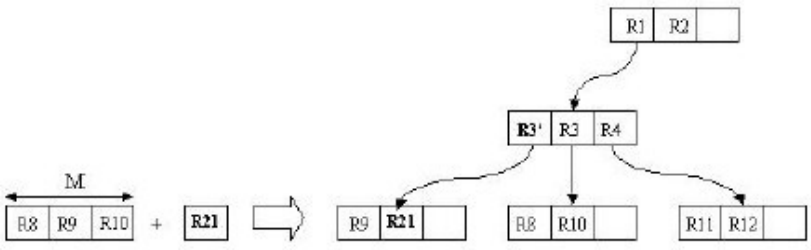
同样，我们用图来更加直观的理解这个插入操作。





我们来通过图分析一下插入操作。现在我们需要插入R21这个矩形。开始时我们进行ChooseLeaf操作。在根结点中有两个条目，分别为R1，R2。其实R1已经完全覆盖了R21，而若向R2中添加R21，则会使R2.I增大很多。显然我们选择R1插入。然后进行下一级的操作。相比于R4，向R3中添加R21会更合适，因为R3覆盖R21所需增大的面积相对较小。这样就在B8，B9，B10所在的叶子结点中插入R21。由于叶子结点没有足够空间，则要进行分裂操作。

插入操作如下图所示：



这个插入操作其实类似于第一节中B树的插入操作，这里不再具体介绍，不过想必看过上面的伪代码大家应该也清楚了。

删除

R树的删除操作与B树的删除操作会有所不同，不过同B树一样，会涉及到压缩等操作。相信读者看完以下的伪代码之后会有所体会。R树的删除同样是比较复杂的，需要用到一些辅助函数来完成整个操作。

伪代码如下：

Function：Delete
描述：将一条记录E从指定的R树中删除。
D1：[找到含有记录的叶子结点] 使用FindLeaf方法找到包含有记录E的叶子结点L。如果搜索失败，则直接终止。
D2：[删除记录] 将E从L中删除。
D3：[传递记录] 对L使用CondenseTree操作
D4：[缩减树] 当经过以上调整后，如果根结点只包含有一个孩子结点，则将这个唯一的孩子结点设为根结点。

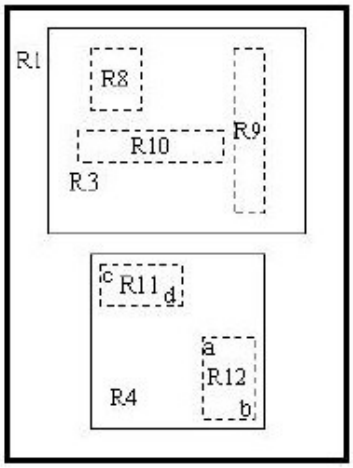
Function：FindLeaf
描述：根结点为T，期望找到包含有记录E的叶子结点。
FL1：[搜索子树] 如果T不是叶子结点，则检查每一条T中的条目F，找出与E所对应的矩形相重合的F（不必完全覆盖）。对于所有满足条件的F，对其指向的孩子结点进行FindLeaf操作，直到寻找到E或者所有条目均以被检查过。
FL2：[搜索叶子结点以找到记录] 如果T是叶子结点，那么检查每一个条目是否有E存在，如果有则返回T。

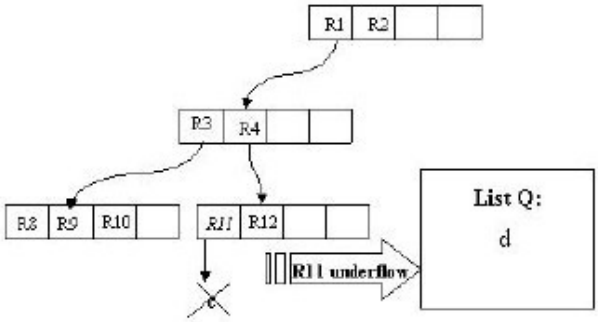
Function：CondenseTree
描述：L为包含有被删除条目的叶子结点。如果L的条目数过少（小于要求的最小值m），则必须将该叶子结点L从树中删除。经过这一删除操作，L中的剩余条目必须重新插入树中。此操作将一直重复直至到达根结点。同样，调整在此修改树的过程所经过的路径上的所有结点对应的矩形大小。
CT1：[初始化] 令N为L。初始化一个用于存储被删除结点包含的条目的链表Q。
CT2：[找到父条目] 如果N为根结点，那么直接跳转至CT6。否则令P为N 的父结点，令EN为P结点中存储的指向N的条目。
CT3：[删除下溢结点] 如果N含有条目数少于m，则从P中删除EN，并把结点N中的条目添加入链表Q中。CT4：[调整覆盖矩形] 如果N没有被删除，则调整EN.I使得其对应矩形能够恰好覆盖N中的所有条目所对应的矩形。
CT5：[向上一层结点进行操作] 令N等于P，从CT2开始重复操作。

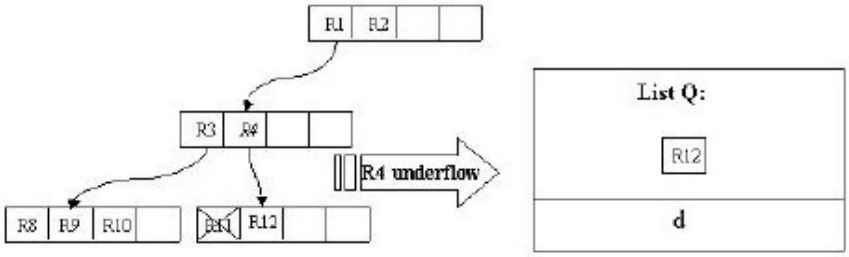
CT6:[重新插入孤立的条目] 所有在Q中的结点中的条目需要被重新插入。原来属于叶子结点的条目可以使用Insert操作进行重新插入，而那些属于非叶子结点的条目必须插入删除之前所在层的结点，以确保它们所指向的子树还处于相同的层。

R树删除记录过程中的CondenseTree操作是不同于B树的。我们知道，B树删除过程中，如果出现结点的记录数少于半满（即下溢）的情况，则直接把这些记录与其他叶子的记录“融合”，也就是说两个相邻结点合并。然而R树却是直接重新插入。

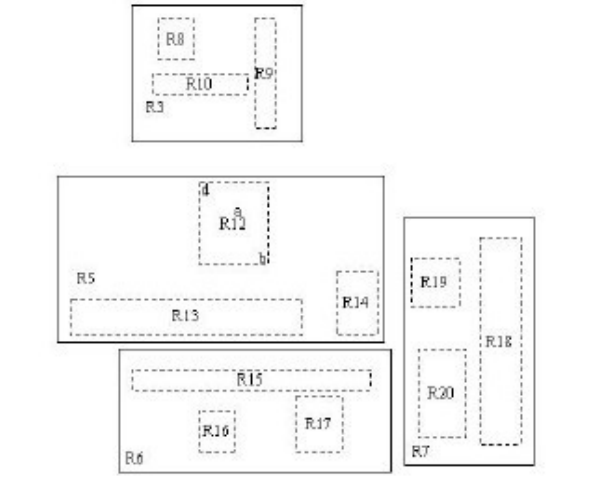
同样，我们用图直观的说明这个操作。

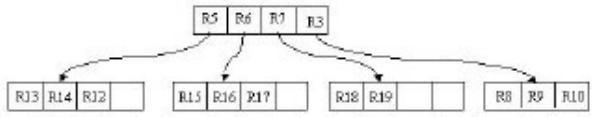






假设结点最大条目数为4，最小条目数为2。在这张图中，我们的目标是删除记录c。首先使用FindLeaf操作找到c所处在的叶子结点的位置——R11。当c从R11删除时，R11就只有一条记录了，少于最小条目数2，出现下溢，此时要调用CondenseTree操作。这样，c被删除，R11剩余的条目——指向记录d的指针——被插入链表Q。然后向更高一层的结点进行此操作。这样R12会被插入链表中。原理是一样的，在这里就不再赘述。





有一点需要解释的是，我们发现这个删除操作向上传递之后，根结点的条目R1也被插入了Q中，这样根结点只剩下了R2。别着急，重新插入操作会有效的解决这个问题。我们插入R3，R12，d至它原来所处的层。这样，我们发现根结点只有一个条目了，此时根据Inert中的操作，我们把这个根结点删除，它的孩子结点，即R5，R6，R7，R3所在的结点被置为根结点。至此，删除操作结束。

结语

R树是一种能够有效进行高维空间搜索的数据结构，它已经被广泛应用在各种数据库及其相关的应用中。但R树的处理也具有局限性，它的最佳应用范围是处理2至6维的数据，更高维的存储会变得非常复杂，这样就不适用了。近年来，R树也出现了很多变体，R*树就是其中的一种。这些变体提升了R树的性能，感兴趣的读者可以参考相关文献。文章有任何错误，还望各位读者不吝赐教。本文完。

参考文献以及推荐阅读：

1. Organization and Maintenance of Large Ordered Indices
2. the ubiquitous B tree
3. http://en.wikipedia.org/wiki/Btree （给出了国外一些开源地址）
4. http://en.wikipedia.org/wiki/Btree#Technical_description
5. http://cis.stvincent.edu/html/tutorials/swd/btree/btree.html （include C++ source code）
6. http://slady.net/java/bt/view.php（如果了解了B-tree结构，该地址可以在线对该结构进行查找（search），插入(insert)，删除(delete)操作。）
7. Guttman, A.; “R-trees: a dynamic index structure for spatial searching,” ACM, 1984, 14
8. http://www.cnblogs.com/CareySon/archive/2012/04/06/2435349.html
9. http://baike.baidu.com/view/298408.htm
10. http://www.cnblogs.com/leoo2sk/archive/2011/07/10/mysql-index.html （介绍了mysql中myisam和innodb这两种引擎的内部索引机制，以及对不同字段的索引时，检索效率上的对比，主要也是基于其内部机制的理解）
11. http://www.oschina.net/news/31988/mysql-indexing-best-practices（MySQL索引最佳实践）
12. http://idlebox.net/2007/stx-btree/（此页面包含B树生成构造的一些演示demo）

版权所有，侵权必究。本blog内任何内容严禁用于任何商业用途，违者永久追究法律责任。

本章导读

一般而言，标题含有“秒杀”，“99%”，“史上最全/最强”等词汇的往往都脱不了哗众取宠之嫌，但进一步来讲，如果读者读罢此文，却无任何收获，那么，我也甘愿背负这样的罪名。

然毕竟受文章和理论之限，本文将摒弃绝大部分的细节，只谈方法/模式论，且注重用最通俗最直白的语言阐述相关问题。最后，有一点必须强调的是，全文行文是基于面试题的分析基础之上的，具体实践过程中，还是得具体情况具体分析，且各个场景下需要考虑的细节也远比本文所描述的任何一种解决方法复杂得多。

OK，若有任何问题，欢迎随时不吝赐教。谢谢。

何谓海量数据处理？

所谓海量数据处理，无非就是基于海量数据上的存储、处理、操作。何谓海量，就是数据量太大，所以导致要么是无法在较短时间内迅速解决，要么是数据太大，导致无法一次性装入内存。

事实上，针对时间，我们可以采用巧妙的算法搭配合适的数据结构，如Bloom filter/Hash/bit-map/堆/数据库或倒排索引/trie树，针对空间，大而化小，分而治之（hash映射），也就是说规模太大的话，就把规模大化为规模小的，最终各个击破。

至于所谓的单机及集群问题，通俗点来讲，单机就是处理装载数据的机器有限(只要考虑cpu，内存，硬盘的数据交互)，而集群，机器有多辆，适合分布式处理，并行计算(更多考虑节点和节点间的数据交互)。

一般说来，处理海量数据问题，有以下六种典型方法：

1. 分而治之/hash映射 + hash统计 + 堆/快速/归并排序；
2. 双层桶划分
3. Bloom filter/Bitmap；
4. Trie树/数据库/倒排索引；
5. 外排序；
6. 分布式处理之Hadoop/Mapreduce。

本章会先简要介绍下set/map/multiset/multimap，及hash_set/hash_map/hash_multiset/hash_multimap之区别**(万丈高楼平地起，基础最重要)**，而本文第二部分，则针对上述那6种方法模式结合对应的海量数据处理面试题分别具体阐述。

关联式容器

一般来说，STL容器分两种，

- 序列式容器(vector/list/deque/stack/queue/heap)，
- 关联式容器。关联式容器又分为set(集合)和map(映射表)两大类，以及这两大类的衍生体multiset(多键集合)和multimap(多键映射表)，这些容器均以RB-tree完成。此外，还有第3类关联式容器，如hashtable(散列表)，以及以hashtable为底层机制完成的hash_set(散列集合)/hash_map(散列映射表)/hash_multiset(散列多键集合)/hash_multimap(散列多键映射表)。也就是说，set/map/multiset/multimap都内含一个RB-tree，而hash_set/hash_map/hash_multiset/hash_multimap都内含一个hashtable。

所谓关联式容器，类似关联式数据库，每笔数据或每个元素都有一个键值(key)和一个实值(value)，即所谓的Key-Value(键-值对)。当元素被插入到关联式容器中时，容器内部结构(RB-tree/hashtable)便依照其键值大小，以某种特定规则将这个元素放置于适当位置。

包括在非关联式数据库中，比如，在MongoDB内，文档(document)是最基本的数据组织形式，每个文档也是以Key-Value（键-值对）的方式组织起来。一个文档可以有多个Key-Value组合，每个Value可以是不同的类型，比如String、Integer、List等等。

```
{ "name" : "July",
"sex" : "male",
"age" : 23 }
```

set/map/multiset/multimap

set，同map一样，所有元素都会根据元素的键值自动被排序，因为set/map两者的所有各种操作，都只是转而调用RB-tree的操作行为，不过，值得注意的是，两者都不允许两个元素有相同的键值。

不同的是：set的元素不像map那样可以同时拥有实值(value)和键值(key)，set元素的键值就是实值，实值就是键值，而map的所有元素都是pair，同时拥有实值(value)和键值(key)，pair的第一个元素被视为键值，第二个元素被视为实值。

至于multiset/multimap，他们的特性及用法和set/map完全相同，唯一的差别就在于它们允许键值重复，即所有的插入操作基于RB-tree的insert_equal()而非insert_unique()。

hash_set/hash_map/hash_multiset/hash_multimap

hash_set/hash_map，两者的一切操作都是基于hashtable之上。不同的是，hash_set同set一样，同时拥有实值和键值，且实质就是键值，键值就是实值，而hash_map同map一样，每一个元素同时拥有一个实值(value)和一个键值(key)，所以其使用方式，和上面的map基本相同。但由于hash_set/hash_map都是基于hashtable之上，所以不具备自动排序功能。为什么?因为hashtable没有自动排序功能。

至于hash_multiset/hash_multimap的特性与上面的multiset/multimap完全相同，唯一的差别就是它们hash_multiset/hash_multimap的底层实现机制是hashtable（而multiset/multimap，上面说了，底层实现机制是RB-tree），所以它们的元素都不会被自动排序，不过也都允许键值重复。

所以，综上，说白了，什么样的结构决定其什么样的性质，因为set/map/multiset/multimap都是基于RB-tree之上，所以有自动排序功能，而hash_set/hash_map/hash_multiset/hash_multimap都是基于hashtable之上，所以不含有自动排序功能，至于加个前缀multi_无非就是允许键值重复而已。

hash表算法

第一部分:Top K 算法详解

问题描述

百度面试题:

搜索引擎会通过日志文件把用户每次检索使用的所有检索串都记录下来,每个查询串的长度为1-255字节。假设目前有一千万个记录(这些查询串的重复度比较高,虽然总数是1千万,但如果除去重复后,不超过3百万个。一个查询串的重复度越高,说明查询它的用户越多,也就是越热门。),请你统计最热门的10个查询串,要求使用的内存不能超过1G。####必备知识 什么是哈希表?

哈希表(Hash table,也叫散列表),是根据关键码值(Key value)而直接进行访问的数据结构。也就是说,它通过把关键码值映射到表中一个位置来访问记录,以加快查找的速度。这个映射函数叫做散列函数,存放记录的数组叫做散列表。

哈希表hashtable(key, value) 的做法其实很简单,就是把Key通过一个固定的算法函数既所谓的哈希函数转换成一个整型数字,然后就将该数字对数组长度进行取余,取余结果就当作数组的下标,将value存储在以该数字为下标的数组空间里。

而当使用哈希表进行查询的时候,就是再次使用哈希函数将key转换为对应的数组下标,并定位到该空间获取value,如此一来,就可以充分利用到数组的定位性能进行数据定位(文章第二、三部分,会针对Hash表详细阐述)。

问题解析:

要统计最热门查询,首先就是要统计每个Query出现的次数,然后根据统计结果,找出Top 10。所以我们可以基于这个思路分两步来设计该算法。

即,此问题的解决分为以下**俩个步骤:**

第一步:Query统计

Query统计有以下俩个方法，可供选择：

1、直接排序法

首先我们最先想到的的算法就是排序了，首先对这个日志里面的所有Query都进行排序，然后再遍历排好序的Query，统计每个Query出现的次数了。

但是题目中有明确要求，那就是内存不能超过1G，一千万条记录，每条记录是255Byte，很显然要占据2.375G内存，这个条件就不满足要求了。

让我们回忆一下数据结构课程上的内容，当数据量比较大而且内存无法装下的时候，我们可以采用外排序的方法来进行排序，这里我们可以采用归并排序，因为归并排序有一个比较好的时间复杂度O(NlgN)。

排完序之后我们再对已经有序的Query文件进行遍历，统计每个Query出现的次数，再次写入文件中。

综合分析一下，排序的时间复杂度是O(NlgN)，而遍历的时间复杂度是O(N)，因此该算法的总体时间复杂度就是O(N+NlgN)=O（NlgN）。

2、Hash Table法

在第1个方法中，我们采用了排序的办法来统计每个Query出现的次数，时间复杂度是NlgN，那么能不能有更好的方法来存储，而时间复杂度更低呢？

题目中说明了，虽然有一千万个Query，但是由于重复度比较高，因此事实上只有300万的Query，每个Query255Byte，因此我们可以考虑把他们都放进内存中去，而现在只是需要一个合适的数据结构，在这里，Hash Table绝对是我们优先的选择，因为Hash Table的查询速度非常的快，几乎是O(1)的时间复杂度。

那么，我们的算法就有了：维护一个Key为Query字串，Value为该Query出现次数的HashTable，每次读取一个Query，如果该字串不在Table中，那么加入该字串，并且将Value值设为1；如果该字串在Table中，那么将该字串的计数加一即可。最终我们在O(N)的时间复杂度内完成了对该海量数据的处理。

本方法相比算法1：在时间复杂度上提高了一个数量级，为O（N），但不仅仅是时间复杂度上的优化，该方法只需要IO数据文件一次，而算法1的IO次数较多的，因此该算法2比算法1在工程上有更好的可操作性。

第二步:找出Top 10

算法一:普通排序

我想对于排序算法大家都已经不陌生了,这里不在赘述,我们要注意的是排序算法的时间复杂度是NlgN,在本题目中,三百万条记录,用1G内存是可以存下的。

算法二:部分排序

题目要求是求出Top 10,因此我们没有必要对所有的Query都进行排序,我们只需要维护一个10个大小的数组,初始化放入10个Query,按照每个Query的统计次数由大到小排序,然后遍历这300万条记录,每读一条记录就和数组最后一个Query对比,如果小于这个Query,那么继续遍历,否则,将数组中最后一条数据淘汰,加入当前的Query。最后当所有的数据都遍历完毕之后,那么这个数组中的10个Query便是我们要找的Top10了。

不难分析出,这样,算法的最坏时间复杂度是N*K,其中K是指top多少。

算法三:堆

在算法二中,我们已经将时间复杂度由NlogN优化到NK,不得不说这是一个比较大的改进了,可是有没有更好的办法呢?

分析一下,在算法二中,每次比较完成之后,需要的操作复杂度都是K,因为要把元素插入到一个线性表之中,而且采用的是顺序比较。这里我们注意一下,该数组是有序的,一次我们每次查找的时候可以采用二分的方法查找,这样操作的复杂度就降到了logK,可是,随之而来的问题就是数据移动,因为移动数据次数增多了。不过,这个算法还是比算法二有了改进。

基于以上的分析,我们想想,有没有一种既能快速查找,又能快速移动元素的数据结构呢?回答是肯定的,那就是堆。

借助堆结构,我们可以在log量级的时间内查找和调整/移动。因此到这里,我们的算法可以改进为这样,维护一个K(该题目中是10)大小的小根堆,然后遍历300万的Query,**分别和根元素进行对比。**

具体过程是,堆顶存放的是整个堆中最小的数,现在遍历N个数,把最先遍历到的k个数存放到最小堆中,并假设它们就是我们要找的最大的k个数,X1>X2…Xmin(堆顶),而后遍历后续的N-K个数,一一与堆顶元素进行比较,

如果遍历到的Xi大于堆顶元素Xmin，则把Xi放入堆中，而后更新整个堆，更新的时间复杂度为logK，如果Xi> 28（右移，除以2^28。记法：左移变大，是乘。右移变小，是除。）

如果数值分配比较均匀的话这种方法能得到不错的结果，但我上面画的那个图的各个元素的值算出来的index都是0——非常失败。也许你还有个问题，value如果很大，value * value不会溢出吗？答案是会的，但我们这个乘法不关心溢出，因为我们根本不是为了获取相乘结果，而是为了获取index。

3，斐波那契（Fibonacci）散列法

平方散列法的缺点是显而易见的，所以我们能不能找出一个理想的乘数，而不是拿value本身当作乘数呢？答案是肯定的。

1. 对于16位整数而言，这个乘数是40503
2. 对于32位整数而言，这个乘数是2654435769
3. 对于64位整数而言，这个乘数是11400714819323198485

这几个“理想乘数”是如何得出来的呢？这跟一个法则有关，叫黄金分割法则，而描述黄金分割法则的最经典表达式无疑就是著名的斐波那契数列，即如此形式的序列：0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610，987, 1597, 2584, 4181, 6765, 10946，…。另外，斐波那契数列的值和太阳系八大行星的轨道半径的比例出奇吻合。

对我们常见的32位整数而言，公式：

index = (value * 2654435769) >> 28

如果用这种斐波那契散列法的话，那上面的图就变成这样了：

注：用斐波那契散列法调整之后会比原来的取摸散列法好很多。

适用范围

快速查找，删除的基本数据结构，通常需要总数据量可以放入内存。

基本原理及要点

hash函数选择，针对字符串，整数，排列，具体相应的hash方法。

碰撞处理，一种是open hashing，也称为拉链法；另一种就是closed hashing，也称开地址法，opened addressing。

扩展

d-left hashing中的d是多个的意思，我们先简化这个问题，看一看2-left hashing。2-left hashing指的是将一个哈希表分成长度相等的两半，分别叫做T1和T2，给T1和T2分别配备一个哈希函数，h1和h2。在存储一个新的key时，同时用两个哈希函数进行计算，得出两个地址h1[key]和h2[key]。这时需要检查T1中的h1[key]位置和T2中的h2[key]位置，哪一个 位置已经存储的（有碰撞的）key比较多，然后将新key存储在负载少的位置。如果两边一样多，比如两个位置都为空或者都存储了一个key，就把新key 存储在左边的T1子表中，2-left也由此而来。在查找一个key时，必须进行两次hash，同时查找两个位置。

问题实例（海量数据处理）

我们知道hash 表在海量数据处理中有着广泛的应用，下面，请看另一道百度面试题：

题目：海量日志数据，提取出某日访问百度次数最多的那个IP。

方案：IP的数目还是有限的，最多2^32个，所以可以考虑使用hash将ip直接存入内存，然后进行统计。

第三部分、最快的Hash表算法

接下来，咱们来具体分析一下一个最快的Hasb表算法。

我们由一个简单的问题逐步入手：有一个庞大的字符串数组，然后给你一个单独的字符串，让你从这个数组中查找是否有这个字符串并找到它，你会怎么做？有一个方法最简单，老老实实从头查到尾，一个一个比较，直到找到为止，我想只要学过程序设计的人都能把这样一个程序作出来，但要是有程序员把这样的程序交给用户，我只能用无语来评价，或许它真的能工作，但…也只能如此了。

最合适的算法自然是使用HashTable(哈希表),先介绍介绍其中的基本知识,所谓Hash,一般是一个整数,通过某种算法,可以把一个字符串"压缩"成一个整数。当然,无论如何,一个32位整数是无法对应回一个字符串的,但在程序中,两个字符串计算出的Hash值相等的可能非常小,下面看看在MPQ中的Hash算法(参看自此文:http://sfsrealm.hopto.org/inside_mopaq/chapter2.htm):

函数一、以下的函数生成一个长度为0x500(合10进制数:1280)的cryptTable[0x500]

```
void prepareCryptTable()
{
    unsigned long seed = 0x00100001, index1 = 0, index2 = 0, i;

    for( index1 = 0; index1 < 0x100; index1++ )
    {
        for( index2 = index1, i = 0; i < 5; i++, index2 += 0x100 )
        {
            unsigned long temp1, temp2;

            seed = (seed * 125 + 3) % 0x2AAAAB;
            temp1 = (seed & 0xFFFF) << 0x10;

            seed = (seed * 125 + 3) % 0x2AAAAB;
            temp2 = (seed & 0xFFFF);

            cryptTable[index2] = ( temp1 | temp2 );
        }
    }
}
```

函数二、以下函数计算lpszFileName 字符串的hash值,其中dwHashType 为hash的类型,在下面的函数三、GetHashTablePos函数中调用此函数二,其可以取的值为0、1、2;该函数返回lpszFileName 字符串的hash值:

```
unsigned long HashString( char *lpszFileName, unsigned long dwHashType )
{
    unsigned char *key  = (unsigned char *)lpszFileName;
unsigned long seed1 = 0x7FED7FED;
unsigned long seed2 = 0xEEEEEEEE;
    int ch;

    while( *key != 0 )
    {
        ch = toupper(*key++);

        seed1 = cryptTable[(dwHashType << 8) + ch] ^ (seed1 + seed2);
        seed2 = ch + seed1 + seed2 + (seed2 << 5) + 3;
```

```
    }
    return seed1;
}
```

Blizzard的这个算法是非常高效的，被称为“One-Way Hash”(A one-way hash is a an algorithm that is constructed in such a way that deriving the original string (set of strings, actually) is virtually impossible) 。举个例子，字符串“unitneutralacritter.grp”通过这个算法得到的结果是0xA26067F3。

是不是把第一个算法改进一下，改成逐个比较字符串的Hash值就可以了呢，答案是，远远不够，要想得到最快的算法，就不能进行逐个的比较，通常是构造一个哈希表(Hash Table)来解决问题，哈希表是一个大数组，这个数组的容量根据程序的要求来定义，例如1024，每一个Hash值通过取模运算 (mod) 对应到数组中的一个位置，这样，只要比较这个字符串的哈希值对应的位置有没有被占用，就可以得到最后的结果了，想想这是什么速度？是的，是最快的O(1)，现在仔细看看这个算法吧：

```
typedef struct
{
    int nHashA;
    int nHashB;
    char bExists;
   ......
} SOMESTRUCTRUE;
```

一种可能的结构体定义？

函数三、下述函数为在Hash表中查找是否存在目标字符串，有则返回要查找字符串的Hash值，无则，return -1.

```
int GetHashTablePos( har *lpszString, SOMESTRUCTURE *lpTable )
//lpszString要在Hash表中查找的字符串，lpTable为存储字符串Hash值的Hash表。
{
    int nHash = HashString(lpszString);  //调用上述函数二，返回要查找字符串lpszString的Hash值。
    int nHashPos = nHash % nTableSize;

    if ( lpTable[nHashPos].bExists  &&  !strcmp( lpTable[nHashPos].pString, lpszString ) )
    {  //如果找到的Hash值在表中存在，且要查找的字符串与表中对应位置的字符串相同，
        return nHashPos;     //则返回上述调用函数二后，找到的Hash值
    }
    else
    {
        return -1;
    }
}
```

看到此，我想大家都在想一个很严重的问题：“如果两个字符串在哈希表中对应的位置相同怎么办？”,毕竟一个数组容量是有限的，这种可能性很大。解决该问题的方法很多，我首先想到的就是用“链表”,感谢大学里学的数据结构教会了这个百试百灵的法宝，我遇到的很多算法都可以转化成链表来解决，只要在哈希表的每个入口挂一个链表，保存所有对应的字符串就OK了。事情到此似乎有了完美的结局，如果是把问题独自交给我解决，此时我可能就要开始定义数据结构然后写代码了。

然而Blizzard的程序员使用的方法则是更精妙的方法。基本原理就是：他们在哈希表中不是用一个哈希值而是用三个哈希值来校验字符串。

MPQ使用文件名哈希表来跟踪内部的所有文件。但是这个表的格式与正常的哈希表有一些不同。首先，它没有使用哈希作为下标，把实际的文件名存储在表中用于验证，实际上它根本就没有存储文件名。而是使用了3种不同的哈希：一个用于哈希表的下标，两个用于验证。这两个验证哈希替代了实际文件名。

当然了，这样仍然会出现2个不同的文件名哈希到3个同样的哈希。但是这种情况发生的概率平均是：1:18889465931478580854784，这个概率对于任何人来说应该都是足够小的。现在再回到数据结构上，Blizzard使用的哈希表没有使用链表，而采用“顺延”的方式来解决问题，看看这个算法：

函数四、lpszString 为要在hash表中查找的字符串；lpTable 为存储字符串hash值的hash表；nTableSize 为hash表的长度：

```
int GetHashTablePos( char *lpszString, MPQHASHTABLE *lpTable, int nTableSize )
{
    const int  HASH_OFFSET = 0, HASH_A = 1, HASH_B = 2;

    int  nHash = HashString( lpszString, HASH_OFFSET );
    int  nHashA = HashString( lpszString, HASH_A );
    int  nHashB = HashString( lpszString, HASH_B );
    int  nHashStart = nHash % nTableSize;
    int  nHashPos = nHashStart;

    while ( lpTable[nHashPos].bExists )
   {
     /*如果仅仅是判断在该表中时候存在这个字符串,就比较这两个hash值就可以了，不用对
     *结构体中的字符串进行比较。这样会加快运行的速度？减少hash表占用的空间?这种
      *方法一般应用在什么场合?*/
        if (    lpTable[nHashPos].nHashA == nHashA
        &&  lpTable[nHashPos].nHashB == nHashB )
       {
            return nHashPos;
```

```
        }
        else
        {
             nHashPos = (nHashPos + 1) % nTableSize;
        }

         if (nHashPos == nHashStart)
              break;
    }
     return -1;
}
```

上述程序解释：

1. 计算出字符串的三个哈希值（一个用来确定位置，另外两个用来校验)
2. 察看哈希表中的这个位置
3. 哈希表中这个位置为空吗?如果为空,则肯定该字符串不存在,返回-1。
4. 如果存在,则检查其他两个哈希值是否也匹配,如果匹配,则表示找到了该字符串,返回其Hash值。
5. 移到下一个位置,如果已经移到了表的末尾,则反绕到表的开始位置起继续查询
6. 看看是不是又回到了原来的位置,如果是,则返回没找到
7. 回到3

ok,这就是本文中所说的最快的Hash表算法。什么?不够快?:D。欢迎，各位批评指正。

补充1、一个简单的hash函数：

```
/*key为一个字符串，nTableLength为哈希表的长度
*该函数得到的hash值分布比较均匀*/
unsigned long getHashIndex( const char *key, int nTableLength )
{
    unsigned long nHash = 0;

    while (*key)
    {
        nHash = (nHash<<5) + nHash + *key++;
    }

    return ( nHash % nTableLength );
}
```

补充2、一个完整测试程序：

哈希表的数组是定长的，如果太大，则浪费，如果太小，体现不出效率。合适的数组大小是哈希表的性能的关键。哈希表的尺寸最好是一个质数。当然，根据不同的数据量，会有不同的哈希表的大小。对于数据量时多时少的应用，最好的设计是使用动态可变尺寸的哈希表，那么如果你发现哈希表尺寸太小了，比如其中的元素是哈希表尺寸的2倍时，我们就需要扩大哈希表尺寸，一般是扩大一倍。

下面是哈希表尺寸大小的可能取值：

17, 37, 79, 163, 331,
673, 1361, 2729, 5471, 10949,
21911, 43853, 87719, 175447, 350899,
701819, 1403641, 2807303, 5614657, 11229331,
22458671, 44917381, 89834777, 179669557, 359339171,
718678369, 1437356741, 2147483647

以下为该程序的完整源码，已在linux下测试通过：

```
#include <stdio.h>
#include <ctype.h>     //多谢citylove指正。
//crytTable[]里面保存的是HashString函数里面将会用到的一些数据，在prepareCryptTable
//函数里面初始化
unsigned long cryptTable[0x500];

//以下的函数生成一个长度为0x500(合10进制数:1280)的cryptTable[0x500]
void prepareCryptTable()
{
    unsigned long seed = 0x00100001, index1 = 0, index2 = 0, i;

    for( index1 = 0; index1 < 0x100; index1++ )
    {
        for( index2 = index1, i = 0; i < 5; i++, index2 += 0x100 )
        {
            unsigned long temp1, temp2;

            seed = (seed * 125 + 3) % 0x2AAAAB;
            temp1 = (seed & 0xFFFF) << 0x10;

            seed = (seed * 125 + 3) % 0x2AAAAB;
            temp2 = (seed & 0xFFFF);

            cryptTable[index2] = ( temp1 | temp2 );
        }
    }
}
```

```
//以下函数计算lpszFileName 字符串的hash值，其中dwHashType 为hash的类型，
//在下面GetHashTablePos函数里面调用本函数,其可以取的值为0、1、2;该函数
//返回lpszFileName 字符串的hash值;
unsigned long HashString( char *lpszFileName, unsigned long dwHashType )
{
    unsigned char *key  = (unsigned char *)lpszFileName;
unsigned long seed1 = 0x7FED7FED;
unsigned long seed2 = 0xEEEEEEEE;
    int ch;

    while( *key != 0 )
    {
        ch = toupper(*key++);

        seed1 = cryptTable[(dwHashType << 8) + ch] ^ (seed1 + seed2);
        seed2 = ch + seed1 + seed2 + (seed2 << 5) + 3;
    }
    return seed1;
}

//在main中测试argv[1]的三个hash值:
//./hash  "arr/units.dat"
//./hash  "unit/neutral/acritter.grp"
int main( int argc, char **argv )
{
    unsigned long ulHashValue;
    int i = 0;

    if ( argc != 2 )
    {
        printf("please input two arguments/n");
        return -1;
    }

     /*初始化数组:crytTable[0x500]*/
     prepareCryptTable();

     /*打印数组crytTable[0x500]里面的值*/
     for ( ; i < 0x500; i++ )
     {
         if ( i % 10 == 0 )
         {
             printf("/n");
         }

         printf("%-12X", cryptTable[i] );
     }

     ulHashValue = HashString( argv[1], 0 );
     printf("/n----%X ----/n", ulHashValue );
```

```
    ulHashValue = HashString( argv[1], 1 );
    printf("----%X ----/n", ulHashValue );

    ulHashValue = HashString( argv[1], 2 );
    printf("----%X ----/n", ulHashValue );

    return 0;
}
```

分而治之:hash映射 + hash_map统计 + 排序

1、海量日志数据,提取出某日访问百度次数最多的那个IP。

思路分析

百度作为国内第一大搜索引擎,每天访问它的IP数量巨大,如果想一次性把所有IP数据装进内存处理,则内存容量明显不够,故针对数据太大,内存受限的情况,可以把大文件化成(取模映射)小文件,从而大而化小,逐个处理。换言之,先映射,而后统计,最后排序。

解法一

1. 分而治之/hash映射:首先把这一天访问百度的日志的所有IP提取出来,然后逐个写入到一个大文件中,接着采用映射的方法,比如%1000,把整个大文件映射为1000个小文件。
2. hash_map统计:当大文件转化了小文件,那么我们便可以采用hash_map(ip, value)来分别对1000个小文件中的IP进行频率统计,再找出每个小文件中出现频率最大的IP及相应的频率。
3. 堆/快速排序:统计出1000个频率最大的IP后,进行排序(可采取堆排序),找出那个频率最大的IP,即为所求。

关于本题,注意两点:

1. Hash取模是一种等价映射,不会存在同一个元素分散到不同小文件中去的情况,即这里采用的是%1000算法,那么同一个IP在hash后,只可能落在同一个文件中,不可能被分散的。
2. 那到底什么是hash映射呢?简单来说,就是为了便于计算机在有限的内存中处理big数据,从而通过一种映射散列的方式让数据均匀分布在对应的内存位置(如大数据通过取余的方式映射成小树存放在内存中,或大文

件映射成多个小文件)，而这个映射散列方式便是我们通常所说的hash函数，设计的好的hash函数能让数据均匀分布而减少冲突。尽管数据映射到了另外一些不同的位置，但数据还是原来的数据，只是代替和表示这些原始数据的形式发生了变化而已。

2、寻找热门查询，300万个查询字符串中统计最热门的10个查询

原题：搜索引擎会通过日志文件把用户每次检索使用的所有检索串都记录下来，每个查询串的长度为1-255字节。假设目前有一千万个记录，请你统计最热门的10个查询串，要求使用的内存不能超过1G。

注：这些查询串的重复度比较高，虽然总数是1千万，但如果除去重复后，不超过3百万个。一个查询串的重复度越高，说明查询它的用户越多，也就是越热门。

思路分析

由上面第1题，我们知道，数据大则划为小的，一亿个Ip求Top 10，可先%1000将ip分到1000个小文件中去，并保证一种ip只出现在一个文件中，再对每个小文件中的ip进行hashmap计数统计并按数量排序，最后归并或者最小堆依次处理每个小文件的top10以得到最后的结。

但对于本题，数据规模比较小，能一次性装入内存，因为根据题目描述，虽然有一千万个Query，但是由于重复度比较高，故去除重复后，事实上只有300万的Query，每个Query255Byte，因此我们可以考虑把他们都放进内存中去（300万个字符串假设没有重复，都是最大长度，那么最多占用内存3M*1K/4=0.75G。所以可以将所有字符串都存放在内存中进行处理），而现在只是需要一个合适的数据结构，在这里，HashTable绝对是我们优先的选择。

所以我们放弃分而治之/hash映射的步骤，直接上hash统计，然后排序。So，针对此类典型的TOP K问题，采取的对策往往是：hashmap + 堆。

解法一

1. hash_map统计：先对这批海量数据预处理。具体方法是：维护一个Key为Query字串，Value为该Query出现次数的HashTable，即hash_map(Query，Value)，每次读取一个Query，如果该字串不在Table中，那么加入该字串，并将Value值设为1；如果该字串在Table中，那么将该

字串的计数加1 即可。最终我们在O(N)的时间复杂度内用Hash表完成了统计；

2. 堆排序：第二步、借助堆这个数据结构，找出Top K，时间复杂度为N‘logK。即借助堆结构，我们可以在log量级的时间内查找和调整/移动。因此，维护一个K(该题目中是10)大小的小根堆，然后遍历300万的Query，分别和根元素进行对比。所以，我们最终的时间复杂度是：O（N）+ N’ * O（logK），（N为1000万，N’为300万）。

关于第2步堆排序，可以维护k个元素的最小堆，即用容量为k的最小堆存储最先遍历到的k个数，并假设它们即是最大的k个数，建堆费时O（k），并调整堆(费时O（logk）)后，有k1>k2>…kmin（kmin设为小顶堆中最小元素）。继续遍历数列，每次遍历一个元素x，与堆顶元素比较，若x>kmin，则更新堆（x入堆，用时logk），否则不更新堆。这样下来，总费时O（k*logk*+（*n-k*）logk）=O（n*logk）。此方法得益于在堆中，查找等各项操作时间复杂度均为logk。

解法二

当然，你也可以采用trie树，关键字域存该查询串出现的次数，没有出现为0。最后用10个元素的最小推来对出现频率进行排序。

3、有一个1G大小的一个文件，里面每一行是一个词，词的大小不超过16字节，内存限制大小是1M。返回频数最高的100个词。

解法一

由上面那两个例题，分而治之 + hash统计 + 堆/快速排序这个套路，我们已经开始有了屡试不爽的感觉。下面，再拿几道再多多验证下。请看此第3题：又是文件很大，又是内存受限，咋办?还能怎么办呢?无非还是：

1. 分而治之/hash映射：顺序读取文件，对于每个词x，取hash(x)%5000，然后把该值存到5000个小文件（记为x0,x1,…x4999）中。这样每个文件大概是200k左右。当然，如果其中有的小文件超过了1M大小，还可以按照类似的方法继续往下分，直到分解得到的小文件的大小都不超过1M。
2. hash_map统计：对每个小文件，采用trie树/hash_map等统计每个文件中出现的词以及相应的频率。
3. 堆/归并排序：取出出现频率最大的100个词（可以用含100个结点的最小堆）后，再把100个词及相应的频率存入文件，这样又得到了5000个文件。

最后就是把这5000个文件进行归并（类似于归并排序）的过程了。

4、海量数据分布在100台电脑中，想个办法高效统计出这批数据的TOP10。

解法一

如果同一个数据元素只出现在某一台机器中，那么可以采取以下步骤统计出现次数TOP10的数据元素：

1. 堆排序：在每台电脑上求出TOP10，可以采用包含10个元素的堆完成（TOP10小，用最大堆，TOP10大，用最小堆，比如求TOP10大，我们首先取前10个元素调整成最小堆，如果发现，然后扫描后面的数据，并与堆顶元素比较，如果比堆顶元素大，那么用该元素替换堆顶，然后再调整为最小堆。最后堆中的元素就是TOP10大）。
2. 求出每台电脑上的TOP10后，然后把这100台电脑上的TOP10组合起来，共1000个数据，再利用上面类似的方法求出TOP10就可以了。

解法二

但如果同一个元素重复出现在不同的电脑中呢，如下例子所述：

就拿2台机器求top2的情况来说：第一台：a(50),b(50),c(49),d(49),e(0),f(0);第二台：a(0),b(0),c(49),d(49),e(50),f(50)

这个时候，你可以有两种方法：

- 遍历一遍所有数据，重新hash取摸，如此使得同一个元素只出现在单独的一台电脑中，然后采用上面所说的方法，统计每台电脑中各个元素的出现次数找出TOP10，继而组合100台电脑上的TOP10，找出最终的TOP10。
- 或者，暴力求解：直接统计统计每台电脑中各个元素的出现次数，然后把同一个元素在不同机器中的出现次数相加，最终从所有数据中找出TOP10。

5、有10个文件，每个文件1G，每个文件的每一行存放的都是用户的query，每个文件的query都可能重复。要求你按照query的频度排序。

解法一

1. hash映射：顺序读取10个文件，按照hash(query)%10的结果将query写入到另外10个文件（记为a0,a1,..a9）中。这样新生成的文件每个的大小大约也1G（假设hash函数是随机的）。
2. hash_map统计：找一台内存在2G左右的机器，依次对用hash_map(query, query_count)来统计每个query出现的次数。注：hash_map(query,query_count)是用来统计每个query的出现次数，不是存储他们的值，出现一次，则count+1。
3. 堆/快速/归并排序：利用快速/堆/归并排序按照出现次数进行排序，将排序好的query和对应的query_cout输出到文件中，这样得到了10个排好序的文件（记为$b_0, b_1, \cdots, b_{10}$）。最后，对这10个文件进行归并排序（内排序与外排序相结合）。根据此解法1，这里有一份实现：。

解法二

一般query的总量是有限的，只是重复的次数比较多而已，可能对于所有的query，一次性就可以加入到内存了。这样，我们就可以采用trie树/hash_map等直接来统计每个query出现的次数，然后按出现次数做快速/堆/归并排序就可以了。

解法三

与解法1类似，但在做完hash，分成多个文件后，可以交给多个文件来处理，采用分布式的架构来处理（比如MapReduce），最后再进行合并。

6、 给定a、b两个文件，各存放50亿个url，每个url各占64字节，内存限制是4G，让你找出a、b文件共同的url？

解法

可以估计每个文件安的大小为5G×64=320G，远远大于内存限制的4G。所以不可能将其完全加载到内存中处理。考虑采取分而治之的方法。

1. 分而治之/hash映射：遍历文件a，对每个url求取$\text{hash}(url)\%1000$，然后根据所取得的值将url分别存储到1000个小文件（记为$a_0, a_2, \cdots, a_{999}$，这里漏写个了a1）中。这样每个小文件的大约为300M。遍历文件b，采取和a相同的方式将url分别存储到1000小文件中（记为$b_0, b_1, \cdots, b_{999}$）。这样处理后，所有可能相同的url都在对应的小文件（a_0 vs b_0, a_1 vs $b_1, \ldots, a_0$ vs b_{999}）中，不对应的小文件不可能有相同的url。然后我们只要求出1000对小文件中相同的url即可。
2. hash_set统计：求每对小文件中相同的url时，可以把其中一个小文件的url存储到hash_set中。然后遍历另一个小文件的每个url，看其是否在刚才构建的hash_set中，如果是，那么就是共同的url，存到文件里面就可以了。

OK，此第一种方法：分而治之/hash映射 + hash统计 + 堆/快速/归并排序，再看最后4道题，如下：

7、怎么在海量数据中找出重复次数最多的一个？

解法：先做hash，然后求模映射为小文件，求出每个小文件中重复次数最多的一个，并记录重复次数。然后找出上一步求出的数据中重复次数最多的一个就是所求（具体参考前面的题）。

8、上千万或上亿数据（有重复），统计其中出现次数最多的前N个数据。

解法：上千万或上亿的数据，现在的机器的内存应该能存下。所以考虑采用hash_map/搜索二叉树/红黑树等来进行统计次数。然后就是取出前N个出现次数最多的数据了，可以用第2题提到的堆机制完成。

9、一个文本文件，大约有一万行，每行一个词，要求统计出其中最频繁出现的前10个词，请给出思想，给出时间复杂度分析。

解法一：这题是考虑时间效率。用trie树统计每个词出现的次数，时间复杂度是O(n*le)（le表示单词的平准长度）。然后是找出出现最频繁的前10个词，可以用堆来实现，前面的题中已经讲到了，时间复杂度是O(n*lg10)。所以总的时间复杂度，是O(n*le)与O(n*lg10)中较大的哪一个。

10. 1000万字符串，其中有些是重复的，需要把重复的全部去掉，保留没有重复的字符串。请怎么设计和实现？

解法一:这题用trie树比较合适，hash_map也行。

解法二:1000w的数据规模插入操作完全不现实,以前试过在stl下100w元素插入set中已经慢得不能忍受,觉得基于hash的实现不会比红黑树好太多,使用vector+sort+unique都要可行许多,建议还是先hash成小文件分开处理再综合。

上述解法2中读者xbzju的方法让我想到了一些问题,即是**set/map,与hash_set/hash_map的性能比较**?共计3个问题,如下:

1. hash_set在千万级数据下,insert操作优于set? 这位blog:http://t.cn/zOibP7t给的实践数据可靠不?
2. 那map和hash_map的性能比较呢? 谁做过相关实验?

```
set VS hash_set VS hash_table(强化版) 性能测试
数据容量 10000000个 查询次数 10000000次
容器中数据范围 [0, 40000000) 查询数据范围[0, 40000000)
--by MoreWindows( http://blog.csdn.net/MoreWindows ) --

-----插入数据-----------
set中有数据8061105个
set 的 insert操作 用时 18782毫秒
hash_set中有数据8061105个
hash_set 的 insert操作 用时 7722毫秒
hash_table中有数据8061105个
Hash_table 的 insert操作 用时 4930毫秒
```

3. 那查询操作呢,如下段文字所述?

> 可以发现在hash_table中最长的链表也只有5个元素，**长度为1和长度为2的链表中的数据占全部数据的89%以上。因此绝大数查询将仅仅访问哈希表1次到2次。这样的查询效率当然会比set（内部使用红黑树——类似于二叉平衡树）高的多**。有了这个图示，无疑已经可以证明hash_set会比set快速高效了。但hash_set还可以动态的增加表的大小，因此我们再实现一个表大小可增加的hash_table。

或者小数据量时用map,构造快，大数据量时用hash_map?

rbtree PK hashtable

据朋友№邦卡猫№的做的红黑树和hash table的性能测试中发现:当数据量基本上int型key时，hash table是rbtree的3-4倍,但hash table一般会浪费大概一半内存。

因为hash table所做的运算就是个%,而rbtree要比较很多,比如rbtree要看value的数据,每个节点要多出3个指针(或者偏移量)如果需要其他功能,比如,统

计某个范围内的key的数量，就需要加一个计数成员。

且1s rbtree能进行大概50w+次插入，hash table大概是差不多200w次。不过很多的时候，其速度可以忍了，例如倒排索引差不多也是这个速度，而且单线程，且倒排表的拉链长度不会太大。正因为基于树的实现其实不比hashtable慢到哪里去，所以数据库的索引一般都是用的B/B+树，而且B+树还对磁盘友好(B树能有效降低它的高度，所以减少磁盘交互次数)。比如现在非常流行的NoSQL数据库，像MongoDB也是采用的B树索引。关于B树系列，请参考本blog内此篇文章：从B树、B+树、B*树谈到R 树。更多请待后续实验论证。

11. 一个文本文件，找出前10个经常出现的词，但这次文件比较长，说是上亿行或十亿行，总之无法一次读入内存，问最优解。

解法一：首先根据用hash并求模，将文件分解为多个小文件，对于单个文件利用上题的方法求出每个文件件中10个最常出现的词。然后再进行归并处理，找出最终的10个最常出现的词。

12. 100w个数中找出最大的100个数。

解法一：采用局部淘汰法。选取前100个元素，并排序，记为序列L。然后一次扫描剩余的元素x，与排好序的100个元素中最小的元素比，如果比这个最小的要大，那么把这个最小的元素删除，并把x利用插入排序的思想，插入到序列L中。依次循环，知道扫描了所有的元素。复杂度为O(100w*100)。

解法二：采用快速排序的思想，每次分割之后只考虑比轴大的一部分，知道比轴大的一部分在比100多的时候，采用传统排序算法排序，取前100个。复杂度为O(100w*100)。

解法三：在前面的题中，我们已经提到了，用一个含100个元素的最小堆完成。复杂度为O(100w*lg100)。

举一反三

现有一200M的文本文件，里面记录着IP地址和对应地域信息，如

202.100.83.56 北京 北京大学

202.100.83.120 北京 人民大学

202.100.83.134 北京 中国青年政治学院

211.93.120.45 长春市 长春大学

211.93.120.129 吉林市 吉林大学

211.93.120.200 长春 长春KTV

现有6亿个IP地址，请编写程序，读取IP地址便转换成IP地址相对应的城市，要求有较好的时间复杂度和空间复杂度。

双层桶划分

双层桶划分—-其实本质上还是分而治之的思想，重在“分”的技巧上！

适用范围：第k大，中位数，不重复或重复的数字

基本原理及要点：因为元素范围很大，不能利用直接寻址表，所以通过多次划分，逐步确定范围，然后最后在一个可以接受的范围内进行。可以通过多次缩小，双层只是一个例子。

扩展：

问题实例：

13、2.5亿个整数中找出不重复的整数的个数，内存空间不足以容纳这2.5亿个整数。

有点像鸽巢原理，整数个数为2^{32},也就是，我们可以将这2^{32}个数，划分为2^8个区域(比如用单个文件代表一个区域)，然后将数据分离到不同的区域，然后不同的区域在利用bitmap就可以直接解决了。也就是说只要有足够的磁盘空间，就可以很方便的解决。

14、5亿个int找它们的中位数。

1. 思路一：这个例子比上面那个更明显。首先我们将int划分为2^16个区域，然后读取数据统计落到各个区域里的数的个数，之后我们根据统计结果

就可以判断中位数落到那个区域，同时知道这个区域中的第几大数刚好是中位数。然后第二次扫描我们只统计落在这个区域中的那些数就可以了。

实际上，如果不是int是int64，我们可以经过3次这样的划分即可降低到可以接受的程度。即可以先将int64分成2^{24}个区域，然后确定区域的第几大数，在将该区域分成2^{20}个子区域，然后确定是子区域的第几大数，然后子区域里的数的个数只有2^20，就可以直接利用direct addr table进行统计了。

2. 思路二@绿色夹克衫：同样需要做两遍统计，如果数据存在硬盘上，就需要读取2次。

方法同基数排序有些像，开一个大小为65536的Int数组，第一遍读取，统计Int32的高16位的情况，也就是0-65535，都算作0,65536 - 131071都算作1。就相当于用该数除以65536。Int32 除以 65536的结果不会超过65536种情况，因此开一个长度为65536的数组计数就可以。每读取一个数，数组中对应的计数+1，考虑有负数的情况，需要将结果加32768后，记录在相应的数组内。

第一遍统计之后，遍历数组，逐个累加统计，看中位数处于哪个区间，比如处于区间k，那么0- k-1的区间里数字的数量sum应该<n/2（2.5亿）。而k+1 - 65535的计数和也<n/2，第二遍统计同上面的方法类似，但这次只统计处于区间k的情况，也就是说(x / 65536) + 32768 = k。统计只统计低16位的情况。并且利用刚才统计的sum，比如sum = 2.49亿，那么现在就是要在低16位里面找100万个数(2.5亿-2.49亿)。这次计数之后，再统计一下，看中位数所处的区间，最后将高位和低位组合一下就是结果了。

Bitmap

Bitmap

什么是Bit-map

所谓的Bit-map就是用一个bit位来标记某个元素对应的Value，而Key即是该元素。由于采用了Bit为单位来存储数据，因此在存储空间方面，可以大大节省。

来看一个具体的例子，假设我们要对0-7内的5个元素(4,7,2,5,3)排序（这里假设这些元素没有重复）。那么我们就可以采用Bit-map的方法来达到排序的目的。要表示8个数，我们就只需要8个Bit（1Bytes），首先我们开辟1Byte的空间，将这些空间的所有Bit位都置为0(如下图：)

然后遍历这5个元素，首先第一个元素是4，那么就把4对应的位置为1（可以这样操作 p+(i/8)|(0×01<<(i%8)) 当然了这里的操作涉及到Big-ending和Little-ending的情况，这里默认为Big-ending）,因为是从零开始的，所以要把第五位置为一（如下图）：

然后再处理第二个元素7，将第八位置为1,，接着再处理第三个元素，一直到最后处理完所有的元素，将相应的位置为1，这时候的内存的Bit位的状态如下：

然后我们现在遍历一遍Bit区域，将该位是一的位的编号输出（2，3，4，5，7），这样就达到了排序的目的。下面的代码给出了一个BitMap的用法：排序。

```
//定义每个Byte中有8个Bit位
#include <memory.h>
#define BYTESIZE 8
void SetBit(char *p, int posi)
{
    for(int i=0; i < (posi/BYTESIZE); i++)
    {
        p++;
    }

    *p = *p|(0x01<<(posi%BYTESIZE));//将该Bit位赋值1
    return;
}

void BitMapSortDemo()
{
    //为了简单起见，我们不考虑负数
    int num[] = {3,5,2,10,6,12,8,14,9};

    //BufferLen这个值是根据待排序的数据中最大值确定的
    //待排序中的最大值是14,因此只需要2个Bytes(16个Bit)
    //就可以了。
    const int BufferLen = 2;
    char *pBuffer = new char[BufferLen];

    //要将所有的Bit位置为0,否则结果不可预知。
    memset(pBuffer,0,BufferLen);
    for(int i=0;i<9;i++)
    {
        //首先将相应Bit位上置为1
        SetBit(pBuffer,num[i]);
    }

    //输出排序结果
    for(int i=0;i<BufferLen;i++)//每次处理一个字节(Byte)
    {
        for(int j=0;j<BYTESIZE;j++)//处理该字节中的每个Bit位
        {
            //判断该位上是否是1,进行输出,这里的判断比较笨。
            //首先得到该第j位的掩码(0x01<<j),将内存区中的
            //位和此掩码作与操作。最后判断掩码是否和处理后的
            //结果相同
            if((*pBuffer&(0x01<<j)) == (0x01<<j))
            {
                printf("%d ",i*BYTESIZE + j);
            }
        }
        pBuffer++;
```

```
    }
}

int _tmain(int argc, _TCHAR* argv[])
{
    BitMapSortDemo();
    return 0;
}
```

可进行数据的快速查找，判重，删除，一般来说数据范围是int的10倍以下

基本原理及要点

使用bit数组来表示某些元素是否存在，比如8位电话号码.

问题实例：

“13、在2.5亿个整数中找出不重复的整数，注，内存不足以容纳这2.5亿个整数。

方案1：采用2-Bitmap（每个数分配2bit，00表示不存在，01表示出现一次，10表示多次，11无意义）进行，共需内存2^32 * 2 bit=1 GB内存，还可以接受。然后扫描这2.5亿个整数，查看Bitmap中相对应位，如果是00变01，01变10，10保持不变。所描完事后，查看bitmap，把对应位是01的整数输出即可。

方案2：也可采用与第1题类似的方法，进行划分小文件的方法。然后在小文件中找出不重复的整数，并排序。然后再进行归并，注意去除重复的元素。”

15、 给40亿个不重复的unsigned int的整数，没排过序的，然后再给一个数，如何快速判断这个数是否在那40亿个数当中？

方案1：frome oo，用位图/Bitmap的方法，申请512M的内存，一个bit位代表一个unsigned int值。读入40亿个数，设置相应的bit位，读入要查询的数，查看相应bit位是否为1，为1表示存在，为0表示不存在。

Bloom Filter

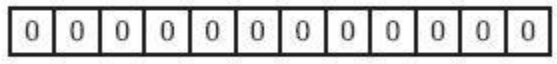
一、什么是Bloom Filter

Bloom Filter，被译作称布隆过滤器，是一种空间效率很高的随机数据结构，Bloom filter可以看做是对bit-map的扩展,它的原理是，**当一个元素被加入集合时，通过K个Hash函数将这个元素映射成一个位阵列（Bit array）中的K个点，把它们置为1**。检索时，我们只要看看这些点是不是都是1就（大约）知道集合中有没有它了：如果这些点有任何一个0，则被检索元素一定不在；如果都是1，则被检索元素很可能在。这就是布隆过滤器的基本思想。

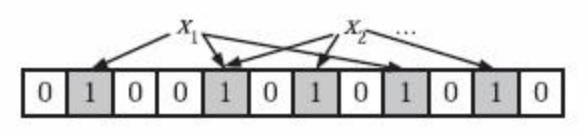
但Bloom Filter的这种高效是有一定代价的：在判断一个元素是否属于某个集合时，有可能会把不属于这个集合的元素误认为属于这个集合（false positive）。因此，Bloom Filter不适合那些“零错误”的应用场合。而在能容忍低错误率的应用场合下，Bloom Filter通过极少的错误换取了存储空间的极大节省。

1.1、集合表示和元素查询

下面我们具体来看Bloom Filter是如何用位数组表示集合的。初始状态时，Bloom Filter是一个包含m位的位数组，每一位都置为0。



为了表达S={x_1, x_2,…,x_n}这样一个n个元素的集合，Bloom Filter使用k个相互独立的哈希函数（Hash Function），它们分别将集合中的每个元素映射到{1,…,m}的范围中。对任意一个元素x，第i个哈希函数映射的位置$h_i(x)$就会被置为1（1≤i≤k）。注意，如果一个位置多次被置为1，那么只有第一次会起作用，后面几次将没有任何效果。在下图中，k=3，且有两个哈希函数选中同一个位置（从左边数第五位，即第二个“1“处）。



在判断y是否属于这个集合时，我们对y应用k次哈希函数，如果所有$h_i(y)$的位置都是1（$1 \le i \le k$），那么我们就认为y是集合中的元素，否则就认为y不是集合中的元素。下图中y_1就不是集合中的元素（因为y1有一处指向了“0”位）。y_2或者属于这个集合，或者刚好是一个false positive。

1.2、错误率估计

前面我们已经提到了，Bloom Filter在判断一个元素是否属于它表示的集合时会有一定的错误率（false positive rate），下面我们就来估计错误率的大小。在估计之前为了简化模型，我们假设kn1, $x_2,\ldots,x_n$}的所有元素都被k个哈希函数映射到m位的位数组中时，这个位数组中某一位还是0的概率是：

[img](http://chart.apis.google.com/chart?cht=tx&chl=p’=\left(1-\frac{1}{m}\right)^{kn}\approx e^{-kn/m})

其中1/m表示任意一个哈希函数选中这一位的概率（前提是哈希函数是完全随机的），(1-1/m)表示哈希一次没有选中这一位的概率。要把S完全映射到位数组中，需要做kn次哈希。某一位还是0意味着kn次哈希都没有选中它，因此这个概率就是（1-1/m）的kn次方。令$p = e^{-kn/m}$是为了简化运算，这里用到了计算e时常用的近似：

img

img

令ρ为位数组中0的比例，则ρ的数学期望E(ρ)= p’。在ρ已知的情况下，要求的错误率（false positive rate）为：

img

(1-ρ)为位数组中1的比例，$(1-\rho)^k$就表示k次哈希都刚好选中1的区域，即false positive rate。上式中第二步近似在前面已经提到了，现在来看第一步近似。p’只是ρ的数学期望，在实际中ρ的值有可能偏离它的数学期望值。M. Mitzenmacher已经证明[2]，位数组中0的比例非常集中地分布在它的数学期望值的附近。因此，第一步的近似得以成立。分别将p和p’代入上式中，得：

f^k)

img

相比p’和f’，使用p和f通常在分析中更为方便。

1.3、最优的哈希函数个数

既然Bloom Filter要靠多个哈希函数将集合映射到位数组中，那么应该选择几个哈希函数才能使元素查询时的错误率降到最低呢？这里有两个互斥的理由：如果哈希函数的个数多，那么在对一个不属于集合的元素进行查询时得到0的概率就大；但另一方面，如果哈希函数的个数少，那么位数组中的0就多。为了得到最优的哈希函数个数，我们需要根据上一小节中的错误率公式进行计算。

先用p和f进行计算。注意到$f = \exp(k \ln(1 - e^{-kn/m}))$，我们令$g = k \ln(1 - e^{-kn/m})$，只要让g取到最小，f自然也取到最小。由于$p = e^{-kn/m}$，我们可以将g写成

img

img

根据对称性法则可以很容易看出当$p = 1/2$，也就是$k = \ln 2 \cdot (m/n)$时，g取得最小值。在这种情况下，最小错误率f等于$(1/2)^k \approx (0.6185)^{m/n}$。另外，注意到p是位数组中某一位仍是0的概率，所以$p = 1/2$对应着位数组中0和1各一半。换句话说，要想保持错误率低，最好让位数组有一半还空着。

需要强调的一点是，$p = 1/2$时错误率最小这个结果并不依赖于近似值p和f。同样对于$f' = \exp(k \ln(1 - (1 - 1/m)^{kn}))$，$g' = k \ln(1 - (1 - 1/m)^{kn})$，$p' = (1 - 1/m)^{kn}$，我们可以将g'写成

$g\backslash\ln(1-p'))$

同样根据对称性法则可以得到当$p' = 1/2$时，g'取得最小值。

1.4、位数组的大小

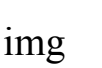
下面我们来看看，在不超过一定错误率的情况下，Bloom Filter至少需要多少位才能表示全集中任意n个元素的集合。假设全集中共有u个元素，允许的最大错误率为ϵ，下面我们来求位数组的位数m。

假设X为全集中任取n个元素的集合，F(X)是表示X的位数组。那么对于集合X中任意一个元素x，在$s = F(X)$中查询x都能得到肯定的结果，即s能够接受x。显然，由于Bloom Filter引入了错误，s能够接受的不仅仅是X中的元素，它还能够$\epsilon (u - n)$个false positive。因此，对于一个确定的位数组来说，它能够接受总共$n + \epsilon (u - n)$个元素。在$n + \epsilon (u - n)$个元素中，s真正表示的只有其中n个，所以一个确定的位数组可以表示

img

个集合。m位的位数组共有2^m个不同的组合，进而可以推出，m位的位数组可以表示

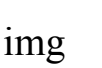
img



img

个集合。全集中n个元素的集合总共有

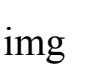
img



img

个，因此要让m位的位数组能够表示所有n个元素的集合，必须有

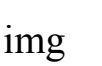
img



img

即：

img

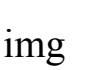


img

上式中的近似前提是n和ϵu相比很小，这也是实际情况中常常发生的。根据上式，我们得出结论：在错误率不大于ϵ的情况下，m至少要等于$n \log_2(1/\epsilon)$才能表示任意n个元素的集合。

上一小节中我们曾算出当$k = \ln 2 \cdot (m/n)$时错误率f最小，这时$f = (1/2)^k = (1/2)^{m \ln 2 / n}$。现在令$f \leq \epsilon$，可以推出

img



img

这个结果比前面我们算得的下界$n \log_2(1/\epsilon)$大了$\log_2 e \approx 1.44$倍。这说明在哈希函数的个数取到最优时，要让错误率不超过ϵ，m至少需要取到最小值的1.44倍。

1.5、概括

在计算机科学中，我们常常会碰到时间换空间或者空间换时间的情况，即为了达到某一个方面的最优而牺牲另一个方面。Bloom Filter在时间空间这两个因素之外又引入了另一个因素：错误率。在使用Bloom Filter判断一个元素是否属于某个集合时，会有一定的错误率。也就是说，有可能把不属于这个集合的元素误认为属于这个集合（False Positive），但不会把属于这个集合的元素误认为不属于这个集合（False Negative）。在增加了错误率这个因素之后，Bloom Filter通过允许少量的错误来节省大量的存储空间。

自从Burton Bloom在70年代提出Bloom Filter之后，Bloom Filter就被广泛用于拼写检查和数据库系统中。近一二十年，伴随着网络的普及和发展，Bloom Filter在网络领域获得了新生，各种Bloom Filter变种和新的应用不断出现。可以预见，随着网络应用的不断深入，新的变种和应用将会继续出现，Bloom Filter必将获得更大的发展。

二、适用范围

可以用来实现数据字典，进行数据的判重，或者集合求交集

三、基本原理及要点

对于原理来说很简单，位数组+k个独立hash函数。将hash函数对应的值的位数组置1，查找时如果发现所有hash函数对应位都是1说明存在，很明显这 个过程并不保证查找的结果是100%正确的。同时也不支持删除一个已经插入的关键字，因为该关键字对应的位会牵动到其他的关键字。所以一个简单的改进就是 counting Bloom filter，用一个counter数组代替位数组，就可以支持删除了。

还有一个比较重要的问题，如 何根据输入元素个数n，确定位数组m的大小及hash函数个数。当hash函数个数$k=(\ln 2)*(m/n)$时错误率最小。在错误率不大于E的情况 下，m至少要等于$n*\lg(1/E)$才能表示任意n个元素的集合。但m还应该更大些，因为还要保证bit数组里至少一半为0，则m应该$>=n\lg(1/E)*\lg e$ 大概就是$n\lg(1/E)1.44$倍(lg表示以2为底的对数)。

举个例子我们假设错误率为0.01，则此时m应大概是n的13倍。这样k大概是8个。

注意这里m与n的单位不同，m是bit为单位，而n则是以元素个数为单位(准确的说是不同元素的个数)。通常单个元素的长度都是有很多bit的。所以使用bloom filter内存上通常都是节省的。

四、扩展

Bloom filter将集合中的元素映射到位数组中，用k（k为哈希函数个数）个映射位是否全1表示元素在不在这个集合中。Counting bloom filter（CBF）将位数组中的每一位扩展为一个counter，从而支持了元素的删除操作。Spectral Bloom Filter（SBF）将其与集合元素的出现次数关联。SBF采用counter中的最小值来近似表示元素的出现频率。

五、问题实例

可以看下上文中的第6题：给你A,B两个文件，各存放50亿条URL，每条URL占用64字节，内存限制是4G，让你找出A,B文件共同的URL。如果是三个乃至n个文件呢？

根据这个问题我们来计算下内存的占用，4G=2^32大概是40亿*8大概是340亿，n=50亿，如果按出错率0.01算需要的大概是650亿个bit。现在可用的是340亿，相差并不多，这样可能会使出错率上升些。另外如果这些urlip是一一对应的，就可以转换成ip，则大大简单了。

同时，上文的第5题：给定a、b两个文件，各存放50亿个url，每个url各占64字节，内存限制是4G，让你找出a、b文件共同的url？如果允许有一定的错误率，可以使用Bloom filter，4G内存大概可以表示340亿bit。将其中一个文件中的url使用Bloom filter映射为这340亿bit，然后挨个读取另外一个文件的url，检查是否与Bloom filter，如果是，那么该url应该是共同的url（注意会有一定的错误率）。”

Trie树

Trie树

适用范围：数据量大，重复多，但是数据种类小可以放入内存

基本原理及要点:实现方式,节点孩子的表示方式

扩展:压缩实现。

问题实例:

1. 上面的**第2题**:寻找热门查询:查询串的重复度比较高,虽然总数是1千万,但如果除去重复后,不超过3百万个,每个不超过255字节。
2. 上面的**第5题**:有10个文件,每个文件1G,每个文件的每一行都存放的是用户的query,每个文件的query都可能重复。要你按照query的频度排序。
3. 1000万字符串,其中有些是相同的(重复),需要把重复的全部去掉,保留没有重复的字符串。请问怎么设计和实现?
4. 上面的**第8题**:一个文本文件,大约有一万行,每行一个词,要求统计出其中最频繁出现的前10个词。其解决方法是:用trie树统计每个词出现的次数,时间复杂度是O(n*le)(le表示单词的平准长度),然后是找出出现最频繁的前10个词。

更多有关Trie树的介绍,请参见此文:从Trie树(字典树)谈到后缀树。

数据库索引

适用范围:大数据量的增删改查

基本原理及要点:利用数据的设计实现方法,对海量数据的增删改查进行处理。

- 关于数据库索引及其优化,更多可参见此文:;
- 关于MySQL索引背后的数据结构及算法原理,这里还有一篇很好的文章:;
- 关于B 树、B+ 树、B* 树及R 树,本blog内有篇绝佳文章:从B树、B+树、B*树谈到R 树。

倒排索引(Inverted index)

适用范围:搜索引擎,关键字查询

基本原理及要点:为何叫倒排索引?一种索引方法,被用来存储在全文搜索下某个单词在一个文档或者一组文档中的存储位置的映射。

以英文为例,下面是要被索引的文本:

```
T0 = "it is what it is"
T1 = "what is it"
T2 = "it is a banana"
```

我们就能得到下面的反向文件索引:

```
"a":      {2}
"banana": {2}
"is":     {0, 1, 2}
"it":     {0, 1, 2}
"what":   {0, 1}
```

检索的条件“what”,“is”和“it”将对应集合的交集。

正向索引开发出来用来存储每个文档的单词的列表。正向索引的查询往往满足每个文档有序频繁的全文查询和每个单词在校验文档中的验证这样的查询。在正向索引中,文档占据了中心的位置,每个文档指向了一个它所包含的索引项的序列。也就是说文档指向了它包含的那些单词,而反向索引则是单词指向了包含它的文档,很容易看到这个反向的关系。

扩展:

问题实例:文档检索系统,查询那些文件包含了某单词,比如常见的学术论文的关键字搜索。

关于倒排索引的应用,更多请参见:

- 第二十三章:杨氏矩阵查找,
- 第二十四章:倒排索引关键词Hash不重复编码实践
- 第二十六章:基于给定的文档生成倒排索引的编码与实践。

外排序

适用范围：大数据的排序，去重

基本原理及要点：外排序的归并方法，置换选择败者树原理，最优归并树

扩展：

问题实例：

问题1

有一个1G大小的一个文件，里面每一行是一个词，词的大小不超过16个字节，内存限制大小是1M。返回频数最高的100个词。

这个数据具有很明显的特点，词的大小为16个字节，但是内存只有1M做hash明显不够，所以可以用来排序。内存可以当输入缓冲区使用。

关于多路归并算法及外排序的具体应用场景，请参见blog内此文。

问题2

输入：给定一个文件，里面最多含有n个不重复的正整数（也就是说可能含有少于n个不重复正整数），且其中每个数都小于等于n，n=10^7。 输出：得到按从小到大升序排列的包含所有输入的整数的列表。 条件：最多有大约1MB的内存空间可用，但磁盘空间足够。且要求运行时间在5分钟以下，10秒为最佳结果。

解法一、位图方案

你可能会想到把磁盘文件进行归并排序，但题目要求你只有1MB的内存空间可用，所以，归并排序这个方法不行。

熟悉位图的朋友可能会想到用位图来表示这个文件集合。例如正如编程珠玑一书上所述，用一个20位长的字符串来表示一个所有元素都小于20的简单的非负整数集合，边框用如下字符串来表示集合{1,2,3,5,8,13}：

0 1 1 1 0 1 0 0 1 0 0 0 0 1 0 0 0 0 0 0

上述集合中各数对应的位置则置1，没有对应的数的位置则置0。

参考编程珠玑一书上的位图方案，针对我们的10^7个数据量的磁盘文件排序问题，我们可以这么考虑，由于每个7位十进制整数表示一个小于1000万的整数。我们可以使用一个具有1000万个位的字符串来表示这个文件，其中，当且仅当整数i在文件中存在时，第i位为1。采取这个位图的方案是因为我们面对的这个问题的特殊性：

1. 输入数据限制在相对较小的范围内，
2. 数据没有重复，
3. 其中的每条记录都是单一的整数，没有任何其它与之关联的数据。

所以，此问题用位图的方案分为以下三步进行解决：

- 第一步，将所有的位都置为0，从而将集合初始化为空。
- 第二步，通过读入文件中的每个整数来建立集合，将每个对应的位都置为1。
- 第三步，检验每一位，如果该位为1，就输出对应的整数。

经过以上三步后，产生有序的输出文件。令n为位图向量中的位数（本例中为1000 0000），程序可以用伪代码表示如下：

```
//磁盘文件排序位图方案的伪代码
//copyright@ Jon Bentley
//July、updated,2011.05.29。

//第一步，将所有的位都初始化为0
for i ={0,....n}
   bit[i]=0;
//第二步，通过读入文件中的每个整数来建立集合，将每个对应的位都置为1。
for each i in the input file
   bit[i]=1;

//第三步，检验每一位，如果该位为1，就输出对应的整数。
for i={0...n}
  if bit[i]==1
    write i on the output file
```

上述的位图方案，共需要扫描输入数据两次，具体执行步骤如下：

第一次，只处理1—4999999之间的数据，这些数都是小于5000000的，对这些数进行位图排序，只需要约5000000/8=625000Byte，也就是0.625M，排序后输出。第二次，扫描输入文件时，只处理4999999-10000000的数据项，也只需要0.625M（可以使用第一次处理申请的内存）。因此，总共也只需要0.625M 位图的的方法有必要强调一下，就是位图的适用范围为针对不重复的数据进行排序，若数据有重复，位图方案就不适用了。

不过很快，我们就将意识到，用此位图方法，严格说来还是不太行，空间消耗10^7/8还是大于1M（1M=1024*1024空间，小于10^7/8）。

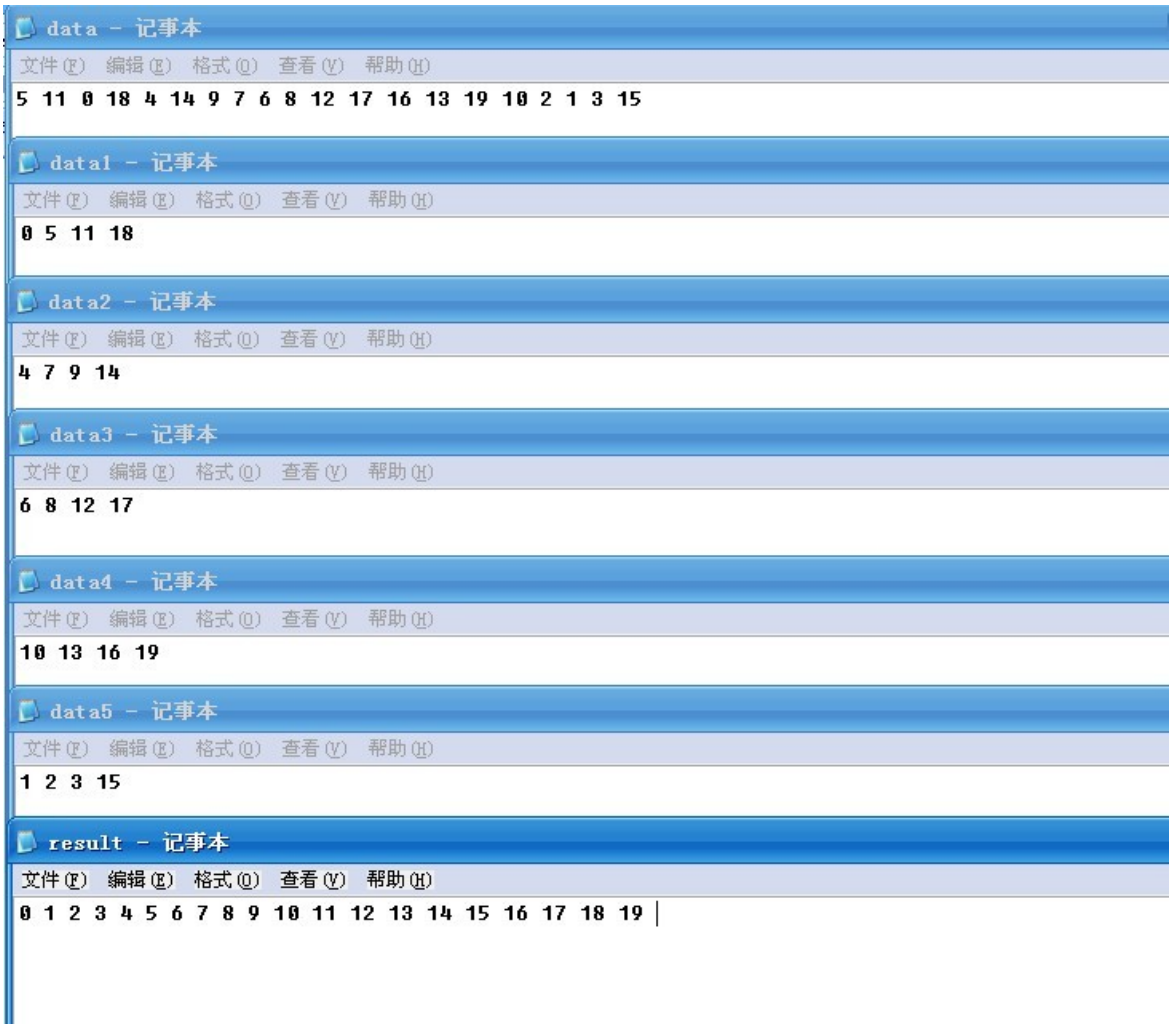
既然如果用位图方案的话，我们需要约1.25MB（若每条记录是8位的正整数的话，则10000000/(1024*1024*8) ~= 1.2M）的空间，而现在只有1MB的可用存储空间，那么究竟该作何处理呢？

解法二、多路归并

诚然，在面对本题时，还可以通过计算分析出可以用如2的位图法解决，但实际上，很多的时候，我们都面临着这样一个问题，文件太大，无法一次性放入内存中计算处理，那这个时候咋办呢？分而治之，大而化小，也就是把整个大文件分为若干大小的几块，然后分别对每一块进行排序，最后完成整个过程的排序。k趟算法可以在kn的时间开销内和n/k的空间开销内完成对最多n个小于n的无重复正整数的排序。

比如可分为2块（k=2，1趟反正占用的内存只有1.25/2M），1$_{4999999}$，和$_{5000000}$9999999。先遍历一趟，首先排序处理1$_{4999999}$之间的整数（用5000000/8=625000个字的存储空间来排序0$_{4999999}$之间的整数），然后再第二趟，对5000001~1000000之间的整数进行排序处理。

我们下面针对20个小数据的文件举例说明，每趟对4个数据进行排序，即5路归并，程序的排序结果如下图所示。



运行时间：

0ms，可以忽略不计了，毕竟是对20个数的小数据量进行排序：

解法总结:

1、关于本章中位图和多路归并两种方案的时间复杂度及空间复杂度的比较，如下:

时间复杂度 空间复杂度

位图 O(N) 0.625M 多位归并 O(Nlogn) 1M

（多路归并，时间复杂度为O（k*n/k*logn/k ），严格来说，还要加上读写磁盘的时间，而此算法绝大部分时间也是浪费在这上面 ）

2 、bit-map

适用范围:可进行数据的快速查找，判重，删除，一般来说数据范围是int的10倍以下 基本原理及要点:使用bit数组来表示某些元素是否存在，比如8位电话号码 扩展:bloom filter可以看做是对bit-map的扩展

问题实例:

1)已知某个文件内包含一些电话号码，每个号码为8位数字，统计不同号码的个数。8位最多99 999 999，大概需要99m个bit，大概10几m字节的内存即可。

2)2.5亿个整数中找出不重复的整数的个数，内存空间不足以容纳这2.5亿个整数。

将bit-map扩展一下，用2bit表示一个数即可，0表示未出现，1表示出现一次，2表示出现2次及以上。或者我们不用2bit来进行表示，我们用两个bit-map即可模拟实现这个2bit-map。

3、[外排序适用范围]大数据的排序，去重基本原理及要点：外排序的归并方法，置换选择败者树原理，最优归并树扩展。问题实例：1).有一个1G大小的一个文件，里面每一行是一个词，词的大小不超过16个字节，内存限制大小是1M。返回频数最高的100个词。这个数据具有很明显的特点，词的大小为16个字节，但是内存只有1m做hash有些不够，所以可以用来排序。内存可以当输入缓冲区使用。

分布式处理之Mapreduce

MapReduce是一种计算模型，简单的说就是将大批量的工作（数据）分解（MAP）执行，然后再将结果合并成最终结果（REDUCE）。这样做的好处是可以在任务被分解后，可以通过大量机器进行并行计算，减少整个操作的时间。但如果你要我再通俗点介绍，那么，说白了，Mapreduce的原理就是一个归并排序。

适用范围：数据量大，但是数据种类小可以放入内存

基本原理及要点：将数据交给不同的机器去处理，数据划分，结果归约。

扩展：

问题实例：

1. The canonical example application of MapReduce is a process to count the appearances of each different word in a set of documents:
2. 海量数据分布在100台电脑中，想个办法高效统计出这批数据的TOP10。
3. 一共有N个机器，每个机器上有N个数。每个机器最多存O(N)个数并对它们操作。如何找到N^2个数的中数(median)？

基础架构

想读懂此文，读者必须先要明确以下几点，以作为阅读后续内容的基础知识储备：

1. Mapreduce是一种模式。
2. Hadoop是一种框架。
3. Hadoop是一个实现了mapreduce模式的开源的分布式并行编程框架。

所以，你现在，知道了什么是mapreduce，什么是hadoop，以及这两者之间最简单的联系，而本文的主旨即是，一句话概括：**在hadoop的框架上采取mapreduce的模式处理海量数据**。下面，咱们可以依次深入学习和了解mapreduce和hadoop这两个东西了。

Mapreduce模式

前面说了，mapreduce是一种模式，一种什么模式呢?一种云计算的核心计算模式，一种分布式运算技术，也是简化的分布式编程模式，它主要用于解决问题的程序开发模型，也是开发人员拆解问题的方法。

Ok，光说不上图，没用。如下图所示，mapreduce模式的主要思想是将自动分割要执行的问题（例如程序）拆解成map（映射）和reduce（化简）的方式，流程图如下图1所示：

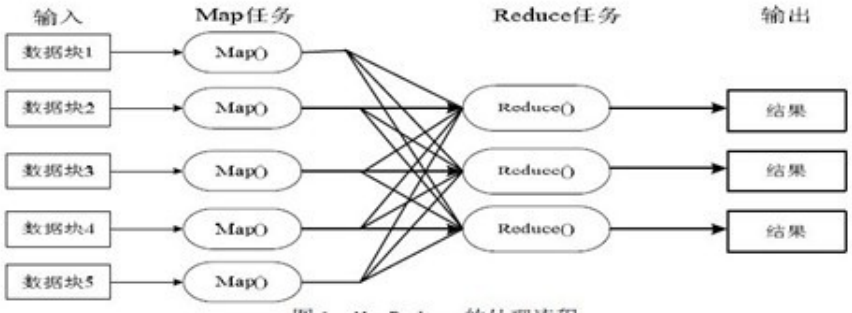


图 1　MapReduce 的处理流程

在数据被分割后通过Map 函数的程序将数据映射成不同的区块，分配给计算机机群处理达到分布式运算的效果，在通过Reduce 函数的程序将结果汇整，从而输出开发者需要的结果。

MapReduce 借鉴了函数式程序设计语言的设计思想，其软件实现是指定一个Map 函数，把键值对(key/value)映射成新的键值对(key/value)，形成一系列中间结果形式的key/value 对，然后把它们传给Reduce(规约)函数，把具有相同中间形式key 的value 合并在一起。Map 和Reduce 函数具有一定的关联性。函

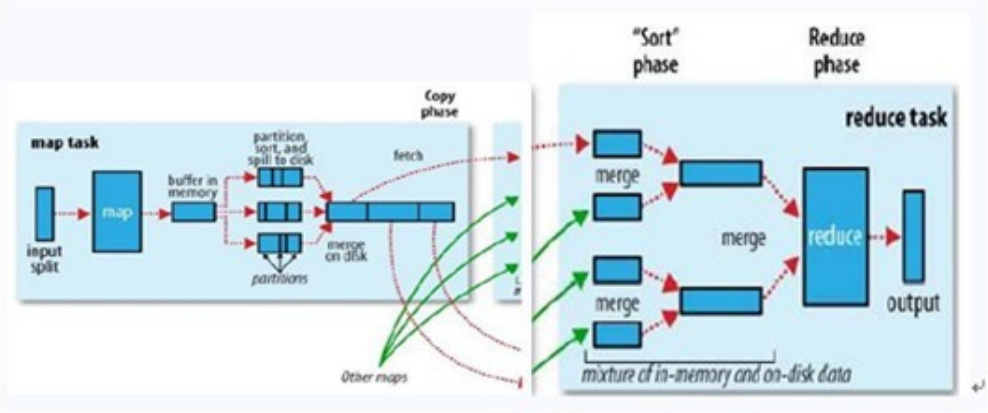
数描述如表1 所示：

表 1　Map 函数和 Reduce 函数的描述

函数	输入	输出	说明
Map	<k1,v1>	List(<k2,v2>)	1.将小数据集进一步解析成一批<key,value>对，输入Map函数中进行处理。 2.每一个输入的<k1,v1>会输出一批<k2,v2>。<k2,v2>是计算的中间结果。
Reduce	<k2,List(v2)>	<k3,v3>	输入的中间结果<k2,List(v2)>中的List(v2)表示是一批属于同一个k2的value

MapReduce致力于解决大规模数据处理的问题，因此在设计之初就考虑了数据的局部性原理，利用局部性原理将整个问题分而治之。MapReduce集群由普通PC机构成，为无共享式架构。在处理之前，将数据集分布至各个节点。处理时，每个节点就近读取本地存储的数据处理（map），将处理后的数据进行合并（combine）、排序（shuffle and sort）后再分发（至reduce节点），避免了大量数据的传输，提高了处理效率。无共享式架构的另一个好处是配合复制（replication）策略，集群可以具有良好的容错性，一部分节点的down机对集群的正常工作不会造成影响。

ok，你可以再简单看看下副图，整幅图是有关hadoop的作业调优参数及原理，图的左边是MapTask运行示意图，右边是ReduceTask运行示意图：



如上图所示，其中map阶段，当map task开始运算，并产生中间数据后并非直接而简单的写入磁盘，它首先利用内存buffer来对已经产生的buffer进行缓存，并在内存buffer中进行一些预排序来优化整个map的性能。而上图右边的reduce阶段则经历了三个阶段，分别Copy->Sort->reduce。我们能明显的看出，其中的Sort是采用的归并排序，即merge sort。

了解了什么是mapreduce，接下来，咱们可以来了解实现了mapreduce模式的开源框架—hadoop。

Hadoop框架

前面说了，hadoop是一个框架，一个什么样的框架呢?Hadoop 是一个实现了MapReduce 计算模型的开源分布式并行编程框架，程序员可以借助Hadoop编写程序，将所编写的程序运行于计算机机群上，从而实现对海量数据的处理。

此外，Hadoop 还提供一个分布式文件系统(HDFS)及分布式数据库(HBase)用来将数据存储或部署到各个计算节点上。所以，你可以大致认为：*Hadoop=HDFS（文件系统，数据存储技术相关）+HBase（数据库）+MapReduce（数据处理）*。Hadoop 框架如图2 所示：

图 2　Hadoop 框架

借助Hadoop 框架及云计算核心技术MapReduce 来实现数据的计算和存储，并且将HDFS 分布式文件系统和HBase 分布式数据库很好的融入到云计算框架中，从而实现云计算的分布式、并行计算和存储，并且得以实现很好的处理大规模数据的能力。

Hadoop的组成部分

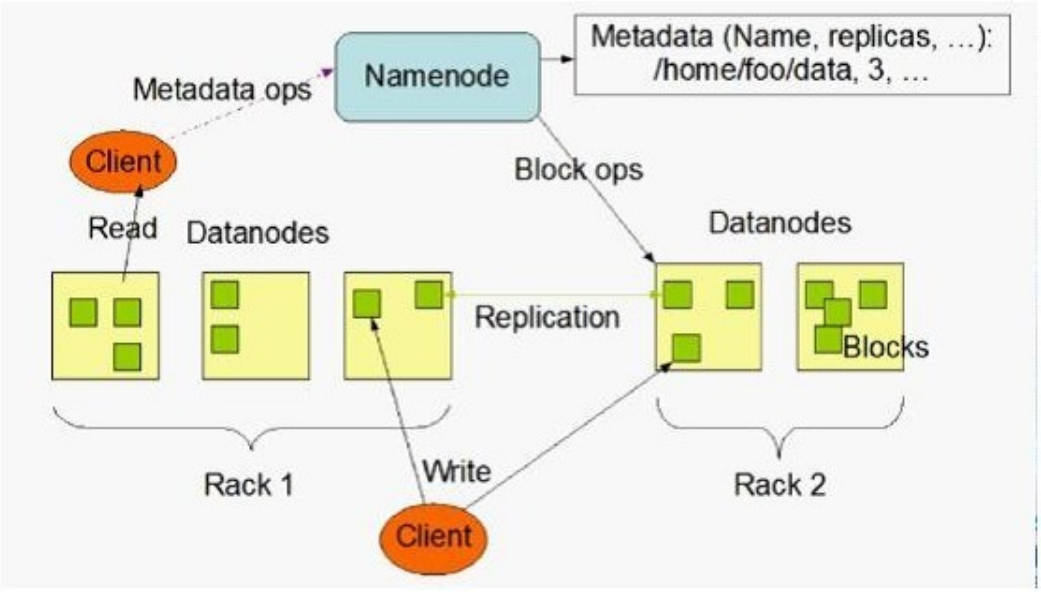
我们已经知道，Hadoop是Google的MapReduce一个Java实现。MapReduce是一种简化的分布式编程模式，让程序自动分布到一个由普通机器组成的超大集群上并发执行。*Hadoop主要由HDFS、MapReduce和HBase*等组成。具体的

hadoop的组成如下图：

由上图，我们可以看到：

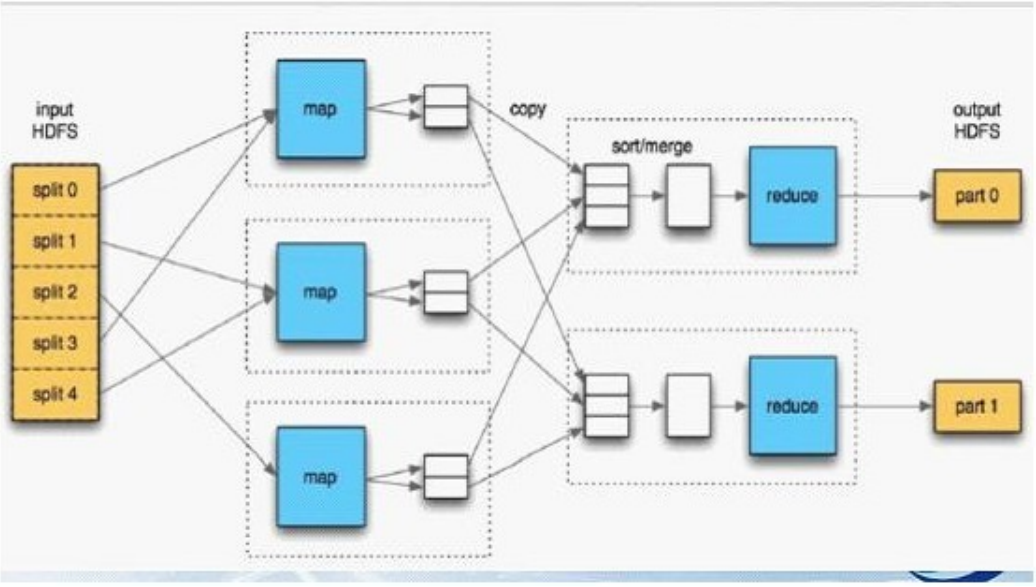
1. **Hadoop HDFS**是Google GFS存储系统的开源实现，主要应用场景是作为并行计算环境（MapReduce）的基础组件，同时也是BigTable（如HBase，HyperTable）的底层分布式文件系统。HDFS采用master/slave架构。一个HDFS集群是有由一个Namenode和一定数目的Datanode组成。Namenode是一个中心服务器，负责管理文件系统的namespace和客户端对文件的访问。Datanode在集群中一般是一个节点一个，负责管理节点上它们附带的存储。在内部，一个文件其实分成一个或多个block，这些

block存储在Datanode集合里。如下图所示（*HDFS体系结构图*）：



2. **Hadoop MapReduce**是一个使用简易的软件框架，基于它写出来的应用程序能够运行在由上千个商用机器组成的大型集群上，并以一种可靠容错的方式并行处理上TB级别的数据集。

一个MapReduce作业（job）通常会把输入的数据集切分为若干独立的数据块，由 Map任务（task）以完全并行的方式处理它们。框架会对Map的输出先进行排序，然后把结果输入给Reduce任务。通常作业的输入和输出都会被存储在文件系统中。整个框架负责任务的调度和监控，以及重新执行已经失败的任务。如下图所示（*Hadoop MapReduce处理流程图*）：



3. **Hive是基于Hadoop的一个数据仓库工具，处理能力强而且成本低廉。**

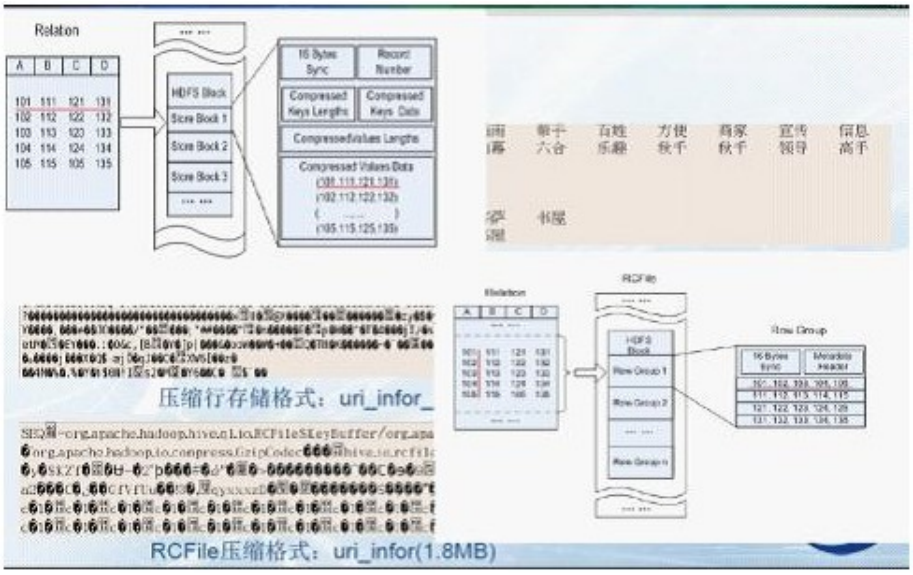
主要特点：

存储方式是将结构化的数据文件映射为一张数据库表。提供类SQL语言，实现完整的SQL查询功能。可以将SQL语句转换为MapReduce任务运行，十分适合数据仓库的统计分析。

不足之处：

采用行存储的方式（SequenceFile）来存储和读取数据。效率低：当要读取数据表某一列数据时需要先取出所有数据然后再提取出某一列的数据，效率很低。同时，它还占用较多的磁盘空间。

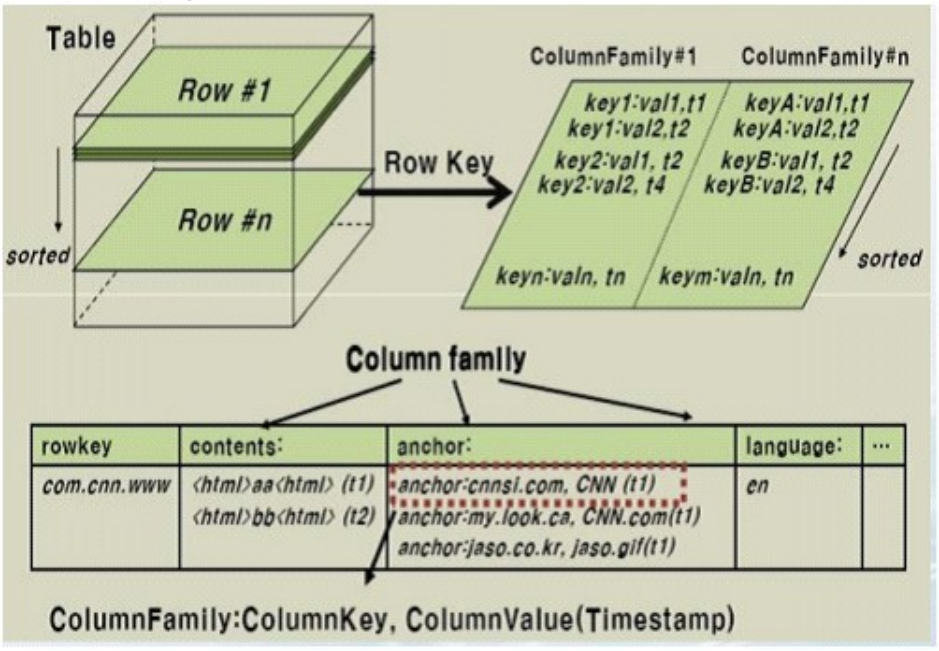
由于以上的不足，有人（查礼博士）介绍了一种将分布式数据处理系统中以记录为单位的存储结构变为以列为单位的存储结构，进而减少磁盘访问数量，提高查询处理性能。这样，由于相同属性值具有相同数据类型和相近的数据特性，以属性值为单位进行压缩存储的压缩比更高，能节省更多的存储空间。如下图所示（*行列存储的比较图*）：



4. **HBase**

HBase是一个分布式的、面向列的开源数据库，它不同于一般的关系数据库，是一个适合于非结构化数据存储的数据库。另一个不同的是HBase基于列的而不是基于行的模式。HBase使用和 BigTable非常相同的数据模型。用户存储数据行在一个表里。一个数据行拥有一个可选择的键和任意数量的列，一个或多个列组成一个ColumnFamily，一个Fmaily下的列

位于一个HFile中，易于缓存数据。表是疏松的存储的，因此用户可以给行定义各种不同的列。在HBase中数据按主键排序，同时表按主键划分为多个HRegion，如下图所示（*HBase 数据表结构图*）：



Ok，行文至此，看似洋洋洒洒近千里，但若给读者造成阅读上的负担，则不是我本意。接下来的内容，我不会再引用诸多繁杂的专业术语，以给读者心里上造成不良影响。

我再给出一副图，算是对上文所说的hadoop框架及其组成部分做个总结，如下图所示，便是hadoop的内部结构，我们可以看到，海量的数据交给hadoop处理后，在hadoop的内部中，正如上文所述：hadoop提供一个分布式文件系统（HDFS）及分布式数据库（Hbase）用来存储或部署到各个计算点上，最终在

内部采取mapreduce的模式对其数据进行处理，然后输出处理结果：

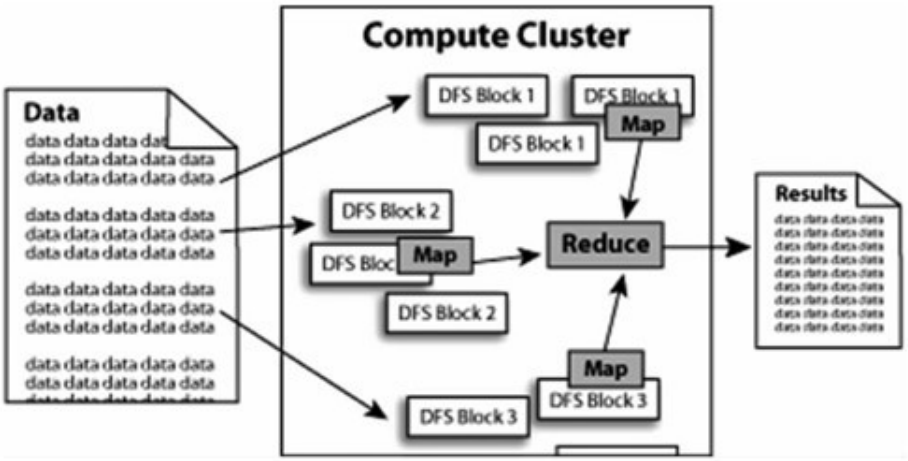


图 1　Hadoop 内部结构图

其它模式/方法论，结合操作系统知识

至此，六种处理海量数据问题的模式/方法已经阐述完毕。据观察，这方面的面试题无外乎以上一种或其变形，然题目为何取为是：秒杀99%的海量数据处理面试题，而不是100%呢。OK，给读者看最后一道题，如下：

非常大的文件，装不进内存。每行一个int类型数据，现在要你随机取100个数。

我们发现上述这道题，无论是以上任何一种模式/方法都不好做，那有什么好的别的方法呢？我们可以借鉴操作系统内存分页系统设计(说白了，就是映射+建索引)。

即针对此题，我们可以做出如下解决方案：

先生成4G的地址表，在把这个表划分为小的4M的小文件做个索引，二级索引。30位前十位表示第几个4M文件，后20位表示在这个4M文件的第几个，等等，基于key value来设计存储，用key来建索引。

但如果现在只有10000个数，然后怎么去随机从这10000个数里面随机取100个数？请读者思考。更多海里数据处理面试题，请参见此文第一部分：。

后记

经过上面这么多海量数据处理面试题的轰炸，我们依然可以看出这类问题是有一定的解决方案/模式的，所以，不必将其神化。然有点，必须负责任的敬告大家：无论是这些海量数据处理面试题也好，还是算法也好，**面试时**，70~80%的人不是倒在这两方面，而是倒在基础之上(诸如语言，数据库，操作系统，网络协议等等)，所以，**无论任何时候，基础最重要**，没了基础，便什么都不是。

sift算法的编译与实现

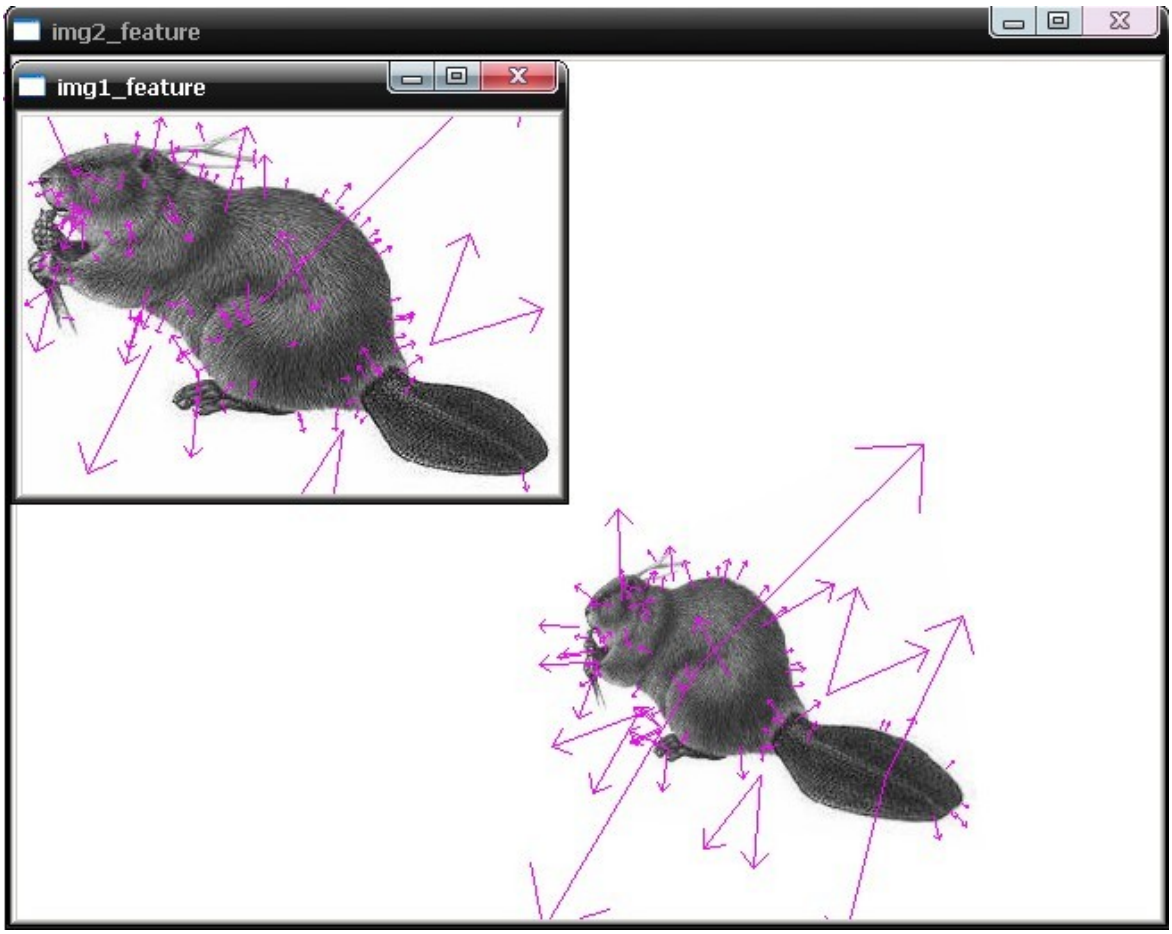
代码：Rob Hess维护的sift 库。

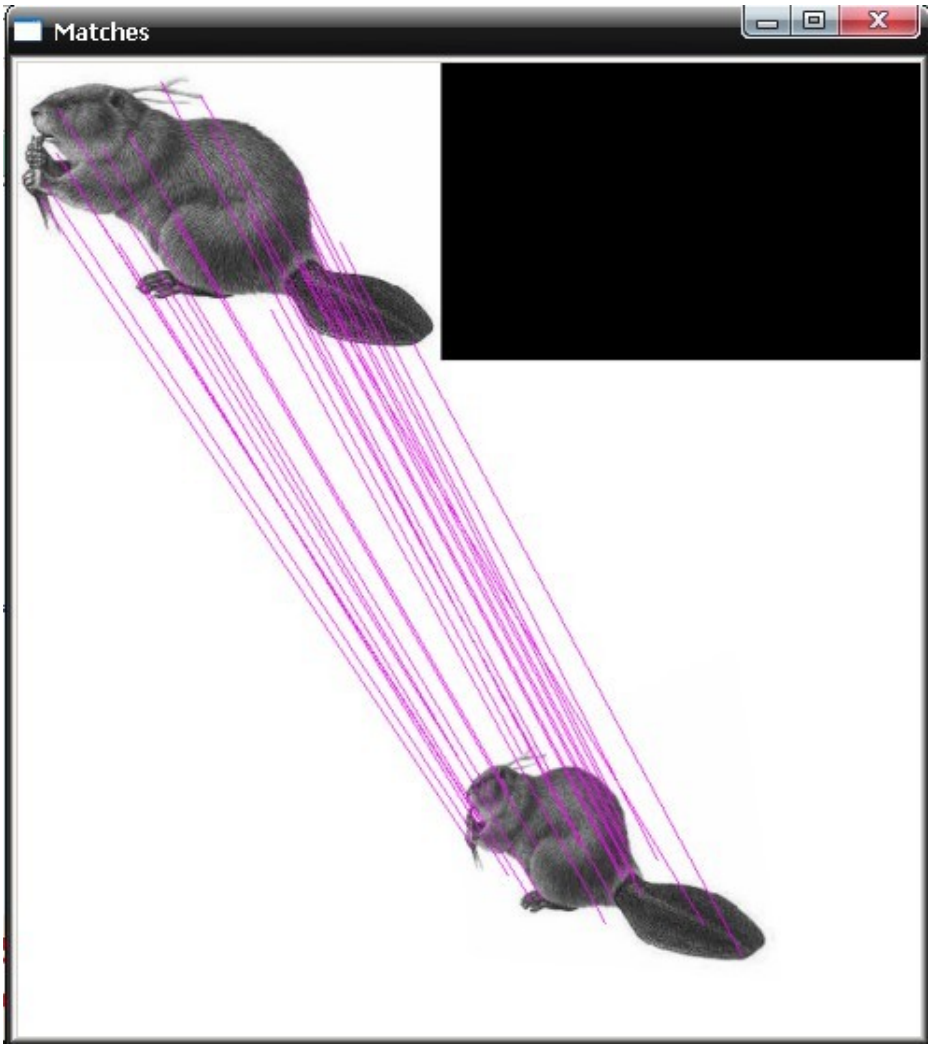
环境：windows xp+vc6.0。

条件：opencv1.0 、gsl-1.8.exe

昨日，下载了Rob Hess的sift库，将其源码粗略的看了看，想要编译时，遇到了不少问题，先修改了下代码，然后下载opencv 、gsl。最后，几经周折，才最终编译成功。

以下便是sift源码库编译后的效果图：


img2_feature
img1_feature



为了给有兴趣实现sift算法的朋友提供个参考，特整理此文如下。要了解什么是sift算法，请参考：九、图像特征提取与匹配之SIFT算法。ok，咱们下面，就来利用Rob Hess维护的sift 库来实现sift算法：

首先，请下载Rob Hess维护的sift 库：
http://blogs.oregonstate.edu/hess/code/sift
下载Rob Hess的这个压缩包后，如果直接解压缩，直接编译，那么会出现下面的错误提示：

编译提示:error C1083: Cannot open include file: 'cxcore.h': No such file or directory,找不到这个头文件。

这个错误，是因为你还没有安装opencv，因为：cxcore.h和cv.h是开源的OPENCV头文件,不是VC++的默认安装文件,所以你还得下载OpenCV并进行安装。然后，可以在OpenCV文件夹下找到你所需要的头文件了。

据网友称，截止2010年4月4日，还没有在VC6.0下成功使用opencv2.0的案例。所以，如果你是VC6.0的用户请下载opencv1.0版本。vs的话，opencv2.0,1.0任意下载。

以下，咱们就以**vc6.0为平台举例，下载并安装opencv1.0版本、gsl**等。当然，你也可以用vs编译，同样下载opencv（具体版本不受限制）、gsl等。

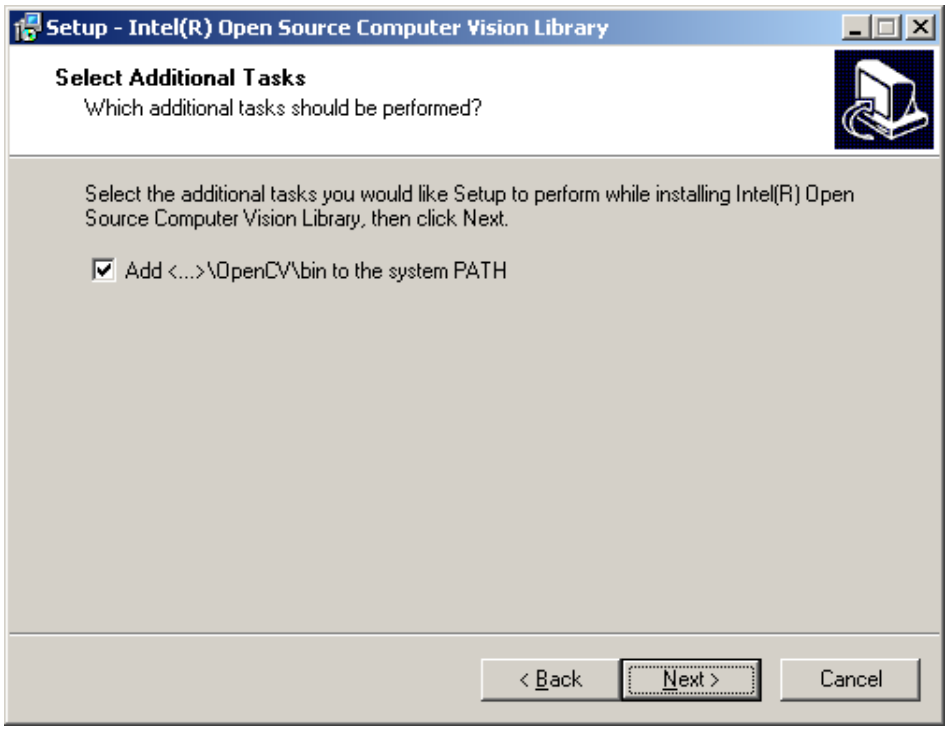
请按以下步骤操作：

一、下载opencv1.0

http://sourceforge.net/projects/opencvlibrary/files/opencv-win/1.0/OpenCV_1.0.exe/download

二、安装opencv1.0，配置Windows环境变量

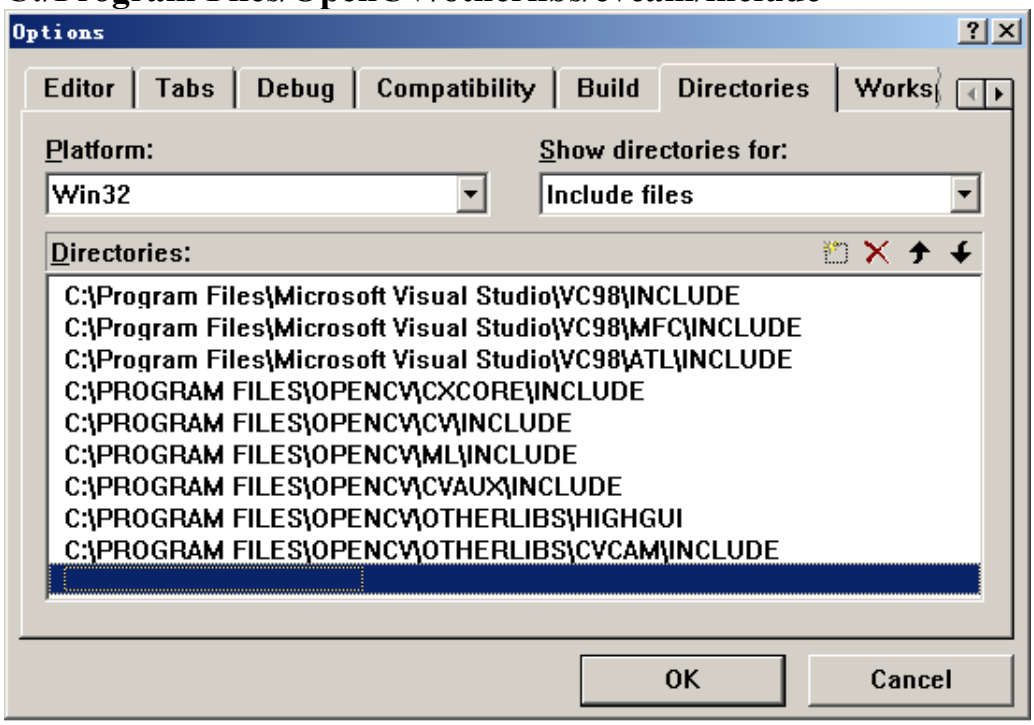
1. **安装注意：**假如你是将OpenCV安装到C:/Program Files/OpenCV（**如果你安装的时候选择不是安装在C盘，则下面所有对应的C盘都改为你所安装在的那个“X盘”，即可**），在安装时选择“将/OpenCV/bin加入系统变量”，打上“勾”。（Add/OpenCV/bin to the systerm PATH。**这一步确认选上了之后，下面的检查环境变量的步骤，便可免去**）



2. **检查环境变量。**为了确保上述步骤中，加入了系统变量，在安装opencv1.0成功后，还得检查C:/Program Files/OpenCV/bin是否已经被加入到环境变量PATH，如果没有，请加入。

3. **最后是配置Visual C++ 6.0。**

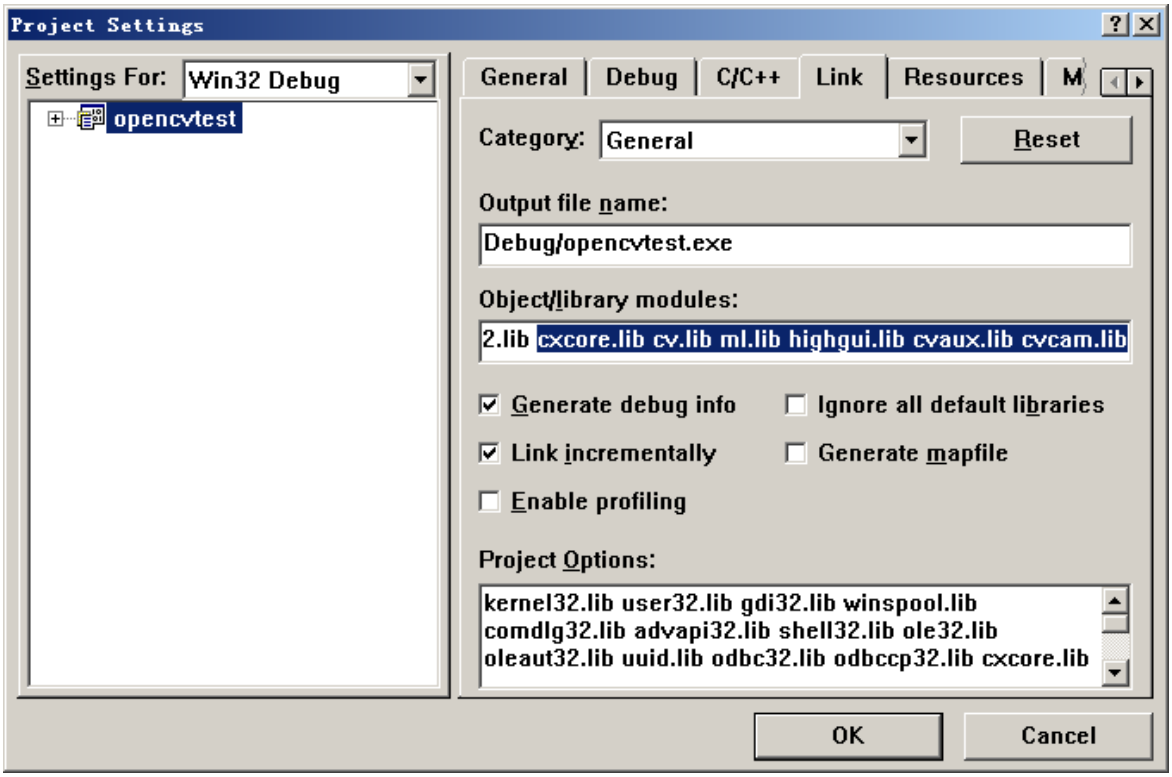
- 全局设置
  菜单Tools->Options->Directories：先设置lib路径，选择Library files，在下方填入路径：
  C:/Program Files/OpenCV/lib
  然后选择include files，在下方填入路径(参考下图)：
  C:/Program Files/OpenCV/cxcore/include
  C:/Program Files/OpenCV/cv/include
  C:/Program Files/OpenCV/cvaux/include
  C:/Program Files/OpenCV/ml/include
  C:/Program Files/OpenCV/otherlibs/highgui
  C:/Program Files/OpenCV/otherlibs/cvcam/include



最后选择source files，在下方填入路径：
C:/Program Files/OpenCV/cv/src
C:/Program Files/OpenCV/cxcore/src
C:/Program Files/OpenCV/cvaux/src

C:/Program Files/OpenCV/otherlibs/highgui
C:/Program Files/OpenCV/otherlibs/cvcam/src/windows

- 项目设置 每创建一个将要使用OpenCV的VC Project，都需要给它指定需要的lib。菜单：Project->Settings，然后将Setting for选为All Configurations，然后选择右边的link标签，在Object/library modules附加上：cxcore.lib cv.lib ml.lib cvaux.lib highgui.lib cvcam.lib 当然，你不需要这么多lib，你可以只添加你需要的lib(见下图)



三、下载gsl，gsl也是一个库，也需要下载：

。在编译时候GSL也是和OpenCV一样要把头文件和lib的路径指定好。

四、配置gsl

将C:/WinGsl/bin中的WinGsl.dll和WinGslD.dll复制到C:/VC6.0/Bin；将整个Gsl目录复制到C:/VC6.0/Bin下；lib目录下的所有.lib文件全部复制到C:/VC6.0/Lib下。
然后，在tools－options－directories中，将C:/WinGsl下的lib，gsl分别加入到库文件和头文件的搜索路径中。

以下是可能会出现的错误情况处理：

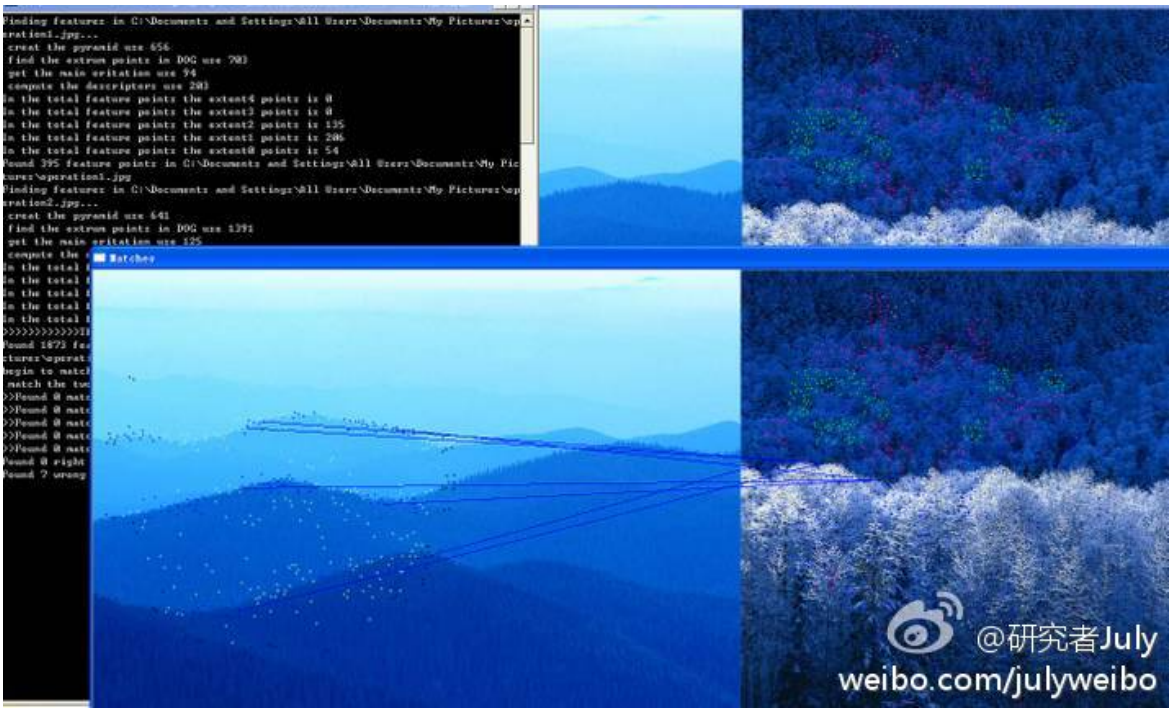
1. OpenCV安装后“没有找到cxcore100.dll”的错误处理 在安装时选择“将/OpenCV/bin加入系统变量”（Add/OpenCV/bin to the systerm PATH）。但该选项并不一定能成功添加到系统变量，如果编写的程序在运行时出现“没有找到cxcore100.dll，因为这个应用程序未能启动。重新安装应用程序可能会修复此问题。”的错误。手动在我的电脑->属性->高级->环境变量->系统变量->path添加c:/program files/opencv/bin;添加完成后需要重启计算机。

2. vc6.0下配置了一下，可是编译程序时遇到如下一个错误：Linking… LINK : fatal error LNK1104: cannot open file“odbccp32.libcxcore.lib” 可能是：在工程设置的时候添加连接库时没加空格或.来把两个文件名（odbccp32.lib cxcore.lib）分开。注意每一次操作后，记得保存。

若经过以上所有的步骤之后，如果还不能正常编译，那就是还要稍微修改下你下载的Rob Hess代码。ok，日后，若有空，再好好详细剖析下此sift的源码。最后，祝你编译顺利。

updated

今天下午试了下sift + KD + BBF，然后用两幅不同的图片做了下匹配（当然，运行结果显示是不匹配的），效果还不错：http://weibo.com/1580904460/yDmzAEwcV#1348475194313!



同时，编译的过程中，直接用的VS2010 + opencv（并没下gsl）。2012.09.24。完。

教你一步一步用c语言实现sift算法、上

参考：Rob Hess维护的sift 库
环境： windows xp+vc6.0
条件：c语言实现。

引言：

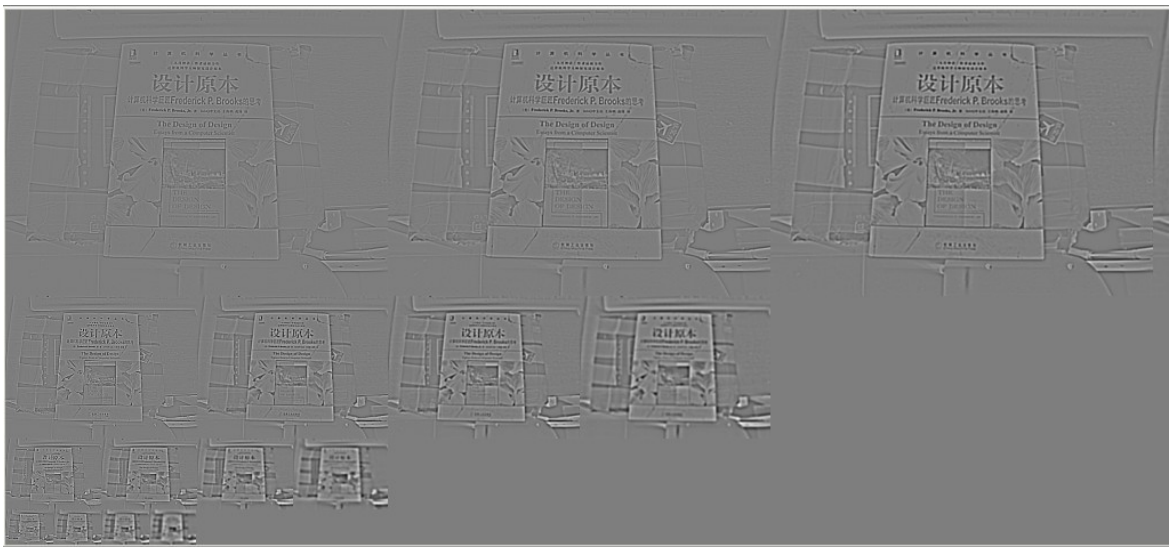
在我写的关于sift算法的前俩篇文章里头，已经对sift算法有了初步的介绍：九、图像特征提取与匹配之SIFT算法，而后在：九（续）、sift算法的编译与实现里，我也简单记录下了如何利用opencv，gsl等库编译运行sift程序。

但据一朋友表示，是否能用c语言实现sift算法，同时，尽量不用到opencv，gsl等第三方库之类的东西。而且，Rob Hess维护的sift 库，也不好懂，有的人根本搞不懂是怎么一回事。

那么本文，就教你如何利用c语言一步一步实现sift算法，同时，你也就能真正明白sift算法到底是怎么一回事了。

ok, 先看一下, 本程序最终运行的效果图, sift 算法分为五个步骤(下文详述), 对应以下**第二~第六幅图**:

```
BuildGaussianOctaves(): Base image dimension is 670x502
BuildGaussianOctaves(): Building 4 octaves
Building octave 0 of dimesion (1340, 1004)
0 scale and blur sigma : 0.707107
1 scale and Blur sigma: 1.000000
2 scale and Blur sigma: 1.414214
3 scale and Blur sigma: 2.000000
4 scale and Blur sigma: 2.828427
Building octave 1 of dimesion (670, 502)
0 scale and blur sigma : 1.414214
1 scale and Blur sigma: 2.000000
2 scale and Blur sigma: 2.828427
3 scale and Blur sigma: 4.000000
4 scale and Blur sigma: 5.656854
Building octave 2 of dimesion (335, 251)
0 scale and blur sigma : 2.828427
1 scale and Blur sigma: 4.000000
2 scale and Blur sigma: 5.656854
3 scale and Blur sigma: 8.000000
4 scale and Blur sigma: 11.313708
Building octave 3 of dimesion (167, 125)
0 scale and blur sigma : 5.656854
1 scale and Blur sigma: 8.000000
2 scale and Blur sigma: 11.313708
3 scale and Blur sigma: 16.000000
4 scale and Blur sigma: 22.627417
the time of build Gaussian pyramid and DOG pyramid is 4005.2
the keypoints number are 747 ;
搜狗拼音 半:
```


设计原本
设计原本
设计原本

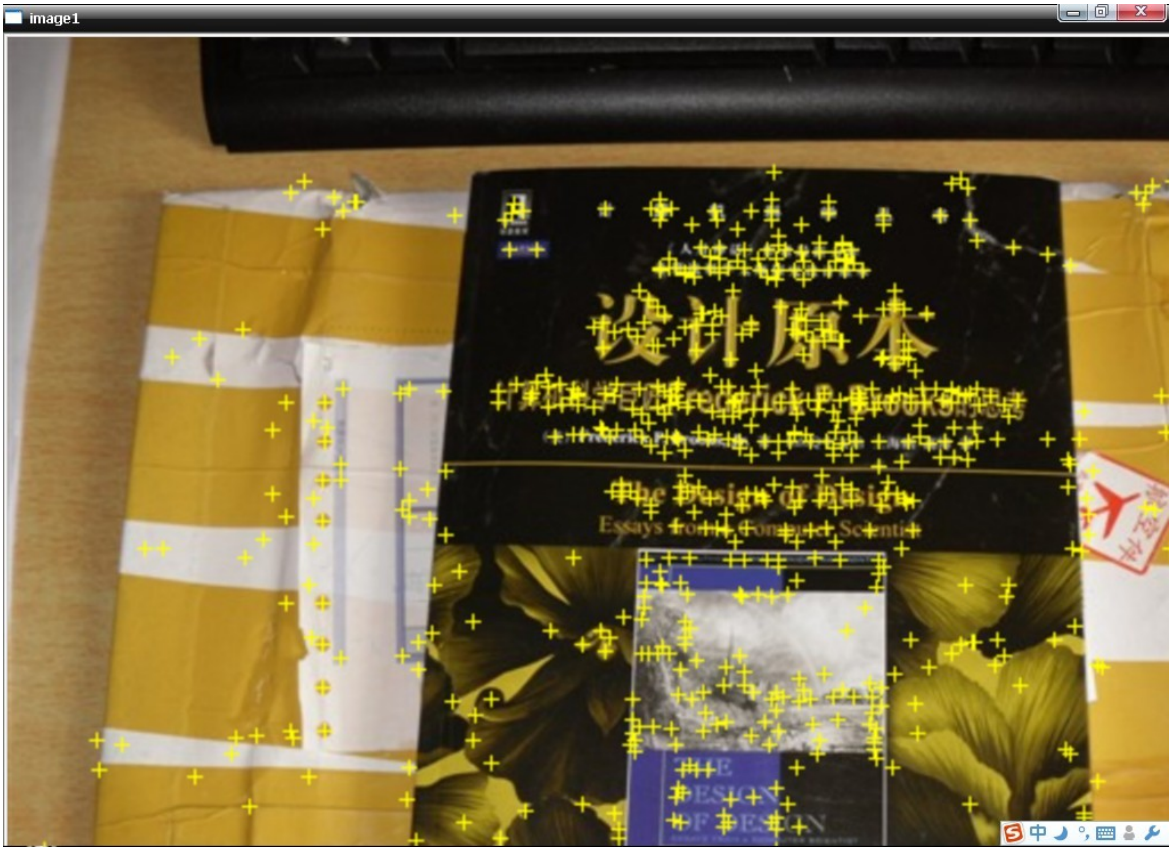


image1
设计原本

image1
设计原本
ClassView
FileView
SIFT-JULY.exe - 0 error(s), 0 warning(s)
组建
调试
在文件1中查找
在文件2中查找
就绪
行 4, 列
读取

sift算法的步骤

要实现一个算法，首先要完全理解这个算法的原理或思想。咱们先来简单了解下，什么叫sift算法：

sift，尺度不变特征转换，是一种电脑视觉的算法用来侦测与描述影像中的局部性特征，它在空间尺度中寻找极值点，并提取出其位置、尺度、旋转不变量，此算法由 David Lowe 在1999年所发表，2004年完善总结。

所谓，Sift算法就是用不同尺度（标准差）的高斯函数对图像进行平滑，然后比较平滑后图像的差别，

差别大的像素就是特征明显的点。

以下是sift算法的五个步骤：

一、建立图像尺度空间(或高斯金字塔)，并检测极值点

首先建立尺度空间，要使得图像具有尺度空间不变形，就要建立尺度空间，sift算法采用了高斯函数来建立尺度空间，高斯函数公式为：

$$G(x,y,\sigma)=\frac{1}{2\pi\sigma^2}e^{-(x^2+y^2)/2\sigma^2}.$$

上述公式G(x,y,e)，即为尺度可变高斯函数。

而，一个图像的尺度空间L（x,y,e) ,定义为原始图像I(x,y)与上述的一个可变尺度的2维高斯函数G(x,y,e) 卷积运算。

即，原始影像I(x，y)在不同的尺度e下，与高斯函数G(x,y,e)进行卷积，得到L(x,y,e)，如下：

$$L(x,y,\sigma)=G(x,y,\sigma)*I(x,y)$$

以上的（x，y）是空间坐标，e，是尺度坐标，或尺度空间因子，e的大小决定平滑程度，大尺度对应图像的概貌特征，小尺度对应图像的细节特征。大的e值对应粗糙尺度(低分辨率)，反之，对应精细尺度(高分辨率)。

尺度，受e这个参数控制的表示。而不同的L（x,y,e）就构成了尺度空间，具体计算的时候，即使连续的高斯函数，都被离散为（一般为奇数大小）(2*k+1) * (2*k+1)矩阵，来和数字图像进行卷积运算。

随着e的变化，建立起不同的尺度空间，或称之为建立起图像的高斯金字塔。

但，像上述L(x,y,e) = G(x,y,e)*I(x,y)的操作，在进行高斯卷积时，整个图像就要遍历所有的像素进行卷积(边界点除外)，于此，就造成了时间和空间上的很大浪费。

为了更有效的在尺度空间检测到稳定的关键点，也为了缩小时间和空间复杂度，对上述的操作作了一个改建：即，提出了高斯差分尺度空间（DOG scale-space）。利用不同尺度的高斯差分与原始图像I(x,y)相乘，卷积生成。

$$\begin{aligned}D(x,y,\sigma) &= (G(x,y,k\sigma)-G(x,y,\sigma))*I(x,y)\\ &= L(x,y,k\sigma)-L(x,y,\sigma).\end{aligned}$$

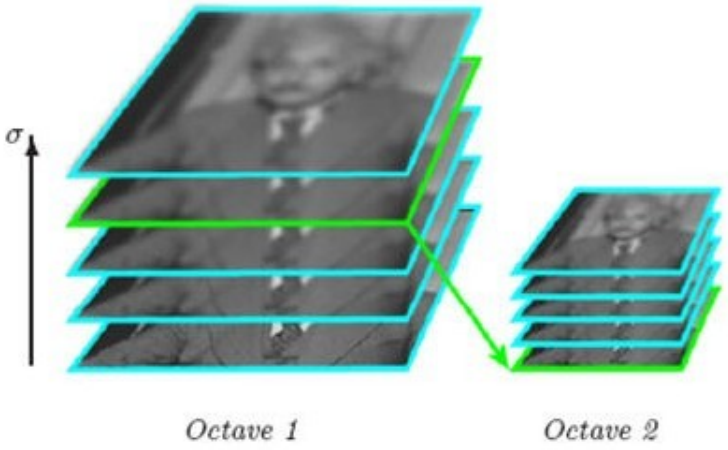
DOG算子计算简单，是尺度归一化的LOG算子的近似。

ok，耐心点，咱们再来总结一下上述内容：

1. 高斯卷积
   在组建一组尺度空间后，再组建下一组尺度空间，对上一组尺度空间的最后一幅图像进行二分之一采样，得到下一组尺度空间的第一幅图像，然后进行像建立第一组尺度空间那样的操作，得到第二组尺度空间，公式定义为

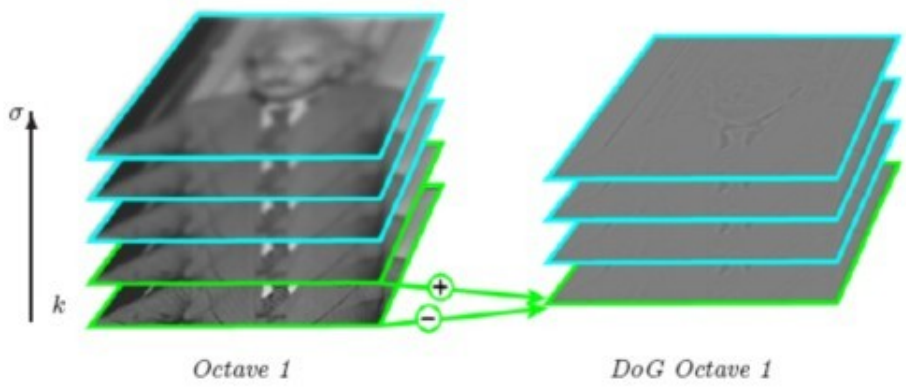
   $L(x,y,e) = G(x,y,e)*I(x,y)$

   图像金字塔的构建：图像金字塔共O组，每组有S层，下一组的图像由上一组图像降采样得到，效果图，**图A**如下(左为上一组，右为下一组)：



2. 高斯差分
   在尺度空间建立完毕后，为了能够找到稳定的关键点，采用高斯差分的方法来检测那些在局部位置的极值点，即采用俩个相邻的尺度中的图像相减，即公式定义为：$D(x,y,e) = ((G(x,y,ke) - G(x,y,e)) * I(x,y) = L(x,y,ke) - L(x,y,e)$

   效果图，图B：



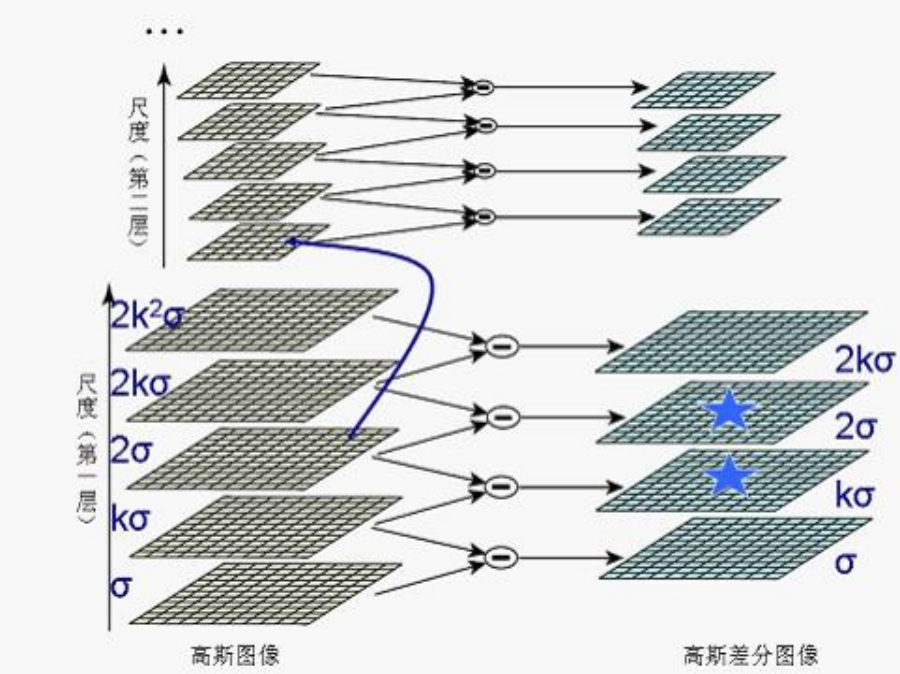
SIFT的精妙之处在于采用图像金字塔的方法解决这一问题，我们可以把两幅图像想象成是连续的，分别以它们作为底面作四棱锥，就像金字塔，那么每一个 截面与原图像相似，那么两个金字塔中必然会有包含大小一致的物体的无穷个截面，但应用只能是离散的，所以我们只能构造有限层，层数越多当然越好，但处理时 间会相应增加，层数太少不行，因为向下采样的截面中可能找不到尺寸大小一致的两个物体的图像。

咱们再来具体阐述下构造D(x,y,e)的详细步骤：

1. 首先采用不同尺度因子的高斯核对图像进行卷积以得到图像的不同尺度空间，将这一组图像作为金子塔图像的第一层。
2. 接着对第一层图像中的2倍尺度图像（相对于该层第一幅图像的2倍尺度）以2倍像素距离进行下采样来得到金子塔图像的第二层中的第一幅图像，对该图像采用不同尺度因子的高斯核进行卷积，以获得金字塔图像中第二层的一组图像。

3. 再以金字塔图像中第二层中的2倍尺度图像（相对于该层第一幅图像的2倍尺度）以2倍像素距离进行下采样来得到金字塔图像的第三层中的第一幅图像，对该图像采用不同尺度因子的高斯核进行卷积，以获得金字塔图像中第三层的一组图像。这样依次类推，从而获得了金字塔图像的每一层中的一组图像，如下图所示：

4. 对上图得到的每一层相邻的高斯图像相减，就得到了高斯差分图像，如下述第一幅图所示。下述第二幅图中的右列显示了将每组中相邻图像相减所生成的高斯差分图像的结果，限于篇幅，图中只给出了第一层和第二层高斯差分图像的计算（下述俩幅图统称为图2）：

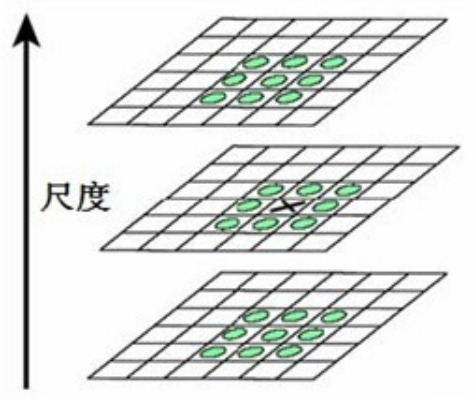


图像金字塔的建立：对于一幅图像I,建立其在不同尺度(scale)的图像，也成为子八度(octave)，这是为了scale-invariant，也就是在任何尺度都能够有对应的特征点，第一个子八度的scale为原图大小，后面每个octave为上一个octave降采样的结果，即原图的1/4(长宽分别减半)，构成下一个子八度(高一层金字塔)。

5. 因为高斯差分函数是归一化的高斯拉普拉斯函数的近似，所以可以从高斯差分金字塔分层结构提取出图像中的极值点作为候选的特征点。对DOG 尺度空间每个点与相邻尺度和相邻位置的点逐个进行比较，得到的局部极值位置即为特征点所处的位置和对应的尺度。

二、检测关键点

为了寻找尺度空间的极值点，每一个采样点要和它所有的相邻点比较，看其是否比它的图像域和尺度域的相邻点大或者小。如下图，图3所示，中间的检测点和它同尺度的8个相邻点和上下相邻尺度对应的9×2个点共26个点比较，以确保在尺度空间和二维图像空间都检测到极值点。



DoG 尺度空间局部极值检测

因为需要同相邻尺度进行比较，所以在一组高斯差分图像中只能检测到两个尺度的极值点（如上述第二幅图中右图的五角星标识），而其它尺度的极值点检测则需要在图像金字塔的上一层高斯差分图像中进行。依次类推，最终在图像金字塔中不同层的高斯差分图像中完成不同尺度极值的检测。

当然这样产生的极值点并不都是稳定的特征点，因为某些极值点响应较弱，而且DOG算子会产生较强的边缘响应。

三、关键点方向的分配

为了使描述符具有旋转不变性，需要利用图像的局部特征为给每一个关键点分配一个方向。利用关键点邻域像素的梯度及方向分布的特性，可以得到梯度模值和方向如下：

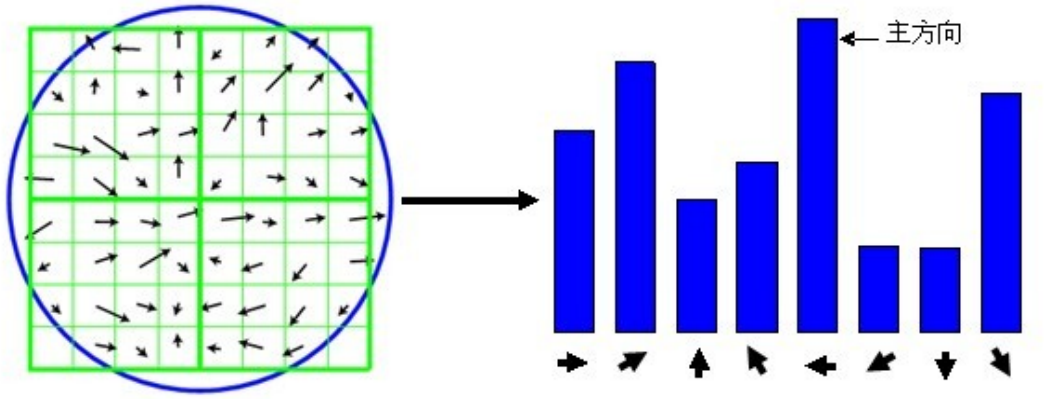
$$m(x,y)=\sqrt{(L(x+1,y)-L(x-1,y))^2+(L(x,y+1)-L(x,y-1))^2}$$
$$\theta(x,y)=\tan^{-1}((L(x,y+1)-L(x,y-1))/L(x+1,y)-L(x-1,y)))$$

其中，尺度为每个关键点各自所在的尺度。

在以关键点为中心的邻域窗口内采样，并用直方图统计邻域像素的梯度方向。梯度直方图的范围是0～360度，其中每10度一个方向，总共36个方向。

直方图的峰值则代表了该关键点处邻域梯度的主方向，即作为该关键点的方向。

在计算方向直方图时，需要用一个参数等于关键点所在尺度1.5倍的高斯权重窗对方向直方图进行加权，上图中用蓝色的圆形表示，中心处的蓝色较重，表示权值最大，边缘处颜色潜，表示权值小。如下图所示，该示例中为了简化给出了8方向的方向直方图计算结果，实际sift创始人David Lowe的原论文中采用36方向的直方图。



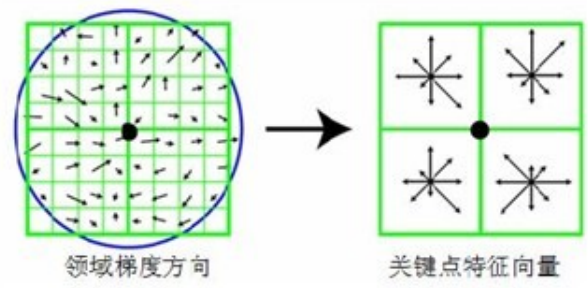
方向直方图的峰值则代表了该特征点处邻域梯度的方向，以直方图中最大值作为该关键点的主方向。为了增强匹配的鲁棒性，只保留峰值大于主方向峰值80%的方向作为该关键点的辅方向。因此，对于同一梯度值的多个峰值的关键点位置，在相同位置和尺度将会有多个关键点被创建但方向不同。仅有15%的关键点被赋予多个方向，但可以明显的提高关键点匹配的稳定性。

至此，图像的关键点已检测完毕，每个关键点有三个信息：位置、所处尺度、方向。由此可以确定一个SIFT特征区域。

四、特征点描述符

通过以上步骤，对于每一个关键点，拥有三个信息：位置、尺度以及方向。接下来就是为每个关键点建立一个描述符，使其不随各种变化而改变，比如光照变化、视角变化等等。并且描述符应该有较高的独特性，以便于提高特征点正确匹配的概率。

首先将坐标轴旋转为关键点的方向，以确保旋转不变性。



接下来以关键点为中心取8×8的窗口。

上图，图5中左部分的中央黑点为当前关键点的位置，每个小格代表关键点邻域所在尺度空间的一个像素，箭头方向代表该像素的梯度方向，箭头长度代表梯度模值，图中蓝色的圈代表高斯加权的范围（越靠近关键点的像素梯度方向信息贡献越大）。

然后在每4×4的小块上计算8个方向的梯度方向直方图，绘制每个梯度方向的累加值，即可形成一个种子点，如图5右部分所示。此图中一个关键点由2×2共4个种子点组成，每个种子点有8个方向向量信息。这种邻域方向性信息联合的思想增强了算法抗噪声的能力，同时对于含有定位误差的特征匹配也提供了较好的容错性。

实际计算过程中，为了增强匹配的稳健性，Lowe建议对每个关键点使用4×4共16个种子点来描述，这样对于一个关键点就可以产生128个数据，即最终形成128维的SIFT特征向量。此时SIFT特征向量已经去除了尺度变化、旋转等几何变形因素的影响，再继续将特征向量的长度归一化，则可以进一步去除光照变化的影响。

五、最后一步

当两幅图像的SIFT特征向量生成后，下一步我们采用关键点特征向量的欧式距离来作为两幅图像中关键点的相似性判定度量。取上图中，**图像A**中的某个关键点，并找出其与**图像B**中欧式距离最近的前两个关键点，在这两个关键点中，如果最近的距离除以次近的距离少于某个比例阈值，则接受这一对匹配点。降低这个比例阈值，SIFT匹配点数目会减少，但更加稳定。关于sift 算法的更多理论介绍请参看此文：。

sift算法的逐步c实现

ok，上文搅了那么多的理论，如果你没有看懂它，咋办列?没关系，下面，咱们来一步一步实现此sift算法，即使你没有看到上述的理论，慢慢的，你也会明白sift算法到底是怎么一回事，sift算法到底是怎么实现的…。

yeah，请看：

前期工作：

在具体编写核心函数之前，得先做几个前期的准备工作：

1. 头文件：

```
#ifdef _CH_
#pragma package <opencv>
#endif

#ifndef _EiC
#include <stdio.h>

#include "stdlib.h"
#include "string.h"
#include "malloc.h"
#include "math.h"
#include <assert.h>
#include <ctype.h>
#include <time.h>
#include <cv.h>
#include <cxcore.h>
#include <highgui.h>
#include <vector>
#endif

#ifdef _EiC
#define WIN32
#endif
```

2. 定义几个宏，及变量，以免下文函数中，突然冒出一个变量，而您却不知道怎么一回事：```c #define NUMSIZE 2
   #define GAUSSKERN 3.5
   #define PI 3.14159265358979323846

   //Sigma of base image – See D.L.’s paper.
   #define INITSIGMA 0.5

//Sigma of each octave – See D.L.’s paper.
#define SIGMA sqrt(3)//1.6//

//Number of scales per octave. See D.L.’s paper.
#define SCALESPEROCTAVE 2
#define MAXOCTAVES 4
int numoctaves;

define CONTRAST_THRESHOLD 0.02

define CURVATURE_THRESHOLD 10.0

define DOUBLE_BASE_IMAGE_SIZE 1

define peakRelThresh 0.8

define LEN 128

// temporary storage
CvMemStorage* storage = 0;
```

3. 然后，咱们还得，声明几个变量，以及建几个数据结构（数据结构是一切程序事物的基础麻，:D。）：

```
//Data structure for a float image.
typedef struct ImageSt {   /*金字塔每一层*/
```
```

```
 float levelsigma;
 int levelsigmalength;
 float absolute_sigma;
 CvMat *Level;        //CvMat是OPENCV的矩阵类,其元素可以是图像的象素值
} ImageLevels;

typedef struct ImageSt1 {       /*金字塔每一阶梯*/
 int row, col;           //Dimensions of image.
 float subsample;
 ImageLevels *Octave;
} ImageOctaves;

ImageOctaves *DOGoctaves;
//DOG pyr,DOG算子计算简单,是尺度归一化的LoG算子的近似。

ImageOctaves *mag_thresh ;
ImageOctaves *mag_pyr ;
ImageOctaves *grad_pyr ;

//keypoint数据结构,Lists of keypoints are linked by the "next" field.
typedef struct KeypointSt
{
 float row, col; /* 反馈回原图像大小,特征点的位置 */
 float sx,sy;    /* 金字塔中特征点的位置*/
 int octave,level;/*金字塔中,特征点所在的阶梯、层次*/

 float scale, ori,mag; /*所在层的尺度sigma,主方向orientation (range [-PI,PI]),以及幅值*/
 float *descrip;       /*特征描述字指针:128维或32维等*/
 struct KeypointSt *next;/* Pointer to next keypoint in list. */
} *Keypoint;

//定义特征点具体变量
Keypoint keypoints=NULL;       //用于临时存储特征点的位置等
Keypoint keyDescriptors=NULL; //用于最后的确定特征点以及特征描述字
```

4. 声明几个图像的基本处理函数:

```
CvMat * halfSizeImage(CvMat * im);      //缩小图像:下采样
CvMat * doubleSizeImage(CvMat * im);    //扩大图像:最近临方法
CvMat * doubleSizeImage2(CvMat * im);   //扩大图像:线性插值
float getPixelBI(CvMat * im, float col, float row);//双线性插值函数
void normalizeVec(float* vec, int dim);//向量归一化
CvMat* GaussianKernel2D(float sigma);   //得到2维高斯核
void normalizeMat(CvMat* mat) ;         //矩阵归一化
float* GaussianKernel1D(float sigma, int dim) ; //得到1维高斯核

//在具体像素处宽度方向进行高斯卷积
float ConvolveLocWidth(float* kernel, int dim, CvMat * src, int x, int y) ;
//在整个图像宽度方向进行1D高斯卷积
```

```
void Convolve1DWidth(float* kern, int dim, CvMat * src, CvMat * dst) ;
//在具体像素处高度方向进行高斯卷积
float ConvolveLocHeight(float* kernel, int dim, CvMat * src, int x, int y) ;
//在整个图像高度方向进行1D高斯卷积
void Convolve1DHeight(float* kern, int dim, CvMat * src, CvMat * dst);
//用高斯函数模糊图像
int BlurImage(CvMat * src, CvMat * dst, float sigma) ;
```

算法核心

本程序中，sift算法被分为以下五个步骤及其相对应的函数（可能表述与上，或与前俩篇文章有所偏差，但都一个意思）：

```
//SIFT算法第一步:图像预处理
CvMat *ScaleInitImage(CvMat * im) ;                    //金字塔初始化

//SIFT算法第二步:建立高斯金字塔函数
ImageOctaves* BuildGaussianOctaves(CvMat * image) ;  //建立高斯金字塔

//SIFT算法第三步:特征点位置检测,最后确定特征点的位置
int DetectKeypoint(int numoctaves, ImageOctaves *GaussianPyr);
void DisplayKeypointLocation(IplImage* image, ImageOctaves *GaussianPyr);

//SIFT算法第四步:计算高斯图像的梯度方向和幅值,计算各个特征点的主方向
void ComputeGrad_DirecandMag(int numoctaves, ImageOctaves *GaussianPyr);

int FindClosestRotationBin (int binCount, float angle);  //进行方向直方图统计
void AverageWeakBins (double* bins, int binCount);        //对方向直方图滤波
//确定真正的主方向
bool InterpolateOrientation (double left, double middle,double right, double
*degreeCorrection, double *peakValue);
//确定各个特征点处的主方向函数
void AssignTheMainOrientation(int numoctaves, ImageOctaves *GaussianPyr,ImageOctaves
*mag_pyr,ImageOctaves *grad_pyr);
//显示主方向
void DisplayOrientation (IplImage* image, ImageOctaves *GaussianPyr);

//SIFT算法第五步:抽取各个特征点处的特征描述字
void ExtractFeatureDescriptors(int numoctaves, ImageOctaves *GaussianPyr);

//为了显示图象金字塔，而作的图像水平、垂直拼接
CvMat* MosaicHorizen( CvMat* im1, CvMat* im2 );
CvMat* MosaicVertical( CvMat* im1, CvMat* im2 );

//特征描述点,网格
#define GridSpacing 4
```

主体实现

ok，以上所有的工作都就绪以后，那么接下来，咱们就先来编写main函数，因为你一看主函数之后，你就立马能发现sift算法的工作流程及其原理了。

（主函数中涉及到的函数，下一篇文章：一、教你一步一步用c语言实现sift算法、下，咱们自会一个一个编写）：

```
int main( void )
{
 //声明当前帧IplImage指针
 IplImage* src = NULL;
 IplImage* image1 = NULL;
 IplImage* grey_im1 = NULL;
 IplImage* DoubleSizeImage = NULL;

 IplImage* mosaic1 = NULL;
 IplImage* mosaic2 = NULL;

 CvMat* mosaicHorizen1 = NULL;
 CvMat* mosaicHorizen2 = NULL;
 CvMat* mosaicVertical1 = NULL;

 CvMat* image1Mat = NULL;
 CvMat* tempMat=NULL;

 ImageOctaves *Gaussianpyr;
 int rows,cols;

#define  Im1Mat(ROW,COL)  ((float  *)(image1Mat->data.fl  +  image1Mat->step/sizeof(float)  *
(ROW)))[(COL)]

 //灰度图象像素的数据结构
#define Im1B(ROW,COL) ((uchar*)(image1->imageData + image1->widthStep*(ROW)))[(COL)*3]
#define Im1G(ROW,COL) ((uchar*)(image1->imageData + image1->widthStep*(ROW)))[(COL)*3+1]
#define Im1R(ROW,COL) ((uchar*)(image1->imageData + image1->widthStep*(ROW)))[(COL)*3+2]

 storage = cvCreateMemStorage(0);

 //读取图片
 if( (src = cvLoadImage( "street1.jpg", 1)) == 0 )  // test1.jpg einstein.pgm back1.bmp
  return -1;

 //为图像分配内存
 image1 = cvCreateImage(cvSize(src->width, src->height),  IPL_DEPTH_8U,3);
 grey_im1 = cvCreateImage(cvSize(src->width, src->height),  IPL_DEPTH_8U,1);
     DoubleSizeImage      =      cvCreateImage(cvSize(2*(src->width),      2*(src->height)),
IPL_DEPTH_8U,3);

 //为图像阵列分配内存，假设两幅图像的大小相同，tempMat跟随image1的大小
 image1Mat = cvCreateMat(src->height, src->width, CV_32FC1);
```

```
 //转化成单通道图像再处理
 cvCvtColor(src, grey_im1, CV_BGR2GRAY);
 //转换进入Mat数据结构,图像操作使用的是浮点型操作
 cvConvert(grey_im1, image1Mat);

 double t = (double)cvGetTickCount();
 //图像归一化
 cvConvertScale( image1Mat, image1Mat, 1.0/255, 0 );

 int dim = min(image1Mat->rows, image1Mat->cols);
 numoctaves = (int) (log((double) dim) / log(2.0)) - 2;     //金字塔阶数
 numoctaves = min(numoctaves, MAXOCTAVES);

 //SIFT算法第一步,预滤波除噪声,建立金字塔底层
 tempMat = ScaleInitImage(image1Mat) ;
 //SIFT算法第二步,建立Guassian金字塔和DOG金字塔
 Gaussianpyr = BuildGaussianOctaves(tempMat) ;

 t = (double)cvGetTickCount() - t;
   printf(  "the  time  of  build  Gaussian  pyramid  and  DOG  pyramid  is  %.1f/n",
t/(cvGetTickFrequency()*1000.) );

#define               ImLevels(OCTAVE,LEVEL,ROW,COL)               ((float               *)
(Gaussianpyr[(OCTAVE)].Octave[(LEVEL)].Level->data.fl                                    +
Gaussianpyr[(OCTAVE)].Octave[(LEVEL)].Level->step/sizeof(float) *(ROW)))[(COL)]
 //显示高斯金字塔
 for (int i=0; i<numoctaves;i++)
 {
  if (i==0)
  {
   mosaicHorizen1=MosaicHorizen( (Gaussianpyr[0].Octave)[0].Level, (Gaussianpyr[0].Octave)
[1].Level );
   for (int j=2;j<SCALESPEROCTAVE+3;j++)
    mosaicHorizen1=MosaicHorizen( mosaicHorizen1, (Gaussianpyr[0].Octave)[j].Level );
   for ( j=0;j<NUMSIZE;j++)
    mosaicHorizen1=halfSizeImage(mosaicHorizen1);
  }
  else if (i==1)
  {
   mosaicHorizen2=MosaicHorizen( (Gaussianpyr[1].Octave)[0].Level, (Gaussianpyr[1].Octave)
[1].Level );
   for (int j=2;j<SCALESPEROCTAVE+3;j++)
    mosaicHorizen2=MosaicHorizen( mosaicHorizen2, (Gaussianpyr[1].Octave)[j].Level );
   for ( j=0;j<NUMSIZE;j++)
    mosaicHorizen2=halfSizeImage(mosaicHorizen2);
   mosaicVertical1=MosaicVertical( mosaicHorizen1, mosaicHorizen2 );
  }
  else
  {
   mosaicHorizen1=MosaicHorizen( (Gaussianpyr[i].Octave)[0].Level, (Gaussianpyr[i].Octave)
[1].Level );
```

```
   for (int j=2;j<SCALESPEROCTAVE+3;j++)
    mosaicHorizen1=MosaicHorizen( mosaicHorizen1, (Gaussianpyr[i].Octave)[j].Level );
   for ( j=0;j<NUMSIZE;j++)
    mosaicHorizen1=halfSizeImage(mosaicHorizen1);
   mosaicVertical1=MosaicVertical( mosaicVertical1, mosaicHorizen1 );
  }
 }
    mosaic1    =    cvCreateImage(cvSize(mosaicVertical1->width,    mosaicVertical1->height),
IPL_DEPTH_8U,1);
 cvConvertScale( mosaicVertical1, mosaicVertical1, 255.0, 0 );
 cvConvertScaleAbs( mosaicVertical1, mosaic1, 1, 0 );

 //  cvSaveImage("GaussianPyramid of me.jpg",mosaic1);
 cvNamedWindow("mosaic1",1);
 cvShowImage("mosaic1", mosaic1);
 cvWaitKey(0);
 cvDestroyWindow("mosaic1");
 //显示DOG金字塔
 for ( i=0; i<numoctaves;i++)
 {
  if (i==0)
  {
     mosaicHorizen1=MosaicHorizen(  (DOGoctaves[0].Octave)[0].Level,  (DOGoctaves[0].Octave)
[1].Level );
   for (int j=2;j<SCALESPEROCTAVE+2;j++)
    mosaicHorizen1=MosaicHorizen( mosaicHorizen1, (DOGoctaves[0].Octave)[j].Level );
   for ( j=0;j<NUMSIZE;j++)
    mosaicHorizen1=halfSizeImage(mosaicHorizen1);
  }
  else if (i==1)
  {
     mosaicHorizen2=MosaicHorizen(  (DOGoctaves[1].Octave)[0].Level,  (DOGoctaves[1].Octave)
[1].Level );
   for (int j=2;j<SCALESPEROCTAVE+2;j++)
    mosaicHorizen2=MosaicHorizen( mosaicHorizen2, (DOGoctaves[1].Octave)[j].Level );
   for ( j=0;j<NUMSIZE;j++)
    mosaicHorizen2=halfSizeImage(mosaicHorizen2);
   mosaicVertical1=MosaicVertical( mosaicHorizen1, mosaicHorizen2 );
  }
  else
  {
     mosaicHorizen1=MosaicHorizen(  (DOGoctaves[i].Octave)[0].Level,  (DOGoctaves[i].Octave)
[1].Level );
   for (int j=2;j<SCALESPEROCTAVE+2;j++)
    mosaicHorizen1=MosaicHorizen( mosaicHorizen1, (DOGoctaves[i].Octave)[j].Level );
   for ( j=0;j<NUMSIZE;j++)
    mosaicHorizen1=halfSizeImage(mosaicHorizen1);
   mosaicVertical1=MosaicVertical( mosaicVertical1, mosaicHorizen1 );
  }
 }
 //考虑到DOG金字塔各层图像都会有正负,所以,必须寻找最负的,以将所有图像抬高一个台阶去显示
```

```
 double min_val=0;
 double max_val=0;
 cvMinMaxLoc( mosaicVertical1, &min_val, &max_val,NULL, NULL, NULL );
 if ( min_val<0.0 )
  cvAddS( mosaicVertical1, cvScalarAll( (-1.0)*min_val ), mosaicVertical1, NULL );
   mosaic2    =    cvCreateImage(cvSize(mosaicVertical1->width,    mosaicVertical1->height),
IPL_DEPTH_8U,1);
 cvConvertScale( mosaicVertical1, mosaicVertical1, 255.0/(max_val-min_val), 0 );
 cvConvertScaleAbs( mosaicVertical1, mosaic2, 1, 0 );

 //  cvSaveImage("DOGPyramid of me.jpg",mosaic2);
 cvNamedWindow("mosaic1",1);
 cvShowImage("mosaic1", mosaic2);
 cvWaitKey(0);

 //SIFT算法第三步:特征点位置检测,最后确定特征点的位置
 int keycount=DetectKeypoint(numoctaves, Gaussianpyr);
 printf("the keypoints number are %d ;/n", keycount);
 cvCopy(src,image1,NULL);
 DisplayKeypointLocation( image1 ,Gaussianpyr);

 cvPyrUp( image1, DoubleSizeImage, CV_GAUSSIAN_5x5 );
 cvNamedWindow("image1",1);
 cvShowImage("image1", DoubleSizeImage);
 cvWaitKey(0);
 cvDestroyWindow("image1");

 //SIFT算法第四步:计算高斯图像的梯度方向和幅值,计算各个特征点的主方向
 ComputeGrad_DirecandMag(numoctaves, Gaussianpyr);
 AssignTheMainOrientation( numoctaves, Gaussianpyr,mag_pyr,grad_pyr);
 cvCopy(src,image1,NULL);
 DisplayOrientation ( image1, Gaussianpyr);

 //  cvPyrUp( image1, DoubleSizeImage, CV_GAUSSIAN_5x5 );
 cvNamedWindow("image1",1);
 //  cvResizeWindow("image1", 2*(image1->width), 2*(image1->height) );
 cvShowImage("image1", image1);
 cvWaitKey(0);

 //SIFT算法第五步:抽取各个特征点处的特征描述字
 ExtractFeatureDescriptors( numoctaves, Gaussianpyr);
 cvWaitKey(0);

 //销毁窗口
 cvDestroyWindow("image1");
 cvDestroyWindow("mosaic1");
 //释放图像
 cvReleaseImage(&image1);
 cvReleaseImage(&grey_im1);
 cvReleaseImage(&mosaic1);
 cvReleaseImage(&mosaic2);
```

```
 return 0;
}
```

更多见下文：一、教你一步一步用c语言实现sift算法、下。本文完。

教你一步一步用c语言实现sift算法、下

本文接上，教你一步一步用c语言实现sift算法、上而来：

函数编写

ok，接上文，咱们一个一个的来编写main函数中所涉及到所有函数，这也是本文的关键部分：

```
//下采样原来的图像，返回缩小2倍尺寸的图像
CvMat * halfSizeImage(CvMat * im)
{
    unsigned int i,j;
    int w = im->cols/2;
    int h = im->rows/2;
    CvMat *imnew = cvCreateMat(h, w, CV_32FC1);

#define Im(ROW,COL) ((float *)(im->data.fl + im->step/sizeof(float) *(ROW)))[(COL)]
#define Imnew(ROW,COL) ((float *)(imnew->data.fl + imnew->step/sizeof(float) *(ROW)))
[(COL)]
    for ( j = 0; j < h; j++)
        for ( i = 0; i < w; i++)
            Imnew(j,i)=Im(j*2, i*2);
    return imnew;
}

//上采样原来的图像，返回放大2倍尺寸的图像
CvMat * doubleSizeImage(CvMat * im)
{
    unsigned int i,j;
    int w = im->cols*2;
    int h = im->rows*2;
    CvMat *imnew = cvCreateMat(h, w, CV_32FC1);

#define Im(ROW,COL) ((float *)(im->data.fl + im->step/sizeof(float) *(ROW)))[(COL)]
#define Imnew(ROW,COL) ((float *)(imnew->data.fl + imnew->step/sizeof(float) *(ROW)))
[(COL)]

    for ( j = 0; j < h; j++)
        for ( i = 0; i < w; i++)
```

```
            Imnew(j,i)=Im(j/2, i/2);

    return imnew;
}

//上采样原来的图像,返回放大2倍尺寸的线性插值图像
CvMat * doubleSizeImage2(CvMat * im)
{
    unsigned int i,j;
    int w = im->cols*2;
    int h = im->rows*2;
    CvMat *imnew = cvCreateMat(h, w, CV_32FC1);

#define Im(ROW,COL) ((float *)(im->data.fl + im->step/sizeof(float) *(ROW)))[(COL)]
#define Imnew(ROW,COL) ((float *)(imnew->data.fl + imnew->step/sizeof(float) *(ROW)))
[(COL)]

    // fill every pixel so we don't have to worry about skipping pixels later
    for ( j = 0; j < h; j++)
    {
        for ( i = 0; i < w; i++)
        {
            Imnew(j,i)=Im(j/2, i/2);
        }
    }
    /*
    A B C
    E F G
    H I J
    pixels A C H J are pixels from original image
    pixels B E G I F are interpolated pixels
    */
    // interpolate pixels B and I
    for ( j = 0; j < h; j += 2)
        for ( i = 1; i < w - 1; i += 2)
            Imnew(j,i)=0.5*(Im(j/2, i/2)+Im(j/2, i/2+1));
    // interpolate pixels E and G
    for ( j = 1; j < h - 1; j += 2)
        for ( i = 0; i < w; i += 2)
            Imnew(j,i)=0.5*(Im(j/2, i/2)+Im(j/2+1, i/2));
    // interpolate pixel F
    for ( j = 1; j < h - 1; j += 2)
        for ( i = 1; i < w - 1; i += 2)
                Imnew(j,i)=0.25*(Im(j/2, i/2)+Im(j/2+1, i/2)+Im(j/2, i/2+1)+Im(j/2+1,
i/2+1));
    return imnew;
}
```

```
//双线性插值,返回像素间的灰度值
float getPixelBI(CvMat * im, float col, float row)
{
    int irow, icol;
    float rfrac, cfrac;
    float row1 = 0, row2 = 0;
    int width=im->cols;
    int height=im->rows;
#define  ImMat(ROW,COL)  ((float  *)(im->data.fl  +  im->step/sizeof(float)  *(ROW)))
[(COL)]

    irow = (int) row;
    icol = (int) col;

    if (irow < 0 || irow >= height
        || icol < 0 || icol >= width)
        return 0;
    if (row > height - 1)
        row = height - 1;
    if (col > width - 1)
        col = width - 1;
    rfrac = 1.0 - (row - (float) irow);
    cfrac = 1.0 - (col - (float) icol);
    if (cfrac < 1)
    {
        row1 = cfrac * ImMat(irow,icol) + (1.0 - cfrac) * ImMat(irow,icol+1);
    }
    else
    {
        row1 = ImMat(irow,icol);
    }
    if (rfrac < 1)
    {
        if (cfrac < 1)
        {
            row2 = cfrac * ImMat(irow+1,icol) + (1.0 - cfrac) * ImMat(irow+1,icol+1);
        } else
        {
            row2 = ImMat(irow+1,icol);
        }
    }
    return rfrac * row1 + (1.0 - rfrac) * row2;
}

//矩阵归一化
void normalizeMat(CvMat* mat)
{
#define  Mat(ROW,COL)  ((float  *)(mat->data.fl  +  mat->step/sizeof(float)  *(ROW)))
```

```
[(COL)]
    float sum = 0;

    for (unsigned int j = 0; j < mat->rows; j++)
        for (unsigned int i = 0; i < mat->cols; i++)
            sum += Mat(j,i);
    for ( j = 0; j < mat->rows; j++)
        for (unsigned int i = 0; i < mat->rows; i++)
            Mat(j,i) /= sum;
}

//向量归一化
void normalizeVec(float* vec, int dim)
{
    unsigned int i;
    float sum = 0;
    for ( i = 0; i < dim; i++)
        sum += vec[i];
    for ( i = 0; i < dim; i++)
        vec[i] /= sum;
}

//得到向量的欧式长度,2-范数
float GetVecNorm( float* vec, int dim )
{
    float sum=0.0;
    for (unsigned int i=0;i<dim;i++)
        sum+=vec[i]*vec[i];
    return sqrt(sum);
}

//产生1D高斯核
float* GaussianKernel1D(float sigma, int dim)
{

    unsigned int i;
       //printf("GaussianKernel1D():  Creating  1x%d  vector  for  sigma=%.3f  gaussian
kernel/n", dim, sigma);

    float *kern=(float*)malloc( dim*sizeof(float) );
    float s2 = sigma * sigma;
    int c = dim / 2;
    float m= 1.0/(sqrt(2.0 * CV_PI) * sigma);
    double v;
    for ( i = 0; i < (dim + 1) / 2; i++)
    {
        v = m * exp(-(1.0*i*i)/(2.0 * s2)) ;
        kern[c+i] = v;
```

```
        kern[c-i] = v;
    }
    //   normalizeVec(kern, dim);
    // for ( i = 0; i < dim; i++)
    //  printf("%f  ", kern[i]);
    //  printf("/n");
    return kern;
}

//产生2D高斯核矩阵
CvMat* GaussianKernel2D(float sigma)
{
    // int dim = (int) max(3.0f, GAUSSKERN * sigma);
    int dim = (int) max(3.0f, 2.0 * GAUSSKERN *sigma + 1.0f);
    // make dim odd
    if (dim % 2 == 0)
        dim++;
      //printf("GaussianKernel(): Creating %dx%d matrix for sigma=%.3f gaussian/n",
dim, dim, sigma);
    CvMat* mat=cvCreateMat(dim, dim, CV_32FC1);
#define  Mat(ROW,COL)  ((float  *)(mat->data.fl  +  mat->step/sizeof(float)  *(ROW)))
[(COL)]
    float s2 = sigma * sigma;
    int c = dim / 2;
    //printf("%d %d/n", mat.size(), mat[0].size());
    float m= 1.0/(sqrt(2.0 * CV_PI) * sigma);
    for (int i = 0; i < (dim + 1) / 2; i++)
    {
        for (int j = 0; j < (dim + 1) / 2; j++)
        {
            //printf("%d %d %d/n", c, i, j);
            float v = m * exp(-(1.0*i*i + 1.0*j*j) / (2.0 * s2));
            Mat(c+i,c+j) =v;
            Mat(c-i,c+j) =v;
            Mat(c+i,c-j) =v;
            Mat(c-i,c-j) =v;
        }
    }
    // normalizeMat(mat);
    return mat;
}

//x方向像素处作卷积
float ConvolveLocWidth(float* kernel, int dim, CvMat * src, int x, int y)
{
#define  Src(ROW,COL)  ((float  *)(src->data.fl  +  src->step/sizeof(float)  *(ROW)))
[(COL)]
    unsigned int i;
```

```
    float pixel = 0;
    int col;
    int cen = dim / 2;
    //printf("ConvolveLoc(): Applying convoluation at location (%d, %d)/n", x, y);
    for ( i = 0; i < dim; i++)
    {
        col = x + (i - cen);
        if (col < 0)
            col = 0;
        if (col >= src->cols)
            col = src->cols - 1;
        pixel += kernel[i] * Src(y,col);
    }
    if (pixel > 1)
        pixel = 1;
    return pixel;
}

//x方向作卷积
void Convolve1DWidth(float* kern, int dim, CvMat * src, CvMat * dst)
{
#define  DST(ROW,COL)  ((float  *)(dst->data.fl  +  dst->step/sizeof(float)  *(ROW)))
[(COL)]
    unsigned int i,j;

    for ( j = 0; j < src->rows; j++)
    {
        for ( i = 0; i < src->cols; i++)
        {
            //printf("%d, %d/n", i, j);
            DST(j,i) = ConvolveLocWidth(kern, dim, src, i, j);
        }
    }
}

//y方向像素处作卷积
float ConvolveLocHeight(float* kernel, int dim, CvMat * src, int x, int y)
{
#define  Src(ROW,COL)  ((float  *)(src->data.fl  +  src->step/sizeof(float)  *(ROW)))
[(COL)]
    unsigned int j;
    float pixel = 0;
    int cen = dim / 2;
    //printf("ConvolveLoc(): Applying convoluation at location (%d, %d)/n", x, y);
    for ( j = 0; j < dim; j++)
    {
        int row = y + (j - cen);
        if (row < 0)
```

```
            row = 0;
        if (row >= src->rows)
            row = src->rows - 1;
        pixel += kernel[j] * Src(row,x);
    }
    if (pixel > 1)
        pixel = 1;
    return pixel;
}

//y方向作卷积
void Convolve1DHeight(float* kern, int dim, CvMat * src, CvMat * dst)
{
#define  Dst(ROW,COL)  ((float  *)(dst->data.fl  +  dst->step/sizeof(float)  *(ROW)))
[(COL)]
    unsigned int i,j;
    for ( j = 0; j < src->rows; j++)
    {
        for ( i = 0; i < src->cols; i++)
        {
            //printf("%d, %d/n", i, j);
            Dst(j,i) = ConvolveLocHeight(kern, dim, src, i, j);
        }
    }
}

//卷积模糊图像
int BlurImage(CvMat * src, CvMat * dst, float sigma)
{
    float* convkernel;
    int dim = (int) max(3.0f, 2.0 * GAUSSKERN * sigma + 1.0f);
    CvMat *tempMat;
    // make dim odd
    if (dim % 2 == 0)
        dim++;
    tempMat = cvCreateMat(src->rows, src->cols, CV_32FC1);
    convkernel = GaussianKernel1D(sigma, dim);

    Convolve1DWidth(convkernel, dim, src, tempMat);
    Convolve1DHeight(convkernel, dim, tempMat, dst);
    cvReleaseMat(&tempMat);
    return dim;
}
```

五个步骤

ok，接下来，进入重点部分，咱们依据上文介绍的sift算法的几个步骤，来一一实现这些函数。

为了版述清晰，再贴一下，主函数，顺便再加强下对sift 算法的五个步骤的认识：

1、SIFT算法第一步：图像预处理

CvMat *ScaleInitImage(CvMat * im) ; //金字塔初始化

2、SIFT算法第二步：建立高斯金字塔函数

ImageOctaves* BuildGaussianOctaves(CvMat * image) ; //建立高斯金字塔

3、SIFT算法第三步：特征点位置检测，最后确定特征点的位置

int DetectKeypoint(int numoctaves, ImageOctaves *GaussianPyr);

4、SIFT算法第四步：计算高斯图像的梯度方向和幅值，计算各个特征点的主方向

void ComputeGrad_DirecandMag(int numoctaves, ImageOctaves *GaussianPyr);

5、SIFT算法第五步：抽取各个特征点处的特征描述字

void ExtractFeatureDescriptors(int numoctaves, ImageOctaves *GaussianPyr);

ok，接下来一一具体实现这几个函数：

SIFT算法第一步

SIFT算法第一步：扩大图像，预滤波剔除噪声，得到金字塔的最底层-第一阶的第一层：

```
CvMat *ScaleInitImage(CvMat * im)
{
    double sigma,preblur_sigma;
    CvMat *imMat;
    CvMat * dst;
    CvMat *tempMat;
    //首先对图像进行平滑滤波,抑制噪声
    imMat = cvCreateMat(im->rows, im->cols, CV_32FC1);
    BlurImage(im, imMat, INITSIGMA);
    //针对两种情况分别进行处理：初始化放大原始图像或者在原图像基础上进行后续操作
    //建立金字塔的最底层
    if (DOUBLE_BASE_IMAGE_SIZE)
    {
        tempMat = doubleSizeImage2(imMat);//对扩大两倍的图像进行二次采样，采样率为0.5，采用线
性插值
#define TEMPMAT(ROW,COL) ((float *)(tempMat->data.fl + tempMat->step/sizeof(float) *
(ROW)))[(COL)]

        dst = cvCreateMat(tempMat->rows, tempMat->cols, CV_32FC1);
        preblur_sigma = 1.0;//sqrt(2 - 4*INITSIGMA*INITSIGMA);
        BlurImage(tempMat, dst, preblur_sigma);

           // The initial blurring for the first image of the first octave of the
pyramid.
        sigma = sqrt( (4*INITSIGMA*INITSIGMA) + preblur_sigma * preblur_sigma );
        //  sigma = sqrt(SIGMA * SIGMA - INITSIGMA * INITSIGMA * 4);
        //printf("Init Sigma: %f/n", sigma);
        BlurImage(dst, tempMat, sigma);       //得到金字塔的最底层-放大2倍的图像
        cvReleaseMat( &dst );
        return tempMat;
    }
    else
    {
        dst = cvCreateMat(im->rows, im->cols, CV_32FC1);
        //sigma = sqrt(SIGMA * SIGMA - INITSIGMA * INITSIGMA);
        preblur_sigma = 1.0;//sqrt(2 - 4*INITSIGMA*INITSIGMA);
        sigma = sqrt( (4*INITSIGMA*INITSIGMA) + preblur_sigma * preblur_sigma );
        //printf("Init Sigma: %f/n", sigma);
        BlurImage(imMat, dst, sigma);        //得到金字塔的最底层:原始图像大小
        return dst;
    }
}
```

SIFT算法第二步

SIFT第二步，建立Gaussian金字塔，给定金字塔第一阶第一层图像后，计算高斯金字塔其他尺度图像，每一阶的数目由变量SCALESPEROCTAVE决定，给定一个基本图像，计算它的高斯金字塔图像，返回外部向量是阶梯指针，内部向量是每一个阶梯内部的不同尺度图像。

```
//SIFT算法第二步
ImageOctaves* BuildGaussianOctaves(CvMat * image)
{
    ImageOctaves *octaves;
    CvMat *tempMat;
    CvMat *dst;
    CvMat *temp;

    int i,j;
    double k = pow(2, 1.0/((float)SCALESPEROCTAVE));  //方差倍数
    float preblur_sigma, initial_sigma , sigma1,sigma2,sigma,absolute_sigma,sigma_f;
    //计算金字塔的阶梯数目
    int dim = min(image->rows, image->cols);
    int numoctaves = (int) (log((double) dim) / log(2.0)) - 2;    //金字塔阶数
    //限定金字塔的阶梯数
    numoctaves = min(numoctaves, MAXOCTAVES);
    //为高斯金塔和DOG金字塔分配内存
    octaves=(ImageOctaves*) malloc( numoctaves * sizeof(ImageOctaves) );
    DOGoctaves=(ImageOctaves*) malloc( numoctaves * sizeof(ImageOctaves) );

      printf("BuildGaussianOctaves(): Base image dimension is %dx%d/n", (int)(0.5*
(image->cols)), (int)(0.5*(image->rows)) );
    printf("BuildGaussianOctaves(): Building %d octaves/n", numoctaves);

    // start with initial source image
    tempMat=cvCloneMat( image );
    // preblur_sigma = 1.0;//sqrt(2 - 4*INITSIGMA*INITSIGMA);
       initial_sigma  =  sqrt(2);//sqrt(  (4*INITSIGMA*INITSIGMA)  +  preblur_sigma  *
preblur_sigma );
    //   initial_sigma = sqrt(SIGMA * SIGMA - INITSIGMA * INITSIGMA * 4);

    //在每一阶金字塔图像中建立不同的尺度图像
    for ( i = 0; i < numoctaves; i++)
    {
        //首先建立金字塔每一阶梯的最底层，其中0阶梯的最底层已经建立好
        printf("Building octave %d of dimesion (%d, %d)/n", i, tempMat->cols,tempMat-
>rows);
        //为各个阶梯分配内存
              octaves[i].Octave=  (ImageLevels*)  malloc(  (SCALESPEROCTAVE  +  3)  *
sizeof(ImageLevels) );
```

```
            DOGoctaves[i].Octave= (ImageLevels*) malloc( (SCALESPEROCTAVE + 2) *
sizeof(ImageLevels) );
        //存储各个阶梯的最底层
        (octaves[i].Octave)[0].Level=tempMat;

        octaves[i].col=tempMat->cols;
        octaves[i].row=tempMat->rows;
        DOGoctaves[i].col=tempMat->cols;
        DOGoctaves[i].row=tempMat->rows;
        if (DOUBLE_BASE_IMAGE_SIZE)
            octaves[i].subsample=pow(2,i)*0.5;
        else
            octaves[i].subsample=pow(2,i);

        if(i==0)
        {
            (octaves[0].Octave)[0].levelsigma = initial_sigma;
            (octaves[0].Octave)[0].absolute_sigma = initial_sigma;
                printf("0 scale and blur sigma : %f /n", (octaves[0].subsample) *
((octaves[0].Octave)[0].absolute_sigma));
        }
        else
        {
                        (octaves[i].Octave)[0].levelsigma = (octaves[i-1].Octave)
[SCALESPEROCTAVE].levelsigma;
                    (octaves[i].Octave)[0].absolute_sigma = (octaves[i-1].Octave)
[SCALESPEROCTAVE].absolute_sigma;
                printf( "0 scale and blur sigma : %f /n", ((octaves[i].Octave)
[0].absolute_sigma) );
        }
        sigma = initial_sigma;
        //建立本阶梯其他层的图像
        for ( j = 1; j < SCALESPEROCTAVE + 3; j++)
        {
            dst = cvCreateMat(tempMat->rows, tempMat->cols, CV_32FC1);//用于存储高斯层
            temp = cvCreateMat(tempMat->rows, tempMat->cols, CV_32FC1);//用于存储DOG层
            // 2 passes of 1D on original
            //   if(i!=0)
            //   {
                  //          sigma1 = pow(k, j - 1) * ((octaves[i-1].Octave)[j-
1].levelsigma);
            //          sigma2 = pow(k, j) * ((octaves[i].Octave)[j-1].levelsigma);
            //       sigma = sqrt(sigma2*sigma2 - sigma1*sigma1);
            sigma_f= sqrt(k*k-1)*sigma;
            //   }
            //   else
            //   {
                 //          sigma = sqrt(SIGMA * SIGMA - INITSIGMA * INITSIGMA *
```

```
4)*pow(k,j);
            //   }
            sigma = k*sigma;
            absolute_sigma = sigma * (octaves[i].subsample);
            printf("%d scale and Blur sigma: %f  /n", j, absolute_sigma);

            (octaves[i].Octave)[j].levelsigma = sigma;
            (octaves[i].Octave)[j].absolute_sigma = absolute_sigma;
            //产生高斯层
             int length=BlurImage((octaves[i].Octave)[j-1].Level, dst, sigma_f);//相应
尺度
            (octaves[i].Octave)[j].levelsigmalength = length;
            (octaves[i].Octave)[j].Level=dst;
            //产生DOG层
             cvSub( ((octaves[i].Octave)[j]).Level, ((octaves[i].Octave)[j-1]).Level,
temp, 0 );
                        //                cvAbsDiff(  ((octaves[i].Octave)[j]).Level,
((octaves[i].Octave)[j-1]).Level, temp );
            ((DOGoctaves[i].Octave)[j-1]).Level=temp;
        }
        // halve the image size for next iteration
        tempMat  = halfSizeImage( ( (octaves[i].Octave)[SCALESPEROCTAVE].Level ) );
    }
    return octaves;
}
```

SIFT算法第三步

SIFT算法第三步，特征点位置检测，最后确定特征点的位置检测DOG金字塔中的局部最大值，找到之后，还要经过两个检验才能确认为特征点：一是它必须有明显的差异，二是他不应该是边缘点，（也就是说，在极值点处的主曲率比应该小于某一个阈值）。

```
//SIFT算法第三步，特征点位置检测，
int DetectKeypoint(int numoctaves, ImageOctaves *GaussianPyr)
{
    //计算用于DOG极值点检测的主曲率比的阈值
    double curvature_threshold;
          curvature_threshold=   ((CURVATURE_THRESHOLD   +   1)*(CURVATURE_THRESHOLD   +
1))/CURVATURE_THRESHOLD;
#define             ImLevels(OCTAVE,LEVEL,ROW,COL)              ((float             *)
(DOGoctaves[(OCTAVE)].Octave[(LEVEL)].Level->data.fl                                    +
DOGoctaves[(OCTAVE)].Octave[(LEVEL)].Level->step/sizeof(float) *(ROW)))[(COL)]

    int   keypoint_count = 0;
```

```
for (int i=0; i<numoctaves; i++)
{
    for(int j=1;j<SCALESPEROCTAVE+1;j++)//取中间的scaleperoctave个层
    {
        //在图像的有效区域内寻找具有显著性特征的局部最大值
        //float sigma=(GaussianPyr[i].Octave)[j].levelsigma;
        //int dim = (int) (max(3.0f, 2.0*GAUSSKERN *sigma + 1.0f)*0.5);
        int dim = (int)(0.5*((GaussianPyr[i].Octave)[j].levelsigmalength)+0.5);
        for (int m=dim;m<((DOGoctaves[i].row)-dim);m++)
            for(int n=dim;n<((DOGoctaves[i].col)-dim);n++)
            {
                if ( fabs(ImLevels(i,j,m,n))>= CONTRAST_THRESHOLD )
                {

                    if ( ImLevels(i,j,m,n)!=0.0 )  //1、首先是非零
                    {
                        float inf_val=ImLevels(i,j,m,n);
                        if(( (inf_val <= ImLevels(i,j-1,m-1,n-1))&&
                            (inf_val <= ImLevels(i,j-1,m  ,n-1))&&
                            (inf_val <= ImLevels(i,j-1,m+1,n-1))&&
                            (inf_val <= ImLevels(i,j-1,m-1,n  ))&&
                            (inf_val <= ImLevels(i,j-1,m  ,n  ))&&
                            (inf_val <= ImLevels(i,j-1,m+1,n  ))&&
                            (inf_val <= ImLevels(i,j-1,m-1,n+1))&&
                            (inf_val <= ImLevels(i,j-1,m  ,n+1))&&
                             (inf_val <= ImLevels(i,j-1,m+1,n+1))&&     //底层的小尺
度9

                            (inf_val <= ImLevels(i,j,m-1,n-1))&&
                            (inf_val <= ImLevels(i,j,m  ,n-1))&&
                            (inf_val <= ImLevels(i,j,m+1,n-1))&&
                            (inf_val <= ImLevels(i,j,m-1,n  ))&&
                            (inf_val <= ImLevels(i,j,m+1,n  ))&&
                            (inf_val <= ImLevels(i,j,m-1,n+1))&&
                            (inf_val <= ImLevels(i,j,m  ,n+1))&&
                            (inf_val <= ImLevels(i,j,m+1,n+1))&&      //当前层8

                            (inf_val <= ImLevels(i,j+1,m-1,n-1))&&
                            (inf_val <= ImLevels(i,j+1,m  ,n-1))&&
                            (inf_val <= ImLevels(i,j+1,m+1,n-1))&&
                            (inf_val <= ImLevels(i,j+1,m-1,n  ))&&
                            (inf_val <= ImLevels(i,j+1,m  ,n  ))&&
                            (inf_val <= ImLevels(i,j+1,m+1,n  ))&&
                            (inf_val <= ImLevels(i,j+1,m-1,n+1))&&
                            (inf_val <= ImLevels(i,j+1,m  ,n+1))&&
                             (inf_val <= ImLevels(i,j+1,m+1,n+1))      //下一层大尺度
9
                           ) ||
```

```
                    ( (inf_val >= ImLevels(i,j-1,m-1,n-1))&&
                    (inf_val >= ImLevels(i,j-1,m  ,n-1))&&
                    (inf_val >= ImLevels(i,j-1,m+1,n-1))&&
                    (inf_val >= ImLevels(i,j-1,m-1,n  ))&&
                    (inf_val >= ImLevels(i,j-1,m  ,n  ))&&
                    (inf_val >= ImLevels(i,j-1,m+1,n  ))&&
                    (inf_val >= ImLevels(i,j-1,m-1,n+1))&&
                    (inf_val >= ImLevels(i,j-1,m  ,n+1))&&
                    (inf_val >= ImLevels(i,j-1,m+1,n+1))&&

                    (inf_val >= ImLevels(i,j,m-1,n-1))&&
                    (inf_val >= ImLevels(i,j,m  ,n-1))&&
                    (inf_val >= ImLevels(i,j,m+1,n-1))&&
                    (inf_val >= ImLevels(i,j,m-1,n  ))&&
                    (inf_val >= ImLevels(i,j,m+1,n  ))&&
                    (inf_val >= ImLevels(i,j,m-1,n+1))&&
                    (inf_val >= ImLevels(i,j,m  ,n+1))&&
                    (inf_val >= ImLevels(i,j,m+1,n+1))&&

                    (inf_val >= ImLevels(i,j+1,m-1,n-1))&&
                    (inf_val >= ImLevels(i,j+1,m  ,n-1))&&
                    (inf_val >= ImLevels(i,j+1,m+1,n-1))&&
                    (inf_val >= ImLevels(i,j+1,m-1,n  ))&&
                    (inf_val >= ImLevels(i,j+1,m  ,n  ))&&
                    (inf_val >= ImLevels(i,j+1,m+1,n  ))&&
                    (inf_val >= ImLevels(i,j+1,m-1,n+1))&&
                    (inf_val >= ImLevels(i,j+1,m  ,n+1))&&
                    (inf_val >= ImLevels(i,j+1,m+1,n+1))
                    ) )       //2、满足26个中极值点
                {
                    //此处可存储
                                    //然后必须具有明显的显著性，即必须大于
CONTRAST_THRESHOLD=0.02
                    if ( fabs(ImLevels(i,j,m,n))>= CONTRAST_THRESHOLD )
                    {
                                    //最后显著处的特征点必须具有足够的曲率比，
CURVATURE_THRESHOLD=10.0,首先计算Hessian矩阵
                                      // Compute the entries of the Hessian matrix at
the extrema location.
                        /*
                        1   0   -1
                        0   0   0
                        -1   0   1          *0.25
                        */
                          // Compute the trace and the determinant of the
Hessian.
                        //Tr_H = Dxx + Dyy;
                        //Det_H = Dxx*Dyy - Dxy^2;
```

```
                                    float Dxx,Dyy,Dxy,Tr_H,Det_H,curvature_ratio;
                                                    Dxx  =  ImLevels(i,j,m,n-1)  +
ImLevels(i,j,m,n+1)-2.0*ImLevels(i,j,m,n);
                                                    Dyy  =  ImLevels(i,j,m-1,n)  +
ImLevels(i,j,m+1,n)-2.0*ImLevels(i,j,m,n);
                                                  Dxy  =  ImLevels(i,j,m-1,n-1)  +
ImLevels(i,j,m+1,n+1) - ImLevels(i,j,m+1,n-1) - ImLevels(i,j,m-1,n+1);
                                    Tr_H = Dxx + Dyy;
                                    Det_H = Dxx*Dyy - Dxy*Dxy;
                                    // Compute the ratio of the principal curvatures.
                                    curvature_ratio = (1.0*Tr_H*Tr_H)/Det_H;
                                            if  ( (Det_H>=0.0)  &&  (curvature_ratio  <=
curvature_threshold) )  //最后得到最具有显著性特征的特征点
                                    {
                                        //将其存储起来,以计算后面的特征描述字
                                        keypoint_count++;
                                        Keypoint k;
                                        /* Allocate memory for the keypoint. */
                                                  k  =  (Keypoint)  malloc(sizeof(struct
KeypointSt));
                                        k->next = keypoints;
                                        keypoints = k;
                                        k->row = m*(GaussianPyr[i].subsample);
                                        k->col =n*(GaussianPyr[i].subsample);
                                        k->sy = m;    //行
                                        k->sx = n;    //列
                                        k->octave=i;
                                        k->level=j;
                                                   k->scale  =  (GaussianPyr[i].Octave)
[j].absolute_sigma;
                                    }//if >curvature_thresh
                                }//if >contrast
                            }//if inf value
                        }//if non zero
                    }//if >contrast
                }  //for concrete image level col
            }//for levels
        }//for octaves
        return keypoint_count;
}

//在图像中,显示SIFT特征点的位置
void DisplayKeypointLocation(IplImage* image, ImageOctaves *GaussianPyr)
{

    Keypoint p = keypoints; // p指向第一个结点
    while(p) // 没到表尾
    {
```

```
        cvLine( image, cvPoint((int)((p->col)-3),(int)(p->row)),
            cvPoint((int)((p->col)+3),(int)(p->row)), CV_RGB(255,255,0),
            1, 8, 0 );
        cvLine( image, cvPoint((int)(p->col),(int)((p->row)-3)),
            cvPoint((int)(p->col),(int)((p->row)+3)), CV_RGB(255,255,0),
            1, 8, 0 );
        //  cvCircle(image,cvPoint((uchar)(p->col),(uchar)(p->row)),
        //   (int)((GaussianPyr[p->octave].Octave)[p->level].absolute_sigma),
        //   CV_RGB(255,0,0),1,8,0);
        p=p->next;
    }
}

// Compute the gradient direction and magnitude of the gaussian pyramid images
void ComputeGrad_DirecandMag(int numoctaves, ImageOctaves *GaussianPyr)
{
    // ImageOctaves *mag_thresh ;
    mag_pyr=(ImageOctaves*) malloc( numoctaves * sizeof(ImageOctaves) );
    grad_pyr=(ImageOctaves*) malloc( numoctaves * sizeof(ImageOctaves) );
      // float sigma=( (GaussianPyr[0].Octave)[SCALESPEROCTAVE+2].absolute_sigma ) /
GaussianPyr[0].subsample;
    // int dim = (int) (max(3.0f, 2 * GAUSSKERN *sigma + 1.0f)*0.5+0.5);
#define             ImLevels(OCTAVE,LEVEL,ROW,COL)             ((float             *)
(GaussianPyr[(OCTAVE)].Octave[(LEVEL)].Level->data.fl                                    +
GaussianPyr[(OCTAVE)].Octave[(LEVEL)].Level->step/sizeof(float) *(ROW)))[(COL)]
    for (int i=0; i<numoctaves; i++)
    {
                    mag_pyr[i].Octave=   (ImageLevels*)   malloc(   (SCALESPEROCTAVE)   *
sizeof(ImageLevels) );
                   grad_pyr[i].Octave=   (ImageLevels*)   malloc(   (SCALESPEROCTAVE)   *
sizeof(ImageLevels) );
        for(int j=1;j<SCALESPEROCTAVE+1;j++)//取中间的scaleperoctave个层
        {
                    CvMat  *Mag  =  cvCreateMat(GaussianPyr[i].row,  GaussianPyr[i].col,
CV_32FC1);
                    CvMat  *Ori  =  cvCreateMat(GaussianPyr[i].row,  GaussianPyr[i].col,
CV_32FC1);
                CvMat  *tempMat1  =  cvCreateMat(GaussianPyr[i].row,  GaussianPyr[i].col,
CV_32FC1);
                CvMat  *tempMat2  =  cvCreateMat(GaussianPyr[i].row,  GaussianPyr[i].col,
CV_32FC1);
            cvZero(Mag);
            cvZero(Ori);
            cvZero(tempMat1);
            cvZero(tempMat2);
#define   MAG(ROW,COL)   ((float   *)(Mag->data.fl   +   Mag->step/sizeof(float)   *(ROW)))
[(COL)]
#define   ORI(ROW,COL)   ((float   *)(Ori->data.fl   +   Ori->step/sizeof(float)   *(ROW)))
```

```
[(COL)]
#define TEMPMAT1(ROW,COL) ((float *)(tempMat1->data.fl + tempMat1->step/sizeof(float)
*(ROW)))[(COL)]
#define TEMPMAT2(ROW,COL) ((float *)(tempMat2->data.fl + tempMat2->step/sizeof(float)
*(ROW)))[(COL)]
            for (int m=1;m<(GaussianPyr[i].row-1);m++)
                for(int n=1;n<(GaussianPyr[i].col-1);n++)
                {
                    //计算幅值
                     TEMPMAT1(m,n) = 0.5*( ImLevels(i,j,m,n+1)-ImLevels(i,j,m,n-1) );
//dx
                     TEMPMAT2(m,n) = 0.5*( ImLevels(i,j,m+1,n)-ImLevels(i,j,m-1,n) );
//dy
                                                                          MAG(m,n)    =
sqrt(TEMPMAT1(m,n)*TEMPMAT1(m,n)+TEMPMAT2(m,n)*TEMPMAT2(m,n));  //mag
                    //计算方向
                    ORI(m,n) =atan( TEMPMAT2(m,n)/TEMPMAT1(m,n) );
                    if (ORI(m,n)==CV_PI)
                        ORI(m,n)=-CV_PI;
                }
                ((mag_pyr[i].Octave)[j-1]).Level=Mag;
                ((grad_pyr[i].Octave)[j-1]).Level=Ori;
                cvReleaseMat(&tempMat1);
                cvReleaseMat(&tempMat2);
        }//for levels
    }//for octaves
}
```

SIFT算法第四步

```
//SIFT算法第四步:计算各个特征点的主方向,确定主方向
void AssignTheMainOrientation(int numoctaves, ImageOctaves *GaussianPyr,ImageOctaves
*mag_pyr,ImageOctaves *grad_pyr)
{
    // Set up the histogram bin centers for a 36 bin histogram.
    int num_bins = 36;
    float hist_step = 2.0*PI/num_bins;
    float hist_orient[36];
    for (int i=0;i<36;i++)
        hist_orient[i]=-PI+i*hist_step;
      float  sigma1=(  ((GaussianPyr[0].Octave)[SCALESPEROCTAVE].absolute_sigma)  )  /
(GaussianPyr[0].subsample);//SCALESPEROCTAVE+2
    int zero_pad = (int) (max(3.0f, 2 * GAUSSKERN *sigma1 + 1.0f)*0.5+0.5);
    //Assign orientations to the keypoints.
#define             ImLevels(OCTAVES,LEVELS,ROW,COL)             ((float             *)
((GaussianPyr[(OCTAVES)].Octave[(LEVELS)].Level)->data.fl                              +
(GaussianPyr[(OCTAVES)].Octave[(LEVELS)].Level)->step/sizeof(float) *(ROW)))[(COL)]
```

```
    int keypoint_count = 0;
    Keypoint p = keypoints; // p指向第一个结点

    while(p) // 没到表尾
    {
        int i=p->octave;
        int j=p->level;
        int m=p->sy;    //行
        int n=p->sx;    //列
        if ((m>=zero_pad)&&(m<GaussianPyr[i].row-zero_pad)&&
            (n>=zero_pad)&&(n<GaussianPyr[i].col-zero_pad) )
        {
                        float  sigma=(  ((GaussianPyr[i].Octave)[j].absolute_sigma)  )  /
(GaussianPyr[i].subsample);
            //产生二维高斯模板
            CvMat* mat = GaussianKernel2D( sigma );
            int dim=(int)(0.5 * (mat->rows));
            //分配用于存储Patch幅值和方向的空间
#define  MAT(ROW,COL)  ((float  *)(mat->data.fl  +  mat->step/sizeof(float)  *(ROW)))
[(COL)]

            //声明方向直方图变量
            double* orienthist = (double *) malloc(36 * sizeof(double));
            for ( int sw = 0 ; sw < 36 ; ++sw)
            {
                orienthist[sw]=0.0;
            }
            //在特征点的周围统计梯度方向
            for (int x=m-dim,mm=0;x<=(m+dim);x++,mm++)
                for(int y=n-dim,nn=0;y<=(n+dim);y++,nn++)
                {
                    //计算特征点处的幅值
                    double dx = 0.5*(ImLevels(i,j,x,y+1)-ImLevels(i,j,x,y-1));  //dx
                    double dy = 0.5*(ImLevels(i,j,x+1,y)-ImLevels(i,j,x-1,y));  //dy
                    double mag = sqrt(dx*dx+dy*dy);  //mag
                    //计算方向
                    double Ori =atan( 1.0*dy/dx );
                                    int  binIdx  =  FindClosestRotationBin(36,  Ori);
//得到离现有方向最近的直方块
                                orienthist[binIdx]  =  orienthist[binIdx]  +  1.0*  mag  *
MAT(mm,nn);//利用高斯加权累加进直方图相应的块
                }
                // Find peaks in the orientation histogram using nonmax suppression.
                AverageWeakBins (orienthist, 36);
                // find the maximum peak in gradient orientation
                double maxGrad = 0.0;
                int maxBin = 0;
```

```
                for (int b = 0 ; b < 36 ; ++b)
                {
                    if (orienthist[b] > maxGrad)
                    {
                        maxGrad = orienthist[b];
                        maxBin = b;
                    }
                }
                // First determine the real interpolated peak high at the maximum bin
                // position, which is guaranteed to be an absolute peak.
                double maxPeakValue=0.0;
                double maxDegreeCorrection=0.0;
                  if ( (InterpolateOrientation ( orienthist[maxBin == 0 ? (36 - 1) :
(maxBin - 1)],
                    orienthist[maxBin], orienthist[(maxBin + 1) % 36],
                    &maxDegreeCorrection, &maxPeakValue)) == false)
                    printf("BUG: Parabola fitting broken");

                     // Now that we know the maximum peak value, we can find other
keypoint
                // orientations, which have to fulfill two criterias:
                //
                //  1. They must be a local peak themselves. Else we might add a very
                //     similar keypoint orientation twice (imagine for example the
                   //      values: 0.4 1.0 0.8, if 1.0 is maximum peak, 0.8 is still
added
                //     with the default threshhold, but the maximum peak orientation
                //     was already added).
                //  2. They must have at least peakRelThresh times the maximum peak
                //     value.
                bool binIsKeypoint[36];
                for ( b = 0 ; b < 36 ; ++b)
                {
                    binIsKeypoint[b] = false;
                    // The maximum peak of course is
                    if (b == maxBin)
                    {
                        binIsKeypoint[b] = true;
                        continue;
                    }
                    // Local peaks are, too, in case they fulfill the threshhold
                    if (orienthist[b] < (peakRelThresh * maxPeakValue))
                        continue;
                    int leftI = (b == 0) ? (36 - 1) : (b - 1);
                    int rightI = (b + 1) % 36;
                          if (orienthist[b] <= orienthist[leftI] || orienthist[b] <=
orienthist[rightI])
                        continue; // no local peak
```

```
                    binIsKeypoint[b] = true;
                }
                // find other possible locations
                double oneBinRad = (2.0 * PI) / 36;
                for ( b = 0 ; b < 36 ; ++b)
                {
                    if (binIsKeypoint[b] == false)
                        continue;
                    int bLeft = (b == 0) ? (36 - 1) : (b - 1);
                    int bRight = (b + 1) % 36;
                    // Get an interpolated peak direction and value guess.
                    double peakValue;
                    double degreeCorrection;

                    double maxPeakValue, maxDegreeCorrection;
                     if (InterpolateOrientation ( orienthist[maxBin == 0 ? (36 - 1) :
(maxBin - 1)],
                        orienthist[maxBin], orienthist[(maxBin + 1) % 36],
                        °reeCorrection, &peakValue) == false)
                    {
                        printf("BUG: Parabola fitting broken");
                    }

                    double degree = (b + degreeCorrection) * oneBinRad - PI;
                    if (degree < -PI)
                        degree += 2.0 * PI;
                    else if (degree > PI)
                        degree -= 2.0 * PI;
                    //存储方向,可以直接利用检测到的链表进行该步主方向的指定;
                    //分配内存重新存储特征点
                    Keypoint k;
                    /* Allocate memory for the keypoint Descriptor. */
                    k = (Keypoint) malloc(sizeof(struct KeypointSt));
                    k->next = keyDescriptors;
                    keyDescriptors = k;
                    k->descrip = (float*)malloc(LEN * sizeof(float));
                    k->row = p->row;
                    k->col = p->col;
                    k->sy = p->sy;    //行
                    k->sx = p->sx;    //列
                    k->octave = p->octave;
                    k->level = p->level;
                    k->scale = p->scale;
                    k->ori = degree;
                    k->mag = peakValue;
                }//for
                free(orienthist);
        }
```

```
        p=p->next;
    }
}

//寻找与方向直方图最近的柱，确定其index
int FindClosestRotationBin (int binCount, float angle)
{
    angle += CV_PI;
    angle /= 2.0 * CV_PI;
    // calculate the aligned bin
    angle *= binCount;
    int idx = (int) angle;
    if (idx == binCount)
        idx = 0;
    return (idx);
}

// Average the content of the direction bins.
void AverageWeakBins (double* hist, int binCount)
{
    // TODO: make some tests what number of passes is the best. (its clear
    // one is not enough, as we may have something like
    // ( 0.4, 0.4, 0.3, 0.4, 0.4 ))
    for (int sn = 0 ; sn < 2 ; ++sn)
    {
        double firstE = hist[0];
        double last = hist[binCount-1];
        for (int sw = 0 ; sw < binCount ; ++sw)
        {
            double cur = hist[sw];
                 double next = (sw == (binCount - 1)) ? firstE : hist[(sw + 1) %
binCount];
            hist[sw] = (last + cur + next) / 3.0;
            last = cur;
        }
    }
}

// Fit a parabol to the three points (-1.0 ; left), (0.0 ; middle) and
// (1.0 ; right).
// Formulas:
// f(x) = a (x - c)^2 + b
// c is the peak offset (where f'(x) is zero), b is the peak value.
// In case there is an error false is returned, otherwise a correction
// value between [-1 ; 1] is returned in 'degreeCorrection', where -1
// means the peak is located completely at the left vector, and -0.5 just
// in the middle between left and middle and > 0 to the right side. In
// 'peakValue' the maximum estimated peak value is stored.
```

```
bool InterpolateOrientation (double left, double middle,double right, double
*degreeCorrection, double *peakValue)
{
    double a = ((left + right) - 2.0 * middle) / 2.0;    //抛物线捏合系数a
    // degreeCorrection = peakValue = Double.NaN;

    // Not a parabol
    if (a == 0.0)
        return false;
    double c = (((left - middle) / a) - 1.0) / 2.0;
    double b = middle - c * c * a;
    if (c < -0.5 || c > 0.5)
        return false;
    *degreeCorrection = c;
    *peakValue = b;
    return true;
}

//显示特征点处的主方向
void DisplayOrientation (IplImage* image, ImageOctaves *GaussianPyr)
{
    Keypoint p = keyDescriptors; // p指向第一个结点
    while(p) // 没到表尾
    {
        float scale=(GaussianPyr[p->octave].Octave)[p->level].absolute_sigma;
        float autoscale = 3.0;
        float uu=autoscale*scale*cos(p->ori);
        float vv=autoscale*scale*sin(p->ori);
        float x=(p->col)+uu;
        float y=(p->row)+vv;
        cvLine( image, cvPoint((int)(p->col),(int)(p->row)),
            cvPoint((int)x,(int)y), CV_RGB(255,255,0),
            1, 8, 0 );
        // Arrow head parameters
          float alpha = 0.33; // Size of arrow head relative to the length of the
vector
         float beta = 0.33;  // Width of the base of the arrow head relative to the
length

        float xx0= (p->col)+uu-alpha*(uu+beta*vv);
        float yy0= (p->row)+vv-alpha*(vv-beta*uu);
        float xx1= (p->col)+uu-alpha*(uu-beta*vv);
        float yy1= (p->row)+vv-alpha*(vv+beta*uu);
        cvLine( image, cvPoint((int)xx0,(int)yy0),
            cvPoint((int)x,(int)y), CV_RGB(255,255,0),
            1, 8, 0 );
        cvLine( image, cvPoint((int)xx1,(int)yy1),
            cvPoint((int)x,(int)y), CV_RGB(255,255,0),
```

```
            1, 8, 0 );
        p=p->next;
    }
}
```

SIFT算法第五步

SIFT算法第五步：抽取各个特征点处的特征描述字，确定特征点的描述字。描述字是Patch网格内梯度方向的描述，旋转网格到主方向，插值得到网格处梯度值。

一个特征点可以用2*2*8=32维的向量，也可以用4*4*8=128维的向量更精确的进行描述。

```
void ExtractFeatureDescriptors(int numoctaves, ImageOctaves *GaussianPyr)
{
    // The orientation histograms have 8 bins
    float orient_bin_spacing = PI/4;
    float orient_angles[8]={-PI,-PI+orient_bin_spacing,-PI*0.5, -orient_bin_spacing,
        0.0, orient_bin_spacing, PI*0.5,  PI+orient_bin_spacing};
    //产生描述字中心各点坐标
    float *feat_grid=(float *) malloc( 2*16 * sizeof(float));
    for (int i=0;i<GridSpacing;i++)
    {
        for (int j=0;j<2*GridSpacing;++j,++j)
        {
            feat_grid[i*2*GridSpacing+j]=-6.0+i*GridSpacing;
            feat_grid[i*2*GridSpacing+j+1]=-6.0+0.5*j*GridSpacing;
        }
    }
    //产生网格
    float *feat_samples=(float *) malloc( 2*256 * sizeof(float));
    for ( i=0;i<4*GridSpacing;i++)
    {
        for (int j=0;j<8*GridSpacing;j+=2)
        {
            feat_samples[i*8*GridSpacing+j]=-(2*GridSpacing-0.5)+i;
            feat_samples[i*8*GridSpacing+j+1]=-(2*GridSpacing-0.5)+0.5*j;
        }
    }
    float feat_window = 2*GridSpacing;
    Keypoint p = keyDescriptors; // p指向第一个结点
    while(p) // 没到表尾
    {
        float scale=(GaussianPyr[p->octave].Octave)[p->level].absolute_sigma;
```

```
        float sine = sin(p->ori);
        float cosine = cos(p->ori);
        //计算中心点坐标旋转之后的位置
        float *featcenter=(float *) malloc( 2*16 * sizeof(float));
        for (int i=0;i<GridSpacing;i++)
        {
            for (int j=0;j<2*GridSpacing;j+=2)
            {
                float x=feat_grid[i*2*GridSpacing+j];
                float y=feat_grid[i*2*GridSpacing+j+1];
                featcenter[i*2*GridSpacing+j]=((cosine * x + sine * y) + p->sx);
                featcenter[i*2*GridSpacing+j+1]=((-sine * x + cosine * y) + p->sy);
            }
        }
        // calculate sample window coordinates (rotated along keypoint)
        float *feat=(float *) malloc( 2*256 * sizeof(float));
        for ( i=0;i<64*GridSpacing;i++,i++)
        {
            float x=feat_samples[i];
            float y=feat_samples[i+1];
            feat[i]=((cosine * x + sine * y) + p->sx);
            feat[i+1]=((-sine * x + cosine * y) + p->sy);
        }
        //Initialize the feature descriptor.
        float *feat_desc = (float *) malloc( 128 * sizeof(float));
        for (i=0;i<128;i++)
        {
            feat_desc[i]=0.0;
            // printf("%f  ",feat_desc[i]);
        }
        //printf("/n");
        for ( i=0;i<512;++i,++i)
        {
            float x_sample = feat[i];
            float y_sample = feat[i+1];
            // Interpolate the gradient at the sample position
            /*
            0   1   0
            1   *   1
            0   1   0   具体插值策略如图示
            */
                        float  sample12=getPixelBI(((GaussianPyr[p->octave].Octave)[p-
>level]).Level, x_sample, y_sample-1);
                        float  sample21=getPixelBI(((GaussianPyr[p->octave].Octave)[p-
>level]).Level, x_sample-1, y_sample);
                        float  sample22=getPixelBI(((GaussianPyr[p->octave].Octave)[p-
>level]).Level, x_sample, y_sample);
```

```
                        float  sample23=getPixelBI(((GaussianPyr[p->octave].Octave)[p-
>level]).Level, x_sample+1, y_sample);
                        float  sample32=getPixelBI(((GaussianPyr[p->octave].Octave)[p-
>level]).Level, x_sample, y_sample+1);
            //float diff_x = 0.5*(sample23 - sample21);
            //float diff_y = 0.5*(sample32 - sample12);
            float diff_x = sample23 - sample21;
            float diff_y = sample32 - sample12;
            float mag_sample = sqrt( diff_x*diff_x + diff_y*diff_y );
            float grad_sample = atan( diff_y / diff_x );
            if(grad_sample == CV_PI)
                grad_sample = -CV_PI;
            // Compute the weighting for the x and y dimensions.
                    float  *x_wght=(float  *)  malloc(  GridSpacing  *  GridSpacing  *
sizeof(float));
                    float  *y_wght=(float  *)  malloc(  GridSpacing  *  GridSpacing  *
sizeof(float));
                  float  *pos_wght=(float  *)  malloc(  8*GridSpacing  *  GridSpacing  *
sizeof(float));;
            for (int m=0;m<32;++m,++m)
            {
                float x=featcenter[m];
                float y=featcenter[m+1];
                x_wght[m/2] = max(1 - (fabs(x - x_sample)*1.0/GridSpacing), 0);
                y_wght[m/2] = max(1 - (fabs(y - y_sample)*1.0/GridSpacing), 0);

            }
            for ( m=0;m<16;++m)
                for (int n=0;n<8;++n)
                    pos_wght[m*8+n]=x_wght[m]*y_wght[m];
            free(x_wght);
            free(y_wght);
            //计算方向的加权，首先旋转梯度场到主方向，然后计算差异
            float diff[8],orient_wght[128];
            for ( m=0;m<8;++m)
            {
                float angle = grad_sample-(p->ori)-orient_angles[m]+CV_PI;
                float temp = angle / (2.0 * CV_PI);
                angle -= (int)(temp) * (2.0 * CV_PI);
                diff[m]= angle - CV_PI;
            }
            // Compute the gaussian weighting.
            float x=p->sx;
            float y=p->sy;
                   float  g  =  exp(-((x_sample-x)*(x_sample-x)+(y_sample-y)*(y_sample-
y))/(2*feat_window*feat_window))/(2*CV_PI*feat_window*feat_window);

            for ( m=0;m<128;++m)
```

```
            {
                                                    orient_wght[m]    =    max((1.0    -
1.0*fabs(diff[m%8])/orient_bin_spacing),0);
                                                feat_desc[m]    =    feat_desc[m]    +
orient_wght[m]*pos_wght[m]*g*mag_sample;
            }
            free(pos_wght);
        }
        free(feat);
        free(featcenter);
        float norm=GetVecNorm( feat_desc, 128);
        for (int m=0;m<128;m++)
        {
            feat_desc[m]/=norm;
            if (feat_desc[m]>0.2)
                feat_desc[m]=0.2;
        }
        norm=GetVecNorm( feat_desc, 128);
        for ( m=0;m<128;m++)
        {
            feat_desc[m]/=norm;
            printf("%f  ",feat_desc[m]);
        }
        printf("/n");
        p->descrip = feat_desc;
        p=p->next;
    }
    free(feat_grid);
    free(feat_samples);
}

//为了显示图象金字塔，而作的图像水平拼接
CvMat* MosaicHorizen( CvMat* im1, CvMat* im2 )
{
    int row,col;
          CvMat   *mosaic   =   cvCreateMat(   max(im1->rows,im2->rows),(im1->cols+im2-
>cols),CV_32FC1);
#define   Mosaic(ROW,COL)   ((float*)(mosaic->data.fl   +   mosaic->step/sizeof(float)*
(ROW)))[(COL)]
#define  Im11Mat(ROW,COL)  ((float  *)(im1->data.fl  +  im1->step/sizeof(float)  *(ROW)))
[(COL)]
#define  Im22Mat(ROW,COL)  ((float  *)(im2->data.fl  +  im2->step/sizeof(float)  *(ROW)))
[(COL)]
    cvZero(mosaic);
    /* Copy images into mosaic1. */
    for ( row = 0; row < im1->rows; row++)
        for ( col = 0; col < im1->cols; col++)
            Mosaic(row,col)=Im11Mat(row,col) ;
```

```
    for (  row = 0; row < im2->rows; row++)
        for (  col = 0; col < im2->cols; col++)
            Mosaic(row, (col+im1->cols) )= Im22Mat(row,col) ;
    return mosaic;
}

//为了显示图象金字塔，而作的图像垂直拼接
CvMat* MosaicVertical( CvMat* im1, CvMat* im2 )
{
    int row,col;
         CvMat  *mosaic  =  cvCreateMat(im1->rows+im2->rows,max(im1->cols,im2->cols),
CV_32FC1);
#define  Mosaic(ROW,COL)  ((float*)(mosaic->data.fl  +  mosaic->step/sizeof(float)*
(ROW)))[(COL)]
#define  Im11Mat(ROW,COL)  ((float  *)(im1->data.fl  +  im1->step/sizeof(float)  *(ROW)))
[(COL)]
#define  Im22Mat(ROW,COL)  ((float  *)(im2->data.fl  +  im2->step/sizeof(float)  *(ROW)))
[(COL)]
    cvZero(mosaic);

    /* Copy images into mosaic1. */
    for ( row = 0; row < im1->rows; row++)
        for ( col = 0; col < im1->cols; col++)
            Mosaic(row,col)= Im11Mat(row,col) ;
    for ( row = 0; row < im2->rows; row++)
        for ( col = 0; col < im2->cols; col++)
            Mosaic((row+im1->rows),col)=Im22Mat(row,col) ;

    return mosaic;
}
```

ok,为了版述清晰，再贴一下上文所述的主函数（注，上文已贴出，此是为了版述清晰，重复造轮）：

```
int main( void )
{
    //声明当前帧IplImage指针
    IplImage* src = NULL;
    IplImage* image1 = NULL;
    IplImage* grey_im1 = NULL;
    IplImage* DoubleSizeImage = NULL;

    IplImage* mosaic1 = NULL;
    IplImage* mosaic2 = NULL;

    CvMat* mosaicHorizen1 = NULL;
    CvMat* mosaicHorizen2 = NULL;
```

```
    CvMat* mosaicVertical1 = NULL;

    CvMat* image1Mat = NULL;
    CvMat* tempMat=NULL;

    ImageOctaves *Gaussianpyr;
    int rows,cols;

#define Im1Mat(ROW,COL) ((float *)(image1Mat->data.fl + image1Mat->step/sizeof(float)
*(ROW)))[(COL)]

    //灰度图象像素的数据结构
#define   Im1B(ROW,COL)   ((uchar*)(image1->imageData   +   image1->widthStep*(ROW)))
[(COL)*3]
#define   Im1G(ROW,COL)   ((uchar*)(image1->imageData   +   image1->widthStep*(ROW)))
[(COL)*3+1]
#define   Im1R(ROW,COL)   ((uchar*)(image1->imageData   +   image1->widthStep*(ROW)))
[(COL)*3+2]

    storage = cvCreateMemStorage(0);

    //读取图片
     if( (src = cvLoadImage( "street1.jpg", 1)) == 0 )   // test1.jpg einstein.pgm
back1.bmp
        return -1;

    //为图像分配内存
    image1 = cvCreateImage(cvSize(src->width, src->height),  IPL_DEPTH_8U,3);
    grey_im1 = cvCreateImage(cvSize(src->width, src->height),  IPL_DEPTH_8U,1);
         DoubleSizeImage   =   cvCreateImage(cvSize(2*(src->width),   2*(src->height)),
IPL_DEPTH_8U,3);

    //为图像阵列分配内存，假设两幅图像的大小相同，tempMat跟随image1的大小
    image1Mat = cvCreateMat(src->height, src->width, CV_32FC1);
    //转化成单通道图像再处理
    cvCvtColor(src, grey_im1, CV_BGR2GRAY);
    //转换进入Mat数据结构,图像操作使用的是浮点型操作
    cvConvert(grey_im1, image1Mat);

    double t = (double)cvGetTickCount();
    //图像归一化
    cvConvertScale( image1Mat, image1Mat, 1.0/255, 0 );

    int dim = min(image1Mat->rows, image1Mat->cols);
    numoctaves = (int) (log((double) dim) / log(2.0)) - 2;     //金字塔阶数
    numoctaves = min(numoctaves, MAXOCTAVES);

    //SIFT算法第一步,预滤波除噪声,建立金字塔底层
```

```
    tempMat = ScaleInitImage(image1Mat) ;
    //SIFT算法第二步,建立Guassian金字塔和DOG金字塔
    Gaussianpyr = BuildGaussianOctaves(tempMat) ;

    t = (double)cvGetTickCount() - t;
       printf( "the time of build Gaussian pyramid and DOG pyramid is %.1f/n",
t/(cvGetTickFrequency()*1000.) );

#define             ImLevels(OCTAVE,LEVEL,ROW,COL)             ((float             *)
(Gaussianpyr[(OCTAVE)].Octave[(LEVEL)].Level->data.fl                                  +
Gaussianpyr[(OCTAVE)].Octave[(LEVEL)].Level->step/sizeof(float) *(ROW)))[(COL)]
    //显示高斯金字塔
    for (int i=0; i<numoctaves;i++)
    {
        if (i==0)
        {
                        mosaicHorizen1=MosaicHorizen(  (Gaussianpyr[0].Octave)[0].Level,
(Gaussianpyr[0].Octave)[1].Level );
            for (int j=2;j<SCALESPEROCTAVE+3;j++)
                 mosaicHorizen1=MosaicHorizen( mosaicHorizen1, (Gaussianpyr[0].Octave)
[j].Level );
            for ( j=0;j<NUMSIZE;j++)
                mosaicHorizen1=halfSizeImage(mosaicHorizen1);
        }
        else if (i==1)
        {
                        mosaicHorizen2=MosaicHorizen(  (Gaussianpyr[1].Octave)[0].Level,
(Gaussianpyr[1].Octave)[1].Level );
            for (int j=2;j<SCALESPEROCTAVE+3;j++)
                 mosaicHorizen2=MosaicHorizen( mosaicHorizen2, (Gaussianpyr[1].Octave)
[j].Level );
            for ( j=0;j<NUMSIZE;j++)
                mosaicHorizen2=halfSizeImage(mosaicHorizen2);
            mosaicVertical1=MosaicVertical( mosaicHorizen1, mosaicHorizen2 );
        }
        else
        {
                        mosaicHorizen1=MosaicHorizen(  (Gaussianpyr[i].Octave)[0].Level,
(Gaussianpyr[i].Octave)[1].Level );
            for (int j=2;j<SCALESPEROCTAVE+3;j++)
                 mosaicHorizen1=MosaicHorizen( mosaicHorizen1, (Gaussianpyr[i].Octave)
[j].Level );
            for ( j=0;j<NUMSIZE;j++)
                mosaicHorizen1=halfSizeImage(mosaicHorizen1);
            mosaicVertical1=MosaicVertical( mosaicVertical1, mosaicHorizen1 );
        }
    }
     mosaic1 = cvCreateImage(cvSize(mosaicVertical1->width, mosaicVertical1->height),
```

```
IPL_DEPTH_8U,1);
    cvConvertScale( mosaicVertical1, mosaicVertical1, 255.0, 0 );
    cvConvertScaleAbs( mosaicVertical1, mosaic1, 1, 0 );

    //  cvSaveImage("GaussianPyramid of me.jpg",mosaic1);
    cvNamedWindow("mosaic1",1);
    cvShowImage("mosaic1", mosaic1);
    cvWaitKey(0);
    cvDestroyWindow("mosaic1");
    //显示DOG金字塔
    for ( i=0; i<numoctaves;i++)
    {
        if (i==0)
        {
                        mosaicHorizen1=MosaicHorizen(  (DOGoctaves[0].Octave)[0].Level,
(DOGoctaves[0].Octave)[1].Level );
            for (int j=2;j<SCALESPEROCTAVE+2;j++)
                 mosaicHorizen1=MosaicHorizen( mosaicHorizen1, (DOGoctaves[0].Octave)
[j].Level );
            for ( j=0;j<NUMSIZE;j++)
                mosaicHorizen1=halfSizeImage(mosaicHorizen1);
        }
        else if (i==1)
        {
                        mosaicHorizen2=MosaicHorizen(  (DOGoctaves[1].Octave)[0].Level,
(DOGoctaves[1].Octave)[1].Level );
            for (int j=2;j<SCALESPEROCTAVE+2;j++)
                 mosaicHorizen2=MosaicHorizen( mosaicHorizen2, (DOGoctaves[1].Octave)
[j].Level );
            for ( j=0;j<NUMSIZE;j++)
                mosaicHorizen2=halfSizeImage(mosaicHorizen2);
            mosaicVertical1=MosaicVertical( mosaicHorizen1, mosaicHorizen2 );
        }
        else
        {
                        mosaicHorizen1=MosaicHorizen(  (DOGoctaves[i].Octave)[0].Level,
(DOGoctaves[i].Octave)[1].Level );
            for (int j=2;j<SCALESPEROCTAVE+2;j++)
                 mosaicHorizen1=MosaicHorizen( mosaicHorizen1, (DOGoctaves[i].Octave)
[j].Level );
            for ( j=0;j<NUMSIZE;j++)
                mosaicHorizen1=halfSizeImage(mosaicHorizen1);
            mosaicVertical1=MosaicVertical( mosaicVertical1, mosaicHorizen1 );
        }
    }
    //考虑到DOG金字塔各层图像都会有正负,所以,必须寻找最负的,以将所有图像抬高一个台阶去显示
    double min_val=0;
    double max_val=0;
```

```
    cvMinMaxLoc( mosaicVertical1, &min_val, &max_val,NULL, NULL, NULL );
    if ( min_val<0.0 )
         cvAddS( mosaicVertical1, cvScalarAll( (-1.0)*min_val ), mosaicVertical1, NULL
);
     mosaic2 = cvCreateImage(cvSize(mosaicVertical1->width, mosaicVertical1->height),
IPL_DEPTH_8U,1);
    cvConvertScale( mosaicVertical1, mosaicVertical1, 255.0/(max_val-min_val), 0 );
    cvConvertScaleAbs( mosaicVertical1, mosaic2, 1, 0 );

    //  cvSaveImage("DOGPyramid of me.jpg",mosaic2);
    cvNamedWindow("mosaic1",1);
    cvShowImage("mosaic1", mosaic2);
    cvWaitKey(0);

    //SIFT算法第三步:特征点位置检测,最后确定特征点的位置
    int keycount=DetectKeypoint(numoctaves, Gaussianpyr);
    printf("the keypoints number are %d ;/n", keycount);
    cvCopy(src,image1,NULL);
    DisplayKeypointLocation( image1 ,Gaussianpyr);

    cvPyrUp( image1, DoubleSizeImage, CV_GAUSSIAN_5x5 );
    cvNamedWindow("image1",1);
    cvShowImage("image1", DoubleSizeImage);
    cvWaitKey(0);
    cvDestroyWindow("image1");

    //SIFT算法第四步:计算高斯图像的梯度方向和幅值,计算各个特征点的主方向
    ComputeGrad_DirecandMag(numoctaves, Gaussianpyr);
    AssignTheMainOrientation( numoctaves, Gaussianpyr,mag_pyr,grad_pyr);
    cvCopy(src,image1,NULL);
    DisplayOrientation ( image1, Gaussianpyr);

    //  cvPyrUp( image1, DoubleSizeImage, CV_GAUSSIAN_5x5 );
    cvNamedWindow("image1",1);
    //  cvResizeWindow("image1", 2*(image1->width), 2*(image1->height) );
    cvShowImage("image1", image1);
    cvWaitKey(0);

    //SIFT算法第五步:抽取各个特征点处的特征描述字
    ExtractFeatureDescriptors( numoctaves, Gaussianpyr);
    cvWaitKey(0);

    //销毁窗口
    cvDestroyWindow("image1");
    cvDestroyWindow("mosaic1");
    //释放图像
    cvReleaseImage(&image1);
    cvReleaseImage(&grey_im1);
```

```
    cvReleaseImage(&mosaic1);
    cvReleaseImage(&mosaic2);
    return 0;
}
```

最后，再看一下，运行效果（图中美女为老乡+朋友，何姐08年照）：

完。

updated

有很多朋友都在本文评论下要求要本程序的完整源码包（注：本文代码未贴全，复制粘贴编译肯定诸多错误），但由于时隔太久，这份代码我自己也找不到了，不过，我可以提供一份sift + KD + BBF，且可以编译正确的代码供大家参考学习，有pudn帐号的朋友可以前去下载：http://www.pudn.com/downloads340/sourcecode/graph/texture_mapping/detail1486667.html（没有pudn账号的同学请加群：169056165，验证信息：sift，至群共享下载），然后用两幅不同的图片做了下匹配（当然，运行结果显示是不匹配的），效果还不错：http://weibo.com/1580904460/yDmzAEwcV#1348475194313! July、二零一二年十月十一日。

十、从头到尾彻底理解傅里叶变换算法、上

推荐阅读：*The Scientist and Engineer's Guide to Digital Signal Processing*，By Steven W. Smith, Ph.D。此书地址：。

博主说明：
I、本文中阐述离散傅里叶变换方法，是根据此书：The Scientist and Engineer's Guide to Digital Signal Processing，By Steven W. Smith, Ph.D.而翻译而成的，此书地址：。
II、同时，有相当一部分内容编辑整理自dznlong的博客，也贴出其博客地

址，向原创的作者表示致敬：http://blog.csdn.net/dznlong 。这年头，真正静下心写来原创文章的人，很少了。

从头到尾彻底理解傅里叶变换算法、上
前言
第一章、傅立叶变换的由来
第二章、实数形式离散傅立叶变换（Real DFT）

从头到尾彻底理解傅里叶变换算法、下
第三章、复数
第四章、复数形式离散傅立叶变换

前言：

“关于傅立叶变换，无论是书本还是在网上可以很容易找到关于傅立叶变换的描述，但是大都是些故弄玄虚的文章，太过抽象，尽是一些让人看了就望而生畏的公式的罗列，让人很难能够从感性上得到理解”—dznlong

那么，到底什么是傅里叶变换算法列?傅里叶变换所涉及到的公式具体有多复杂列?
傅里叶变换（Fourier transform）是一种线性的积分变换。因其基本思想首先由法国学者傅里叶系统地提出，所以以其名字来命名以示纪念。

哦，傅里叶变换原来就是一种变换而已，只是这种变换是从时间转换为频率的变化。这下，你就知道了，傅里叶就是一种变换，一种什么变换列?就是一种从时间到频率的变化或其相互转化。

ok，咱们再来总体了解下傅里叶变换，让各位对其有个总体大概的印象，也顺便看看傅里叶变换所涉及到的公式，究竟有多复杂：
以下就是傅里叶变换的4种变体（摘自，维基百科）:

连续傅里叶变换

一般情况下，若“傅里叶变换”一词不加任何限定语，则指的是“连续傅里叶变换”。连续傅里叶变换将平方可积的函数f(t)表示成复指数函数的积分或级数形式。

$$F(\omega) = \mathcal{F}[f(t)] = \int_{-\infty}^{\infty} f(t) e^{-i\omega t}\,dt.$$

这是将频率域的函数F(ω)表示为时间域的函数f(t)的积分形式。

连续傅里叶变换的逆变换 (inverse Fourier transform)为：

$$f(t) = \mathcal{F}^{-1}[F(\omega)] = \frac{1}{2\pi} \int_{-\infty}^{\infty} F(\omega) e^{i\omega t}\,d\omega.$$

即将时间域的函数f(t)表示为频率域的函数F(ω)的积分。

一般可称函数**f(t)**为原函数，而称函数**F(ω)**为傅里叶变换的像函数，原函数和像函数构成一个傅里叶变换对（transform pair）。

除此之外，还有其它型式的变换对，以下两种型式亦常被使用。在通信或是信号处理方面，常以$f = \frac{\omega}{2\pi}$来代换，而形成新的变换对:

$$X(f) = \mathcal{F}[x(t)] = \int_{-\infty}^{\infty} x(t)\ e^{-i2\pi f t}\,dt$$

$$x(t) = \mathcal{F}^{-1}[X(f)] = \int_{-\infty}^{\infty} X(f) e^{i2\pi f t}\,df.$$

或者是因系数重分配而得到新的变换对：

$$F(\omega) = \mathcal{F}[f(t)] = \int_{-\infty}^{\infty} f(t) e^{-i\omega t}\,dt$$

$$f(t) = \mathcal{F}^{-1}[F(\omega)] = \frac{1}{2\pi} \int_{-\infty}^{\infty} F(\omega) e^{i\omega t}\,d\omega.$$

一种对连续傅里叶变换的推广称为分数傅里叶变换（Fractional Fourier Transform）。分数傅里叶变换(fractional Fourier transform,FRFT)指的就是傅里叶变换(Fourier transform,FT)的广义化。
分数傅里叶变换的物理意义即做傅里叶变换 a 次，其中 a 不一定要为整数；而做了分数傅里叶变换之后，信号或输入函数便会出现在介于时域(time domain)与频域(frequency domain)之间的分数域(fractional domain)。

当f(t)为偶函数（或奇函数）时，其正弦（或余弦）分量将消亡，而可以称这时的变换为余弦变换（cosine transform）或正弦变换（sine transform）.

另一个值得注意的性质是，当f(t)为纯实函数时，F(−ω) = F*(ω)成立.

傅里叶级数

连续形式的傅里叶变换其实是傅里叶级数 (Fourier series)的推广，因为积分其实是一种极限形式的求和算子而已。对于周期函数，其傅里叶级数是存在的：

$$f(x) = \sum_{n=-\infty}^{\infty} F_n\, e^{inx},$$

其中Fn为复幅度。对于实值函数，函数的傅里叶级数可以写成：

$$f(x) = a_0 + \sum_{n=1}^{\infty} \left[a_n \cos(nx) + b_n \sin(nx)\right]$$

其中an和bn是实频率分量的幅度。

离散时域傅里叶变换

离散傅里叶变换是离散时间傅里叶变换(DTFT)的特例(有时作为后者的近似)。DTFT在时域上离散,在频域上则是周期的。DTFT可以被看作是傅里叶级数的逆变换。

离散傅里叶变换

离散傅里叶变换(DFT),是连续傅里叶变换在时域和频域上都离散的形式,将时域信号的采样变换为在离散时间傅里叶变换(DTFT)频域的采样。在形式上,变换两端(时域和频域上)的序列是有限长的,而实际上这两组序列都应当被认为是离散周期信号的主值序列。即使对有限长的离散信号作DFT,也应当将其看作经过周期延拓成为周期信号再作变换。在实际应用中通常采用快速傅里叶变换以高效计算DFT。

为了在科学计算和数字信号处理等领域使用计算机进行傅里叶变换,必须将函数xn定义在离散点而非连续域内,且须满足有限性或周期性条件。这种情况下,使用*离散傅里叶变换(DFT)*,将函数xn表示为下面的求和形式:

$$x_n = \sum_{k=0}^{N-1} X_k e^{i\frac{2\pi}{N}kn} \qquad n = 0, \ldots, N-1$$

其中Xk是傅里叶幅度。直接使用这个公式计算的计算复杂度为O(n*n),而*快速傅里叶变换(FFT)可以将复杂度改进为O(n*lgn)*。(后面会具体阐述FFT是如何将复杂度降为O(n*lgn)的。)计算复杂度的降低以及数字电路计算能力的发展使得DFT成为在信号处理领域十分实用且重要的方法。

下面,比较下上述傅立叶变换的4种变体,

变换	时间	频率
连续傅里叶变换	连续,非周期性	连续,非周期性
傅里叶级数	连续,周期性	离散,非周期性
离散时间傅里叶变换	离散,非周期性	连续,周期性

离散傅里叶变换　　离散，周期性　　离散，周期性

如上，容易发现：函数在时（频）域的离散对应于其像函数在频（时）域的周期性。反之连续则意味着在对应域的信号的非周期性。也就是说，时间上的离散性对应着频率上的周期性。同时，注意，离散时间傅里叶变换，时间离散，频率不离散，它在频域依然是连续的。

如果，读到此，你不甚明白，大没关系，不必纠结于以上4种变体，继续往下看，你自会豁然开朗。（有什么问题，也恳请提出，或者批评指正）

ok，本文，接下来，由傅里叶变换入手，后重点阐述离散傅里叶变换、快速傅里叶算法，到最后彻底实现FFT算法，全篇力求通俗易懂、阅读顺畅，教你从头到尾彻底理解傅里叶变换算法。由于傅里叶变换，也称傅立叶变换，下文所称为*傅立叶变换*，同一个变换，不同叫法，读者不必感到奇怪。

第一章、傅立叶变换的由来

要理解傅立叶变换，先得知道傅立叶变换是怎么变换的，当然，也需要一定的高等数学基础，最基本的是级数变换，其中傅立叶级数变换是傅立叶变换的基础公式。

一、傅立叶变换的提出

傅立叶是一位法国数学家和物理学家，原名是Jean Baptiste Joseph Fourier(1768-1830), Fourier于1807年在法国科学学会上发表了一篇论文，论文里描述运用正弦曲线来描述温度分布，论文里有个在当时具有争议性的决断：任何连续周期信号都可以由一组适当的正弦曲线组合而成。

当时审查这个论文拉格朗日坚决反对此论文的发表，而后在近50年的时间里，拉格朗日坚持认为傅立叶的方法无法表示带有棱角的信号，如在方波中出现非连续变化斜率。直到拉格朗日死后15年这个论文才被发表出来。

谁是对的呢？拉格朗日是对的：正弦曲线无法组合成一个带有棱角的信号。但是，我们可以用正弦曲线来非常逼近地表示它，逼近到两种表示方法不存在能量差别，基于此，傅立叶是对的。

为什么我们要用正弦曲线来代替原来的曲线呢？如我们也还可以用方波或三角波来代替呀，分解信号的方法是无穷多的，但分解信号的目的是为了更加简单地处理原来的信号。

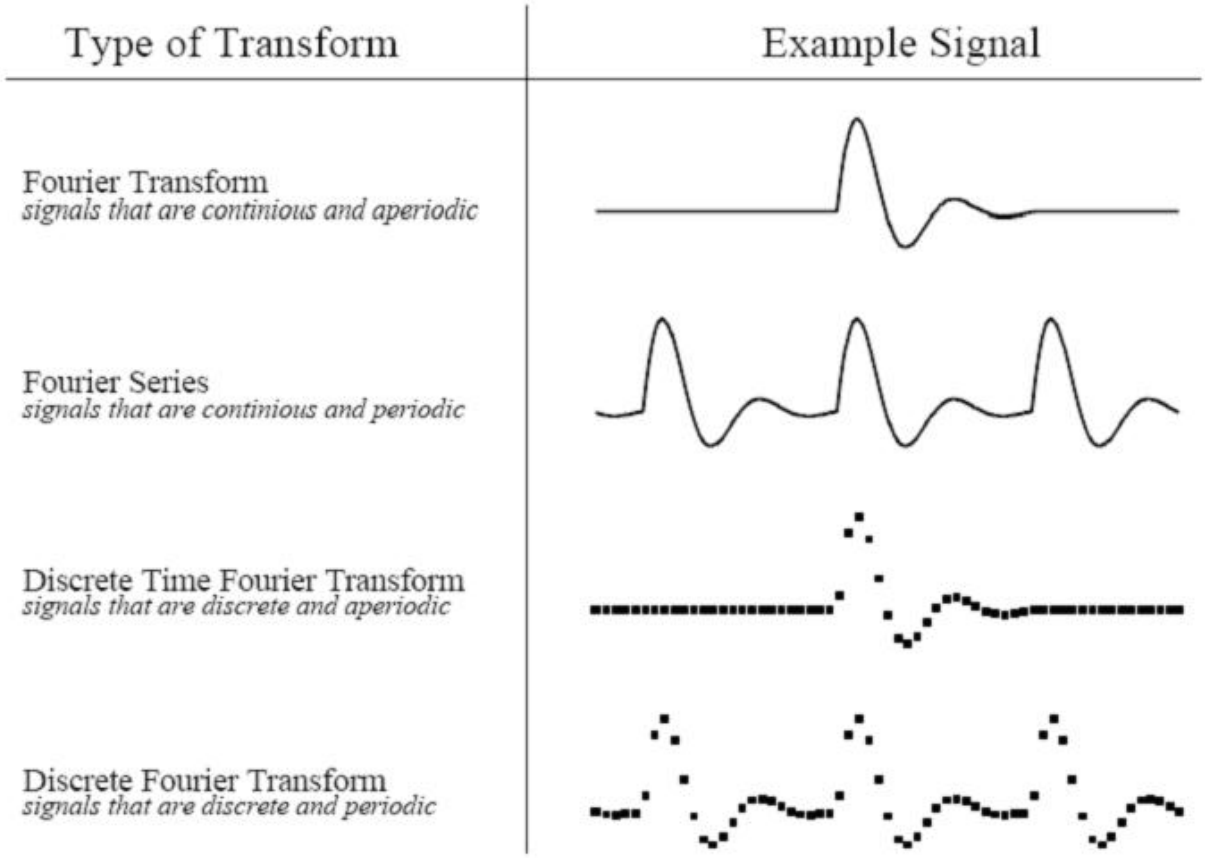
用正余弦来表示原信号会更加简单，因为正余弦拥有原信号所不具有的性质：正弦曲线保真度。一个正余弦曲线信号输入后，输出的仍是正余弦曲线，只有幅度和相位可能发生变化，但是频率和波的形状仍是一样的。且只有正余弦曲线才拥有这样的性质，正因如此我们才不用方波或三角波来表示。

二、傅立叶变换分类

根据原信号的不同类型，我们可以把傅立叶变换分为四种类别：

1. 非周期性连续信号 傅立叶变换（Fourier Transform）
2. 周期性连续信号 傅立叶级数(Fourier Series)
3. 非周期性离散信号 离散*时域*傅立叶变换（Discrete Time Fourier Transform）
4. 周期性离散信号 离散傅立叶变换(Discrete Fourier Transform)

下图是四种原信号图例（从上到下，依次是FT，FS，DTFT，DFT）：



这四种傅立叶变换都是针对正无穷大和负无穷大的信号，即信号的的长度是无穷大的，我们知道这对于计算机处理来说是不可能的，**那么有没有针对长度有限的傅立叶变换呢？**没有。因为正余弦波被定义成从负无穷小到正无穷大，我们无法把一个长度无限的信号组合成长度有限的信号。

面对这种困难，方法是：把长度有限的信号表示成长度无限的信号。如，可以把信号无限地从左右进行延伸，延伸的部分用零来表示，这样，这个信号就可以被看成是*非周期性*离散信号，我们可以用到*离散时域傅立叶变换（DTFT）*的方法。也可以把信号用复制的方法进行延伸，这样信号就变成了*周期性*离散信号，这时我们就可以用*离散傅立叶变换方法（DFT）*进行变换。本章我们要讲的是离散信号，对于连续信号我们不作讨论，因为计算机只能处理离散的数值信号，我们的最终目的是运用计算机来处理信号的。

但是对于非周期性的信号，我们需要用无穷多不同频率的正弦曲线来表示，这对于计算机来说是不可能实现的。所以对于离散信号的变换只有

离散傅立叶变换（DFT）才能被适用，对于计算机来说只有离散的和有限长度的数据才能被处理，对于其它的变换类型只有在数学演算中才能用到，在计算机面前我们只能用DFT方法，后面我们要理解的也正是DFT方法。

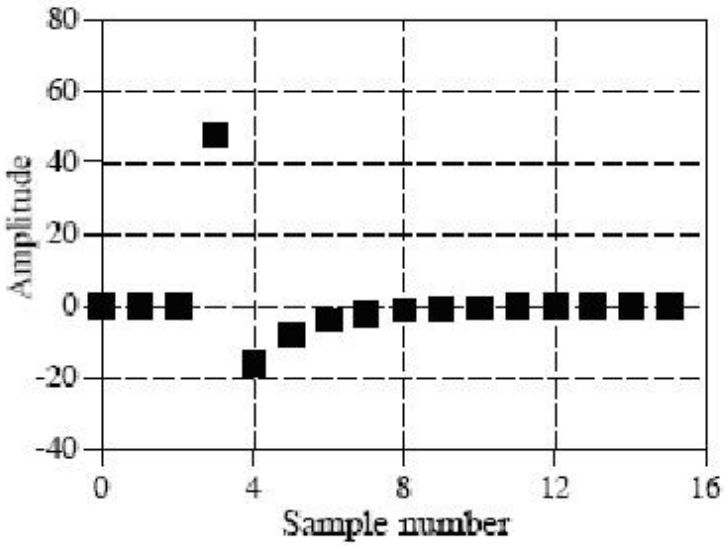
这里要理解的是我们使用周期性的信号目的是为了能够用数学方法来解决问题，至于考虑周期性信号是从哪里得到或怎样得到是无意义的。

每种傅立叶变换都分成实数和复数两种方法，对于实数方法是最好理解的，但是复数方法就相对复杂许多了，需要懂得有关复数的理论知识，不过，如果理解了实数离散傅立叶变换(real DFT)，再去理解复数傅立叶变换就更容易了，所以我们先把复数的傅立叶变换放到一边去，先来理解实数傅立叶变换，在后面我们会先讲讲关于复数的基本理论，然后在理解了实数傅立叶变换的基础上再来理解复数傅立叶变换。

还有，这里我们所要说的变换(transform)虽然是数学意义上的变换，但跟函数变换是不同的，函数变换是符合一一映射准则的，对于离散数字信号处理（DSP），有许多的变换：傅立叶变换、拉普拉斯变换、Z变换、希尔伯特变换、离散余弦变换等，这些都扩展了函数变换的定义，允许输入和输出有多种的值，简单地说变换就是把一堆的数据变成另一堆的数据的方法。

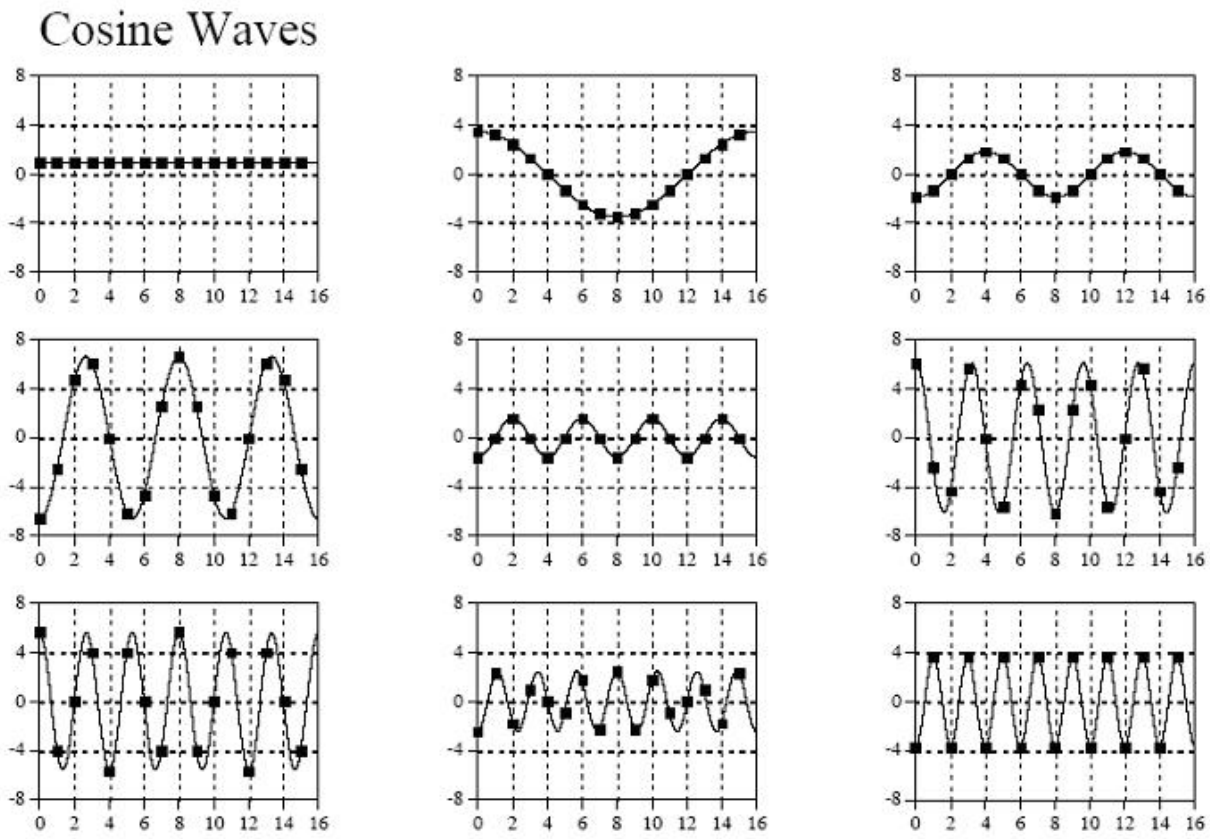
三、一个关于实数离散傅立叶变换(Real DFT)的例子

先来看一个变换实例，下图是一个原始信号图像：

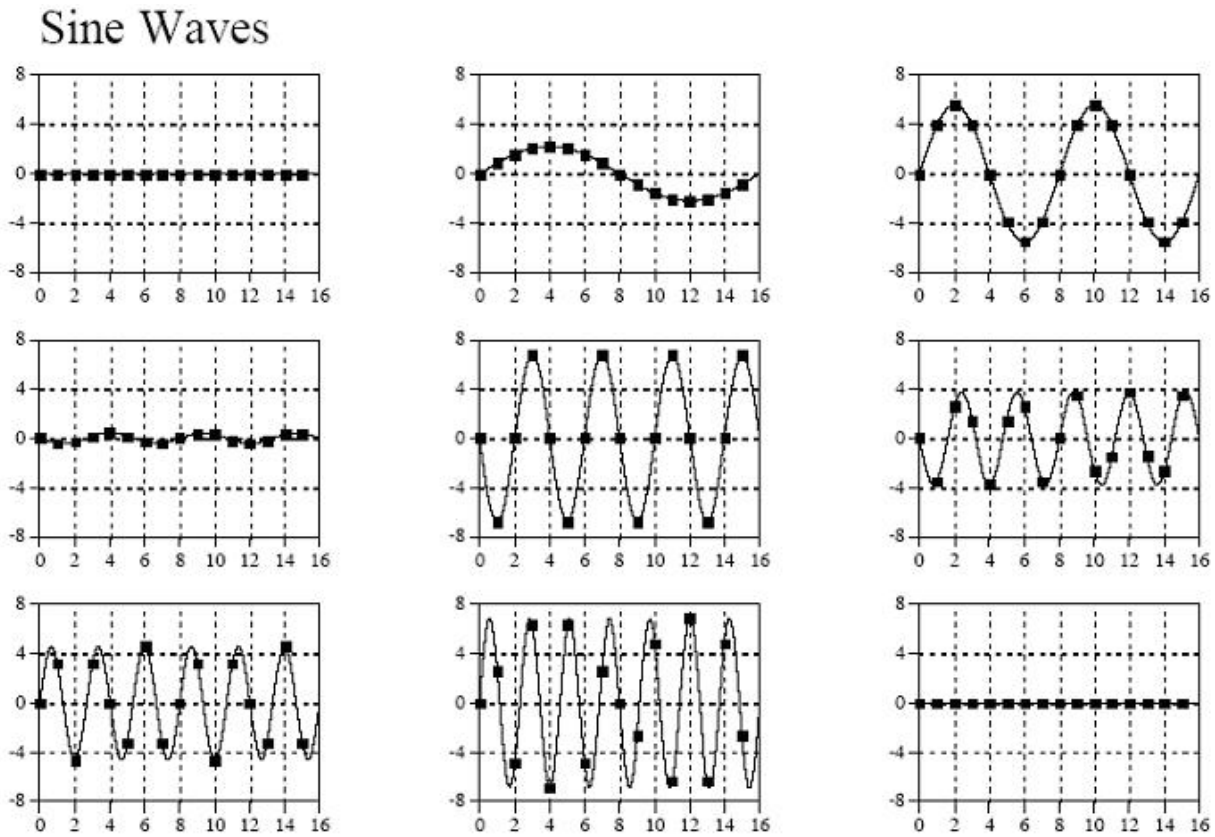


这个信号的长度是16，于是可以把这个信号分解9个余弦波和9个正弦波（一个长度为N的信号可以分解成N/2+1个正余弦信号，这是为什么呢？结合下面的18个正余弦图,我想从计算机处理精度上就不难理解，一个长度为N的信号，最多只能有N/2+1个不同频率，再多的频率就超过了计算机所能所处理的精度范围），如下图：

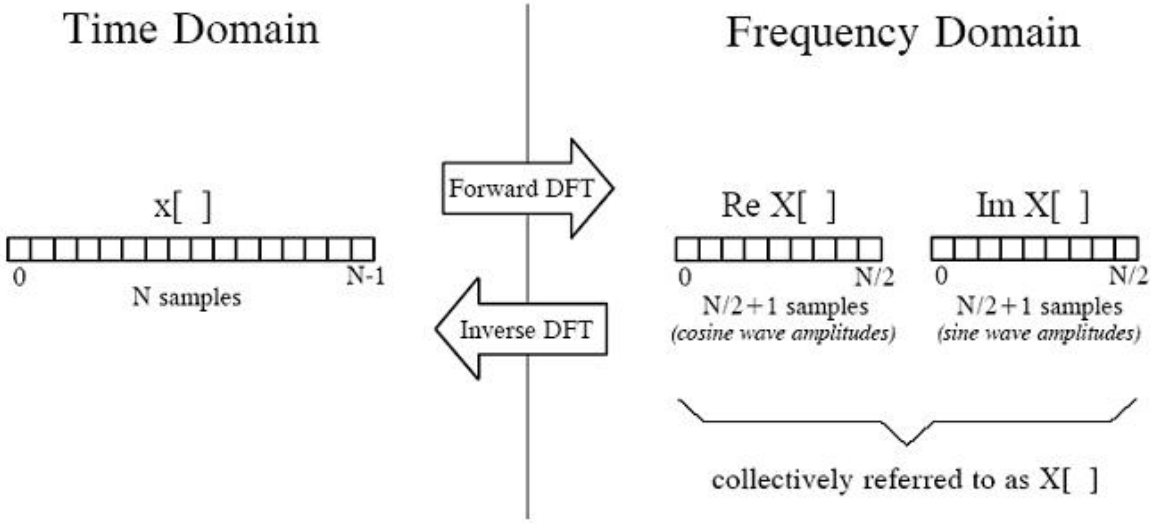
9个余弦信号：



9个正弦信号：



把以上所有信号相加即可得到原始信号，至于是怎么分别变换出9种不同频率信号的，我们先不急，先看看对于以上的变换结果，在程序中又是该怎么表示的，我们可以看看下面这个示例图：



上图中左边表示时域中的信号，右边是频域信号表示方法，

从左向右，–>，表示**正向转换**(Forward DFT)，从右向左，<–，表示**逆向转换**(Inverse DFT)，用小写x[]表示信号在每个时间点上的幅度值数组，用大写X[]表示每种频率的副度值数组（即时间x–>频率**X**），

因为有N/2+1种频率，所以该数组长度为N/2+1，

X[]数组又分两种，一种是表示余弦波的不同频率幅度值：Re X[]，

另一种是表示正弦波的不同频率幅度值：Im X[]，

Re是实数(Real)的意思，Im是虚数(Imagine)的意思，采用复数的表示方法把正余弦波组合起来进行表示，但这里我们不考虑复数的其它作用，只记住是一种组合方法而已，目的是为了便于表达（在后面我们会知道，复数形式的傅立叶变换长度是N，而不是N/2+1）。如此，再回过头去，看上面的正余弦各9种频率的变化，相信，问题不大了。

第二章、实数形式离散傅立叶变换（Real DFT）

上一章，我们看到了一个实数形式离散傅立叶变换的例子，通过这个例子能够让我们先对傅立叶变换有一个较为形象的感性认识，现在就让我们来看看实数形式离散傅立叶变换的正向和逆向是怎么进行变换的。在此，我们先来看一下频率的多种表示方法。

一、频域中关于频率的四种表示方法

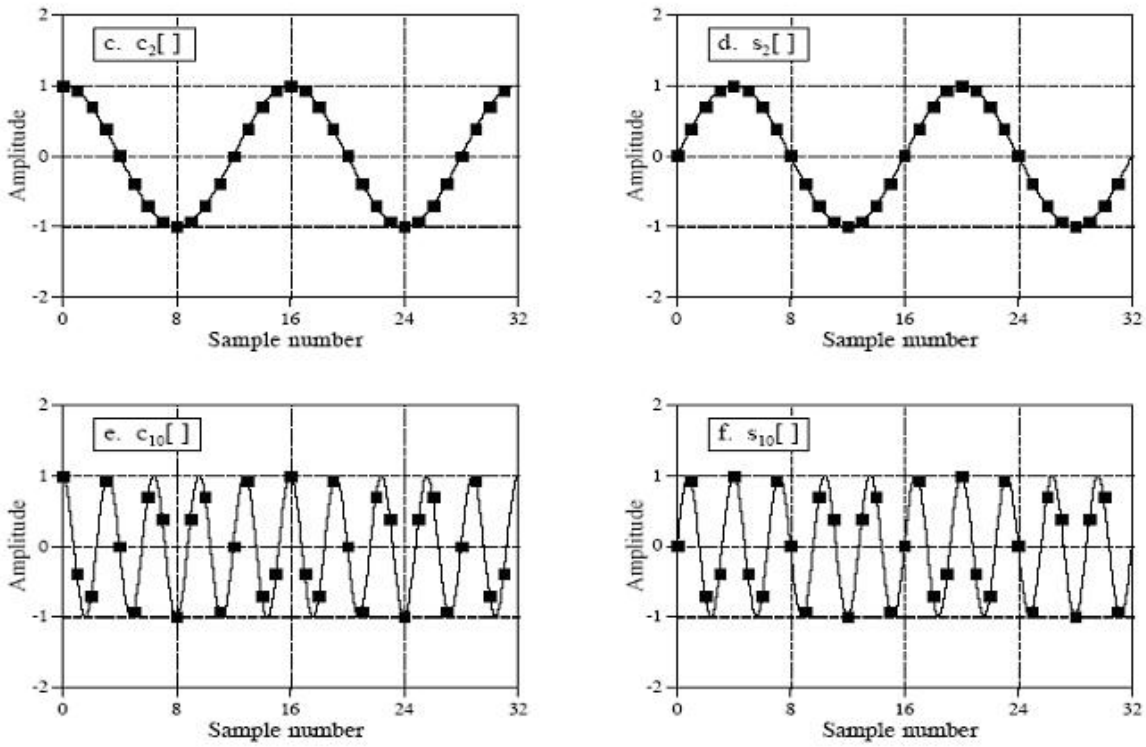
1. 序号表示方法，根据时域中信号的样本数取0 ~ N/2，用这种方法在程序中使用起来可以更直接地取得每种频率的幅度值，因为频率值跟数组的序号是一一对应的: X[k]，取值范围是0 ~ N/2；
2. 分数表示方法，根据时域中信号的样本数的比例值取0 ~ 0.5: X[f]，f = k/N，取值范围是0 ~ 1/2；
3. 用弧度值来表示，把f乘以一个2π得到一个弧度值，这种表示方法叫做自然频率(natural frequency)：X[ω]，ω = 2πf = 2πk/N，取值范围是0 ~ π；
4. 以赫兹(Hz)为单位来表示，这个一般是应用于一些特殊应用，如取样率为10 kHz表示每秒有10,000个样本数：取值范围是0到取样率的

一半。

二、DFT基本函数

ck[i] = cos(2πki/N) sk[i] = sin(2πki/N)

其中k表示每个正余弦波的频率，如为2表示在0到N长度中存在两个完整的周期，10即有10个周期，如下图：



上图中至于每个波的振幅(amplitude)值(Re X[k],Im X[k])是怎么算出来的,这个是DFT的核心，也是最难理解的部分，我们先来看看如何把分解出来的正余弦波合成原始信号(Inverse DFT)。

三、合成运算方法(Real Inverse DFT)

DFT合成等式（合成原始***时间*信号，频率–>时间，逆向变换**）：

$$x[i] = \sum_{k=0}^{N/2} Re\bar{X}[k]\cos(2\pi ki/N) + \sum_{k=0}^{N/2} Im\bar{X}[k]\sin(2\pi ki/N)$$

如果有学过傅立叶级数,对这个等式就会有似曾相识的感觉,不错!这个等式跟傅立叶级数是非常相似的:

$$f(x) = \frac{a_0}{2} + \sum_{k=1}^{\infty}(a_k \cos kx + b_k \sin kx)$$

当然,差别是肯定是存在的,因为这两个等式是在两个不同条件下运用的,至于怎么证明DFT合成公式,这个我想需要非常强的高等数学理论知识了,这是研究数学的人的工作,对于普通应用者就不需要如此的追根究底了,但是傅立叶级数是好理解的,我们起码可以从傅立叶级数公式中看出DFT合成公式的合理性。

DFT合成等式中的 $Im\ \bar{X}[k]$ 和 $Re\ \bar{X}[k]$ *跟之前提到的* Im X[k]和Re X[k]是不一样的,下面是转换方法(*关于此公式的解释,见下文*):

$$Re\bar{X}[k] = \frac{ReX[k]}{N/2}$$

$$Im\bar{X}[k] = -\frac{ImX[k]}{N/2}$$

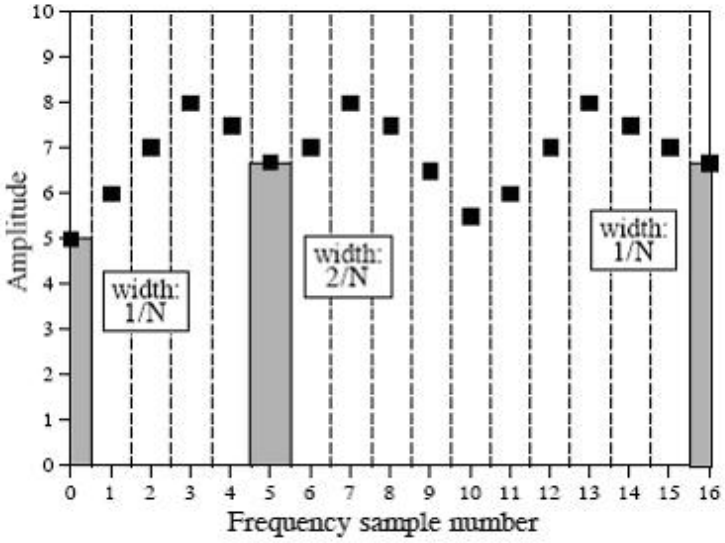
但k等于0和N/2时,实数部分的计算要用下面的等式:

$$Re\bar{X}[0] = \frac{ReX[0]}{N}$$

$$Re\bar{X}[N/2] = \frac{ReX[N/2]}{N}$$

上面四个式中的N是时域中点的总数，k是从0到N/2的序号。

为什么要这样进行转换呢？这个可以从频谱密度(spectral density)得到理解，如下图就是个频谱图：



这是一个频谱图，横坐标表示频率大小，纵坐标表示振幅大小，原始信号长度为N（这里是32），经DFT转换后得到的**17**个频率的频谱，频谱密度表示每单位带宽中为多大的振幅，那么带宽是怎么计算出来的呢？看上图，除了头尾两个，其余点的所占的宽度是2/N，这个宽度便是每个点的带宽，头尾两个点的带宽是1/N，而Im X[k]和Re X[k]表示的是频谱密度，即每一个单位带宽的振幅大小，但 $\mathrm{Im}\ \bar{X}[k]$和 $\mathrm{Re}\ \bar{X}[k]$ 表示2/N（或1/N）带宽的振幅大小， **所以** $\mathrm{Im}\ \bar{X}[k]$和 $\mathrm{Re}\ \bar{X}[k]$**分别应当是Im X[k]和Re X[k]的2/N（或1/N）**。

频谱密度就象物理中物质密度，原始信号中的每一个点就象是一个混合物，这个混合物是由不同密度的物质组成的，混合物中含有的每种物质的质量是一样的，除了最大和最小两个密度的物质外，这样我们只要把每种物质的密度加起来就可以得到该混合物的密度了，又该混合物的质量是单位质量，所以得到的密度值跟该混合物的质量值是一样的。

至于为什么虚数部分是负数，这是为了跟复数DFT保持一致，这个我们将在后面会知道这是数学计算上的需要（Im X[k]在计算时就已经加上了一个负号（*稍后，由下文，便可知*），$\mathrm{Im}\ \bar{X}[k]$再加上负号，结果便是正的，等于没有变化）。

如果已经得到了DFT结果，这时要进行**逆转换，即合成原始信号**，则可按如下步骤进行转换：

1. 先根据上面四个式子计算得出$\mathrm{Im}\ \bar{X}[k]$和$\mathrm{Re}\ \bar{X}[k]$的值；
2. 再根据DFT合成等式得到原始信号数据。

下面是用BASIC语言来实现的转换源代码：

```
'DFT逆转换方法 '/XX[]数组存储计算结果（时域中的原始信号） '/REX[]数组存储频域中的实数分量，IMX[]为虚分量 ' DIM XX[511] DIM REX[256] DIM IMX[256] ' PI = 3.14159265 N% = 512 ' GOSUB XXXX '转到子函数去获取REX[]和IMX[]数据 ' ' ' FOR K% = 0 TO 256 REX[K%] = REX[K%] / (N%/2) IMX[K%] = -IMX[K%] / (N%/2) NEXT k% ' REX[0] = REX[0] / N REX[256] = REX[256] / N ' ' 初始化XX[]数组 FOR I% = 0 TO 511 XX[I%] = 0 NEXT I% ' ' ' ' ' FOR K% =0 TO 256 FOR I%=0 TO 511 ' XX[I%] = XX[I%] + REX[K%] * COS(2 * PI * K% * I% / N%) XX[I%] = XX[I%] + IMX[K%] * SIN(2 * PI * K% * I% / N%) ' NEXT I% NEXT K% ' END
```

上面代码中420至490换成如下形式也许更好理解，但结果都是一样的：

```
FOR I% =0 TO 511 FOR K%=0 TO 256 ' XX[I%] = XX[I%] + REX[K%] * COS(2 * PI * K% * I% / N%) XX[I%] = XX[I%] + IMX[K%] * SIN(2 * PI * K% * I% / N%) ' NEXT I% NEXT K%
```

四、分解运算方法（DFT）

有三种完全不同的方法进行DFT：*一种方法是通过联立方程进行求解*，从代数的角度看，要从N个已知值求N个未知值，需要N个联立方程，且N个联立方程必须是线性独立的，但这是这种方法计算量非常的大且极其复杂，所以很少被采用；*第二种方法是利用信号的相关性（correlation）*进行计算，这个是我们后面将要介绍的方法；*第三种方法是快速傅立叶变换（FFT）*，这是一个非常具有创造性和革命性的的方法，因为它大大提高了运算速度，使得傅立叶变换能够在计算机中被广泛应用，但这种算法是根据复数形式的傅立叶变换来实现的，它把N个点的信号分解

成长度为N的频域，这个跟我们现在所进行的实域DFT变换不一样，而且这种方法也较难理解，这里我们先不去理解，等先理解了复数DFT后，再来看一下FFT。有一点很重要，那就是这三种方法所得的变换结果是一样的，经过实践证明，当频域长度为32时，利用相关性方法进行计算效率最好，否则FFT算法效率较高。现在就让我们来看一下相关性算法。

利用**第一种方法、信号的相关性**(correlation)可以从噪声背景中检测出已知的信号，我们也可以利用这个方法检测信号波中是否含有某个频率的信号波：把一个待检测信号波乘以另一个信号波，得到一个新的信号波，再把这个新的信号波所有的点进行相加，从相加的结果就可以判断出这两个信号的相似程度。如下图：

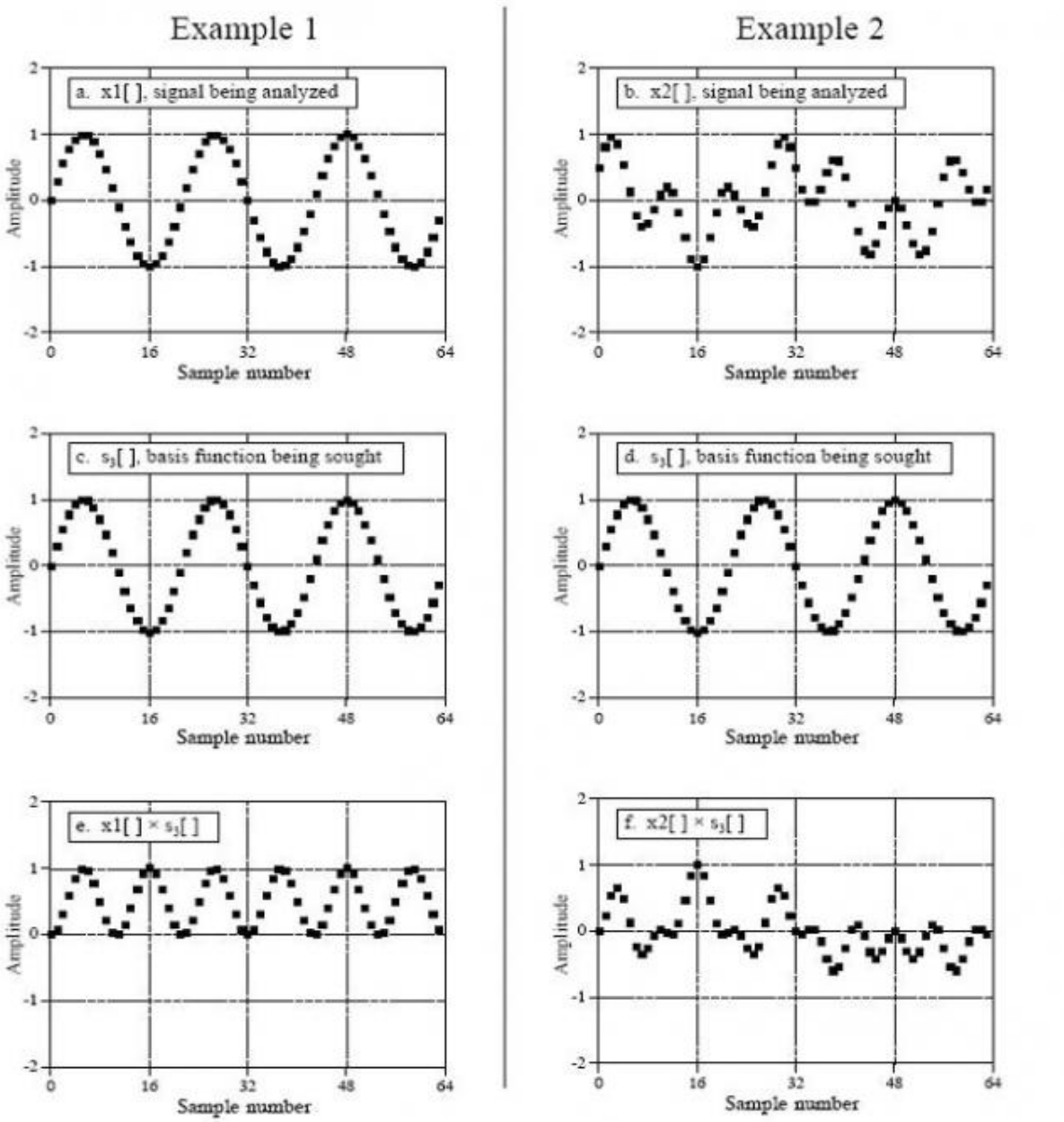


FIGURE 8-8
Two example signals, (a) and (b), are analyzed for containing the specific basis function shown in (c) and (d). Figures (e) and (f) show the result of multiplying each example signal by the basis function. Figure (e) has an average of 0.5, indicating that *x*1[ ]contains the basis function with an amplitude of 1.0. Conversely, (f) has a zero average, indicating that *x*2[ ] does not contain the basis function.

上面a和 b两个图是待检测信号波，图a很明显可以看出是个3个周期的正弦信号波，图b的信号波则看不出是否含有正弦或余弦信号，图c和d都是个3个周期的正弦信号波，图e和f分别是a、b两图跟c、d两图相乘后的结果，图e所有点的平均值是0.5，说明信号a含有振幅为1的正弦信号c，但

图f所有点的平均值是0,则说明信号b不含有信号d。这个就是通过信号相关性来检测是否含有某个信号的方法。

第二种方法: 相应地,我也可以通过把输入信号和每一种频率的正余弦信号进行相乘(**关联操作**),从而得到原始信号与每种频率的关联程度(即总和大小),这个结果便是我们所要的傅立叶变换结果,下面两个等式便是我们所要的计算方法:

$$ReX[k] = \sum_{i=0}^{N-1} x[i] \cos(2\pi k i / N)$$

$$ImX[k] = -\sum_{i=0}^{N-1} x[i] \sin(2\pi k i / N)$$

第二个式子中加了个负号,是为了保持复数形式的一致,*前面我们知道在计算* $\mathrm{Im}\ \bar{X}[k]$ *时又加了个负号*,所以这只是个形式的问题,并没有实际意义,你也可以把负号去掉,并在计算 $\mathrm{Im}\ \bar{X}[k]$ 时也不加负号。

这里有一点必须明白一个正交的概念:两个函数相乘,如果结果中的每个点的总和为0,则可认为这两个函数为正交函数。要确保关联性算法是正确的,则必须使得跟原始信号相乘的信号的函数形式是正交的,我们知道所有的正弦或余弦函数是正交的,这一点我们可以通过简单的高数知识就可以证明它,所以我们可以通过关联的方法把原始信号分离出正余弦信号。当然,其它的正交函数也是存在的,如:方波、三角波等形式的脉冲信号,所以原始信号也可被分解成这些信号,但这只是说可以这样做,却是没有用的。

下面是实域傅立叶变换的BASIC语言代码:

```
100 'THE DISCRETE FOURIER TRANSFORM
110 'The frequency domain signals, held in REX[ ] and IMX[ ], are calculated from
120 'the time domain signal, held in XX[ ].
130 '
140 DIM XX[511]                    'XX[ ]  holds the time domain signal
150 DIM REX[256]                  'REX[ ] holds the real part of the frequency domain
160 DIM IMX[256]                  'IMX[ ] holds the imaginary part of the frequency domain
170 '
180 PI = 3.14159265               'Set the constant, PI
190 N% = 512                      'N% is the number of points in XX[ ]
200 '
210 GOSUB XXXX                    'Mythical subroutine to load data into XX[ ]
220 '
230 '
240 FOR K% = 0 TO 256   'Zero REX[ ] & IMX[ ] so they can be used as accumulators
250   REX[K%] = 0
260   IMX[K%] = 0
270 NEXT K%
280 '
290 '                             'Correlate XX[ ] with the cosine and sine waves, Eq. 8-4
300 '
310 FOR K% = 0 TO 256             'K% loops through each sample in REX[ ] and IMX[ ]
320   FOR I% = 0 TO 511   'I% loops through each sample in XX[ ]
330     '
340     REX[K%] = REX[K%] + XX[I%] * COS(2*PI*K%*I%/N%)
350     IMX[K%] = IMX[K%]  - XX[I%] * SIN(2*PI*K%*I%/N%)
360     '
370   NEXT I%
380 NEXT K%
390 '
400 END
```

到此为止，我们对傅立叶变换便有了感性的认识了吧。但要记住，这只是在实域上的离散傅立叶变换，其中虽然也用到了复数的形式，但那只是个替代的形式，并无实际意义，现实中一般使用的是复数形式的离散傅立叶变换，且**快速傅立叶变换**是根据复数离散傅立叶变换来设计算法的，在后面我们先来复习一下有关复数的内容，然后再在理解实域离散傅立叶变换的基础上来理解复数形式的离散傅立叶变换。**更多见下文：**

十、从头到尾彻底理解傅里叶变换算法、下（July、dznlong）

十、从头到尾彻底理解傅里叶变换算法、下

推荐阅读：*The Scientist and Engineer's Guide to Digital Signal Processing*，By Steven W. Smith, Ph.D。**此书地址：**。

从头到尾彻底理解傅里叶变换算法、上
前言
第一章、傅立叶变换的由来
第二章、实数形式离散傅立叶变换（Real DFT）

从头到尾彻底理解傅里叶变换算法、下
第三章、复数
第四章、复数形式离散傅立叶变换

前期回顾，在上一篇：十、从头到尾彻底理解傅里叶变换算法、上里，我们讲了傅立叶变换的由来、和实数形式离散傅立叶变换（Real DFT）俩个问题，

本文接上文，着重讲下复数、和复数形式离散傅立叶变换等俩个问题。

第三章、复数

复数扩展了我们一般所能理解的数的概念，复数包含了实数和虚数两部分，利用复数的形式可以把由两个变量表示的表达式变成由一个变量(复变量)来表达，使得处理起来更加自然和方便。

我们知道傅立叶变换的结果是由两部分组成的，使用复数形式可以缩短变换表达式，使得我们可以单独处理一个变量（这个在后面的描述中我们就可以更加确切地知道），而且快速傅立叶变换正是基于复数形式的，所以几乎所有描述的傅立叶变换形式都是复数的形式。

但是复数的概念超过了我们日常生活中所能理解的概念，要理解复数是较难的，所以我们在理解复数傅立叶变换之前，先来专门复习一下有关复数的知识，这对后面的理解非常重要。

一、复数的提出

在此，先让我们看一个物理实验：把一个球从某点向上抛出，然后根据初速度和时间来计算球所在高度，这个方法可以根据下面的式子计算得出：

$$h = \frac{-gt^2}{2} + vt$$

其中h表示高度，g表示重力加速度(9.8m/s2)，v表示初速度，t表示时间。现在反过来，假如知道了高度，要求计算到这个高度所需要的时间，这时我们又可以通过下式来计算：

$$t = 1 \pm \sqrt{1 - h/4.9}$$

（*多谢JERRY_PRI提出*：

1. 根据公式h=-(gt2/2)+Vt（gt后面的2表示t的平方），我们可以讨论最终情况，也就是说小球运动到最高点时，v=gt，所以，可以得到t=sqt(2h/g)，且在您给的公式中，根号下为1-(2h)/g，化成分数形式为(g-2h)/g，g和h不能直接做加减运算。
2. g是重力加速度，单位是m/s2，h的单位是m，他们两个相减的话在物理上没有意义，而且使用您给的那个公式反向回去的话推出的是h=-(gt2/2)+gt啊（gt后面的2表示t的平方）。
3. 直接推到可以得出t=v/g±sqt((v2-2hg)/g2)（v和g后面的2都表示平方），那么也就是说当v2<2hg时会产生复数，但是如果从实际的v2是不可能小于2hg的，所以我感觉复数不能从实际出发去推到，只能从抽象的角度说明一下。

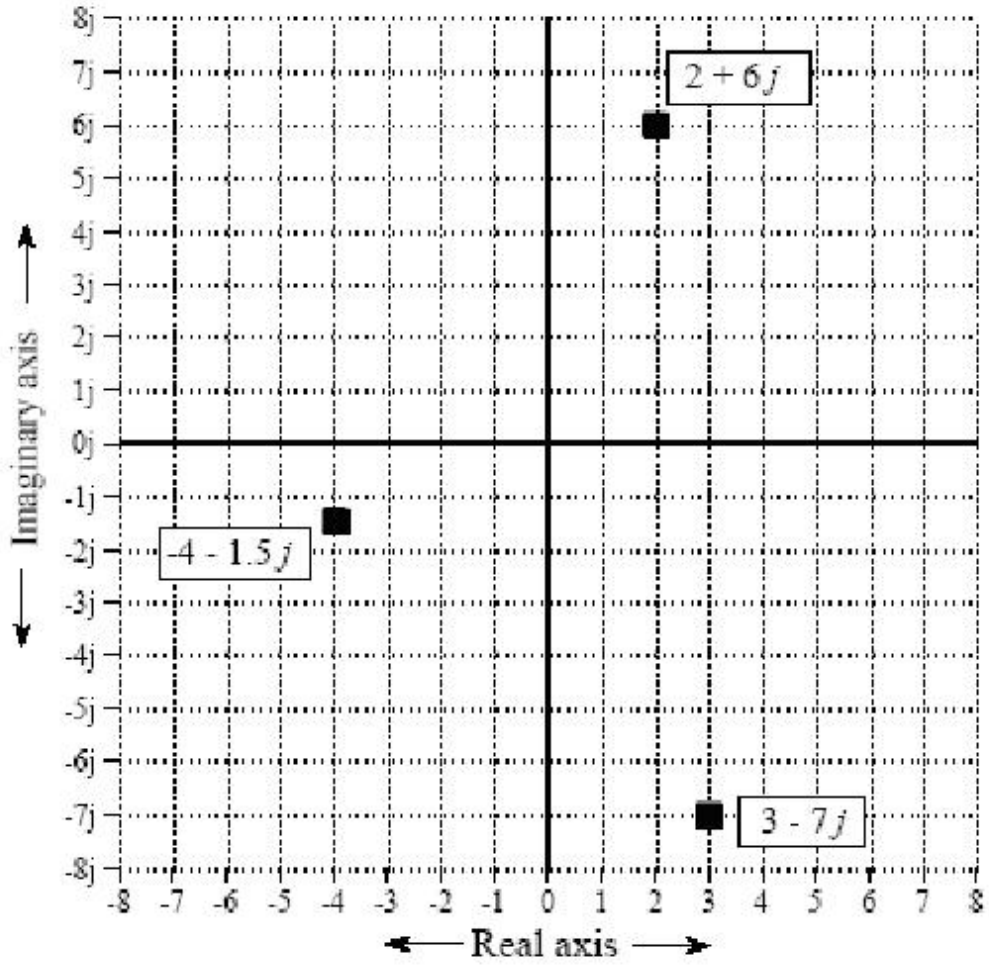
）

经过计算我们可以知道，当高度是3米时，有两个时间点到达该高度：球向上运动时的时间是0.38秒，球向下运动时的时间是1.62秒。但是如果高度等于10时，结果又是什么呢？根据上面的式子可以发现存在对负数进行开平方运算，我们知道这肯定是不现实的。

第一次使用这个不一般的式子的人是意大利数学家Girolamo Cardano（1501-1576），两个世纪后，德国伟大数学家Carl Friedrich Gause（1777-1855）提出了复数的概念，为后来的应用铺平了道路，他对复数进行这样表示：复数由实数（real）和虚数(imaginary)两部分组成，虚数中的

根号负1用i来表示(在这里我们用j来表示,因为i在电力学中表示电流的意思)。

我们可以把横坐标表示成实数,纵坐标表示成虚数,则坐标中的每个点的向量就可以用复数来表示,如下图:



上图中的ABC三个向量可以表示成如下的式子:

```
A = 2 + 6j
B = -4 - 1.5j
C = 3 - 7j
```

这样子来表达方便之处在于运用一个符号就能把两个原来难以联系起来的数组合起来了，不方便的是我们要分辨哪个是实数和哪个是虚数，我们一般是用Re( )和Im( )来表示实数和虚数两部分,如:

```
Re A = 2      Im A = 6
Re B = -4     Im B = -1.5
Re C = 3      Im C = -7
```

复数之间也可以进行加减乘除运算：

$$(a+bj)+(c+dj) = (a+c)+j(b+d)$$

$$(a+bj)-(c+dj) = (a-c)+j(b-d)$$

$$(a+bj)(c+dj) = (ac-bd)+j(bc+ad)$$

$$\frac{(a+bj)}{(c+dj)} = \left(\frac{ac+bd}{c^2+d^2}\right)+j\left(\frac{bc-ad}{c^2+d^2}\right)$$

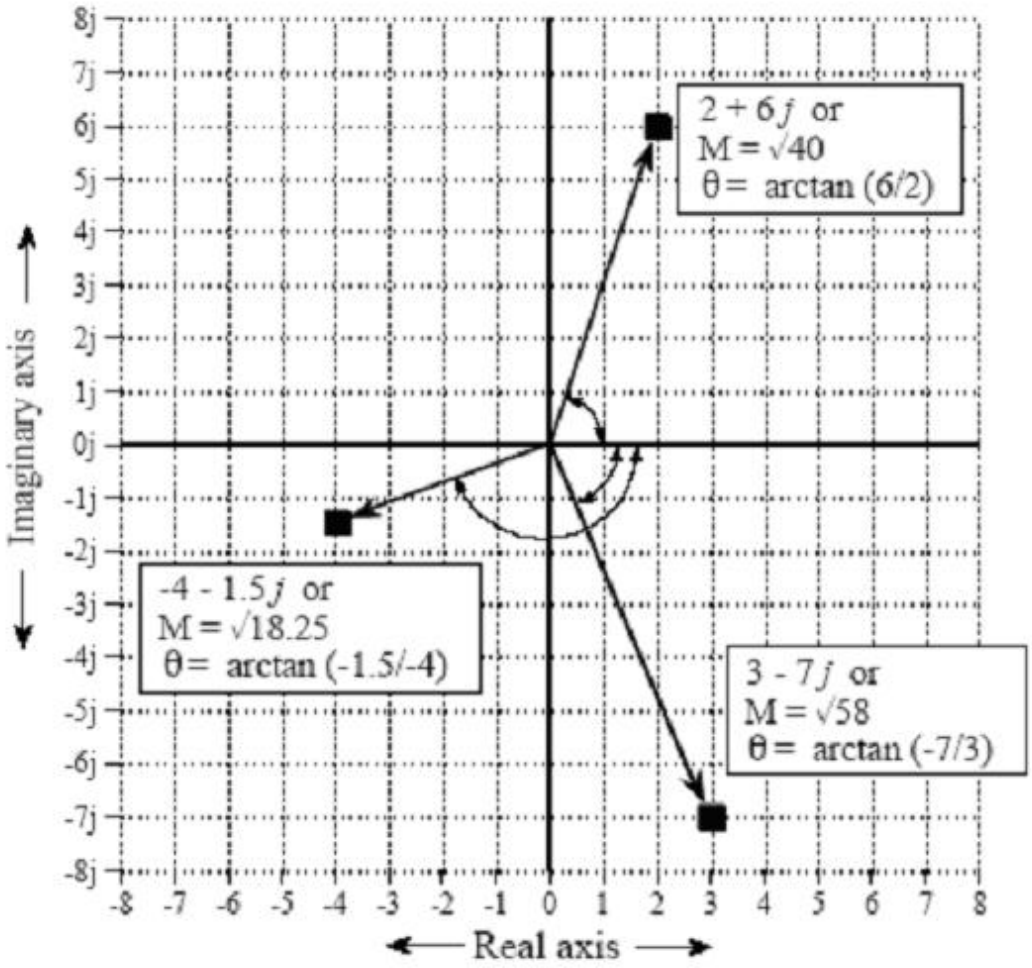
这里有个特殊的地方是j2等于-1，上面第四个式子的计算方法是把分子和分母同时乘以c – dj，这样就可消去分母中的j了。

复数也符合代数运算中的交换律、结合律、分配律：

```
A B = B A
(A + B) + C = A + (B + C)
A(B + C) = AB + AC
```

二、复数的极坐标表示形式

前面提到的是运用直角坐标来表示复数，其实更为普遍应用的是极坐标的表示方法，如下图：



上图中的M即是数量积(magnitude)，表示从原点到坐标点的距离，θ是相位角(phase angle)，表示从X轴正方向到某个向量的夹角，下面四个式子是计算方法：

$$M = \sqrt{(Re\ \mathrm{A})^2 + (Im\ \mathrm{A})^2}$$

$$\theta = \arctan\left[\frac{Im\ \mathrm{A}}{Re\ \mathrm{A}}\right]$$

$$Re\ \mathrm{A} = M\cos(\theta)$$

$$Im\ \mathrm{A} = M\sin(\theta)$$

我们还可以通过下面的式子进行极坐标到直角坐标的转换：

```
a + jb = M (cosθ + j sinθ)
```

上面这个等式中左边是直角坐标表达式，右边是极坐标表达式。

还有一个更为重要的等式——欧拉等式（欧拉，瑞士的著名数学家，Leonhard Euler，1707-1783）：

```
ejx = cos x + j sin x
```

这个等式可以从下面的级数变换中得到证明：

$$e^{jx} = \sum_{n=0}^{\infty}\frac{(jx)^n}{n!} = \left[\sum_{k=0}^{\infty}(-1)^k\frac{x^{2k}}{(2k)!}\right] + j\left[\sum_{k=0}^{\infty}(-1)^k\frac{x^{2k+1}}{(2k+1)!}\right]$$

上面中右边的两个式子分别是cos(x)和sin(x)的泰勒(Taylor)级数。

这样子我们又可以把复数的表达式表示成指数的形式了：

```
a + jb = M ejθ （这便是复数的两个表达式）
```

指数形式是数字信号处理中数学方法的支柱，也许是因为用指数形式进行复数的乘除运算极为简单的缘故吧：

$$M_1 e^{j\theta_1} M_2 e^{j\theta_2} = M_1 M_2 e^{j(\theta_1 + \theta_2)}$$

$$\frac{M_1 e^{j\theta_1}}{M_2 e^{j\theta_2}} = \left[\frac{M_1}{M_2}\right] e^{j(\theta_1 - \theta_2)}$$

三、复数是数学分析中的一个工具

为什么要使用复数呢？其实它只是个工具而已，就如钉子和锤子的关系，复数就象那锤子，作为一种使用的工具。我们把要解决的问题表达成复数的形式（因为有些问题用复数的形式进行运算更加方便），然后对复数进行运算，最后再转换回来得到我们所需要的结果。

有两种方法使用复数，一种是用复数进行简单的替换，如前面所说的向量表达式方法和前一节中我们所讨论的实域DFT，另一种是更高级的方法：数学等价(mathematical equivalence)，复数形式的傅立叶变换用的便是数学等价的方法，但在这里我们先不讨论这种方法，这里我们先来看一下用复数进行替换中的问题。

用复数进行替换的基本思想是：把所要分析的物理问题转换成复数的形式，其中只是简单地添加一个复数的符号j，当返回到原来的物理问题时，则只是把符号j去掉就可以了。

有一点要明白的是并不是所有问题都可以用复数来表示，必须看用复数进行分析是否适用，有个例子可以看出用复数来替换原来问题的表达方式明显是谬误的：假设一箱的苹果是5美元，一箱的桔子是10美元，于是我们把它表示成 5 + 10j，有一个星期你买了6箱苹果和2箱桔子，我们又

把它表示成6 + 2j，最后计算总共花的钱是(5 + 10j)(6 + 2j) = 10 + 70j，结果是买苹果花了10美元的，买桔子花了70美元，这样的结果明显是错了，所以复数的形式不适合运用于对这种问题的解决。

四、用复数来表示正余弦函数表达式

对于象M cos (ωt + φ)和A cos(ωt) + B sin(ωt)表达式，用复数来表示，可以变得非常简洁，对于直角坐标形式可以按如下形式进行转换：

$$A\cos(\omega t) + B\sin(\omega t) \quad \rightleftarrows \quad a + jb$$

(conventional representation) *(complex number)*

上式中余弦幅值A经变换生成a，正弦幅值B的相反数经变换生成b：A <=> a，B<=> -b，但要注意的是，这不是个等式，只是个替换形式而已。

对于极坐标形式可以按如下形式进行转换：

$$M\cos(\omega t + \phi) \quad \rightleftarrows \quad Me^{j\theta}$$

(conventional representation) *(complex number)*

上式中，M <=> M，θ<=>φ。

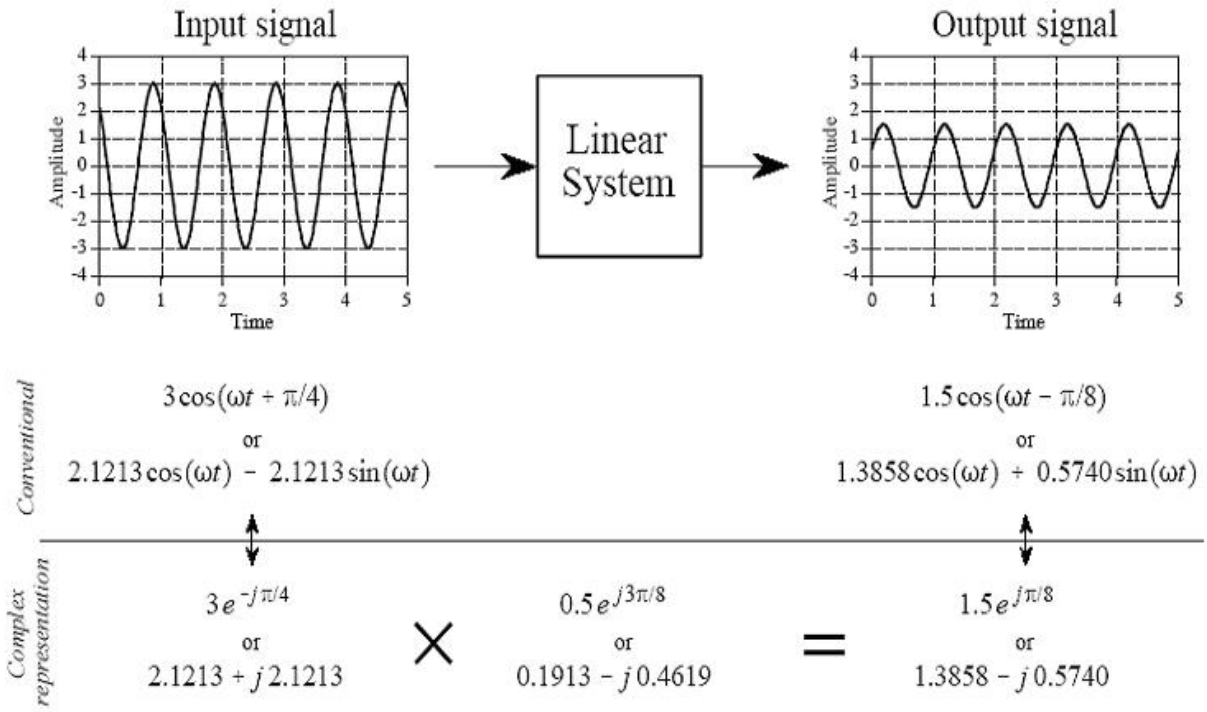
这里虚数部分采用负数的形式主要是为了跟复数傅立叶变换表达式保持一致，对于这种替换的方法来表示正余弦，符号的变换没有什么好处，但替换时总会被改变掉符号以跟更高级的等价变换保持形式上的一致。

在离散信号处理中，运用复数形式来表示正余弦波是个常用的技术，这是因为利用复数进行各种运算得到的结果跟原来的正余弦运算结果是一致的，但是，我们要小心使用复数操作，如加、减、乘、除，有些操作是不能用的，如两个正弦信号相加，采用复数形式进行相加，得到的结果跟替

换前的直接相加的结果是一样的，但是如果两个正弦信号相乘，则采用复数形式来相乘结果是不一样的。幸运的是，我们已严格定义了正余弦复数形式的运算操作条件：

1. 参加运算的所有正余弦的频率必须是一样的；
2. 运算操作必须是线性的，如两个正弦信号可以进行相加减，但不能进行乘除，象信号的放大、衰减、高低通滤波等系统都是线性的，象平方、缩短、取限等则不是线性的。要记住的是卷积和傅立叶分析也只有线性操作才可以进行。

下图是一个相量变换(我们把正弦或余弦波变成复数的形式称为相量变换，Phasor transform)的例子，一个连续信号波经过一个线性处理系统生成另一个信号波，从计算过程我们可以看出采用复数的形式使得计算变化十分的简洁：



在第二章中我们描述的实数形式傅立叶变换也是一种替换形式的复数变换，但要注意的是那还不是复数傅立叶变换，只是一种代替方式而已。下一章、即，第四章，我们就会知道复数傅立叶变换是一种更高级的变换，而不是这种简单的替换形式。

第四章、复数形式离散傅立叶变换

复数形式的离散傅立叶变换非常巧妙地运用了复数的方法，使得傅立叶变换变换更加自然和简洁，它并不是只是简单地运用替换的方法来运用复数，而是完全从复数的角度来分析问题，这一点跟实数DFT是完全不一样的。

一、把正余弦函数表示成复数的形式

通过欧拉等式可以把正余弦函数表示成复数的形式：

```
cos( x ) = 1/2 e j(-x) + 1/2 ejx
sin( x ) = j (1/2 e j(-x) - 1/2 ejx)
```

从这个等式可以看出，如果把正余弦函数表示成复数后，它们变成了由正负频率组成的正余弦波，相反地，一个由正负频率组成的正余弦波，可以通过复数的形式来表示。

我们知道，在实数傅立叶变换中，它的频谱是$0 \sim \pi(0 \sim N/2)$,但无法表示$-\pi \sim 0$的频谱，可以预见，如果把正余弦表示成复数形式，则能够把负频率包含进来。

二、把变换前后的变量都看成复数的形式

复数形式傅立叶变换把原始信号x[n]当成是一个用复数来表示的信号，其中实数部分表示原始信号值，虚数部分为0，变换结果X[k]也是个复数的形式，但这里的虚数部分是有值的。

在这里要用复数的观点来看原始信号，是理解复数形式傅立叶变换的关键（如果有学过复变函数则可能更好理解，即把x[n]看成是一个复数变量，然后象对待实数那样对这个复数变量进行相同的变换）。

三、对复数进行相关性算法（正向傅立叶变换）

从实数傅立叶变换中可以知道，我们可以通过原始信号乘以一个正交函数形式的信号，然后进行求总和，最后就能得到这个原始信号所包含的

正交函数信号的分量。

现在我们的原始信号变成了复数，我们要得到的当然是复数的信号分量，我们是不是可以把它乘以一个复数形式的正交函数呢？答案是肯定的，正余弦函数都是正交函数，变成如下形式的复数后，仍旧还是正交函数（这个从正交函数的定义可以很容易得到证明）：

```
cos x + j sin x, cos x – j sin x,……
```

这里我们采用上面的第二个式子进行相关性求和，为什么用第二个式子呢?，我们在后面会知道，正弦函数在虚数中变换后得到的是负的正弦函数，这里我们再加上一个负号，使得最后的得到的是正的正弦波，根据这个于是我们很容易就可以得到了复数形式的**DFT正向变换等式**:

$$X[k] = \frac{1}{N}\sum_{n=0}^{N-1} x[n]\left(\cos(2\pi kn/N) - j\sin(2\pi kn/N)\right)$$

这个式子很容易可以得到欧拉变换式子：

$$X[k] = \frac{1}{N}\sum_{n=0}^{N-1} x[n]e^{-j2\pi kn/N}$$

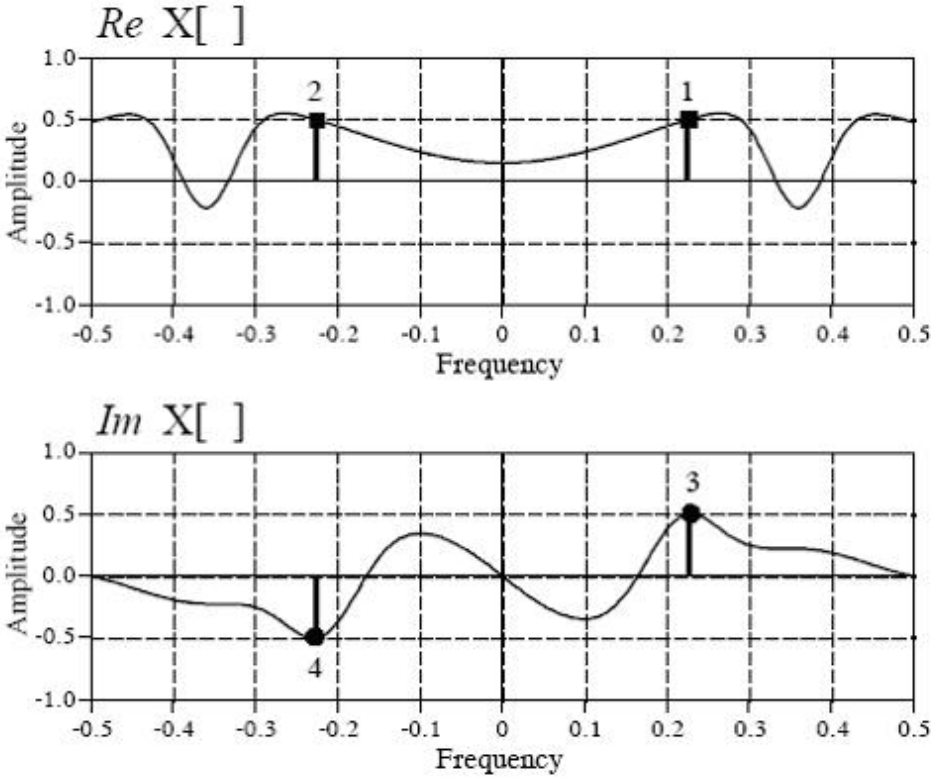
其实我们是为了表达上的方便才用到欧拉变换式，在解决问题时我们还是较多地用到正余弦表达式。

对于上面的等式，我们要清楚如下几个方面（也是区别于实数DFT的地方）：

1. X[k] 、x[n]都是复数，但x[n]的虚数部分都是由0组成的，实数部分表示原始信号；
2. k的取值范围是0 ~ N-1 (也可以表达成0 ~ 2π)，其中0 ~ N/2（或0 ~ π）是正频部分，N/2 ~ N-1（π~ 2π）是负频部分，由于正余弦函数的对称

性，所以我们把 –π~ 0表示成π~ 2π，这是出于计算上方便的考虑。

3. 其中的j是一个不可分离的组成部分，就象一个等式中的变量一样，不能随便去掉，去掉之后意义就完全不一样了，但我们知道在实数DFT中，j只是个符号而已，把j去掉，整个等式的意义不变；
4. 下图是个连续信号的频谱，但离散频谱也是与此类似的，所以不影响我们对问题的分析：



上面的频谱图把负频率放到了左边，是为了迎合我们的思维习惯，但在实际实

现中我们一般是把它移到正的频谱后面的。

从上图可以看出，时域中的正余弦波（用来组成原始信号的正余弦波）在复数DFT的频谱中被分成了正、负频率的两个组成部分，基于此等式中前面的比例系数是1/N（或1/2π），而不是2/N，这是因为现在把频谱延伸到了2π,但把正负两个频率相加即又得到了2/N,又还原到了实数DFT的形式，这个在后面的描述中可以更清楚地看到。

由于复数DFT生成的是一个完整的频谱，原始信号中的每一个点都是由正、负两个频率组合而成的，所以频谱中每一个点的带宽是一样的，都是1/N，相对实数DFT，两端带宽比其它点的带宽少了一半；复数DFT的频谱特征具有周期性：-N/2 ~ 0与N/2 ~ N-1是一样的，实域频谱呈偶对称性（表示余弦波频谱），虚域频谱呈奇对称性（表示正弦波频谱）。

四、逆向傅立叶变换

假设我们已经得到了复数形式的频谱X[k]，现在要把它还原到复数形式的原始信号x[n]，当然应该是把X[k]乘以一个复数，然后再进行求和，最后得到原始信号x[n]，这个跟X[k]相乘的复数首先让我们想到的应该是上面进行相关性计算的复数：

```
cos(2πkn/N) – j si(2πkn/N),
```

但其中的负号其实是为了使得进行逆向傅立叶变换时把正弦函数变为正的符号，因为虚数j的运算特殊性，使得原来应该是正的正弦函数变为了负的正弦函数（我们从后面的推导会看到这一点），所以这里的负号只是为了纠正符号的作用，在进行逆向DFT时，我们可以把负号去掉，于是我们便得到了这样的**逆向DFT变换等式**：

```
x[n] = X[k] (cos(2πkn/N) + j sin(2πkn/N))
```

我们现在来分析这个式子，会发现这个式其实跟实数傅立叶变换是可以得到一样结果的。我们先把X[k]变换一下：

```
X[k] = Re X[k] + j Im X[k]
```

这样我们就可以对x[n]再次进行变换，如：

```
x[n] = (Re X[k] + j Im X[k]) (cos(2πkn/N) + j sin(2πkn/N))

    = ( Re X[k] cos(2πkn/N) + j Im X[k] cos(2πkn/N) +j Re X[k] sin(2πkn/N) -  Im X[k] sin(2πkn/N) )

    = ( Re X[k] (cos(2πkn/N) + j sin(2πkn/N)) +     ----------------------(1)

    Im X[k] ( - sin(2πkn/N) + j cos(2πkn/N))       ----------------------(2)
```

这时我们就把原来的等式分成了两个部分，第一个部分是跟实域中的频谱相乘，第二个部分是跟虚域中的频谱相乘，根据频谱图我们可以知道，Re X[k]是个偶对称的变量，Im X[k]是个奇对称的变量，即

```
Re X[k] = Re X[- k]
Im X[k] = - Im X[-k]
```

但k的范围是0 ~ N-1，$0_{N/2表示正频率,N/2}$N-1表示负频率，为了表达方便我们把N/2~N-1用-k来表示，这样在从0到N-1的求和过程中对于(1)和(2)式分别有N/2对的k和-k的和，对于（1）式有：

```
Re X[k] (cos(2πkn/N) + j sin(2πkn/N)) + Re X[- k] (cos( - 2πkn/N) + j sin( -2πkn/N))
```

根据偶对称性和三角函数的性质，把上式化简得到：

```
Re X[k] (cos(2πkn/N) + j sin(2πkn/N)) + Re X[ k] (cos( 2πkn/N) - j sin( 2πkn/N))
```

这个式子最后的结果是：

```
2 Re X[ k] cos(2πkn/N)。
```

再考虑到求Re X[k]等式中有个比例系数1/N，把1/N乘以2，这样的结果不就是跟实数DFT中的式子一样了吗？

对于(2)式，用同样的方法，我们也可以得到这样的结果：

```
-2 Im X[k] sin(2πkn/N)
```

注意上式前面多了个负符号，这是由于虚数变换的特殊性造成的，当然我们肯定不能把负符号的正弦函数跟余弦来相加，还好，我们前面是用cos(2πkn/N) – j sin(2πkn/N)进行相关性计算，得到的Im X[k]中有个负的符号，这样最后的结果中正弦函数就没有负的符号了，这就是为什么在进行相关性计算时虚数部分要用到负符号的原因（我觉得这也许是复数形式DFT美中不足的地方，让人有一种拼凑的感觉）。

从上面的分析中可以看出，实数傅立叶变换跟复数傅立叶变换，在进行逆变换时得到的结果是一样的，只不过是殊途同归吧。本文完。（July、dznlong）

支持向量机通俗导论（理解SVM的三层境界）

前言

动笔写这个支持向量机(support vector machine)是费了不少劲和困难的，原因很简单，一者这个东西本身就并不好懂，要深入学习和研究下去需花费不少时间和精力，二者这个东西也不好讲清楚，尽管网上已经有朋友写得不错了(见文末参考链接)，但在描述数学公式的时候还是显得不够。得益于同学白石的数学证明，我还是想尝试写一下，希望本文在兼顾通俗易懂的基础上，真真正正能足以成为一篇完整概括和介绍支持向量机的导论性的文章。

本文在写的过程中，参考了不少资料，包括《支持向量机导论》、《统计学习方法》及网友pluskid的支持向量机系列等等，于此，还是一篇学习笔记，只是加入了自己的理解和总结，有任何不妥之处，还望海涵。全文宏观上整体认识支持向量机的概念和用处，微观上深究部分定理的来龙去脉，证明及原理细节，力保逻辑清晰 & 通俗易懂。

同时，阅读本文时建议大家尽量使用chrome等浏览器，如此公式才能更好的显示，再者，阅读时可拿张纸和笔出来，把本文所有定理.公式都亲自推导一遍或者直接打印下来（可直接打印网页版或本文文末附的PDF，享受随时随地思考、演算的极致快感），在文稿上演算。

Ok，还是那句原话，有任何问题，欢迎任何人随时不吝指正 & 赐教，感谢。

第一层、了解SVM

1.0、什么是支持向量机SVM

要明白什么是SVM，便得从分类说起。

分类作为数据挖掘领域中一项非常重要的任务，它的目的是学会一个分类函数或分类模型(或者叫做分类器)，而支持向量机本身便是一种监督式学习的方法(至于具体什么是监督学习与非监督学习，请参见此系列Machine L&Data Mining第一篇)，它广泛的应用于统计分类以及回归分析中。

支持向量机（SVM）是90年代中期发展起来的基于统计学习理论的一种机器学习方法，通过寻求结构化风险最小来提高学习机泛化能力，实现经验风险和置信范围的最小化，从而达到在统计样本量较少的情况下，亦能获得良好统计规律的目的。

通俗来讲，它是一种二类分类模型，其基本模型定义为特征空间上的间隔最大的线性分类器，即支持向量机的学习策略便是间隔最大化，最终可转化为一个凸二次规划问题的求解。

对于不想深究SVM原理的同学或比如就只想看看SVM是干嘛的，那么，了解到这里便足够了，不需上层。而对于那些喜欢深入研究一个东西的同学，甚至究其本质的，咱们则还有很长的一段路要走，万里长征，咱们开始迈第一步吧，相信你能走完。

1.1、线性分类

OK，在讲SVM之前，咱们必须先弄清楚一个概念：线性分类器(也可以叫做感知机，这里的机表示的是一种算法，本文第三部分、证明SVM中会详细阐述)。

1.1.1、分类标准

这里我们考虑的是一个两类的分类问题，数据点用 x 来表示，这是一个 n 维向量，w^T中的T代表转置，而类别用 y 来表示，可以取 1 或者 -1，分别代表两个不同的类。一个线性分类器的学习目标就是要在 n 维的数据空间中找到一个分类超平面，其方程可以表示为：

$$w^T x + b = 0$$

上面给出了线性分类的定义描述，但或许读者没有想过：为何用y取1 或者 -1来表示两个不同的类别呢？其实，这个1或-1的分类标准起源于logistic回归，为了完整和过渡的自然性，咱们就再来看看这个logistic回归。

1.1.2、1或-1分类标准的起源：logistic回归

Logistic回归目的是从特征学习出一个0/1分类模型，而这个模型是将特性的线性组合作为自变量，由于自变量的取值范围是负无穷到正无穷。因此，使用logistic函数（或称作sigmoid函数）将自变量映射到(0,1)上，映射后的值被认为是属于y=1的概率。

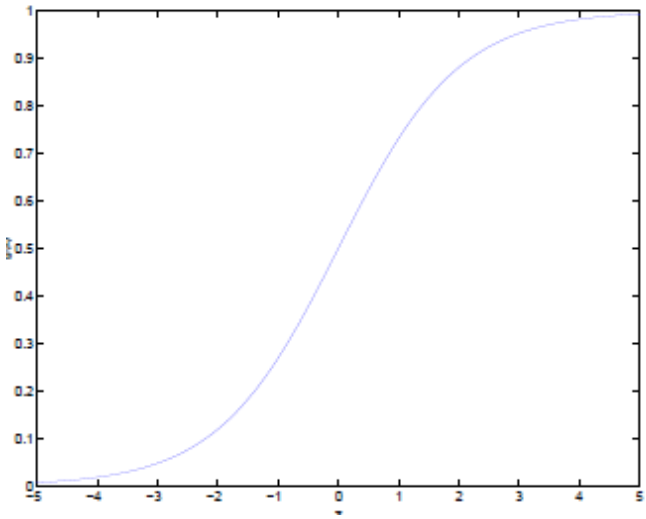
形式化表示就是

假设函数

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}},$$

其中x是n维特征向量，函数g就是logistic函数。

而 $g(z) = \frac{1}{1 + e^{-z}}$ 的图像是



可以看到，将无穷映射到了(0,1)。而假设函数就是特征属于y=1的概率。

$$P(y=1 \mid x;\theta) = h_\theta(x)$$
$$P(y=0 \mid x;\theta) = 1 - h_\theta(x)$$

当我们要判别一个新来的特征属于哪个类时，只需求，若大于0.5就是y=1的类，反之属于y=0类。

再审视一下$h_\theta(x)$，发现$h_\theta(x)$只和$\theta^T x$有关，$\theta^T x>0$，那么$h_\theta(x)>0.5$，g(z)只不过是用来映射，真实的类别决定权还在$\theta^T x$。还有当$\theta^T x>>0$时，$h_\theta(x)=1$，反之$h_\theta(x)=0$。如果我们只从$\theta^T x$出发，希望模型达到的目标无非就是让训练数据中y=1的特征$\theta^T x>>0$，而是y=0的特征$\theta^T x<<0$。Logistic回归就是要学习得到θ，使得正例的特征远大于0，负例的特征远小于0，强调在全部训练实例上达到这个目标。

1.1.3、形式化标示

我们这次使用的结果标签是y=-1,y=1，替换在logistic回归中使用的y=0和y=1。同时将θ替换成w和b。以前的$\theta^T x = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$，其中认为$x_0 = 1$。现在我们替换为b，后面替换$\theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$为$w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$（即$w^T x$）。这样，我们让$\theta^T x = w^T x + b$，进一步$h_\theta(x) = g(\theta^T x) = g(w^T x + b)$。也就是说除了y由y=0变为y=-1，只是标记不同外，与logistic回归的形式化表示没区别。

再明确下假设函数

$$h_{w,b}(x) = g(w^T x + b)$$

上面提到过我们只需考虑的$\theta^T x$正负问题，而不用关心g(z)，因此我们这里将g(z)做一个简化，将其简单映射到y=-1和y=1上。映射关系如下：

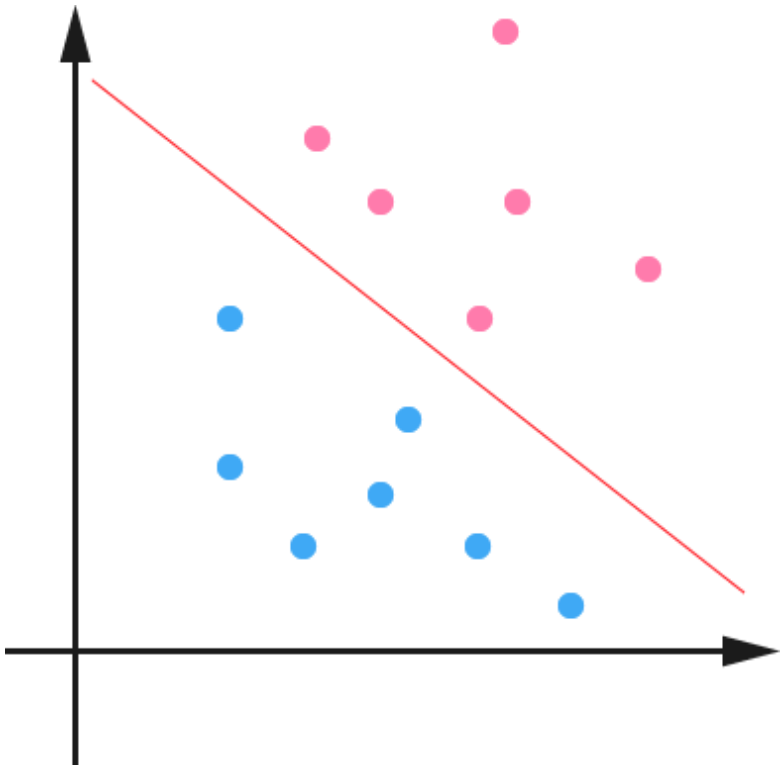
$$g(z) = \begin{cases} 1, & z \geq 0 \\ -1, & z < 0 \end{cases}$$

于此，想必已经解释明白了为何线性分类的标准一般用1 或者-1 来标示。

注：上小节来自jerrylead所作的斯坦福机器学习课程的笔记。

1.2、线性分类的一个例子

下面举个简单的例子，一个二维平面(一个超平面，在二维空间中的例子就是一条直线)，如下图所示，平面上有两种不同的点，分别用两种不同的颜色表示，一种为红颜色的点，另一种则为蓝颜色的点，红颜色的线表示一个可行的超平面。



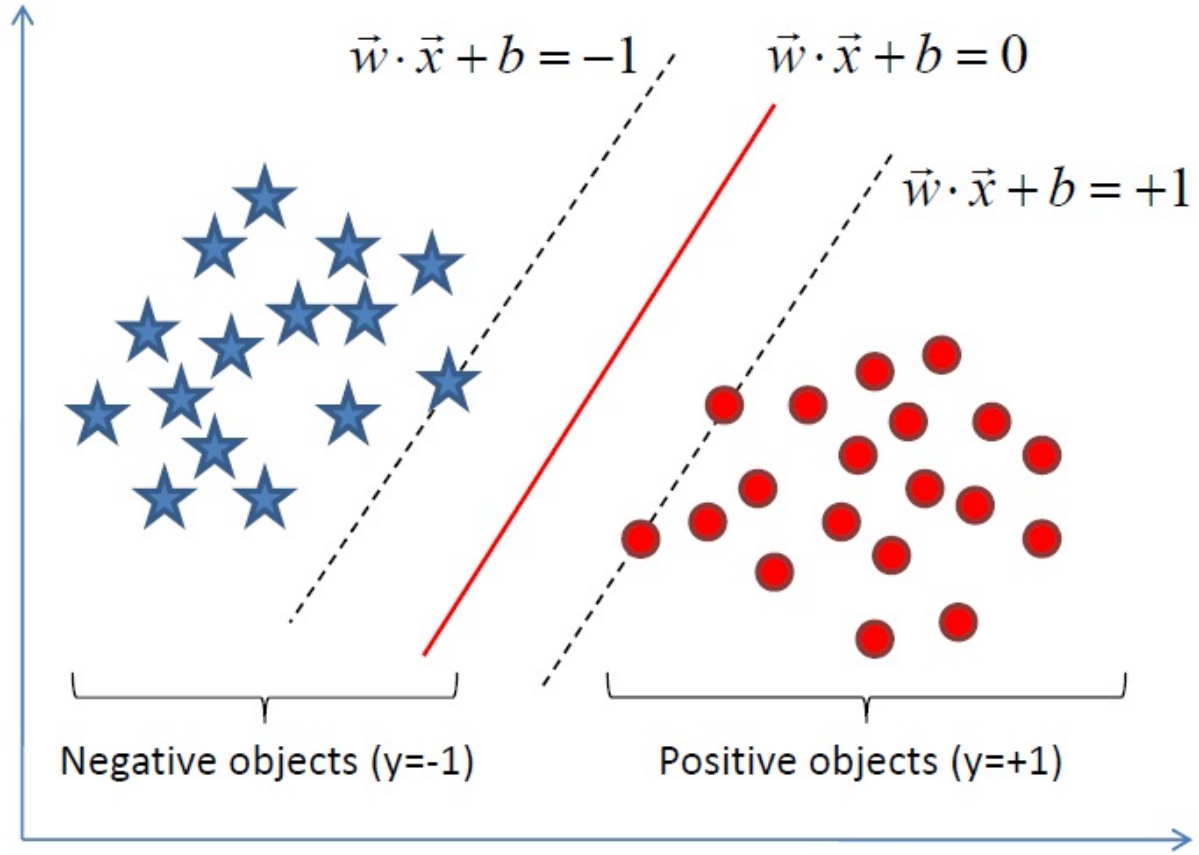
从上图中我们可以看出，这条红颜色的线把红颜色的点和蓝颜色的点分开来了。而这条红颜色的线就是我们上面所说的超平面，也就是说，这个所谓的超平面的的确确便把这两种不同颜色的数据点分隔开来，在超平面一边的数据点所对应的 y 全是 -1 ，而在另一边全是 1 。

接着，我们可以令分类函数（提醒：下文很大篇幅都在讨论着这个分类函数）：

$$f(x) = w^T x + b$$

显然，如果 f(x)=0 ，那么 x 是位于超平面上的点。我们不妨要求对于所有满足 f(x)<0 的点，其对应的 y 等于 -1 ，而 f(x)>0 则对应 y=1 的数据

点。



注:上图中,定义特征到结果的输出函数 $u=\vec{w}\cdot\vec{x}-b$,与我们之前定义的 $f(x)=w^Tx+b$ 实质是一样的。为什么?因为无论是,还是,不影响最终优化结果。下文你将看到,当我们转化到优化

$$\max\frac{1}{\|w\|},\quad s.t.,y_i(w^Tx_i+b)\geq 1,i=1,\ldots,n$$

的时候,为了求解方便,会把yf(x)令为1,即yf(x)是y(w^x + b),还是y(w^x - b),对我们要优化的式子max1/||w||已无影响。(有一朋友飞狗来自Mare_Desiderii,看了上面的定义之后,问道:请教一下SVM functional margin 为 $\hat{\gamma}$ =y(wTx+b)=yf(x)中的Y是只取1和-1 吗?y的唯一作用就是确保functional margin的非负性?真是这样的么?当然不是,详情请见本文评论下第43楼)

当然，有些时候，或者说大部分时候数据并不是线性可分的，这个时候满足这样条件的超平面就根本不存在(不过关于如何处理这样的问题我们后面会讲)，这里先从最简单的情形开始推导，就假设数据都是线性可分的，亦即这样的超平面是存在的。

更进一步，我们在进行分类的时候，将数据点 x代入 f(x) 中，如果得到的结果小于 0 ，则赋予其类别 -1 ，如果大于 0 则赋予类别 1 。如果 f(x)=0，则很难办了，分到哪一类都不是。

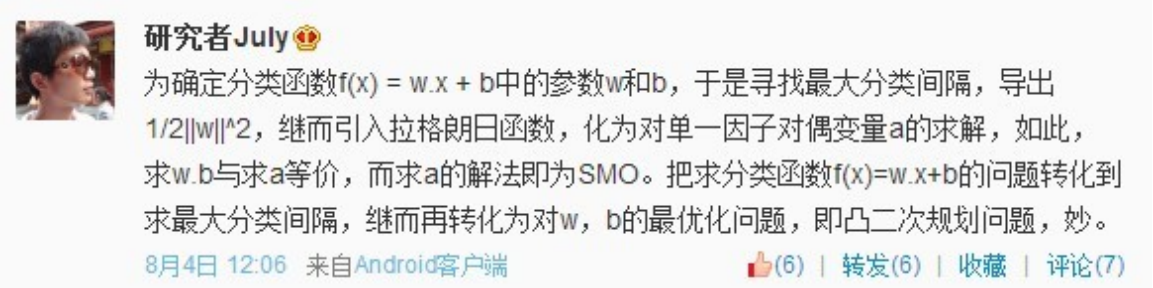
请读者注意，下面的篇幅将按下述3点走：

1. 咱们就要确定上述分类函数f(x) = w.x + b（w.x表示w与x的内积）中的两个参数w和b，通俗理解的话w是法向量，b是截距（再次说明：定义特征到结果$u = \vec{w} \cdot \vec{x} - b$的输出函数，与我们最开始定义的$f(x) = w^T x + b$实质是一样的）；

2. 那如何确定w和b呢？答案是寻找两条边界端或极端划分直线中间的最大间隔（之所以要寻最大间隔是为了能更好的划分不同类的点，下文你将看到：为寻最大间隔，导出1/2||w||^2，继而引入拉格朗日函数和对偶变量a，化为对单一因数对偶变量a的求解，当然，这是后话），从而确定最终的最大间隔分类超平面hyper plane和分类函数；

3. 进而把寻求分类函数f(x) = w.x + b的问题转化为对w，b的最优化问题，最终化为对偶因子的求解。

总结成一句话即是：从最大间隔出发（目的本就是为了确定法向量w），转化为求对变量w和b的凸二次规划问题。亦或如下图所示（有点需要注意，如读者@酱爆小八爪所说：从最大分类间隔开始，就一直是凸优化问题）：



1.3、函数间隔Functional margin与几何间隔Geometrical margin

一般而言，一个点距离超平面的远近可以表示为分类预测的确信或准确程度。

- 在超平面w*x+b=0确定的情况下，|w*x+b|能够相对的表示点x到距离超平面的远近，而w*x+b的符号与类标记y的符号是否一致表示分类是否正确，所以，可以用量y*(w*x+b)的正负性来判定或表示分类的正确性和确信度。

于此，我们便引出了定义样本到分类间隔距离的函数间隔functional margin的概念。

1.3.1、函数间隔Functional margin

我们定义函数间隔functional margin 为：

$$\hat{\gamma} = y(w^T x + b) = yf(x)$$

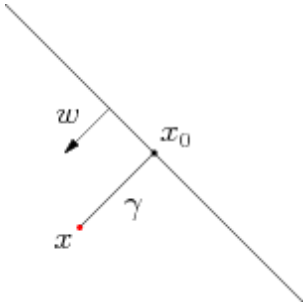
接着，我们定义超平面(w，b)关于训练数据集T的函数间隔为超平面(w，b)关于T中所有样本点(xi，yi)的函数间隔最小值，其中，x是特征，y是结果标签，i表示第i个样本，有：

$\hat{\gamma}$ = min $\hat{\gamma}$ i (i=1，…n)

然与此同时，问题就出来了。上述定义的函数间隔虽然可以表示分类预测的正确性和确信度，但在选择分类超平面时，只有函数间隔还远远不够，因为如果成比例的改变w和b，如将他们改变为2w和2b，虽然此时超平面没有改变，但函数间隔的值f(x)却变成了原来的2倍。

其实，我们可以对法向量w加些约束条件，使其表面上看起来规范化，如此，我们很快又将引出真正定义点到超平面的距离–几何间隔geometrical margin的概念（很快你将看到，几何间隔就是函数间隔除以个||w||，即yf(x) / ||w||）。

1.3.2、点到超平面的距离定义：几何间隔Geometrical margin



在给出几何间隔的定义之前，咱们首先来看下，如上图所示，对于一个点 x，令其垂直投影到超平面上的对应的为 x0，由于 w 是垂直于超平面的一个向量，$\hat{\gamma}$ 为样本x到分类间隔的距离，我们有

$$x = x_0 + \gamma \frac{w}{\|w\|}$$

（||w||表示的是范数，关于范数的概念参见这里）

又由于 x0 是超平面上的点，满足 f(x0)=0，代入超平面的方程即可算出：

$$\gamma = \frac{w^T x + b}{\|w\|} = \frac{f(x)}{\|w\|}$$

（有的书上会写成把||w|| 分开相除的形式，如本文参考文献及推荐阅读条目11，其中，||w||为w的二阶泛数）

不过这里的 $\hat{\gamma}$ 是带符号的，我们需要的只是它的绝对值，因此类似地，也乘上对应的类别 y即可，因此实际上我们定义 几何间隔geometrical margin 为(注：别忘了，上面 $\hat{\gamma}$ 的定义，$\hat{\gamma}$ =y(wTx+b)=yf(x))：

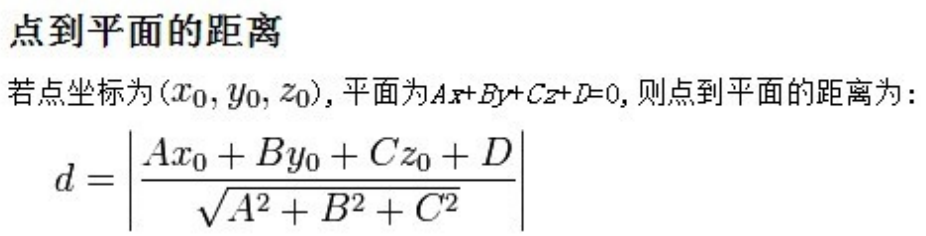
$$\tilde{\gamma} = y\gamma = \frac{\hat{\gamma}}{\|w\|}$$

（代入相关式子可以得出：yi*(w/||w|| + b/||w||)）

正如本文评论下读者popol1991留言：函数间隔y*(wx+b)*=yf(x)实际上就是|f(x)|，只是人为定义的一个间隔度量；而几何间隔|f(x)|/||w||才是直观上

的点到超平面距离。

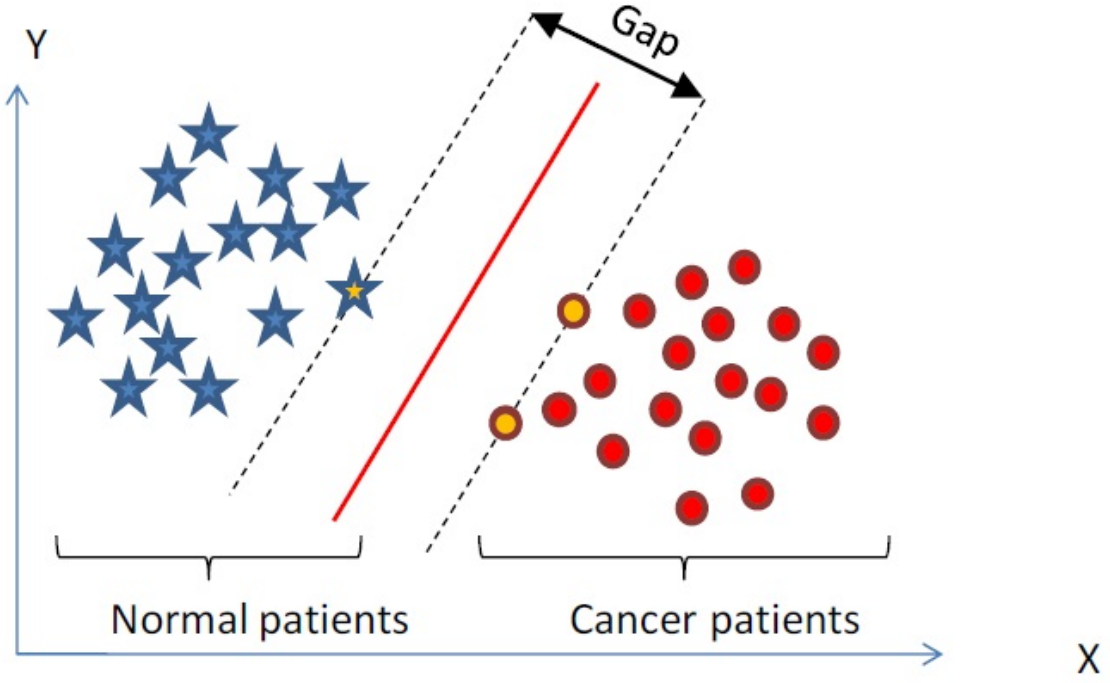
想想二维空间里的点到直线公式：假设一条直线的方程为ax+by+c=0,点P的坐标是(x0,y0)，则点到直线距离为|ax0+by0+c|/sqrt($a^{2+b}2$)。如下图所示：



那么如果用向量表示，设w=(a,b),f(x)=wx+c,那么这个距离正是|f(p)|/||w||。

1.4、最大间隔分类器Maximum Margin Classifier的定义

于此，我们已经很明显的看出，函数间隔functional margin 和 几何间隔geometrical margin 相差一个的缩放因子。按照我们前面的分析，对一个数据点进行分类，当它的 margin 越大的时候，分类的 confidence 越大。对于一个包含 n 个点的数据集，我们可以很自然地定义它的 margin 为所有这 n 个点的 margin 值中最小的那个。于是，为了使得分类的 confidence 高，我们希望所选择的超平面hyper plane 能够最大化这个 margin 值。



通过上节，我们已经知道：

1. functional margin 明显是不太适合用来最大化的一个量，因为在 hyper plane 固定以后，我们可以等比例地缩放 w 的长度和 b 的值，这样可以使得 $f(x) = w^T x + b$ 的值任意大，亦即 functional margin 可以在 hyper plane 保持不变的情况下被取得任意大，

2. 而 geometrical margin 则没有这个问题，因为除上了 $\|w\|$ 这个分母，所以缩放 w 和 b 的时候 $\hat{\gamma}$ 的值是不会改变的，它只随着 hyper plane 的变动而变动，因此，这是更加合适的一个 margin 。

这样一来，我们的 maximum margin classifier 的目标函数可以定义为：

$$\max \tilde{\gamma}$$

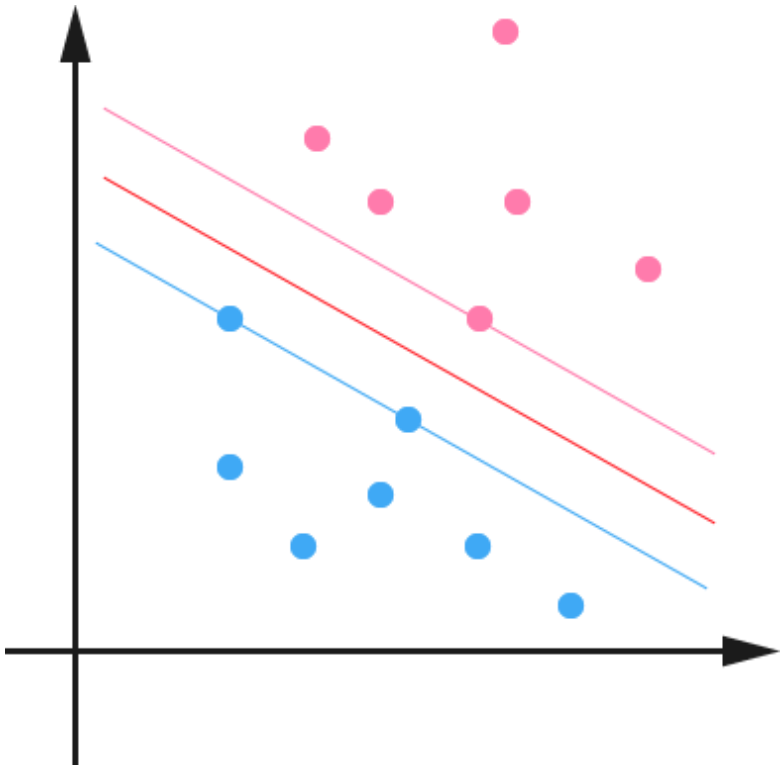
当然，还需要满足一些条件，根据 margin 的定义，我们有

$$y_i(w^T x_i + b) = \hat{\gamma}_i \geq \hat{\gamma}, \quad i = 1, \ldots, n$$

其中 $\hat{\gamma} = \tilde{\gamma}\|w\|$ (等价于 $\hat{\gamma}$ = $\hat{\gamma}$ /||w||，故有稍后的 $\hat{\gamma}$ =1 时， $\hat{\gamma}$ = 1 / ||w||)，处于方便推导和优化的目的，我们可以令 $\hat{\gamma}$ =1(对目标函数的优化没有影响，至于为什么，请见本文评论下第42楼回复)，此时，上述的目标函数 $\hat{\gamma}$ 转化为(其中，s.t.，即subject to的意思，它导出的是约束条件)：

$$\max \frac{1}{\|w\|}, \quad s.t., y_i(w^T x_i + b) \geq 1, i = 1, \ldots, n$$

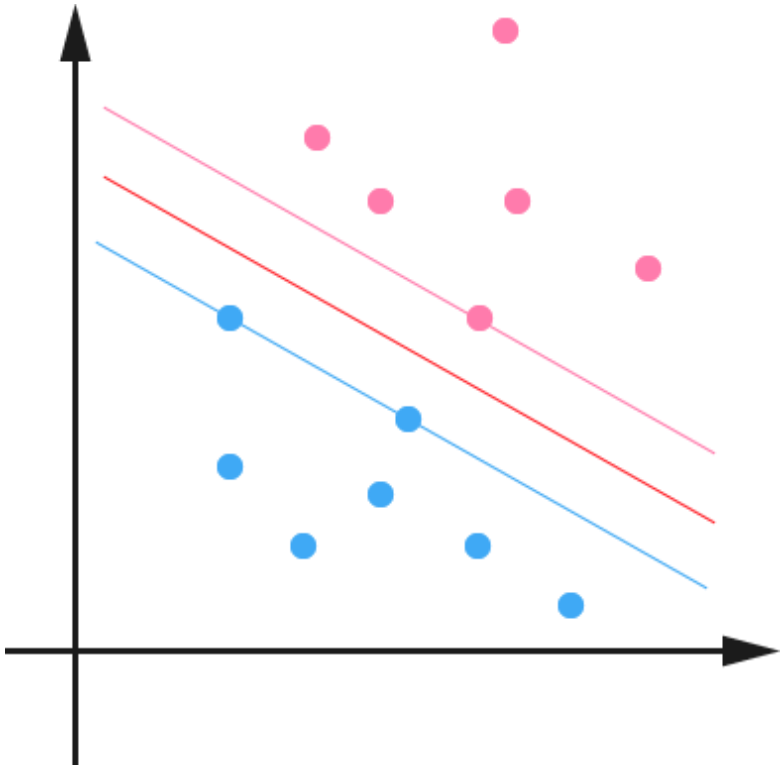
通过求解这个问题，我们就可以找到一个 margin 最大的 classifier，如下图所示，中间的红色线条是 Optimal Hyper Plane，另外两条线到红线的距离都是等于 $\hat{\gamma}$ 的($\hat{\gamma}$ 便是上文所定义的geometrical margin，当令 $\hat{\gamma}$ =1 时， $\hat{\gamma}$ 便为1/||w||，而我们上面得到的目标函数便是在相应的约束条件下，要最大化这个1/||w||值)：



通过最大化 margin，我们使得该分类器对数据进行分类时具有了最大的 confidence，从而设计决策最优分类超平面。

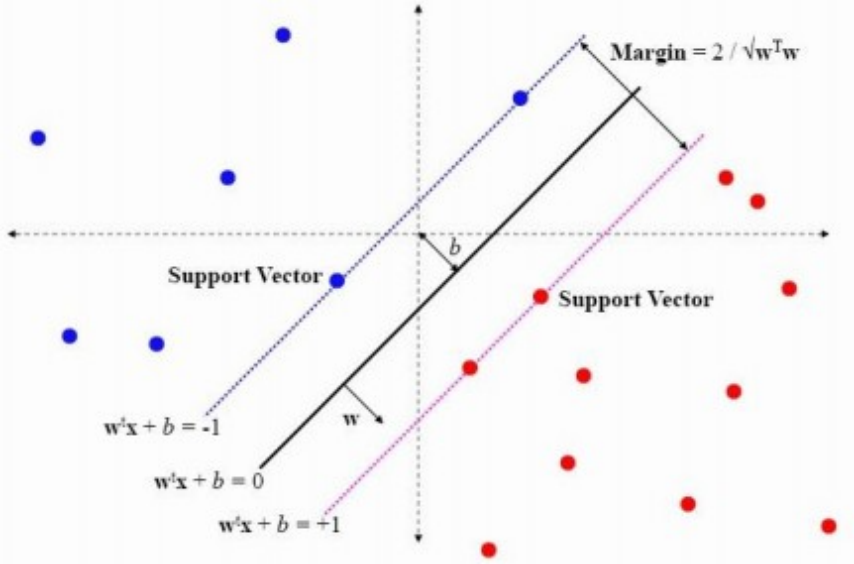
1.5、到底什么是Support Vector

上节，我们介绍了Maximum Margin Classifier，但并没有具体阐述到底什么是Support Vector，本节，咱们来重点阐述这个概念。咱们不妨先来回忆一下上节1.4节最后一张图：



可以看到两个支撑着中间的 gap 的超平面，它们到中间的纯红线 separating hyper plane 的距离相等，即我们所能得到的最大的 geometrical margin，而“支撑”这两个超平面的必定会有一些点，而这些“支撑”的点便叫做支持向量Support Vector。

或亦可看下来自此PPT中的一张图，Support Vector便是那蓝色虚线和粉红色虚线上的点：



很显然，由于这些 supporting vector 刚好在边界上，所以它们满足 $y(w^Tx+b)=1$（还记得我们把 functional margin 定为 1 了吗？上节中：“处于方便推导和优化的目的，我们可以令=1”），而对于所有不是支持向量的点，也就是在“阵地后方”的点，则显然有 $y(w^Tx+b)>1$。当然，除了从几何直观上之外，支持向量的概念也可以从下文优化过程的推导中得到。

OK，到此为止，算是了解到了SVM的第一层，对于那些只关心怎么用SVM的朋友便已足够，不必再更进一层深究其更深的原理。

第二层、深入SVM

2.1、从线性可分到线性不可分

2.1.1、从原始问题到对偶问题的求解

虽然上文1.4节给出了目标函数，却没有讲怎么来求解。现在就让我们来处理这个问题。回忆一下之前得到的目标函数（subject to导出的则是约束条件）：

$$\max \frac{1}{\|w\|}, \quad s.t., y_i(w^Tx_i+b) \geq 1, i=1,\ldots,n$$

由于求$\frac{1}{||w||}$的最大值相当于求$\frac{1}{2}||w||^2$的最小值，所以上述问题等价于（w由分母变成分子，从而也有原来的max问题变为min问题，很明显，两者问题等价）：

$$\min \frac{1}{2}\|w\|^2 \quad s.t., y_i(w^T x_i + b) \geq 1, i = 1, \ldots, n$$

1. 转化到这个形式后，我们的问题成为了一个凸优化问题，或者更具体的说，因为现在的目标函数是二次的，约束条件是线性的，所以它是一个凸二次规划问题。这个问题可以用任何现成的 QP (Quadratic Programming) 的优化包进行求解，归结为一句话即是：在一定的约束条件下，目标最优，损失最小；
2. 但虽然这个问题确实是一个标准的 QP 问题，但是它也有它的特殊结构，通过 Lagrange Duality 变换到对偶变量 (dual variable) 的优化问题之后，可以找到一种更加有效的方法来进行求解，而且通常情况下这种方法比直接使用通用的 QP 优化包进行优化要高效得多。

也就说，除了用解决QP问题的常规方法之外，还可以通过求解对偶问题得到最优解，这就是线性可分条件下支持向量机的对偶算法，这样做的优点在于：一者对偶问题往往更容易求解；二者可以自然的引入核函数，进而推广到非线性分类问题。

至于上述提到，关于什么是Lagrange duality？简单地来说，通过给每一个约束条件加上一个 Lagrange multiplier(拉格朗日乘值)，即引入拉格朗日乘子α，如此我们便可以通过拉格朗日函数将约束条件融和到目标函数里去(也就是说把条件融合到一个函数里头，现在只用一个函数表达式便能清楚的表达出我们的问题)：

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2}\|w\|^2 - \sum_{i=1}^{n} \alpha_i \left(y_i(w^T x_i + b) - 1 \right)$$

然后我们令

$$\theta(w) = \max_{\alpha_i \geq 0} \mathcal{L}(w, b, \alpha)$$

容易验证，当某个约束条件不满足时，例如$y_i(w^Tx_i+b)<1$，那么我们显然有$\theta(w)=\infty$（只要令$\alpha_i=\infty$即可）。而当所有约束条件都满足时，则有$\theta(w)=\frac{1}{2}\|w\|^2$，亦即我们最初要最小化的量$\frac{1}{2}\|w\|^2$。因此，在要求约束条件得到满足的情况下最小化$\theta(w)$，实际上等价于直接最小化$\alpha_i$（当然，这里也有约束条件，就是≥0,i=1,…,n），因为如果约束条件没有得到满足，$\theta(w)$会等于无穷大，自然不会是我们所要求的最小值。具体写出来，我们现在的目标函数变成了：

$$\min_{w,b}\theta(w)=\min_{w,b}\max_{\alpha_i\geq 0}\mathcal{L}(w,b,\alpha)=p^*$$

这里用p^*表示这个问题的最优值，这个问题和我们最初的问题是等价的。不过，现在我们来把最小和最大的位置交换一下（稍后，你将看到，当下面式子满足了一定的条件之后，这个式子d 便是上式P 的对偶形式表示）：

$$\max_{\alpha_i\geq 0}\min_{w,b}\mathcal{L}(w,b,\alpha)=d^*$$

当然，交换以后的问题不再等价于原问题，这个新问题的最优值用d^*来表示。并且，我们有$d^*\leq p^*$，这在直观上也不难理解，最大值中最小的一个总也比最小值中最大的一个要大吧！总之，第二个问题的最优值d^*在这里提供了一个第一个问题的最优值p^*的一个下界，在满足某些条件的情况下，这两者相等，这个时候我们就可以通过求解第二个问题来间接地求解第一个问题。

也就是说，下面我们将先求L 对w、b的极小，再求L 对α的极大。而且，之所以从minmax的原始问题p^*，转化为maxmin的对偶问题d^*，一者因为d^*是p^*的近似解，二者，转化为对偶问题后，更容易求解。

2.1.2 KKT条件

与此同时，上段说“在满足某些条件的情况下”，这所谓的“满足某些条件”就是要满足KKT条件。那KKT条件的表现形式是什么呢？据维基百科：KKT 条件的介绍，一般地，一个最优化数学模型能够表示成下列标准形式：

$$\begin{aligned}
&\min. f(\mathbf{x}) \\
&\text{s.t.} \quad h_j(\mathbf{x}) = 0, j = 1,\ldots, p, \\
&\qquad g_k(\mathbf{x}) \le 0, k = 1,\ldots, q, \\
&\qquad \mathbf{x} \in \mathbf{X} \subset \Re^n
\end{aligned}$$

其中，f(x)是需要最小化的函数，h(x)是等式约束，g(x)是不等式约束，p和q分别为等式约束和不等式约束的数量。同时，我们得明白以下两个定理：

- 凸优化的概念：$\mathcal{X} \subset \mathbb{R}^n$为一凸集，$f : \mathcal{X} \to \mathbb{R}$ 为一凸函数。凸优化就是要找出一点 $x^* \in \mathcal{X}$，使得每一 $x \in \mathcal{X}$ 满足 $f(x^*) \le f(x)$。
- KKT条件的意义：它是一个非线性规划（Nonlinear Programming）问题能有最优化解法的必要和充分条件。

那到底什么是所谓Karush-Kuhn-Tucker条件呢？KKT条件就是指上面最优化数学模型的标准形式中的最小点 x* 必须满足下面的条件：

$$\begin{aligned}
&1. \quad h_j(\mathbf{x}_*) = 0, j = 1,\ldots, p, \; g_k(\mathbf{x}_*) \le 0, k = 1,\ldots, q, \\
&2. \quad \nabla f(\mathbf{x}_*) + \sum_{j=1}^{p} \lambda_j \nabla h_j(\mathbf{x}_*) + \sum_{k=1}^{q} \mu_k \nabla g_k(\mathbf{x}_*) = \mathbf{0}, \\
&\lambda_j \ne 0, \; \mu_k \ge 0, \; \mu_k g_k(\mathbf{x}_*) = 0.
\end{aligned}$$

经过论证，我们这里的问题是满足 KKT 条件的（首先已经满足Slater condition，再者f和gi也都是可微的，即L对w和b都可导），因此现在我们便转化为求解第二个问题。也就是说，现在，咱们的原问题通过满足一定的

条件，已经转化成了对偶问题。而求解这个对偶学习问题，分为3个步骤，首先要让L(w，b，a) 关于 w 和 b 最小化，然后求对α的极大，最后利用SMO算法求解对偶因子。

2.1.3、对偶问题求解的3个步骤

（）1)、首先固定α，要让 L 关于 w 和 b 最小化，我们分别对w，b求偏导数，即令 ∂L/∂w 和 ∂L/∂b 等于零（对w求导结果的解释请看本文评论下第45楼回复）：

$$\frac{\partial \mathcal{L}}{\partial w} = 0 \Rightarrow w = \sum_{i=1}^{n} \alpha_i y_i x_i$$
$$\frac{\partial \mathcal{L}}{\partial b} = 0 \Rightarrow \sum_{i=1}^{n} \alpha_i y_i = 0$$

以上结果代回上述的 L：

$$\mathcal{L}(w,b,\alpha) = \frac{1}{2}\|w\|^2 - \sum_{i=1}^{n} \alpha_i \Big(y_i (w^T x_i + b) - 1 \Big)$$

得到：

$$\begin{aligned}\mathcal{L}(w,b,\alpha) &= \frac{1}{2}\sum_{i,j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^T x_j - \sum_{i,j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^T x_j - b\sum_{i=1}^{n} \alpha_i y_i + \sum_{i=1}^{n} \alpha_i \\ &= \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i,j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^T x_j\end{aligned}$$

提醒：有读者可能会问上述推导过程如何而来？说实话，其具体推导过程是比较复杂的，如下图所示：

$$\mathcal{L}(\mathrm{w},\mathrm{b},\alpha)=\frac{1}{2}\|w\|^2-\sum_{i=1}^{m}\alpha_i\left[y^{(i)}\left(w^Tx^{(i)}+b\right)-1\right]$$

$$=\frac{1}{2}\mathrm{w}^T\mathrm{w}-\sum_{i=1}^{m}\alpha_i y^{(i)}w^Tx^{(i)}-\sum_{i=1}^{m}\alpha_i y^{(i)}b+\sum_{i=1}^{m}\alpha_i$$

$$=\frac{1}{2}\mathrm{w}^T\sum_{i=1}^{m}\alpha_i y^{(i)}x^{(i)}-\sum_{i=1}^{m}\alpha_i y^{(i)}w^Tx^{(i)}-\sum_{i=1}^{m}\alpha_i y^{(i)}b+\sum_{i=1}^{m}\alpha_i$$

$$=\frac{1}{2}\mathrm{w}^T\sum_{i=1}^{m}\alpha_i y^{(i)}x^{(i)}-w^T\sum_{i=1}^{m}\alpha_i y^{(i)}x^{(i)}-\sum_{i=1}^{m}\alpha_i y^{(i)}b+\sum_{i=1}^{m}\alpha_i$$

$$=-\frac{1}{2}\mathrm{w}^T\sum_{i=1}^{m}\alpha_i y^{(i)}x^{(i)}-\sum_{i=1}^{m}\alpha_i y^{(i)}b+\sum_{i=1}^{m}\alpha_i$$

$$=-\frac{1}{2}\mathrm{w}^T\sum_{i=1}^{m}\alpha_i y^{(i)}x^{(i)}-b\sum_{i=1}^{m}\alpha_i y^{(i)}+\sum_{i=1}^{m}\alpha_i$$

$$=-\frac{1}{2}\left(\sum_{i=1}^{m}\alpha_i y^{(i)}x^{(i)}\right)^T\sum_{i=1}^{m}\alpha_i y^{(i)}x^{(i)}-b\sum_{i=1}^{m}\alpha_i y^{(i)}+\sum_{i=1}^{m}\alpha_i$$

$$=-\frac{1}{2}\sum_{i=1}^{m}\alpha_i y^{(i)}\left(x^{(i)}\right)^T\sum_{i=1}^{m}\alpha_i y^{(i)}x^{(i)}-b\sum_{i=1}^{m}\alpha_i y^{(i)}+\sum_{i=1}^{m}\alpha_i$$

$$=-\frac{1}{2}\sum_{i=1,j=1}^{m}\alpha_i y^{(i)}\left(x^{(i)}\right)^T\alpha_j y^{(j)}x^{(j)}-b\sum_{i=1}^{m}\alpha_i y^{(i)}+\sum_{i=1}^{m}\alpha_i$$

最后，得到：

$$\mathcal{L}(w,b,\alpha)=\frac{1}{2}\sum_{i,j=1}^{n}\alpha_i\alpha_j y_i y_j x_i^T x_j-\sum_{i,j=1}^{n}\alpha_i\alpha_j y_i y_j x_i^T x_j-b\sum_{i=1}^{n}\alpha_i y_i+\sum_{i=1}^{n}\alpha_i$$
$$=\sum_{i=1}^{n}\alpha_i-\frac{1}{2}\sum_{i,j=1}^{n}\alpha_i\alpha_j y_i y_j x_i^T x_j$$

如 jerrylead所说:“倒数第4步”推导到“倒数第3步”使用了线性代数的转置运算,由于ai和yi都是实数,因此转置后与自身一样。“倒数第3步”推导到“倒数第2步”使用了(a+b+c+…)(a+b+c+…)=aa+ab+ac+ba+bb+bc+…的乘法运算法则。最后一步是上一步的顺序调整。

L(从上面的最后一个式子,我们可以看出,此时的拉格朗日函数只包含了一个变量,那就是α_i,然后下文的第2步,求出了α_i便能求出w,和b,由此可见,上文第1.2节提出来的核心问题:分类函数$f(x)=w^Tx+b$也就可以轻而易举的求出来了。

(2)、求对α的极大,即是关于对偶问题的最优化问题,从上面的式子得到:

(不得不提醒下读者:经过上面第一个步骤的求w和b,得到的拉格朗日函数式子已经没有了变量w,b,只有α,而反过来,求得的将能导出w,b的解,最终得出分离超平面和分类决策函数。为何呢?因为如果求出了α_i,根据$w=\sum_{i=1}^{m}\alpha_i y^{(i)}x^{(i)}$,即可求出w。然后通过$b^*=-\frac{\max_{i:y^{(i)}=-1}w^{*T}x^{(i)}+\min_{i:y^{(i)}=1}w^{*T}x^{(i)}}{2}$,即可求出b)

$$\max_{\alpha}\sum_{i=1}^{n}\alpha_i-\frac{1}{2}\sum_{i,j=1}^{n}\alpha_i\alpha_j y_i y_j x_i^T x_j$$
$$s.t.,\ \alpha_i\geq 0, i=1,\ldots,n$$
$$\sum_{i=1}^{n}\alpha_i y_i=0$$

如前面所说,这个问题有更加高效的优化算法,即我们常说的SMO算法。

2.1.4、序列最小最优化SMO算法

细心的读者读至上节末尾处，怎么拉格朗日乘子α的值可能依然心存疑惑。实际上，关于α的求解可以用一种快速学习算法即SMO算法，这里先简要介绍下。

OK，当：

$$\begin{aligned} \max_\alpha \quad & W(\alpha) = \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} y^{(i)} y^{(j)} \alpha_i \alpha_j \langle x^{(i)}, x^{(j)} \rangle \\ \text{s.t.} \quad & 0 \le \alpha_i \le C, \quad i = 1, \ldots, m \\ & \sum_{i=1}^{m} \alpha_i y^{(i)} = 0, \end{aligned}$$

要解决的是在参数$\{\alpha_1, \alpha_2, \ldots, \alpha_n\}$上求最大值W的问题，至于$x^{(i)}$和$y^{(i)}$都是已知数（其中 是一个参数，用于控制目标函数中两项（“寻找 margin 最大的超平面”和“保证数据点偏差量最小”）之间的权重。和上文最后的式子对比一下，可以看到唯一的区别就是现在 dual variable α 多了一个上限 C，关于C的具体由来请查看下文第2.3节）。

要了解这个SMO算法是如何推导的，请跳到下文第3.5节、SMO算法。
####*2.1.5、线性不可分的情况*

OK，为过渡到下节2.2节所介绍的核函数，让我们再来看看上述推导过程中得到的一些有趣的形式。首先就是关于我们的 hyper plane，对于一个数据点 x 进行分类，实际上是通过把 x 带入到 $f(x) = w^T x + b$ 算出结果然后根据其正负号来进行类别划分的。而前面的推导中我们得到

$w = \sum_{i=1}^{n} \alpha_i y_i x_i$

因此分类函数为：

$$f(x) = \left(\sum_{i=1}^{n} \alpha_i y_i x_i\right)^T x + b$$
$$= \sum_{i=1}^{n} \alpha_i y_i \langle x_i, x \rangle + b$$

这里的形式的有趣之处在于，对于新点 x的预测，只需要计算它与训练数据点的内积即可（<·,·>表示向量内积），这一点至关重要，是之后使用 Kernel 进行非线性推广的基本前提。此外，所谓 Supporting Vector 也在这里显示出来——事实上，所有非Supporting Vector 所对应的系数α都是等于零的，因此对于新点的内积计算实际上只要针对少量的“支持向量”而不是所有的训练数据即可。

为什么非支持向量对应的α等于零呢？直观上来理解的话，就是这些“后方”的点——正如我们之前分析过的一样，对超平面是没有影响的，由于分类完全有超平面决定，所以这些无关的点并不会参与分类问题的计算，因而也就不会产生任何影响了。

回忆一下我们2.1.1节中通过 Lagrange multiplier得到的目标函数：

$$\max_{\alpha_i \geq 0} \mathcal{L}(w, b, \alpha) = \max_{\alpha_i \geq 0} \frac{1}{2} \|w\|^2 - \sum_{i=1}^{n} \alpha_i \left(y_i(w^T x_i + b) - 1\right)$$

注意到如果 xi 是支持向量的话，上式中红颜色的部分是等于 0 的（因为支持向量的 functional margin 等于 1），而对于非支持向量来说，functional margin 会大于 1，因此红颜色部分是大于零的，而α_i又是非负的，为了满足最大化，α_i必须等于 0。这也就是这些非Supporting Vector 的点的局限性。

从1.5节到上述所有这些东西，便得到了一个maximum margin hyper plane classifier，这就是所谓的支持向量机（Support Vector Machine）。当然，到目前为止，我们的 SVM 还比较弱，只能处理线性的情况，不过，在得到了对偶dual 形式之后，通过 Kernel 推广到非线性的情况就变成了一件非常容易的事情了(相信，你还记得本节开头所说的：“通过求解对偶问题得到最优解，这就是线性可分条件下支持向量机的对偶算法，这样做的

优点在于:一者对偶问题往往更容易求解;二者可以自然的引入核函数,进而推广到非线性分类问题")。

2.2、核函数Kernel

2.2.1、特征空间的隐式映射:核函数

咱们首先给出核函数的来头:

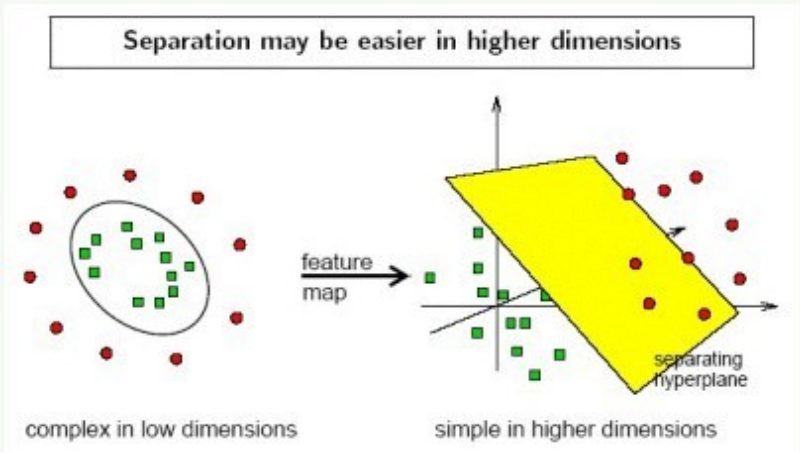
- 在上文中,我们已经了解到了SVM处理线性可分的情况,而对于非线性的情况,SVM 的处理方法是选择一个核函数 $\kappa(\cdot,\cdot)$,通过将数据映射到高维空间,来解决在原始空间中线性不可分的问题。由于核函数的优良品质,这样的非线性扩展在计算量上并没有比原来复杂多少,这一点是非常难得的。当然,这要归功于核方法——除了SVM 之外,任何将计算表示为数据点的内积的方法,都可以使用核方法进行非线性扩展。

也就是说,Minsky和Papert早就在20世纪60年代就已经明确指出线性学习器计算能力有限。为什么呢?因为总体上来讲,现实世界复杂的应用需要有比线性函数更富有表达能力的假设空间,也就是说,目标概念通常不能由给定属性的简单线性函数组合产生,而是应该一般地寻找待研究数据的更为一般化的抽象特征。

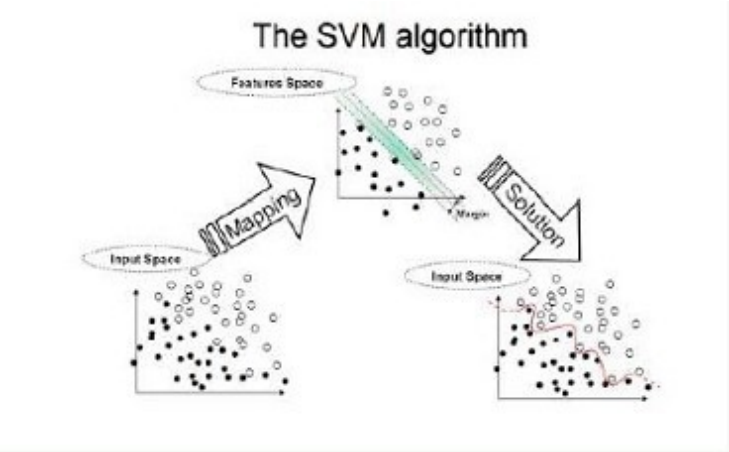
而下文我们将具体介绍的核函数则提供了此种问题的解决途径,从下文你将看到,核函数通过把数据映射到高维空间来增加第一节所述的线性学习器的能力,使得线性学习器对偶空间的表达方式让分类操作更具灵活性和可操作性。因为训练样例一般是不会独立出现的,它们总是以成对样例的内积形式出现,而用对偶形式表示学习器的优势在为在该表示中可调参数的个数不依赖输入属性的个数,通过使用恰当的核函数来替代内积,可以隐式得将非线性的训练数据映射到高维空间,而不增加可调参数的个数(当然,前提是核函数能够计算对应着两个输入特征向量的内积)。

1、 简而言之:在线性不可分的情况下,支持向量机通过某种事先选择的非线性映射(核函数)将输入变量映射到一个高维特征空间,在这个空间中构造最优分类超平面。我们使用SVM进行数据集分类工作的过程

首先是同预先选定的一些非线性映射将输入空间映射到高维特征空间(下图很清晰的表达了通过映射到高维特征空间，而把平面上本身不好分的非线性数据分了开来)：



使得在高维属性空间中有可能最训练数据实现超平面的分割，避免了在原输入空间中进行非线性曲面分割计算。SVM数据集形成的分类函数具有这样的性质：它是一组以支持向量为参数的非线性函数的线性组合，因此分类函数的表达式仅和支持向量的数量有关，而独立于空间的维度，在处理高维输入空间的分类时，这种方法尤其有效，其工作原理如下图所示：



2、具体点说：在我们遇到核函数之前，如果用原始的方法，那么在用线性学习器学习一个非线性关系，需要选择一个非线性特征集，并且将数据写成新的表达形式，这等价于应用一个固定的非线性映射，将数据映射

到特征空间，在特征空间中使用线性学习器，因此，考虑的假设集是这种类型的函数：

$$f(\mathbf{x}) = \sum_{i=1}^{N} w_i \phi_i(\mathbf{x}) + b,$$

这里ϕ：X->F是从输入空间到某个特征空间的映射，这意味着建立非线性学习器分为两步：

1. 首先使用一个非线性映射将数据变换到一个特征空间F，
2. 然后在特征空间使用线性学习器分类。

在上文我提到过对偶形式，而这个对偶形式就是线性学习器的一个重要性质，这意味着假设可以表达为训练点的线性组合，因此决策规则可以用测试点和训练点的内积来表示：

$$f(\mathbf{x}) = \sum_{i=1}^{\ell} \alpha_i y_i \langle \boldsymbol{\phi}(\mathbf{x}_i) \cdot \boldsymbol{\phi}(\mathbf{x}) \rangle + b.$$

如果有一种方式可以在特征空间中直接计算内积〈φ(xi · φ(x)〉，就像在原始输入点的函数中一样，就有可能将两个步骤融合到一起建立一个非线性的学习器，这样直接计算法的方法称为核函数方法，于是，核函数便横空出世了。

这里我直接给出一个定义：核是一个函数K，对所有x，z(-X，满足

$$K(\mathbf{x}, \mathbf{z}) = \langle \boldsymbol{\phi}(\mathbf{x}) \cdot \boldsymbol{\phi}(\mathbf{z}) \rangle,$$

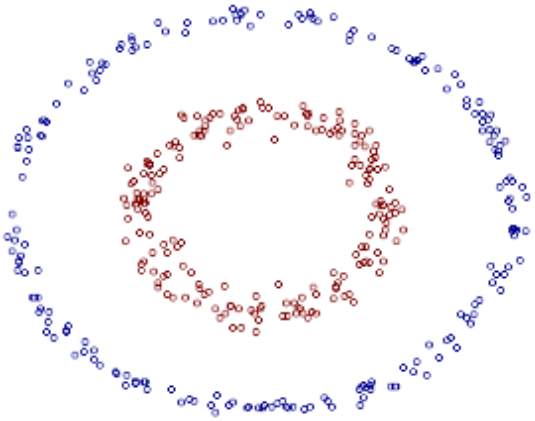
，这里φ是从X到内积特征空间F的映射。

3、 总而言之，举个简单直接点的例子，如@Wind所说：如果不是用核技术，就会先计算线性映射phy(x1)和phy(x2),然后计算这两个特征的内积，使用了核技术之后，先把phy(x1)和phy(x2)的通用表达式子：< phy(x1), phy(x2) >=k( )计算出来，注意到这里的< ， >表示内积，k(,)就是对应的核函数，这个表达往往非常简单，所以计算非常方便。

OK，接下来，咱们就进一步从外到里，来探探这个核函数的真面目。

2.2.2、核函数：如何处理非线性数据

在2.1节中我们介绍了线性情况下的支持向量机，它通过寻找一个线性的超平面来达到对数据进行分类的目的。不过，由于是线性方法，所以对非线性的数据就没有办法处理。举个例子来说，则是如下图所示的两类数据，分别分布为两个圆圈的形状，这样的数据本身就是线性不可分的，此时咱们该如何把这两类数据分开呢(下文将会有一个相应的三维空间图)？



事实上，上图所述的这个数据集，是用两个半径不同的圆圈加上了少量的噪音生成得到的，所以，一个理想的分界应该是一个“圆圈”而不是一条线（超平面）。如果用 X1 和 X2 来表示这个二维平面的两个坐标的话，我们知道一条二次曲线（圆圈是二次曲线的一种特殊情况）的方程可以写作这样的形式：

$$a_1X_1 + a_2X_1^2 + a_3X_2 + a_4X_2^2 + a_5X_1X_2 + a_6 = 0$$

注意上面的形式，如果我们构造另外一个五维的空间，其中五个坐标的值分别为 Z1=X1, Z2=X21, Z3=X2, Z4=X22, Z5=X1X2，那么显然，上面的方程在新的坐标系下可以写作：

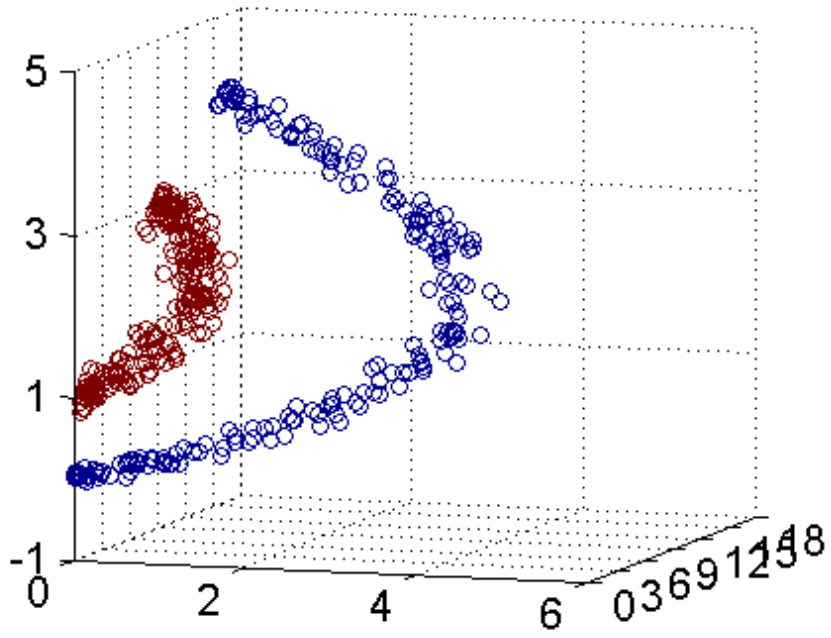
$$\sum_{i=1}^{5} a_i Z_i + a_6 = 0$$

关于新的坐标 Z，这正是一个 hyper plane 的方程！也就是说，如果我们做一个映射 ϕ:R2→R5，将 X 按照上面的规则映射为 Z，那么在新的空间中原来的数据将变成线性可分的，从而使用之前我们推导的线性分类算法就可以进行处理了。这正是 Kernel 方法处理非线性问题的基本思想。

再进一步描述 Kernel 的细节之前，不妨再来看看这个例子映射过后的直观例子。当然，你我可能无法把 5 维空间画出来，不过由于我这里生成数据的时候就是用了特殊的情形，具体来说，我这里的超平面实际的方程是这个样子（圆心在 X2 轴上的一个正圆）：

$$\sum_{i=1}^{5} a_i Z_i + a_6 = 0$$

因此我只需要把它映射到 Z1=X21, Z2=X22, Z3=X2 这样一个三维空间中即可，下图即是映射之后的结果，将坐标轴经过适当的旋转，就可以很明显地看出，数据是可以通过一个平面来分开的(pluskid：下面的 gif 动画，先用 Matlab 画出一张张图片，再用 Imagemagick 拼贴成)：



现在让我们再回到 SVM 的情形，假设原始的数据时非线性的，我们通过一个映射 ϕ(·) 将其映射到一个高维空间中，数据变得线性可分了，这个

时候，我们就可以使用原来的推导来进行计算，只是所有的推导现在是在新的空间，而不是原始空间中进行。当然，推导过程也并不是可以简单地直接类比的，例如，原本我们要求超平面的法向量 w，但是如果映射之后得到的新空间的维度是无穷维的（确实会出现这样的情况，比如后面会提到的 高斯核Gaussian Kernel），要表示一个无穷维的向量描述起来就比较麻烦。于是我们不妨先忽略过这些细节，直接从最终的结论来分析，回忆一下，我们上一次2.1节中得到的最终的分类函数是这样的：

$$f(x) = \sum_{i=1}^{n} \alpha_i y_i \langle x_i, x \rangle + b$$

现在则是在映射过后的空间，即：

$$f(x) = \sum_{i=1}^{n} \alpha_i y_i \langle \phi(x_i), \phi(x) \rangle + b$$

而其中的 α 也是通过求解如下 dual 问题而得到的：

$$\begin{aligned} &\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{n} \alpha_i \alpha_j y_i y_j \langle \phi(x_i), \phi(x_j) \rangle \\ &s.t., \alpha_i \geq 0, i = 1, \ldots, n \\ &\quad \sum_{i=1}^{n} \alpha_i y_i = 0 \end{aligned}$$

这样一来问题就解决了吗？似乎是的：拿到非线性数据，就找一个映射 ϕ(·)，然后一股脑把原来的数据映射到新空间中，再做线性 SVM 即可。不过事实上没有这么简单！其实刚才的方法稍想一下就会发现有问题：在最初的例子里，我们对一个二维空间做映射，选择的新空间是原始空间的所有一阶和二阶的组合，得到了五个维度；如果原始空间是三维，那么我们会得到 19 维的新空间，这个数目是呈爆炸性增长的，这给 ϕ(·) 的计算带来了非常大的困难，而且如果遇到无穷维的情况，就根本无从计算了。所以就需要 Kernel 出马了。

不妨还是从最开始的简单例子出发，设两个向量 $x_1 = (\eta_1, \eta_2)^T$ 和 $x_2 = (\xi_1, \xi_2)^T$，而ϕ(·)即是到前面2.2.1节说的五维空间的映射，因此映射过后的内积为：

$$\langle \phi(x_1), \phi(x_2) \rangle = \eta_1\xi_1 + \eta_1^2\xi_1^2 + \eta_2\xi_2 + \eta_2^2\xi_2^2 + \eta_1\eta_2\xi_1\xi_2$$

（公式说明：上面的这两个推导过程中，所说的前面的五维空间的映射，这里说的前面便是文中2.2.1节的所述的映射方式，仔细看下2.2.1节的映射规则，再看那第一个推导，其实就是计算x1，x2各自的内积，然后相乘相加即可，第二个推导则是直接平方，去掉括号，也很容易推出来）

另外，我们又注意到：

$$(\langle x_1, x_2 \rangle + 1)^2 = 2\eta_1\xi_1 + \eta_1^2\xi_1^2 + 2\eta_2\xi_2 + \eta_2^2\xi_2^2 + 2\eta_1\eta_2\xi_1\xi_2 + 1$$

二者有很多相似的地方，实际上，我们只要把某几个维度线性缩放一下，然后再加上一个常数维度，具体来说，上面这个式子的计算结果实际上和映射

$$\varphi(X_1, X_2) = (\sqrt{2}X_1, X_1^2, \sqrt{2}X_2, X_2^2, \sqrt{2}X_1X_2, 1)^T$$

之后的内积 $\langle \varphi(x_1), \varphi(x_2) \rangle$ 的结果是相等的，那么区别在于什么地方呢？

一个是映射到高维空间中，然后再根据内积的公式进行计算；而另一个则直接在原来的低维空间中进行计算，而不需要显式地写出映射后的结果。（公式说明：上面之中，最后的两个式子，第一个算式，是带内积的完全平方式，可以拆开，然后，通过凑一个得到，第二个算式，也是根据第一个算式凑出来的）

回忆刚才提到的映射的维度爆炸，在前一种方法已经无法计算的情况下，后一种方法却依旧能从容处理，甚至是无穷维度的情况也没有问题。

我们把这里的计算两个向量在隐式映射过后的空间中的内积的函数叫做核函数 (Kernel Function)，例如，在刚才的例子中，我们的核函数为：

$$\kappa(x_1, x_2) = (\langle x_1, x_2 \rangle + 1)^2$$

核函数能简化映射空间中的内积运算——刚好“碰巧”的是，在我们的SVM 里需要计算的地方数据向量总是以内积的形式出现的。对比刚才我们上面写出来的式子，现在我们的分类函数为：

$$\sum_{i=1}^{n} \alpha_i y_i \kappa(x_i, x) + b$$

其中 由如下 dual 问题计算而得：

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{n} \alpha_i \alpha_j y_i y_j \kappa(x_i, x_j)$$
$$s.t., \alpha_i \geq 0, i = 1, \ldots, n$$
$$\sum_{i=1}^{n} \alpha_i y_i = 0$$

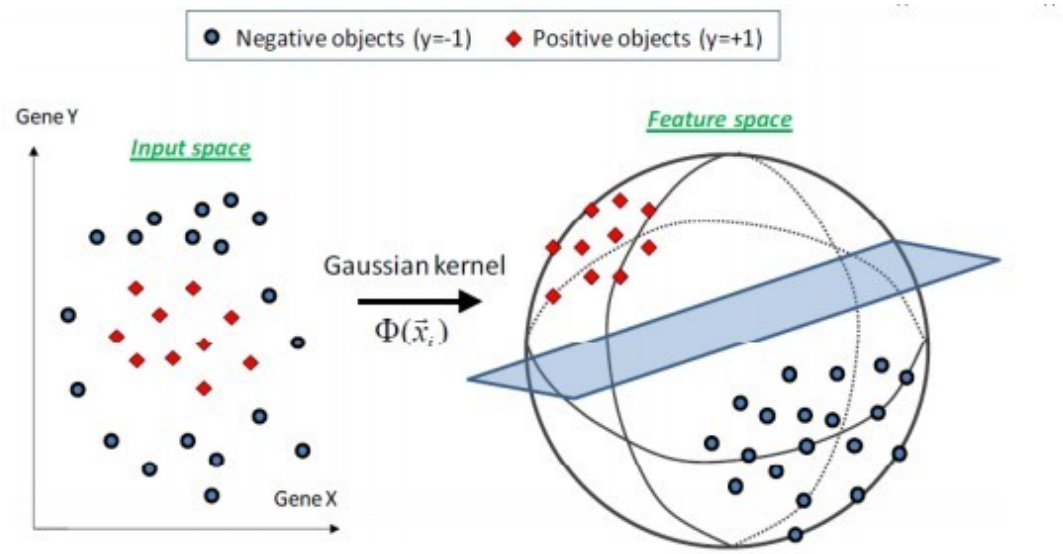
这样一来计算的问题就算解决了，避开了直接在高维空间中进行计算，而结果却是等价的！当然，因为我们这里的例子非常简单，所以我可以手工构造出对应于ϕ(·)的核函数出来，如果对于任意一个映射，想要构造出对应的核函数就很困难了。

2.2.3、几个核函数

通常人们会从一些常用的核函数中选择（根据问题和数据的不同，选择不同的参数，实际上就是得到了不同的核函数），例如：

- 多项式核 $\kappa(x_1, x_2) = (\langle x_1, x_2 \rangle + R)^d$，显然刚才我们举的例子是这里多项式核的一个特例（R = 1, d = 2）。虽然比较麻烦，而且没有必要，不过这个核所对应的映射实际上是可以写出来的，该空间的维度是$\binom{m+d}{d}$，其中m是原始空间的维度。

- 高斯核 $\kappa(x_1, x_2) = \exp\left(-\frac{\|x_1 - x_2\|^2}{2\sigma^2}\right)$，这个核就是最开始提到过的会将原始空间映射为无穷维空间的那个家伙。不过，如果σ选得很大的话，高次特征上的权重实际上衰减得非常快，所以实际上（数值上近似一下）相当于一个低维的子空间；反过来，如果σ选得很小，则可以将任意的数据映射为线性可分——当然，这并不一定是好事，因为随之而来的可能是非常严重的过拟合问题。不过，总的来说，通过调控参数σ，高斯核实际上具有相当高的灵活性，也是使用最广泛的核函数之一。下图所示的例子便是把低维线性不可分的数据通过高斯核函数映射到了高维空间：



- 线性核 $\kappa(x_1, x_2) = \langle x_1, x_2 \rangle$，这实际上就是原始空间中的内积。这个核存在的主要目的是使得“映射后空间中的问题”和“映射前空间中的问题”两者在形式上统一起来了(意思是说，咱们有的时候，写代码，或写公式的时候，只要写个模板或通用表达式，然后再代入不同的核，便可以了，于此，便在形式上统一了起来，不用再分别写一个线性的，和一个非线性的)。

2.2.4、核函数的本质

上面说了这么一大堆，读者可能还是没明白核函数到底是个什么东西？我再简要概括下，即以下三点：

1. 实际中，我们会经常遇到线性不可分的样例，此时，我们的常用做法是把样例特征映射到高维空间中去(如上文2.2节最开始的那幅图所示，映射到高维空间后，相关特征便被分开了，也就达到了分类的目的)；

2. 但进一步，如果凡是遇到线性不可分的样例，一律映射到高维空间，那么这个维度大小是会高到可怕的(如上文中19维乃至无穷维的例子)。那咋办呢？

3. 此时，核函数就隆重登场了，核函数的价值在于它虽然也是讲特征进行从低维到高维的转换，但核函数绝就绝在它事先在低维上进行计算，而将实质上的分类效果表现在了高维上，也就如上文所说的避免了直接在高维空间中的复杂计算。

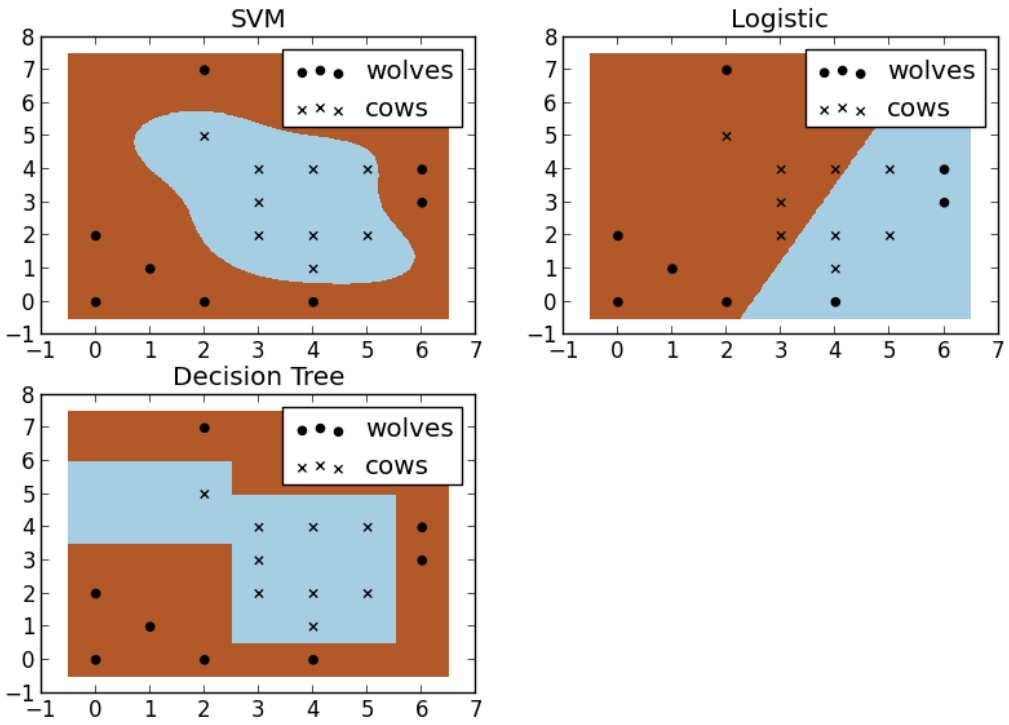
经过前面内容的讲解，我们已经知道，当把内积$<x_i,x_j>$变成$<\Phi(x_i),\Phi(x_j)>$之后，求$<\Phi(x_i),\Phi(x_j)>$将有两种方法：

1、先找到这种映射，然后将输入空间中的样本映射到新的空间中，最后在新空间中去求内积$<\Phi(x_i),\Phi(x_j)>$。以多项式$x_1+x_2+x_1^2+x_2^2+c=0$为例，对其进行变换，$c_1=x_1$，$c_2=x_2$，$c_3=x_1^2$，$c_4=x_2^2$，得到：$c_1+c_2+c_3+c_4+c=0$，也就是说通过把输入空间从二维向四维映射后，样本由线性不可分变成了线性可分，但是这种转化带来的直接问题是维度变高了，这意味着，首先可能导致后续计算变复杂，其次可能出现维度之咒，对于学习器而言就是：特征空间维数可能最终无法计算，而它的泛化能力(学习器对训练样本以外数据的适应性)会随着维度的增长而大大降低，这也违反了“奥坎姆的剃刀”，最终可能会使得内积$<\Phi(x_i),\Phi(x_j)>$无法求出，于是也就失去了这种转化的优势了；

1、或者是找到某种方法，它不需要显式的将输入空间中的样本映射到新的空间中而能够在输入空间中直接计算出内积$<\Phi(x_i),\Phi(x_j)>$。它其实是对输入空间向高维空间的一种隐式映射，它不需要显式的给出那个映射，在输入空间就可以计算$<\Phi(x_i),\Phi(x_j)>$，这就是传说中的核函数方法。

最后引用这里的一个例子举例说明下核函数解决非线性问题的直观效果。

假设现在你是一个农场主，圈养了一批羊群，但为预防狼群袭击羊群，你需要搭建一个篱笆来把羊群围起来。但是篱笆应该建在哪里呢？你很可能需要依据牛群和狼群的位置建立一个“分类器”，比较下图这几种不同的分类器，我们可以看到SVM完成了一个很完美的解决方案。



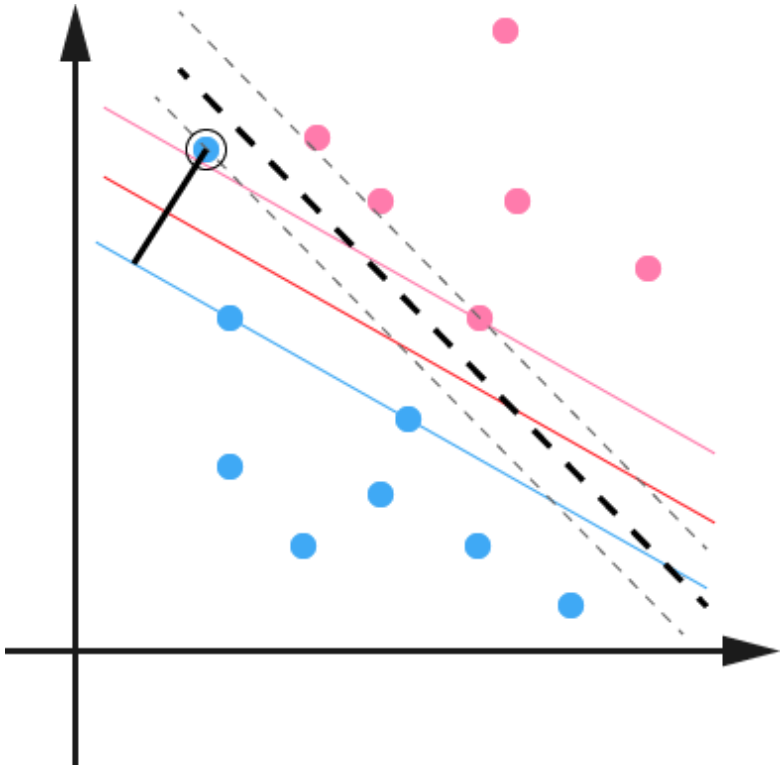
这个例子从侧面简单说明了SVM使用非线性分类器的优势，而逻辑模式以及决策树模式都是使用了直线方法。

OK，不再做过多介绍了，对核函数有进一步兴趣的，还可以看看此文。

2.3、使用松弛变量处理 outliers 方法

在本文第一节最开始讨论支持向量机的时候，我们就假定，数据是线性可分的，亦即我们可以找到一个可行的超平面将数据完全分开。后来为了处理非线性数据，在上文2.2节使用 Kernel 方法对原来的线性 SVM 进行了推广，使得非线性的的情况也能处理。虽然通过映射 $\phi(\cdot)$ 将原始数据映射到高维空间之后，能够线性分隔的概率大大增加，但是对于某些情况还是很难处理。

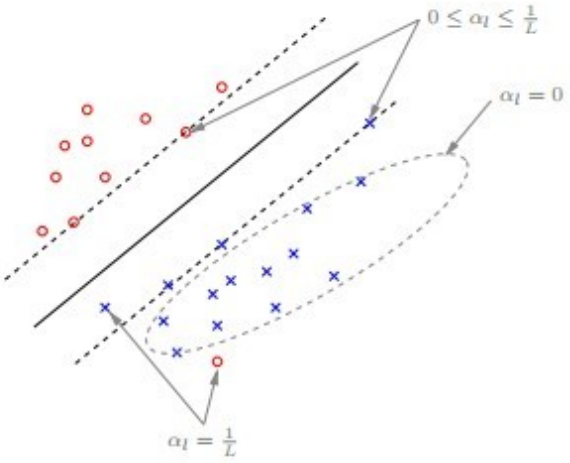
例如可能并不是因为数据本身是非线性结构的，而只是因为数据有噪音。对于这种偏离正常位置很远的数据点，我们称之为 outlier，在我们原来的 SVM 模型里，outlier 的存在有可能造成很大的影响，因为超平面本身就是只有少数几个 support vector 组成的，如果这些 support vector 里又存在 outlier 的话，其影响就很大了。例如下图：



用黑圈圈起来的那个蓝点是一个 outlier，它偏离了自己原本所应该在的那个半空间，如果直接忽略掉它的话，原来的分隔超平面还是挺好的，但是由于这个 outlier 的出现，导致分隔超平面不得不被挤歪了，变成途中黑色虚线所示（这只是一个示意图，并没有严格计算精确坐标），同时 margin 也相应变小了。当然，更严重的情况是，如果这个 outlier 再往右上移动一些距离的话，我们将无法构造出能将数据分开的超平面来。

为了处理这种情况，SVM 允许数据点在一定程度上偏离一下超平面。例如上图中，黑色实线所对应的距离，就是该 outlier 偏离的距离，如果把它移动回来，就刚好落在原来的超平面上，而不会使得超平面发生变形了。

插播下一位读者@Copper_PKU的理解:“换言之,在有松弛的情况下outline点也属于支持向量SV,同时,对于不同的支持向量,拉格朗日参数的值也不同,如此篇论文《Large Scale Machine Learning》中的下图所示:



对于远离分类平面的点值为0;对于边缘上的点值在[0, 1/L]之间,其中,L为训练数据集个数,即数据集大小;对于outline数据和内部的数据值为1/L。更多请参看本文文末参考条目第51条。”

OK,继续回到咱们的问题。我们,原来的约束条件为:

$$y_i(w^T x_i + b) \geq 1, \quad i = 1, \ldots, n$$

现在考虑到outlier问题,约束条件变成了:

$$y_i(w^T x_i + b) \geq 1 - \xi_i, \quad i = 1, \ldots, n$$

其中 $\xi_i \geq 0$ 称为松弛变量 (slack variable),对应数据点 x_i 允许偏离的 functional margin 的量。当然,如果我们运行 ξ_i 任意大的话,那任意的超平面都是符合条件的了。所以,我们在原来的目标函数后面加上一项,使得这些 ξ_i 的总和也要最小:

$$\min \frac{1}{2} \|w\|^2 + C \sum_{i=1}^{n} \xi_i$$

其中 C 是一个参数，用于控制目标函数中两项（“寻找 margin 最大的超平面”和“保证数据点偏差量最小”）之间的权重。注意，其中 ξ 是需要优化的变量（之一），而 C 是一个事先确定好的常量。完整地写出来是这个样子：

$$\min \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i$$
$$s.t., y_i(w^T x_i + b) \geq 1 - \xi_i, i = 1, \ldots, n$$
$$\xi_i \geq 0, i = 1, \ldots, n$$

用之前的方法将限制或约束条件加入到目标函数中，得到新的拉格朗日函数，如下所示：

$$\mathcal{L}(w, b, \xi, \alpha, r) = \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i - \sum_{i=1}^{n}\alpha_i\left(y_i(w^T x_i + b) - 1 + \xi_i\right) - \sum_{i=1}^{n} r_i\xi_i$$

分析方法和前面一样，转换为另一个问题之后，我们先让$\mathcal{L}$针对w、b和ξ_i最小化：

$$\frac{\partial \mathcal{L}}{\partial w} = 0 \Rightarrow w = \sum_{i=1}^{n}\alpha_i y_i x_i$$
$$\frac{\partial \mathcal{L}}{\partial b} = 0 \Rightarrow \sum_{i=1}^{n}\alpha_i y_i = 0$$
$$\frac{\partial \mathcal{L}}{\partial \xi_i} = 0 \Rightarrow C - \alpha_i - r_i = 0, \quad i = 1, \ldots, n$$

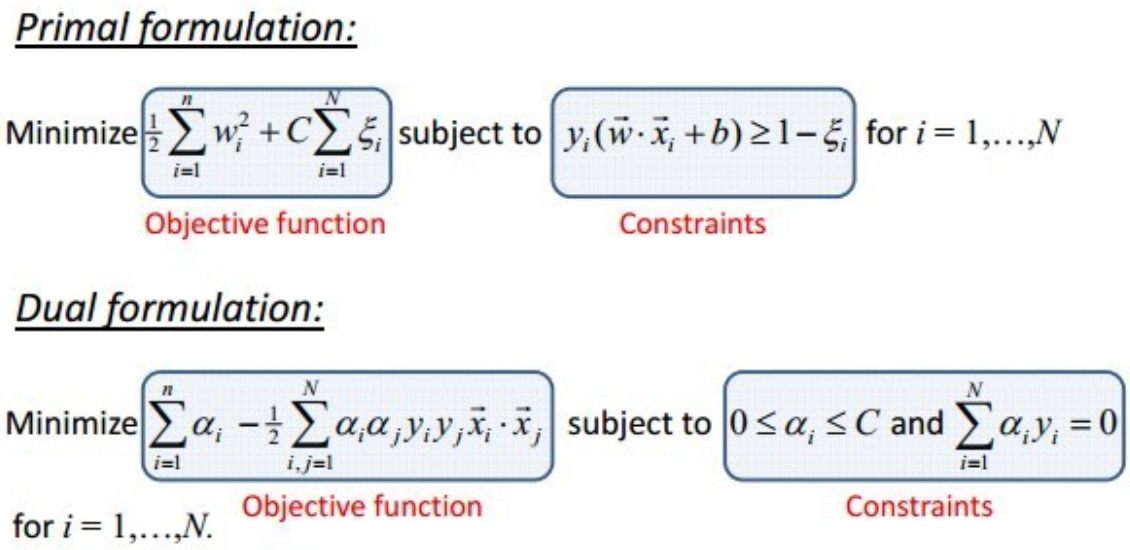
将 w 带回 $\mathcal{L}$ 并化简，得到和原来一样的目标函数：

$$\max_{\alpha}\sum_{i=1}^{n}\alpha_i - \frac{1}{2}\sum_{i,j=1}^{n}\alpha_i\alpha_j y_i y_j \langle x_i, x_j\rangle$$

不过，由于我们得到 $C - \alpha_i - r_i = 0$ 而又有 ri >= 0（作为 Lagrange multiplier 的条件），因此有 $\alpha_i \leq C$，所以整个 dual 问题现在写作：

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{n} \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle$$
$$s.t., 0 \le \alpha_i \le C, i = 1, \ldots, n$$
$$\sum_{i=1}^{n} \alpha_i y_i = 0$$

把前后的结果对比一下（错误修正：图中的Dual formulation中的Minimize应为maxmize）：



可以看到唯一的区别就是现在 dual variable α 多了一个上限 C 。而 Kernel 化的非线性形式也是一样的，只要把 $\langle x_i, x_j \rangle$ 换成 $\kappa(x_i, x_j)$ 即可。这样一来，一个完整的，可以处理线性和非线性并能容忍噪音和 outliers 的支持向量机才终于介绍完毕了。

行文至此，可以做个小结，不准确的说，SVM它本质上即是一个分类方法，用$w^{T}+b$定义分类函数，于是求w、b，为寻最大间隔，引出$1/2\|w\|2$，继而引入拉格朗日因子，化为对拉格朗日乘子a的求解（求解过程中会涉及到一系列最优化或凸二次规划等问题），如此，求w.b与求a等价，而a的求解可以用一种快速学习算法SMO，至于核函数，是为处理非线性情况，若直接映射到高维计算恐维度爆炸，故在低维计算，等效高维表现。

OK,理解到这第二层,已经能满足绝大部分人一窥SVM原理的好奇心,然对于那些想在证明层面理解SVM的则还很不够,但进入第三层理解境界之前,你必须要有比较好的数理基础和逻辑证明能力,不然你会跟我一样,吃不少苦头的。

第三层、证明SVM

说实话，凡是涉及到要证明的东西.理论，便一般不是怎么好惹的东西。绝大部分时候，看懂一个东西不难，但证明一个东西则需要点数学功底，进一步，证明一个东西也不是特别难，难的是从零开始发明创造这个东西的时候，则显艰难(因为任何时代，大部分人的研究所得都不过是基于前人的研究成果，前人所做的是开创性工作，而这往往是最艰难最有价值的，他们被称为真正的先驱。牛顿也曾说过，他不过是站在巨人的肩上。你，我则更是如此)。

正如陈希孺院士在他的著作《数理统计学简史》的第4章、最小二乘法中所讲:在科研上诸多观念的革新和突破是有着很多的不易的,或许某个定理在某个时期由某个人点破了,现在的我们看来一切都是理所当然,但在一切没有发现之前,可能许许多多的顶级学者毕其功于一役,耗尽一生,努力了几十年最终也是无功而返。

话休絮烦，要证明一个东西先要弄清楚它的根基在哪，即构成它的基础是哪些理论。OK,以下内容基本是上文中未讲到的一些定理的证明,包括其背后的逻辑、来源背景等东西,还是读书笔记。

本部分导述

- 3.1节线性学习器中，主要阐述感知机算法；
- 3.2节非线性学习器中，主要阐述mercer定理；
- 3.3节、损失函数；
- 3.4节、最小二乘法；
- 3.5节、SMO算法；
- 3.6节、简略谈谈SVM的应用；
- 3.1、线性学习器

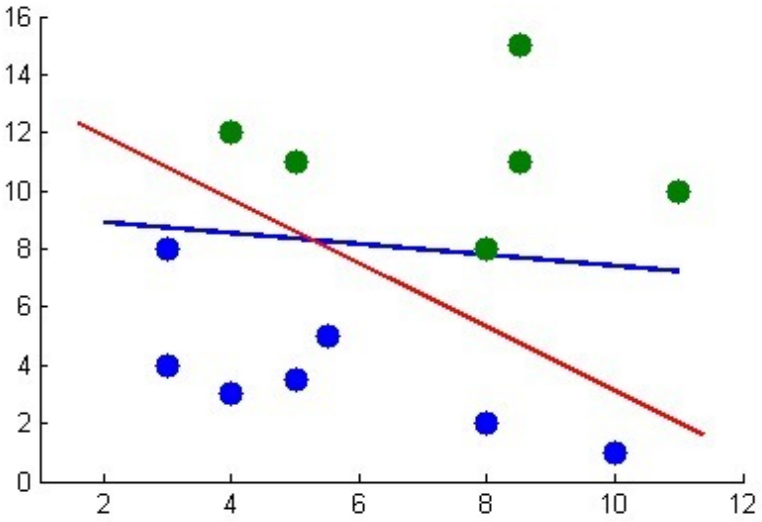
3.1.1、感知机算法

这个感知机算法是1956年提出的，年代久远，依然影响着当今，当然，可以肯定的是，此算法亦非最优，后续会有更详尽阐述。不过，有一点，你必须清楚，这个算法是为了干嘛的：不断的训练试错以期寻找一个合适的超平面(是的，就这么简单)。

表 2.1　感知机算法（原始形式）

给定线性可分的数据集S和学习率$\eta \in \mathbb{R}^+$
$\mathbf{w}_0 \leftarrow \mathbf{0}; b_0 \leftarrow 0; k \leftarrow 0$
$R \leftarrow \max_{1 \leqslant i \leqslant \ell} \|\mathbf{x}_i\|$
重复
　　for $i = 1$ to ℓ
　　　　if $y_i((\mathbf{w}_k \cdot \mathbf{x}_i) + b_k) \leqslant 0$ then
　　　　　　$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k + \eta y_i \mathbf{x}_i$
　　　　　　$b_{k+1} \leftarrow b_k + \eta y_i R^2$
　　　　　　$k \leftarrow k + 1$
　　　　end if
　　end for
直到在for 循环中没有错误发生
返回$(\mathbf{w}_k, b_k)$，这里k是错误次数

下面，举个例子。如下图所示，凭我们的直觉可以看出，图中的红线是最优超平面，蓝线则是根据感知机算法在不断的训练中，最终，若蓝线能通过不断的训练移动到红线位置上，则代表训练成功。

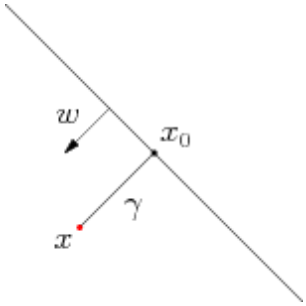


既然需要通过不断的训练以让蓝线最终成为最优分类超平面，那么，到底需要训练多少次呢？Novikoff定理告诉我们当间隔是正的时候感知机算法会在有限次数的迭代中收敛，也就是说Novikoff定理证明了感知机算法的收敛性，即能得到一个界，不至于无穷循环下去。

- Novikoff定理：如果分类超平面存在，仅需在序列 S 上迭代几次，在界为$\left(\frac{2R}{\gamma}\right)^2$的错误次数下就可以找到分类超平面，算法停止。

这里$R=\max_{1\le i\le l}\|\mathbf{x}_i\|$，为扩充间隔。根据误分次数公式可知，迭代次数与对应于扩充(包括偏置)权重的训练集的间隔有关。

顺便再解释下这个所谓的扩充间隔γ，γ即为样本到分类间隔的距离，即从γ引出的最大分类间隔。OK，还记得上文第1.3.2节开头的内容么？如下：“



在给出几何间隔的定义之前，咱们首先来看下，如上图所示，对于一个点 x，令其垂直投影到超平面上的对应的为 x0，由于 w 是垂直于超平面的一个向量，γ 为样本x到分类间隔的距离，我们有

$$x = x_0 + \gamma \frac{w}{\|w\|}$$

然后后续怎么推导出最大分类间隔请回到本文第一、二部分，此处不重复板书。

同时有一点得注意：感知机算法虽然可以通过简单迭代对线性可分数据生成正确分类的超平面，但不是最优效果，那怎样才能得到最优效果呢，就是上文中第一部分所讲的寻找最大分类间隔超平面。此外，Novikoff定理的证明请见这里。

3.2、非线性学习器

3.2.1 、Mercer定理

Mercer定理：如果函数K是 $\mathbb{R}^n \times \mathbb{R}^n \to \mathbb{R}$ 上的映射（也就是从两个n维向量映射到实数域）。那么如果K是一个有效核函数（也称为Mercer核函数），那么当且仅当对于训练样例 $\{x^{(1)}, x^{(2)}, \ldots, x^{(m)}\}$，其相应的核函数矩阵是对称半正定的。

要理解这个Mercer定理，先要了解什么是半正定矩阵，要了解什么是半正定矩阵，先得知道什么是正定矩阵（矩阵理论“博大精深”，我自己也未能彻底理清，等我理清了再续写此节，顺便推荐我正在看的一本《矩阵分析与应用》）。然后这里有一个此定理的证明，可以看下。

正如@Copper_PKU所说：核函数在SVM的分类效果中起了重要的作用，最后这里有个tutorial可以看看。

3.3、损失函数

在本文1.0节有这么一句话“支持向量机(SVM)是90年代中期发展起来的基于统计学习理论的一种机器学习方法，通过寻求结构化风险最小来提

高学习机泛化能力，实现经验风险和置信范围的最小化，从而达到在统计样本量较少的情况下，亦能获得良好统计规律的目的。”但初次看到的读者可能并不了解什么是结构化风险，什么又是经验风险。要了解这两个所谓的“风险”，还得又从监督学习说起。

监督学习实际上就是一个经验风险或者结构风险函数的最优化问题。风险函数度量平均意义下模型预测的好坏，模型每一次预测的好坏用损失函数来度量。它从假设空间F中选择模型f作为决策函数，对于给定的输入X，由f(X)给出相应的输出Y，这个输出的预测值f(X)与真实值Y可能一致也可能不一致，用一个损失函数来度量预测错误的程度。损失函数记为L(Y, f(X))。

常用的损失函数有以下几种（基本引用自《统计学习方法》）：

（1）0-1 损失函数

$$L(Y,f(X))=\begin{cases}1, Y\neq f(X)\\ 0, Y=f(X)\end{cases}$$

（2）平方损失函数

$$L(Y,f(X))=(Y-f(X))^2$$

（3）绝对损失函数

$$L(Y,f(X))=|Y-f(X)|$$

（4）对数损失函数

$$L(Y,P(Y|X))=-\log P(Y|X)$$

给定一个训练数据集

$$T=\{(x_1,y_1),(x_2,y_2),\ldots(x_i,y_i)\ldots,(x_N,y_N)\}$$

模型 f(X)关于训练数据集的平均损失称为经验风险，如下：

$$R_{emp}(f)=\frac{1}{N}\sum_{i=1}^{N}L(y_i,f(x_i))$$

关于如何选择模型，监督学习有两种策略：经验风险最小化和结构风险最小化。

经验风险最小化的策略认为，经验风险最小的模型就是最优的模型，则按照经验风险最小化求最优模型就是求解如下最优化问题：

$$\min_{f \in F} \frac{1}{N} \sum_{i=1}^{N} L(y_i, f(x_i)) \qquad (1)$$

当样本容量很小时，经验风险最小化的策略容易产生过拟合的现象。结构风险最小化可以防止过拟合。结构风险是在经验风险的基础上加上表示模型复杂度的正则化项或罚项，结构风险定义如下：

$$R_{srm}(f) = \frac{1}{N} \sum_{i=1}^{N} L(y_i, f(x_i)) + \lambda J(f)$$

其中 J(f)为模型的复杂度，模型 f 越复杂，J(f)值就越大，模型越简单，J(f)值就越小，也就是说 J(f)是对复杂模型的惩罚。$\lambda \geq 0$ 是系数，用以权衡经验风险和模型复杂度。结构风险最小化的策略认为结构风险最小的模型是最优的模型，所以求最优的模型就是求解下面的最优化问题：

$$\min_{f \in F} \frac{1}{N} \sum_{i=1}^{N} L(y_i, f(x_i)) + \lambda J(f) \qquad (2)$$

这样，监督学习问题就变成了经验风险或结构风险函数的最优化问题，如式（1）和式（2）。

如此，SVM有第二种理解，即最优化+损失最小，或如@夏粉_百度所说“可从损失函数和优化算法角度看SVM, boosting, LR等算法，可能会有不同收获”。

OK, 关于更多统计学习方法的问题，请参看此文。

关于损失函数，如下文读者评论中所述：可以看看张潼的这篇《Statistical behavior and consistency of classification methods based on convex risk minimization》。各种算法中常用的损失函数基本都具有fisher一致性，优化这些损失函数得到的分类器可以看作是后验概率的“代理”。

此外，他还有另外一篇论文《Statistical analysis of some multi-category large margin classification methods》，在多分类情况下margin loss的分析，这两篇对Boosting和SVM使用的损失函数分析的很透彻。

3.4、最小二乘法

3.4.1、什么是最小二乘法？

既然本节开始之前提到了最小二乘法，那么下面引用《正态分布的前世今生》里的内容稍微简单阐述下。

我们口头中经常说：一般来说，平均来说。如平均来说，不吸烟的健康优于吸烟者，之所以要加“平均”二字，是因为凡事皆有例外，总存在某个特别的人他吸烟但由于经常锻炼所以他的健康状况可能会优于他身边不吸烟的朋友。而最小二乘法的一个最简单的例子便是算术平均。

最小二乘法（又称最小平方法）是一种数学优化技术。它通过最小化误差的平方和寻找数据的最佳函数匹配。利用最小二乘法可以简便地求得未知的数据，并使得这些求得的数据与实际数据之间误差的平方和为最小。用函数表示为：

$$\min_{\vec{x}} \sum_{i=1}^{n} (y_m - y_i)^2.$$

使误差「所谓误差，当然是观察值与实际真实值的差量」平方和达到最小以寻求估计值的方法，就叫做最小二乘法，用最小二乘法得到的估计，叫做最小二乘估计。当然，取平方和作为目标函数只是众多可取的方法之一。

最小二乘法的一般形式可表示为：

$$\min_{\vec{x}} \|\vec{y}_m(\vec{x}) - \vec{y}\|_2 \ .$$

有效的最小二乘法是勒让德在 1805 年发表的，基本思想就是认为测量中有误差，所以所有方程的累积误差为

$$\text{累积误差} = \sum (\text{观测值} - \text{理论值})^2$$

我们求解出导致累积误差最小的参数即可：

$$\begin{aligned}\hat{\beta} &= argmin_{\beta}\sum_{i=1}^{n}e_i^2 \\ &= argmin_{\beta}\sum_{i=1}^{n}[y_i-(\beta_0+\beta_1 x_{1i}+\cdots+\beta_p x_{pi})]^2\end{aligned}$$

勒让德在论文中对最小二乘法的优良性做了几点说明：

最小二乘使得误差平方和最小，并在各个方程的误差之间建立了一种平衡，从而防止某一个极端误差取得支配地位

计算中只要求偏导后求解线性方程组，计算过程明确便捷

最小二乘可以导出算术平均值作为估计值

对于最后一点，从统计学的角度来看是很重要的一个性质。推理如下：假设真值为 θ, x1,⋯,xn为n次测量值, 每次测量的误差为ei=xi−θ，按最小二乘法，误差累积为

$$L(\theta)=\sum_{i=1}^{n}e_i^2=\sum_{i=1}^{n}(x_i-\theta)^2$$

求解 θ 使达到最小$L(\theta)$，正好是算术平均$\bar{x}=\frac{\sum_{i-1}^{n}x_i}{n}$。

由于算术平均是一个历经考验的方法，而以上的推理说明，算术平均是最小二乘的一个特例，所以从另一个角度说明了最小二乘方法的优良性，使我们对最小二乘法更加有信心。

最小二乘法发表之后很快得到了大家的认可接受，并迅速的在数据分析实践中被广泛使用。不过历史上又有人把最小二乘法的发明归功于高斯，这又是怎么一回事呢。高斯在1809年也发表了最小二乘法，并且声称自己已经使用这个方法多年。高斯发明了小行星定位的数学方法，并在数据分析中使用最小二乘方法进行计算，准确的预测了谷神星的位置。

说了这么多，貌似跟本文的主题SVM没啥关系呀，别急，请让我继续阐述。本质上说，最小二乘法即是一种参数估计方法，说到参数估计，咱们

得从一元线性模型说起。

3.4.2、最小二乘法的解法

什么是一元线性模型呢？请允许我引用这里的内容，先来梳理下几个基本概念：

- 监督学习中，如果预测的变量是离散的，我们称其为分类（如决策树，支持向量机等），如果预测的变量是连续的，我们称其为回归。

- 回归分析中，如果只包括一个自变量和一个因变量，且二者的关系可用一条直线近似表示，这种回归分析称为一元线性回归分析。

- 如果回归分析中包括两个或两个以上的自变量，且因变量和自变量之间是线性关系，则称为多元线性回归分析。对于二维空间线性是一条直线；对于三维空间线性是一个平面，对于多维空间线性是一个超平面…

- 对于一元线性回归模型，假设从总体中获取了n组观察值（X1，Y1），（X2，Y2），…，（Xn，Yn）。对于平面中的这n个点，可以使用无数条曲线来拟合。要求样本回归函数尽可能好地拟合这组值。综合起来看，这条直线处于样本数据的中心位置最合理。

选择最佳拟合曲线的标准可以确定为：使总的拟合误差（即总残差）达到最小。有以下三个标准可以选择：

1. 用“残差和最小”确定直线位置是一个途径。但很快发现计算“残差和”存在相互抵消的问题。
2. 用“残差绝对值和最小”确定直线位置也是一个途径。但绝对值的计算比较麻烦。
3. 最小二乘法的原则是以“残差平方和最小”确定直线位置。用最小二乘法除了计算比较方便外，得到的估计量还具有优良特性。这种方法对异常值非常敏感。

最常用的是普通最小二乘法（Ordinary Least Square，OLS）：所选择的回归模型应该使所有观察值的残差平方和达到最小，即采用平方损失函

数。

我们定义样本回归模型为：

$$Y_i = \hat{\beta}_0 + \hat{\beta}_1 X_i + e_i$$
$$\Rightarrow e_i = Y_i - \hat{\beta}_0 - \hat{\beta}_1 X_i$$

其中ei为样本（Xi, Yi）的误差。接着，定义平方损失函数Q：

$$Q = \sum_{i=1}^{n} e_i^2 = \sum_{i=1}^{n} (Y_i - \hat{Y}_i)^2 = \sum_{i=1}^{n} (Y_i - \hat{\beta}_0 - \hat{\beta}_1 X_i)^2$$

则通过Q最小确定这条直线，即确定$\hat{\beta}_0, \hat{\beta}_1$，以$\hat{\beta}_0, \hat{\beta}_1$为变量，把它们看作是Q的函数，就变成了一个求极值的问题，可以通过求导数得到。

求Q对两个待估参数的偏导数：

$$\begin{cases} \dfrac{\partial Q}{\partial \hat{\beta}_0} = 2\sum_{i=1}^{n} (Y_i - \hat{\beta}_0 - \hat{\beta}_1 X_i)(-1) = 0 \\ \dfrac{\partial Q}{\partial \hat{\beta}_1} = 2\sum_{i=1}^{n} (Y_i - \hat{\beta}_0 - \hat{\beta}_1 X_i)(-X_i) = 0 \end{cases}$$

根据数学知识我们知道，函数的极值点为偏导为0的点。

解得：

$$\hat{\beta}_2 = \frac{n\sum X_i Y_i - \sum X_i \sum Y_i}{n\sum X_i^2 - (\sum X_i)^2}$$

$$\hat{\beta}_1 = \frac{\sum X_i^2 \sum Y_i - \sum X_i \sum X_i Y_i}{n\sum X_i^2 - (\sum X_i)^2}$$

这就是最小二乘法的解法，就是求得平方损失函数的极值点。自此，你看到求解最小二乘法与求解SVM问题何等相似，尤其是定义损失函数，而后通过偏导求得极值。

OK，更多请参看陈希孺院士的《数理统计学简史》的第4章、最小二乘法，和本文参考条目第59条《凸函数》。

3.5、SMO算法

在上文2.1.2节中，我们提到了求解对偶问题的序列最小最优化SMO算法，但并未提到其具体解法。

事实上，SMO算法是由Microsoft Research的John C. Platt在1998年发表的一篇论文《Sequential Minimal Optimization A Fast Algorithm for Training Support Vector Machines》中提出，它很快成为最快的二次规划优化算法，特别针对线性SVM和数据稀疏时性能更优。

接下来，咱们便参考John C. Platt的这篇文章来看看SMO的解法是怎样的。

3.5.1、SMO算法的解法

咱们首先来定义特征到结果的输出函数为

$$u = \vec{w} \cdot \vec{x} - b$$

再三强调，这个u与我们之前定义的$f(x)=w^Tx+b$实质是一样的。

接着，咱们重新定义咱们原始的优化问题，权当重新回顾，如下：

$$\min_{w,b}\frac{1}{2}\|\vec{w}\|^2 \text{ subject to } y_i(\vec{w}\cdot\vec{x}_i-b)\geq 1,\forall i$$

求导得到：

$$\vec{w}=\sum_{i=1}^{N}y_i\alpha_i\vec{x}_i,\quad b=\vec{w}\cdot\vec{x}_k-y_k \text{ for some } \alpha_k>0$$

代入$u=\vec{w}\cdot\vec{x}-b$中，可得 $$u=\sum_{j=1}^{N}y_j\alpha_j K(\vec{x}_j,\vec{x})-b$$ 。

引入对偶因子后，得：

$$\min_{\vec{\alpha}}\Psi(\vec{\alpha})=\min_{\vec{\alpha}}\frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N}y_iy_j(\vec{x}_i\cdot\vec{x}_j)\alpha_i\alpha_j-\sum_{i=1}^{N}\alpha_i$$

s.t：$\alpha_i\geq 0,\forall i,$ 且 $\sum_{i=1}^{N}y_i\alpha_i=0.$

注：这里得到的min函数与我们之前的max函数实质也是一样，因为把符号变下，即有min转化为max的问题，且yi也与之前的$y^{(i)}$等价，yj亦如此。

经过加入松弛变量后，模型修改为：

$$\min_{w,b,\xi}\frac{1}{2}\|\vec{w}\|^2+C\sum_{i=1}^{N}\xi_i \text{ subject to } y_i(\vec{w}\cdot\vec{x}_i-b)\geq 1-\xi_i,\forall i$$

$$0\leq\alpha_i\leq C,\forall i$$

从而最终我们的问题变为：

$$\min_{\vec{\alpha}} \Psi(\vec{\alpha}) = \min_{\vec{\alpha}} \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} y_i y_j K(\vec{x}_i, \vec{x}_j) \alpha_i \alpha_j - \sum_{i=1}^{N} \alpha_i$$
$$0 \le \alpha_i \le C, \forall i,$$
$$\sum_{i=1}^{N} y_i \alpha_i = 0.$$

继而，根据KKT条件可以得出其中取值的意义为：

$$\alpha_i = 0 \Leftrightarrow y_i u_i \ge 1,$$
$$0 < \alpha_i < C \Leftrightarrow y_i u_i = 1,$$
$$\alpha_i = C \Leftrightarrow y_i u_i \le 1.$$

这里的还是拉格朗日乘子(问题通过拉格朗日乘法数来求解)

1. 对于第1种情况，表明α_i是正常分类，在边界内部（我们知道正确分类的点yi*f(xi)>=0）；

2. 对于第2种情况，表明了α_i是支持向量，在边界上；

3. 对于第3种情况，表明了α_i是在两条边界之间；

而最优解需要满足KKT条件，即上述3个条件都得满足，以下几种情况出现将会出现不满足：

$y_i u_i$<=1但是α_i=1但是α_i>0则是不满足的而原本α_i=0

$y_i u_i$=1但是α_i=0或者α_i=C则表明不满足的，而原本应该是0。

3. @孤独之守望者：“最后，推出svm的cost function 是hinge loss，然后对比其他的方法的cost function，说明其实他们的目标函数很像，那么问题是svm为什么这么popular呢？您可以再加些VC dimension跟一些error bound的数学，点一下，提供一个思路和方向”。– 。

4. @夏粉_百度:“在面试时,考察SVM可考察机器学习各方面能力:目标函数,优化过程,并行方法,算法收敛性,样本复杂度,适用场景,调参经验,不过个人认为考察boosting和LR也还不错啊。此外,随着统计机器学习不断进步,SVM只被当成使用了一个替代01损失hinge研究,更通用的方法被提出,损失函数研究替代损失与贝叶斯损失关系,算法稳定性研究替代损失与推广性能关系,凸优化研究如何求解凸目标函数,SVM,boosting等算法只是这些通用方法的一个具体组建而已。”

5. @居里猴姐:关于SVM损失函数的问题，可以看看张潼老师的这篇《Statistical behavior and consistency of classification methods based on convex risk minimization》。各种算法中常用的损失函数基本都具有fisher一致性,优化这些损失函数得到的分类器可以看作是后验概率的“代理”。此外,张潼老师还有另外一篇论文《Statistical analysis of some multi-category large margin classification methods》,在多分类情况下margin loss的分析,这两篇对Boosting和SVM使用的损失函数分析的很透彻。

6. @夏粉_百度:SVM用了hinge损失，hinge损失不可导，不如其它替代损失方便优化并且转换概率麻烦。核函数也不太用，现在是大数据时代，样本非常大，无法想象一个n^2的核矩阵如何存储和计算。而且,现在现在非线性一般靠深度学习了。//@Copper_PKU:请教svm在工业界的应用典型的有哪些？工业界如何选取核函数,经验的方法？svm的训练过程如何优化？

7. @Copper_PKU:July的svm tutorial 我个人觉得还可以加入和修改如下部分:(1) 对于支持向量解释,可以结合图和拉格朗日参数来表达,松弛中sv没有写出来. (2) SMO算法部分,加入Joachims论文中提到的算法，以及SMO算法选取workset的方法,包括SMO算法的收敛判断,还有之前共轭梯度求解方法,虽然是较早的算法,但是对于理解SMO算法有很好的效果。模型的优化和求解都是迭代的过程,加入历史算法增强立体感。–http://weibo.com/1580904460/Akw6dl3Yk#_rnd1385474436177。

8. //@廖临川: 之所以sgd对大训练集的效果更好，1.因为SGD优化每次迭代使用样本子集，比使用训练全集（尤其是百万数量级）要

快得多；2.如果目标函数是凸的或者伪凸的，SGD几乎必然可以收敛到全局最优;否则,则收敛到局部最优;3.SGD一般不需要收敛到全局最优，只要得到足够好的解，就可以立即停止。//@Copper_PKU:sgd的核心思想:是迭代训练,每拿到一个样本就算出基于当前w(t) 的loss function,t代表训练第t次,然后进行下一w(t+1)的更新,w(t+1)=w(t)-(learning rate) * loss function的梯度,这个类比神经网络中bp中的参数训练方法。sample by sample就是每次仅处理一个样本 而不是一个batch。

9. //@Copper_PKU:从损失函数角度说:primal问题可以理解为正则化项+lossfunction,求解目标是在两个中间取平衡 如果强调loss function最小则会overfitting,所以有C参数。//@研究者July:SVM还真就是在一定限定条件下，即约束条件下求目标函数的最优值问题，同时，为减少误判率，尽量让损失最小。
10.
11. … 非常享受这种全民大讨论的年代,没有谁一定就对或一定就错,而是各自发表各自的理解见解,真棒!

参考文献及推荐阅读

1. 《支持向量机导论》,[美] Nello Cristianini / John Shawe-Taylor 著;
2. 支持向量机导论一书的支持网站:;
3. 《数据挖掘导论》,[美] Pang-Ning Tan / Michael Steinbach / Vipin Kumar 著;
4. 《数据挖掘:概念与技术》,(加)Jiawei Han;Micheline Kamber 著;
5. 《数据挖掘中的新方法:支持向量机》,邓乃扬 田英杰 著;
6. 《支持向量机–理论、算法和扩展》,邓乃扬 田英杰 著;
7. 支持向量机系列,pluskid:;
8. ;
9. 数据挖掘十大经典算法初探;
10. 《模式识别支持向量机指南》,C.J.C Burges 著;
11. 《统计学习方法》，李航著(第7章有不少内容参考自支持向量机导论一书，不过，可以翻翻看看);
12. 《统计自然语言处理》，宗成庆编著，第十二章、文本分类;
13. SVM入门系列,Jasper:;
14. 最近邻决策和SVM数字识别的实现和比较,作者不详;

15. 斯坦福大学机器学习课程原始讲义:;
16. 斯坦福机器学习课程笔记:;
17. ;
18. SMO算法的数学推导:;
19. 数据挖掘掘中所需的概率论与数理统计知识、上;
20. 关于机器学习方面的文章,可以读读:;
21. 数学系教材推荐:;
22. 《神经网络与机器学习(原书第三版)》,[加] Simon Haykin 著;
23. 正态分布的前世今生:;
24. 《数理统计学简史》,陈希孺院士著;
25. 《最优化理论与算法(第2版)》,陈宝林编著;
26. A Gentle Introduction to Support Vector Machines in Biomedicine:,此PPT很赞,除了对引入拉格朗日对偶变量后的凸二次规划问题的深入度不够之外,其它都挺好,配图很精彩,本文有几张图便引自此PPT中;
27. 来自卡内基梅隆大学carnegie mellon university(CMU)的讲解SVM的PPT:;
28. 发明libsvm的台湾林智仁教授06年的机器学习讲义SVM:;
29. ;
30. Introduction to Support Vector Machines (SVM)，By Debprakash Patnai M.E (SSA)，https://www.google.com.hk/url?sa=t&rct=j&q=&esrc=s&source=web&cd=1&ved=0CCwQFjAA&url=http%3a%2f%2fwww%2epws%2estu%2eedu%2etw%2fccfang%2findex%2efiles%2fAI%2fAI%26ML-Support%2520Vector%2520Machine-1%2eppt&ei=JRR6UqT5C-iyiQfWyIDgCg&usg=AFQjCNGw1fTbpH4ltQjjmx1d25ZqbCN9nA;
31. 多人推荐过的libsvm:;
32. 《machine learning in action》,中文版为《机器学习实战》;
33. SMO算法的提出:Sequential Minimal Optimization A Fast Algorithm for Training Support Vector Machines:;
34. 《统计学习理论的本质》，[美] Vladimir N. Vapnik著,非常晦涩,不做过多推荐;
35. 张兆翔，机器学习第五讲之支持向量机;
36. VC维的理论解释：http://www.svms.org/vc-dimension/>,中文VC维解释<http://xiaoxia001.iteye.com/blog/1163338;
37. 来自NEC Labs America的Jason Weston关于SVM的讲义;

38. 来自MIT的SVM讲义：；
39. PAC问题：；
40. 百度张潼老师的两篇论文：《Statistical behavior and consistency of classification methods based on convex risk minimization》http://home.olemiss.edu/~xdang/676/Consistency_of_Classification_Convex_Risk_Minimization.pdf，《Statistical analysis of some multi-category large margin classification methods》；
41. ；
42. 《矩阵分析与应用》，清华张贤达著；
43. SMO算法的实现：；
44. 常见面试之机器学习算法思想简单梳理：；
45. 矩阵的wikipedia页面：；
46. 最小二乘法及其实现：；
47. 统计学习方法概论：；
48. ；
49. A Tutorial on Support Vector Regression：http://alex.smola.org/papers/2003/SmoSch03b.pdf>；SVR 简明版：<http://www.cmlab.csie.ntu.edu.tw/~cyy/learning/tutorials/SVR.pdf。
50. SVM Org：；
51. R. Collobert. Large Scale Machine Learning. Université Paris VI phd thesis. 2004：；
52. Making Large-Scale SVM Learning Practical：；
53. 文本分类与SVM：；
54. Working Set Selection Using Second Order Information for Training Support Vector Machines：；
55. SVM Optimization: Inverse Dependence on Training Set Size：；
56. Large-Scale Support Vector Machines: Algorithms and Theory：；
57. 凸优化的概念：；
58. 《凸优化》，作者：Stephen Boyd / Lieven Vandenberghe，原作名：Convex Optimization；
59. Large-scale Non-linear Classification: Algorithms and Evaluations，Zhuang Wang，讲了很多SVM算法的新进展：；

后记

OK，此文从最初2012年5月开始动笔，到后续不断的修改，创造了三个之最，即所写时间最长，所花心血最大，所改次数最多，因为我的目标是让没有任何机器学习基础的都能看懂此文，所以总是不停的改，不停的改，不想放过任何一个小的细节。再者，引用侯捷的一句话是：天下大作，必作于细。

最后，非常感谢pluskid及诸多朋友们的文章及著作，让我有机会在其基础上总结、深入。有任何问题，敬请广大读者随时不吝批评指正，感谢。

updated、本文PDF版

- 13年11月25日，用chrome浏览器打开文章，右键打印，弹出打印框，把左上角的目标更改为“另存为PDF”，成第一个PDF：。
- 13年12月7日，用“印象笔记”提取出博客正文，放到office内编辑成此PDF：，较上一版本添加了完整的书签。
- 14年 2月18日，用Latex全部重排了本文所有公式，而且给所有公式和图片全部做了标记，Latex版下载地址为：。

本文会一直不断翻新，再者，上述3个PDF的阅读体验也还不是最好的，如果有朋友制作了更好的PDF，欢迎分享给我：，谢谢。

July、二零一四年二月十一日于天通苑。

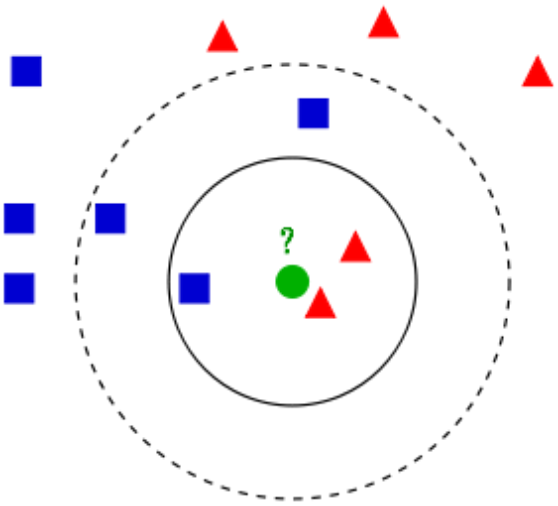
K近邻算法与距离度量表示法

第一部分、K近邻算法

1.1、什么是K近邻算法

何谓K近邻算法，即K-Nearest Neighbor algorithm，简称KNN算法，单从名字来猜想，可以简单粗暴的认为是：K个最近的邻居，当K=1时，算法便成了最近邻算法，即寻找最近的那个邻居。为何要找邻居？打个比方来说，假设你来到一个陌生的村庄，现在你要找到与你有着相似特征的人群融入他们，所谓入伙。

用官方的话来说，所谓K近邻算法，即是给定一个训练数据集，对新的输入实例，在训练数据集中找到与该实例最邻近的K个实例（也就是上面所说的K个邻居），这K个实例的多数属于某个类，就把该输入实例分类到这个类中。根据这个说法，咱们来看下引自维基百科上的一幅图：



如上图所示，有两类不同的样本数据，分别用蓝色的小正方形和红色的小三角形表示，而图正中间的那个绿色的圆所标示的数据则是待分类的数据。也就是说，现在，我们不知道中间那个绿色的数据是从属于哪一类（蓝色小正方形or红色小三角形），下面，我们就要解决这个问题：给这个绿色的圆分类。

- 我们常说，物以类聚，人以群分，判别一个人是一个什么样品质特征的人，常常可以从他/她身边的朋友入手，所谓观其友，而识其人。我们不是要判别上图中那个绿色的圆是属于哪一类数据么，好说，从它的邻居下手。但一次性看多少个邻居呢？从上图中，你还能看到：
- 如果K=3，绿色圆点的最近的3个邻居是2个红色小三角形和1个蓝色小正方形，少数从属于多数，基于统计的方法，判定绿色的这个待分类点属于红色的三角形一类。 如果K=5，绿色圆点的最近的5个邻居是2个红色三角形和3个蓝色的正方形，还是少数从属于多数，基于统计的方法，判定绿色的这个待分类点属于蓝色的正方形一类。

于此我们看到，当无法判定当前待分类点是从属于已知分类中的哪一类时，我们可以依据统计学的理论看它所处的位置特征，衡量它周围邻居的权重，而把它归为(或分配)到权重更大的那一类。这就是K近邻算法的核心思想。

1.2、近邻的距离度量表示法

上文第一节，我们看到，K近邻算法的核心在于找到实例点的邻居，这个时候，问题就接踵而至了，如何找到邻居，邻居的判定标准是什么，用什么来度量。这一系列问题便是下面要讲的距离度量表示法。但有的读者可能就有疑问了，我是要找邻居，找相似性，怎么又跟距离扯上关系了？

这是因为特征空间中两个实例点的距离和反应出两个实例点之间的相似性程度。K近邻模型的特征空间一般是n维实数向量空间，使用的距离可以使欧式距离，也是可以是其它距离，既然扯到了距离，下面就来具体阐述下都有哪些距离度量的表示法，权当扩展。

1.**欧氏距离**，最常见的两点之间或多点之间的距离表示法，又称之为欧几里得度量，它定义于欧几里得空间中，如点 x = (x1,…,xn) 和 y = (y1,…,yn) 之间的距离为：

$$d(x,y) := \sqrt{(x_1-y_1)^2+(x_2-y_2)^2+\cdots+(x_n-y_n)^2} = \sqrt{\sum_{i=1}^{n}(x_i-y_i)^2}$$

(1)二维平面上两点a(x1,y1)与b(x2,y2)间的欧氏距离：

$$d_{12} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

(2)三维空间两点a(x1,y1,z1)与b(x2,y2,z2)间的欧氏距离：

$$d_{12} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}$$

(3)两个n维向量a(x11,x12,…,x1n)与 b(x21,x22,…,x2n)间的欧氏距离：

$$d_{12} = \sqrt{\sum_{k=1}^{n} (x_{1k} - x_{2k})^2}$$

也可以用表示成向量运算的形式：

$$d_{12} = \sqrt{(a - b)(a - b)^T}$$

其上，二维平面上两点欧式距离，代码可以如下编写：

```
//unixfy:计算欧氏距离
double euclideanDistance(const vector<double>& v1, const vector<double>& v2)
{
     assert(v1.size() == v2.size());
     double ret = 0.0;
     for (vector<double>::size_type i = 0; i != v1.size(); ++i)
     {
         ret += (v1[i] - v2[i]) * (v1[i] - v2[i]);
     }
     return sqrt(ret);
 }
```

2.**曼哈顿距离**，我们可以定义曼哈顿距离的正式意义为L1-距离或城市区块距离，也就是在欧几里得空间的固定直角坐标系上两点所形成的线段对轴产生的投影的距离总和。例如在平面上，坐标（x1, y1）的点P1与坐标（x2, y2）的点P2的曼哈顿距离为：$|x_1 - x_2| + |y_1 - y_2|$，要注意的是，曼哈顿距离依赖座标系统的转度，而非系统在座标轴上的平移或映射。

通俗来讲，想象你在曼哈顿要从一个十字路口开车到另外一个十字路口，驾驶距离是两点间的直线距离吗？显然不是，除非你能穿越大楼。而

实际驾驶距离就是这个“曼哈顿距离”，此即曼哈顿距离名称的来源，同时，曼哈顿距离也称为城市街区距离(City Block distance)。

(1)二维平面两点a(x1,y1)与b(x2,y2)间的曼哈顿距离

$$d_{12} = |x_1 - x_2| + |y_1 - y_2|$$

(2)两个n维向量a(x11,x12,…,x1n)与 b(x21,x22,…,x2n)间的曼哈顿距离

$$\mathrm{d}_{12} = \sum_{k=1}^{n} |\mathrm{x}_{1k} - x_{2k}|$$

3.**切比雪夫距离**，若二个向量或二个点p 、and q，其座标分别为及，则两者之间的切比雪夫距离定义如下：$D_{\text{Chebyshev}}(p,q) := \max_i(|p_i - q_i|)$.，

这也等于以下Lp度量的极值：$\lim_{k\to\infty}\left(\sum_{i=1}^{n}|p_i - q_i|^k\right)^{1/k}$，因此切比雪夫距离也称为L∞度量。

以数学的观点来看，切比雪夫距离是由一致范数（uniform norm）（或称为上确界范数）所衍生的度量，也是超凸度量（injective metric space）的一种。

在平面几何中，若二点p及q的直角坐标系坐标为(x_1, y_1)及(x_2, y_2)，则切比雪夫距离为：$D_{\text{Chess}} = \max\left(|x_2 - x_1|, |y_2 - y_1|\right)$.。

玩过国际象棋的朋友或许知道，国王走一步能够移动到相邻的8个方格中的任意一个。那么国王从格子(x1,y1)走到格子(x2,y2)最少需要多少步？。你会发现最少步数总是max(| x2-x1 | , | y2-y1 |) 步 。有一种类似的一种距离度量方法叫切比雪夫距离。

(1)二维平面两点a(x1,y1)与b(x2,y2)间的切比雪夫距离

$$d_{12} = \max(|x_1 - x_2|, |y_1 - y_2|)$$

(2)两个n维向量a(x11,x12,…,x1n)与 b(x21,x22,…,x2n)间的切比雪夫距离

$$d_{12} = \max_i(|x_{1i} - x_{2i}|)$$

这个公式的另一种等价形式是 $$\mathrm{d}_{12} = \lim_{k\to\infty}\left(\sum_{i=1}^{n}|x_{1i} - x_{2i}|^k\right)^{1/k}$$

4.**闵可夫斯基距离(Minkowski Distance)**，闵氏距离不是一种距离，而是一组距离的定义。

1. 闵氏距离的定义

两个n维变量a(x11,x12,…,x1n)与 b(x21,x22,…,x2n)间的闵可夫斯基距离定义为：

$$d_{12} = \sqrt[p]{\sum_{k=1}^{n}|x_{1k} - x_{2k}|^p}$$

其中p是一个变参数。

当p=1时，就是曼哈顿距离

当p=2时，就是欧氏距离

当p→∞时，就是切比雪夫距离

根据变参数的不同，闵氏距离可以表示一类的距离。

5.**标准化欧氏距离 (Standardized Euclidean distance)**，标准化欧氏距离是针对简单欧氏距离的缺点而作的一种改进方案。标准欧氏距离的思路：既然数据各维分量的分布不一样，那先将各个分量都“标准化”到均值、方差相等。至于均值和方差标准化到多少，先复习点统计学知识。

假设样本集X的数学期望或均值(mean)为m，标准差(standard deviation，方差开根)为s，那么X的“标准化变量”X*表示为：(X-m)/s，而且标准化变量的数学期望为0，方差为1。即，样本集的标准化过程(standardization)用公式描述就是：

$$X^* = \frac{X - m}{s}$$

标准化后的值 = (标准化前的值 － 分量的均值) /分量的标准差

经过简单的推导就可以得到两个n维向量a(x11,x12,…,x1n)与b(x21,x22,…,x2n)间的标准化欧氏距离的公式：

$$d_{12} = \sqrt{\sum_{k=1}^{n} \left(\frac{x_{1k} - x_{2k}}{s_k} \right)^2}$$

如果将方差的倒数看成是一个权重，这个公式可以看成是一种加权欧氏距离(Weighted Euclidean distance)。

6.**马氏距离(Mahalanobis Distance)**

（1）马氏距离定义

有M个样本向量X1~Xm，协方差矩阵记为S，均值记为向量μ，则其中样本向量X到u的马氏距离表示为：

$$D(X) = \sqrt{(X - \mu)^T S^{-1} (X - \mu)}$$

（协方差矩阵中每个元素是各个矢量元素之间的协方差Cov(X,Y)，Cov(X,Y) = E{ [X-E(X)] [Y-E(Y)]}，其中E为数学期望）

而其中向量Xi与Xj之间的马氏距离定义为：

$$D(X_i, X_j) = \sqrt{(X_i - X_j)^T S^{-1} (X_i - X_j)}$$

若协方差矩阵是单位矩阵（各个样本向量之间独立同分布），则公式就成了：

$$D(X_i,X_j)=\sqrt{(X_i-X_j)^T(X_i-X_j)}$$

也就是欧氏距离了。

若协方差矩阵是对角矩阵，公式变成了标准化欧氏距离。

(2)马氏距离的优缺点：量纲无关，排除变量之间的相关性的干扰。

「微博上的seafood高清版点评道：原来马氏距离是根据协方差矩阵演变，一直被老师误导了，怪不得看Killian在05年NIPS发表的LMNN论文时候老是看到协方差矩阵和半正定，原来是这回事」

7.**巴氏距离（Bhattacharyya Distance）**，在统计中，Bhattacharyya距离测量两个离散或连续概率分布的相似性。它与衡量两个统计样品或种群之间的重叠量的Bhattacharyya系数密切相关。Bhattacharyya距离和Bhattacharyya系数以20世纪30年代曾在印度统计研究所工作的一个统计学家A. Bhattacharya命名。同时，Bhattacharyya系数可以被用来确定两个样本被认为相对接近的，它是用来测量中的类分类的可分离性。

(1)巴氏距离的定义

对于离散概率分布 p和q在同一域 X，它被定义为：

$$D_B(p,q)=-\ln\left(BC(p,q)\right)$$

其中：

$$BC(p,q)=\sum_{x\in X}\sqrt{p(x)q(x)}$$

是Bhattacharyya系数。

对于连续概率分布，Bhattacharyya系数被定义为：

$$BC(p,q)=\int\sqrt{p(x)q(x)}\,dx$$

在 $0\le BC\le 1$ and $0\le D_B\le\infty$ 这两种情况下，巴氏距离D_B并没有服从三角不等式.（值得一提的是，Hellinger距离不服从三角不等式

$0 \leq D_B \leq \infty D_B \sqrt{1 - BC}$)。

对于多变量的高斯分布 $p_i = N(m_i, P_i)$ ，$D_B = \frac{1}{8}(m_1 - m_2)^T P^{-1}(m_1 - m_2) + \frac{1}{2} \ln \left(\frac{\det P}{\sqrt{\det P_1 \det P_2}} \right)$，和是手段和协方差的分布 $P = \frac{P_1 + P_2}{2}$ 。

需要注意的是，在这种情况下，第一项中的Bhattacharyya距离与马氏距离有关联。

（2）Bhattacharyya系数

Bhattacharyya系数是两个统计样本之间的重叠量的近似测量，可以被用于确定被考虑的两个样本的相对接近。

计算Bhattacharyya系数涉及集成的基本形式的两个样本的重叠的时间间隔的值的两个样本被分裂成一个选定的分区数，并且在每个分区中的每个样品的成员的数量，在下面的公式中使用

$$\text{Bhattacharyya} = \sum_{i=1}^{n} \sqrt{(\Sigma \mathbf{a}_i \cdot \Sigma \mathbf{b}_i)}$$

考虑样品a 和 b ，n是的分区数，并且 $\Sigma \mathbf{a}_i$, $\Sigma \mathbf{b}_i$ 被一个 和 b i的日分区中的样本数量的成员。更多介绍请参看：。

8.**汉明距离(Hamming distance)**， 两个等长字符串s1与s2之间的汉明距离定义为将其中一个变为另外一个所需要作的最小替换次数。例如字符串“1111”与“1001”之间的汉明距离为2。应用：信息编码（为了增强容错性，应使得编码间的最小汉明距离尽可能大）。或许，你还没明白我再说什么，不急，看下上篇blog中第78题的第3小题整理的一道面试题目，便一目了然了。如下图所示：

3.给定一个源串和目标串，能够对源串进行如下操作：

1.在给定位置上插入一个字符

2.替换任意字符

3.删除任意字符

写一个程序，返回最小操作数，使得对源串进行这些操作后等于目标串，源串和目标串的长度都小于2000。

点评：

1、此题反复出现，如上文第38题第4小题9月26日百度一二面试题，10月9日腾讯面试题第1小题，及上面第69题10月13日百度2013校招北京站笔试题第二大道题第3小题，同时，还可以看下这个链

```
//动态规划：

//f[i,j]表示s[0...i]与t[0...j]的最小编辑距离。
f[i,j] = min { f[i-1,j]+1,  f[i,j-1]+1,  f[i-1,j-1]+(s[i]==t[j]?0:1) }

//分别表示:添加1个,删除1个,替换1个(相同就不用替换)。
```

与此同时，面试官还可以继续问下去：那么，请问，如何设计一个比较两篇文章相似性的算法？（这个问题的讨论可以看看这里：http://t.cn/zl82CAH> ，及这里关于simhash算法的介绍：<http://www.cnblogs.com/linecong/archive/2010/08/28/simhash.html），接下来，便引出了下文关于夹角余弦的讨论。（*上篇blog中第78题的第3小题给出了多种方法，读者可以参看之。同时，程序员编程艺术系列第二十八章将详细阐述这个问题*）

9.**夹角余弦(Cosine)**，几何中夹角余弦可用来衡量两个向量方向的差异，机器学习中借用这一概念来衡量样本向量之间的差异。

(1)在二维空间中向量A(x1,y1)与向量B(x2,y2)的夹角余弦公式：

$$\cos\theta = \frac{x_1x_2 + y_1y_2}{\sqrt{x_1^2 + y_1^2}\sqrt{x_2^2 + y_2^2}}$$

2. 两个n维样本点a(x11,x12,…,x1n)和b(x21,x22,…,x2n)的夹角余弦

$$\cos(\theta) = \frac{a \cdot b}{|a|\,|b|}$$

类似的，对于两个n维样本点a(x11,x12,…,x1n)和b(x21,x22,…,x2n)，可以使用类似于夹角余弦的概念来衡量它们间的相似程度，即：

$$\cos(\theta) = \frac{\sum_{k=1}^{n} x_{1k}\, x_{2k}}{\sqrt{\sum_{k=1}^{n} x_{1k}^{\,2}}\ \sqrt{\sum_{k=1}^{n} x_{2k}^{\,2}}}$$

夹角余弦取值范围为[-1,1]。夹角余弦越大表示两个向量的夹角越小，夹角余弦越小表示两向量的夹角越大。当两个向量的方向重合时夹角余弦取最大值1，当两个向量的方向完全相反夹角余弦取最小值-1。

10.**杰卡德相似系数(Jaccard similarity coefficient)**

1. 杰卡德相似系数

两个集合A和B的交集元素在A，B的并集中所占的比例，称为两个集合的杰卡德相似系数，用符号J(A,B)表示。

$$\mathrm{J}(\mathrm{A},\mathrm{B}) = \frac{|A \cap B|}{|A \cup B|}$$

杰卡德相似系数是衡量两个集合的相似度一种指标。

2. 杰卡德距离

与杰卡德相似系数相反的概念是杰卡德距离(Jaccard distance)。

杰卡德距离可用如下公式表示：

$$\mathrm{J}_{\delta}(\mathrm{A},\mathrm{B}) = 1 - J(A,B) = \frac{|A \cup B| - |A \cap B|}{|A \cup B|}$$

杰卡德距离用两个集合中不同元素占所有元素的比例来衡量两个集合的区分度。

3. 杰卡德相似系数与杰卡德距离的应用

可将杰卡德相似系数用在衡量样本的相似度上。

举例：样本A与样本B是两个n维向量，而且所有维度的取值都是0或1，例如：A(0111)和B(1011)。我们将样本看成是一个集合，1表示集合包含该元素，0表示集合不包含该元素。

M11 :样本A与B都是1的维度的个数

M01:样本A是0,样本B是1的维度的个数

M10:样本A是1,样本B是0 的维度的个数

M00:样本A与B都是0的维度的个数

依据上文给的杰卡德相似系数及杰卡德距离的相关定义,样本A与B的杰卡德相似系数J可以表示为:

$$J = \frac{M_{11}}{M_{01} + M_{10} + M_{11}}.$$

这里M11+M01+M10可理解为A与B的并集的元素个数,而M11是A与B的交集的元素个数。而样本A与B的杰卡德距离表示为J':

$$J' = \frac{M_{01} + M_{10}}{M_{01} + M_{10} + M_{11}}.$$

11.**皮尔逊系数(Pearson Correlation Coefficient)**

在具体阐述皮尔逊相关系数之前,有必要解释下什么是相关系数 (Correlation coefficient)与相关距离(Correlation distance)。

相关系数 (Correlation coefficient)的定义是:

$$\rho_{XY} = \frac{\mathrm{Cov(X,Y)}}{\sqrt{D(X)}\sqrt{D(Y)}} = \frac{E((\mathrm{X} - \mathrm{EX})(Y - EY))}{\sqrt{D(X)}\sqrt{D(Y)}}$$

(其中,E为数学期望或均值,D为方差,D开根号为标准差,E{ [X-E(X)] [Y-E(Y)]}称为随机变量X与Y的协方差,记为Cov(X,Y),即Cov(X,Y) = E{ [X-E(X)] [Y-E(Y)]},而两个变量之间的协方差和标准差的商则称为随机变量X与Y的相关系数,记为ρ_{XY})

相关系数衡量随机变量X与Y相关程度的一种方法,相关系数的取值范围是[-1,1]。相关系数的绝对值越大,则表明X与Y相关度越高。当X与Y线性相关时,相关系数取值为1(正线性相关)或-1(负线性相关)。

具体的，如果有两个变量：X、Y，最终计算出的相关系数的含义可以有如下理解：

当相关系数为0时，X和Y两变量无关系。

当X的值增大（减小），Y值增大（减小），两个变量为正相关，相关系数在0.00与1.00之间。

当X的值增大（减小），Y值减小（增大），两个变量为负相关，相关系数在-1.00与0.00之间。

相关距离的定义是：

$$D_{xy} = 1 - \rho_{XY}$$

OK，接下来，咱们来重点了解下皮尔逊相关系数。

在统计学中，皮尔逊积矩相关系数（英语：Pearson product-moment correlation coefficient，又称作 PPMCC或PCCs，用r表示）用于度量两个变量X和Y之间的相关（线性相关），其值介于-1与1之间。

通常情况下通过以下取值范围判断变量的相关强度：

相关系数
0.8-1.0 极强相关
0.6-0.8 强相关
0.4-0.6 中等程度相关
0.2-0.4 弱相关
0.0-0.2 极弱相关或无相关

在自然科学领域中，该系数广泛用于度量两个变量之间的相关程度。它是由卡尔·皮尔逊从弗朗西斯·高尔顿在19世纪80年代提出的一个相似却又稍有不同的想法演变而来的。这个相关系数也称作“皮尔森相关系数r”。

(1)皮尔逊系数的定义：

两个变量之间的皮尔逊相关系数定义为两个变量之间的协方差和标准差的商：

$$\rho_{X,Y} = \frac{\operatorname{cov}(X,Y)}{\sigma_X \sigma_Y} = \frac{E[(X-\mu_X)(Y-\mu_Y)]}{\sigma_X \sigma_Y},$$

以上方程定义了总体相关系数, 一般表示成希腊字母ρ(rho)。基于样本对协方差和方差进行估计, 可以得到样本标准差, 一般表示成r：

$$r = \frac{\sum_{i=1}^{n}(X_i-\bar{X})(Y_i-\bar{Y})}{\sqrt{\sum_{i=1}^{n}(X_i-\bar{X})^2}\sqrt{\sum_{i=1}^{n}(Y_i-\bar{Y})^2}}.$$

一种等价表达式的是表示成标准分的均值。基于(Xi, Yi)的样本点, 样本皮尔逊系数是

$$r = \frac{1}{n-1}\sum_{i=1}^{n}\left(\frac{X_i-\bar{X}}{s_X}\right)\left(\frac{Y_i-\bar{Y}}{s_Y}\right)$$

$\frac{X_i-\bar{X}}{s_X}$ 、$\bar{X}$及s_X, 分别是标准分、样本平均值和样本标准差。

或许上面的讲解令你头脑混乱不堪, 没关系, 我换一种方式讲解, 如下：

假设有两个变量X 、Y, 那么两变量间的皮尔逊相关系数可通过以下公式计算：

- 公式一：

$$\rho_{X,Y} = \frac{\operatorname{cov}(X,Y)}{\sigma_X \sigma_Y} = \frac{E((X-\mu_X)(Y-\mu_Y))}{\sigma_X \sigma_Y} = \frac{E(XY)-E(X)E(Y)}{\sqrt{E(X^2)-E^2(X)}\sqrt{E(Y^2)-E^2(Y)}}$$

  注：勿忘了上面说过，“皮尔逊相关系数定义为两个变量之间的协方差和标准差的商”，其中标准差的计算公式为：

  随机变量的标准差计算公式

  一随机变量 X 的标准差定义为：

  $$\sigma = \sqrt{\mathrm{E}((X-\mathrm{E}(X))^2)} = \sqrt{\mathrm{E}(X^2)-(\mathrm{E}(X))^2}$$

- 公式二：$$\rho_{X,Y}=\frac{N\sum XY-\sum X\sum Y}{\sqrt{N\sum X^2-(\sum X)^2}\sqrt{N\sum Y^2-(\sum Y)^2}}$$
- 公式三：$$\rho_{X,Y}=\frac{\sum(X-\overline{X})(Y-\overline{Y})}{\sqrt{\sum(X-\overline{X})^2\sum(Y-\overline{Y})^2}}$$
- 公式四：$$\rho_{X,Y}=\frac{\sum XY-\frac{\sum X\sum Y}{N}}{\sqrt{(\sum X^2-\frac{(\sum X)^2}{N})(\sum Y^2-\frac{(\sum Y)^2}{N})}}$$

以上列出的四个公式等价，其中E是数学期望，cov表示协方差，N表示变量取值的个数。

(2)皮尔逊相关系数的适用范围

当两个变量的标准差都不为零时，相关系数才有定义，皮尔逊相关系数适用于：

1. 两个变量之间是线性关系，都是连续数据。
2. 两个变量的总体是正态分布，或接近正态的单峰分布。
3. 两个变量的观测值是成对的，每对观测值之间相互独立。

(3)如何理解皮尔逊相关系数

rubyist：皮尔逊相关系数理解有两个角度

其一，按照高中数学水平来理解，它很简单，可以看做将两组数据首先做Z分数处理之后，然后两组数据的乘积和除以样本数，Z分数一般代表正态分布中，数据偏离中心点的距离.等于变量减掉平均数再除以标准差.(就是高考的标准分类似的处理)

样本标准差则等于变量减掉平均数的平方和，再除以样本数，最后再开方，也就是说，方差开方即为标准差，样本标准差计算公式为：

$$\sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i-\overline{x})^2}$$

所以, 根据这个最朴素的理解,我们可以将公式依次精简为:

$$r_{xy}=\frac{\sum ZxZy}{N}$$

$$=\frac{\sum(\frac{X-\bar{X}}{Sx})(\frac{Y-\bar{Y}}{Sy})}{N}$$

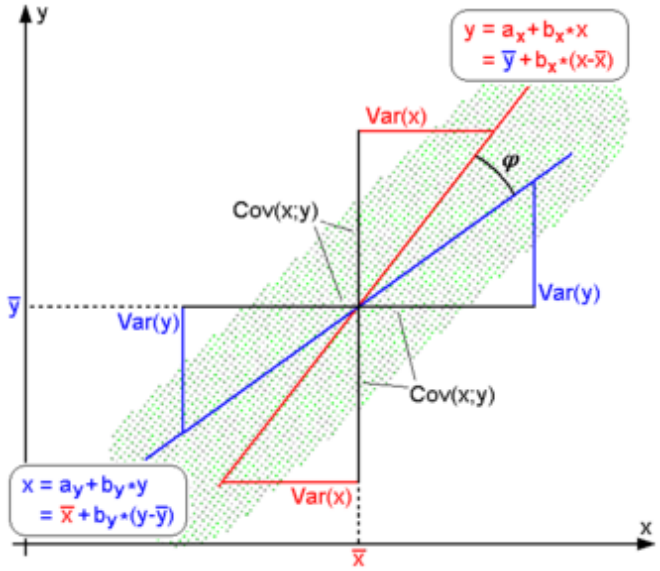
$$=\frac{\sum(X-\bar{X})(Y-\bar{Y})}{N\bullet SxSy}$$

$$=\frac{\sum(X-\bar{X})(Y-\bar{Y})}{N\bullet(\sqrt{\frac{1}{n}\sum_{i=1}^{n}(X_i-\bar{X})^2})(\sqrt{\frac{1}{n}\sum_{i=1}^{n}(Y_i-\bar{Y})^2})}$$

$$=\frac{\sum(X-\bar{X})(Y-\bar{Y})}{(\sqrt{\sum_{i=1}^{n}(X_i-\bar{X})^2})(\sqrt{\sum_{i=1}^{n}(Y_i-\bar{Y})^2})}$$

其二, 按照大学的线性数学水平来理解, 它比较复杂一点,可以看做是两组数据的向量夹角的余弦。下面是关于此皮尔逊系数的几何学的解释,

先来看一幅图，如下所示：



回归直线：y=gx(x) [红色] 和 x=gy(y) [蓝色]

如上图，对于没有中心化的数据，相关系数与两条可能的回归线y=gx(x)和 x=gy(y) 夹角的余弦值一致。 对于没有中心化的数据 (也就是说，数据移动一个样本平均值以使其均值为0)，相关系数也可以被视作由两个随机变量 向量 夹角 的 余弦值（见下方）。

举个例子，例如，有5个国家的国民生产总值分别为 10, 20, 30, 50 和 80 亿美元。 假设这5个国家 (顺序相同) 的贫困百分比分别为 11%, 12%, 13%, 15%, and 18% 。令 x 和 y 分别为包含上述5个数据的向量: x = (1, 2, 3, 5, 8) 和 y = (0.11, 0.12, 0.13, 0.15, 0.18)。

利用通常的方法计算两个向量之间的夹角 (参见 数量积)，未中心化 的相关系数是:

$$\cos\theta = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \, \|\mathbf{y}\|} = \frac{2.93}{\sqrt{103}\sqrt{0.0983}} = 0.920814711.$$

我们发现以上的数据特意选定为完全相关: y = 0.10 + 0.01 x。于是，皮尔逊相关系数应该等于1。将数据中心化 (通过E(x) = 3.8移动 x 和通过 E(y) = 0.138 移动 y) 得到 x = (−2.8, −1.8, −0.8, 1.2, 4.2) 和 y = (−0.028,

−0.018, −0.008, 0.012, 0.042), 从中

$$\cos\theta = \frac{\mathbf{x}\cdot\mathbf{y}}{\|\mathbf{x}\|\,\|\mathbf{y}\|} = \frac{0.308}{\sqrt{30.8}\sqrt{0.00308}} = 1 = \rho_{xy},$$

(4)皮尔逊相关的约束条件

从以上解释, 也可以理解皮尔逊相关的约束条件:

1. 两个变量间有线性关系
2. 变量是连续变量
3. 变量均符合正态分布,且二元分布也符合正态分布
4. 两变量独立

在实践统计中,一般只输出两个系数,一个是相关系数,也就是计算出来的相关系数大小,在-1到1之间;另一个是独立样本检验系数,用来检验样本一致性。

简单说来，各种“距离”的应用场景简单概括为，空间：欧氏距离，路径：曼哈顿距离，国际象棋国王：切比雪夫距离，以上三种的统一形式:闵可夫斯基距离，加权：标准化欧氏距离，排除量纲和依存：马氏距离，向量差距：夹角余弦，编码差别：汉明距离，集合近似度：杰卡德类似系数与距离，相关：相关系数与相关距离。

1.3 K值的选择

除了上述1.2节如何定义邻居的问题之外,还有一个选择多少个邻居,即K值定义为多大的问题。不要小看了这个K值选择问题，因为它对K近邻算法的结果会产生重大影响。如李航博士的一书「统计学习方法」上所说:

1. 如果选择较小的K值，就相当于用较小的领域中的训练实例进行预测，“学习”近似误差会减小，只有与输入实例较近或相似的训练实例才会对预测结果起作用，与此同时带来的问题是“学习”的估计误差会增大，换句话说，K值的减小就意味着整体模型变得复杂，容易发生过拟合；
2. 如果选择较大的K值，就相当于用较大领域中的训练实例进行预测，其优点是可以减少学习的估计误差，但缺点是学习的近似误

差会增大。这时候，与输入实例较远（不相似的）训练实例也会对预测器作用，使预测发生错误，且K值的增大就意味着整体的模型变得简单。

3. K=N，则完全不足取，因为此时无论输入实例是什么，都只是简单的预测它属于在训练实例中最多的累，模型过于简单，忽略了训练实例中大量有用信息。

在实际应用中，K值一般取一个比较小的数值，例如采用交叉验证法（简单来说，就是一部分样本做训练集，一部分做测试集）来选择最优的K值。

The Art of Programming By July

July

2014